All Necessary Fits - No Color Stuff

December 20, 2021

[3]:
```
import numpy as np
import matplotlib
from matplotlib import pyplot as plot
from matplotlib.font_manager import FontProperties
from sklearn.linear_model import LinearRegression
from sklearn.decomposition import PCA
import sklearn.metrics
from sklearn.metrics import explained_variance_score, mean_squared_error,␣
 ↪r2_score # https://scikit-learn.org/stable/auto_examples/linear_model/
 ↪plot_ols.html
import statsmodels.api as sm
import pandas
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import scipy as sp
from scipy.odr import *
import random
import operator
import os
import seaborn as sns
from mpl_toolkits.mplot3d import axes3d

# For 3d plots. This import is necessary to have 3D plotting below
from mpl_toolkits.mplot3d import Axes3D

# For statistics. Requires statsmodels 5.0 or more
from statsmodels.formula.api import ols
from statsmodels.formula.api import wls
# Analysis of Variance (ANOVA) on linear models
from statsmodels.stats.anova import anova_lm

# Adapted from:
# https://www.datasciencecentral.com/profiles/blogs/
 ↪linear-regression-in-python-use-of-numpy-scipy-and-statsmodels,
# https://realpython.com/linear-regression-in-python/#multiple-linear-regression
```

[4]:
```
plt.rcParams['figure.facecolor'] = 'w'
plt.rcParams['figure.edgecolor'] = 'k'
```

[5]:
```
M0StarData = np.transpose(np.asarray(np.
↪genfromtxt('ObservablePlus_FundMStarData.csv',delimiter=',',skip_header=0)))␣
↪# No header, so don't skip
M1StarData = np.transpose(np.asarray(np.
↪genfromtxt('ObservablePlus_FirstOvertoneMStarData.
↪csv',delimiter=',',skip_header=0))) # Has header, so skip
C0StarData = np.transpose(np.asarray(np.
↪genfromtxt('ObservablePlus_FundamentalCStarData.
↪csv',delimiter=',',skip_header=0))) # Has header, so skip
C1StarData = np.transpose(np.asarray(np.
↪genfromtxt('ObservablePlus_FirstOvertoneCStarData.
↪csv',delimiter=',',skip_header=0))) # No header, so don't skip

MdotIndex=5 # Column of imported files that has Mdot data
MdotErrorIndex=6 # Absolute error in Mdot
MdotDustIndex=2 # Column of imported files that has Mdot_dust Data
LIndex=8 # Column of imported files that has LBol data
LErrorIndex=9 # Absolute error in L
PIndex=13 # Column of imported files that has Period data
JIndex=14 # Column of imported files that has J-band magnitude
HIndex=15 # Column of imported files that has J-band magnitude
KIndex=16 # Column of imported files that has J-band magnitude
ThreePointSixIndex=17 # Column of imported files that has 3.6 band magnitude
FourPointFiveIndex=18 # Column of imported files that has 3.6 band magnitude
FivePointEightIndex=19 # Column of imported files that has 3.6 band magnitude
EightPointZeroIndex=20 # Column of imported files that has 8.0 band magnitude
MIndex=22 # Column of imported files that has Mass data
RIndex=23 # Column of imported files that has Radius data
TeffObsvIndex=12 # Column of imported files that has Observed Teff Data
TeffCalcIndex=24 # Column of imported files that has Calculated Teff Data
tauIndex=27 # Column of imported files that has Optical Depth data
```

[6]:
```
def critical_zone_only (star_data,print_star_cuts = True):
    c_zone_indices = []
    under_c_indices = []
    over_c_indices = []
    i = 0
    while i <len(np.transpose(star_data)):
        #print('logMdot = ',np.log10(star_data[MdotIndex][i]))
        #print('logMdotCrit = ',np.log10(star_data[MIndex][i]/(2*10**6)))
        if np.log10(star_data[MdotIndex][i]) <= np.log10(star_data[MIndex][i]/
↪(2*10**6))-1: # 2*10**6 is the characteristic time for mass-loss (evolution␣
↪time)
            i+=1
```

```
            under_c_indices.append(i)
        elif np.log10(star_data[MdotIndex][i]) >= np.log10(star_data[MIndex][i]/
↪(2*10**6))+1:
            i+=1
            over_c_indices.append(i)
        else:
            c_zone_indices.append(i)
            i+=1
    trimmed_array = np.transpose(star_data)[c_zone_indices]
    if print_star_cuts == True:
        print(len(np.transpose(star_data))-len(c_zone_indices), 'stars cut.',␣
↪len(trimmed_array), 'stars remaining.')
        print(len(over_c_indices), 'stars over the critical mass-loss rate.
↪',len(under_c_indices), 'stars under the critical mass-loss rate.')
        print('_____')
    return np.transpose(trimmed_array)
```

[7]:
```
def correct_mdot (star_data,multiplier):
    new_mdot_data = multiplier*star_data[MdotDustIndex]
    new_mdot_error_data = multiplier*star_data[MdotDustIndex+1]
    return np.
↪asarray([star_data[0],star_data[1],star_data[2],star_data[3],star_data[4],new_mdot_data,new
↪])
```

[8]:
```
def find_max_crit_zone_multiplier (star_data, min_guess=1, max_guess = 10000):
    crit_zone_sizes_at_multiplier_value = []
    multiplier_values = []
    for multiplier in range(min_guess, max_guess):
        scaled_mdot_star_data = correct_mdot(star_data, multiplier)
        critical_zone_only_data = critical_zone_only (scaled_mdot_star_data,␣
↪print_star_cuts = False)
        crit_zone_sizes_at_multiplier_value.append(len(np.
↪transpose(critical_zone_only_data)))
        multiplier_values.append(multiplier)
    print ('Data maximized with',np.
↪nanmax(crit_zone_sizes_at_multiplier_value),' stars with multiplier(s)',␣
↪multiplier_values[np.argmax(crit_zone_sizes_at_multiplier_value)])
    return multiplier_values[np.argmax(crit_zone_sizes_at_multiplier_value)]
```

M0Multiplier = find_max_crit_zone_multiplier (M0StarData,min_guess=3800, max_guess = 3900) M1Multiplier = find_max_crit_zone_multiplier (M1StarData,min_guess=5000, max_guess = 5200) C0Multiplier = find_max_crit_zone_multiplier (C0StarData,min_guess=2500, max_guess = 2600) C1Multiplier = find_max_crit_zone_multiplier (C1StarData,min_guess=7900, max_guess = 8000)

M0Multiplier = 500 # Riebel 2012, conclusions M1Multiplier = 500 # Riebel 2012, conclusions

C0Multiplier = 200 # Riebel 2012, conclusions C1Multiplier = 200 # Riebel 2012, conclusions

[9]:
```
def linear_fit_1var (xdata, ydata):
    x = xdata
    y = ydata

    data = pandas.DataFrame({'x': x, 'y': y})
    model = ols("y ~ x", data).fit()
    # Parameters are returned in the format intercept, slope
    return model._results.params, model._results.rsquared, model.bse, model.
 ↪normalized_cov_params
```

[10]:
```
def multilinear_fit_2var (xdata, ydata):
    x0 = xdata[0]
    x1 = xdata[1]
    y = ydata

    data = pandas.DataFrame({'x0': x0, 'x1': x1, 'y': y})
    model = ols("y ~ x0 + x1", data).fit()

    # Parameters are returned in the format intercept, slope 1, slope 2
    return model._results.params, model._results.rsquared, model.bse, model.
 ↪normalized_cov_params
```

[11]:
```
# Check current multiplier
def check_gas_to_dust_ratio (star_data):
    star_by_star_ratio = star_data[MdotIndex]/star_data[MdotDustIndex]
    #print(star_by_star_ratio)
    print(np.nanmean(star_by_star_ratio))
```

[12]:
```
def van_Loon_psi_for_non_solar (spec_type, solar_ratio, input_z):
    # Take spectral type (M or C), the solar ratio (dust-to-gas ratio for␣
 ↪sun-like metallicity), intial metallicity z (divided by solar) for target,␣
 ↪to find
    # dust-to-gas ratio for target metallicity
    if spec_type == 'M':
        # First, use the solar ratio to determine the constant in formula. Zsun␣
 ↪~ 0.0124
        constant = np.log10(solar_ratio) - 0.7*np.log10(0.0124/0.0124)
        constant_lower = np.log10(solar_ratio) - (0.7 - 0.3)*np.log10(0.0124/0.
 ↪0124)
        constant_upper = np.log10(solar_ratio) - (0.7 + 0.6)*np.log10(0.0124/0.
 ↪0124)
        dtg_ratio = 10**(0.7*np.log10(input_z)+constant)
        dtg_ratio_lower = 10**((0.7- 0.3)*np.log10(input_z)+constant_lower)
        dtg_ratio_upper = 10**((0.7+ 0.6)*np.log10(input_z)+constant_upper)
    elif spec_type == 'C':
```

```
        constant = np.log10(solar_ratio) - 1*np.log10(0.0124/0.0124)
        constant_lower = np.log10(solar_ratio) - (1 - 0.3)*np.log10(0.0124/0.
↪0124)
        constant_upper = np.log10(solar_ratio) - (1 + 0.3)*np.log10(0.0124/0.
↪0124)
        dtg_ratio = 10**(0.7*np.log10(input_z)+constant)
        dtg_ratio_lower = 10**((1- 0.3)*np.log10(input_z)+constant_lower)
        dtg_ratio_upper = 10**((1+ 0.3)*np.log10(input_z)+constant_upper)
    else:
        print('Error. Invalid Spectral Type.')
    return np.asarray([dtg_ratio_lower, dtg_ratio, dtg_ratio_upper])
```

[13]:
```
ratios_M = (van_Loon_psi_for_non_solar('M',1/500,0.001/0.0124))**-1
```

[14]:
```
(van_Loon_psi_for_non_solar('M',1/500,0.003/0.0124))**-1
```

[14]:
```
array([ 882.0438506 , 1350.14466801, 3163.44700853])
```

[15]:
```
ratios_C = (van_Loon_psi_for_non_solar('C',1/200,0.001/0.0124))**-1
```

[ ]:
```

```

[16]:
```
(van_Loon_psi_for_non_solar('C',1/200,0.003/0.0124))**-1
```

[16]:
```
array([ 540.05786721,  540.05786721, 1265.37880341])
```

[17]:
```
M0StarData = correct_mdot (M0StarData, int(ratios_M[1]))
M1StarData = correct_mdot (M1StarData, int(ratios_M[1]))
C0StarData = correct_mdot (C0StarData, int(ratios_C[1]))
C1StarData = correct_mdot (C1StarData, int(ratios_C[1]))
```

[18]:
```
check_gas_to_dust_ratio(M0StarData)
check_gas_to_dust_ratio(M1StarData)
check_gas_to_dust_ratio(C0StarData)
check_gas_to_dust_ratio(C1StarData)
```

```
2913.0
2913.0
1165.0
1165.0
```

[19]:
```
def clear_rows_with_nAn_for_color (RawStarData, band_1_index, band_2_index ):
    band_1_nAn_index = []
    band_2_nAn_index = []
    band_1_data = RawStarData[band_1_index]
    band_2_data = RawStarData[band_2_index]
    band_1_nAn = np.isnan(band_1_data)
```

```
    band_2_nAn = np.isnan(band_2_data)
    index_list_1 = np.where(band_1_nAn == True)[0]
    index_list_2 = np.where(band_2_nAn == True)[0]
    full_index_list = np.concatenate((index_list_1, index_list_2))
    full_index_list = np.unique(full_index_list) # Note: This function also␣
 ↪*sorts* the array
    working_data = np.transpose(RawStarData)
    reduced_data = np.delete(working_data, full_index_list, 0)
    print('Stars removed (nAn):', len(np.
 ↪transpose(RawStarData))-len(reduced_data))
    return np.transpose(reduced_data)
def clear_rows_with_color_less_than_value (RawStarData, band_1_index,␣
 ↪band_2_index,value=0 ):
    band_1_data = RawStarData[band_1_index]
    band_2_data = RawStarData[band_2_index]
    color = band_1_data - band_2_data
    less_than_value = []
    for i in range (0,len(color)):
        if color[i] < value:
            less_than_value.append(i)
    #print(less_than_value)
    full_index_list = np.unique(less_than_value) # Note: This function also␣
 ↪*sorts* the array
    working_data = np.transpose(RawStarData)
    reduced_data = np.delete(working_data, full_index_list, 0)
    print('Stars removed (color<'+str(value)+'):', len(np.
 ↪transpose(RawStarData))-len(reduced_data))
    return np.transpose(reduced_data)
def calculate_color (band_1, band_2):
    return band_1 - band_2
```

[20]:
```
# Calculates the mass-loss rate from a hyperbolic fit to color
#                  a
#   log Mdot = _________ + c
#              color + b
def calc_mdot_from_color (color_data, a, b, c): # Color_data should be a numpy␣
 ↪array. Using the function written, turn it into Mdot data for a given star␣
 ↪type
    color_log_mdot = a/(color_data+b) + c
    return color_log_mdot
```

[21]:
```
def plot_x_y_data (xdata,ydata):
    plot.scatter(xdata,ydata,s=2)
    plot.show()
    plot.close()
```

[22]:
```
# First, define function for ODR
def f10(B, x):
    '''Multi-Linear function y = B[0]*x0 + B[1]*x[1] + B[2]'''
    # B is a vector of the parameters.
    # x is an array of the current x values.
    # x is in the same format as the x passed to Data or RealData.
    #
    # Return an array in the same format as y passed to Data or RealData.
    return B[0]*x + B[1]
def do_odr_fit (data_to_fit):
    linear = Model(f10)
    logMdotData = np.log10(data_to_fit[5])
    logPData = np.log10(data_to_fit[13])
    logLData = np.log10(data_to_fit[8])
    mydata = Data(logPData,logMdotData)
    myodr = ODR(mydata,linear,beta0=[1,1,1])
    myrun = myodr.run()
    myrun.pprint()
    return myrun.beta # Return the information we need to create the fitting␣
 ↪function and plot it
```

[23]:
```
def chi_sq_fit_2d (data_x, data_y, print_model_results = 0):
    # First, let's look at all of the Fundamental mode data at once.
    # Convert the data into a Pandas DataFrame to use the formulas framework
    # in statsmodels

    # First we need to flatten the data: it's 2D layout is not relevent.
    x = data_x
    y = data_y

    data = pandas.DataFrame({'x': data_x, 'y': data_y})

    # Fit the model
    model = ols("y ~ x ", data).fit()

    # Print the summary
    if print_model_results != 0:
        print(model.summary())

        print("\nRetrieving manually the parameter estimates:")
        print(model._results.params)

        print("\nRetrieving manually the R^2 value:")
        print(model._results.rsquared)
        # Peform analysis of variance on fitted linear model
        anova_results = anova_lm(model)
        print('\nANOVA results')
```

```
        print(anova_results)

    parameters = model._results.params
    rsquared = model._results.rsquared
    return parameters, rsquared
```

[24]:
```
def calc_PE_scale (stellar_data):
    Mass = stellar_data[MIndex]
    Radius = stellar_data[RIndex]
    potential_energy_scale = np.log10(Mass/Radius)
    return potential_energy_scale

def calc_g_scale (stellar_data):
    Mass = stellar_data[MIndex]
    Radius = stellar_data[RIndex]
    g_scale = np.log10(Mass/Radius**2)
    return g_scale

def calc_lum_pressure_scale (stellar_data):
    Luminosity = stellar_data[LIndex]
    Radius = stellar_data[RIndex]
    L_Pressure_scale = np.log10(Luminosity/Radius**2)
    return L_Pressure_scale

def calc_core_lum_param (stellar_data):
    Luminosity = stellar_data[LIndex]
    Mass = stellar_data[MIndex]
    core_lum_param = np.log10(Luminosity/Mass)
    return core_lum_param
```

[25]:
```
def plot_logmdot_vs_parameter (mdot, parameter, title='',␣
↪parameter_label='',save_fig = 0, save_folder = '',testing = 0):
    plot.scatter(parameter,mdot,s=2,label='Data')
    chisqparams = chi_sq_fit_2d (parameter, mdot)
    fit_parameters = chisqparams[0]
    fit_x = np.asarray([np.nanmin(parameter), np.nanmax(parameter)])
    if testing != 0:
        print('parameter:',parameter)
        print('np.min(parameter):',np.nanmin(parameter))
        print('np.max(parameter):',np.nanmax(parameter))
        print('fit_x:',fit_x)
    fit_y = fit_parameters[0] + fit_parameters[1]*fit_x
    if testing != 0:
        print('fit_y:',fit_x)
    fit_label = "{:6e}".format(fit_parameters[0]) +'+'+ "{:6e}".
↪format(fit_parameters[1])+'*('+parameter_label+'); R^2 ='+ "{:6e}".
↪format(chisqparams[1])
```

```
    plot.plot(fit_x,fit_y,color='orange', label = fit_label)
    plot.title(title)
    plot.xlabel(parameter_label)
    plot.ylabel('logMdot_obsv')
    plot.legend()
    if save_fig != 0:
        plot.tight_layout()
        plot_name = save_folder + '/' + 'logMdot_versus_' + parameter_label + '/
↪' + title
        plot_name = plot_name.replace('(M/R)', '(MdivR)') # Illegal character␣
↪on Windows. Replace the Directory character with a simple string
        plot_name = plot_name.replace('(M/R^2)', '(MdivR^2)') # Illegal␣
↪character on Windows. Replace the Directory character with a simple string
        plot_name = plot_name.replace('(L/R)', '(LdivR)') # Illegal character␣
↪on Windows. Replace the Directory character with a simple string
        plot_name = plot_name.replace('(L/R^2)', '(LdivR^2)') # Illegal␣
↪character on Windows. Replace the Directory character with a simple string
        plot_name = plot_name.replace('(L/M)', '(LdivM)') # Illegal character␣
↪on Windows. Replace the Directory character with a simple string
        plot.savefig(plot_name+'.png')
        plot.savefig(plot_name+'.pdf') # Why do journals require this format...
        plot.show()
    else:
        plot.tight_layout()
        plot.show()
    plot.close()
```

[26]:
```
def plot_residual_vs_parameter (mdot_resid, parameter, title='',␣
↪parameter_label='',save_fig = 0, save_folder = ''):
    plot.scatter(parameter,mdot_resid,s=2, label = 'Data')
    chisqparams = chi_sq_fit_2d (parameter, mdot_resid)
    fit_parameters = chisqparams[0]
    fit_x = np.asarray([np.nanmin(parameter), np.nanmax(parameter)])
    fit_y = fit_parameters[0] + fit_parameters[1]*fit_x
    fit_label = "{:6e}".format(fit_parameters[0]) +'+'+ "{:6e}".
↪format(fit_parameters[1])+'*('+parameter_label+'); R^2 ='+ "{:6e}".
↪format(chisqparams[1])
    plot.plot(fit_x,fit_y,color='orange', label = fit_label)
    plot.title(title)
    plot.xlabel(parameter_label)
    plot.ylabel('logMdot_calc-logMdot_obsv')
    plot.legend()
    if save_fig != 0:
        plot.tight_layout()
        plot_name = save_folder + '/' + 'Residuals/'+ 'residual_versus_'+␣
↪parameter_label + '/' + title
```

```
        plot_name = plot_name.replace('(M/R)', '(MdivR)') # Illegal character␣
↪on Windows. Replace the Directory character with a simple string
        plot_name = plot_name.replace('(M/R^2)', '(MdivR^2)') # Illegal␣
↪character on Windows. Replace the Directory character with a simple string
        plot_name = plot_name.replace('(L/R)', '(LdivR)') # Illegal character␣
↪on Windows. Replace the Directory character with a simple string
        plot_name = plot_name.replace('(L/R^2)', '(LdivR^2)') # Illegal␣
↪character on Windows. Replace the Directory character with a simple string
        plot_name = plot_name.replace('(L/M)', '(LdivM)') # Illegal character␣
↪on Windows. Replace the Directory character with a simple string
        plot.savefig(plot_name+'.png')
        plot.savefig(plot_name+'.pdf') # Why do journals require this format...
    else:
        plot.tight_layout()
        plot.show()
    plot.close()
    return fit_parameters
```

[27]:
```
def plot_logmdot_vs_logmdot (mdot, calc_mdot, title='',save_fig = 0,␣
↪save_folder = ''):
    plot.scatter(mdot,calc_mdot,s=2, label = 'Data')
    chisqparams = chi_sq_fit_2d (mdot, calc_mdot)
    fit_parameters = chisqparams[0]
    line_coords = np.asarray([np.min([np.nanmin(mdot),np.nanmin(calc_mdot)]),np.
↪nanmax([np.max(mdot),np.nanmax(calc_mdot)])])
    fit_x = line_coords
    fit_y = fit_parameters[0] + fit_parameters[1]*fit_x
    fit_label = "{:6e}".format(fit_parameters[0]) +'+'+ "{:6e}".
↪format(fit_parameters[1])+'*'+'logMdot_obsv'+'; R^2 ='+ "{:6e}".
↪format(chisqparams[1])
    plot.plot(line_coords,line_coords,color='black', label = '1:1 Line')
    plot.plot(fit_x,fit_y,color='orange', label = fit_label)
    plot.title(title)
    plot.xlabel('logMdot_obsv')
    plot.ylabel('logMdot_calc')
    plot.legend()
    if save_fig != 0:
        plot.tight_layout()
        plot_name = save_folder + '/' + 'logMdot_calc_versus_logMdot_obsv' + '/
↪' + title
        plot_name = plot_name.replace('(M/R)', '(MdivR)') # Illegal character␣
↪on Windows. Replace the Directory character with a simple string
        plot_name = plot_name.replace('(M/R^2)', '(MdivR^2)') # Illegal␣
↪character on Windows. Replace the Directory character with a simple string
        plot_name = plot_name.replace('(L/R)', '(LdivR)') # Illegal character␣
↪on Windows. Replace the Directory character with a simple string
```

```
        plot_name = plot_name.replace('(L/R^2)', '(LdivR^2)') # Illegal␣
↪character on Windows. Replace the Directory character with a simple string
        plot_name = plot_name.replace('(L/M)', '(LdivM)') # Illegal character␣
↪on Windows. Replace the Directory character with a simple string
        plot.savefig(plot_name+'.png')
        plot.savefig(plot_name+'.pdf') # Why do journals require this format...
    else:
        plot.tight_layout()
        plot.show()
    plot.close()
```

[28]:
```
np.shape(COStarData)
```

[28]:
```
(28, 1995)
```

[29]:
```
def sort_xCO(stellar_data): # Function to separate 'extreme' AGB stars from the␣
↪carbon star population
    '''
    "As in Blum et al. (2006) and Srinivasan et al. (2009), we identify x-AGB␣
↪stars as those brighter than the 3.6  m TRGB and
    with J -[3.6] > 3.1 mag. Some of the most heavily dust-enshrouded x-AGB␣
↪stars are totally undetected in the near-IR.
    Therefore, if there is no near-IR detection, but [3.6] - [8] > 0.8 and the␣
↪star is brighter than the 3.6  m TRGB, then it is
    also included in the initial list of x-AGB stars."
    - Boyer et. al, 2011 https://iopscience.iop.org/article/10.1088/0004-6256/
↪142/4/103/pdf
    '''
    mag_J = stellar_data[JIndex]
    mag_36 = stellar_data[ThreePointSixIndex]
    mag_8 = stellar_data[EightPointZeroIndex]
    cut_line_1 = np.asarray([3.1,3.1])
    line_coords_1 = np.asarray([np.nanmin(mag_36 - mag_8),np.nanmax(mag_36 -␣
↪mag_8)])
    # Plot 1: CCD of J-[3.6] vs [3.6]-[8.0], with cutline 1
    plot.scatter(mag_36 - mag_8, mag_J - mag_36,s=2)
    plot.plot(line_coords_1, cut_line_1)
    plot.vlines(0.8,np.nanmin(mag_J - mag_36),np.nanmax(mag_J - mag_36))
    #plot.vlines(0.7,np.nanmin(mag_J - mag_36),np.nanmax(mag_J -␣
↪mag_36),linestyles='--')
    #plot.vlines(0.6,np.nanmin(mag_J - mag_36),np.nanmax(mag_J -␣
↪mag_36),linestyles='--')
    plot.xlabel('[3.6]-[8.0]')
    plot.ylabel('J-[3.6]')
    #plt.gca().invert_yaxis()
    #plt.gca().invert_xaxis()
```

```
    plot.tight_layout()
    plot.show()
    plot.close()

    # Plot 2: CMD of [3.6] vs [3.6]-[8.0], with cutline 1
    plot.scatter(mag_36 - mag_8, mag_36,s=2)
    plot.vlines(0.8,np.nanmin(mag_36),np.nanmax(mag_36))
    #plot.vlines(0.7,np.nanmin(mag_36),np.nanmax(mag_36),linestyles='--')
    #plot.vlines(0.6,np.nanmin(mag_36),np.nanmax(mag_36),linestyles='--')
    plot.xlabel('[3.6]-[8.0]')
    plot.ylabel('[3.6]')
    plt.gca().invert_yaxis()
    #plt.gca().invert_xaxis()
    plot.tight_layout()
    plot.show()
    plot.close()

    # Now, the acutal sorting function
    normal_indices = []
    extreme_indices = []
    for i in range(0,np.shape(stellar_data)[1]): # Iterate over every entry in␣
↪stellar_data
        if np.isnan(mag_J[i]) == False and np.isnan(mag_36[i]) == False: # If␣
↪there is an entry for J and [3.6]
            if mag_J[i]-mag_36[i] > 3.1: # Stars bluer than this should be␣
↪classified as extreme
                extreme_indices.append(i)
            else: # Otherwise they are not
                normal_indices.append(i)
        elif np.isnan(mag_36[i]) == False and np.isnan(mag_8[i]) == False: # If␣
↪there is an entry for [3.6] and [8.0]
            if mag_36[i]-mag_8[i]>0.8:
                extreme_indices.append(i)
            else:
                normal_indices.append(i)
        else:
            normal_indices.append(i)
            print('Non-sortable star found at index ',i,'. Assumed to not be␣
↪extreme.')

    norm_array = []
    extr_array = []
    for i in range(0, len(normal_indices)):
        norm_array.append(np.transpose(stellar_data)[normal_indices[i]])
    np_norm_array = np.asarray(norm_array)
    for i in range(0, len(extreme_indices)):
        extr_array.append(np.transpose(stellar_data)[extreme_indices[i]])
```

```
    np_extr_array = np.asarray(extr_array)

    mag_J_n = np.transpose(np_norm_array)[JIndex]
    mag_36_n = np.transpose(np_norm_array)[ThreePointSixIndex]
    mag_8_n = np.transpose(np_norm_array)[EightPointZeroIndex]
    mag_J_x = np.transpose(np_extr_array)[JIndex]
    mag_36_x = np.transpose(np_extr_array)[ThreePointSixIndex]
    mag_8_x = np.transpose(np_extr_array)[EightPointZeroIndex]

    # Plot 3: CCD of J-[3.6] vs [3.6]-[8.0], with cutline 1
    plot.scatter(mag_36_n - mag_8_n, mag_J_n - mag_36_n,s=2,label='C0')
    plot.scatter(mag_36_x - mag_8_x, mag_J_x - mag_36_x,s=2,label='C0x')
    plot.plot(line_coords_1, cut_line_1,color='black')
    plot.vlines(0.8,np.nanmin(mag_J - mag_36),np.nanmax(mag_J - mag_36))
    #plot.vlines(0.7,np.nanmin(mag_J - mag_36),np.nanmax(mag_J -␣
↪mag_36),linestyles='--')
    #plot.vlines(0.6,np.nanmin(mag_J - mag_36),np.nanmax(mag_J -␣
↪mag_36),linestyles='--')
    plot.xlabel('[3.6]-[8.0]')
    plot.ylabel('J-[3.6]')
    plot.legend()
    #plt.gca().invert_yaxis()
    #plt.gca().invert_xaxis()
    plot.tight_layout()
    plot.savefig('Single_Parameter_Fits/carbon_star_split_main.
↪png',format='png')
    plot.savefig('Single_Parameter_Fits/carbon_star_split_main.
↪pdf',format='pdf')
    plot.show()
    plot.close()

    # Plot 4: CMD of [3.6] vs [3.6]-[8.0], with cutline 1
    plot.scatter(mag_36_n - mag_8_n, mag_36_n,s=2,label='C0')
    plot.scatter(mag_36_x - mag_8_x, mag_36_x,s=2,label='C0x')
    plot.vlines(0.8,np.nanmin(mag_36),np.nanmax(mag_36))
    #plot.vlines(0.7,np.nanmin(mag_36),np.nanmax(mag_36),linestyles='--')
    #plot.vlines(0.6,np.nanmin(mag_36),np.nanmax(mag_36),linestyles='--')
    plot.xlabel('[3.6]-[8.0]')
    plot.ylabel('[3.6]')
    plot.legend()
    plt.gca().invert_yaxis()
    #plt.gca().invert_xaxis()
    plot.tight_layout()
    plot.savefig('Single_Parameter_Fits/carbon_star_split_secondary.
↪png',format='png')
    plot.savefig('Single_Parameter_Fits/carbon_star_split_secondary.
↪pdf',format='pdf')
```

```
    plot.show()
    plot.close()

    return np.transpose(np_norm_array), np.transpose(np_extr_array)
```

[30]:
```
def find_multiplier_for_flat_residual (star_data, multiplier =␣
↪1,spec_mode_label=''):
    logLData = np.log10(star_data[LIndex])
    logPData = np.log10(star_data[PIndex])
    logMdotData = np.log10(star_data[MdotIndex])
    logMdotDustData = np.log10(star_data[MdotDustIndex])

    data_fit = multilinear_fit_2var([logLData,logPData],logMdotDustData)
    LPCoeffs = data_fit[0]
    our_log_mdot = LPCoeffs[0]+LPCoeffs[1]*logLData+LPCoeffs[2]*logPData
    form_vs_fed_fit =␣
↪linear_fit_1var(logMdotDustData,our_log_mdot-logMdotDustData)
    # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
    form_vs_fed_Coeffs = form_vs_fed_fit[0]
    # This first fit should by construction center us at 0.
    print(form_vs_fed_fit)
    residual =␣
↪our_log_mdot-logMdotDustData-(form_vs_fed_Coeffs[1]*logMdotDustData)
    resid_adjust = (np.nanmax(residual) + np.nanmin(residual))/2
    plot.
↪scatter(logMdotDustData,our_log_mdot-logMdotDustData-(form_vs_fed_Coeffs[1]*logMdotDustData
    # To correct to 0 residual and 0 slope, we have to subtract a linear term␣
↪slope_residual_fit*logMdotfit-avg_residual
    plot.xlim(-12,-8)
    plot.ylim(-5,3)
    plot.title(spec_mode_label)
    plot.plot([-12,-2],[0,0],color = 'black', alpha =␣
↪1,linestyle='--',linewidth=1.5)
    plot.tight_layout()
    plot.show()
    plot.close()

    beta = form_vs_fed_Coeffs[1]

    old_avg_log_Mdot = np.nanmean(logMdotData)
    #print('Old Mean:',old_avg_log_Mdot)
    # To get rid of the slope, we can correct using the linear fit.
    # This implies logMdot = resid_adjust + (slope+1)*logMdotDust
    # Beta = form_vs_fed_Coeffs[1]
    # Guess: Alpha = old multiplier + the residual adjustment above
    logMdotData = np.log10(multiplier)+resid_adjust + (beta+1)*logMdotDustData
```

```
    guess_avg_log_Mdot = np.nanmean(logMdotData)
    #print('Guess Mean:',guess_avg_log_Mdot)
    correction = old_avg_log_Mdot-guess_avg_log_Mdot
    logalpha = np.log10(multiplier)+resid_adjust+correction
    logMdotData = logalpha + (beta + 1)*logMdotDustData
    new_avg_log_Mdot = np.nanmean(logMdotData)
    #print('New Mean:',new_avg_log_Mdot)

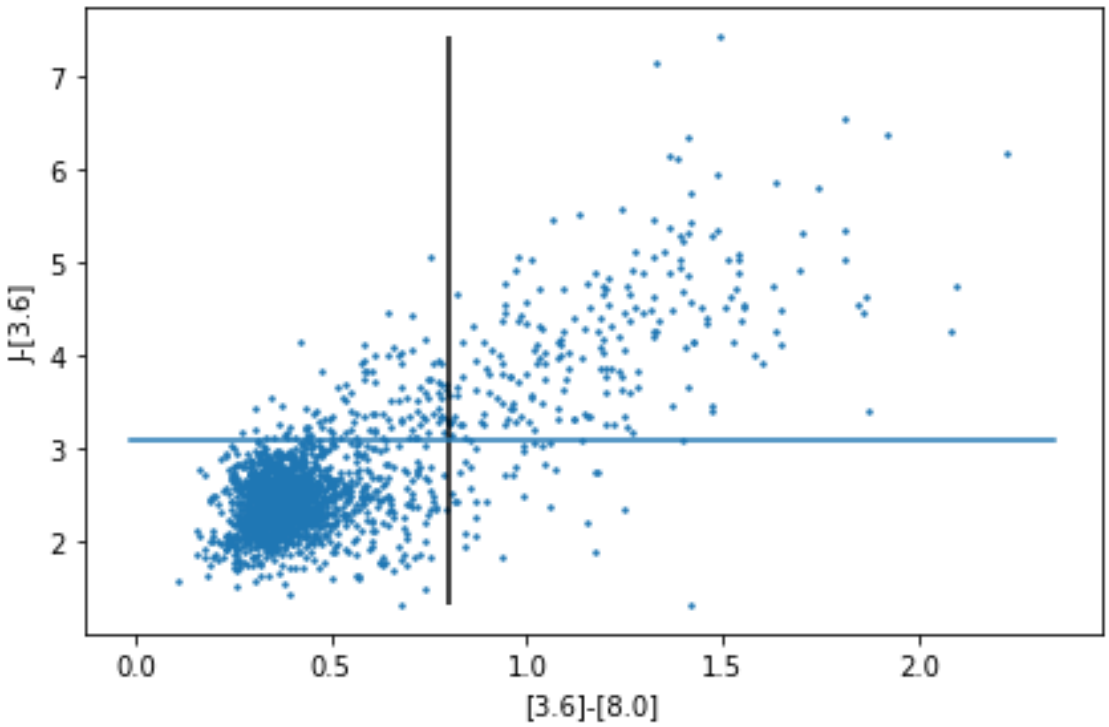
    data_fit = multilinear_fit_2var([logLData,logPData],logMdotData)
    LPCoeffs = data_fit[0]
    our_log_mdot = LPCoeffs[0]+LPCoeffs[1]*logLData+LPCoeffs[2]*logPData
    form_vs_fed_fit = linear_fit_1var(logMdotData,our_log_mdot-logMdotData)
    #print('Fit to logMdot:', form_vs_fed_fit)
    print('Fit to logMdot:', '\n Fit:',form_vs_fed_fit[0], '\n R^2:
↪',form_vs_fed_fit[1], '\n BSE:',form_vs_fed_fit[2], '\n Covar:
↪',form_vs_fed_fit[3])
    plot.scatter(logMdotData,our_log_mdot - logMdotData,s=2)
    plot.xlim(-8,-4)
    plot.ylim(-5,3)
    plot.title(spec_mode_label)
    plot.plot([-12,-2],[0,0],color = 'black', alpha =␣
↪1,linestyle='--',linewidth=1.5)
    plot.tight_layout()
    plot.show()
    plot.close()

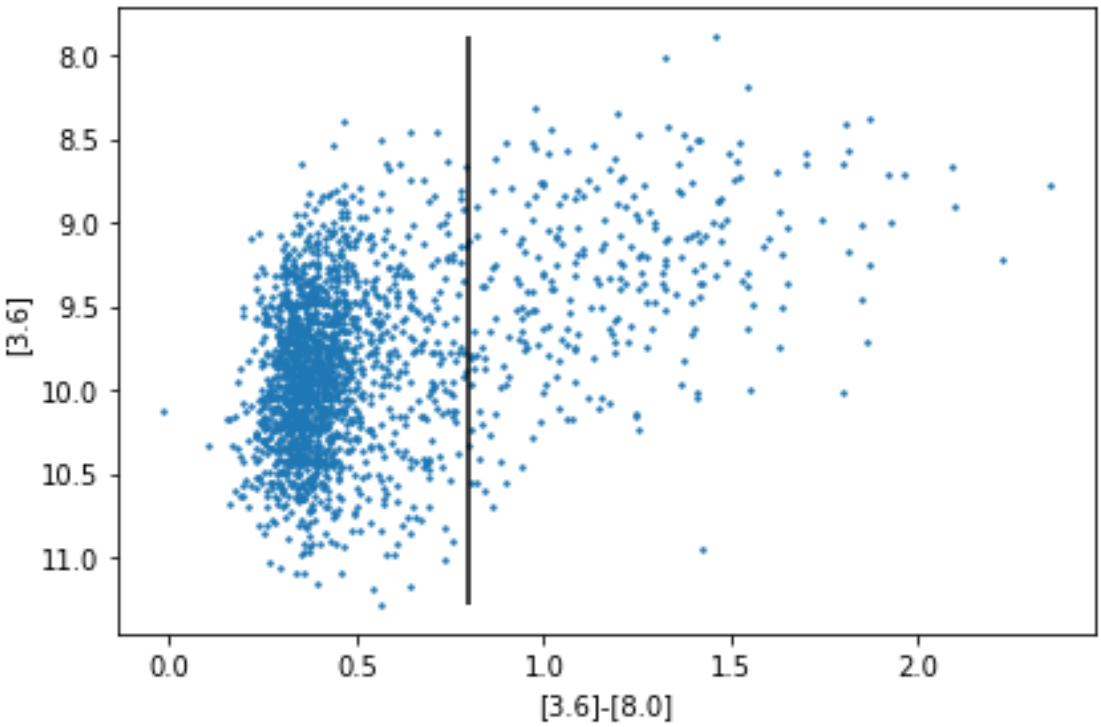
    # Test: Let our multiplier gamma be a function alpha*mdot^beta with beta =␣
↪0.5, 1, 1.5
    # or, log gamma = log(alpha) + beta*log(mdot)
    beta_test_list = [0.5, 1, 1.5]
    for i in range(0, len(beta_test_list)):
        beta_test = beta_test_list[i]
        gamma_test = multiplier*(10**logMdotDustData)**beta_test
        logMdot_test_list = []
        for k in range(0, len(gamma_test)):
            logMdot_test_list.append(np.
↪log10(gamma_test[k]*10**(logMdotDustData[k])))
        logMdot_test_array = np.asarray(logMdot_test_list)
        print('Beta=',beta_test,':')
        data_fit = multilinear_fit_2var([logLData,logPData],logMdot_test_array)
        LPCoeffs = data_fit[0]
        our_log_mdot = LPCoeffs[0]+LPCoeffs[1]*logLData+LPCoeffs[2]*logPData
        form_vs_fed_fit =␣
↪linear_fit_1var(logMdot_test_array,our_log_mdot-logMdot_test_array)
        # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
        form_vs_fed_Coeffs = form_vs_fed_fit[0]
        # This first fit should by construction center us at 0.
```

```
        print(form_vs_fed_fit)
        residual =␣
↪our_log_mdot-logMdot_test_array-(form_vs_fed_Coeffs[1]*logMdot_test_array)

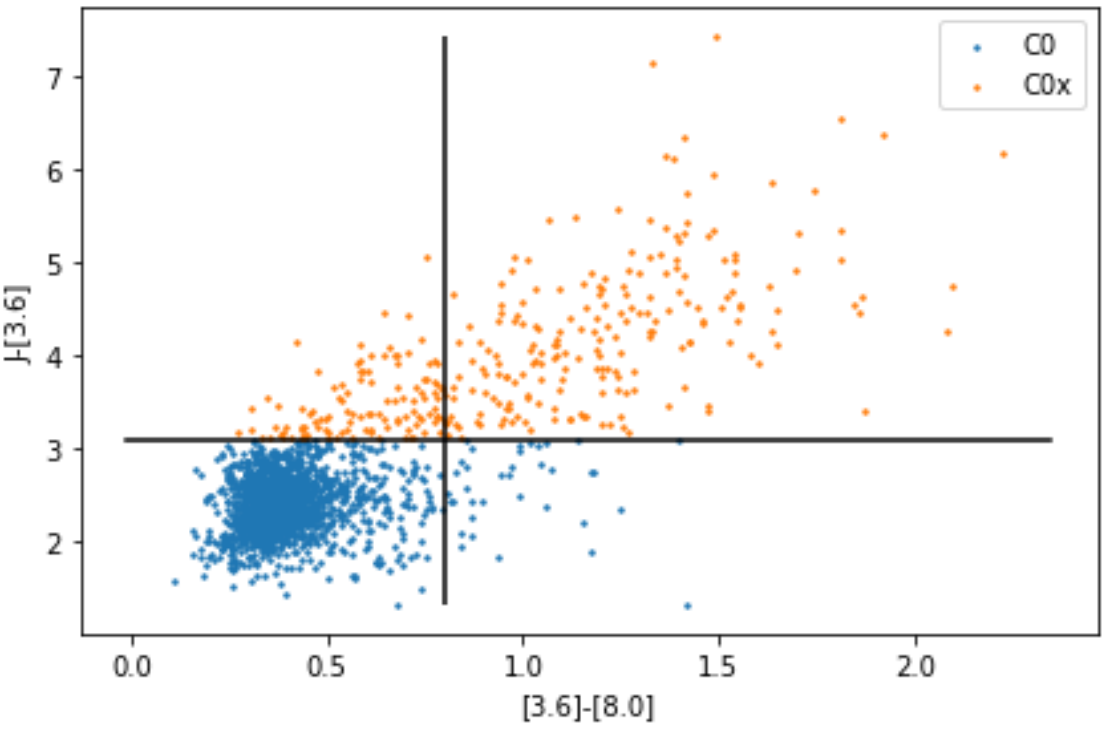
    # Test: What range of multipliers do we find? Rho = alpha*MdotDust^{Beta}
    rho_array = (10**logalpha)*(10**logMdotDustData)**beta
    print('Alpha:', 10**logalpha,'; beta:', beta)
    #print('10**logMdotDustData:',10**logMdotDustData)
    #print('10**logMdotDustData**beta:',(10**logMdotDustData)**beta)
    print('Minimum rho:',np.nanmin(rho_array), '; Maximum rho:', np.
↪nanmax(rho_array),'; Range:',np.nanmax(rho_array)-np.nanmin(rho_array))
    print('_________________________')
```

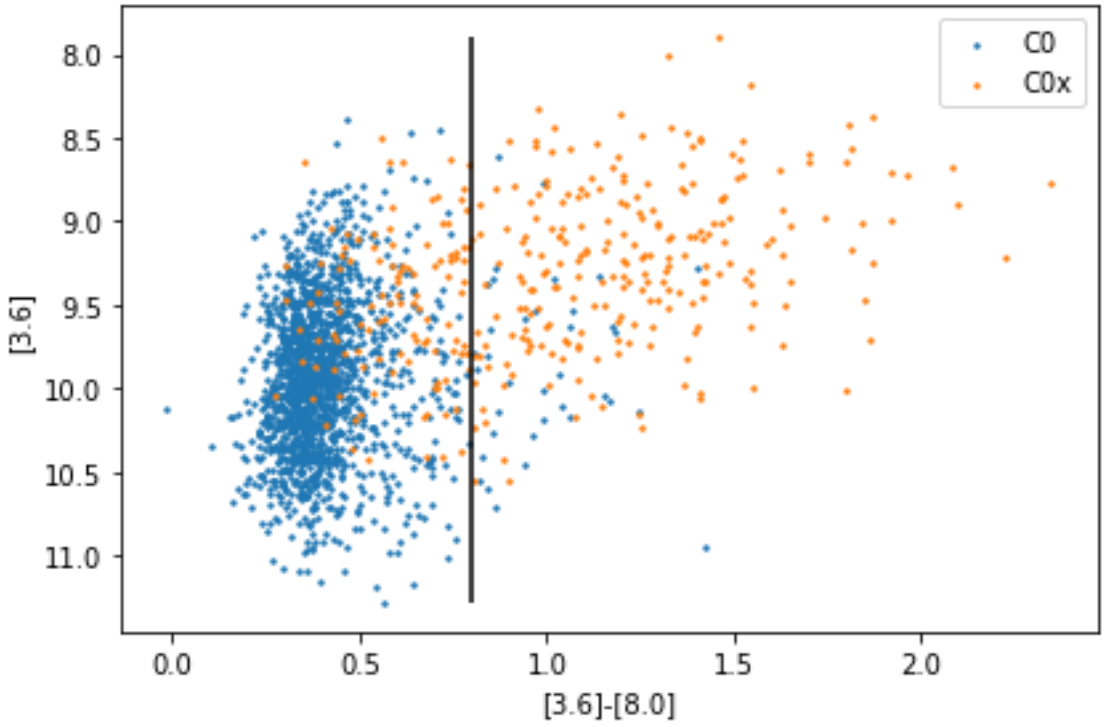
[31]:
```
C0StarData_n,C0StarData_x  = sort_xC0(C0StarData)
```






```
Non-sortable star found at index  0 . Assumed to not be extreme.
Non-sortable star found at index  227 . Assumed to not be extreme.
Non-sortable star found at index  243 . Assumed to not be extreme.
Non-sortable star found at index  250 . Assumed to not be extreme.
Non-sortable star found at index  748 . Assumed to not be extreme.
Non-sortable star found at index  917 . Assumed to not be extreme.
Non-sortable star found at index  1035 . Assumed to not be extreme.
Non-sortable star found at index  1189 . Assumed to not be extreme.
Non-sortable star found at index  1331 . Assumed to not be extreme.
Non-sortable star found at index  1339 . Assumed to not be extreme.
Non-sortable star found at index  1385 . Assumed to not be extreme.
Non-sortable star found at index  1597 . Assumed to not be extreme.
```


C0
C0x
J-[3.6]
[3.6]-[8.0]
7
6
5
4
3
2
0.0
0.5
1.0
1.5
2.0


C0
C0x
[3.6]
[3.6]-[8.0]
8.0
8.5
9.0
9.5
10.0
10.5
11.0
0.0
0.5
1.0
1.5
2.0

[32]:
```
# Functions to sort a list
def examine_index_third(elem):
    # Sorting Key. Will sort a list based on the specified index hard coded␣
 ↪here.
    return elem[tauIndex]

def sort_star_data_third(DataToSort):
    TransposedArray=np.transpose(DataToSort)
    TempList=TransposedArray.tolist()
    TempList=sorted(TempList,key = operator.itemgetter(1)) # use 2 because we␣
 ↪index from 0
    return np.transpose(np.asarray(TempList))

def RangeSelect (DataToTrim, LowerBound, UpperBound ):
    return np.transpose(np.
 ↪transpose(sort_star_data_third(DataToTrim))[LowerBound:UpperBound])
```

[33]:
```
# Save folder: Mdot_resid_v_color
def plot_mdot_resids_vs_color (star_data, band_1_ind, band_2_ind,␣
 ↪mdot_color_coeffs, comp_plus_mode = '', save_fig = False, save_folder = '',␣
 ↪less_than_value_color_1 = -np.inf, less_than_value_color_2 = -np.inf):
    # first, we will need to trim the data sets so they are the same size after␣
 ↪eliminating possible nAn entries.
    reduced_for_K_minus_eight = clear_rows_with_nAn_for_color(star_data,␣
 ↪KIndex, EightPointZeroIndex) # Hard code this color because we have already␣
 ↪settled on that as our first to examine. Could be changed if necessary.
    if less_than_value_color_1 != -np.inf:
        reduced_for_K_minus_eight =␣
 ↪clear_rows_with_color_less_than_value(reduced_for_K_minus_eight, KIndex,␣
 ↪EightPointZeroIndex, value=less_than_value_color_1 )
    reduced_for_chosen_color =␣
 ↪clear_rows_with_nAn_for_color(reduced_for_K_minus_eight, band_1_ind,␣
 ↪band_2_ind)
    if less_than_value_color_2 != -np.inf:
        reduced_for_chosen_color =␣
 ↪clear_rows_with_color_less_than_value(reduced_for_chosen_color, band_1_ind,␣
 ↪band_2_ind, value=less_than_value_color_2 )
    color_1 = calculate_color␣
 ↪(reduced_for_chosen_color[KIndex],reduced_for_chosen_color[EightPointZeroIndex])␣
 ↪# Hard code as K-[8.0]
    color_2 = calculate_color␣
 ↪(reduced_for_chosen_color[band_1_ind],reduced_for_chosen_color[band_2_ind])
    a = mdot_color_coeffs[0]
    b = mdot_color_coeffs[1]
    c = mdot_color_coeffs[2]
    mdot_from_color = calc_mdot_from_color(color_1, a, b, c)
```

```
    mdot_from_model = np.log10(reduced_for_chosen_color[MdotIndex])
    mdot_residual = mdot_from_color - mdot_from_model
    #print(mdot_from_model)
    chisqparams = chi_sq_fit_2d (color_2, mdot_residual )
    fit_parameters = chisqparams[0]
    line_coords = np.asarray([np.nanmin(color_2),np.nanmax(color_2)])
    print(fit_parameters)
    fit_x = line_coords
    fit_y = fit_parameters[0] + fit_parameters[1]*fit_x
    fit_label = "{:6e}".format(fit_parameters[0]) +'+'+ "{:6e}".
↪format(fit_parameters[1])+'*'+'Color'+'; R^2 ='+ "{:6e}".
↪format(chisqparams[1])
    plot.scatter(color_2,mdot_residual,s=2, label='Data')
    plot.plot(fit_x,fit_y, color='orange',label = fit_label)
    #plot.title(comp_plus_mode)
    # (J, H, K, 3.6micron, 4.5micron, 5.8micron, and 8.0micron)
    if band_1_ind == JIndex:
        band_1_str = 'J'
    if band_1_ind == HIndex:
        band_1_str = 'H'
    if band_1_ind == KIndex:
        band_1_str = 'K'
    if band_1_ind == ThreePointSixIndex:
        band_1_str = '[3.6]'
    if band_1_ind == FourPointFiveIndex:
        band_1_str = '[4.5]'
    if band_1_ind == FivePointEightIndex:
        band_1_str = '[5.8]'
    if band_1_ind == EightPointZeroIndex:
        band_1_str = '[8.0]'
    if band_2_ind == JIndex:
        band_2_str = 'K'
    if band_2_ind == HIndex:
        band_2_str = 'H'
    if band_2_ind == KIndex:
        band_2_str = 'K'
    if band_2_ind == ThreePointSixIndex:
        band_2_str = '[3.6]'
    if band_2_ind == FourPointFiveIndex:
        band_2_str = '[4.5]'
    if band_2_ind == FivePointEightIndex:
        band_2_str = '[5.8]'
    if band_2_ind == EightPointZeroIndex:
        band_2_str = '[8.0]'
    plot.xlabel('Color: ' + band_1_str + '-' + band_2_str)
    plot.ylabel(r'$\log \dot{M}_{color} - \log \dot{M}_{GRAMS\ model}$')
    plot.legend()
```

```
    if save_fig == True:
        plot.tight_layout()
        plot_name = save_folder + '/' + band_1_str + '-' + band_2_str + '_'+␣
↪comp_plus_mode
        plot.savefig(plot_name+'.png')
        plot.savefig(plot_name+'.pdf') # Why do journals require this format...
        plot.show()
    else:
        plot.tight_layout()
        plot.show()
    plot.close()

    return reduced_for_chosen_color
```

[34]:
```
def hyperbolic_func (C, P0, P1, P2):
    # logMdot = a/(C + b) + c
    return ((P0/(C+P1)))+P2

def hyperbolic_func_pls_lin (C, P0, P1, P2, P3):
    # logMdot = a/(K - [8.0] + b) -   col + (c -  ) => logMdot = P0/(C1 + {1}) -␣
↪P2*C2 + P3
    C0 = np.asarray(C[0])
    C1 = np.asarray(C[1])
    return ((P0/(C0+P1)))-P2*C1 + P3
```

[35]:
```
def fit_to_hyperbolic (xdata, ydata, save_fig = 0, plot_title = '',␣
↪x_label='',y_label='',y_label_sanitized='', save_folder = '', flip_axes = 0,␣
↪z_color_by_data=[],manual_data_bin_tau=0,␣
↪z_data_to_sort=[],initial_guess=[]):
    if len (initial_guess) >0:
        fit_parameters, fit_covariances = sp.optimize.
↪curve_fit(hyperbolic_func, xdata, ydata,p0=initial_guess)
    else:
        fit_parameters, fit_covariances = sp.optimize.
↪curve_fit(hyperbolic_func, xdata, ydata)
    fit_left = np.min(xdata)
    fit_right = np.max(xdata)
    #fit_left = 0
    #fit_right = 5
    #print(fit_left,fit_right)
    fit_data = [fit_left]
    new_elem = fit_left + np.abs((fit_right-fit_left))/100
    while len(fit_data)<100:
        fit_data.append(new_elem)
        new_elem = new_elem + np.abs((fit_right-fit_left))/100
    #print(fit_data)
```

```
   fit_curve = hyperbolic_func (fit_data, fit_parameters[0],␣
↪fit_parameters[1], fit_parameters[2])
   standard_dev_of_coeff = np.sqrt(np.diag(fit_covariances))
   fit_curve_up = hyperbolic_func (fit_data,␣
↪fit_parameters[0]-standard_dev_of_coeff[0],␣
↪fit_parameters[1]+standard_dev_of_coeff[1],␣
↪fit_parameters[2]-standard_dev_of_coeff[2])
   fit_curve_low = hyperbolic_func (fit_data,␣
↪fit_parameters[0]+standard_dev_of_coeff[0],␣
↪fit_parameters[1]-standard_dev_of_coeff[1],␣
↪fit_parameters[2]+standard_dev_of_coeff[2])

   fig, ax = plt.subplots()
   if flip_axes == 0:
       if len(z_color_by_data)>0:
           ax.scatter(xdata, ydata,␣
↪label='Data',s=2,c=z_color_by_data,cmap='Dark2')
       elif manual_data_bin_tau >0:
           zdata = np.asarray(z_data_to_sort)
           data_to_sort = np.transpose(np.asarray([xdata, ydata, zdata])) #␣
↪Stack our data so we can associate data points with each other
           a = data_to_sort
           ind=np.argsort(a[:,-1])
           sorted_data=np.transpose(np.asarray(a[ind]))
           #print('sorted_data:',sorted_data)
           #print(len(data_to_sort))
           #print('sorted_data:',sorted_data)
           #print(data_to_sort)
           bin_size = len(data_to_sort)//manual_data_bin_tau
           bin_bottom = 0
           bin_top = bin_size
           print('len(data_to_sort):',len(data_to_sort))
           i=0
           while i < manual_data_bin_tau:
               if i ==0: # When on first bin
                   working_x=sorted_data[0][:bin_top]
                   working_y=sorted_data[1][:bin_top]
                   data_label = 'tau<'+str( np.max(sorted_data[2][:
↪bin_size]))[0:7]
                   ax.scatter(working_x, working_y, label=data_label,s=2)
               elif i == manual_data_bin_tau-1: # If we are on the last bin
                   working_x=sorted_data[0][bin_bottom:]
                   working_y=sorted_data[1][bin_bottom:]
                   data_label = 'tau>'+str(np.min(sorted_data[2][bin_bottom:
↪]))[0:7]
                   ax.scatter(working_x, working_y, label=data_label,s=2)
```

```
                else: # We are on some intermediate bin
                    working_x=sorted_data[0][bin_bottom:bin_top]
                    working_y=sorted_data[1][bin_bottom:bin_top]
                    #print('tau range:',sorted_data[2][bin_bottom:bin_top])
                    data_label = str(np.min(sorted_data[2][bin_bottom:
↪bin_top]))[0:7] +'<tau<'+str(np.max(sorted_data[2][bin_bottom:bin_top]))[0:7]
                    ax.scatter(working_x, working_y, label=data_label,s=2)
                bin_bottom = bin_bottom+bin_size
                print('bin_bottom:',bin_bottom)
                bin_top = bin_top + bin_size
                print('bin_top:',bin_top)
                i = i + 1
        else:
            ax.scatter(xdata, ydata, label='Data',s=2)
        ax.plot(fit_data,fit_curve,label='fit: a=%5.3f, b=%5.3f, c=%5.3f' %␣
↪tuple(fit_parameters),color='orange')
        ax.text(0.03, .
↪97,plot_title,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
        #plot.plot(fit_data,fit_curve_up ,color='orange',linestyle='--')
        #plot.plot(fit_data,fit_curve_low ,color='orange',linestyle='--')
        #ax.set_title(plot_title, loc='left', pad=-100)
        #plot.legend()
        #plot.xlim(0,5)
        ax.set_ylim((np.min(ydata)-0.1), (np.max(ydata)+0.1))
        #plot.ylim(-8,-4.5)
        plot.xlabel(x_label)
        plot.ylabel(y_label)
        plot.tight_layout()
        if save_fig !=0:
            if y_label_sanitized != '':
                plot.savefig(save_folder+'/
↪'+plot_title+'_'+y_label_sanitized+'_vs_'+x_label+'.pdf')
                plot.savefig(save_folder+'/
↪'+plot_title+'_'+y_label_sanitized+'_vs_'+x_label+'.png')
            else:
                plot.savefig(save_folder+'/
↪'+plot_title+'_'+y_label+'_vs_'+x_label+'.pdf')
                plot.savefig(save_folder+'/
↪'+plot_title+'_'+y_label+'_vs_'+x_label+'.png')
            plot.show()
        else:
            plot.show()
        plot.close()
    else:
        if len(z_color_by_data)>0:
```

```
            ax.scatter(xdata, ydata,␣
↪label='Data',s=2,c=z_color_by_data,cmap='Dark2')
        elif manual_data_bin_tau >0:
            zdata = np.asarray(z_data_to_sort)
            data_to_sort = np.transpose(np.asarray([xdata, ydata, zdata])) #␣
↪Stack our data so we can associate data points with each other
            a = data_to_sort
            ind=np.argsort(a[:,-1])
            sorted_data=np.transpose(np.asarray(a[ind]))
            print('sorted_data:',sorted_data)
            #print(len(data_to_sort))
            #print('sorted_data:',sorted_data)
            #print(data_to_sort)
            bin_size = len(data_to_sort)//manual_data_bin_tau
            bin_bottom = 0
            bin_top = bin_size
            print('len(data_to_sort):',len(data_to_sort))
            i=0
            while i < manual_data_bin_tau:
                if i ==0: # When on first bin
                    working_x=sorted_data[0][:bin_top]
                    working_y=sorted_data[1][:bin_top]
                    data_label = 'tau<'+str( np.max(sorted_data[2][:
↪bin_size]))[0:7]
                    ax.scatter(working_x, working_y, label=data_label,s=2)
                elif i == manual_data_bin_tau-1: # If we are on the last bin
                    working_x=sorted_data[0][bin_bottom:]
                    working_y=sorted_data[1][bin_bottom:]
                    data_label = 'tau>'+str(np.min(sorted_data[2][bin_bottom:
↪]))[0:7]
                    ax.scatter(working_x, working_y, label=data_label,s=2)
                else: # We are on some intermediate bin
                    working_x=sorted_data[0][bin_bottom:bin_top]
                    working_y=sorted_data[1][bin_bottom:bin_top]
                    #print('tau range:',sorted_data[2][bin_bottom:bin_top])
                    data_label = str(np.min(sorted_data[2][bin_bottom:
↪bin_top]))[0:7] +'<tau<'+str(np.max(sorted_data[2][bin_bottom:bin_top]))[0:7]
                    ax.scatter(working_x, working_y, label=data_label,s=2)
                bin_bottom = bin_bottom+bin_size
                print('bin_bottom:',bin_bottom)
                bin_top = bin_top + bin_size
                print('bin_top:',bin_top)
                i = i + 1
        else:
            ax.scatter(xdata, ydata, label='Data',s=2)
        ax.plot(fit_curve,fit_data,label='fit: a=%5.3f, b=%5.3f, c=%5.3f' %␣
↪tuple(fit_parameters),color='orange')
```

```
        #ax.set_title(plot_title, loc='left', pad=-100)
        ax.text(0.03, .
↪97,plot_title,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
        #plot.legend()
        plot.xlabel(y_label)
        plot.ylabel(x_label)
        plot.tight_layout()
        if save_fig !=0:
            if y_label_sanitized != '':
                plot.savefig(save_folder+'/
↪'+plot_title+'_'+y_label_sanitized+'_vs_'+x_label+'.pdf')
                plot.savefig(save_folder+'/
↪'+plot_title+'_'+y_label_sanitized+'_vs_'+x_label+'.png')
            else:
                plot.savefig(save_folder+'/
↪'+plot_title+'_'+y_label+'_vs_'+x_label+'.pdf')
                plot.savefig(save_folder+'/
↪'+plot_title+'_'+y_label+'_vs_'+x_label+'.png')
            plot.show()
        else:
            plot.show()
        plot.close()
    return [fit_parameters[0], fit_parameters[1], fit_parameters[2]],␣
↪fit_covariances
```

[36]:
```
# Calculates the mass-loss rate from a hyperbolic fit to color
#                    a
#    log Mdot = _________ + c
#               color + b
def calc_mdot_from_color_1 (color_data, coeff_data): # Color_data should be a␣
↪numpy array. Using the function written, turn it into Mdot data for a given␣
↪star type
    color_log_mdot = coeff_data[0]/(color_data+coeff_data[1]) + coeff_data[2]
    return color_log_mdot
def calc_mdot_from_color (color_data, coeff_data): # Color_data should be a␣
↪numpy array. Using the function written, turn it into Mdot data for a given␣
↪star type
    color_log_mdot = coeff_data[0]/(color_data[0]+coeff_data[1]) -␣
↪coeff_data[2]*color_data[1] + coeff_data[3]
    return color_log_mdot
```

[37]:
```
def plot_x_y_data (xdata,ydata):
    plot.scatter(xdata,ydata,s=2)
    plot.show()
```

```
    plot.close()
```

[38]:
```
def calc_PE_scale (stellar_data):
    Mass = stellar_data[MIndex]
    Radius = stellar_data[RIndex]
    potential_energy_scale = np.log10(Mass/Radius)
    return potential_energy_scale

def calc_g_scale (stellar_data):
    Mass = stellar_data[MIndex]
    Radius = stellar_data[RIndex]
    g_scale = np.log10(Mass/Radius**2)
    return g_scale

def calc_lum_pressure_scale (stellar_data):
    Luminosity = stellar_data[LIndex]
    Radius = stellar_data[RIndex]
    L_Pressure_scale = np.log10(Luminosity/Radius**2)
    return L_Pressure_scale

def calc_core_lum_param (stellar_data):
    Luminosity = stellar_data[LIndex]
    Mass = stellar_data[MIndex]
    core_lum_param = np.log10(Luminosity/Mass)
    return core_lum_param
```

[39]:
```
def plot_logmdot_vs_parameter (mdot, parameter, title='',␣
↪parameter_label='',save_fig = 0, save_folder = '',testing = 0):
    fig, ax = plt.subplots()
    plot.scatter(parameter,mdot,s=2,label='Data')
    chisqparams = chi_sq_fit_2d (parameter, mdot)
    fit_parameters = chisqparams[0]
    fit_x = np.asarray([np.nanmin(parameter), np.nanmax(parameter)])
    if testing != 0:
        print('parameter:',parameter)
        print('np.min(parameter):',np.nanmin(parameter))
        print('np.max(parameter):',np.nanmax(parameter))
        print('fit_x:',fit_x)
    fit_y = fit_parameters[0] + fit_parameters[1]*fit_x
    if testing != 0:
        print('fit_y:',fit_x)
    fit_label = "{:6e}".format(fit_parameters[0]) +'+'+ "{:6e}".
↪format(fit_parameters[1])+'*('+parameter_label+'); R^2 ='+ "{:6e}".
↪format(chisqparams[1])
    plot.plot(fit_x,fit_y,color='orange', label = fit_label)
    #plot.title(title)
```

```
    ax.text(0.03, .
↪97,title,horizontalalignment='center',verticalalignment='center',transform =␣
↪ax.transAxes,fontsize=10)
    plot.xlabel(parameter_label)
    plot.ylabel('logMdot_obsv')
    plot.legend(loc=8)
    if save_fig != 0:
            plot_name = save_folder + '/' + 'logMdot_versus_' + parameter_label␣
↪+ '/' + title
            plot_name = plot_name.replace('(M/R)', '(MdivR)') # Illegal␣
↪character on Windows. Replace the Directory character with a simple string
            plot_name = plot_name.replace('(M/R^2)', '(MdivR^2)') # Illegal␣
↪character on Windows. Replace the Directory character with a simple string
            plot_name = plot_name.replace('(L/R)', '(LdivR)') # Illegal␣
↪character on Windows. Replace the Directory character with a simple string
            plot_name = plot_name.replace('(L/R^2)', '(LdivR^2)') # Illegal␣
↪character on Windows. Replace the Directory character with a simple string
            plot_name = plot_name.replace('(L/M)', '(LdivM)') # Illegal␣
↪character on Windows. Replace the Directory character with a simple string
            plot.savefig(plot_name+'.png')
            plot.savefig(plot_name+'.pdf') # Why do journals require this␣
↪format...
    else:
        plot.show()
    plot.close()
```

[40]:
```
def plot_residual_vs_parameter (mdot_resid, parameter, title='',␣
↪parameter_label='',save_fig = 0, save_folder = ''):
    fig, ax = plt.subplots()
    plot.scatter(parameter,mdot_resid,s=2, label = 'Data')
    chisqparams = chi_sq_fit_2d (parameter, mdot_resid)
    fit_parameters = chisqparams[0]
    fit_x = np.asarray([np.nanmin(parameter), np.nanmax(parameter)])
    fit_y = fit_parameters[0] + fit_parameters[1]*fit_x
    fit_label = "{:6e}".format(fit_parameters[0]) +'+'+ "{:6e}".
↪format(fit_parameters[1])+'*('+parameter_label+'); R^2 ='+ "{:6e}".
↪format(chisqparams[1])
    plot.plot(fit_x,fit_y,color='orange', label = fit_label)
    #plot.title(title)
    ax.text(0.03, .
↪97,title,horizontalalignment='center',verticalalignment='center',transform =␣
↪ax.transAxes,fontsize=10)
    plot.xlabel(parameter_label)
    plot.ylabel('logMdot_calc-logMdot_obsv')
    plot.legend(loc=8)
    if save_fig != 0:
```

```
            plot_name = save_folder + '/' + 'Residuals/'+ 'residual_versus_'+␣
↪parameter_label + '/' + title
            plot_name = plot_name.replace('(M/R)', '(MdivR)') # Illegal␣
↪character on Windows. Replace the Directory character with a simple string
            plot_name = plot_name.replace('(M/R^2)', '(MdivR^2)') # Illegal␣
↪character on Windows. Replace the Directory character with a simple string
            plot_name = plot_name.replace('(L/R)', '(LdivR)') # Illegal␣
↪character on Windows. Replace the Directory character with a simple string
            plot_name = plot_name.replace('(L/R^2)', '(LdivR^2)') # Illegal␣
↪character on Windows. Replace the Directory character with a simple string
            plot_name = plot_name.replace('(L/M)', '(LdivM)') # Illegal␣
↪character on Windows. Replace the Directory character with a simple string
            plot.savefig(plot_name+'.png')
            plot.savefig(plot_name+'.pdf') # Why do journals require this␣
↪format...
    else:
        plot.show()
    plot.close()
    return fit_parameters
```

[41]:
```
def plot_logmdot_vs_logmdot (mdot, calc_mdot, title='',save_fig = 0,␣
↪save_folder = ''):
    fig, ax = plt.subplots()
    plot.scatter(mdot,calc_mdot,s=2, label = 'Data')
    chisqparams = chi_sq_fit_2d (mdot, calc_mdot)
    fit_parameters = chisqparams[0]
    line_coords = np.asarray([np.min([np.nanmin(mdot),np.nanmin(calc_mdot)]),np.
↪nanmax([np.max(mdot),np.nanmax(calc_mdot)])])
    fit_x = line_coords
    fit_y = fit_parameters[0] + fit_parameters[1]*fit_x
    fit_label = "{:6e}".format(fit_parameters[0]) +'+'+ "{:6e}".
↪format(fit_parameters[1])+'*'+'logMdot_obsv'+'; R^2 ='+ "{:6e}".
↪format(chisqparams[1])
    plot.plot(line_coords,line_coords,color='black', label = '1:1 Line')
    plot.plot(fit_x,fit_y,color='orange', label = fit_label)
    #plot.title(title)
    ax.text(0.03, .
↪97,title,horizontalalignment='center',verticalalignment='center',transform =␣
↪ax.transAxes,fontsize=10)
    plot.xlabel('logMdot_obsv')
    plot.ylabel('logMdot_calc')
    plot.legend(loc=8)
    plot.tight_layout()
    if save_fig != 0:
            plot_name = save_folder + '/' + 'logMdot_calc_versus_logMdot_obsv'␣
↪+ '/' + title
```

```
            plot_name = plot_name.replace('(M/R)', '(MdivR)') # Illegal␣
↪character on Windows. Replace the Directory character with a simple string
            plot_name = plot_name.replace('(M/R^2)', '(MdivR^2)') # Illegal␣
↪character on Windows. Replace the Directory character with a simple string
            plot_name = plot_name.replace('(L/R)', '(LdivR)') # Illegal␣
↪character on Windows. Replace the Directory character with a simple string
            plot_name = plot_name.replace('(L/R^2)', '(LdivR^2)') # Illegal␣
↪character on Windows. Replace the Directory character with a simple string
            plot_name = plot_name.replace('(L/M)', '(LdivM)') # Illegal␣
↪character on Windows. Replace the Directory character with a simple string
            plot.savefig(plot_name+'.png')
            plot.savefig(plot_name+'.pdf') # Why do journals require this␣
↪format...
    else:
        plot.show()
    plot.close()
```

[42]:
```
def full_parameter_analysis (stellar_data_x, stellar_data_mdot,␣
↪full_stellar_data, stellar_data_xlabel='', plot_label = '', save_folder =␣
↪'', save_fig = 0):
    #fig, ax = plt.subplots()
    # Data Analysis: Do a chi^2 fit of our chosen parameter vs Mdot for our␣
↪star, and find the residuals of that fit
    chisqfit_returns = chi_sq_fit_2d (stellar_data_x,stellar_data_mdot)
    Betas = chisqfit_returns[0]
    RSquared = chisqfit_returns[1]
    print('\n Fit: logMdot=', Betas[1],'*x+',Betas[0])
    logMdot_calc = Betas[1]*stellar_data_x+Betas[0]
    logMdot_residual = logMdot_calc- stellar_data_mdot
    # First Plot: Chosen Parameter vs Mdot, so we can see what the chi^2 fit␣
↪looks like
    plot_logmdot_vs_parameter (stellar_data_mdot, stellar_data_x,␣
↪title=plot_label, parameter_label=stellar_data_xlabel,save_folder =␣
↪save_folder,save_fig = save_fig)
    #plot.scatter(stellar_data_x,stellar_data_mdot,s=2)
    #fit_x = np.asarray([np.min(stellar_data_x), np.max(stellar_data_x)])
    #fit_y = Betas[0] + Betas[1]*fit_x
    #plot.plot(fit_x,fit_y,color='orange')
    #plot.xlabel(stellar_data_xlabel)
    #plot.ylabel('logMdot_obsv')
    #plot.title(plot_label)
    #ax.text(0.03, .
↪97,plot_label,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
    #plot.show()
    #plot.close()
```

```
    # Data Analysis: For the input, calculate some proxies for stellar␣
↪parameters (so they don't have to be fed to the function individually)
    logPE = calc_PE_scale(full_stellar_data)
    logg = calc_g_scale(full_stellar_data)
    logLP = calc_lum_pressure_scale(full_stellar_data)
    logCL = calc_core_lum_param(full_stellar_data)
    # More plots
    plot_residual_vs_parameter(logMdot_residual,np.
↪log10(full_stellar_data[PIndex]),title=plot_label,parameter_label='logP',save_folder␣
↪= save_folder,save_fig = save_fig)
    plot_residual_vs_parameter(logMdot_residual,np.
↪log10(full_stellar_data[LIndex]),title=plot_label,parameter_label='logL',save_folder␣
↪= save_folder,save_fig = save_fig)
    plot_residual_vs_parameter(logMdot_residual,np.
↪log10(full_stellar_data[MIndex]),title=plot_label,parameter_label='logM',save_folder␣
↪= save_folder,save_fig = save_fig)
  ␣
↪plot_residual_vs_parameter(logMdot_residual,logPE,title=plot_label,parameter_label='log(M/
↪R)',save_folder = save_folder,save_fig = save_fig)
  ␣
↪plot_residual_vs_parameter(logMdot_residual,logg,title=plot_label,parameter_label='log(M/
↪R^2)',save_folder = save_folder,save_fig = save_fig)
  ␣
↪plot_residual_vs_parameter(logMdot_residual,logLP,title=plot_label,parameter_label='log(L/
↪R^2)',save_folder = save_folder,save_fig = save_fig)
  ␣
↪plot_residual_vs_parameter(logMdot_residual,logCL,title=plot_label,parameter_label='log(L/
↪M)',save_folder = save_folder,save_fig = save_fig)

    color_J_minus_ThreePointSix = np.asarray(full_stellar_data[JIndex])-np.
↪asarray(full_stellar_data[ThreePointSixIndex])
    color_ThreePointSix_minus_EightPointZero = np.
↪asarray(full_stellar_data[ThreePointSixIndex])-np.
↪asarray(full_stellar_data[EightPointZeroIndex])
    color_J_minus_K = np.asarray(full_stellar_data[JIndex])-np.
↪asarray(full_stellar_data[KIndex])
    color_K_minus_EightPointZero = np.asarray(full_stellar_data[KIndex])-np.
↪asarray(full_stellar_data[EightPointZeroIndex])
    plot_logmdot_vs_parameter␣
↪(stellar_data_mdot,color_ThreePointSix_minus_EightPointZero ,␣
↪title=plot_label, parameter_label='[3.6]-[8.0]')
    plot_logmdot_vs_parameter (stellar_data_mdot, color_K_minus_EightPointZero,␣
↪title=plot_label, parameter_label='K-[8.0]')
  ␣
↪plot_residual_vs_parameter(logMdot_residual,color_ThreePointSix_minus_EightPointZero,title=
↪6]-[8.0]',save_folder = save_folder,save_fig = save_fig)
```

```
  ␣
↪plot_residual_vs_parameter(logMdot_residual,color_K_minus_EightPointZero,title=plot_label,p
↪0]',save_folder = save_folder,save_fig = save_fig)

   plot_logmdot_vs_logmdot(stellar_data_mdot, logMdot_calc,␣
↪title=plot_label,save_folder = save_folder,save_fig = save_fig)
```

[43]:
```
def multilinear_fit_2var_WLS (xdata, ydata, xdata_error, ydata_error):
    std_error_x0 = np.nanstd(xdata_error)
    std_error_y = np.nanstd(ydata_error)
    x0 = xdata[0] # logL
    x1 = xdata[1] # logP
    y = ydata # logMdot

    wls_model = sm.WLS(ydata, xdata, [1/std_error_x0**2, 1, 1/std_error_y**2])

    # Parameters are returned in the format intercept, slope 1, slope 2
    return model._results.params, model._results.rsquared, model.bse
```

[44]:
```
def do_final_fits(bulk_raw_data,mdot_data, fed_label = '',star_type_label=''):
    break_line =␣
↪'_____\n'

    ############################################################
    # Fit 1: Multilinear of L,P vs provided Mdot
    ############################################################
    fig, ax = plt.subplots()
    logLData = np.log10(bulk_raw_data[LIndex])
    logPData = np.log10(bulk_raw_data[PIndex])
    print('Fit 1: Multilinear of L,P vs provided Mdot')
    LPmultilinear_fit = multilinear_fit_2var([logLData,logPData],mdot_data)
    ## PRINT: Coefficients of fit, functional form
    if LPmultilinear_fit[0][1] >= 0:
        if LPmultilinear_fit[0][2] >= 0:
            print('Fit:', LPmultilinear_fit[0][0], '+',␣
↪LPmultilinear_fit[0][1],'logL', '+', LPmultilinear_fit[0][2],'logP')
        else:
            print('Fit:', LPmultilinear_fit[0][0], '+',␣
↪LPmultilinear_fit[0][1],'logL', '-', np.abs(LPmultilinear_fit[0][2]),'logP')
    else:
        if LPmultilinear_fit[0][2] >= 0:
            print('Fit:', LPmultilinear_fit[0][0], '-',np.
↪abs(LPmultilinear_fit[0][1]),'logL', '+', LPmultilinear_fit[0][2],'logP')
        else:
```

```
            print('Fit:', LPmultilinear_fit[0][0], '-', np.
↪abs(LPmultilinear_fit[0][1]),'logL', '-', np.
↪abs(LPmultilinear_fit[0][2]),'logP')
    print('R^2:', LPmultilinear_fit[1])
    print('Standard Errors (x0 = logL, x1 = logP):\n', LPmultilinear_fit[2])
    LPCoeffs = LPmultilinear_fit[0]
    plot.scatter(logLData,mdot_data, color='tab:blue',s=2)
    plot.xlabel(r'$\log L$')
    if fed_label == "GRAMS":
        plot.ylabel(r'$log \dot{M}\ \mathrm{(GRAMS)}$')
    elif fed_label == "Color":
        plot.ylabel(r'$log \dot{M}\ \mathrm{(Color)}$')
    else:
        plot.ylabel(r'$log \dot{M}\ \mathrm{(Data)}$')
    #plot.title(star_type_label)
    ax.text(0.03, .
↪97,star_type_label,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
    plot_name = Directory + '/' + star_type_label + '_Mdot'+ fed_label +'_vs_'␣
↪+ 'logL'
    plot.savefig(plot_name+'.png')
    plot.savefig(plot_name+'.pdf') # Why do journals require this format...
    plot.show()
    plot.close()
    fig, ax = plt.subplots()
    plot.scatter(logPData,mdot_data, color='tab:blue',s=2)
    plot.xlabel(r'$\log P$')
    if fed_label == "GRAMS":
        plot.ylabel(r'$log \dot{M}\ \mathrm{(GRAMS)}$')
    elif fed_label == "Color":
        plot.ylabel(r'$log \dot{M}\ \mathrm{(Color)}$')
    else:
        plot.ylabel(r'$log \dot{M}\ \mathrm{(Data)}$')
    #plot.title(star_type_label)
    ax.text(0.03, .
↪97,star_type_label,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
    plot_name = Directory + '/' + star_type_label + '_Mdot'+ fed_label +'_vs_'␣
↪+ 'logP'
    plot.tight_layout()
    plot.savefig(plot_name+'.png')
    plot.savefig(plot_name+'.pdf') # Why do journals require this format...
    plot.show()
    plot.close()
    #fit_1_flat = np.
↪asarray([multilinear_fit[0][0],multilinear_fit[0][1],multilinear_fit[0][2],multilinear_fit[
```

```
    ## Append to returned array: Coefficients of fit
    print(break_line)

    ############################################################
    # Fit 2: provided Mdot vs FormulaMdot
    ############################################################
    # Figure 1: Fit vs data
    fig, ax = plt.subplots()
    if fed_label == "GRAMS":
        print('Fit 2: GRAMS Mdot vs Formula Mdot')
    elif fed_label == "Color":
        print('Fit 2: Color Mdot vs Formula Mdot')
    else:
        print('Fit 2: Data Mdot vs Formula Mdot')
    FormulaMdot = LPCoeffs[0] + LPCoeffs[1]*logLData + LPCoeffs[2]*logPData
    form_vs_fed_fit = linear_fit_1var(FormulaMdot,mdot_data)
    form_vs_fed_Coeffs = form_vs_fed_fit[0]
    print('form_vs_fed_Coeffs',form_vs_fed_Coeffs)
    plot.scatter(FormulaMdot,mdot_data, color='tab:blue',s=2)
    plot.xlabel(r'$\log \dot{M}\  \mathrm{(Formula)}$')
    if fed_label == "GRAMS":
        plot.ylabel(r'$\log \dot{M} (GRAMS)$')
    elif fed_label == "Color":
        plot.ylabel(r'$\log \dot{M} (Color)$')
    else:
        plot.ylabel(r'$\log \dot{M} (Data)$')
    #plot.title(star_type_label)
    ax.text(0.03, .
↪97,star_type_label,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
    # And the fit line...
    limits = np.asarray([np.nanmin([np.nanmin(FormulaMdot),np.
↪nanmin(mdot_data)]), np.nanmax([np.nanmax(FormulaMdot),np.
↪nanmax(mdot_data)])])
    min_val = limits[0]
    max_val = limits[1]
    quality_line_x = np.asarray([min_val,max_val])
    quality_line_y = form_vs_fed_Coeffs[0] +␣
↪form_vs_fed_Coeffs[1]*quality_line_x
    plot.plot(quality_line_x,quality_line_y,color='orange')
    #print('limits',limits)
    plot.
↪plot([limits[0],limits[1]],[limits[0],limits[1]],color='black',linestyle='-.
↪')
    plot.xlim(limits[0]-0.1,limits[1]+0.1)
    plot.ylim(limits[0]-0.1,limits[1]+0.1)
    plot.tight_layout()
```

```
    plot_name = Directory + '/' + star_type_label + '_Mdot'+ fed_label +'_vs_'␣
↪+ 'MdotFormula'
    plot.savefig(plot_name+'.png')
    plot.savefig(plot_name+'.pdf') # Why do journals require this format...
    plot.show()
    plot.close()
    ## PRINT: Coefficients of fit, formula of fit line
    print('Quality of Fit Line:')
    if fed_label == "GRAMS":
        print('Fit: logMdot_GRAMS=%5.3f + %5.3f * logMdot_Formula' %␣
↪tuple(form_vs_fed_Coeffs) )
    elif fed_label == "Color":
        print('Fit: logMdot_Color=%5.3f + %5.3f * logMdot_Formula' %␣
↪tuple(form_vs_fed_Coeffs) )
    else:
        print('Fit: logMdot_Data=%5.3f + %5.3f * logMdot_Formula' %␣
↪tuple(form_vs_fed_Coeffs) )
    print('R^2:',form_vs_fed_fit[1])
    ## Append to returned array: Coefficients of fit
    residual = mdot_data - FormulaMdot
    # Figure 2: logMdot (data) - ClogP vs logL
    fig, ax = plt.subplots()
    plot.scatter(logLData, mdot_data - LPCoeffs[2]*logPData,s=2)
    plot.plot(logLData, FormulaMdot -␣
↪LPCoeffs[2]*logPData,marker='',color='orange')
    plot.xlabel(r'$\log L$')

    if LPmultilinear_fit[2][2] < 0:
        i = 2 # start after decimal, knowing the error is less than 1
    else:
        i = 0
    while True:
        if str(np.abs(LPmultilinear_fit[2][2]))[i] == '0':
            i += 1
        else:
            break
    rounded_coeff = np.round(LPCoeffs[2],i)
    plot.ylabel(r'$\log \dot{M} -' + str(rounded_coeff)+r'\log P$')
    plot_name = Directory + '/' + star_type_label + '_Mdot-P'+ fed_label␣
↪+'_vs_' + 'MdotFormula'
    #plot.title(star_type_label)
    ax.text(0.03, .
↪97,star_type_label,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
    plot.tight_layout()
    plot.savefig(plot_name+'.png')
```

```
    plot.savefig(plot_name+'.pdf') # Why do journals require this format...
    plot.show()
    plot.close()
    # Figure 3: logMdot (data) - ClogP vs logP
    fig, ax = plt.subplots()
    plot.scatter(logPData, mdot_data - LPCoeffs[1]*logLData,s=2)
    plot.plot(logPData, FormulaMdot - LPCoeffs[1]*logLData,marker='',color='orange')
    plot.xlabel(r'$\log P$')

    if LPmultilinear_fit[2][1] < 0:
        i = 2 # start after decimal, knowing the error is less than 1
    else:
        i = 0
    while True:
        if str(np.abs(LPmultilinear_fit[2][1]))[i] == '0':
            i += 1
        else:
            break
    rounded_coeff = np.round(LPCoeffs[1],i)
    plot.ylabel(r'$\log \dot{M} +' + str(rounded_coeff)+r'\log L$')
    plot_name = Directory + '/' + star_type_label + '_Mdot-L'+ fed_label+'_vs_' + 'MdotFormula'
    #plot.title(star_type_label)
    ax.text(0.03, .97,star_type_label,horizontalalignment='center',verticalalignment='center',transform= ax.transAxes,fontsize=10)
    plot.tight_layout()
    plot.savefig(plot_name+'.png')
    plot.savefig(plot_name+'.pdf') # Why do journals require this format...
    plot.show()
    plot.close()
    print(break_line)

    ############################################################
    # Fit 3: Residual provided Mdot-FormulaMdot vs Luminosity
    ############################################################
    fig, ax = plt.subplots()
    if fed_label == "GRAMS":
        print('Fit 3: Mdot Residual (GRAMS-Formula) vs logL')
    elif fed_label == "Color":
        print('Fit 3: Mdot Residual (Color-Formula) vs logL')
    else:
        print('Fit 3: Mdot Residual (Data-Formula) vs logL')
    resid_vs_L_fit = linear_fit_1var(logLData,residual)
    resid_vs_L_Coeffs = resid_vs_L_fit[0]
    plot.scatter(logLData, residual, color='tab:blue',s=2)
```

```
    plot.xlabel('logL')
    if fed_label == "GRAMS":
        plot.ylabel(r'$\log \dot{M}\ \mathrm{(GRAMS)} - \log \dot{M}\␣
↪\mathrm{(Formula)}$')
    elif fed_label == "Color":
        plot.ylabel(r'$\log \dot{M}\ \mathrm{(Color)} - \log \dot{M}\␣
↪\mathrm{(Formula)}$')
    else:
        plot.ylabel(r'$\log \dot{M}\ \mathrm{(Data)} - \log \dot{M}\␣
↪\mathrm{(Formula)}$')
    #plot.title(star_type_label)
    ax.text(0.03, .
↪97,star_type_label,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
    fit_line_x = np.asarray([np.nanmin(logLData),np.nanmax(logLData)])
    fit_line_y = resid_vs_L_Coeffs[0] + resid_vs_L_Coeffs[1]*fit_line_x
    plot.plot(fit_line_x,fit_line_y,color='orange')
    plot.tight_layout()
    plot_name = Directory + '/' + star_type_label + '_Mdot'+ fed_label +'-' +␣
↪'MdotFormula' + '_vs_' + 'logL'
    plot.savefig(plot_name+'.png')
    plot.savefig(plot_name+'.pdf') # Why do journals require this format...
    plot.show()
    plot.close()
    ## PRINT: Coefficients of fit, formula of fit line
    print('Quality of Residual Fit:')
    if fed_label == "GRAMS":
        print('Fit: logMdot (GRAMS) - logMdot (Formula)=%5.3f + %5.3f * logL' %␣
↪tuple(resid_vs_L_Coeffs) )
    elif fed_label == "Color":
        print('Fit: logMdot (Color) - logMdot (Formula)=%5.3f + %5.3f * logL' %␣
↪tuple(resid_vs_L_Coeffs) )
    else:
        print('Fit: logMdot (Data) - logMdot (Formula)=%5.3f + %5.3f * logL' %␣
↪tuple(resid_vs_L_Coeffs) )
    print('R^2:',resid_vs_L_fit[1])
    ## Append to returned array: Coefficients of fit
    print(break_line)

    ############################################################
    # Fit 4: Residual provided Mdot-FormulaMdot vs Period
    ############################################################
    fig, ax = plt.subplots()
    if fed_label == "GRAMS":
        print('Fit 4: Mdot Residual (GRAMS-Formula) vs logP')
    elif fed_label == "Color":
```

```
        print('Fit 4: Mdot Residual (Color-Formula) vs logP')
    else:
        print('Fit 4: Mdot Residual (Data-Formula) vs logP')
    resid_vs_P_fit = linear_fit_1var(logPData,residual)
    resid_vs_P_Coeffs = resid_vs_P_fit[0]
    plot.scatter(logPData, residual, color='tab:blue',s=2)
    plot.xlabel('logP')
    if fed_label == "GRAMS":
        plot.ylabel(r'$\log \dot{M}\ \mathrm{(GRAMS)} - \log \dot{M}\␣
↪\mathrm{(Formula)}$')
    elif fed_label == "Color":
        plot.ylabel(r'$\log \dot{M}\ \mathrm{(Color)} - \log \dot{M}\␣
↪\mathrm{(Formula)}$')
    else:
        plot.ylabel(r'$\log \dot{M}\ \mathrm{(Data)} - \log \dot{M}\␣
↪\mathrm{(Formula)}$')
    #plot.title(star_type_label)
    ax.text(0.03, .
↪97,star_type_label,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
    fit_line_x = np.asarray([np.nanmin(logPData),np.nanmax(logPData)])
    fit_line_y = resid_vs_P_Coeffs[0] + resid_vs_P_Coeffs[1]*fit_line_x
    plot.plot(fit_line_x,fit_line_y,color='orange')
    plot.tight_layout()
    plot_name = Directory + '/' + star_type_label + '_Mdot'+ fed_label +'-' +␣
↪'MdotFormula' + '_vs_' + 'logP'
    plot.savefig(plot_name+'.png')
    plot.savefig(plot_name+'.pdf') # Why do journals require this format...
    plot.show()
    plot.close()
    ## PRINT: Coefficients of fit, formula of fit line
    print('Quality of Residual Fit:')
    if fed_label == "GRAMS":
        print('Fit: logMdot (GRAMS) - logMdot (Formula)=%5.3f + %5.3f * logP' %␣
↪tuple(resid_vs_P_Coeffs) )
    elif fed_label == "Color":
        print('Fit: logMdot (Color) - logMdot (Formula)=%5.3f + %5.3f * logP' %␣
↪tuple(resid_vs_P_Coeffs) )
    else:
        print('Fit: logMdot (Data) - logMdot (Formula)=%5.3f + %5.3f * logP' %␣
↪tuple(resid_vs_P_Coeffs) )
    print('R^2:',resid_vs_P_fit[1])
    ## PRINT: Coefficients of fit, formula of fit line
    ## Append to returned array: Coefficients of fit
```

```
    return LPCoeffs # Information for the fits. Need this to find the␣
↪differentials between pairs of formulas
```

[45]:
```
def helpful_plots_different_mdot (bulk_raw_data,mdot_data, fed_label =␣
↪'',star_type_label=''):
    color_mdot = mdot_data[0]
    GRAMS_mdot = mdot_data[1]
    logLData = np.log10(bulk_raw_data[LIndex])
    logPData = np.log10(bulk_raw_data[PIndex])

    ####################
    # Fit L, P to Mdot
    ####################
    LPmultilinear_fit_color =␣
↪multilinear_fit_2var([logLData,logPData],color_mdot)
    LPmultilinear_fit_GRAMS =␣
↪multilinear_fit_2var([logLData,logPData],GRAMS_mdot)
    LPCoeffs_color = LPmultilinear_fit_color[0]
    LPCoeffs_GRAMS = LPmultilinear_fit_GRAMS[0]

    ####################
    # Calculate Mdot from two formulas (the fit)
    ####################
    FormulaMdot_color = LPCoeffs_color[0] + LPCoeffs_color[1]*logLData +␣
↪LPCoeffs_color[2]*logPData
    FormulaMdot_GRAMS = LPCoeffs_GRAMS[0] + LPCoeffs_GRAMS[1]*logLData +␣
↪LPCoeffs_GRAMS[2]*logPData

    # Plot 1
    fig, ax1 = plot.subplots()
    ax1.set_xlabel(r'$\log L$')
    ax1.set_ylabel(r'$\log \dot{M}_{\mathrm{color}}-$'+str(LPCoeffs_color[2])[0:
↪4]+r'$\times \log P$')
    ax1.scatter(logLData, color_mdot-LPCoeffs_color[2]*logPData, color = 'tab:
↪blue',s=2,alpha=0.3)
    ax1.plot(logLData, FormulaMdot_color-LPCoeffs_color[2]*logPData, color =␣
↪'tab:blue',label='Color')

    ax2 = ax1.twinx()
    ax2.set_ylabel(r'$\log \dot{M}_{\mathrm{GRAMS}}-$'+str(LPCoeffs_GRAMS[2])[0:
↪4]+r'$\times \log P$')
    ax2.scatter(logLData, GRAMS_mdot-LPCoeffs_GRAMS[2]*logPData, color = 'tab:
↪orange',s=2,alpha=0.3)
    ax2.plot(logLData, FormulaMdot_GRAMS-LPCoeffs_GRAMS[2]*logPData, color =␣
↪'tab:orange',label='GRAMS')
```

```
    #plot.title(star_type_label)
    ax1.text(0.03, .
↪97,star_type_label,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax1.transAxes,fontsize=10)
    fig.tight_layout()
    plot_name = Directory + '/' + star_type_label + '_logMdot-ClogP_vs_' +␣
↪'logL'
    plot.savefig(plot_name+'.png')
    plot.savefig(plot_name+'.pdf') # Why do journals require this format...
    plot.show()
    plot.close()

    # Plot 2
    fig, ax1 = plot.subplots()
    ax1.set_xlabel(r'$\log P$')
    ax1.set_ylabel(r'$\log \dot{M}_{\mathrm{color}}+$'+str(LPCoeffs_color[1])[1:
↪5]+r'$\times \log L$')
    ax1.scatter(logPData, color_mdot-LPCoeffs_color[1]*logLData, color = 'tab:
↪blue',s=2,alpha=0.25)
    ax1.plot(logPData, FormulaMdot_color-LPCoeffs_color[1]*logLData, color =␣
↪'tab:blue',label='Color')

    ax2 = ax1.twinx()
    ax2.set_ylabel(r'$\log \dot{M}_{\mathrm{GRAMS}}+$'+str(LPCoeffs_GRAMS[1])[1:
↪5]+r'$\times \log L$')
    ax2.scatter(logPData, GRAMS_mdot-LPCoeffs_GRAMS[1]*logLData, color = 'tab:
↪orange',s=2,alpha=0.25)
    ax2.plot(logPData, FormulaMdot_GRAMS-LPCoeffs_GRAMS[1]*logLData, color =␣
↪'tab:orange',label='GRAMS')


    #plot.title(star_type_label)
    ax1.text(0.03, .
↪97,star_type_label,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax1.transAxes,fontsize=10)
    fig.tight_layout()
    plot_name = Directory + '/' + star_type_label + '_logMdot-BlogL_vs_' +␣
↪'logP'
    plot.savefig(plot_name+'.png')
    plot.savefig(plot_name+'.pdf') # Why do journals require this format...
    plot.show()
    plot.close()
```

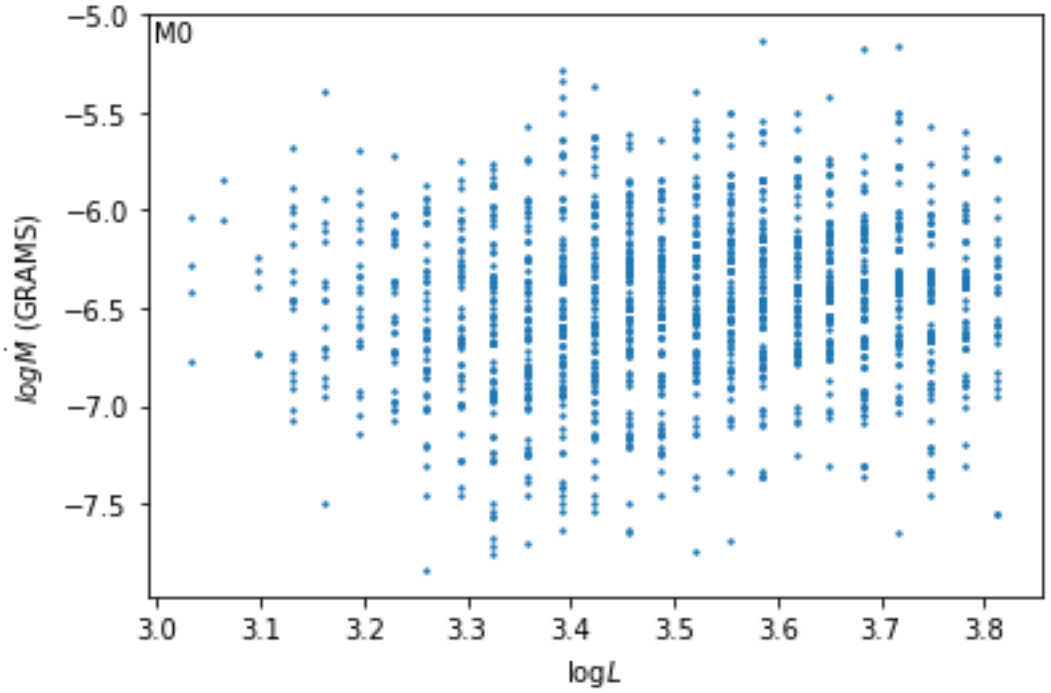
[46]:
```
Directory = 'Final_Fit/Color_and_GRAMS'
```

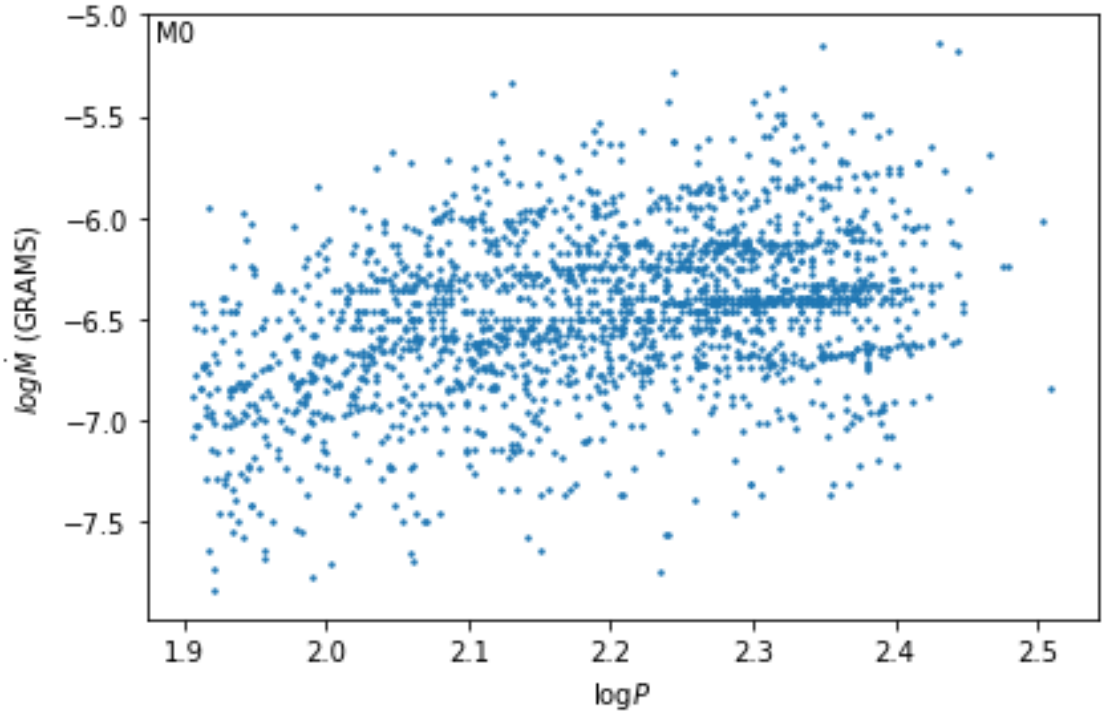
[47]:
```
Directory = 'Final_Fit/Color'
```

[48]:
```
Directory = 'Final_Fit/GRAMS'
```

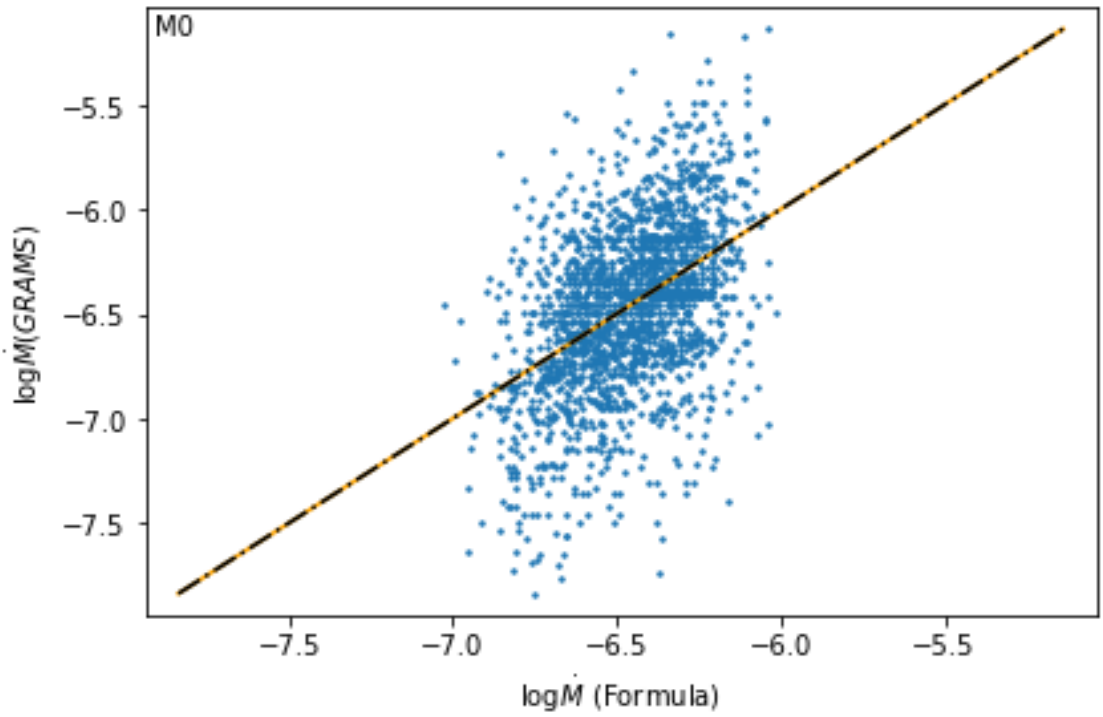
[49]:
```
LPCoeffsM0_GRAMS = do_final_fits(M0StarData,np.log10(M0StarData[MdotIndex]),␣
↪fed_label = 'GRAMS',star_type_label='M0')
```

```
Fit 1: Multilinear of L,P vs provided Mdot
Fit: -7.388158771513019 - 1.005665316322142 logL + 2.0387720751424845 logP
R^2: 0.20636545213502622
Standard Errors (x0 = logL, x1 = logP):
 Intercept    0.168589
x0            0.076281
x1            0.093964
dtype: float64
```

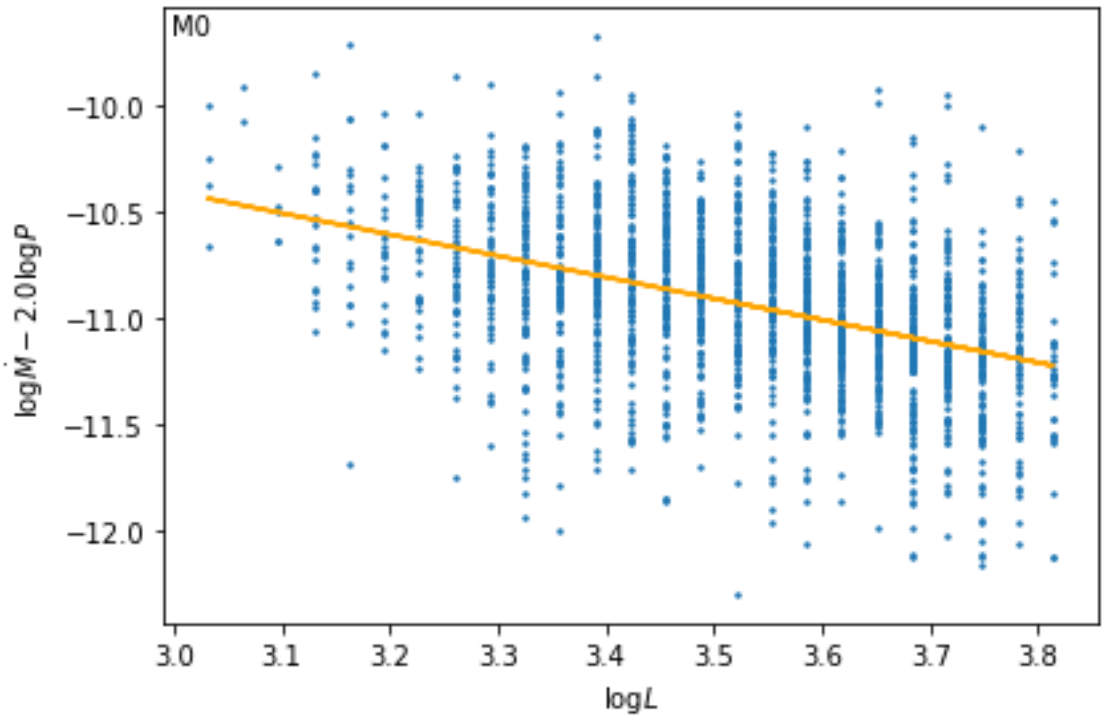


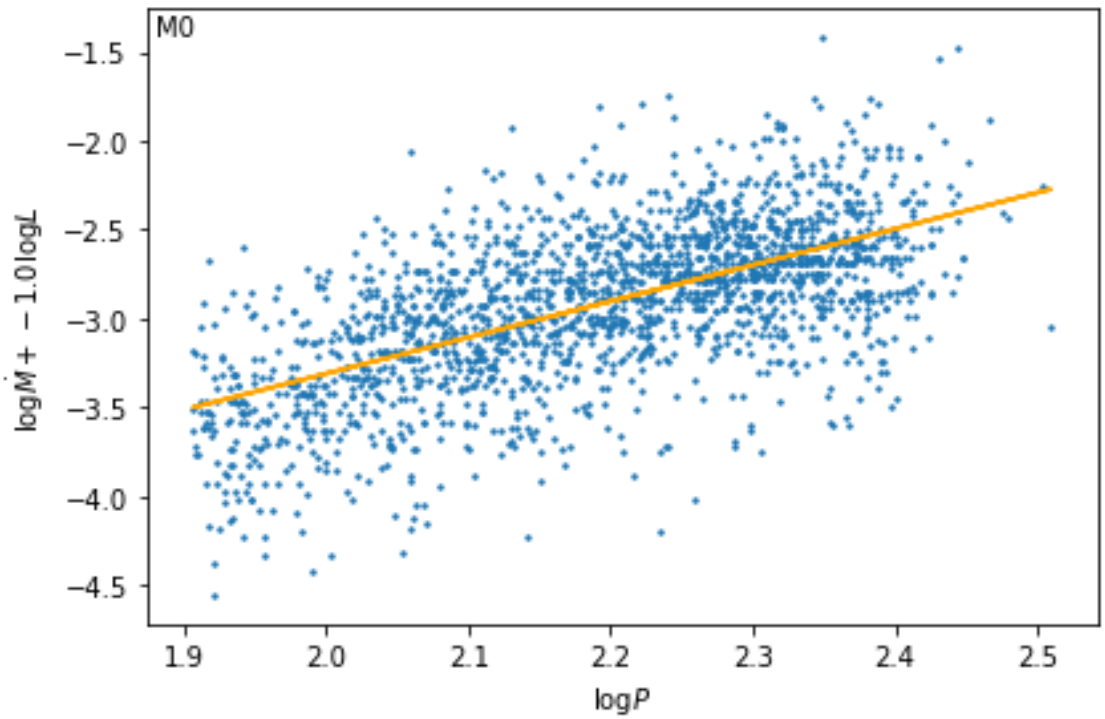


```
Fit 2: GRAMS Mdot vs Formula Mdot
form_vs_fed_Coeffs [5.23331378e-14 1.00000000e+00]
```

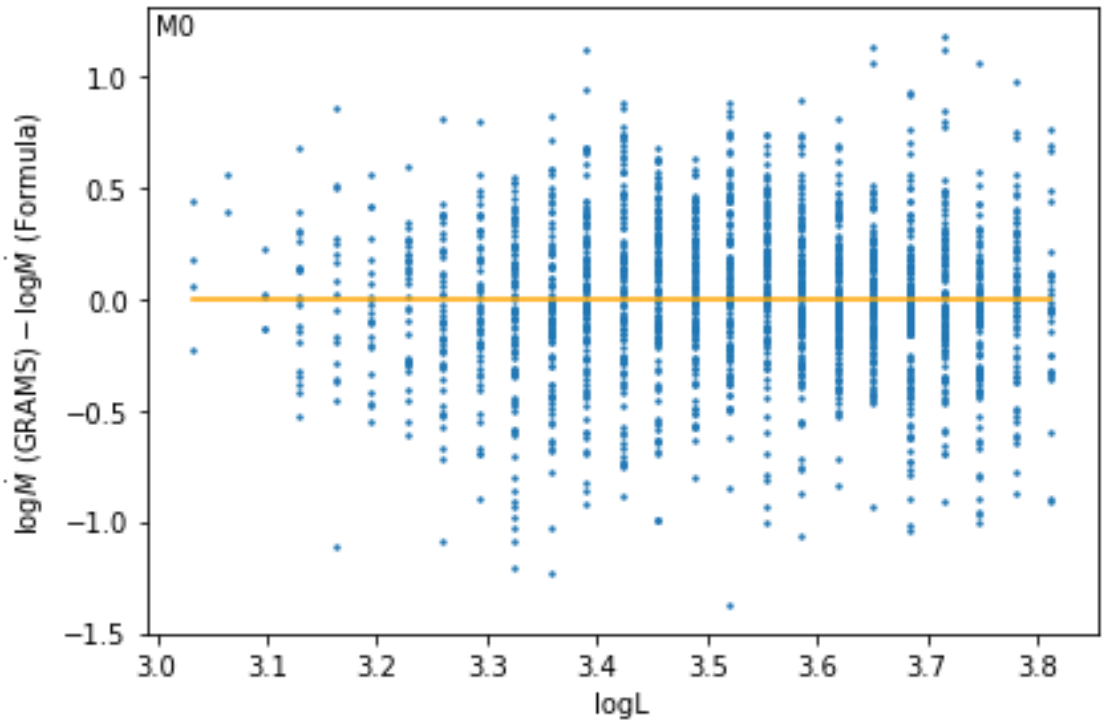



```
Quality of Fit Line:
Fit: logMdot_GRAMS=0.000 + 1.000 * logMdot_Formula
R^2: 0.206365452135026
```


M0
−10.0
−10.5
−11.0
−11.5
−12.0
3.0
3.1
3.2
3.3
3.4
3.5
3.6
3.7
3.8
logL
logṀ − 2.0 logP

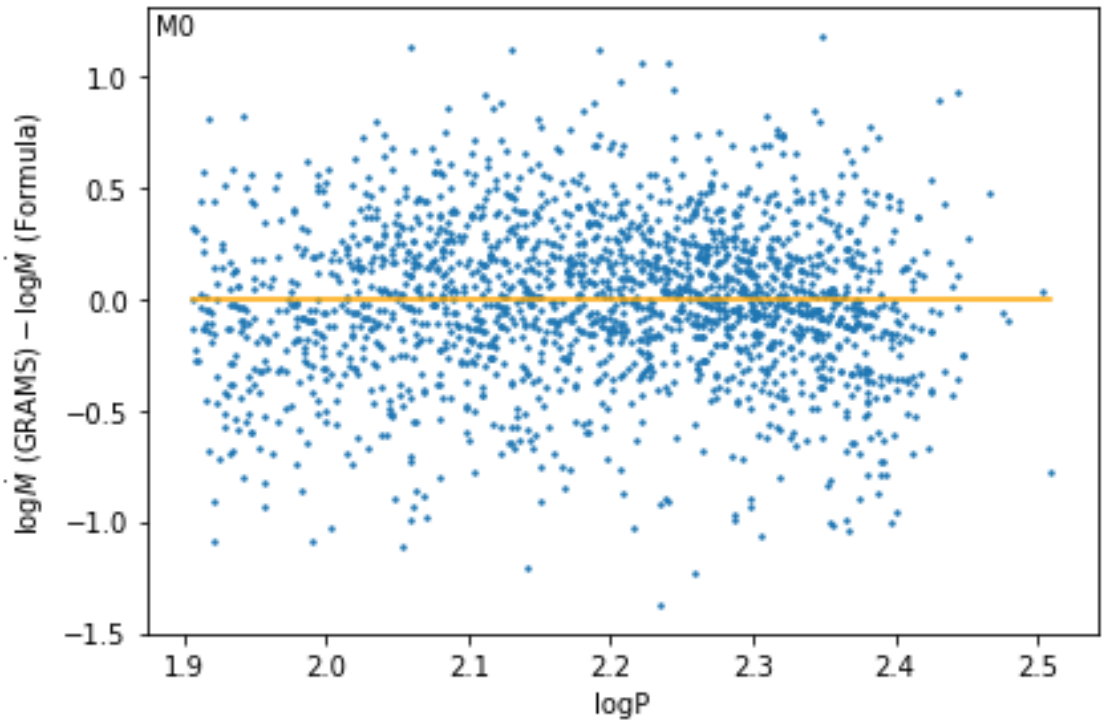

M0
−1.5
−2.0
−2.5
−3.0
−3.5
−4.0
−4.5
1.9
2.0
2.1
2.2
2.3
2.4
2.5
logP
logṀ + − 1.0 logL

Fit 3: Mdot Residual (GRAMS-Formula) vs logL



Quality of Residual Fit:
Fit: logMdot (GRAMS) - logMdot (Formula)=-0.000 + 0.000 * logL
R^2: 2.220446049250313e-16

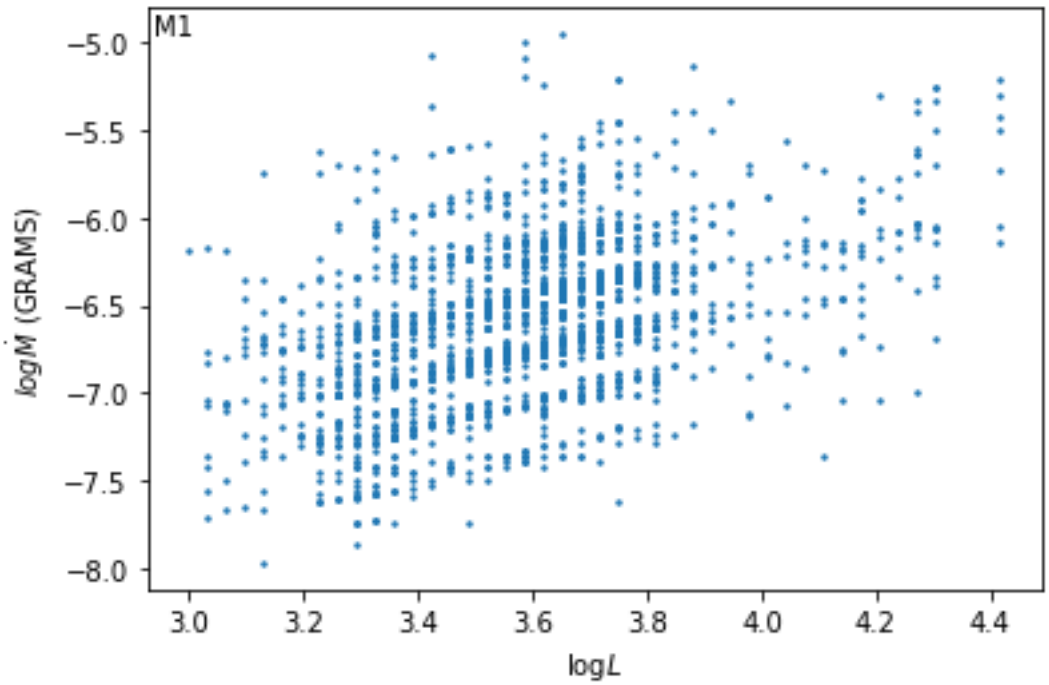
Fit 4: Mdot Residual (GRAMS-Formula) vs logP

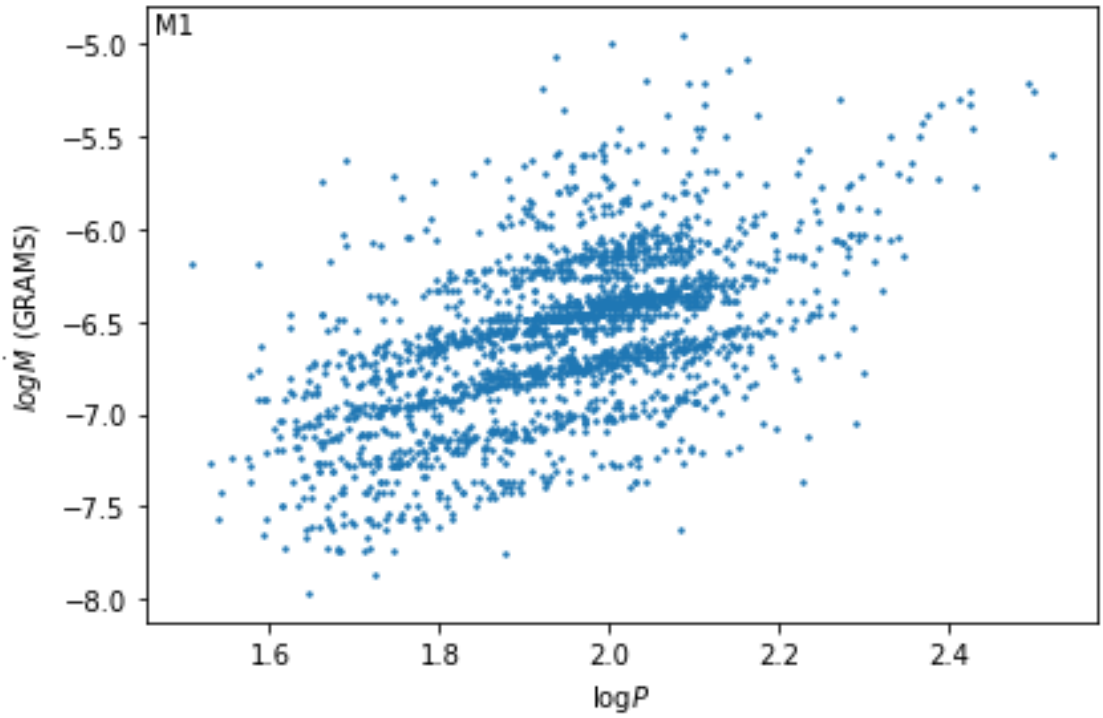



```
Quality of Residual Fit:
Fit: logMdot (GRAMS) - logMdot (Formula)=-0.000 + 0.000 * logP
R^2: 2.220446049250313e-16
```

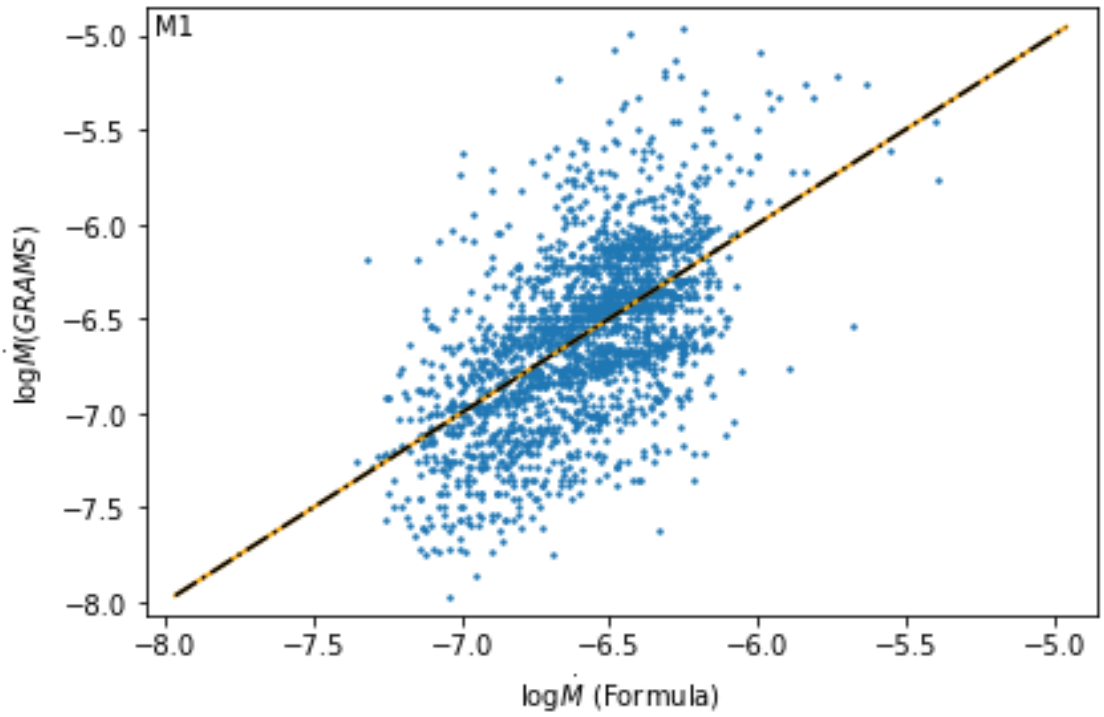
[50]:
```
LPCoeffsM1_GRAMS = do_final_fits(M1StarData,np.log10(M1StarData[MdotIndex]),␣
↪fed_label = 'GRAMS',star_type_label='M1')
```

```
Fit 1: Multilinear of L,P vs provided Mdot
Fit: -9.089358285438802 - 0.7730420667303209 logL + 2.7128930575022596 logP
R^2: 0.34292097808195965
Standard Errors (x0 = logL, x1 = logP):
 Intercept    0.129781
x0            0.091410
x1            0.135346
dtype: float64
```

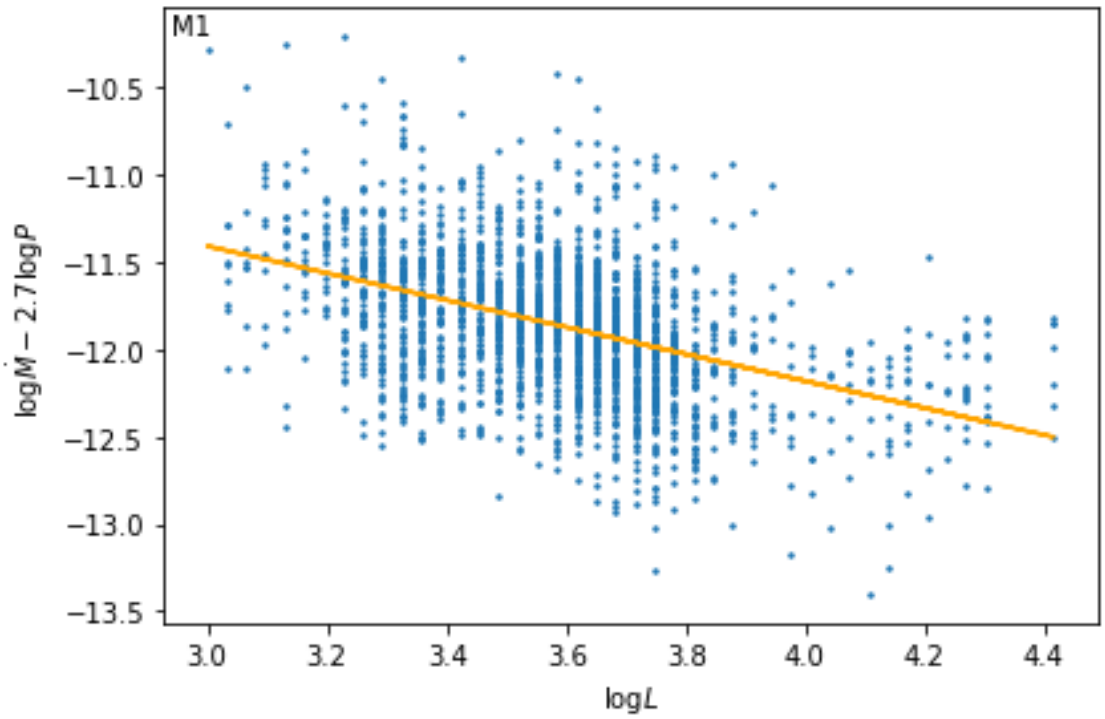

M1
−5.0
−5.5
−6.0
−6.5
−7.0
−7.5
−8.0
3.0
3.2
3.4
3.6
3.8
4.0
4.2
4.4
logL
logṀ (GRAMS)

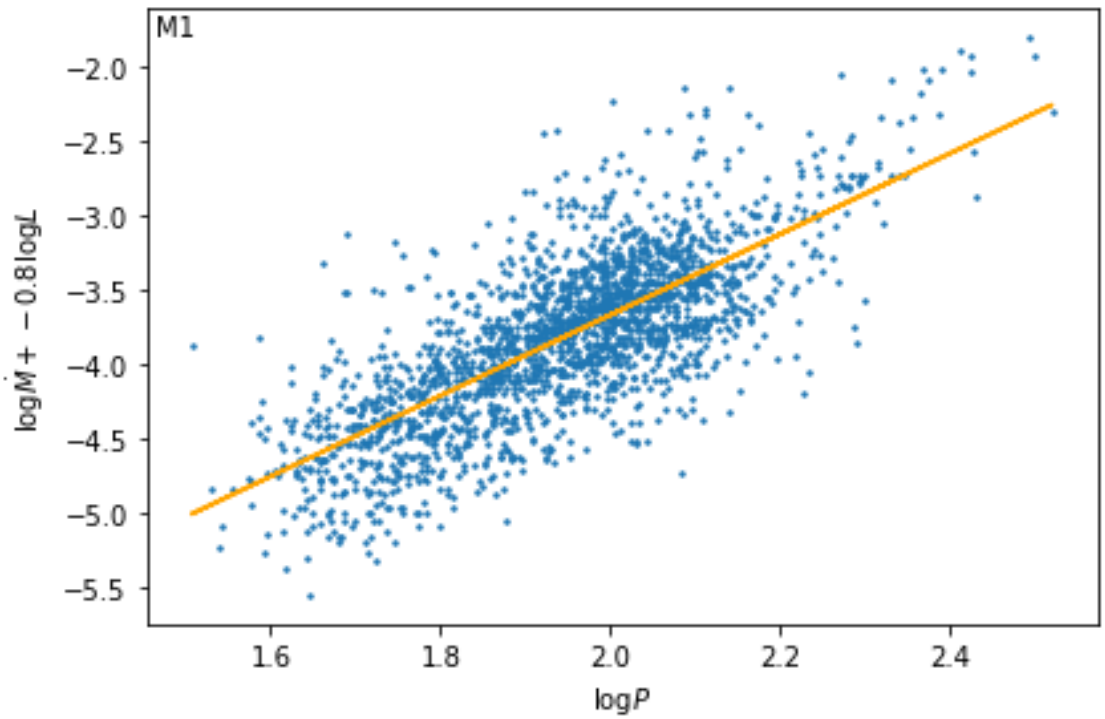

M1
−5.0
−5.5
−6.0
−6.5
−7.0
−7.5
−8.0
1.6
1.8
2.0
2.2
2.4
logP
logṀ (GRAMS)

Fit 2: GRAMS Mdot vs Formula Mdot
form_vs_fed_Coeffs [1.73194792e-14 1.00000000e+00]

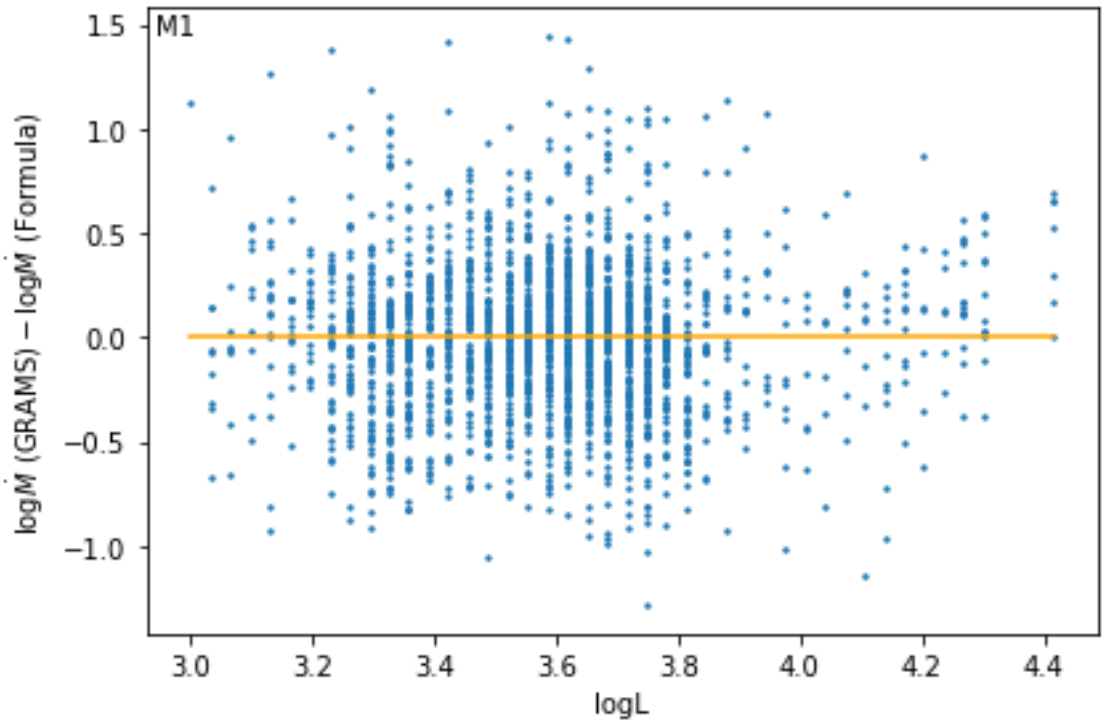


Quality of Fit Line:
Fit: logMdot_GRAMS=0.000 + 1.000 * logMdot_Formula
R^2: 0.34292097808195976


M1
−10.5
−11.0
−11.5
−12.0
−12.5
−13.0
−13.5
3.0
3.2
3.4
3.6
3.8
4.0
4.2
4.4
logL
logṀ − 2.7logP

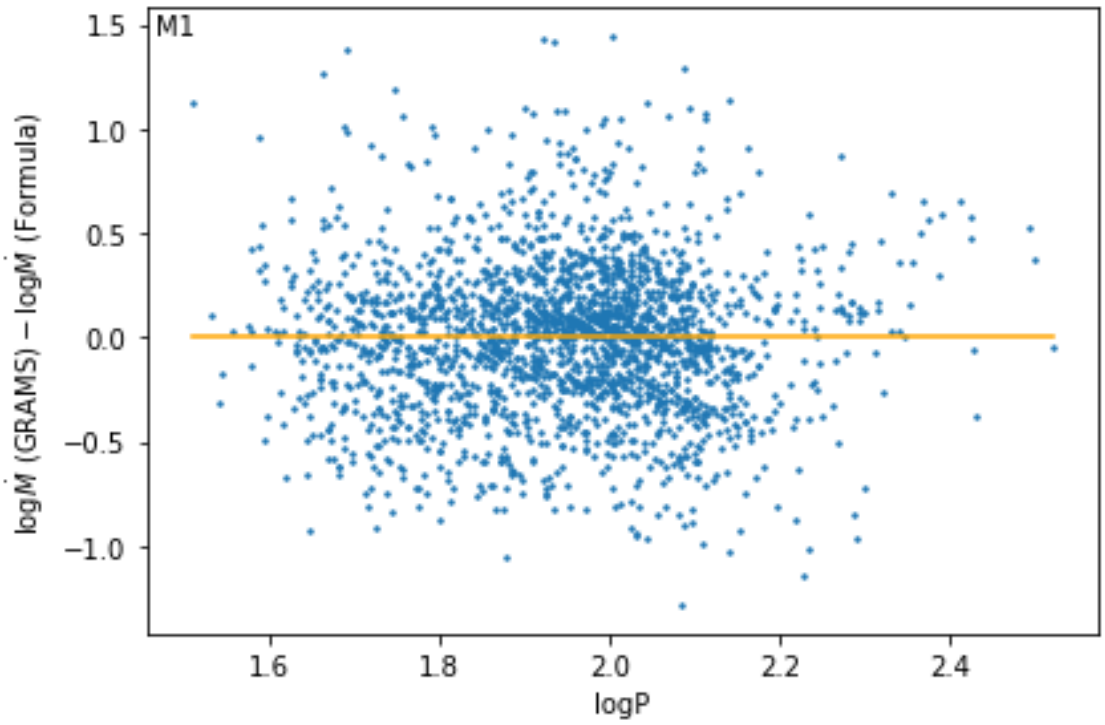

M1
−2.0
−2.5
−3.0
−3.5
−4.0
−4.5
−5.0
−5.5
1.6
1.8
2.0
2.2
2.4
logP
logṀ + −0.8logL

Fit 3: Mdot Residual (GRAMS-Formula) vs logL



Quality of Residual Fit:
Fit: logMdot (GRAMS) - logMdot (Formula)=-0.000 + 0.000 * logL
R^2: -2.220446049250313e-16

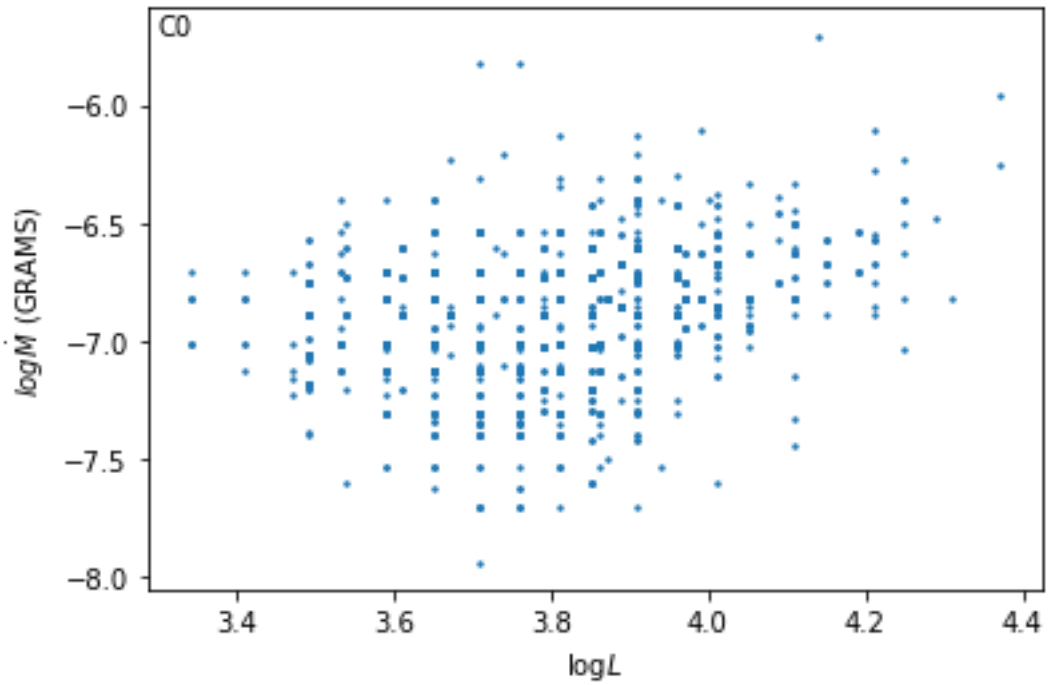
Fit 4: Mdot Residual (GRAMS-Formula) vs logP

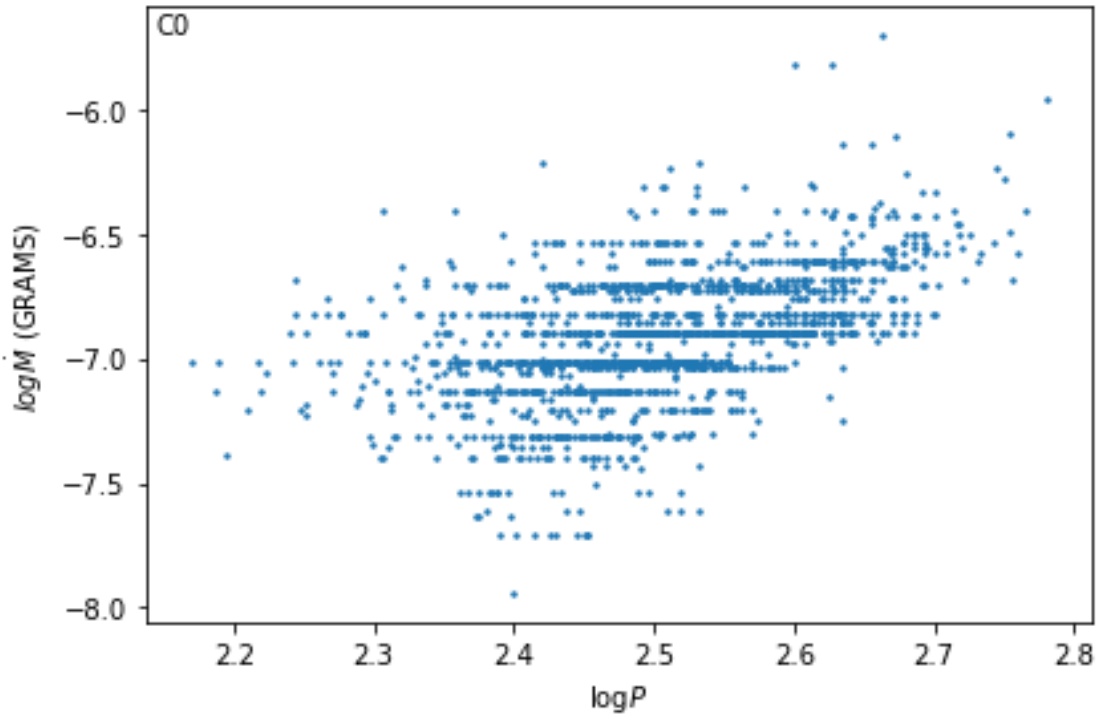



```
Quality of Residual Fit:
Fit: logMdot (GRAMS) - logMdot (Formula)=-0.000 + 0.000 * logP
R^2: 0.0
```

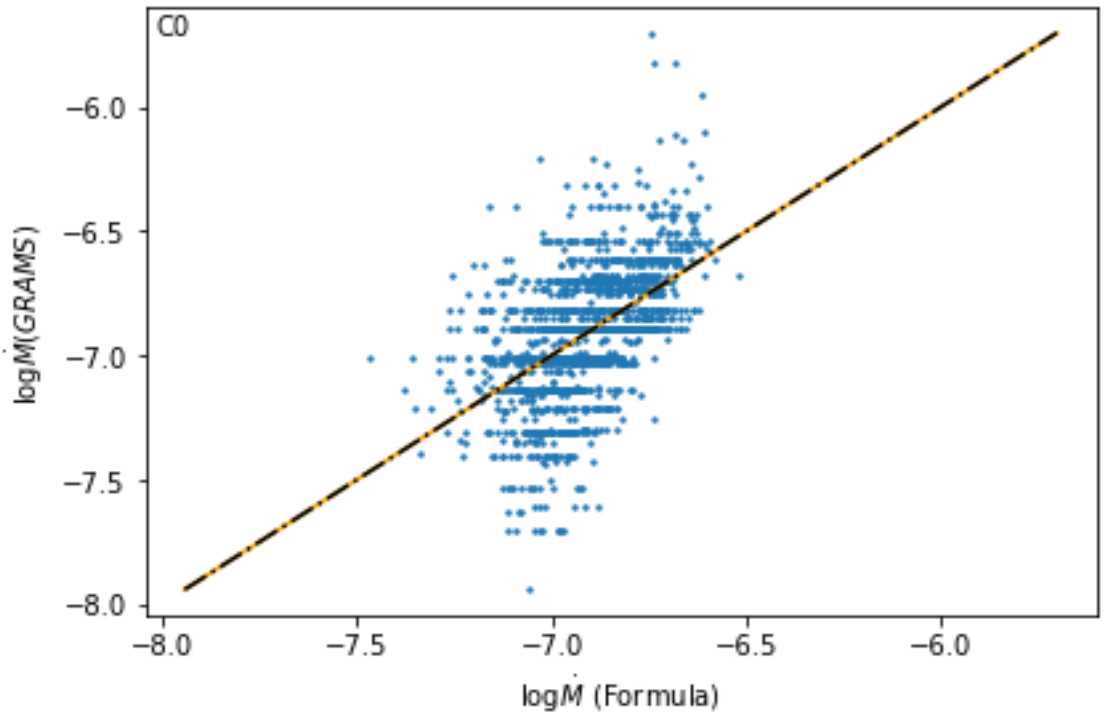
[51]:
```
LPCoeffsC0_GRAMS = do_final_fits(C0StarData_n,np.
↪log10(C0StarData_n[MdotIndex]), fed_label = 'GRAMS',star_type_label='C0')
```

```
Fit 1: Multilinear of L,P vs provided Mdot
Fit: -9.996397243650726 - 0.2789546923594095 logL + 1.6553157873965993 logP
R^2: 0.2707589145251824
Standard Errors (x0 = logL, x1 = logP):
 Intercept    0.146462
x0           0.049505
x1           0.079347
dtype: float64
```

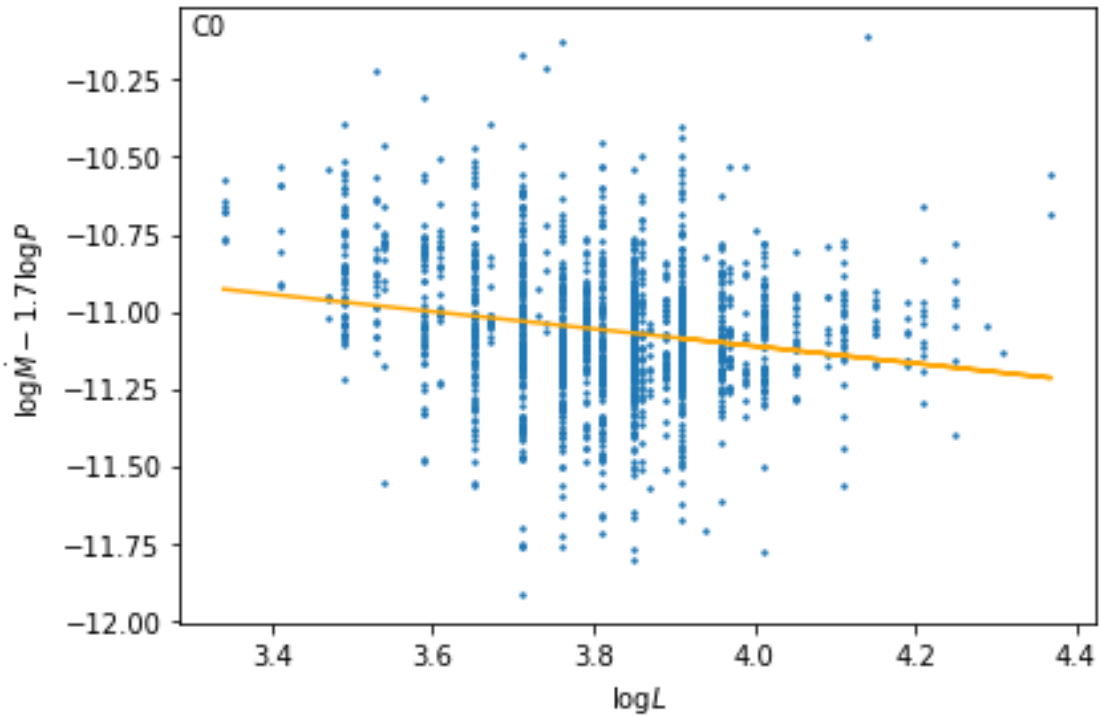

C0
−6.0
−6.5
−7.0
−7.5
−8.0
3.4
3.6
3.8
4.0
4.2
4.4
$\log\dot{M}$ (GRAMS)
$\log L$

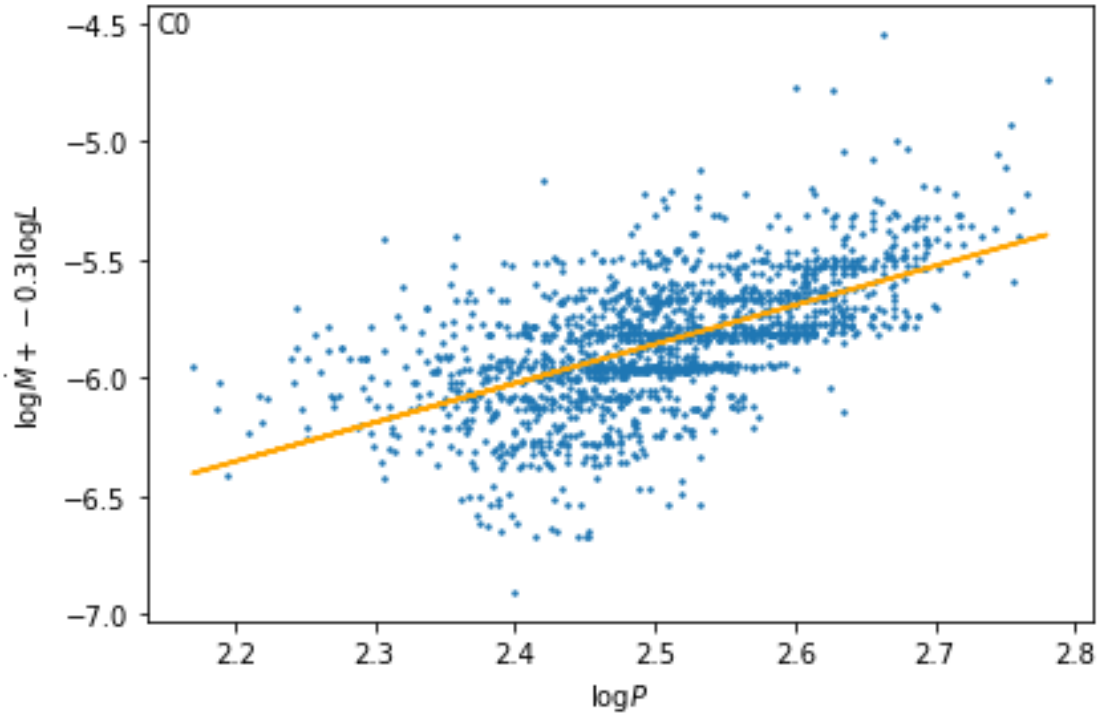

C0
−6.0
−6.5
−7.0
−7.5
−8.0
2.2
2.3
2.4
2.5
2.6
2.7
2.8
$\log\dot{M}$ (GRAMS)
$\log P$

Fit 2: GRAMS Mdot vs Formula Mdot
form_vs_fed_Coeffs [5.66213743e-15 1.00000000e+00]

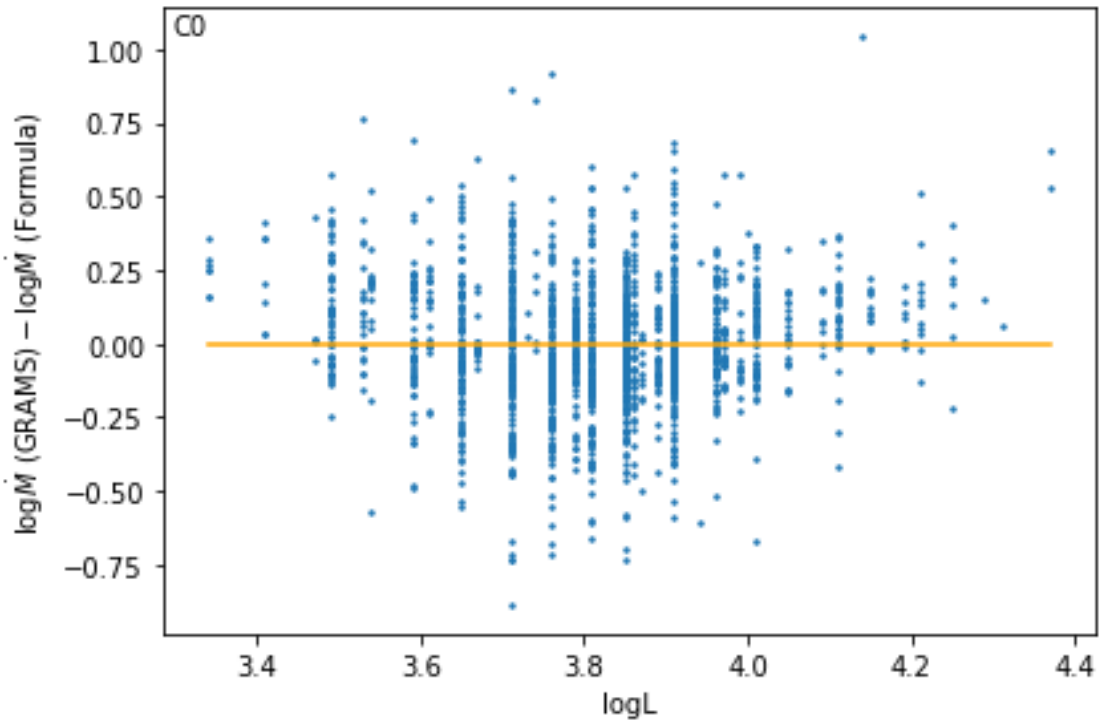


Quality of Fit Line:
Fit: logMdot_GRAMS=0.000 + 1.000 * logMdot_Formula
R^2: 0.2707589145251822


C0
−10.25
−10.50
−10.75
−11.00
−11.25
−11.50
−11.75
−12.00
3.4
3.6
3.8
4.0
4.2
4.4
$\log\dot{M} - 1.7\log P$
$\log L$

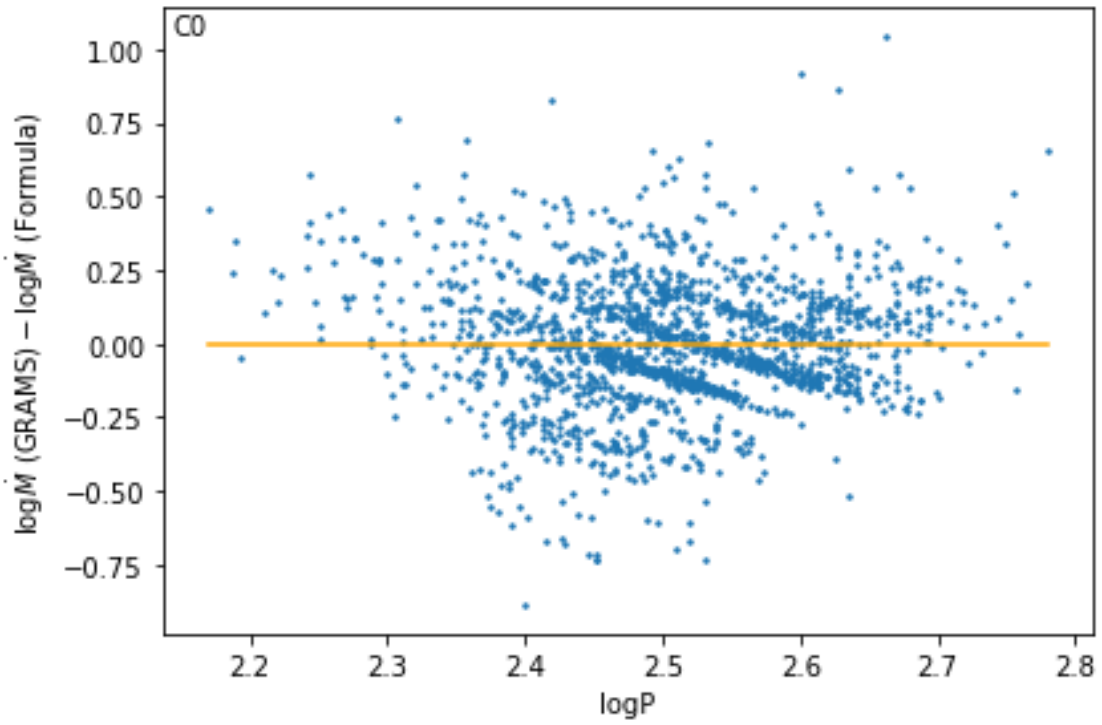

C0
−4.5
−5.0
−5.5
−6.0
−6.5
−7.0
2.2
2.3
2.4
2.5
2.6
2.7
2.8
$\log\dot{M} + -0.3\log L$
$\log P$

Fit 3: Mdot Residual (GRAMS-Formula) vs logL



Quality of Residual Fit:
Fit: logMdot (GRAMS) - logMdot (Formula)=-0.000 + 0.000 * logL
R^2: -2.220446049250313e-16

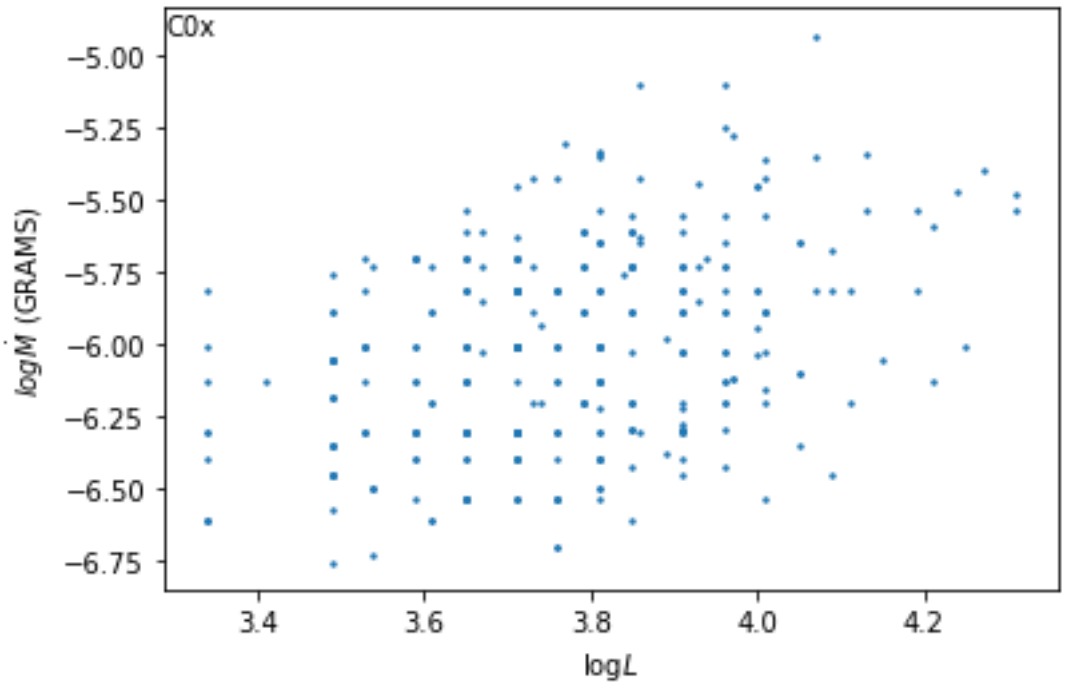
Fit 4: Mdot Residual (GRAMS-Formula) vs logP

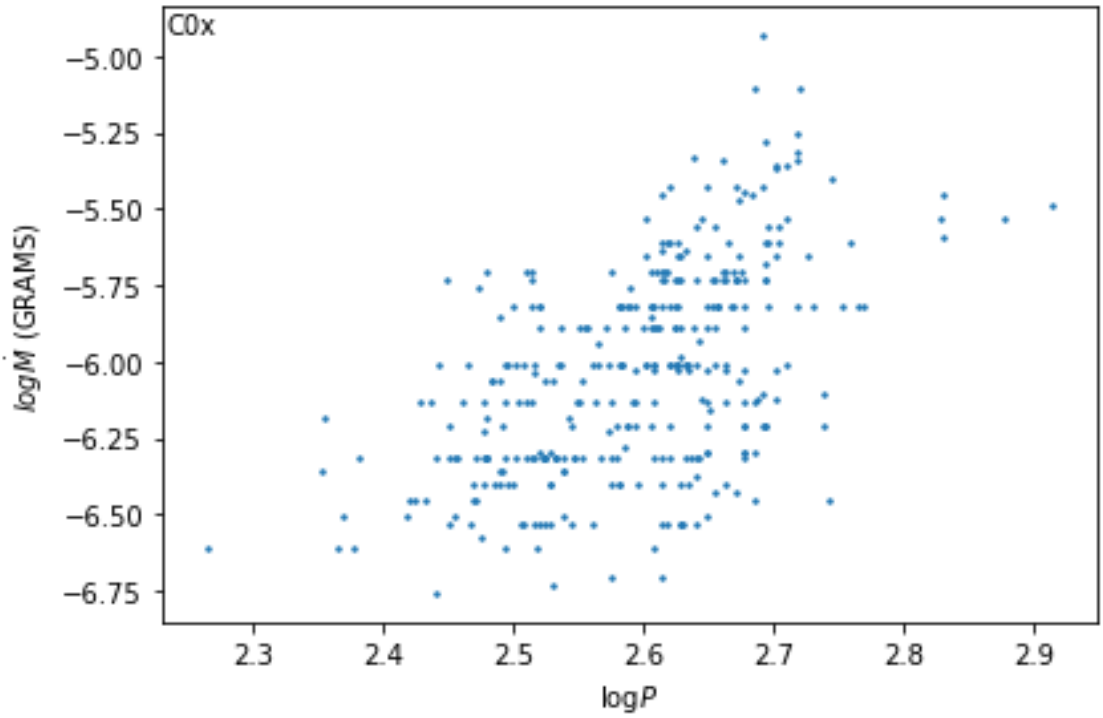



```
Quality of Residual Fit:
Fit: logMdot (GRAMS) - logMdot (Formula)=-0.000 + 0.000 * logP
R^2: -2.220446049250313e-16
```

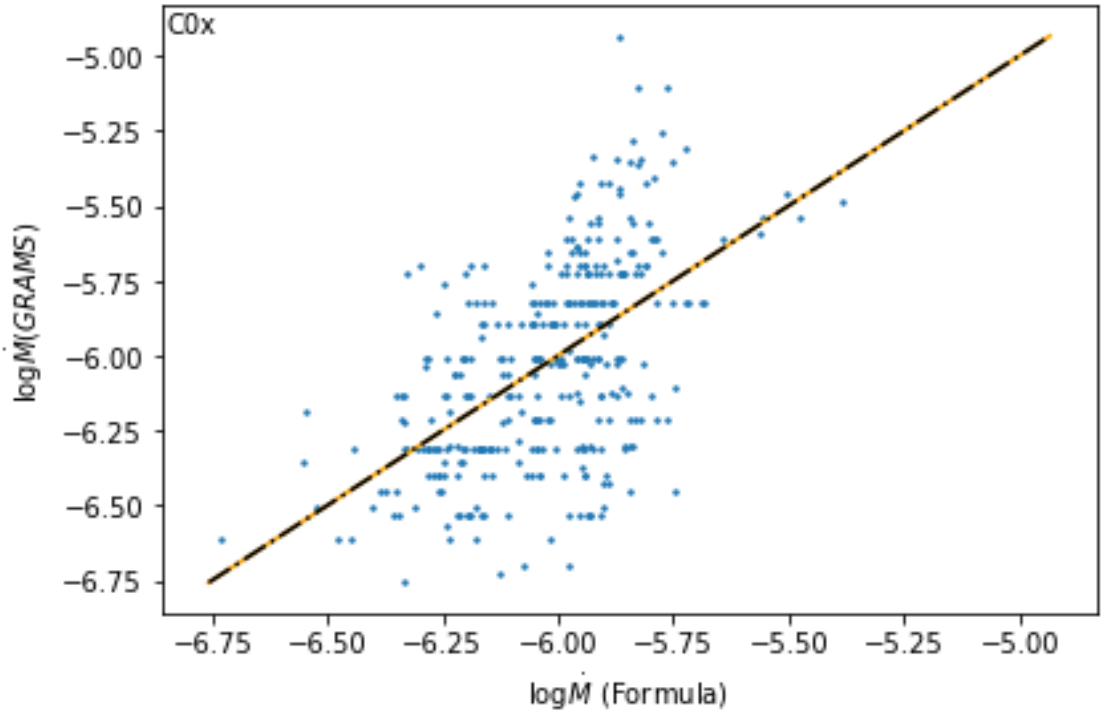
[52]:
```
LPCoeffsC0x_GRAMS = do_final_fits(C0StarData_x,np.
↪log10(C0StarData_x[MdotIndex]), fed_label = 'GRAMS',star_type_label='C0x')
```

```
Fit 1: Multilinear of L,P vs provided Mdot
Fit: -11.462209794685188 - 0.2817162588472474 logL + 2.5037891787567554 logP
R^2: 0.30157616903474194
Standard Errors (x0 = logL, x1 = logP):
 Intercept    0.450097
x0           0.141329
x1           0.290497
dtype: float64
```

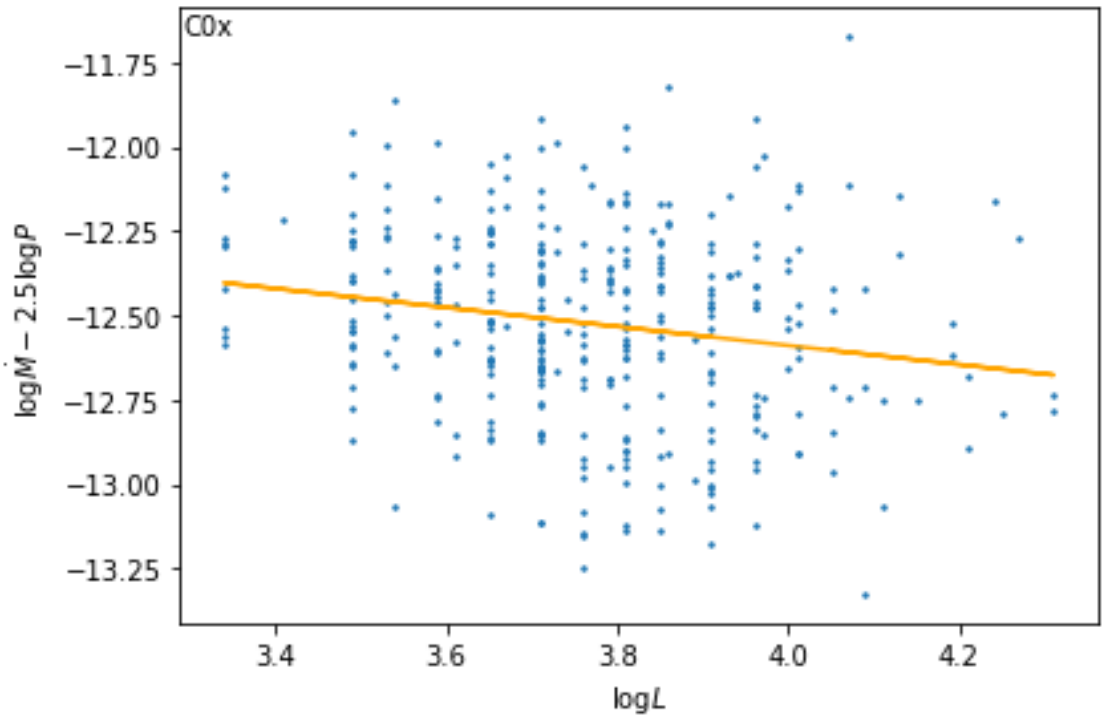

C0x
−5.00
−5.25
−5.50
−5.75
−6.00
−6.25
−6.50
−6.75
3.4
3.6
3.8
4.0
4.2
logL
logṀ (GRAMS)

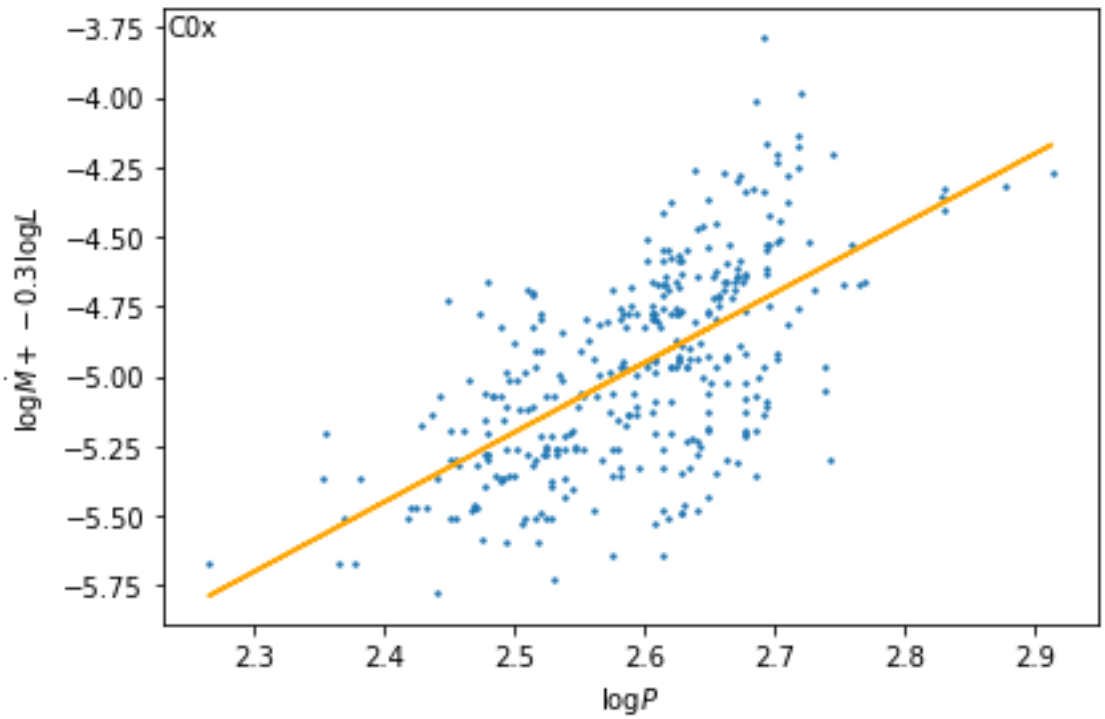

C0x
−5.00
−5.25
−5.50
−5.75
−6.00
−6.25
−6.50
−6.75
2.3
2.4
2.5
2.6
2.7
2.8
2.9
logP
logṀ (GRAMS)

Fit 2: GRAMS Mdot vs Formula Mdot
form_vs_fed_Coeffs [-1.98729921e-14 1.00000000e+00]

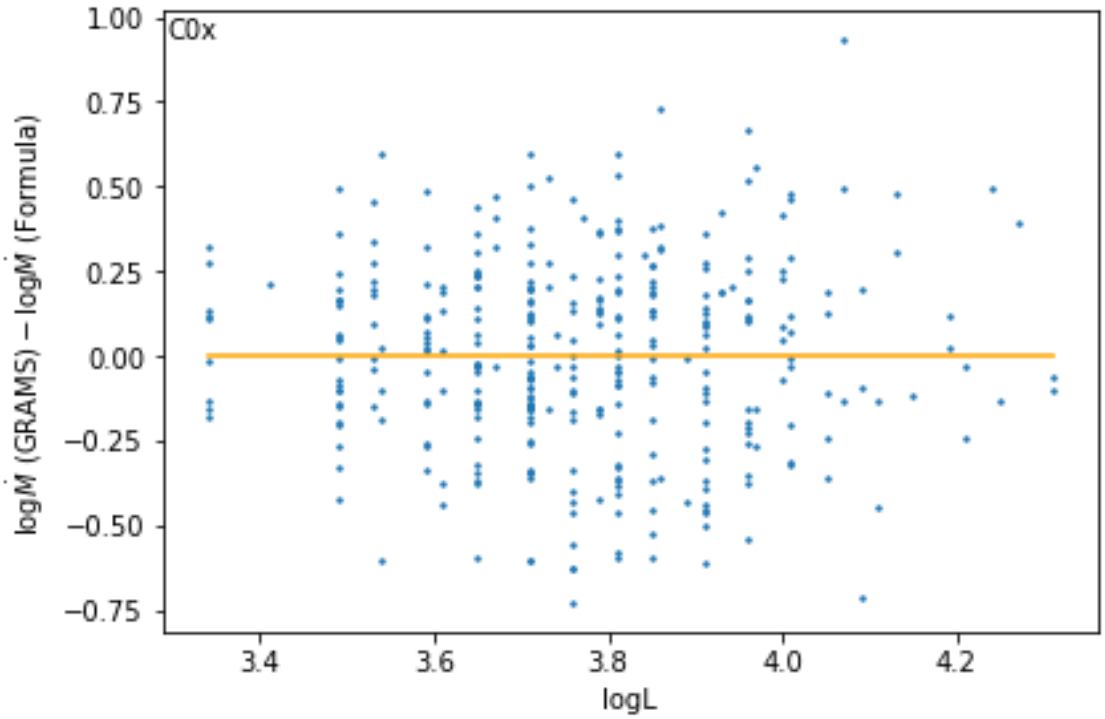


Quality of Fit Line:
Fit: logMdot_GRAMS=-0.000 + 1.000 * logMdot_Formula
R^2: 0.3015761690347418


C0x
−11.75
−12.00
−12.25
−12.50
−12.75
−13.00
−13.25
3.4
3.6
3.8
4.0
4.2
logL
logṀ − 2.5logP

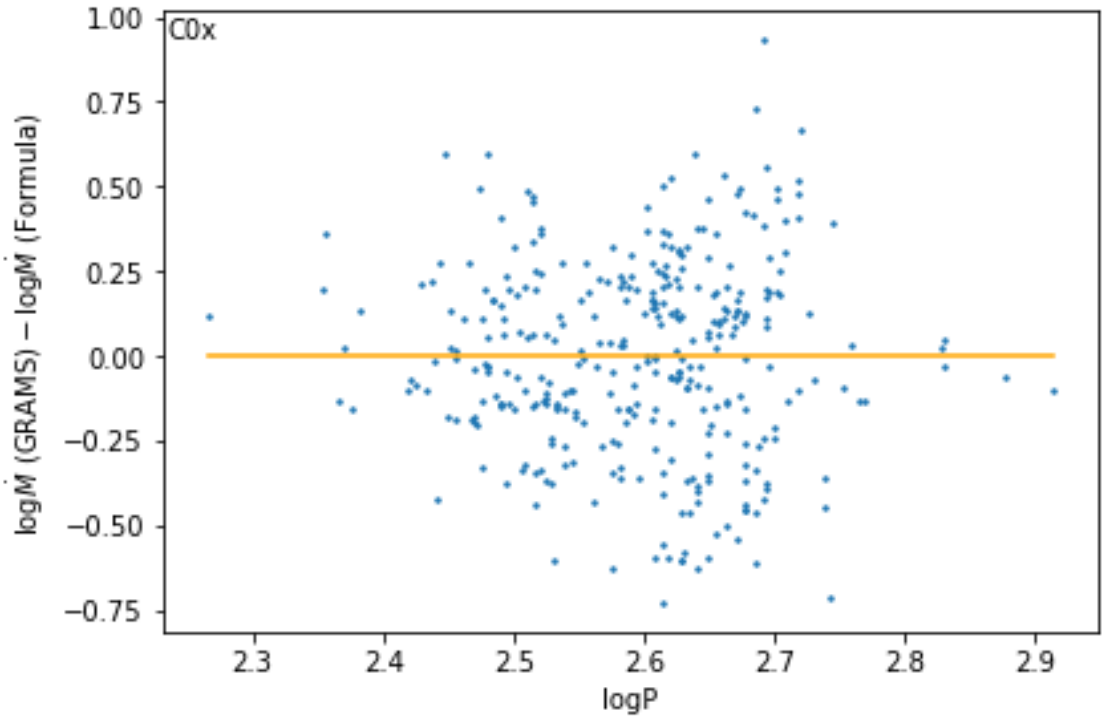

C0x
−3.75
−4.00
−4.25
−4.50
−4.75
−5.00
−5.25
−5.50
−5.75
2.3
2.4
2.5
2.6
2.7
2.8
2.9
logP
logṀ + −0.3logL

Fit 3: Mdot Residual (GRAMS-Formula) vs logL



Quality of Residual Fit:
Fit: logMdot (GRAMS) - logMdot (Formula)=-0.000 + -0.000 * logL
R^2: 1.1102230246251565e-16

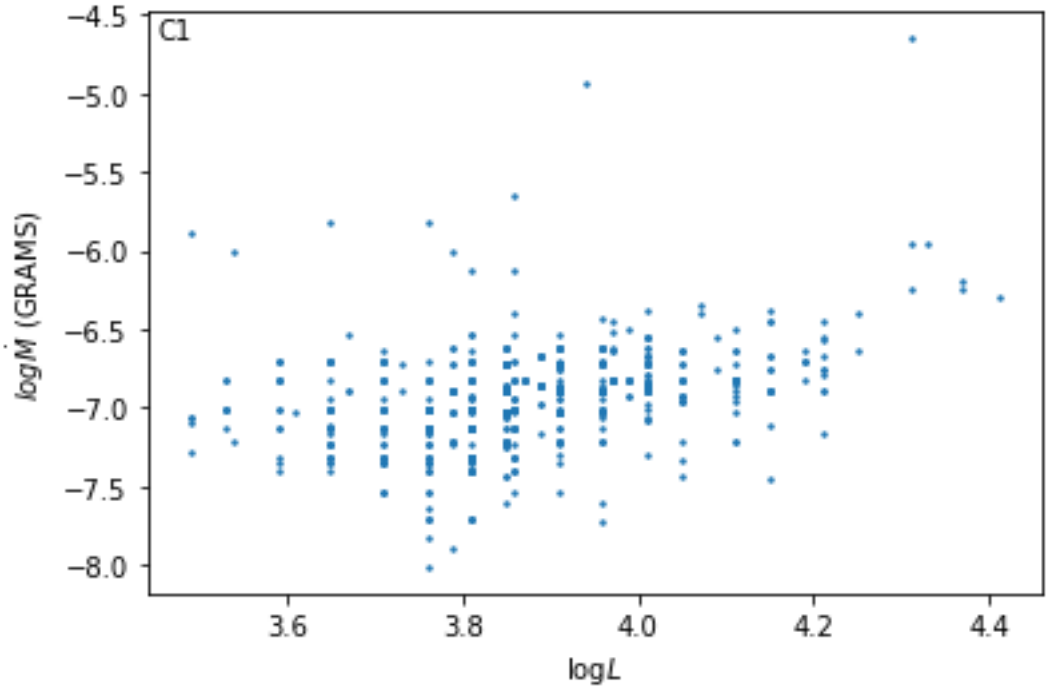
Fit 4: Mdot Residual (GRAMS-Formula) vs logP

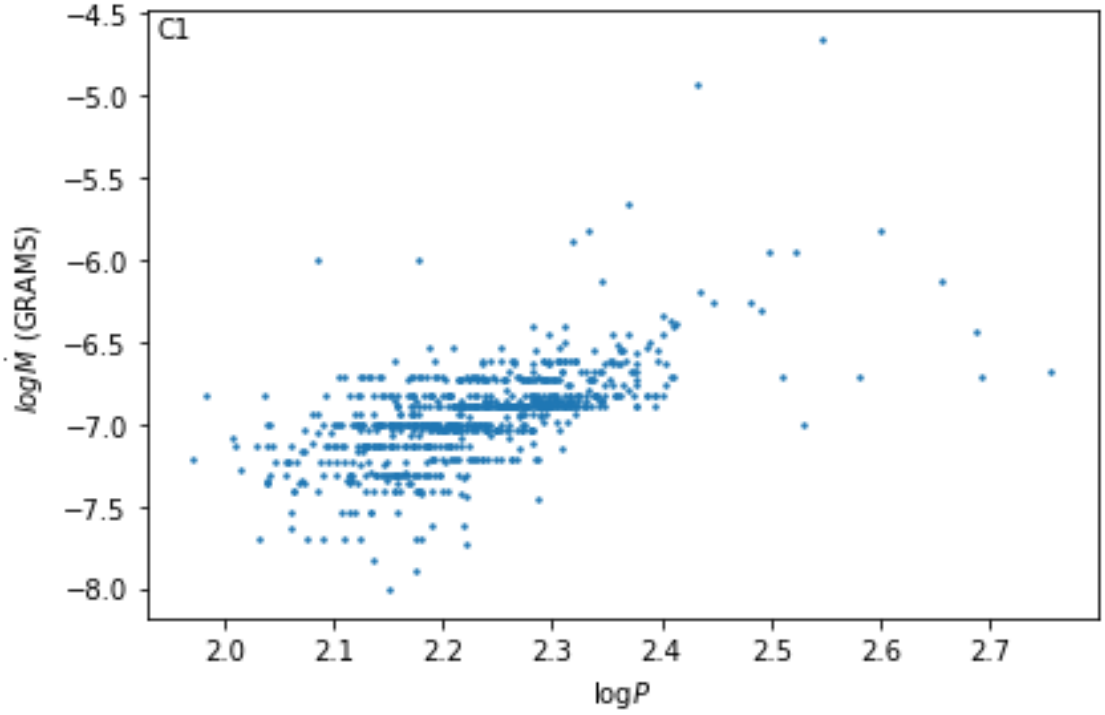



```
Quality of Residual Fit:
Fit: logMdot (GRAMS) - logMdot (Formula)=-0.000 + -0.000 * logP
R^2: 1.1102230246251565e-16
```

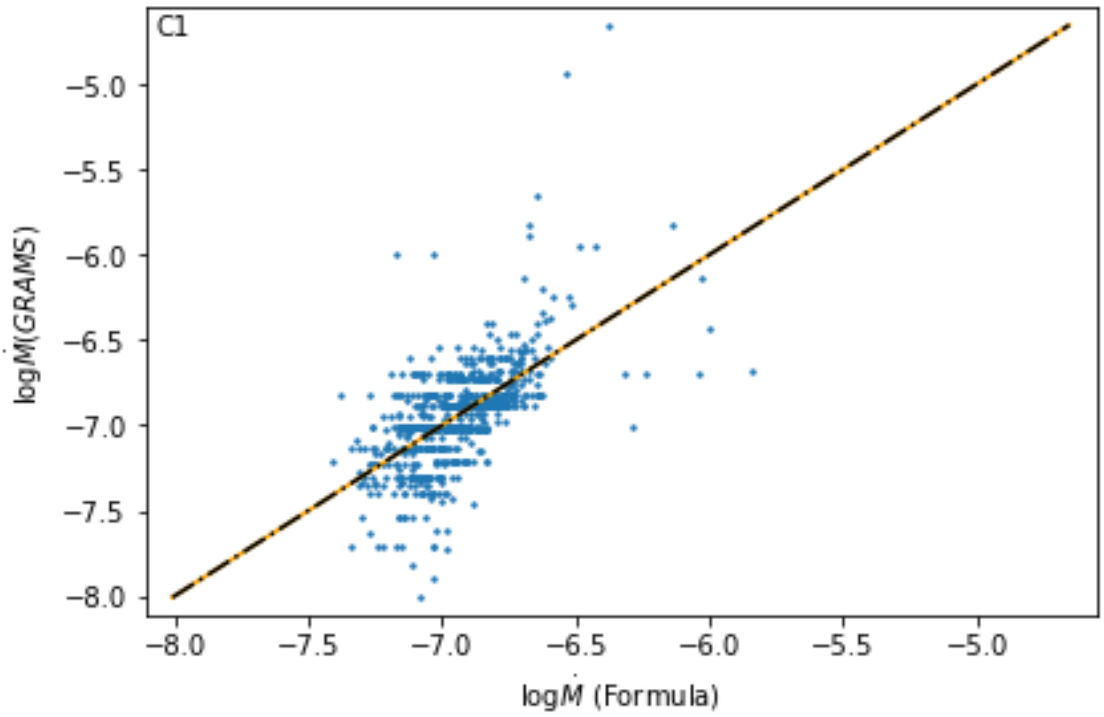
[53]:
```
LPCoeffsC1_GRAMS = do_final_fits(C1StarData,np.log10(C1StarData[MdotIndex]),␣
↪fed_label = 'GRAMS',star_type_label='C1')
```

```
Fit 1: Multilinear of L,P vs provided Mdot
Fit: -10.766834965320808 - 0.2240625182840398 logL + 2.103540946168206 logP
R^2: 0.38212210637017974
Standard Errors (x0 = logL, x1 = logP):
 Intercept    0.229507
x0            0.079929
x1            0.119122
dtype: float64
```

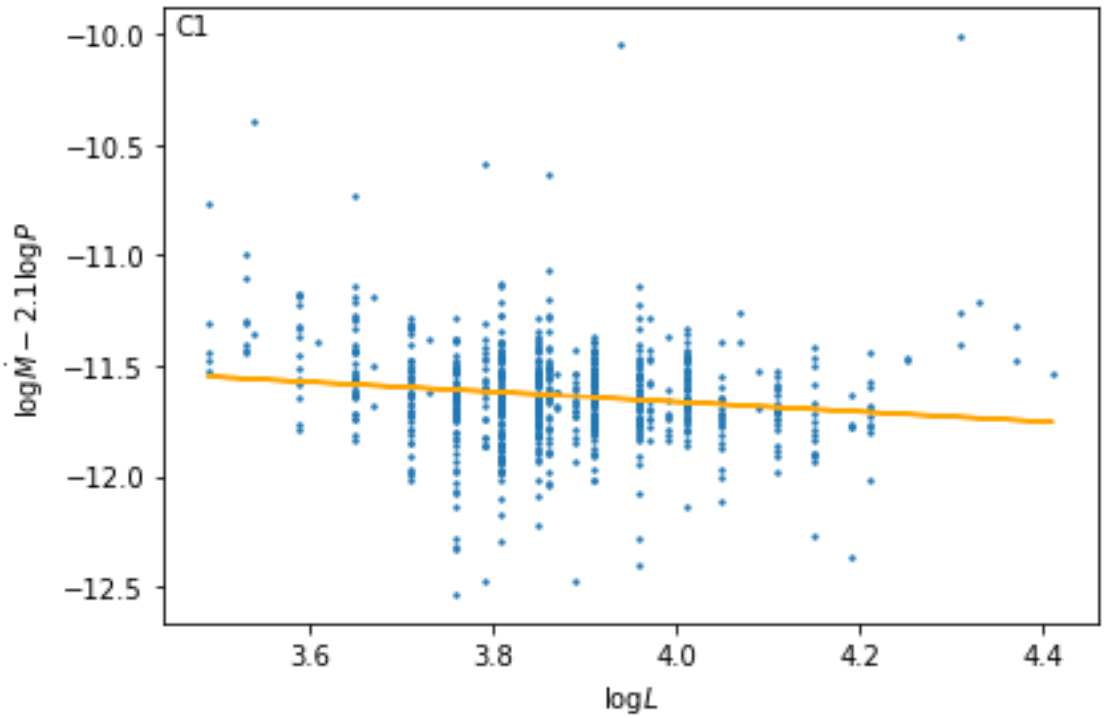

C1
−4.5
−5.0
−5.5
−6.0
−6.5
−7.0
−7.5
−8.0
3.6
3.8
4.0
4.2
4.4
log$\dot{M}$ (GRAMS)
log$L$

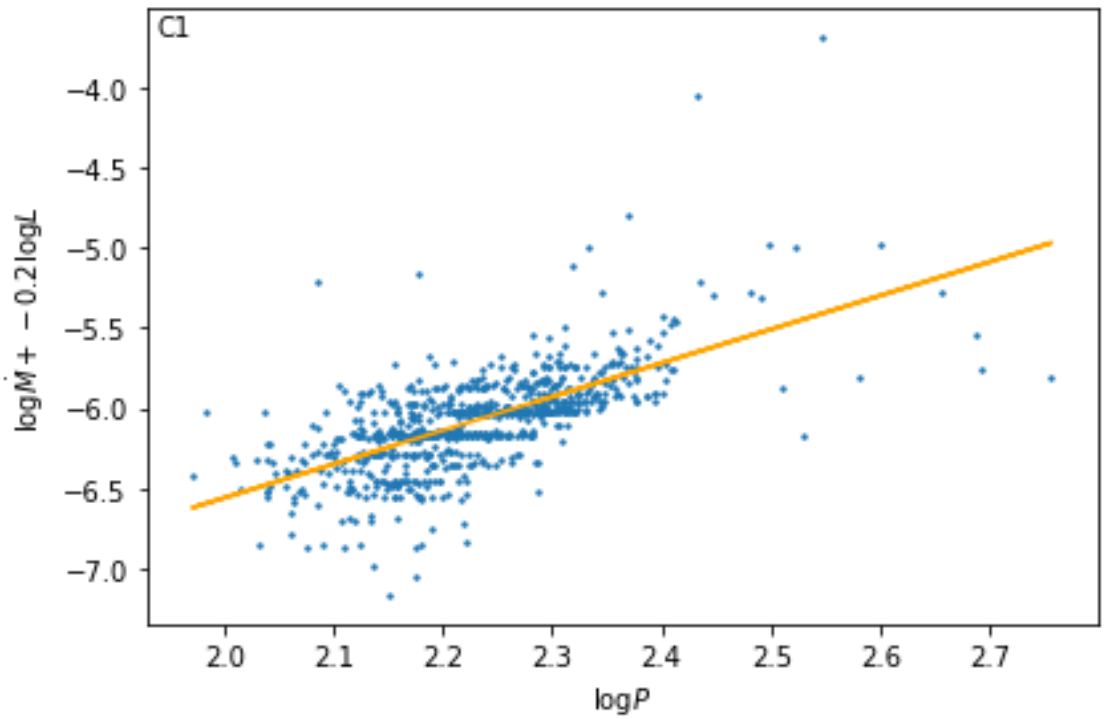

C1
−4.5
−5.0
−5.5
−6.0
−6.5
−7.0
−7.5
−8.0
2.0
2.1
2.2
2.3
2.4
2.5
2.6
2.7
log$\dot{M}$ (GRAMS)
log$P$

Fit 2: GRAMS Mdot vs Formula Mdot
form_vs_fed_Coeffs [-3.88578059e-16 1.00000000e+00]

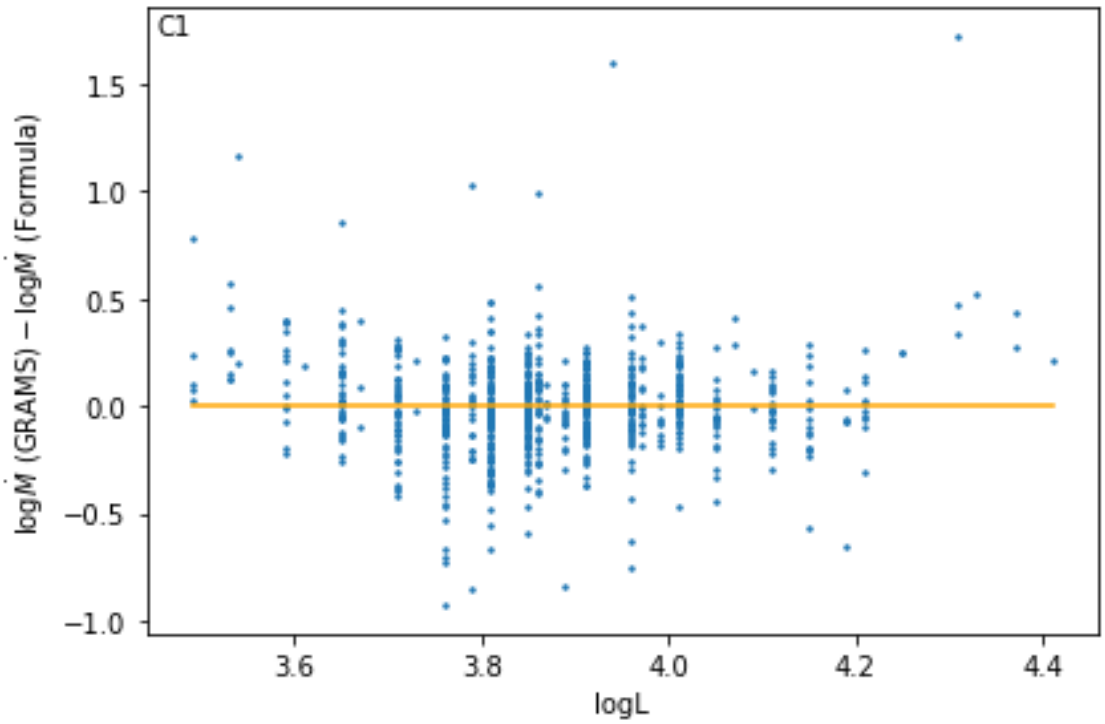


Quality of Fit Line:
Fit: logMdot_GRAMS=-0.000 + 1.000 * logMdot_Formula
R^2: 0.38212210637017974


C1
−10.0
−10.5
−11.0
−11.5
−12.0
−12.5
3.6
3.8
4.0
4.2
4.4
$\log\dot{M} - 2.1\log P$
$\log L$

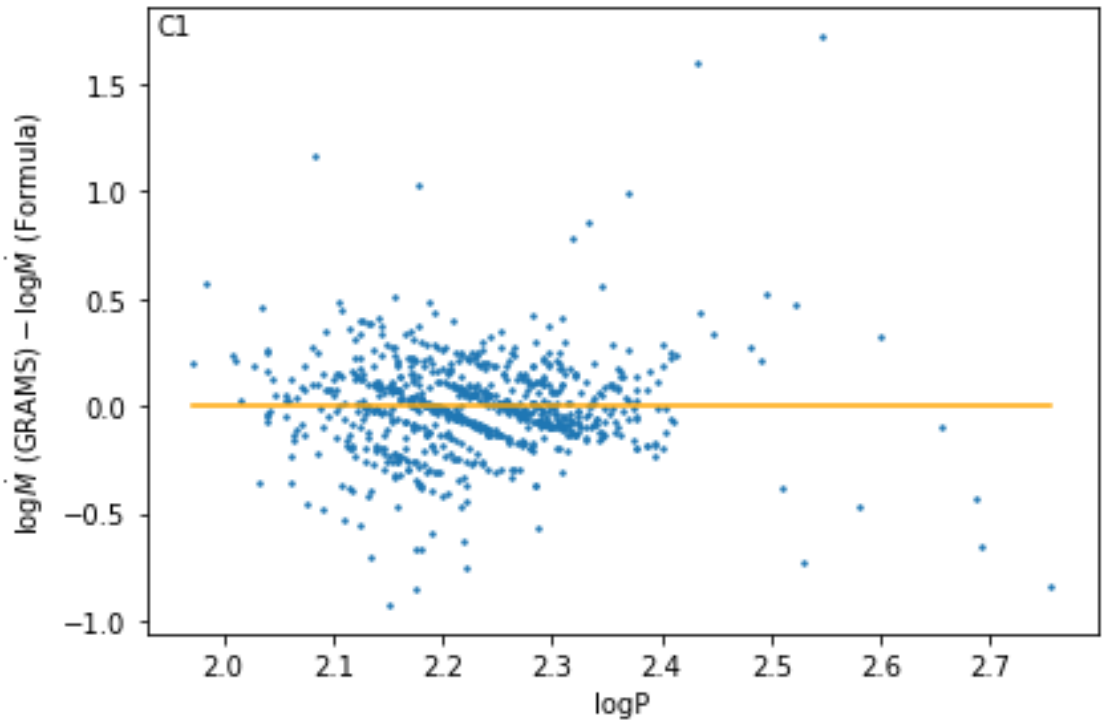

C1
−4.0
−4.5
−5.0
−5.5
−6.0
−6.5
−7.0
2.0
2.1
2.2
2.3
2.4
2.5
2.6
2.7
$\log\dot{M} + -0.2\log L$
$\log P$

Fit 3: Mdot Residual (GRAMS-Formula) vs logL



Quality of Residual Fit:
Fit: logMdot (GRAMS) - logMdot (Formula)=-0.000 + 0.000 * logL
R^2: -2.220446049250313e-16

Fit 4: Mdot Residual (GRAMS-Formula) vs logP




```
Quality of Residual Fit:
Fit: logMdot (GRAMS) - logMdot (Formula)=-0.000 + -0.000 * logP
R^2: -2.220446049250313e-16
```

[54]:
```
def find_formula_differentials (coeffs_1, coeffs_2,␣
↪data_set,plot_title,debug=False):
    # Using two sets of coefficients with some matching property (mode or␣
↪stellar composition), calculate the difference between
    # the predictions for the correct formula (using coeffs_1) and incorrect␣
↪formula (coeffs_2). Then, plot this data.
    if debug == True:
        print('coeffs_1:', coeffs_1)
        print('coeffs_2:', coeffs_2)
        print('logL Range:',np.nanmax(np.log10(data_set[LIndex]))-np.nanmin(np.
↪log10(data_set[LIndex])))
        print('coeffs_1[1] * logL Range:',coeffs_1[1]*np.nanmax(np.
↪log10(data_set[LIndex]))-np.nanmin(np.log10(data_set[LIndex])))
        print('coeffs_2[1] * logL Range:',coeffs_2[1]*np.nanmax(np.
↪log10(data_set[LIndex]))-np.nanmin(np.log10(data_set[LIndex])))
        print('logP Range:',np.nanmax(np.log10(data_set[PIndex]))-np.nanmin(np.
↪log10(data_set[PIndex])))
        print('coeffs_1[2] * logP Range:',coeffs_1[2]*np.nanmax(np.
↪log10(data_set[PIndex]))-np.nanmin(np.log10(data_set[PIndex])))
```

```
        print('coeffs_2[2] * logP Range:',coeffs_2[2]*np.nanmax(np.
↪log10(data_set[PIndex]))-np.nanmin(np.log10(data_set[PIndex])))
    MdotPredict_1 = coeffs_1[0] + coeffs_1[1] * np.log10(data_set[LIndex]) +␣
↪coeffs_1[2] * np.log10(data_set[PIndex])
    MdotPredict_2 = coeffs_2[0] + coeffs_2[1] * np.log10(data_set[LIndex]) +␣
↪coeffs_2[2] * np.log10(data_set[PIndex])
    if debug == True:
        print ('MdotPredict_1:', MdotPredict_1)
        print ('MdotPredict_2:', MdotPredict_2)
    differential = np.log10(-(10**MdotPredict_1 - 10**MdotPredict_2))
    print(differential)
    limits = np.asarray([np.nanmin([np.nanmin(MdotPredict_1),np.
↪nanmin(MdotPredict_2)]), np.nanmax([np.nanmax(MdotPredict_1),np.
↪nanmax(MdotPredict_2)])])

    fig, ax = plt.subplots()
    plot.scatter(MdotPredict_1, differential,s = 2)
    #plot.
↪plot([limits[0],limits[1]],[limits[0],limits[1]],color='black',linestyle='-.
↪')
    #plot.xlim(limits[0]-0.1,limits[1]+0.1)
    #plot.ylim(limits[0]-0.1,limits[1]+0.1)
    plot.xlabel('logMdotPredict_1')
    plot.ylabel('np.log10(-(10**MdotPredict_1 - 10**MdotPredict_2))')
    plot.title(plot_title)
    plot.show()
    plot.close()

    return differential
```

[55]:
```
def calc_vw_mdot (log_period, log_mass):
    # log Mdot = -11.4 + 0.0125(P-100(M-2.5))
    # http://cdsads.u-strasbg.fr/pdf/1993ApJ...413..641V
    logMdot_array = []
    for i in range (0, len(log_period)):
        mass = 10**(log_mass[i])
        period = 10**(log_period[i])
        if mass > 2.5:
            # eq. 5 in above reference
            logMdot = -11.4 + 0.0125*(period - 100*(mass-2.5))
        else:
            # Eq. 2 in above reference
            logMdot = -11.4 + 0.0123*(period)
        logMdot_array.append(logMdot)
    return np.asarray(logMdot_array)

def calc_de_beck_mdot (log_period):
```

```
    # log Mdot = -7.37 + 3.42*10^-3 * P (P<850)
    #          = -4.46                  (P>850)
    logMdot_array = []
    for i in range (0, len(log_period)):
        period = 10**(log_period[i])
        if period < 850:
            logMdot = -7.37 + 3.42*10**(-3)*period
        else:
            logMdot = -4.46
        logMdot_array.append(logMdot)
    return np.asarray(logMdot_array)
```

[56]:
```
# PML coefficients can be constructed from RLM and PMR relations
# logR = AR + BRlogL + CRlogM
# logP = AP + BPlogM + CPlogR
# => logP = AP + BPlogM + CP(AR + BRlogL + CRlogM) = (AP + CP*AR) +␣
 ↪(BP+CP*CR)logM + (CP*BR)logL = A + BlogM + C logL
RLM_M = [-0.241, 0.690, -0.251]
RLM_C = [-0.323, 0.737, -0.371]
PMR_M0 = [-2.234, -0.7802, 2.027]
PMR_M1 = [-1.554, -0.529, 1.570]
PMR_C0 = [-2.285, -0.826, 2.043]
PMR_C1 = [-1.554, -0.529, 1.570]

def calculate_PML_coeffs (RLM_coeffs, PMR_coeffs):
    A = PMR_coeffs[0] + PMR_coeffs[2]*RLM_coeffs[0]
    B = PMR_coeffs[1] + PMR_coeffs[2]*RLM_coeffs[2]
    C = PMR_coeffs[2]*RLM_coeffs[1]
    return np.asarray([A, B, C])

PML_M0 = calculate_PML_coeffs(RLM_M,PMR_M0)
PML_M1 = calculate_PML_coeffs(RLM_M,PMR_M1)
PML_C0 = calculate_PML_coeffs(RLM_C,PMR_C0)
PML_C1 = calculate_PML_coeffs(RLM_C,PMR_C1)
```

[57]:
```
def histogram_check_mdot_by_crit_mdot (star_data, spec_mode_label,␣
 ↪bins=10,tev=2*10**6):
    log_mdot_by_mdotcrit = np.log10(star_data[MdotIndex]/(star_data[MIndex]/
 ↪tev))
    plot.hist(log_mdot_by_mdotcrit,bins)
    plot.xlabel(r'$\log(\dot{M}/\dot{M}_{\mathrm{crit}})$')
    plot.title(spec_mode_label)
    plot.show()
    plot.close('all')
```

[58]:
```
def histogram_check_log_index (star_data, index,spec_mode_label, xlab,bins=10,␣
↪tev=2*10**6):
    hist_values = np.log10(star_data[index])
    plot.hist(hist_values,bins)
    plot.title(spec_mode_label)
    plot.xlabel(xlab)
    plot.show()
    plot.close('all')

def histogram_check_log_index_multi_list (star_data, index,spec_mode_label,␣
↪xlab,bins=10, tev=2*10**6):
    # Type 1: Overlay
    hist_values_list = []
    for i in range(0, len(star_data)):
        hist_values_list.append(star_data[i][index])
    hist_values_array = np.asarray(hist_values_list)
    plot.hist(hist_values_array,bins,histtype='bar')
    plot.title(spec_mode_label)
    plot.xlabel(xlab)
    plot.show()
    plot.close('all')
    # Type 2: Stacked
    hist_values_list = []
    for i in range(0, len(star_data)):
        hist_values_list.append(star_data[i][index])
    hist_values_array = np.asarray(hist_values_list)
    plot.hist(hist_values_array,bins,histtype='barstacked')
    plot.title(spec_mode_label)
    plot.xlabel(xlab)
    plot.show()
    plot.close('all')
```

[59]:
```
combined_data_set_raw = np.hstack([M0StarData[0:28],M1StarData[0:
↪28],C0StarData,C1StarData])
```

[60]:
```
M0StarData_critzone = critical_zone_only(M0StarData)
M1StarData_critzone = critical_zone_only(M1StarData)
C0StarData_critzone = critical_zone_only(C0StarData_n)
C0xStarData_critzone = critical_zone_only(C0StarData_x)
C1StarData_critzone = critical_zone_only(C1StarData)
```

```
69 stars cut. 1882 stars remaining.
10 stars over the critical mass-loss rate. 59 stars under the critical mass-loss
rate.

-----
134 stars cut. 2028 stars remaining.
10 stars over the critical mass-loss rate. 124 stars under the critical mass-
```

```
loss rate.

-----
319 stars cut. 1335 stars remaining.
0 stars over the critical mass-loss rate. 319 stars under the critical mass-loss
rate.

-----
3 stars cut. 338 stars remaining.
3 stars over the critical mass-loss rate. 0 stars under the critical mass-loss
rate.

-----
174 stars cut. 607 stars remaining.
2 stars over the critical mass-loss rate. 172 stars under the critical mass-loss
rate.

-----
```

[61]:
```
combined_data_set = np.hstack([M0StarData_critzone[0:28],M1StarData_critzone[0:
↪28],C0StarData_critzone,C1StarData_critzone,C0xStarData_critzone])
```

[62]:
```
def width_by_slicing (star_data):
    logLData = np.log10(star_data[LIndex])
    logMData = np.log10(star_data[MIndex])
    unique_logL = np.unique(logLData)
    num_strips = len(unique_logL)
    logL_bars = []
    for i in range(0, len(unique_logL)):
        logL_bars.append(np.argwhere(logLData==unique_logL[i]))
    logL_bars=np.asarray(logL_bars)
    #print(logL_bars)
    delta_logM_values = []
    range_lb = int(np.rint(num_strips/3))
    range_ub = int(np.rint(2*num_strips/3))
    for i in range(range_lb, range_ub):
        logMvalues = logMData[logL_bars[i]]
        if len(logMvalues) > 0:
            delta_logM_value = np.nanmax(logMvalues)-np.nanmin(logMvalues)
            delta_logM_values.append(delta_logM_value)
    #print('delta log M (mean):',np.nanmean(delta_logM_values))
    #print('delta log M (median):',np.nanmedian(delta_logM_values))
    #print('CHECK: Range of log M is:', np.nanmax(logMData) - np.
↪nanmin(logMData))

    # Now, do the same for delta log L. To be fair to the data, we will split␣
↪the data verically into the same number of
    # horizontal strips we have.
    num_strips = len(unique_logL)
    strip_width = (np.nanmax(logMData) - np.nanmin(logMData))/num_strips
    #print('strip_width:',strip_width)
```

```
    delta_logL_values = []
    #print('CHECK: Max and Min of logM are', np.nanmax(logMData),' ', np.
↪nanmin(logMData))
    for i in range(range_lb, range_ub):
        #print(np.argwhere(logMData>strip_width*i))
        data_set_lb = np.nanmin(logMData) + strip_width*i
        #print('data_set_lb:',data_set_lb)
        data_set_ub = np.nanmin(logMData) + strip_width*(i+1)
        #print('data_set_ub:',data_set_ub)
        data_set_over_lb = np.argwhere(logMData>data_set_lb)
        #print('data_set_over_lb',data_set_over_lb)
        data_set_under_ub = np.argwhere(logMData<data_set_ub)
        #print('data_set_under_ub',data_set_under_ub)
        #print('np.in1d(data_set_lb,data_set_ub)==True):',np.argwhere(np.
↪in1d(data_set_over_lb,data_set_under_ub)==True))
        data_set = data_set_over_lb[np.argwhere(np.
↪in1d(data_set_over_lb,data_set_under_ub)==True)]
        #print(data_set)
        #print(len(data_set))
        if len(data_set >0):
            delta_logL_value = np.nanmax(logLData[data_set])-np.
↪nanmin(logLData[data_set])
            delta_logL_values.append(delta_logL_value)

    #print('delta log L (mean):',np.nanmean(delta_logL_values))
    #print('delta log L (median):',np.nanmedian(delta_logL_values))
    #print('CHECK: Range of log L is:', np.nanmax(logLData) - np.
↪nanmin(logLData))

    return np.nanmedian(delta_logL_values), np.nanmedian(delta_logM_values)
width_by_slicing(M1StarData)
```

[62]:
```
(0.4228428249018725, 0.2832345758749221)
```

[63]:
```
def deathline_analysis_testing (star_data,spec_mode_label =␣
↪'',std_multiplier=1,text_x=0.
↪03,delta_mdot_multiplier=1,trend_correct=False,make_plots = True,␣
↪print_results = True):
    logLData = np.log10(star_data[LIndex])
    logMData = np.log10(star_data[MIndex])
    logMdotData = np.log10(star_data[MdotIndex])
    # Take data, fit logM v logL (this line is the deathline)
    deathline_fit = linear_fit_1var(np.log10(star_data[LIndex]),np.
↪log10(star_data[MIndex]))
    deathline_coeffs = deathline_fit[0]
    # Plot logM v logL and fit line for checking
```

```
    line_x_vals = np.asarray([np.nanmin(np.log10(star_data[LIndex])),np.
↪nanmax(np.log10(star_data[LIndex]))])
    line_y_vals = deathline_coeffs[0] + deathline_coeffs[1]*line_x_vals
    #bound_size = std_multiplier*np.nanstd(np.log10(star_data[MIndex]))
    stnd_dev_around_DL = np.nanstd(np.
↪log10(star_data[MIndex])-deathline_coeffs[0]-deathline_coeffs[1]*np.
↪log10(star_data[LIndex]))

    if print_results == True:
        print('sigmaDL=',stnd_dev_around_DL)
    # Take the Standard Deviation around the deathline to be the standard␣
↪deviation of the difference between the dependent logM and the fit of logL
    bound_size = std_multiplier*stnd_dev_around_DL*np.cos(np.
↪arctan(deathline_coeffs[1]))


    if print_results == True:
        print('Bound Size (logM):', bound_size)
    line_y_vals_up = deathline_coeffs[0] + deathline_coeffs[1]*line_x_vals +␣
↪bound_size
    line_y_vals_down = deathline_coeffs[0] + deathline_coeffs[1]*line_x_vals -␣
↪bound_size
    # Find width of deltaLogL and deltaLogM using fit line, and plot them
    logL_left = (np.average(line_y_vals) - deathline_coeffs[0]-bound_size)/
↪deathline_coeffs[1]
    logL_right = (np.average(line_y_vals) - deathline_coeffs[0]+bound_size)/
↪deathline_coeffs[1]
    delta_logL = logL_right - logL_left
    delta_logM = line_y_vals_up[1] - line_y_vals_down[1]

    # OVERRIDE
    #delta_logL, delta_logM = width_by_slicing (star_data)

    if print_results == True:
        print(' logL =',delta_logL)
        print(' logM =',delta_logM)
    # Using data, calculate deltaLogMdot
    '''
    To accurately determine delta log Mdot, we need to characterize its width␣
↪within the contect of the strip defintiion above.
    To do that, we will find all stars within those bounds, and then make␣
↪histogram of the mass-loss rates. This should let us
    determine an appropriate vale of delta log Mdot visually.
    '''
    star_data_inside_bound = np.asarray([])
    for i in range(0, len(star_data[MIndex])):
```

```
        if logMData[i] <= deathline_coeffs[0] + deathline_coeffs[1]*logLData[i]␣
↪+ bound_size:
            if logMData[i] >= deathline_coeffs[0] +␣
↪deathline_coeffs[1]*logLData[i] - bound_size:
                if len(star_data_inside_bound) == 0:
                    star_data_inside_bound = np.transpose(star_data)[i]
                else:
                    star_data_inside_bound = np.
↪vstack((star_data_inside_bound,np.transpose(star_data)[i]))
    star_data_inside_bound = np.transpose(star_data_inside_bound)
    fig, ax = plt.subplots()
    n_hist, bins_hist, patches_hist = plot.hist(np.
↪log10(star_data_inside_bound[MdotIndex]))
    plot.close()
    logMdotBoundedData = np.log10(star_data_inside_bound[MdotIndex])
    logMdotBoundedData_mean = np.nanmean(logMdotBoundedData)
    logMdotBoundedData_std = np.nanstd(logMdotBoundedData)
    logMBoundedData = np.log10(star_data_inside_bound[MIndex])
    logLBoundedData = np.log10(star_data_inside_bound[LIndex])

    FWHM = 2*np.sqrt(2*np.log(2))

    # Histogram
    if make_plots == True:
        plot.vlines([logMdotBoundedData_mean-0.
↪5*logMdotBoundedData_std,logMdotBoundedData_mean+0.
↪5*logMdotBoundedData_std],0,np.nanmax(n_hist),label='sigma=0.5',color='tab:
↪orange')
        plot.
↪vlines([logMdotBoundedData_mean-logMdotBoundedData_std,logMdotBoundedData_mean+logMdotBound
↪nanmax(n_hist),label='sigma=1',color='tab:red')
        plot.vlines([logMdotBoundedData_mean-(FWHM/
↪2)*logMdotBoundedData_std,logMdotBoundedData_mean+(FWHM/
↪2)*logMdotBoundedData_std],0,np.nanmax(n_hist),label='sigma=FWHM',color='tab:
↪green')
        plot.
↪vlines([logMdotBoundedData_mean-2*logMdotBoundedData_std,logMdotBoundedData_mean+2*logMdotB
↪nanmax(n_hist),label='sigma=2',color='tab:gray')
        plot.
↪vlines([logMdotBoundedData_mean-3*logMdotBoundedData_std,logMdotBoundedData_mean+3*logMdotB
↪nanmax(n_hist),label='sigma=3',color='tab:olive')
        ax.text(0.15, .
↪97,'std_multiplier='+str(std_multiplier),horizontalalignment='center',verticalalignment='ce
↪= ax.transAxes,fontsize=10)
        plot.legend()
        plot.show()
```

```
        plot.close()

    delta_logMdot = delta_mdot_multiplier*np.nanstd(np.
↪log10(star_data_inside_bound[MdotIndex]))
    if print_results == True:
        print(' logMdot =',delta_logMdot)
    # Calculate B = deltaLogMdot/deltaLogL and C = deltaLogMdot/deltaLogM
    B = delta_logMdot/delta_logL
    C = -delta_logMdot/delta_logM


    # Plot
    if make_plots == True:
        fig, ax = plt.subplots()
        plot.scatter(logLData, logMData,s=2)
        plot.scatter(logLBoundedData, logMBoundedData,s=1,color='tab:orange')
        plot.plot(line_x_vals,line_y_vals,linestyle='--',color='black')
        plot.plot(line_x_vals,line_y_vals_up,linestyle='--',color='tab:red')
        plot.plot(line_x_vals,line_y_vals_down,linestyle='--',color='tab:red')
        plot.xlabel(r'$\log L$')
        plot.ylabel(r'$\log M$')
        #plot.title(spec_mode_label)
        ax.text(text_x, .
↪97,spec_mode_label,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
        ax.text(0.15, .
↪92,'std_multiplier='+str(std_multiplier),horizontalalignment='center',verticalalignment='ce
↪= ax.transAxes,fontsize=10)
        plot.vlines(np.average(line_x_vals),np.average(line_y_vals)-delta_logM/
↪2,np.average(line_y_vals)+delta_logM/2,color='tab:
↪green',linestyle='-',linewidth=4)
        if trend_correct ==True:
            plot.vlines(np.average(line_x_vals),np.
↪average(line_y_vals)-new_delta_logM/2,np.average(line_y_vals)+new_delta_logM/
↪2,color='tab:red',linestyle='--',linewidth=2)
        plot.plot([logL_left,logL_right],[np.average(line_y_vals),np.
↪average(line_y_vals)],color='tab:purple',linewidth=4)
        if trend_correct ==True:
            plot.plot([np.average(line_x_vals)-new_delta_logL/2,np.
↪average(line_x_vals)+new_delta_logL/2],[np.average(line_y_vals),np.
↪average(line_y_vals)],color='tab:olive',linewidth=2,linestyle='--')
        if type(std_multiplier) == int:
            plot_name =␣
↪'deathzone_'+spec_mode_label+'_stnd_dev_'+str(std_multiplier)
        #elif std_multiplier/(1/2*std_multiplier) == 2:
        #    plot_name = 'deathzone_'+spec_mode_label+'_stnd_dev_of_all_double'
```

```
        else:
            plot_name = 'deathzone_'+spec_mode_label+'_stnd_dev_of_all'
        plot.savefig(plot_name+'.png')
        plot.savefig(plot_name+'.pdf') # Why do journals require this format...
        plot.show()
        plot.close('all')
    if print_results == True:
        print('B:',B)
        print('C:',C)

    return logMdotBoundedData_std, B, C
```

[64]:
```
def where_become_scatter (star_data, terminal_value = 0.1,delta_sigma_n_mult =␣
↪0.05,initial_value=0.5,plot_label = '',text_x=0.03, print_results=False):
    # Plot 1: std. dev. of log Mdot vs strip width
    sigma_mdot = 0
    delta_sigma_mdot = 1
    sigma_mdot_old = 0
    step = 0
    sigma_n_mult = initial_value
    B_list = []
    C_list = []
    sigma_mdot_list = []
    n_list = []
    if print_results == True:
         print('sigma_n_mult:',sigma_n_mult)
    while sigma_n_mult <= terminal_value:
        sigma_mdot, B, C = deathline_analysis_testing␣
↪(star_data,std_multiplier=sigma_n_mult, delta_mdot_multiplier= sigma_n_mult,␣
↪trend_correct=False,make_plots = False, print_results = print_results)
        B_list.append(B)
        C_list.append(C)
        sigma_mdot_list.append(sigma_mdot)
        n_list.append(sigma_n_mult)
        sigma_n_mult = sigma_n_mult + delta_sigma_n_mult
        if print_results == True:
            print('sigma_mdot:',sigma_mdot)
            print('sigma_n_mult:',sigma_n_mult)
    B_list = np.asarray(B_list)
    C_list = np.asarray(C_list)
    sigma_mdot_list = np.asarray(sigma_mdot_list)
    n_list = np.asarray(n_list)
    fig, ax = plt.subplots()
    vlines_min = np.nanmin(sigma_mdot_list)
    vlines_max = np.nanmin(sigma_mdot_list)
    plot.vlines([0.5,1,FWHM/2,1.5,2],vlines_min,vlines_max,linestyles = '--')
```

```
   ax.text(text_x, .
↪97,plot_label,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
   plot.scatter(n_list,sigma_mdot_list,color='tab:blue',s=2)
   plot.xlabel(r'$n$'+' (strip width)')
   plot.ylabel(r'$\sigma(\dot{M})$')
   plot.show()
   plot.close()

   fig, ax = plt.subplots()
   ax.text(text_x, .
↪97,plot_label,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
   plot.scatter(n_list,B_list,color='tab:blue',s=2)
   plot.xlabel(r'$n$'+' (strip width)')
   plot.ylabel(r'$B$')
   vlines_min = np.nanmin(B_list)
   vlines_max = np.nanmin(B_list)
   plot.vlines([0.5,1,FWHM/2,1.5,2],vlines_min,vlines_max,linestyles = '--')
   plot.show()
   plot.close()

   fig, ax = plt.subplots()
   ax.text(text_x, .
↪97,plot_label,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
   plot.scatter(n_list,B_list/sigma_mdot_list,color='tab:blue',s=2)
   plot.xlabel(r'$n$'+' (strip width)')
   plot.ylabel(r'$B/\sigma(\dot{M})$')
   vlines_min = np.nanmin(B_list/sigma_mdot_list)
   vlines_max = np.nanmin(B_list/sigma_mdot_list)
   plot.vlines([0.5,1,FWHM/2,1.5,2],vlines_min,vlines_max,linestyles = '--')
   plot.show()
   plot.close()


   fig, ax = plt.subplots()
   ax.text(text_x, .
↪97,plot_label,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
   plot.scatter(n_list,C_list,color='tab:blue',s=2)
   plot.xlabel(r'$n$'+' (strip width)')
   plot.ylabel(r'$C$')
   vlines_min = np.nanmin(C_list)
   vlines_max = np.nanmin(C_list)
   plot.vlines([0.5,1,FWHM/2,1.5,2],vlines_min,vlines_max,linestyles = '--')
   plot.show()
```

```
    plot.close()

    fig, ax = plt.subplots()
    ax.text(text_x, .
↪97,plot_label,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
    plot.scatter(n_list,C_list/sigma_mdot_list,color='tab:blue',s=2)
    plot.xlabel(r'$n$'+' (strip width)')
    plot.ylabel(r'$C/\sigma(\dot{M})$')
    vlines_min = np.nanmin(C_list/sigma_mdot_list)
    vlines_max = np.nanmin(C_list/sigma_mdot_list)
    plot.vlines([0.5,1,FWHM/2,1.5,2],vlines_min,vlines_max,linestyles = '--')
    plot.show()
    plot.close()


    #print('Standard Dev. of logMdot of selection becomes stationary at␣
↪sigma_n=', sigma_n_mult)
```

[65]:
```
FWHM = 2*np.sqrt(2*np.log(2))
```

where_become_scatter(M0StarData,terminal_value = 4,delta_sigma_n_mult = 0.01,initial_value=0.25,plot_label='M0', print_results = True)

where_become_scatter(M1StarData,terminal_value = 4,delta_sigma_n_mult = 0.01,initial_value=0.25,plot_label='M1')

where_become_scatter(combined_data_set_C0,terminal_value = 4,delta_sigma_n_mult = 0.01,initial_value=0.25,plot_label='C0')

where_become_scatter(C1StarData,terminal_value = 4,delta_sigma_n_mult = 0.01,initial_value=0.25,plot_label='C1')

[66]:
```
combined_data_set_C0 = np.hstack([C0StarData_n, C0StarData_x])
```

[67]:
```
def deathline_analysis_testing (star_data,spec_mode_label =␣
↪'',std_multiplier=1,text_x=0.
↪03,delta_mdot_multiplier=1,trend_correct=False,make_plots = True,␣
↪print_results = True):
    logLData = np.log10(star_data[LIndex])
    logMData = np.log10(star_data[MIndex])
    logMdotData = np.log10(star_data[MdotIndex])
    # Take data, fit logM v logL (this line is the deathline)
    deathline_fit = linear_fit_1var(np.log10(star_data[LIndex]),np.
↪log10(star_data[MIndex]))
    deathline_coeffs = deathline_fit[0]
    # Plot logM v logL and fit line for checking
```

```
    line_x_vals = np.asarray([np.nanmin(np.log10(star_data[LIndex])),np.
↪nanmax(np.log10(star_data[LIndex]))])
    line_y_vals = deathline_coeffs[0] + deathline_coeffs[1]*line_x_vals
    #bound_size = std_multiplier*np.nanstd(np.log10(star_data[MIndex]))
    stnd_dev_around_DL = np.nanstd(np.
↪log10(star_data[MIndex])-deathline_coeffs[0]-deathline_coeffs[1]*np.
↪log10(star_data[LIndex]))

    if print_results == True:
        print('sigmaDL=',stnd_dev_around_DL)
    # Take the Standard Deviation around the deathline to be the standard␣
↪deviation of the difference between the dependent logM and the fit of logL
    bound_size = std_multiplier*stnd_dev_around_DL*np.cos(np.
↪arctan(deathline_coeffs[1]))


    if print_results == True:
        print('Bound Size (logM):', bound_size)
    line_y_vals_up = deathline_coeffs[0] + deathline_coeffs[1]*line_x_vals +␣
↪bound_size
    line_y_vals_down = deathline_coeffs[0] + deathline_coeffs[1]*line_x_vals -␣
↪bound_size
    # Find width of deltaLogL and deltaLogM using fit line, and plot them
    logL_left = (np.average(line_y_vals) - deathline_coeffs[0]-bound_size)/
↪deathline_coeffs[1]
    logL_right = (np.average(line_y_vals) - deathline_coeffs[0]+bound_size)/
↪deathline_coeffs[1]
    delta_logL = logL_right - logL_left
    delta_logM = line_y_vals_up[1] - line_y_vals_down[1]

    # OVERRIDE
    #delta_logL, delta_logM = width_by_slicing (star_data)

    if print_results == True:
        print(' logL =',delta_logL)
        print(' logM =',delta_logM)
    # Using data, calculate deltaLogMdot
    '''
    To accurately determine delta log Mdot, we need to characterize its width␣
↪within the contect of the strip defintiion above.
    To do that, we will find all stars within those bounds, and then make␣
↪histogram of the mass-loss rates. This should let us
    determine an appropriate vale of delta log Mdot visually.
    '''
    star_data_inside_bound = np.asarray([])
    for i in range(0, len(star_data[MIndex])):
```

```
        if logMData[i] <= deathline_coeffs[0] + deathline_coeffs[1]*logLData[i]␣
↪+ bound_size:
            if logMData[i] >= deathline_coeffs[0] +␣
↪deathline_coeffs[1]*logLData[i] - bound_size:
                if len(star_data_inside_bound) == 0:
                    star_data_inside_bound = np.transpose(star_data)[i]
                else:
                    star_data_inside_bound = np.
↪vstack((star_data_inside_bound,np.transpose(star_data)[i]))
    star_data_inside_bound = np.transpose(star_data_inside_bound)
    fig, ax = plt.subplots()
    n_hist, bins_hist, patches_hist = plot.hist(np.
↪log10(star_data_inside_bound[MdotIndex]))
    plot.close()
    logMdotBoundedData = np.log10(star_data_inside_bound[MdotIndex])
    logMdotBoundedData_mean = np.nanmean(logMdotBoundedData)
    logMdotBoundedData_std = np.nanstd(logMdotBoundedData)
    logMBoundedData = np.log10(star_data_inside_bound[MIndex])
    logLBoundedData = np.log10(star_data_inside_bound[LIndex])

    FWHM = 2*np.sqrt(2*np.log(2))

    # Histogram
    if make_plots == True:
        plot.vlines([logMdotBoundedData_mean-0.
↪5*logMdotBoundedData_std,logMdotBoundedData_mean+0.
↪5*logMdotBoundedData_std],0,np.nanmax(n_hist),label='sigma=0.5',color='tab:
↪orange')
        plot.
↪vlines([logMdotBoundedData_mean-logMdotBoundedData_std,logMdotBoundedData_mean+logMdotBound
↪nanmax(n_hist),label='sigma=1',color='tab:red')
        plot.vlines([logMdotBoundedData_mean-(FWHM/
↪2)*logMdotBoundedData_std,logMdotBoundedData_mean+(FWHM/
↪2)*logMdotBoundedData_std],0,np.nanmax(n_hist),label='sigma=FWHM',color='tab:
↪green')
        plot.
↪vlines([logMdotBoundedData_mean-2*logMdotBoundedData_std,logMdotBoundedData_mean+2*logMdotB
↪nanmax(n_hist),label='sigma=2',color='tab:gray')
        plot.
↪vlines([logMdotBoundedData_mean-3*logMdotBoundedData_std,logMdotBoundedData_mean+3*logMdotB
↪nanmax(n_hist),label='sigma=3',color='tab:olive')
        ax.text(0.15, .
↪97,'std_multiplier='+str(std_multiplier),horizontalalignment='center',verticalalignment='ce
↪= ax.transAxes,fontsize=10)
        plot.legend()
        plot.show()
```

```
        plot.close()

    delta_logMdot = FWHM*np.nanstd(np.log10(star_data_inside_bound[MdotIndex]))
    # Calculate B = deltaLogMdot/deltaLogL and C = deltaLogMdot/deltaLogM
    B = delta_logMdot/delta_logL
    C = -delta_logMdot/delta_logM


    return logMdotBoundedData_std, B, C
```

[68]:
```
np.full((2,3),1.0)
```

[68]:
```
array([[1., 1., 1.],
       [1., 1., 1.]])
```

[69]:
```
def fit_golf (star_data, n=2, initial_m=0.2, terminal_m=3.0, delta_m=0.1,
              plot_label ='',trend_correct = False):
    # Goal: script that calculates A, B, C for fit. Then, take those values and␣
↪compare them to ideal. Knowing that,
    # rank them based on quality. Why golf? Closer to 0 is better!
    logLData = np.log10(star_data[LIndex])
    logMData = np.log10(star_data[MIndex])
    logMdotData = np.log10(star_data[MdotIndex])
    # Take data, fit logM v logL (this line is the deathline)
    deathline_fit = linear_fit_1var(logLData,logMData)
    deathline_coeffs = deathline_fit[0]
    # Plot logM v logL and fit line for checking
    line_x_vals = np.asarray([np.nanmin(np.log10(star_data[LIndex])),np.
↪nanmax(np.log10(star_data[LIndex]))])
    line_y_vals = deathline_coeffs[0] + deathline_coeffs[1]*line_x_vals
    #bound_size = std_multiplier*np.nanstd(np.log10(star_data[MIndex]))
    stnd_dev_around_DL = np.
↪nanstd(logMData-deathline_coeffs[0]-deathline_coeffs[1]*logLData)

    # Take the Standard Deviation around the deathline to be the standard␣
↪deviation of the difference between the dependent logM and the fit of logL
    # After this, we need to loop as we change n and m.
    #array_width = int(np.rint((terminal_n - initial_n)/delta_n))
    array_height = int(np.rint((terminal_m - initial_m)/delta_m))
    #print('array_width:',array_width)
    print('array_height:', array_height)

    A_array = np.full((array_height),100.0)
    B_array = np.full((array_height),100.0)
    C_array = np.full((array_height),100.0)
    #n_array = np.full((array_height),100.0)
    m_array = np.full((array_height),100.0)
```

```
    fit_quality_array = np.full((array_height),100.0) # Array to hold the slope␣
↪of calc vs obsv. Ideal is 1.
    fit_significance_array = np.full((array_height),100.0) # Array to hold the␣
↪slope of calc vs obsv R^2. Ideal is 1.
    fit_quality_array_weighted = np.full((array_height),100.0)
    fit_significance_array_weighted = np.full((array_height),100.0)

    current_n = n
    width_index = 0
    bound_size = current_n*stnd_dev_around_DL*np.cos(np.
↪arctan(deathline_coeffs[1]))
    line_y_vals_up = deathline_coeffs[0] + deathline_coeffs[1]*line_x_vals +␣
↪bound_size
    line_y_vals_down = deathline_coeffs[0] + deathline_coeffs[1]*line_x_vals -␣
↪bound_size
    # Find width of deltaLogL and deltaLogM using fit line, and plot them
    logL_left = (np.average(line_y_vals) - deathline_coeffs[0]-bound_size)/
↪deathline_coeffs[1]
    logL_right = (np.average(line_y_vals) - deathline_coeffs[0]+bound_size)/
↪deathline_coeffs[1]
    delta_logL = logL_right - logL_left
    delta_logM = line_y_vals_up[1] - line_y_vals_down[1]
    star_data_inside_bound = np.asarray([])
    for i in range(0, len(star_data[MIndex])):
        if logMData[i] <= deathline_coeffs[0] + deathline_coeffs[1]*logLData[i]␣
↪+ bound_size:
            if logMData[i] >= deathline_coeffs[0] +␣
↪deathline_coeffs[1]*logLData[i] - bound_size:
                if len(star_data_inside_bound) == 0:
                    star_data_inside_bound = np.transpose(star_data)[i]
                else:
                    star_data_inside_bound = np.
↪vstack((star_data_inside_bound,np.transpose(star_data)[i]))
    star_data_inside_bound = np.transpose(star_data_inside_bound)
    logMdotBoundedData = np.log10(star_data_inside_bound[MdotIndex])
    logMdotBoundedData_mean = np.nanmean(logMdotBoundedData)
    logMdotBoundedData_std = np.nanstd(logMdotBoundedData)
    logMBoundedData = np.log10(star_data_inside_bound[MIndex])
    logLBoundedData = np.log10(star_data_inside_bound[LIndex])
    current_m = initial_m
    height_index = 0
    while current_m < terminal_m:
        delta_logMdot = current_m*logMdotBoundedData_std
        # Calculate B = deltaLogMdot/deltaLogL and C = deltaLogMdot/deltaLogM
        B = delta_logMdot/delta_logL
        C = -delta_logMdot/delta_logM
```

```
        # Simpler, we can force A to be whatever is necessary for the mean␣
↪residual to be 0
        # <logMdot_obsv - logMdot_calc> = 0 = sum(logMdot_obsv[i] - (A +␣
↪BlogL[i] + ClogM[i]))/len(data)
        # => <A> = sum(logMdot_obsv[i] - BlogL[i] - ClogM[i])/len(data)
        A = 0
        for i in range(0, np.shape(star_data)[1]):
            if np.isnan(np.log10(star_data[MdotIndex][i]))==True:
                print('Index ',i,' skipped due to NaN in logMdot')
            elif np.isnan(np.log10(star_data[LIndex][i]))==True:
                print('Index ',i,' skipped due to NaN in logL')
            elif np.isnan(np.log10(star_data[MIndex][i]))==True:
                print('Index ',i,' skipped due to NaN in logM')
            else:
                A = A + np.log10(star_data[MdotIndex][i])-B*np.
↪log10(star_data[LIndex][i]) - C*np.log10(star_data[MIndex][i])
        A = A/np.shape(star_data)[1]
        logMdot_predict = A + B*np.log10(star_data[LIndex]) + C*np.
↪log10(star_data[MIndex])
        correction = np.nanmean(np.log10(star_data[MdotIndex])-logMdot_predict)
        #print('Correction to A:',correction)
        A = A + correction
        if trend_correct ==True:
            logMdot_guess = A + B*np.log10(star_data[LIndex]) + C*np.
↪log10(star_data[MIndex])
            resid = np.log10(star_data[MdotIndex]) - logMdot_guess
            trend = linear_fit_1var(np.log10(star_data[MIndex]),resid)[0][1]
            #print('Old C:',C)
            #print('Old C/New C:',C/(C+trend))
            C = C + trend
            #print('New C:',C)
            trend = linear_fit_1var(np.log10(star_data[LIndex]),resid)[0][1]
            #print('Old B:',B)
            #print('Old B/New B:',B/(B+trend))
            B = B + trend
            #print('New B:',B)
            new_delta_logL = delta_logMdot/B
            #print('new_delta_logL',new_delta_logL)
            new_delta_logM = -delta_logMdot/C
            #print('new_delta_logM',new_delta_logM)
        #print('Editing position (',height_index, ',', width_index,')')
        A_array[height_index] = A
        B_array[height_index] = B
        C_array[height_index] = C
        calc_logMdot = A + B*logLData + C*logMData
        fit = linear_fit_1var(logMdotData, calc_logMdot)
        print('fit:',fit)
```

```
        fit_params = fit[0]
        fitRSq = fit[1]
        fit_quality_array[height_index] = fit_params[1]
        fit_significance_array[height_index] = fitRSq
        #print('m=',current_m, 'complete')
        #n_array[height_index] = current_n
        m_array[height_index] = current_m
        height_index = height_index + 1
        current_m = current_m + delta_m
        decimals_m = int(np.rint(-np.log10(delta_m)+1))
        #print('decimals:',decimals_m)
        current_m=np.round(current_m,decimals=decimals_m)

    golfing_array = np.abs(np.sqrt((fit_quality_array_weighted)**2 +␣
↪(fit_significance_array_weighted)**2))
    # Golfing array should now be filled with values around 0. Wherever this is␣
↪minimum is where we have found the function that
    # replicates the observed mdots best.
    #print(np.nanmin(golfing_array))
    #print(np.argmin(golfing_array))
    pos_id = np.argmin(golfing_array)
    A_array_flat = A_array.flatten()
    B_array_flat = B_array.flatten()
    C_array_flat = C_array.flatten()
    #n_array_flat = n_array.flatten()
    m_array_flat = m_array.flatten()
    fit_quality_array_flat = fit_quality_array.flatten()
    fit_significance_array_flat = fit_significance_array.flatten()

    print('Lowest Score Achieved at position ',np.argmin(golfing_array), ' with␣
↪A=', A_array_flat[pos_id],', B=',
          B_array_flat[pos_id],', and C=',C_array_flat[pos_id], '\n where␣
↪quality of fit (slope) is',
          fit_quality_array_flat[pos_id], 'and signifciance (R^2) is',␣
↪fit_significance_array_flat[pos_id])
    print('For these exponents, n=',n,' and m=',m_array_flat[pos_id])

    print('A_array_flat:',A_array_flat)
    print('B_array_flat:',B_array_flat)
    print('C_array_flat:',C_array_flat)
    #print('n_array_flat:',n_array_flat)
    print('m_array_flat:',m_array_flat)
    print('fit_quality_array_flat:',fit_quality_array_flat)
    print('fit_significance_array_flat:',fit_significance_array_flat)

    return A_array_flat[pos_id], B_array_flat[pos_id], C_array_flat[pos_id]
```

[70]:
```
def fit_seek_one_fixed_n (star_data, n=2, initial_m=0.2, terminal_m=3.0,
                plot_label ='',iteration_limit = 1000, digit_precision = 6):
    '''
    Goal: script that calculates A, B, C for fit. Then, take those values and␣
↪compare them to ideal. Knowing that,
    compare to previous fit. If better, continue. If worse, increase resolution␣
↪then continue from previous.
    Repeat until a limit on the number of iterations is reached or we are as␣
↪close to 1 as we can measure with
    numerical precision.
    '''
    '''
    First, we will handle all of the calculations that are simply properties of␣
↪the data set. No need to recalculate
    these on every iteration. These calculations are finding the line of best␣
↪fit for the data (presumably the deathline),
    finding the standard deviation of the data around the deathline,␣
↪calculating the vertical displacement of the upper and
    lower bounds given a multiplier n, calculating the horizontal displacement␣
↪of the upper and lower bounds from the fit line,
    using those two calculations to determine the height and width of the strip␣
↪(respectively), and sorting out the data found
    inside our LM strip.
    '''
    logLData = np.log10(star_data[LIndex])
    logMData = np.log10(star_data[MIndex])
    logMdotData = np.log10(star_data[MdotIndex])
    # Take data, fit logM v logL (this line is the deathline)
    deathline_fit = linear_fit_1var(logLData,logMData)
    deathline_coeffs = deathline_fit[0]
    # Plot logM v logL and fit line for checking
    line_x_vals = np.asarray([np.nanmin(np.log10(star_data[LIndex])),np.
↪nanmax(np.log10(star_data[LIndex]))])
    line_y_vals = deathline_coeffs[0] + deathline_coeffs[1]*line_x_vals
    #bound_size = std_multiplier*np.nanstd(np.log10(star_data[MIndex]))
    stnd_dev_around_DL = np.
↪nanstd(logMData-deathline_coeffs[0]-deathline_coeffs[1]*logLData)
    bound_size = n*stnd_dev_around_DL*np.cos(np.arctan(deathline_coeffs[1]))
    line_y_vals_up = deathline_coeffs[0] + deathline_coeffs[1]*line_x_vals +␣
↪bound_size
    line_y_vals_down = deathline_coeffs[0] + deathline_coeffs[1]*line_x_vals -␣
↪bound_size
    # Find width of deltaLogL and deltaLogM using fit line, and plot them
    logL_left = (np.average(line_y_vals) - deathline_coeffs[0]-bound_size)/
↪deathline_coeffs[1]
```

```
    logL_right = (np.average(line_y_vals) - deathline_coeffs[0]+bound_size)/
↪deathline_coeffs[1]
    delta_logL = logL_right - logL_left
    delta_logM = line_y_vals_up[1] - line_y_vals_down[1]
    star_data_inside_bound = np.asarray([])
    for i in range(0, len(star_data[MIndex])):
        if logMData[i] <= deathline_coeffs[0] + deathline_coeffs[1]*logLData[i]␣
↪+ bound_size:
            if logMData[i] >= deathline_coeffs[0] +␣
↪deathline_coeffs[1]*logLData[i] - bound_size:
                if len(star_data_inside_bound) == 0:
                    star_data_inside_bound = np.transpose(star_data)[i]
                else:
                    star_data_inside_bound = np.
↪vstack((star_data_inside_bound,np.transpose(star_data)[i]))
    star_data_inside_bound = np.transpose(star_data_inside_bound)
    logMdotBoundedData = np.log10(star_data_inside_bound[MdotIndex])
    logMdotBoundedData_mean = np.nanmean(logMdotBoundedData)
    logMdotBoundedData_std = np.nanstd(logMdotBoundedData)
    logMBoundedData = np.log10(star_data_inside_bound[MIndex])
    logLBoundedData = np.log10(star_data_inside_bound[LIndex])

    '''
    Now onto the loop. For some number of iterations, we are going to seek␣
↪better fits for logMdot = A + BlogL + ClogM.
    From previous experimentation (see above function, fit_golf), we know that␣
↪as we adjust m, the significance of the fit
    is more-or-less fixed using this method. Therefore, we can keep iterating␣
↪to improve the quality of the fit.
    '''
    m = initial_m
    delta_m = 0.1 # An initial incrementing value. We'll start with rough␣
↪adjustment and change this as we get closer to a good fit
    i = 0
    early_cancel_flag = False
    while i < iteration_limit:
        delta_logMdot = m*logMdotBoundedData_std
        # Calculate B = deltaLogMdot/deltaLogL and C = deltaLogMdot/deltaLogM
        B = delta_logMdot/delta_logL
        C = -delta_logMdot/delta_logM
        # Simpler, we can force A to be whatever is necessary for the mean␣
↪residual to be 0
        # <logMdot_obsv - logMdot_calc> = 0 = sum(logMdot_obsv[i] - (A +␣
↪BlogL[i] + ClogM[i]))/len(data)
        # => <A> = sum(logMdot_obsv[i] - BlogL[i] - ClogM[i])/len(data)
        A = 0
```

```
        for k in range(0, np.shape(star_data)[1]):
            if np.isnan(logMdotData[k])==True:
                print('Index ',i,' skipped due to NaN in logMdot')
            elif np.isnan(logLData[k])==True:
                print('Index ',i,' skipped due to NaN in logL')
            elif np.isnan(logMData[k])==True:
                print('Index ',i,' skipped due to NaN in logM')
            else:
                A = A + logMdotData[k]-B*logLData[k] - C*logMData[k]
        A = A/np.shape(star_data)[1]
        logMdot_predict = A + B*np.log10(star_data[LIndex]) + C*np.
↪log10(star_data[MIndex])
        correction = np.nanmean(np.log10(star_data[MdotIndex])-logMdot_predict)
        A = A + correction
        calc_logMdot = A + B*logLData + C*logMData

        fit = linear_fit_1var(logMdotData, calc_logMdot)
        fit_params = fit[0]
        fitRSq = fit[1]

        current_fit_quality = 1 - fit_params[1]
        if i != 0: # We have nothing to compare against on the first␣
↪itteration, so we need an exception built-in here
            if current_fit_quality > 0:
                if current_fit_quality < previous_fit_quality: # If we are␣
↪closer to 1 than before, continue with our routine
                    m = m + delta_m
                    previous_previous_fit_quality = previous_fit_quality
                    previous_fit_quality = current_fit_quality # Override the␣
↪previous fit quality with what we just found
                elif previous_fit_quality < current_fit_quality:
                    delta_m = delta_m/10. # Increase our resolution, then␣
↪repeat the loop with the new, smaller change
                else:
                    print('Stationary at m=',m,'. n/m=',n/m)
                    i = iteration_limit
                    early_cancel_flag = True
            elif current_fit_quality < 0: # Oops, we overshot! That's okay,␣
↪though. We'll go back a step and try with a smaller resolution.
                m = m - delta_m
                delta_m = delta_m/10. # Increase our resolution, then repeat␣
↪the loop with the new, smaller change
                previous_fit_quality = previous_previous_fit_quality
                print('New delta m:',delta_m)
                if np.log10(delta_m) < -digit_precision:
```

```
                    print('Stationary at m=',m,'. n/m=',n/m,' as recognized by␣
↪delta_m limit.')
                    i = iteration_limit
                    early_cancel_flag = True
            else:
                print('Arbitrarily close to 1 at m=',m,'.')
                i = iteration_limit # Break the loop early. No reason to␣
↪continue.
                early_cancel_flag = True
        else: # What to do if we are on the first iteration
            previous_previous_fit_quality = current_fit_quality
            previous_fit_quality = current_fit_quality # Assign our first fit␣
↪quality to a variable so we can update and check against it
            m = initial_m + delta_m
        if m >= terminal_m:
            print('Loop terminated due to reaching limit of allowed values for␣
↪m.')
            i = iteration_limit
            early_cancel_flag = True
        i += 1 # Increment
    if early_cancel_flag == False:
        print('Itteration limit reached.')
    print('Fit quality: Slope=',1-current_fit_quality)
    print('Fit significance: R^2=', fitRSq)
    print('log A, B, C:', A, B, C)
    print('\n')
    return A, B, C

def fit_seek_one_percentile_m (star_data, initial_n=0.5, terminal_n=3.0,␣
↪delta_n=0.1,percentile_to_trim = 0.05,
              spec_mode_label ='', digit_precision = 6,text_x=0.03):
    '''
    Goal: script that calculates A, B, C for fit. Then, take those values and␣
↪compare them to ideal. Knowing that,
    compare to previous fit. If better, continue. If worse, increase resolution␣
↪then continue from previous.
    Repeat until a limit on the number of iterations is reached or we are as␣
↪close to 1 as we can measure with
    numerical precision.
    '''
    '''
    First, we will handle all of the calculations that are simply properties of␣
↪the data set. No need to recalculate
    these on every iteration. These calculations are finding the line of best␣
↪fit for the data (presumably the deathline),
```

```
   finding the standard deviation of the data around the deathline,␣
↪calculating the vertical displacement of the upper and
   lower bounds given a multiplier n, calculating the horizontal displacement␣
↪of the upper and lower bounds from the fit line,
   using those two calculations to determine the height and width of the strip␣
↪(respectively), and sorting out the data found
   inside our LM strip.
   '''
   logLData = np.log10(star_data[LIndex])
   logMData = np.log10(star_data[MIndex])
   logMdotData = np.log10(star_data[MdotIndex])
   # Take data, fit logM v logL (this line is the deathline)
   deathline_fit = linear_fit_1var(logLData,logMData)
   deathline_coeffs = deathline_fit[0]
   print('deathline_fit:',deathline_fit)
   # Plot logM v logL and fit line for checking
   line_x_vals = np.asarray([np.nanmin(np.log10(star_data[LIndex])),np.
↪nanmax(np.log10(star_data[LIndex]))])
   line_y_vals = deathline_coeffs[0] + deathline_coeffs[1]*line_x_vals
   #bound_size = std_multiplier*np.nanstd(np.log10(star_data[MIndex]))
   stnd_dev_around_DL = np.
↪nanstd(logMData-deathline_coeffs[0]-deathline_coeffs[1]*logLData)

   i = 0
   n = initial_n
   #delta_n = 0.1 # An initial incrementing value. We'll start with rough␣
↪adjustment and change this as we get closer to a good fit
   early_cancel_flag = False

   n_array = []
   Exponents_array = []
   fit_qual_array = []
   stand_dev_array = []
   rsq_array = []
   deltas_array = []
   while n<=terminal_n:
       n_array.append(n)
       bound_size = n*stnd_dev_around_DL*np.cos(np.arctan(deathline_coeffs[1]))
       line_y_vals_up = deathline_coeffs[0] + deathline_coeffs[1]*line_x_vals␣
↪+ bound_size
       line_y_vals_down = deathline_coeffs[0] +␣
↪deathline_coeffs[1]*line_x_vals - bound_size
       # Find width of deltaLogL and deltaLogM using fit line, and plot them
       logL_left = (np.average(line_y_vals) - deathline_coeffs[0]-bound_size)/
↪deathline_coeffs[1]
```

```
        logL_right = (np.average(line_y_vals) - deathline_coeffs[0]+bound_size)/
↪deathline_coeffs[1]
        delta_logL = logL_right - logL_left
        delta_logM = line_y_vals_up[1] - line_y_vals_down[1]
        star_data_inside_bound = np.asarray([])
        for l in range(0, len(star_data[MIndex])):
            if logMData[l] <= deathline_coeffs[0] +␣
↪deathline_coeffs[1]*logLData[l] + bound_size:
                if logMData[l] >= deathline_coeffs[0] +␣
↪deathline_coeffs[1]*logLData[l] - bound_size:
                    if len(star_data_inside_bound) == 0:
                        star_data_inside_bound = np.transpose(star_data)[l]
                    else:
                        star_data_inside_bound = np.
↪vstack((star_data_inside_bound,np.transpose(star_data)[l]))
        star_data_inside_bound = np.transpose(star_data_inside_bound)
        logMdotBoundedData = np.log10(star_data_inside_bound[MdotIndex])
        logMdotBoundedData_mean = np.nanmean(logMdotBoundedData)
        logMdotBoundedData_std = np.nanstd(logMdotBoundedData)
        logMBoundedData = np.log10(star_data_inside_bound[MIndex])
        logLBoundedData = np.log10(star_data_inside_bound[LIndex])

        '''
        Now onto the loop. For some number of iterations, we are going to seek␣
↪better fits for logMdot = A + BlogL + ClogM.
        From previous experimentation (see above function, fit_golf), we know␣
↪that as we adjust m/n, the significance of the fit
        is more-or-less fixed using this method. Therefore, we can keep␣
↪iterating to improve the quality of the fit. We are doing
        this over n and are adjusting m to be to trim the most extreme␣
↪mass-loss rates
        '''
        logMdot_spread_from_mean = np.abs(logMdotBoundedData -␣
↪logMdotBoundedData_mean)
        num_to_check = int(np.
↪round_(percentile_to_trim*len(logMdot_spread_from_mean)))
        #print('num_to_check:',num_to_check)
        indices_to_trim = logMdot_spread_from_mean.argsort()[-num_to_check:]
        trimmed_logMdotBoundedData = np.
↪delete(logMdotBoundedData,indices_to_trim)
        delta_logMdot = np.nanmax(trimmed_logMdotBoundedData)-np.
↪nanmin(trimmed_logMdotBoundedData)
        deltas_array.append([delta_logL, delta_logM,delta_logMdot])
        #delta_logMdot = m*logMdotBoundedData_std
        # Calculate B = deltaLogMdot/deltaLogL and C = deltaLogMdot/deltaLogM
        B = delta_logMdot/delta_logL
```

```
        C = -delta_logMdot/delta_logM
        # Simpler, we can force A to be whatever is necessary for the mean␣
↪residual to be 0
        # <logMdot_obsv - logMdot_calc> = 0 = sum(logMdot_obsv[i] - (A +␣
↪BlogL[i] + ClogM[i]))/len(data)
        # => <A> = sum(logMdot_obsv[i] - BlogL[i] - ClogM[i])/len(data)
        A = 0
        for k in range(0, np.shape(star_data)[1]):
            if np.isnan(logMdotData[k])==True:
                #print('Index ',i,' skipped due to NaN in logMdot')
                continue
            elif np.isnan(logLData[k])==True:
                #print('Index ',i,' skipped due to NaN in logL')
                continue
            elif np.isnan(logMData[k])==True:
                #print('Index ',i,' skipped due to NaN in logM')
                continue
            else:
                A = A + logMdotData[k]-B*logLData[k] - C*logMData[k]
        A = A/np.shape(star_data)[1]
        logMdot_predict = A + B*np.log10(star_data[LIndex]) + C*np.
↪log10(star_data[MIndex])
        correction = np.nanmean(np.log10(star_data[MdotIndex])-logMdot_predict)
        A = A + correction
        calc_logMdot = A + B*logLData + C*logMData
        Exponents_array.append([A,B,C])
        fit = linear_fit_1var(logMdotData, calc_logMdot)
        #print('fit:',fit)
        fit_params = fit[0]
        fitRSq = fit[1]
        fit_qual_array.append(fit_params[1])
        rsq_array.append(fitRSq)
        residuals = logMdotData-calc_logMdot
        stand_dev_array.append( np.nanstd(residuals))

        n = n + delta_n
    fit_qual_array = np.asarray(fit_qual_array)
    fit_qual_testing = np.abs(fit_qual_array - 1)
    where_min = np.argmin(fit_qual_testing)

    print('Optimal fit found at ',where_min)
    print('n=',n_array[where_min])
    print('Fit quality: Slope=',fit_qual_array[where_min])
    print('Fit significance: R^2=', rsq_array[where_min])
    print('Fit standard dev: sigma=', stand_dev_array[where_min])
    print('delta logL, delta logM, delta logMdot:', deltas_array[where_min])
    print('log A, B, C:', Exponents_array[where_min])
```

```
    print('\n')

    print('Lower bound fit:')
    print('n=',n_array[0])
    print('Fit quality: Slope=',fit_qual_array[0])
    print('Fit significance: R^2=', rsq_array[0])
    print('Fit standard dev: sigma=', stand_dev_array[0])
    print('delta logL, delta logM, delta logMdot:', deltas_array[0])
    print('log A, B, C:', Exponents_array[0])
    print('\n')

    print('Upper bound fit:')
    print('n=',n_array[-1])
    print('Fit quality: Slope=',fit_qual_array[-1])
    print('Fit significance: R^2=', rsq_array[-1])
    print('Fit standard dev: sigma=', stand_dev_array[-1])
    print('delta logL, delta logM, delta logMdot:', deltas_array[-1])
    print('log A, B, C:', Exponents_array[-1])
    print('\n')

    # Get all of the looping values with our optimal n
    n= n_array[where_min]
    bound_size = n*stnd_dev_around_DL*np.cos(np.arctan(deathline_coeffs[1]))
    line_y_vals_up = deathline_coeffs[0] + deathline_coeffs[1]*line_x_vals +␣
↪bound_size
    line_y_vals_down = deathline_coeffs[0] + deathline_coeffs[1]*line_x_vals -␣
↪bound_size
    # Find width of deltaLogL and deltaLogM using fit line, and plot them
    logL_left = (np.average(line_y_vals) - deathline_coeffs[0]-bound_size)/
↪deathline_coeffs[1]
    logL_right = (np.average(line_y_vals) - deathline_coeffs[0]+bound_size)/
↪deathline_coeffs[1]
    delta_logL = logL_right - logL_left
    delta_logM = line_y_vals_up[1] - line_y_vals_down[1]
    star_data_inside_bound = np.asarray([])
    for l in range(0, len(star_data[MIndex])):
        if logMData[l] <= deathline_coeffs[0] + deathline_coeffs[1]*logLData[l]␣
↪+ bound_size:
            if logMData[l] >= deathline_coeffs[0] +␣
↪deathline_coeffs[1]*logLData[l] - bound_size:
                if len(star_data_inside_bound) == 0:
                    star_data_inside_bound = np.transpose(star_data)[l]
                else:
                    star_data_inside_bound = np.
↪vstack((star_data_inside_bound,np.transpose(star_data)[l]))
    star_data_inside_bound = np.transpose(star_data_inside_bound)
```

```
    logMdotBoundedData = np.log10(star_data_inside_bound[MdotIndex])
    logMdotBoundedData_mean = np.nanmean(logMdotBoundedData)
    logMdotBoundedData_std = np.nanstd(logMdotBoundedData)
    logMBoundedData = np.log10(star_data_inside_bound[MIndex])
    logLBoundedData = np.log10(star_data_inside_bound[LIndex])
    logMdot_spread_from_mean = np.abs(logMdotBoundedData -␣
↪logMdotBoundedData_mean)
    num_to_check = int(np.
↪round_(percentile_to_trim*len(logMdot_spread_from_mean)))
    #print('num_to_check:',num_to_check)
    indices_to_trim = logMdot_spread_from_mean.argsort()[-num_to_check:]
    #print('indices_to_trim',indices_to_trim)
    trimmed_logMdotBoundedData = np.delete(logMdotBoundedData,indices_to_trim)
    #print('Indices removed?:')
    delta_logMdot = np.nanmax(trimmed_logMdotBoundedData)-np.
↪nanmin(trimmed_logMdotBoundedData)
    deltas_array.append([delta_logL, delta_logM,delta_logMdot])
    #delta_logMdot = m*logMdotBoundedData_std
    # Calculate B = deltaLogMdot/deltaLogL and C = deltaLogMdot/deltaLogM
    B = delta_logMdot/delta_logL
    C = -delta_logMdot/delta_logM
    # Simpler, we can force A to be whatever is necessary for the mean residual␣
↪to be 0
    # <logMdot_obsv - logMdot_calc> = 0 = sum(logMdot_obsv[i] - (A + BlogL[i] +␣
↪ClogM[i]))/len(data)
    # => <A> = sum(logMdot_obsv[i] - BlogL[i] - ClogM[i])/len(data)
    A = 0
    for k in range(0, np.shape(star_data)[1]):
        if np.isnan(logMdotData[k])==True:
            #print('Index ',i,' skipped due to NaN in logMdot')
            continue
        elif np.isnan(logLData[k])==True:
            #print('Index ',i,' skipped due to NaN in logL')
            continue
        elif np.isnan(logMData[k])==True:
            #print('Index ',i,' skipped due to NaN in logM')
            continue
        else:
            A = A + logMdotData[k]-B*logLData[k] - C*logMData[k]
    A = A/np.shape(star_data)[1]
    logMdot_predict = A + B*np.log10(star_data[LIndex]) + C*np.
↪log10(star_data[MIndex])
    correction = np.nanmean(np.log10(star_data[MdotIndex])-logMdot_predict)
    A = A + correction
    calc_logMdot = A + B*logLData + C*logMData
    fit = linear_fit_1var(logMdotData, calc_logMdot)
```

```
    fit_params = fit[0]
    fitRSq = fit[1]
    residuals = logMdotData-calc_logMdot


    # Plot 1: The LM Strip
    fig, ax = plt.subplots()
    plot.scatter(np.log10(star_data[LIndex]), np.log10(star_data[MIndex]),s=2)
    plot.plot(line_x_vals,line_y_vals,linestyle='-',color='black')
    plot.plot(line_x_vals,line_y_vals_up,linestyle='--',color='black')
    plot.plot(line_x_vals,line_y_vals_down,linestyle='--',color='black')
    plot.xlabel(r'$\log L$')
    plot.ylabel(r'$\log M$')
    #plot.title(spec_mode_label)
    ax.text(text_x, .
↪97,spec_mode_label,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
    plot.vlines(np.average(line_x_vals),np.average(line_y_vals)-delta_logM/2,np.
↪average(line_y_vals)+delta_logM/2,color='tab:
↪green',linestyle='-',linewidth=4)
    plot.plot([logL_left,logL_right],[np.average(line_y_vals),np.
↪average(line_y_vals)],color='tab:purple',linewidth=4)
    if type(n) == float:
        plot_name = 'deathzone_'+spec_mode_label+'_stnd_dev_'+str(n)[:4]
    #elif std_multiplier/(1/2*std_multiplier) == 2:
    #    plot_name = 'deathzone_'+spec_mode_label+'_stnd_dev_of_all_double'
    else:
        plot_name = 'deathzone_'+spec_mode_label+'_stnd_dev_of_all'
    plot.savefig(plot_name+'.png')
    plot.savefig(plot_name+'.pdf') # Why do journals require this format...
    plot.show()
    plot.close('all')

    # Plot 2: logMdot Histograms
    fig, ax = plt.subplots()
    n_hist, bins_hist, patches_hist = plot.hist(np.
↪log10(star_data_inside_bound[MdotIndex]))
    ax.text(text_x, .
↪97,spec_mode_label,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
    print('Full range:',np.nanmax(logMdotBoundedData),np.
↪nanmin(logMdotBoundedData))
    print('vlines locations:',np.nanmax(trimmed_logMdotBoundedData),np.
↪nanmin(trimmed_logMdotBoundedData))
    plot.vlines([np.nanmax(trimmed_logMdotBoundedData),np.
↪nanmin(trimmed_logMdotBoundedData)],0,np.nanmax(n_hist))
```

```
    if type(n) == int:
        plot_name = 'logMdot_histogram_'+spec_mode_label+'_stnd_dev_'+str(n)
    elif type(n) == float:
        plot_name = 'logMdot_histogram_'+spec_mode_label+'_stnd_dev_'+str(n)[0:
↪5]
    else:
        plot_name = 'logMdot_histogram_'+spec_mode_label+'_stnd_dev_of_all'
    #plot.legend(loc=4)
    plot.savefig(plot_name+'.png')
    plot.savefig(plot_name+'.pdf') # Why do journals require this format...

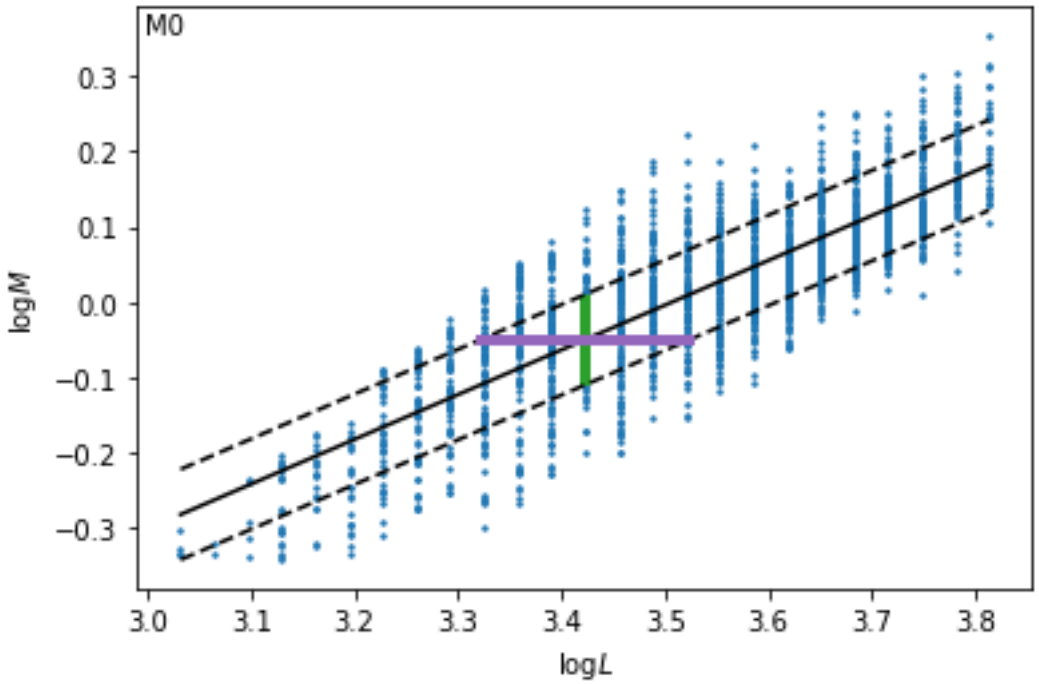

    return A, B, C
```

[71]:
```
#M0_optimal_exponents = fit_golf (M0StarData, n=0.75,initial_m=0.3,␣
↪terminal_m=5., delta_m=0.1,trend_correct = False)
#M0_optimal_exponents = fit_seek_one (M0StarData, n=0.5, initial_m=0.3,␣
↪terminal_m=4.0,plot_label ='',iteration_limit = 100,digit_precision=5)
M0_optimal_exponents = fit_seek_one_percentile_m (M0StarData, initial_n=0.5,␣
↪terminal_n=3.0, delta_n=0.0001,percentile_to_trim = 0.025,
             spec_mode_label='M0')
```

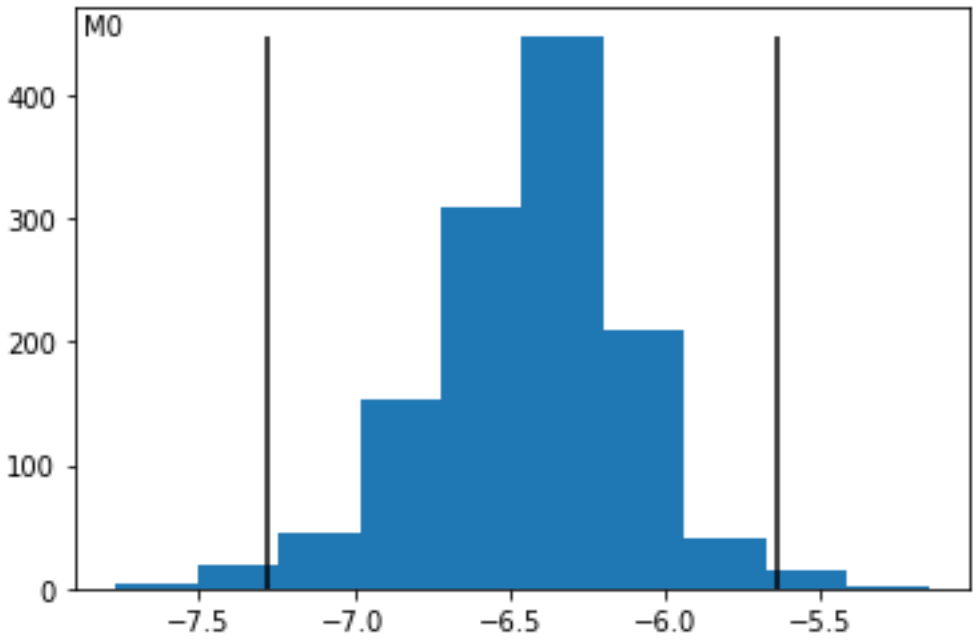
```
deathline_fit: (array([-2.08166269,  0.59359611]), 0.6939735315292701, Intercept
0.031530
x            0.008929
dtype: float64,            Intercept         x
Intercept   0.233686 -0.066104
x          -0.066104  0.018740)
Optimal fit found at  5694
n= 1.0693999999999373
Fit quality: Slope= 1.0000037850481107
Fit significance: R^2= 0.19180011520815876
Fit standard dev: sigma= 0.802789559750684
delta logL, delta logM, delta logMdot: [0.20198361631524042,
0.11989668960937344, 1.6423545861871354]
log A, B, C: [-34.975963392304195, 8.131127742677283, -13.698081169196328]


Lower bound fit:
n= 0.5
Fit quality: Slope= 2.12430541120037
Fit significance: R^2= 0.1918001152081602
Fit standard dev: sigma= 1.7611346721938628
delta logL, delta logM, delta logMdot: [0.09443782322575789,
0.05605792482204064, 1.6312182200691758]
log A, B, C: [-67.03506056677335, 17.272933284047376, -29.09879781043589]
```

```
Upper bound fit:
n= 2.999900000001945
Fit quality: Slope= 0.4093764453350075
Fit significance: R^2= 0.1918001152081592
Fit standard dev: sigma= 0.4016934426344642
delta logL, delta logM, delta logMdot: [0.5666080517902725, 0.33633633734749735,
1.886056647693164]
log A, B, C: [-18.13441585975965, 3.3286795726497718, -5.607650551728879]
```




```
Full range: -5.155448273660727 -7.7648075037301885
vlines locations: -5.638032424081891 -7.280387010269027
```



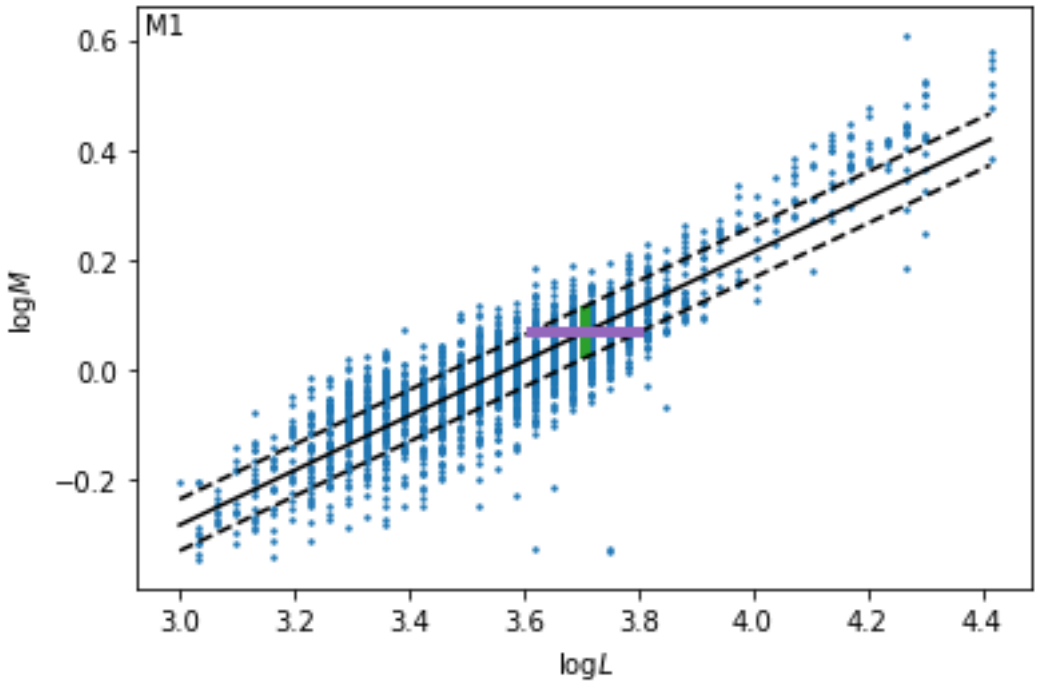
[72]:
```
#M1_optimal_exponents = fit_golf (M1StarData, initial_n=0.25, terminal_n=3.,␣
 ↪initial_m=0.25, terminal_m=4., delta_n=0.05, delta_m=0.05,trend_correct =␣
 ↪False)
#M1_optimal_exponents = fit_seek_one_fixed_n (M1StarData, n=0.5, initial_m=0.3,␣
 ↪terminal_m=4.0,plot_label ='',iteration_limit = 100,digit_precision=5)
M1_optimal_exponents = fit_seek_one_percentile_m (M1StarData, initial_n=0.5,␣
 ↪terminal_n=3.0, delta_n=0.0001,percentile_to_trim = 0.025,
              spec_mode_label='M1')
```

```
deathline_fit: (array([-1.77015472,  0.4958621 ]), 0.7647445924769394, Intercept
0.021242
x            0.005918
dtype: float64,            Intercept         x
Intercept   0.117549 -0.032683
x          -0.032683  0.009123)
Optimal fit found at  3451
n= 0.845099999999962
Fit quality: Slope= 0.9999967327250667
Fit significance: R^2= 0.12227636713260515
Fit standard dev: sigma= 1.1878406074267007
delta logL, delta logM, delta logMdot: [0.1891101235031245, 0.09377254366392473,
1.9199034860015933]
log A, B, C: [-42.82979371999287, 10.152304120143375, -20.474047210262466]

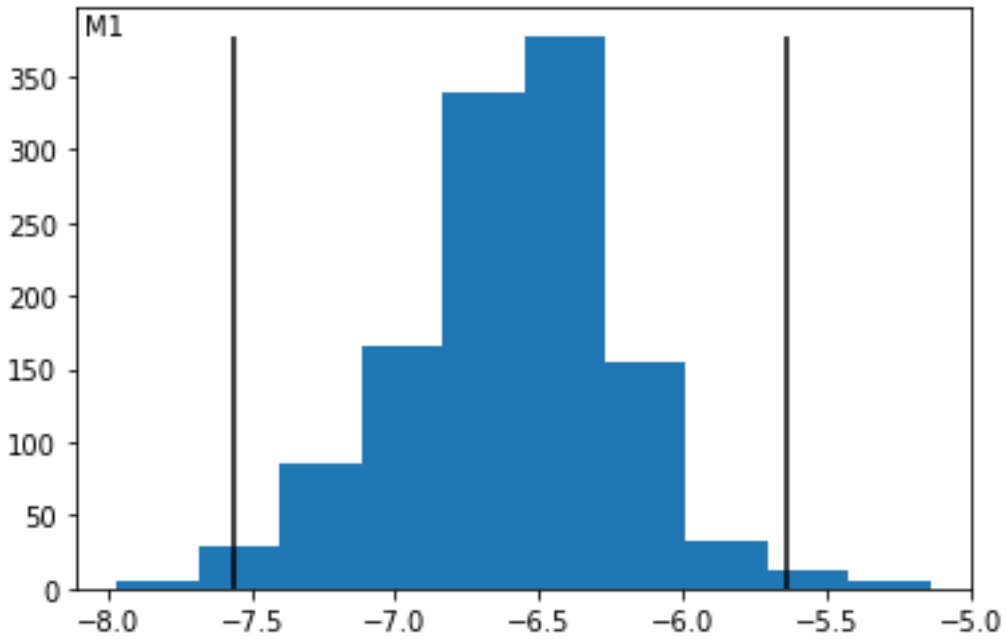

Lower bound fit:
```

n= 0.5
Fit quality: Slope= 1.5986696535219758
Fit significance: R^2= 0.12227636713260459
Fit standard dev: sigma= 1.9174306237799505
delta logL, delta logM, delta logMdot: [0.11188624038760686,
0.05548014652936273, 1.8159398112730436]
log A, B, C: [-64.52710704694141, 16.230233538834568, -32.7313449021257]

Upper bound fit:
n= 2.999900000001945
Fit quality: Slope= 0.2956713860252729
Fit significance: R^2= 0.12227636713260315
Fit standard dev: sigma= 0.46995832024867795
delta logL, delta logM, delta logMdot: [0.6712950650780005, 0.3328697831470864,
2.0150637464358763]
log A, B, C: [-17.30338823114128, 3.001755638113827, -6.053609695012397]



Full range: -5.137719506700295 -7.967457791305338
vlines locations: -5.638032424081891 -7.557935910083485

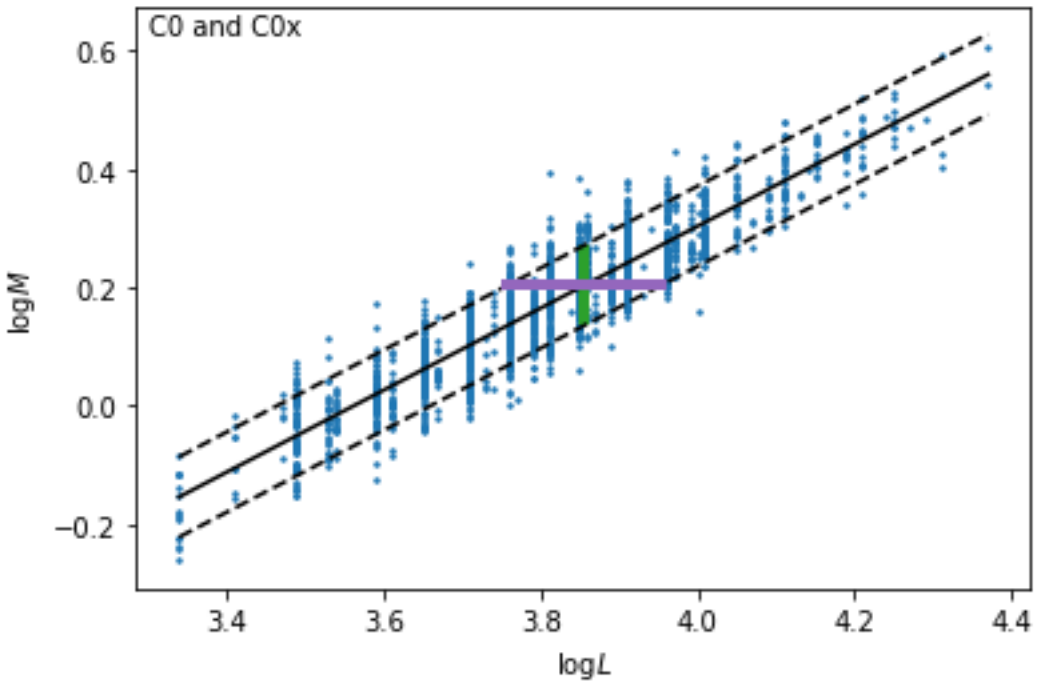


[73]:
```
#C0_optimal_exponents = fit_golf (combined_data_set_C0, initial_n=0.25,␣
 ↪terminal_n=3., initial_m=0.25, terminal_m=4., delta_n=0.05, delta_m=0.
 ↪05,trend_correct = False)
#C0_optimal_exponents = fit_seek_one_fixed_n (combined_data_set_C0, n=0.5,␣
 ↪initial_m=0.3, terminal_m=4.0, plot_label ='',iteration_limit =␣
 ↪100,digit_precision=5)
C0_optimal_exponents = fit_seek_one_percentile_m (combined_data_set_C0,␣
 ↪initial_n=0.5, terminal_n=3.0, delta_n=0.0001,percentile_to_trim = 0.025,
              spec_mode_label ='C0 and C0x', text_x = 0.10)
```

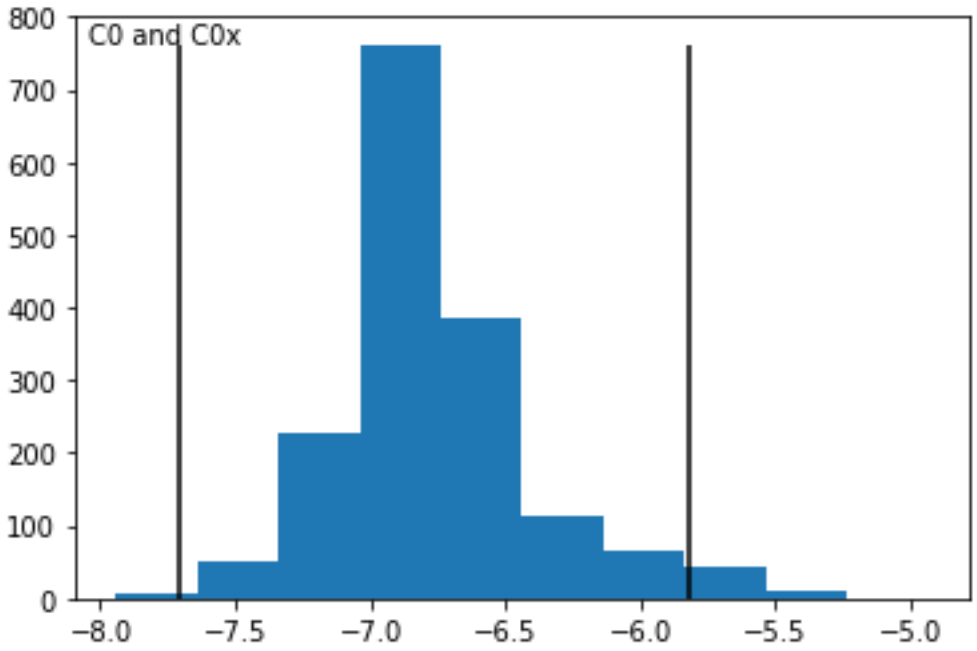
```
deathline_fit: (array([-2.46712362,  0.69265989]), 0.8425134640215445, Intercept
0.025548
x            0.006710
dtype: float64,            Intercept         x
Intercept   0.279136 -0.073246
x          -0.073246  0.019255)
Optimal fit found at  12040
n= 1.7039999999998674
Fit quality: Slope= 1.0000291742728242
Fit significance: R^2= 0.40738409029419853
Fit standard dev: sigma= 0.517546827333577
delta logL, delta logM, delta logMdot: [0.19548072568275332,
0.13540165823092454, 1.8834944309285628]
log A, B, C: [-41.078937026141794, 9.635192545710597, -13.910423664947327]
```

```
Lower bound fit:
n= 0.5
Fit quality: Slope= 2.5481476018874734
Fit significance: R^2= 0.40738409029419465
Fit standard dev: sigma= 1.4766142733923044
delta logL, delta logM, delta logMdot: [0.05735936786466267,
0.03973053351846678, 1.40823996531185]
log A, B, C: [-94.20685284118863, 24.551176516354587, -35.444778627432704]


Upper bound fit:
n= 2.999900000001945
Fit quality: Slope= 0.6106583959313372
Fit significance: R^2= 0.40738409029419986
Fit standard dev: sigma= 0.35747531349839023
delta logL, delta logM, delta logMdot: [0.34414473531462164, 0.238375255004252,
2.0248235837250323]
log A, B, C: [-27.716614374612355, 5.883639573547193, -8.494269187835435]
```




```
Full range: -4.933674074637962 -7.942447998945467
vlines locations: -5.819730722331125 -7.703225153259688
```



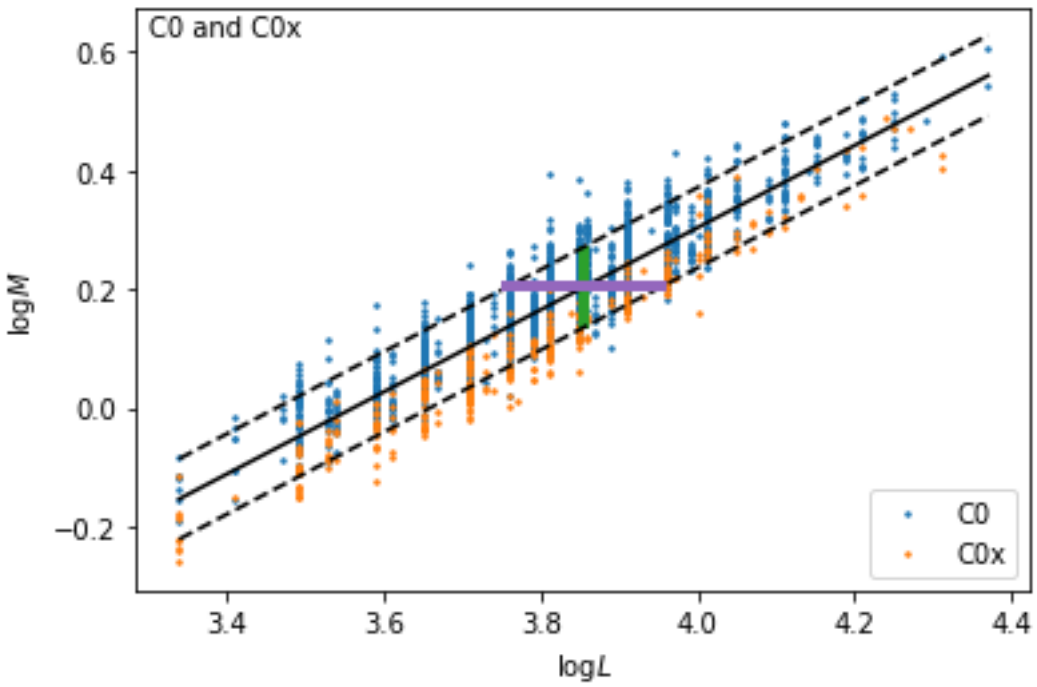
[75]:
```
# Need to make a special plot for these stars because it is two sets. Change n␣
 ↪to match what was found above, and recreate it
# with the sets being split
n_C0C0x=1.7039999999998674
logLData_C0C0x = np.log10(combined_data_set_C0[LIndex])
logMData_C0C0x = np.log10(combined_data_set_C0[MIndex])
logMdotData_C0C0x = np.log10(combined_data_set_C0[MdotIndex])
    # Take data, fit logM v logL (this line is the deathline)
deathline_fit_C0C0x = linear_fit_1var(logLData_C0C0x,logMData_C0C0x)
deathline_coeffs_C0C0x = deathline_fit_C0C0x[0]
    # Plot logM v logL and fit line for checking
line_x_vals_C0C0x = np.asarray([np.nanmin(logLData_C0C0x),np.
 ↪nanmax(logLData_C0C0x)])
line_y_vals_C0C0x = deathline_coeffs_C0C0x[0] +␣
 ↪deathline_coeffs_C0C0x[1]*line_x_vals_C0C0x
    #bound_size = std_multiplier*np.nanstd(np.log10(star_data[MIndex]))
stnd_dev_around_DL_C0C0x = np.
 ↪nanstd(logMData_C0C0x-deathline_coeffs_C0C0x[0]-deathline_coeffs_C0C0x[1]*logLData_C0C0x)
bound_size_C0C0x = n_C0C0x*stnd_dev_around_DL_C0C0x*np.cos(np.
 ↪arctan(deathline_coeffs_C0C0x[1]))
line_y_vals_up_C0C0x = deathline_coeffs_C0C0x[0] +␣
 ↪deathline_coeffs_C0C0x[1]*line_x_vals_C0C0x + bound_size_C0C0x
line_y_vals_down_C0C0x = deathline_coeffs_C0C0x[0] +␣
 ↪deathline_coeffs_C0C0x[1]*line_x_vals_C0C0x - bound_size_C0C0x
# Find width of deltaLogL and deltaLogM using fit line, and plot them
```

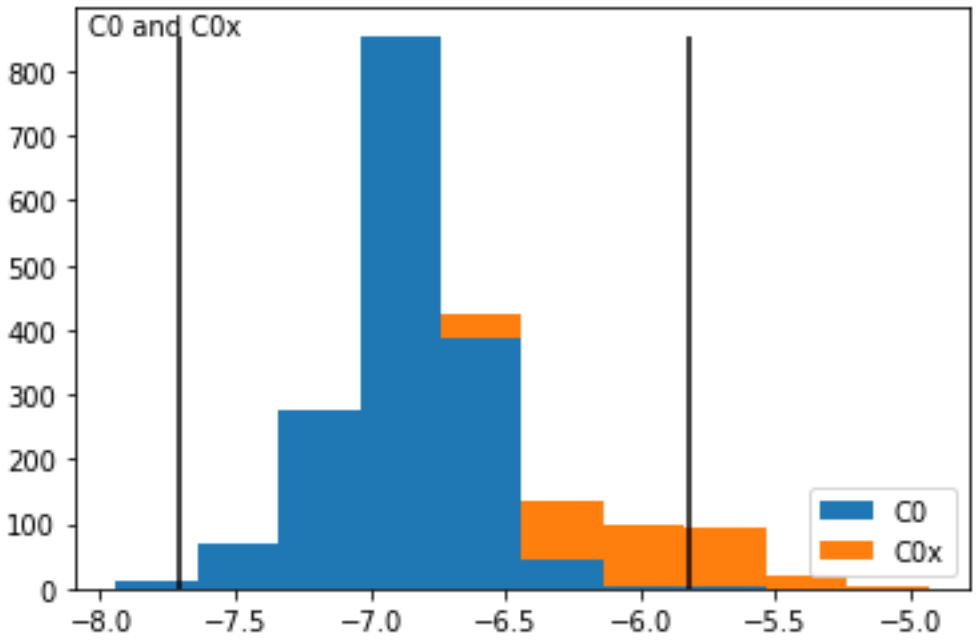
```
logL_left_C0C0x = (np.average(line_y_vals_C0C0x) -␣
 ↪deathline_coeffs_C0C0x[0]-bound_size_C0C0x)/deathline_coeffs_C0C0x[1]
logL_right_C0C0x = (np.average(line_y_vals_C0C0x) -␣
 ↪deathline_coeffs_C0C0x[0]+bound_size_C0C0x)/deathline_coeffs_C0C0x[1]

# Plot 1: LM Strip
fig, ax = plt.subplots()
plot.scatter(np.log10(C0StarData_n[LIndex]), np.
 ↪log10(C0StarData_n[MIndex]),s=2,color='tab:blue',label='C0')
plot.scatter(np.log10(C0StarData_x[LIndex]), np.
 ↪log10(C0StarData_x[MIndex]),s=2,color='tab:orange',label='C0x')
plot.plot(line_x_vals_C0C0x,line_y_vals_C0C0x,linestyle='-',color='black')
plot.plot(line_x_vals_C0C0x,line_y_vals_up_C0C0x,linestyle='--',color='black')
plot.plot(line_x_vals_C0C0x,line_y_vals_down_C0C0x,linestyle='--',color='black')
plot.xlabel(r'$\log L$')
plot.ylabel(r'$\log M$')
#plot.title(spec_mode_label)
text_x_C0C0x = 0.10
spec_mode_label_C0C0x = 'C0 and C0x'
delta_logM = line_y_vals_up_C0C0x-line_y_vals_down_C0C0x
ax.text(text_x_C0C0x, .
 ↪97,spec_mode_label_C0C0x,horizontalalignment='center',verticalalignment='center',transform␣
 ↪= ax.transAxes,fontsize=10)
plot.vlines(np.average(line_x_vals_C0C0x),np.
 ↪average(line_y_vals_C0C0x)-delta_logM/2,np.
 ↪average(line_y_vals_C0C0x)+delta_logM/2,color='tab:
 ↪green',linestyle='-',linewidth=4)
plot.plot([logL_left_C0C0x,logL_right_C0C0x],[np.average(line_y_vals_C0C0x),np.
 ↪average(line_y_vals_C0C0x)],color='tab:purple',linewidth=4)
if type(n_C0C0x) == float:
    plot_name = 'deathzone_'+spec_mode_label_C0C0x+'_stnd_dev_'+str(n_C0C0x)[:4]
    #elif std_multiplier/(1/2*std_multiplier) == 2:
    #    plot_name = 'deathzone_'+spec_mode_label+'_stnd_dev_of_all_double'
else:
    plot_name = 'deathzone_'+spec_mode_label_C0C0x+'_stnd_dev_of_all'
plot.legend(loc=4)
plot.savefig(plot_name+'.png')
plot.savefig(plot_name+'.pdf') # Why do journals require this format...
plot.show()
plot.close('all')

# Plot 2: Histogram
# Plot 2: logMdot Histograms
fig, ax = plt.subplots()
```

```
n_hist_C0C0x, bins_hist_C0C0x, patches_hist_C0C0x = plot.hist([np.
 ↪log10(C0StarData_n[MdotIndex]),np.
 ↪log10(C0StarData_x[MdotIndex])],stacked=True,label=['C0','C0x'])
ax.text(text_x_C0C0x, .
 ↪97,spec_mode_label_C0C0x,horizontalalignment='center',verticalalignment='center',transform␣
 ↪= ax.transAxes,fontsize=10)
plot.vlines([-5.819730722331125,-7.703225153259688],0,np.nanmax(n_hist_C0C0x))
if type(n_C0C0x) == int:
    plot_name =␣
 ↪'logMdot_histogram_'+spec_mode_label_C0C0x+'_stnd_dev_'+str(n_C0C0x)
elif type(n_C0C0x) == float:
    plot_name =␣
 ↪'logMdot_histogram_'+spec_mode_label_C0C0x+'_stnd_dev_'+str(n_C0C0x)[0:5]
else:
    plot_name = 'logMdot_histogram_'+spec_mode_label_C0C0x+'_stnd_dev_of_all'
plot.legend(loc=4)
plot.savefig(plot_name+'.png')
plot.savefig(plot_name+'.pdf') # Why do journals require this format...
```





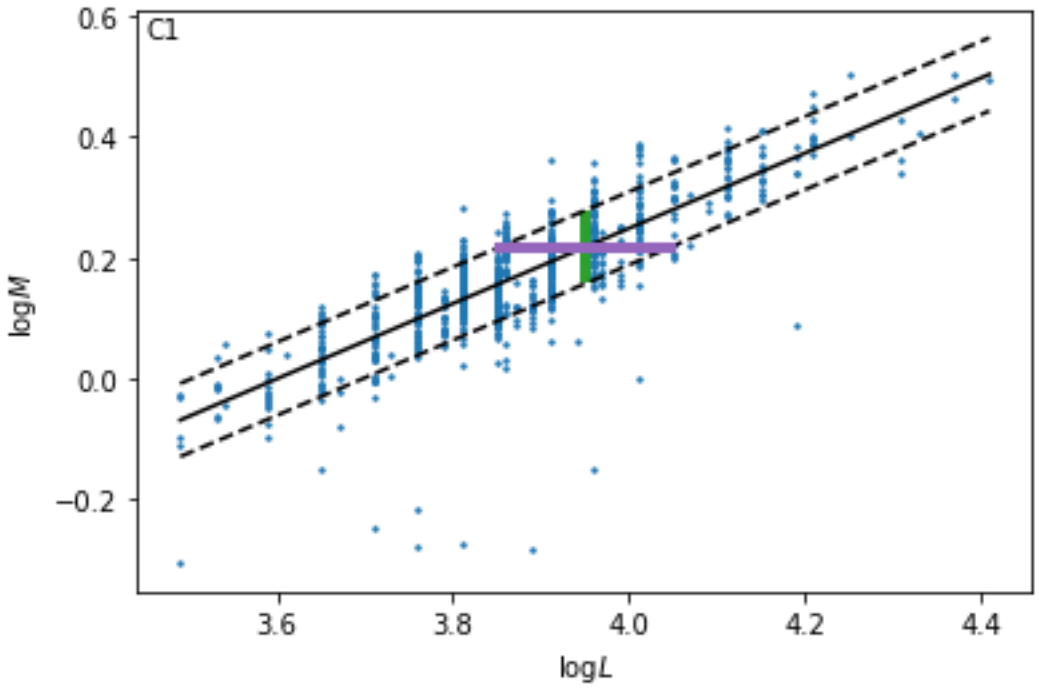
[74]:
```
#C1_optimal_exponents = fit_golf (C1StarData, initial_n=0.25, terminal_n=3.,␣
 ↪initial_m=0.25, terminal_m=4., delta_n=0.05, delta_m=0.05,trend_correct =␣
 ↪False)
#C1_optimal_exponents = fit_seek_one_fixed_n (C1StarData, n=0.5, initial_m=0.3,␣
 ↪terminal_m=4.0,plot_label ='',iteration_limit = 100,digit_precision=5)

C1_optimal_exponents = fit_seek_one_percentile_m (C1StarData, initial_n=0.5,␣
 ↪terminal_n=3.0, delta_n=0.0001,percentile_to_trim = 0.025,
              spec_mode_label ='C1')
```

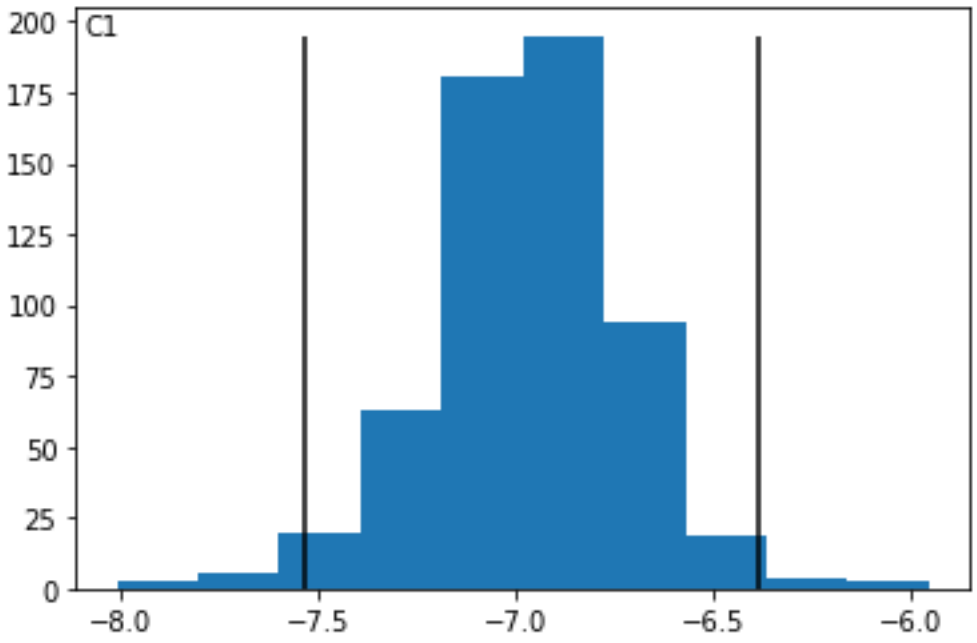
```
deathline_fit: (array([-2.2461344 ,  0.62357106]), 0.6712522002248873, Intercept
0.060517
x            0.015635
dtype: float64,            Intercept         x
Intercept   0.944136 -0.243762
x          -0.243762  0.063021)
Optimal fit found at  6504
n= 1.1503999999999284
Fit quality: Slope= 1.0000181660375453
Fit significance: R^2= 0.24765173220056858
Fit standard dev: sigma= 0.509201851017138
delta logL, delta logM, delta logMdot: [0.19474764044216197,
0.12143899263765517, 1.1461280356782382]
log A, B, C: [-28.14202038380528, 5.88519600584648, -9.437891494192556]
```

```
Lower bound fit:
n= 0.5
Fit quality: Slope= 2.5762157564270125
Fit significance: R^2= 0.24765173220056436
Fit standard dev: sigma= 1.3902636729215105
delta logL, delta logM, delta logMdot: [0.08464344595018058,
0.05278120333695352, 1.2833012287035501]
log A, B, C: [-61.55486794571157, 15.16125925991807, -24.31360309296088]


Upper bound fit:
n= 2.999900000001945
Fit quality: Slope= 0.4854249505701496
Fit significance: R^2= 0.24765173220056647
Fit standard dev: sigma= 0.28929934301635696
delta logL, delta logM, delta logMdot: [0.5078437470122208, 0.31667666378125925,
1.4507923159973135]
log A, B, C: [-17.23347391118014, 2.856769084059238, -4.581304787900101]
```




```
Full range: -5.955950469349115 -8.009394788576081
vlines locations: -6.389606030287687 -7.535734065965925
```



[299]:
```
def deathline_analysis (star_data,spec_mode_label =␣
↪'',std_multiplier=1,text_x=0.03,delta_mdot_multiplier=1,trend_correct=False):
    logLData = np.log10(star_data[LIndex])
    logMData = np.log10(star_data[MIndex])
    logMdotData = np.log10(star_data[MdotIndex])
    # Take data, fit logM v logL (this line is the deathline)
    deathline_fit = linear_fit_1var(np.log10(star_data[LIndex]),np.
↪log10(star_data[MIndex]))
    deathline_coeffs = deathline_fit[0]
    # Plot logM v logL and fit line for checking
    line_x_vals = np.asarray([np.nanmin(np.log10(star_data[LIndex])),np.
↪nanmax(np.log10(star_data[LIndex]))])
    line_y_vals = deathline_coeffs[0] + deathline_coeffs[1]*line_x_vals
    #bound_size = std_multiplier*np.nanstd(np.log10(star_data[MIndex]))
    stnd_dev_around_DL = np.nanstd(np.
↪log10(star_data[MIndex])-deathline_coeffs[0]-deathline_coeffs[1]*np.
↪log10(star_data[LIndex]))
    print('sigmaDL=',stnd_dev_around_DL)
    # Take the Standard Deviation around the deathline to be the standard␣
↪deviation of the difference between the dependent logM and the fit of logL
    bound_size = std_multiplier*stnd_dev_around_DL*np.cos(np.
↪arctan(deathline_coeffs[1]))
```

```
    print('Bound Size (logM):', bound_size)
    line_y_vals_up = deathline_coeffs[0] + deathline_coeffs[1]*line_x_vals +␣
↪bound_size
    line_y_vals_down = deathline_coeffs[0] + deathline_coeffs[1]*line_x_vals -␣
↪bound_size
    # Find width of deltaLogL and deltaLogM using fit line, and plot them
    logL_left = (np.average(line_y_vals) - deathline_coeffs[0]-bound_size)/
↪deathline_coeffs[1]
    logL_right = (np.average(line_y_vals) - deathline_coeffs[0]+bound_size)/
↪deathline_coeffs[1]
    delta_logL = logL_right - logL_left
    delta_logM = line_y_vals_up[1] - line_y_vals_down[1]

    # OVERRIDE
    #delta_logL, delta_logM = width_by_slicing (star_data)

    print(' logL =',delta_logL)
    print(' logM =',delta_logM)
    # Using data, calculate deltaLogMdot
    '''
    delta_logMdot = np.nanmax(np.log10(star_data[MdotIndex])) - np.nanmin(np.
↪log10(star_data[MdotIndex]))
    print('Dumb delta_logMdot:',delta_logMdot)
    '''
    '''
    We should be more sophisticated than what we have above. Taking the maximum␣
↪or minimum of the mass loss rate
    of the unbounded data will lead to significant skew. With the other two␣
↪variables, we are saying something like
    "if we hold M (or L) constant, what range of L (or M) do we measure?". We␣
↪need to keep this restriction on the
    Mdot data, so instead we could ask "what range of Mdot do we measure after␣
↪bounding?" (a valid approximation if B is large)
    or we could have the same confidence restriction, just in Mdot. So, for␣
↪example, say what is the range of logMdot that falls
    within 2-sigma of the mean.
    '''
    '''
    # Option 1: Range of the *bounded* data
    stars_in_bound = []
    print(len(star_data[MdotIndex]))
    for i in range(0, len(star_data[MdotIndex])):
        if np.log10(star_data[MIndex][i]) < deathline_coeffs[0] +␣
↪deathline_coeffs[1]*np.log10(star_data[LIndex][i]) + bound_size:
            #print('True value found.')
```

```
            if np.log10(star_data[MIndex][i]) > deathline_coeffs[0] +␣
↪deathline_coeffs[1]*np.log10(star_data[LIndex][i]) - bound_size:
                #print('Double True value found.')
                stars_in_bound.append(np.transpose(star_data)[i])
            else:
                continue
        else:
            continue
    stars_in_bound = np.transpose(np.asarray(stars_in_bound))
    #print('stars_in_bound:',stars_in_bound)
    delta_logMdot = np.nanmax(np.log10(stars_in_bound[MdotIndex])) - np.
↪nanmin(np.log10(stars_in_bound[MdotIndex]))
    print('Option 1 delta_logMdot:',delta_logMdot)

    # Option 2: What is the width of the distribution of mdot?
    #delta_logMdot = 2*std_multiplier*np.nanstd(np.log10(star_data[MdotIndex]))
    std_dev_logMdot = np.nanstd(np.log10(star_data[MdotIndex]))
    min_logmdot = np.nanmin(np.log10(star_data[MdotIndex]))
    max_logmdot = np.nanmax(np.log10(star_data[MdotIndex]))
    midpoint_logMdot = (min_logmdot + max_logmdot)/2.
    print('max_logmdot:',max_logmdot)
    print('min_logmdot:',min_logmdot)
    print('midpoint_logMdot:',midpoint_logMdot)
    midpoint_logMdot = np.nanmean(np.log10(star_data[MdotIndex]))
    delta_logMdot = delta_mdot_multiplier*std_dev_logMdot
    print('Option 2 delta_logMdot:',delta_logMdot)
    FWHM = 2*np.sqrt(2*np.log(2))
    fig, ax = plt.subplots()
    plot.hist(np.log10(star_data[MdotIndex]))
    ax.text(text_x, .
↪97,spec_mode_label,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
    plot.
↪vlines([midpoint_logMdot-std_dev_logMdot,midpoint_logMdot+std_dev_logMdot],0,300,colors='ta
↪orange')
    plot.vlines([midpoint_logMdot-FWHM/2*std_dev_logMdot,midpoint_logMdot+FWHM/
↪2*std_dev_logMdot],0,300,colors='tab:pink')
    plot.vlines([midpoint_logMdot-1.5*std_dev_logMdot,midpoint_logMdot+1.
↪5*std_dev_logMdot],0,300,colors='tab:red')
    plot.
↪vlines([midpoint_logMdot-2*std_dev_logMdot,midpoint_logMdot+2*std_dev_logMdot],0,300,colors
↪purple')
    plot.vlines([midpoint_logMdot-2.5*std_dev_logMdot,midpoint_logMdot+2.
↪5*std_dev_logMdot],0,300,colors='k')
    plot.show()
    plot.close()
```

```
    '''
    '''
    To accurately determine delta log Mdot, we need to characterize its width␣
↪within the contect of the strip defintiion above.
    To do that, we will find all stars within those bounds, and then make␣
↪histogram of the mass-loss rates. This should let us
    determine an appropriate vale of delta log Mdot visually.
    '''
    star_data_inside_bound = np.asarray([])
    for i in range(0, len(star_data[MIndex])):
        if logMData[i] <= deathline_coeffs[0] + deathline_coeffs[1]*logLData[i]␣
↪+ bound_size:
            if logMData[i] >= deathline_coeffs[0] +␣
↪deathline_coeffs[1]*logLData[i] - bound_size:
                if len(star_data_inside_bound) == 0:
                    star_data_inside_bound = np.transpose(star_data)[i]
                else:
                    star_data_inside_bound = np.
↪vstack((star_data_inside_bound,np.transpose(star_data)[i]))
    star_data_inside_bound = np.transpose(star_data_inside_bound)
    fig, ax = plt.subplots()
    n_hist, bins_hist, patches_hist = plot.hist(np.
↪log10(star_data_inside_bound[MdotIndex]))
    logMdotBoundedData = np.log10(star_data_inside_bound[MdotIndex])
    logMdotBoundedData_mean = np.nanmean(logMdotBoundedData)
    logMdotBoundedData_std = np.nanstd(logMdotBoundedData)
    logMBoundedData = np.log10(star_data_inside_bound[MIndex])
    logLBoundedData = np.log10(star_data_inside_bound[LIndex])
    ax.text(text_x, .
↪97,spec_mode_label,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
    plot.
↪vlines([logMdotBoundedData_mean-delta_mdot_multiplier*logMdotBoundedData_std,logMdotBounded
↪nanmax(n_hist))
    delta_logMdot = 2*delta_mdot_multiplier*logMdotBoundedData_std
    if type(std_multiplier) == int:
        plot_name =␣
↪'logMdot_histogram_'+spec_mode_label+'_stnd_dev_'+str(std_multiplier)
    elif type(std_multiplier) == float:
        plot_name =␣
↪'logMdot_histogram_'+spec_mode_label+'_stnd_dev_'+str(std_multiplier)
    else:
        plot_name = 'logMdot_histogram_'+spec_mode_label+'_stnd_dev_of_all'
    #plot.legend(loc=4)
    plot.savefig(plot_name+'.png')
    plot.savefig(plot_name+'.pdf') # Why do journals require this format...
```

```
    #delta_logMdot = 2
    print(' logMdot =',delta_logMdot)
    # Calculate B = deltaLogMdot/deltaLogL and C = deltaLogMdot/deltaLogM
    B = delta_logMdot/delta_logL
    C = -delta_logMdot/delta_logM

    # From actually testing this method, we know there is a possibility of a␣
↪residual trend in log M or log L. We can correct for this
    # immediately by setting A = 0, calculating a residual, and finding the␣
↪slope (intercept is irrelevant) when plotted vs logM.
    if trend_correct ==True:
        A = 0.
        logMdot_guess = A + B*np.log10(star_data[LIndex]) + C*np.
↪log10(star_data[MIndex])
        resid = np.log10(star_data[MdotIndex]) - logMdot_guess
        trend = linear_fit_1var(np.log10(star_data[MIndex]),resid)[0][1]
        print('Old C:',C)
        print('Old C/New C:',C/(C+trend))
        C = C + trend
        print('New C:',C)
        trend = linear_fit_1var(np.log10(star_data[LIndex]),resid)[0][1]
        print('Old B:',B)
        print('Old B/New B:',B/(B+trend))
        B = B + trend
        print('New B:',B)
        new_delta_logL = delta_logMdot/B
        print('new_delta_logL',new_delta_logL)
        new_delta_logM = -delta_logMdot/C
        print('new_delta_logM',new_delta_logM)

    # Plot
    fig, ax = plt.subplots()
    plot.scatter(np.log10(star_data[LIndex]), np.log10(star_data[MIndex]),s=2)
    plot.plot(line_x_vals,line_y_vals,linestyle='-',color='black')
    plot.plot(line_x_vals,line_y_vals_up,linestyle='--',color='black')
    plot.plot(line_x_vals,line_y_vals_down,linestyle='--',color='black')
    plot.xlabel(r'$\log L$')
    plot.ylabel(r'$\log M$')
    #plot.title(spec_mode_label)
    ax.text(text_x, .
↪97,spec_mode_label,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
    plot.vlines(np.average(line_x_vals),np.average(line_y_vals)-delta_logM/2,np.
↪average(line_y_vals)+delta_logM/2,color='tab:
↪green',linestyle='-',linewidth=4)
```

```
    if trend_correct ==True:
        plot.vlines(np.average(line_x_vals),np.
↪average(line_y_vals)-new_delta_logM/2,np.average(line_y_vals)+new_delta_logM/
↪2,color='tab:red',linestyle='--',linewidth=2)
    plot.plot([logL_left,logL_right],[np.average(line_y_vals),np.
↪average(line_y_vals)],color='tab:purple',linewidth=4)
    if trend_correct ==True:
        plot.plot([np.average(line_x_vals)-new_delta_logL/2,np.
↪average(line_x_vals)+new_delta_logL/2],[np.average(line_y_vals),np.
↪average(line_y_vals)],color='tab:orange',linewidth=2,linestyle='--')
    if type(std_multiplier) == int:
        plot_name =␣
↪'deathzone_'+spec_mode_label+'_stnd_dev_'+str(std_multiplier)
    #elif std_multiplier/(1/2*std_multiplier) == 2:
    #    plot_name = 'deathzone_'+spec_mode_label+'_stnd_dev_of_all_double'
    else:
        plot_name = 'deathzone_'+spec_mode_label+'_stnd_dev_of_all'
    plot.savefig(plot_name+'.png')
    plot.savefig(plot_name+'.pdf') # Why do journals require this format...
    plot.show()
    plot.close('all')




    # Determine A
    '''
    # Idea: Find all points within 0.0125335 std. devs. of deathline (the␣
↪closest 1% of stars), and assume they are
    # 'on' the deathline
    indices_on_DL = []
    for i in range (0, len(star_data[MdotIndex])):
        PredictedLogM = deathline_coeffs[0]+deathline_coeffs[1]*np.
↪log10(star_data[LIndex][i])
        if np.log10(star_data[MIndex][i]) < PredictedLogM + 0.
↪0125335*stnd_dev_around_DL:
            if np.log10(star_data[MIndex][i]) > PredictedLogM - 0.
↪0125335*stnd_dev_around_DL:
                indices_on_DL.append(i)
    print('Number of Stars within 1% of DL:', len(indices_on_DL))
    # Then, find their mass-loss rate.
    closest_logMdots = np.log10(star_data[MdotIndex][indices_on_DL])
    #print('DEBUG: closest_logMdots=',closest_logMdots)
    # Then, take that mass-loss rate and set equal to A+BlogL+ClogM, solve for␣
↪A.
```

```
    AList = closest_logMdots - B*np.log10(star_data[LIndex][indices_on_DL]) +␣
↪C*np.log10(star_data[MIndex][indices_on_DL])
    #print('DEBUG: AList=',AList)
    # Finally, average this value of logA over all included points
    A = np.mean(AList)
    ATestResults = A + B*np.log10(star_data[LIndex][indices_on_DL]) - C*np.
↪log10(star_data[MIndex][indices_on_DL])
    #print('DEBUG: ATestResults', ATestResults)
    '''
    '''
    # Idea:
    # We have B, C, logM = X + Y*logL, logMdot = A + BlogL + ClogM. Pick a␣
↪series of masses and determine log L.
    # Assuming logMdot = logMdotcrit, we have what we need to calculate A for a␣
↪given mass and luminosity.
    min_mass = np.nanmin(star_data[MIndex])
    max_mass = np.nanmax(star_data[MIndex])
    sample_logM = np.log10(np.arange(min_mass, max_mass, (max_mass-min_mass)/
↪200))
    sample_logL = (sample_logM - deathline_coeffs[0])/deathline_coeffs[1]
    sample_logMdot = np.log10((10**sample_logM)/(2*10**6))
    A_list = sample_logMdot - B*sample_logL - C*sample_logM
    A_fit = linear_fit_1var(sample_logM,A_list)
    #print('A_list:', A_list)
    print('A_fit: Intercept=',A_fit[0][0], ', Slope=',A_fit[0][1])
    A = np.mean(A_list)
    plot.scatter(sample_logM,A_list,s=2)
    plot.xlabel(r'$\log M$')
    plot.ylabel(r'$A$')
    plot.title(spec_mode_label)
    plot.show()
    plot.close('all')
    '''
    # Simpler, we can force A to be whatever is necessary for the mean residual␣
↪to be 0
    # <logMdot_obsv - logMdot_calc> = 0 = sum(logMdot_obsv[i] - (A + BlogL[i] +␣
↪ClogM[i]))/len(data)
    # => <A> = sum(logMdot_obsv[i] - BlogL[i] - ClogM[i])/len(data)
    A = 0
    for i in range(0, np.shape(star_data)[1]):
        if np.isnan(np.log10(star_data[MdotIndex][i]))==True:
            print('Index ',i,' skipped due to NaN in logMdot')
        elif np.isnan(np.log10(star_data[LIndex][i]))==True:
            print('Index ',i,' skipped due to NaN in logL')
        elif np.isnan(np.log10(star_data[MIndex][i]))==True:
            print('Index ',i,' skipped due to NaN in logM')
```

```
        else:
            A = A + np.log10(star_data[MdotIndex][i])-B*np.
↪log10(star_data[LIndex][i]) - C*np.log10(star_data[MIndex][i])
    A = A/np.shape(star_data)[1]
    logMdot_predict = A + B*np.log10(star_data[LIndex]) + C*np.
↪log10(star_data[MIndex])
    correction = np.nanmean(np.log10(star_data[MdotIndex])-logMdot_predict)
    print('Correction to A:',correction)
    A = A + correction


    # Check: Make histogram of log(Mdot/Mdotcrit); this should be about 2 dex␣
↪wide and that will match the theoretical death zone width.
    log_mdot_by_mdotcrit = np.log10(star_data[MdotIndex]/(star_data[MIndex]/
↪(2*10**6)))
    fig, ax = plt.subplots()
    plot.hist(log_mdot_by_mdotcrit,10)
    ax.text(text_x, .
↪97,spec_mode_label,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
    #plot.title(spec_mode_label)
    plot.show()
    plot.close('all')


    # Check: Make histogram of logL/logMdot and logM/logMdot

    # Check: Make a plot of our formula here (calculated data) vs. the observed␣
↪data, and compare to a 1-to-1 line
    our_log_mdot = A + B*np.log10(star_data[LIndex]) + C*np.
↪log10(star_data[MIndex])
    logMdotData = np.log10(star_data[MdotIndex])
    logMData = np.log10(star_data[MIndex])
    logLData = np.log10(star_data[LIndex])

    # Now an overlaid version
    # Type 1: Calc Mdot vs Obsv Mdot
    fig, ax = plt.subplots()
    alpha_set=0.33
    plot.scatter(logMdotData,our_log_mdot,s=2,label=r'$\log \dot{M}(\log L,␣
↪\log M)$',alpha=alpha_set,color='tab:blue')
    fit = linear_fit_1var(logMdotData,our_log_mdot)
    fit_params = fit[0]
    fit_x = np.asarray([-9,-3])
    fit_y = fit_params[0]+fit_params[1]*fit_x
    plot.plot(fit_x,fit_y,color = 'tab:blue')
```

```
   #plot.title(spec_mode_label)
   ax.text(text_x, .
↪97,spec_mode_label,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
   plot.xlabel(r'$\log \dot{M}_{obsv.}$')
   plot.ylabel(r'$\log \dot{M}_{calc.}$')
   plot.plot([-9,-3],[-9,-3],color = 'black', alpha =␣
↪1,linestyle='--',linewidth=1.5)
   plot.xlim([-8,-4])
   plot.ylim([-8,-4])
   plot.tight_layout()
   plot.legend(loc=4)
   plot.show()
   plot.close()

   # Type 2: Residual vs Obsv Mdot
   fig, ax = plt.subplots()
   plot.scatter(logMdotData,our_log_mdot-logMdotData,s=2,label=r'$\log␣
↪\dot{M}(\log L, \log M)$',alpha=alpha_set,color='tab:blue')
   fit = linear_fit_1var(logMdotData,our_log_mdot-logMdotData)
   fit_params = fit[0]
   fit_x = np.asarray([-9,-3])
   fit_y = fit_params[0]+fit_params[1]*fit_x
   plot.plot(fit_x,fit_y,color = 'tab:blue')
   print('Mdot(L,P) Slope=',fit_params[1])
   print('Mdot(L,P) R^2=',fit[1])

   #plot.title(spec_mode_label)
   ax.text(text_x, .
↪97,spec_mode_label,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
   plot.xlabel(r'$\log \dot{M}_{obsv.}$')
   plot.ylabel(r'$\log \dot{M}_{calc.}-\log \dot{M}_{obsv.}$')
   plot.plot([-8,-4],[0,0],color = 'black', alpha =␣
↪1,linestyle='--',linewidth=1.5)
   plot.xlim([-8,-4])
   plot.tight_layout
   plot.legend(loc=4)
   plot.show()
   plot.close()

   # Make a test plot. Log L vs log Mdot.
   fig, ax = plt.subplots()
   alpha_set=0.33
```

```
   plot.
↪scatter(logMdotData,logLData,s=2,label='Data',alpha=alpha_set,color='tab:
↪blue')
   fit = linear_fit_1var(logMdotData,logLData)
   fit_params = fit[0]
   fit_x = np.asarray([-9,-3])
   fit_y = fit_params[0]+fit_params[1]*fit_x
   plot.plot(fit_x,fit_y,color = 'tab:blue')
   ax.text(text_x, .
↪97,spec_mode_label,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
   plot.xlabel(r'$\log \dot{M}$')
   plot.ylabel(r'$\log L$')
   plot.plot([-9,-3],[-9,-3],color = 'black', alpha =␣
↪1,linestyle='--',linewidth=1.5)
   plot.xlim([-8,-4])
   plot.ylim([2.75,4])
   plot.tight_layout()
   plot.legend(loc=4)
   plot.show()
   plot.close()

   # Make another test plot. LogMdot vs logM.
   fig, ax = plt.subplots()
   alpha_set=0.33
   plot.
↪scatter(logMdotData,logMData,s=2,label='Data',alpha=alpha_set,color='tab:
↪blue')
   fit = linear_fit_1var(logMdotData,logMData)
   fit_params = fit[0]
   fit_x = np.asarray([-9,-3])
   fit_y = fit_params[0]+fit_params[1]*fit_x
   plot.plot(fit_x,fit_y,color = 'tab:blue')
   ax.text(text_x, .
↪97,spec_mode_label,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
   plot.xlabel(r'$\log \dot{M}$')
   plot.ylabel(r'$\log M$')
   plot.plot([-9,-3],[-9,-3],color = 'black', alpha =␣
↪1,linestyle='--',linewidth=1.5)
   plot.xlim([-8,-4])
   plot.ylim([-0.5,0.5])
   plot.tight_layout()
   plot.legend(loc=4)
   plot.show()
   plot.close()
```

```
    print('For set '+spec_mode_label+': A=',A,', B=', B,', C=', -C)
    print('_____ _____ _____ _____ _____')
    return [A, B, C], star_data_inside_bound
```

[300]:
```
def deathline_analysis_multidata (star_data,spec_mode_label =␣
↪'',std_multiplier=1,text_x=0.03,delta_mdot_multiplier=1,trend_correct=False):


    # Make Title Label
    title_string = ''
    for i in range(0, len(spec_mode_label)):
        if i == 0:
            title_string = spec_mode_label[0]
        elif i < len(spec_mode_label)-1:
            title_string = title_string + ', '+spec_mode_label[i]
        else:
            if len(spec_mode_label) == 2:
                title_string = title_string + ' and ' + spec_mode_label[i]
            else:
                title_string = title_string + ', and '+ spec_mode_label[i]
    #plot.title(title_string)

    star_data_stacked = np.hstack(star_data) # Assume a list of data that would␣
↪be used in version above. Need the stacked version for calculations.
    logLData = np.log10(star_data_stacked[LIndex])
    logMData = np.log10(star_data_stacked[MIndex])
    logMdotData = np.log10(star_data_stacked[MdotIndex])
    # Take data, fit logM v logL (this line is the deathline)
    deathline_fit = linear_fit_1var(np.log10(star_data_stacked[LIndex]),np.
↪log10(star_data_stacked[MIndex]))
    deathline_coeffs = deathline_fit[0]
    np.nanstd(np.log10(star_data_stacked[MIndex]))
    # Plot logM v logL and fit line for checking
    line_x_vals = np.asarray([np.nanmin(np.log10(star_data_stacked[LIndex])),np.
↪nanmax(np.log10(star_data_stacked[LIndex]))])
    line_y_vals = deathline_coeffs[0] + deathline_coeffs[1]*line_x_vals
    #bound_size = std_multiplier*np.nanstd(np.log10(star_data[MIndex]))
    stnd_dev_around_DL = np.nanstd(np.
↪log10(star_data_stacked[MIndex])-deathline_coeffs[0]-deathline_coeffs[1]*np.
↪log10(star_data_stacked[LIndex]))
    print('sigmaDL=',stnd_dev_around_DL)
    # Take the Standard Deviation around the deathline to be the standard␣
↪deviation of the difference between the dependent logM and the fit of logL
```

```
    bound_size = std_multiplier*stnd_dev_around_DL*np.cos(np.
↪arctan(deathline_coeffs[1]))



    print('Bound Size (logM):', bound_size)
    line_y_vals_up = deathline_coeffs[0] + deathline_coeffs[1]*line_x_vals +␣
↪bound_size
    line_y_vals_down = deathline_coeffs[0] + deathline_coeffs[1]*line_x_vals -␣
↪bound_size
    # Find width of deltaLogL and deltaLogM using fit line, and plot them
    logL_left = (np.average(line_y_vals) - deathline_coeffs[0]-bound_size)/
↪deathline_coeffs[1]
    logL_right = (np.average(line_y_vals) - deathline_coeffs[0]+bound_size)/
↪deathline_coeffs[1]
    delta_logL = logL_right - logL_left
    delta_logM = line_y_vals_up[1] - line_y_vals_down[1]

    # OVERRIDE
    #delta_logL, delta_logM = width_by_slicing (star_data_stacked)

    print(' logL =',delta_logL)
    print(' logM =',delta_logM)
    # Using data, calculate deltaLogMdot
    #delta_logMdot = np.nanmax(np.log10(star_data_stacked[MdotIndex])) - np.
↪nanmin(np.log10(star_data_stacked[MdotIndex]))
    #print('Dumb delta_logMdot:',delta_logMdot)
    '''
    We should be more sophisticated than what we have above. Taking the maximum␣
↪or minimum of the mass loss rate
    of the unbounded data will lead to significant skew. With the other two␣
↪variables, we are saying something like
    "if we hold M (or L) constant, what range of L (or M) do we measure?". We␣
↪need to keep this restriction on the
    Mdot data, so instead we could ask "what range of Mdot do we measure after␣
↪bounding?" (a valid approximation if B is large)
    or we could have the same confidence restriction, just in Mdot. So, for␣
↪example, say what is the range of logMdot that falls
    within 2-sigma of the mean.
    '''
    '''
    # Option 1: Range of the *bounded* data
    stars_in_bound = []
    print(len(star_data_stacked[MdotIndex]))
    for i in range(0, len(star_data_stacked[MdotIndex])):
```

```
        if np.log10(star_data_stacked[MIndex][i]) < deathline_coeffs[0] +␣
↪deathline_coeffs[1]*np.log10(star_data_stacked[LIndex][i]) + bound_size:
            #print('True value found.')
            if np.log10(star_data_stacked[MIndex][i]) > deathline_coeffs[0] +␣
↪deathline_coeffs[1]*np.log10(star_data_stacked[LIndex][i]) - bound_size:
                #print('Double True value found.')
                stars_in_bound.append(np.transpose(star_data_stacked)[i])
            else:
                continue
        else:
            continue
    stars_in_bound = np.transpose(np.asarray(stars_in_bound))
    #print('stars_in_bound:',stars_in_bound)
    delta_logMdot = np.nanmax(np.log10(stars_in_bound[MdotIndex])) - np.
↪nanmin(np.log10(stars_in_bound[MdotIndex]))
    print('Option 1 delta_logMdot:',delta_logMdot)

    # Option 2: What is the width of the distribution of mdot?
    #delta_logMdot = 2*std_multiplier*np.nanstd(np.log10(star_data[MdotIndex]))
    std_dev_logMdot = np.nanstd(np.log10(star_data_stacked[MdotIndex]))
    min_logmdot = np.nanmin(np.log10(star_data_stacked[MdotIndex]))
    max_logmdot = np.nanmax(np.log10(star_data_stacked[MdotIndex]))
    midpoint_logMdot = (min_logmdot + max_logmdot)/2.
    print('max_logmdot:',max_logmdot)
    print('min_logmdot:',min_logmdot)
    print('midpoint_logMdot:',midpoint_logMdot)
    midpoint_logMdot = np.nanmean(np.log10(star_data_stacked[MdotIndex]))
    delta_logMdot = delta_mdot_multiplier*std_dev_logMdot
    print('Option 2 delta_logMdot:',delta_logMdot)
    FWHM = 2*np.sqrt(2*np.log(2))
    fig, ax = plt.subplots()
    plot.hist(np.log10(star_data_stacked[MdotIndex]))
    ax.text(text_x, .
↪97,title_string,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
    plot.
↪vlines([midpoint_logMdot-std_dev_logMdot,midpoint_logMdot+std_dev_logMdot],0,300,colors='ta
↪orange')
    plot.vlines([midpoint_logMdot-FWHM/2*std_dev_logMdot,midpoint_logMdot+FWHM/
↪2*std_dev_logMdot],0,300,colors='tab:pink')
    plot.vlines([midpoint_logMdot-1.5*std_dev_logMdot,midpoint_logMdot+1.
↪5*std_dev_logMdot],0,300,colors='tab:red')
    plot.
↪vlines([midpoint_logMdot-2*std_dev_logMdot,midpoint_logMdot+2*std_dev_logMdot],0,300,colors
↪purple')
```

```
    plot.vlines([midpoint_logMdot-2.5*std_dev_logMdot,midpoint_logMdot+2.
↪5*std_dev_logMdot],0,300,colors='k')
    plot.show()
    plot.close()
    '''

    star_data_inside_bound = np.asarray([])
    for i in range(0, len(star_data_stacked[MIndex])):
        if logMData[i] <= deathline_coeffs[0] + deathline_coeffs[1]*logLData[i]␣
↪+ bound_size:
            if logMData[i] >= deathline_coeffs[0] +␣
↪deathline_coeffs[1]*logLData[i] - bound_size:
                if len(star_data_inside_bound) == 0:
                    star_data_inside_bound = np.transpose(star_data_stacked)[i]
                else:
                    star_data_inside_bound = np.
↪vstack((star_data_inside_bound,np.transpose(star_data_stacked)[i]))
    star_data_inside_bound = np.transpose(star_data_inside_bound)
    fig, ax = plt.subplots()
    n_hist, bins_hist, patches_hist = plot.hist(np.
↪log10(star_data_inside_bound[MdotIndex]))
    logMdotBoundedData = np.log10(star_data_inside_bound[MdotIndex])
    logMdotBoundedData_mean = np.nanmean(logMdotBoundedData)
    logMdotBoundedData_std = np.nanstd(logMdotBoundedData)
    logMBoundedData = np.log10(star_data_inside_bound[MIndex])
    logLBoundedData = np.log10(star_data_inside_bound[LIndex])
    plot.
↪vlines([logMdotBoundedData_mean-delta_mdot_multiplier*logMdotBoundedData_std,logMdotBounded
↪nanmax(n_hist))
    ax.text(text_x, .
↪97,title_string,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
    logMdotBoundedData = np.log10(star_data_inside_bound[MdotIndex])
    logMdotBoundedData_mean = np.nanmean(logMdotBoundedData)
    logMdotBoundedData_std = np.nanstd(logMdotBoundedData)
    logMBoundedData = np.log10(star_data_inside_bound[MIndex])
    logLBoundedData = np.log10(star_data_inside_bound[LIndex])
    delta_logMdot = 2*delta_mdot_multiplier*logMdotBoundedData_std
    if type(std_multiplier) == int:
        plot_name =␣
↪'logMdot_histogram_'+title_string+'_stnd_dev_'+str(std_multiplier)
    elif type(std_multiplier) == float:
        plot_name =␣
↪'logMdot_histogram_'+title_string+'_stnd_dev_'+str(std_multiplier)
    else:
        plot_name = 'logMdot_histogram_'+title_string+'_stnd_dev_of_all'
```

```
    #plot.legend(loc=4)
    plot.savefig(plot_name+'.png')
    plot.savefig(plot_name+'.pdf') # Why do journals require this format...

    print(' logMdot =',delta_logMdot)
    # Calculate B = deltaLogMdot/deltaLogL and C = deltaLogMdot/deltaLogM
    B = delta_logMdot/delta_logL
    C = -delta_logMdot/delta_logM


    # From actually testing this method, we know there is a possibility of a␣
↪residual trend in log M or log L. We can correct for this
    # immediately by setting A = 0, calculating a residual, and finding the␣
↪slope (intercept is irrelevant) when plotted vs logM.
    if trend_correct ==True:
        A = 0.
        logMdot_guess = A + B*np.log10(star_data_stacked[LIndex]) + C*np.
↪log10(star_data_stacked[MIndex])
        resid = np.log10(star_data_stacked[MdotIndex]) - logMdot_guess
        trend = linear_fit_1var(np.log10(star_data_stacked[MIndex]),resid)[0][1]
        print('Old C:',C)
        print('Old C/New C:',C/(C+trend))
        C = C + trend
        print('New C:',C)
        trend = linear_fit_1var(np.log10(star_data_stacked[LIndex]),resid)[0][1]
        print('Old B:',B)
        print('Old B/New B:',B/(B+trend))
        B = B + trend
        print('New B:',B)
        new_delta_logL = delta_logMdot/B
        print('new_delta_logL',new_delta_logL)
        new_delta_logM = -delta_logMdot/C
        print('new_delta_logM',new_delta_logM)

    # Plot
    fig, ax = plt.subplots()
    for i in range(0,len(star_data)):
        plot.scatter(np.log10(star_data[i][LIndex]), np.
↪log10(star_data[i][MIndex]),s=2,label=spec_mode_label[i])
    plot.plot(line_x_vals,line_y_vals,linestyle='-',color='black')
    plot.plot(line_x_vals,line_y_vals_up,linestyle='--',color='black')
    plot.plot(line_x_vals,line_y_vals_down,linestyle='--',color='black')
    plot.xlabel(r'$\log L$')
    plot.ylabel(r'$\log M$')
    ax.text(text_x, .
↪97,title_string,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
```

```
    plot.vlines(np.average(line_x_vals),np.average(line_y_vals)-delta_logM/2,np.
↪average(line_y_vals)+delta_logM/2,color='tab:
↪green',linestyle='-',linewidth=4)
    plot.plot([logL_left,logL_right],[np.average(line_y_vals),np.
↪average(line_y_vals)],color='tab:purple',linewidth=4)
    if trend_correct ==True:
        plot.vlines(np.average(line_x_vals),np.
↪average(line_y_vals)-new_delta_logM/2,np.average(line_y_vals)+new_delta_logM/
↪2,color='tab:red',linestyle='--',linewidth=2)
        plot.plot([np.average(line_x_vals)-new_delta_logL/2,np.
↪average(line_x_vals)+new_delta_logL/2],[np.average(line_y_vals),np.
↪average(line_y_vals)],color='tab:orange',linewidth=2,linestyle='--')
    if type(std_multiplier) == int:
        plot_name = 'deathzone_'+title_string+'_stnd_dev_'+str(std_multiplier)
    elif type(std_multiplier) == float:
        plot_name = 'deathzone_'+title_string+'_stnd_dev_'+str(std_multiplier)
    else:
        plot_name = 'deathzone_'+title_string+'_stnd_dev_of_all'
    plot.legend(loc=4)
    plot.savefig(plot_name+'.png')
    plot.savefig(plot_name+'.pdf') # Why do journals require this format...
    plot.show()
    plot.close('all')
    # Determine A
    '''
    # Idea: Find all points within 0.0125335 std. devs. of deathline (the␣
↪closest 1% of stars), and assume they are
    # 'on' the deathline
    indices_on_DL = []
    for i in range (0, len(star_data[MdotIndex])):
        PredictedLogM = deathline_coeffs[0]+deathline_coeffs[1]*np.
↪log10(star_data[LIndex][i])
        if np.log10(star_data[MIndex][i]) < PredictedLogM + 0.
↪0125335*stnd_dev_around_DL:
            if np.log10(star_data[MIndex][i]) > PredictedLogM - 0.
↪0125335*stnd_dev_around_DL:
                indices_on_DL.append(i)
    print('Number of Stars within 1% of DL:', len(indices_on_DL))
    # Then, find their mass-loss rate.
    closest_logMdots = np.log10(star_data[MdotIndex][indices_on_DL])
    #print('DEBUG: closest_logMdots=',closest_logMdots)
    # Then, take that mass-loss rate and set equal to A+BlogL+ClogM, solve for␣
↪A.
    AList = closest_logMdots - B*np.log10(star_data[LIndex][indices_on_DL]) +␣
↪C*np.log10(star_data[MIndex][indices_on_DL])
    #print('DEBUG: AList=',AList)
```

```
    # Finally, average this value of logA over all included points
    A = np.mean(AList)
    ATestResults = A + B*np.log10(star_data[LIndex][indices_on_DL]) - C*np.
↪log10(star_data[MIndex][indices_on_DL])
    #print('DEBUG: ATestResults', ATestResults)
    '''
    '''
    # Idea:
    # We have B, C, logM = X + Y*logL, logMdot = A + BlogL + ClogM. Pick a␣
↪series of masses and determine log L.
    # Assuming logMdot = logMdotcrit, we have what we need to calculate A for a␣
↪given mass and luminosity.
    min_mass = np.nanmin(star_data[MIndex])
    max_mass = np.nanmax(star_data[MIndex])
    sample_logM = np.log10(np.arange(min_mass, max_mass, (max_mass-min_mass)/
↪200))
    sample_logL = (sample_logM - deathline_coeffs[0])/deathline_coeffs[1]
    sample_logMdot = np.log10((10**sample_logM)/(2*10**6))
    A_list = sample_logMdot - B*sample_logL - C*sample_logM
    A_fit = linear_fit_1var(sample_logM,A_list)
    #print('A_list:', A_list)
    print('A_fit: Intercept=',A_fit[0][0], ', Slope=',A_fit[0][1])
    A = np.mean(A_list)
    plot.scatter(sample_logM,A_list,s=2)
    plot.xlabel(r'$\log M$')
    plot.ylabel(r'$A$')
    plot.title(spec_mode_label)
    plot.show()
    plot.close('all')
    '''
    # Simpler, we can force A to be whatever is necessary for the mean residual␣
↪to be 0
    # <logMdot_obsv - logMdot_calc> = 0 = sum(logMdot_obsv[i] - (A + BlogL[i] +␣
↪ClogM[i]))/len(data)
    # => <A> = sum(logMdot_obsv[i] - BlogL[i] - ClogM[i])/len(data)
    A = 0
    for i in range(0, np.shape(star_data_stacked)[1]):
        if np.isnan(np.log10(star_data_stacked[MdotIndex][i]))==True:
            print('Index ',i,' skipped due to NaN in logMdot')
        elif np.isnan(np.log10(star_data_stacked[LIndex][i]))==True:
            print('Index ',i,' skipped due to NaN in logL')
        elif np.isnan(np.log10(star_data_stacked[MIndex][i]))==True:
            print('Index ',i,' skipped due to NaN in logM')
        else:
```

```
            A = A + np.log10(star_data_stacked[MdotIndex][i])-B*np.
↪log10(star_data_stacked[LIndex][i]) - C*np.
↪log10(star_data_stacked[MIndex][i])
    A = A/np.shape(star_data_stacked)[1]

    logMdot_predict = A + B*np.log10(star_data_stacked[LIndex]) + C*np.
↪log10(star_data_stacked[MIndex])
    correction = np.nanmean(np.
↪log10(star_data_stacked[MdotIndex])-logMdot_predict)
    print('Correction to A:',correction)
    A = A + correction

    # Check: Make histogram of log(Mdot)
    fig, ax = plt.subplots()
    logMdot_data = []
    for i in range (0, len(star_data)):
        logMdot_data.append(np.log10(star_data[i][MdotIndex]))
    plot.hist(logMdot_data,10,stacked=True)
    ax.text(text_x, .
↪97,title_string,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
    #plot.title(title_string)
    plot.show()
    plot_name = 'stacked_histogram_'+title_string
    plot.savefig(plot_name+'.png')
    plot.savefig(plot_name+'.pdf') # Why do journals require this format...
    plot.close('all')

    # Check: Make histogram of log(Mdot/Mdotcrit); this should be about 2 dex␣
↪wide and that will match the theoretical death zone width.
    fig, ax = plt.subplots()
    log_mdot_by_mdotcrit = []
    for i in range(0, len(star_data)):
        log_mdot_by_mdotcrit.append(np.log10(star_data[i][MdotIndex]/
↪(star_data[i][MIndex]/(2*10**6))))
    #log_mdot_by_mdotcrit = np.log10(star_data_stacked[MdotIndex]/
↪(star_data_stacked[MIndex]/(2*10**6)))
    plot.hist(log_mdot_by_mdotcrit,10,stacked=True)
    ax.text(text_x, .
↪97,title_string,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
    #plot.title(title_string)
    plot.show()
    plot.close('all')
```

```
    # Check: Make histogram of logL/logMdot and logM/logMdot



    # Check: Make a plot of our formula here (calculated data) vs. the observed␣
↪data, and compare to a 1-to-1 line
    our_log_mdot = A + B*np.log10(star_data_stacked[LIndex]) + C*np.
↪log10(star_data_stacked[MIndex])
    logMdotData = np.log10(star_data_stacked[MdotIndex])

    # Now an overlaid version
    # Type 1: Calc Mdot vs Obsv Mdot
    fig, ax = plt.subplots()
    alpha_set=0.33
    plot.scatter(logMdotData,our_log_mdot,s=2,label=r'$\log \dot{M}(\log L,␣
↪\log P)$',alpha=alpha_set,color='tab:blue')
    fit = linear_fit_1var(logMdotData,our_log_mdot)
    fit_params = fit[0]
    fit_x = np.asarray([-9,-3])
    fit_y = fit_params[0]+fit_params[1]*fit_x
    plot.plot(fit_x,fit_y,color ='tab:blue')

    #plot.title(spec_mode_label)
    ax.text(text_x, .
↪97,spec_mode_label,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
    plot.xlabel(r'$\log \dot{M}_{obsv.}$')
    plot.ylabel(r'$\log \dot{M}_{calc.}$')
    plot.plot([-9,-3],[-9,-3],color = 'black', alpha =␣
↪1,linestyle='--',linewidth=1.5)
    plot.xlim([-8,-4])
    plot.ylim([-8,-4])
    plot.tight_layout()
    plot.legend(loc=4)
    spec_mode_label_str = ''
    for i in range(0,len(spec_mode_label)):
        if len(spec_mode_label) == 2:
            spec_mode_label_str = spec_mode_label[0] + '_and_' +␣
↪spec_mode_label[1]
        else:
            if i == len(spec_mode_label)-1:
                spec_mode_label_str =␣
↪spec_mode_label_str+'and_'+spec_mode_label[i]
            elif i == 0:
                spec_mode_label_str = spec_mode_label[i]+'_'
            else:
```

```
            spec_mode_label_str = spec_mode_label_str + spec_mode_label[i]␣
↪+ '_'
    #plot.savefig('Final_Fit/Attenuation_Demonstration/
↪'+spec_mode_label_str+'_all_calc_vs_obsv'+'.pdf',format='pdf')
    #plot.savefig('Final_Fit/Attenuation_Demonstration/
↪'+spec_mode_label_str+'_all_calc_vs_obsv'+'.png',format='png')
    plot.show()
    plot.close()

    # Type 2: Residual vs Obsv Mdot
    fig, ax = plt.subplots()
    plot.scatter(logMdotData,our_log_mdot-logMdotData,s=2,label=r'$\log␣
↪\dot{M}(\log L, \log P)$',alpha=alpha_set,color='tab:blue')
    fit = linear_fit_1var(logMdotData,our_log_mdot-logMdotData)
    fit_params = fit[0]
    fit_x = np.asarray([-9,-3])
    fit_y = fit_params[0]+fit_params[1]*fit_x
    plot.plot(fit_x,fit_y,color = 'tab:blue')
    print('Mdot(L,P) Slope=',fit_params[1])
    print('Mdot(L,P) R^2=',fit[1])

    #plot.title(spec_mode_label)
    ax.text(text_x, .
↪97,spec_mode_label,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
    plot.xlabel(r'$\log \dot{M}_{obsv.}$')
    plot.ylabel(r'$\log \dot{M}_{calc.}-\log \dot{M}_{obsv.}$')
    plot.plot([-8,-4],[0,0],color = 'black', alpha =␣
↪1,linestyle='--',linewidth=1.5)
    plot.xlim([-8,-4])
    plot.tight_layout
    plot.legend(loc=4)
    #plot.savefig('Final_Fit/Attenuation_Demonstration/
↪'+spec_mode_label_str+'_all_resid_vs_obsv'+'.pdf',format='pdf')
    #plot.savefig('Final_Fit/Attenuation_Demonstration/
↪'+spec_mode_label_str+'_all_resid_vs_obsv'+'.png',format='png')
    plot.show()
    plot.close()


    print('For set '+title_string+': A=',A,', B=', B,', C=', -C)
    print('_____ _____ _____ _____ _____')
    return [A, B, C], star_data_inside_bound
```

[335]:
```
FWHM = 2*np.sqrt(2*np.log(2))
```

```
#deathzone_values_M0, M0_in_bound = deathline_analysis(M0StarData,␣
↪spec_mode_label = 'M0',std_multiplier=2,delta_mdot_multiplier=2.
↪75,trend_correct=True)
deathzone_values_M0 = M0_optimal_exponents
```

[336]:
```
#deathzone_values_M1, M1_in_bound = deathline_analysis(M1StarData,␣
↪spec_mode_label = 'M1',std_multiplier=2,delta_mdot_multiplier=2.
↪75,trend_correct=True)
deathzone_values_M1 = M1_optimal_exponents
```

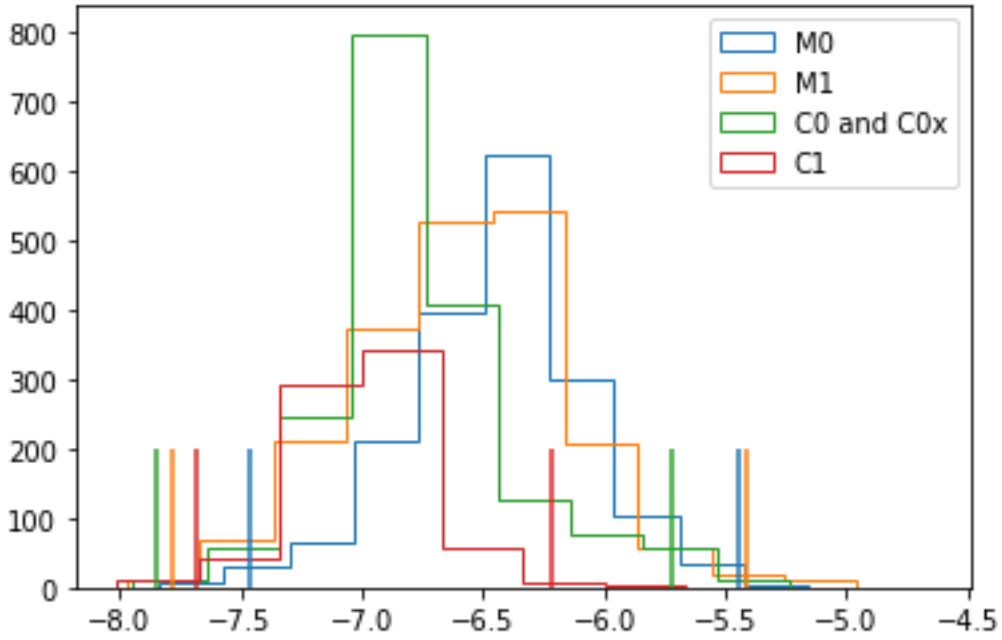
[337]:
```
#print('sigma=',FWHM)
#deathzone_values_C1, C1_in_bound = deathline_analysis(C1StarData,␣
↪spec_mode_label = 'C1',std_multiplier=2,delta_mdot_multiplier=2.
↪75,trend_correct=True)
deathzone_values_C1 = C1_optimal_exponents
```

[338]:
```
#deathzone_values_C0_and_C0x, C0_and_C0x_in_bound =␣
↪deathline_analysis_multidata([C0StarData_n, C0StarData_x], spec_mode_label =␣
↪['C0', 'C0x'],std_multiplier=2,delta_mdot_multiplier=2.
↪75,trend_correct=True,text_x=0.09)
deathzone_values_C0_and_C0x = C0_optimal_exponents
#deathzone_values_M0_and_M1 = deathline_analysis_multidata([M0StarData,␣
↪M1StarData], spec_mode_label = ['M0', 'M1'],std_multiplier=2)
```

[339]:
```
plot.hist(np.log10(M0_in_bound[MdotIndex]),histtype='step',color='tab:
↪blue',label='M0')
plot.vlines([np.nanmean(np.log10(M0_in_bound[MdotIndex]))-2.75*np.nanstd(np.
↪log10(M0_in_bound[MdotIndex])),np.nanmean(np.
↪log10(M0_in_bound[MdotIndex]))+2.75*np.nanstd(np.
↪log10(M0_in_bound[MdotIndex]))],0,200,color='tab:blue')
plot.hist(np.log10(M1_in_bound[MdotIndex]),histtype='step',color='tab:
↪orange',label='M1')
plot.vlines([np.nanmean(np.log10(M1_in_bound[MdotIndex]))-2.75*np.nanstd(np.
↪log10(M1_in_bound[MdotIndex])),np.nanmean(np.
↪log10(M1_in_bound[MdotIndex]))+2.75*np.nanstd(np.
↪log10(M1_in_bound[MdotIndex]))],0,200,color='tab:orange')
plot.hist(np.log10(C0_and_C0x_in_bound[MdotIndex]),histtype='step',color='tab:
↪green',label='C0 and C0x')
plot.vlines([np.nanmean(np.log10(C0_and_C0x_in_bound[MdotIndex]))-2.75*np.
↪nanstd(np.log10(C0_and_C0x_in_bound[MdotIndex])),np.nanmean(np.
↪log10(C0_and_C0x_in_bound[MdotIndex]))+2.75*np.nanstd(np.
↪log10(C0_and_C0x_in_bound[MdotIndex]))],0,200,color='tab:green')
plot.hist(np.log10(C1_in_bound[MdotIndex]),histtype='step',color='tab:
↪red',label='C1')
```

```
plot.vlines([np.nanmean(np.log10(C1_in_bound[MdotIndex]))-2.75*np.nanstd(np.
 ↪log10(C1_in_bound[MdotIndex])),np.nanmean(np.
 ↪log10(C1_in_bound[MdotIndex]))+2.75*np.nanstd(np.
 ↪log10(C1_in_bound[MdotIndex]))],0,200,color='tab:red')
plot.legend(loc=1)
```

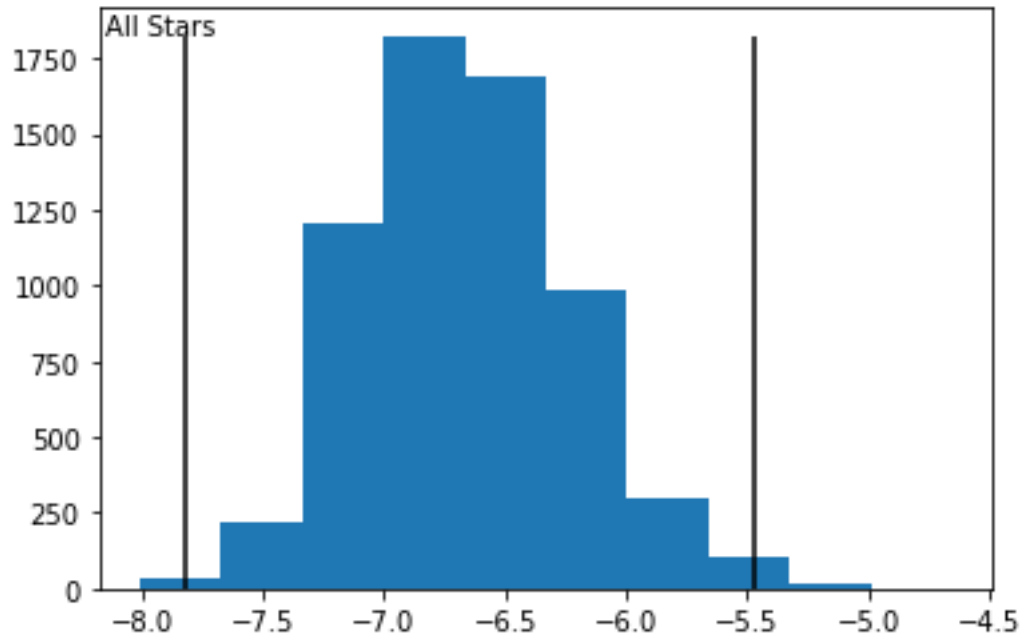
[339]: <matplotlib.legend.Legend at 0x223a869edc8>

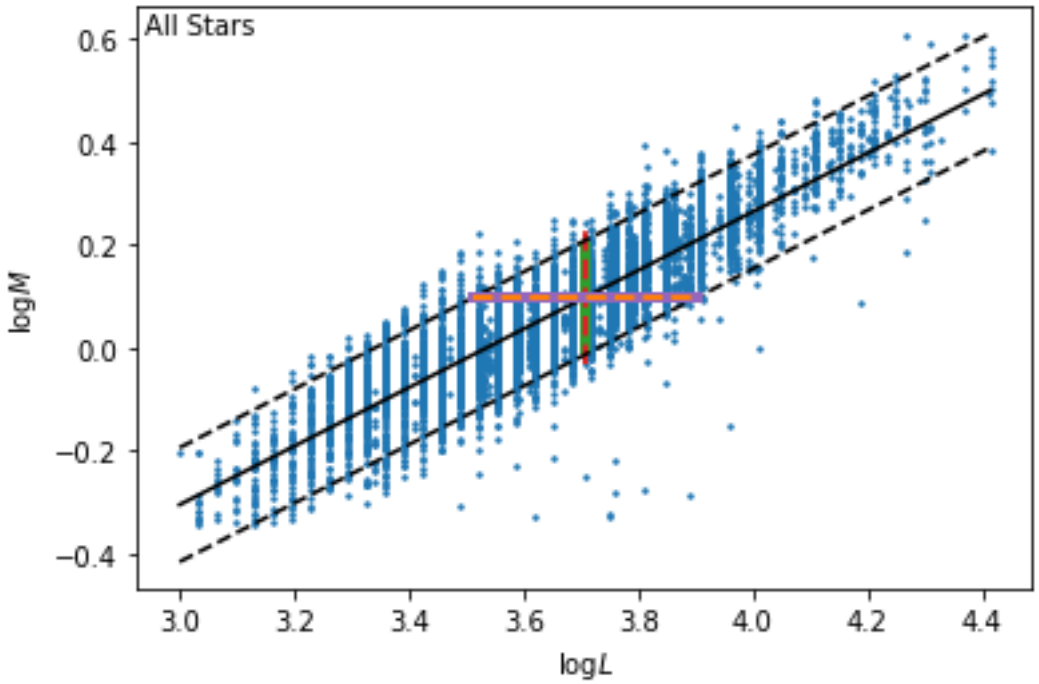


[340]:
```
combined_data_set = np.hstack([M0StarData[0:28],M1StarData[0:
 ↪28],C0StarData_n,C1StarData, C0StarData_x])
deathzone_values_All = deathline_analysis(combined_data_set, spec_mode_label =␣
 ↪'All Stars',std_multiplier=2,delta_mdot_multiplier=2.
 ↪75,trend_correct=True,text_x=0.07)
```

```
sigmaDL= 0.06381699617454836
Bound Size (logM): 0.1109088097393586
 logL = 0.3894874714313086
 logM = 0.22181761947871714
 logMdot = 2.3466640996111767
Old C: -10.579250219734385
Old C/New C: 1.1753115027498777
New C: -9.001230903451638
Old B: 6.025005351231286
Old B/New B: 0.9973651851469224
New B: 6.040922062407602
new_delta_logL 0.3884612440564937
```

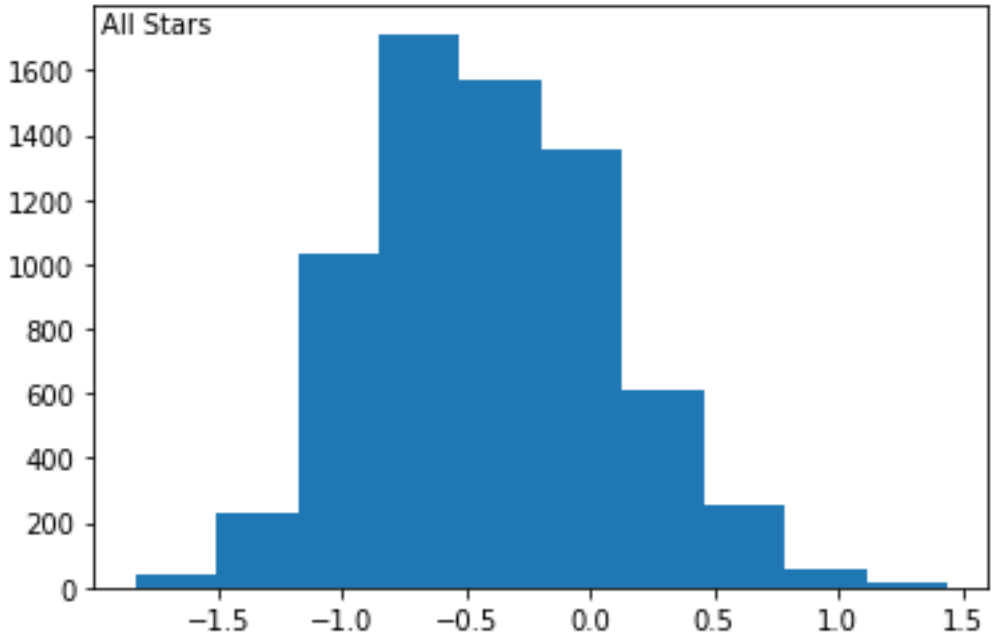
```
new_delta_logM 0.26070479968593163
```

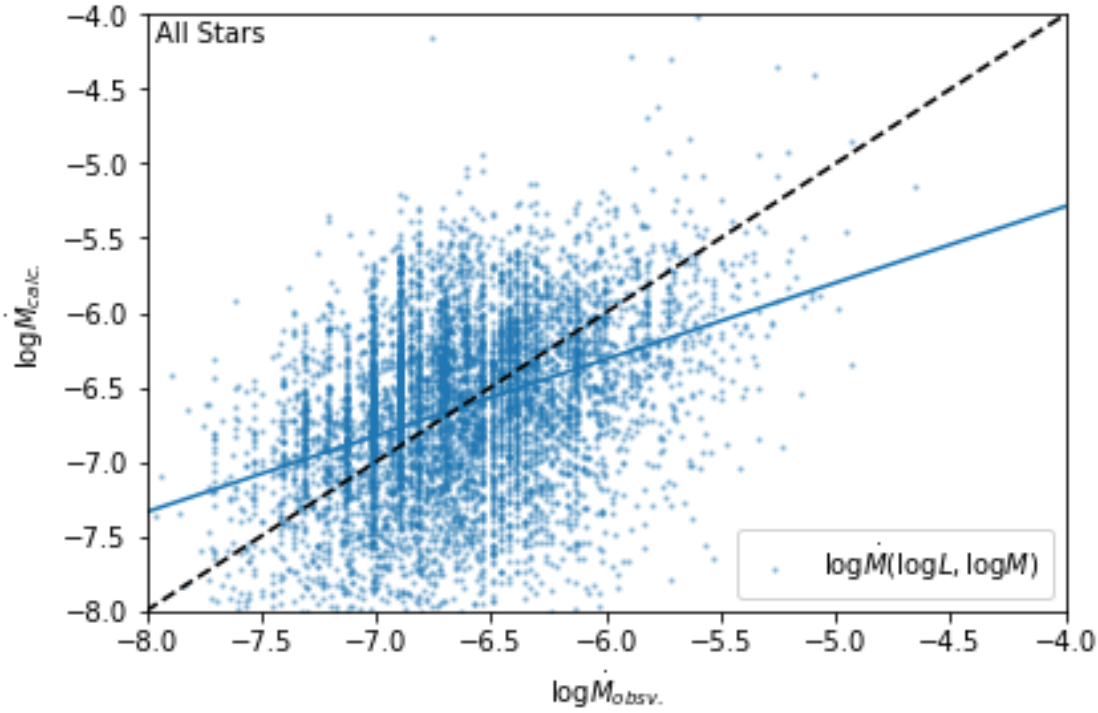




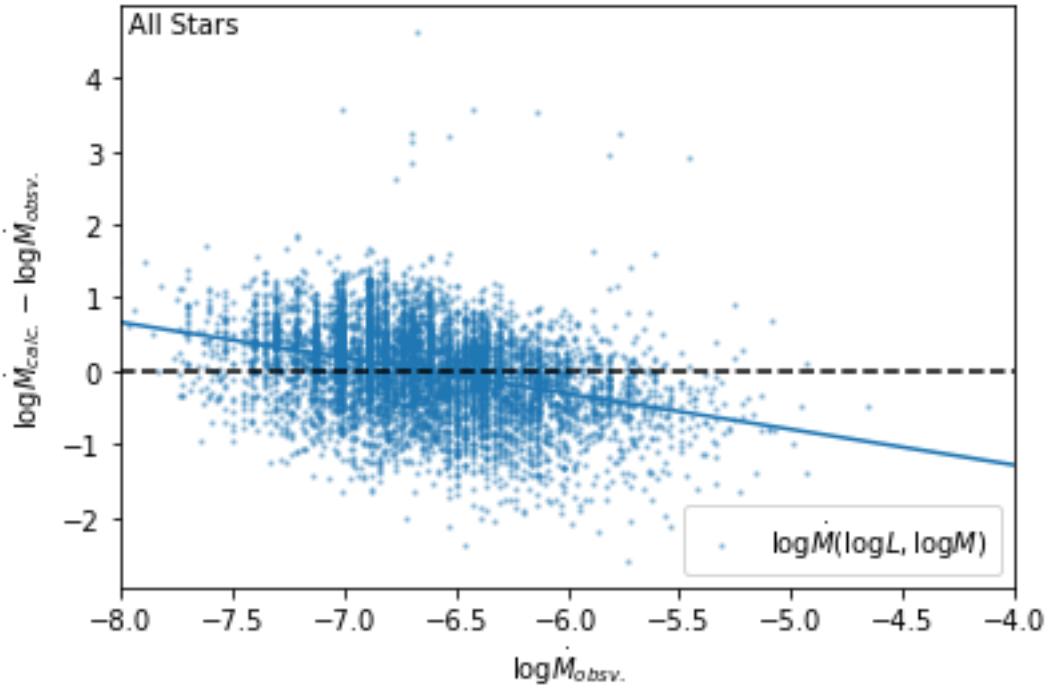

```
Index  4113  skipped due to NaN in logMdot
Correction to A: -0.004081449490800517
```

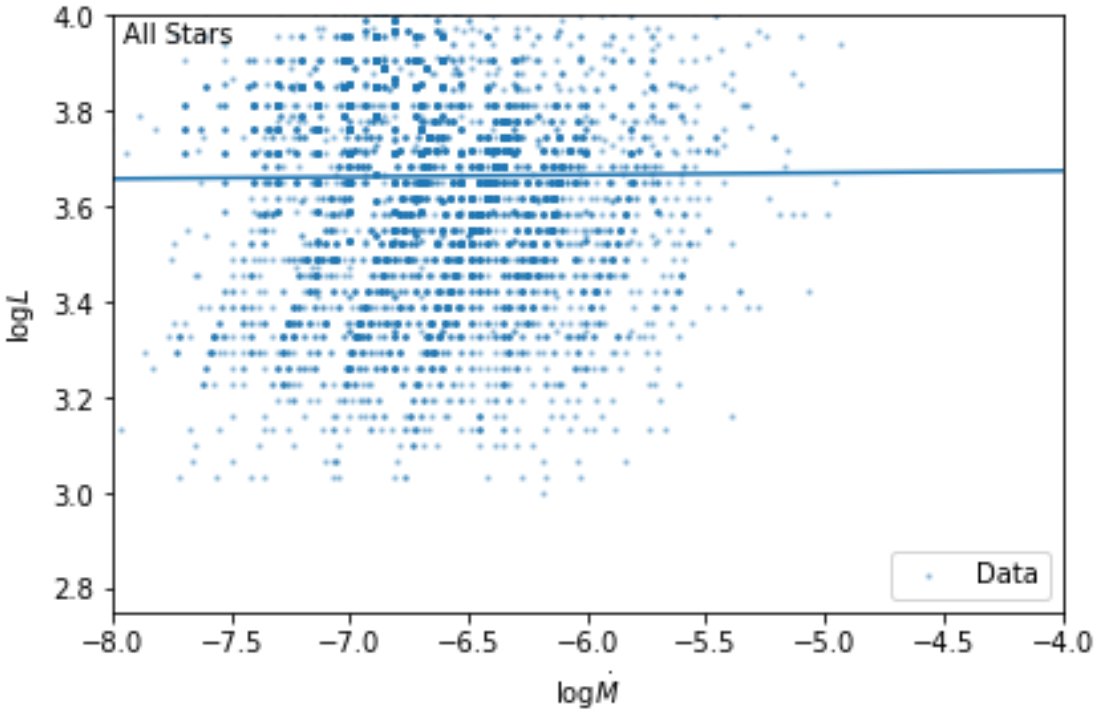
```
C:\ProgramData\Anaconda3\lib\site-packages\numpy\lib\histograms.py:839:
RuntimeWarning: invalid value encountered in greater_equal
  keep = (tmp_a >= first_edge)
C:\ProgramData\Anaconda3\lib\site-packages\numpy\lib\histograms.py:840:
RuntimeWarning: invalid value encountered in less_equal
  keep &= (tmp_a <= last_edge)
```

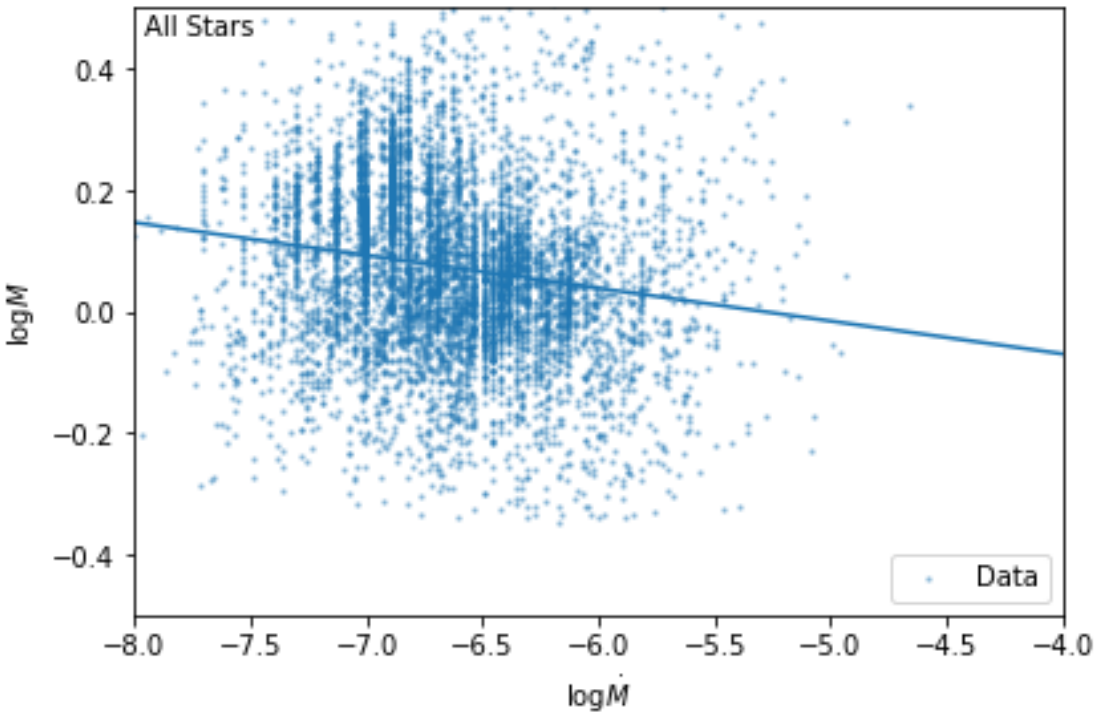





```
Mdot(L,P) Slope= -0.4874006663579684
Mdot(L,P) R^2= 0.12427710208402021
```




All Stars
4.0
3.8
3.6
3.4
3.2
3.0
2.8
log$L$
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
log$\dot{M}$
Data


All Stars
0.4
0.2
0.0
−0.2
−0.4
log$M$
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
log$\dot{M}$
Data

```
For set All Stars: A= -28.11710553716294 , B= 6.040922062407602 , C=
9.001230903451638

----- ----- ----- ----- -----
```

[341]:
```
deathzone_values_All_legend = deathline_analysis_multidata([M0StarData,␣
↪M1StarData, C0StarData_n, C0StarData_x, C1StarData],
                                                           spec_mode_label =␣
↪['M0', 'M1', 'C0', 'C0x', 'C1'],std_multiplier=2,md
                                                          text_x=0.19)
```

```
  File "<ipython-input-341-f537e858faa1>", line 3
    text_x=0.19)
         ^
SyntaxError: invalid syntax
```

combined_data_std = np.nanstd(np.log10(combined_data_set[MIndex]))

M0_critzone_M_std = np.nanstd(np.log10(M0StarData_critzone[MIndex])) M1_critzone_M_std = np.nanstd(np.log10(M1StarData_critzone[MIndex])) C0_critzone_M_std = np.nanstd(np.log10(C0StarData_critzone[MIndex])) C1_critzone_M_std = np.nanstd(np.log10(C1StarData_critzone[MIndex])) C0x_critzone_M_std = np.nanstd(np.log10(C0xStarData_critzone[MIndex]))

deathline_analysis(M0StarData_critzone, spec_mode_label = 'M0',std_multiplier=combined_data_std/M0_cr
deathline_analysis(M1StarData_critzone, spec_mode_label = 'M1',std_multiplier=combined_data_std/M1_cr
deathline_analysis(C0StarData_critzone, spec_mode_label = 'C0',std_multiplier=combined_data_std/C0_crit
deathline_analysis(C1StarData_critzone, spec_mode_label = 'C1',std_multiplier=combined_data_std/C1_crit
deathline_analysis(C0xStarData_critzone, spec_mode_label = 'C0x',std_multiplier=combined_data_std/C0x_

deathline_analysis(M0StarData_critzone, spec_mode_label = 'M0',std_multiplier=2*combined_data_std/M0_c*
*deathline_analysis(M1StarData_critzone, spec_mode_label = 'M1',std_multiplier=2*combined_data_std/M1_cr
deathline_analysis(C0StarData_critzone, spec_mode_label = 'C0',std_multiplier=2*combined_data_std/C0_cri*
*deathline_analysis(C1StarData_critzone, spec_mode_label = 'C1',std_multiplier=2*combined_data_std/C1_crit
deathline_analysis(C0xStarData_critzone, spec_mode_label = 'C0x',std_multiplier=2*combined_data_std/C0

deathline_analysis_with_errors(M0StarData_critzone, spec_mode_label = 'M0') deathline_analysis_with_errors(M1StarData_critzone, spec_mode_label = 'M1') deathline_analysis_with_errors(C0StarData_critzone, spec_mode_label = 'C0') deathline_analysis_with_errors(C1StarData_critzone, spec_mode_label = 'C1') deathline_analysis_with_errors(C0xStarData_critzone, spec_mode_label = 'C0x') deathline_analysis_with_errors(combined_data_set, spec_mode_label = 'All Stars')

[342]:
```
def mdot_vs_mdot_dz (star_data, deathzone_exponents, spec_mode_label = ''):
    mdot_data = np.log10(star_data[MdotIndex])
```

```
    predict_log_mdot = deathzone_exponents[0] + deathzone_exponents[1]*np.
↪log10(star_data[LIndex]) - deathzone_exponents[2]*np.log10(star_data[MIndex])
    plot.scatter(predict_log_mdot,mdot_data,s=2)

    limits = np.asarray([np.nanmin([np.nanmin(predict_log_mdot),np.
↪nanmin(mdot_data)]), np.nanmax([np.nanmax(predict_log_mdot),np.
↪nanmax(mdot_data)])])
    min_val = limits[0]
    max_val = limits[1]
    form_vs_fed_fit = linear_fit_1var(predict_log_mdot,mdot_data)
    form_vs_fed_Coeffs = form_vs_fed_fit[0]
    quality_line_x = np.asarray([min_val,max_val])
    quality_line_y = form_vs_fed_Coeffs[0] +␣
↪form_vs_fed_Coeffs[1]*quality_line_x
    plot.plot(quality_line_x,quality_line_y,color='orange')
    #print('limits',limits)
    plot.
↪plot([limits[0],limits[1]],[limits[0],limits[1]],color='black',linestyle='-.
↪')
    plot.xlim(limits[0]-0.1,limits[1]+0.1)
    plot.ylim(limits[0]-0.1,limits[1]+0.1)
    plot.tight_layout()
    plot.xlabel(r'$\log \dot{M}$'+' - Formula')
    plot.ylabel(r'$\log \dot{M}$'+' - GRAMS')
    plot.title(spec_mode_label)
```

mdot_vs_mdot_dz (M0StarData, deathzone_values_M0, spec_mode_label = 'M0')

[343]:
```
def get_parallelogram_points (obsv_log_mdot_data, residual_data):
    # First, do a linear fit of given data
    form_vs_fed_fit = linear_fit_1var(obsv_log_mdot_data,residual_data) #␣
↪Returns [(b=intercept, m=slope), r^2] for equation x = my + b
    form_vs_fed_Coeffs = form_vs_fed_fit[0] # Get Coeffs from above

    # Second, find two standard deviations away from fit line
    np.nanstd(residual_data)
    stnd_dev_around_fit = np.
↪nanstd(residual_data-form_vs_fed_Coeffs[0]-form_vs_fed_Coeffs[1]*obsv_log_mdot_data)
    bound_size = 2*stnd_dev_around_fit*np.cos(np.arctan(form_vs_fed_Coeffs[1]))

    # Third, find upper and lower bounding line equations
    upper_bound_coeffs = form_vs_fed_Coeffs + np.asarray([bound_size,0])
    lower_bound_coeffs = form_vs_fed_Coeffs - np.asarray([bound_size,0])
    # Fourth, find left-most and right-most log Mdots (these are edges)
    left_bound = np.nanmin(obsv_log_mdot_data)
    right_bound = np.nanmax(obsv_log_mdot_data)
```

```
    # Fifth, use edges to find the 4 points on bounding lines
    point_list = []
    point_list.
↪append([left_bound,upper_bound_coeffs[0]+upper_bound_coeffs[1]*left_bound])
    point_list.
↪append([left_bound,lower_bound_coeffs[0]+lower_bound_coeffs[1]*left_bound])
    point_list.
↪append([right_bound,lower_bound_coeffs[0]+lower_bound_coeffs[1]*right_bound])
    point_list.
↪append([right_bound,upper_bound_coeffs[0]+upper_bound_coeffs[1]*right_bound])
    print('Point List:', point_list)
    # Return array ready for pyplot.fill_between
    x_values = [left_bound,right_bound]
    y_values =␣
↪[[upper_bound_coeffs[0]+upper_bound_coeffs[1]*left_bound,upper_bound_coeffs[0]+upper_bound_
                ␣
↪[lower_bound_coeffs[0]+lower_bound_coeffs[1]*left_bound,lower_bound_coeffs[0]+lower_bound_c

    print('x-values:',np.asarray(x_values),'\n y-values:', np.asarray(y_values))
    return np.asarray(x_values), np.asarray(y_values)
```

[378]:
```
def mdot_formula_comparison (star_data, spec_mode_label = '',marker_size=2):
    # Use artificial star data to create plots demonstrating how our formulas␣
↪differ from eachother and other notable formulas
    logLData = np.log10(star_data[LIndex])
    logMData = np.log10(star_data[MIndex])
    logPData = np.log10(star_data[PIndex])
    logRData = np.log10(star_data[RIndex])
    logTData = np.log10(star_data[TeffCalcIndex])
    logMdotData = np.log10(star_data[MdotIndex])


    color_list = ['tab:blue','tab:orange','tab:green','tab:red','tab:
↪purple','tab:brown', 'tab:pink', 'tab:gray', 'tab:olive', 'tab:cyan']

    LPmultilinear_fit = multilinear_fit_2var([logLData,logPData],logMdotData)
    LPCoeffs = LPmultilinear_fit[0]
    our_log_mdot = LPCoeffs[0] + LPCoeffs[1]*logLData + LPCoeffs[2]*logPData
    # log Mdot = A + BlogL + ClogP
    if spec_mode_label[0] == 'M':
        RLM = RLM_M
    if spec_mode_label[0] == 'C':
        RLM = RLM_C
    if spec_mode_label == 'M0':
        PMR = PMR_M0
        dz_values = deathzone_values_M0
```

```
    if spec_mode_label == 'M1':
        PMR = PMR_M1
        dz_values = deathzone_values_M1
    if spec_mode_label == 'C0':
        PMR = PMR_C0
        dz_values = deathzone_values_C0
    if spec_mode_label == 'C0x':
        PMR = PMR_C0
        dz_values = deathzone_values_C0x
    if spec_mode_label == 'C0 and C0x':
        PMR = PMR_C0
        dz_values = deathzone_values_C0_and_C0x
    if spec_mode_label == 'C1':
        PMR = PMR_C1
        dz_values = deathzone_values_C1
    if spec_mode_label == 'C0 and C0x':
        PMR = PMR_C0
        dz_values = deathzone_values_C0_and_C0x

    #artificial_data = make_artificial_set([star_data],0.2,grid_spacing = 75j)
    #artificial_logLData = artificial_data[0]
    #artificial_logMData = artificial_data[1]
    #artificial_logRData = RLM[0] + RLM[1]*artificial_logLData +␣
↪RLM[2]*artificial_logMData
    #artificial_logPData = PMR[0] + PMR[1]*artificial_logMData +␣
↪PMR[2]*artificial_logRData
    #artificial_logTData = np.log10(5778*np.sqrt(10**(artificial_logRData)/np.
↪sqrt(10**artificial_logLData)))

    a = LPCoeffs[0]
    arlm = RLM[0]
    aRLM = arlm
    apmr = PMR[0]
    aPMR = apmr
    c = LPCoeffs[2]
    cpmr = PMR[2]
    cPMR = cpmr
    b = LPCoeffs[1]
    brlm = RLM[1]
    bRLM = brlm
    bpmr = PMR[1]
    bPMR = bpmr
    crlm = RLM[2]
    cRLM = crlm
    PMLCoeffs = calculate_PML_coeffs(RLM, PMR)
    aPML = PMLCoeffs[0]
    bPML = PMLCoeffs[1]
```

```
    cPML = PMLCoeffs[2]

    Anew = a - arlm + apmr*c + arlm*c*cpmr
    Bnew = b - brlm + brlm*c*cpmr
    Cnew = bpmr*c - crlm + c*cpmr*crlm
    print('log Mdot ~ logL + logP:')
    print('LRM form: logMdot =',Anew,' + ',Bnew,'logL + logR -',-Cnew,'logM')
    print('LM form: logMdot =',Anew+arlm,' + ',Bnew+brlm,'logL␣
↪-',-(Cnew+crlm),'logM')

    print('_______________')

    print('log Mdot ~ logL + logM')
    print('LM form: logMdot =',dz_values[0],' + ',dz_values[1],'logL -',np.
↪abs(dz_values[2]),'logM')

    print('_______________')


    Reimers_log_mdot = np.log10(4*10**-13) + logLData + logRData - logMData
    # Mdot = (4*10^-13) * eta * LR/M => logMdot = log(4*10^-13) + log(eta) +␣
↪logL + logR - logM
    # 1975
    print('Reimers 1975 LRM form: logMdot =',np.log10(4*10**-13),'+ logL + logR␣
↪- logM')
    Aprint = np.log10(4*10**-13) + aRLM
    Bprint = 1 + bRLM
    Cprint = -1 + cRLM
    print('Reimers 1975 LM form: logMdot =', Aprint,'+',Bprint,'logL - ',np.
↪abs(Cprint),'logM')
    fit = linear_fit_1var(logMdotData,Reimers_log_mdot)
    print('Calc vs Obsv Slope:', fit[0][1])
    print('Calc vs Obsv R^2:',fit[1])
    print('SDev of Residual:',np.nanstd(Reimers_log_mdot-logMdotData))
    print('Mean distance from 1-to-1 line:',np.nanmean(np.
↪abs(Reimers_log_mdot-logMdotData)))

    print('_______________')

    Reimers_log_mdot_2 = Reimers_log_mdot +np.log10(0.35)
    # Mdot = (4*10^-13) * 0.35 * LR/M => logMdot = log(4*10^-13) + log(0.35) +␣
↪logL + logR - logM
    # http://cdsads.u-strasbg.fr/pdf/1977A%26A....61..217R, 1977
    print('Reimers 1977 LRM form: logMdot =',np.log10(4*10**-13)+np.log10(0.
↪35),'+ logL + logR - logM')
    Aprint = np.log10(4*10**-13)+np.log10(0.35) + aRLM
```

```
    Bprint = 1 + bRLM
    Cprint = -1 + cRLM
    print('Reimers 1977 LM form: logMdot =', Aprint,'+',Bprint,'logL - ',np.
↪abs(Cprint),'logM')
    fit = linear_fit_1var(logMdotData,Reimers_log_mdot_2)
    print('Calc vs Obsv Slope:', fit[0][1])
    print('Calc vs Obsv R^2:',fit[1])
    print('SDev of Residual:',np.nanstd(Reimers_log_mdot_2-logMdotData))
    print('Mean distance from 1-to-1 line:',np.nanmean(np.
↪abs(Reimers_log_mdot_2-logMdotData)))

    print('_______________')

    VW_log_mdot = calc_vw_mdot(logPData, logMData)
    # log Mdot = -11.4 + 0.0125(P-100(M-2.5))
    # http://adsabs.harvard.edu/abs/1993ApJ...413..641V, 1993
    apprx_fit = multilinear_fit_2var([logLData,logMData],VW_log_mdot)
    apprx_fit_coeffs = apprx_fit[0]
    print('VW LRM form: N/A')
    if apprx_fit_coeffs[2]>0:
        print('VW LM form apprx.:
↪',apprx_fit_coeffs[0],'+',apprx_fit_coeffs[1],'logL','+',apprx_fit_coeffs[2],'logM')
    else:
        print('VW LM form apprx.:
↪',apprx_fit_coeffs[0],'+',apprx_fit_coeffs[1],'logL','-',np.
↪abs(apprx_fit_coeffs[2]),'logM')
    fit = linear_fit_1var(logMdotData,VW_log_mdot)
    print('Calc vs Obsv Slope:', fit[0][1])
    print('Calc vs Obsv R^2:',fit[1])
    print('SDev of Residual:',np.nanstd(VW_log_mdot-logMdotData))
    print('Mean distance from 1-to-1 line:',np.nanmean(np.
↪abs(VW_log_mdot-logMdotData)))

    print('_______________')

    Bloecker_log_mdot = np.log10(1.932*10**-21) + 3.7*logLData + logRData - 3.
↪1*logMData
    # Mdot = (4.83*10^-9) (M^-2.1)(L^2.7)(Mdot_R=(4*10^-13) * eta * LR/M) = (1.
↪932*10^-21)(L^3.7)(R)(M^-3.1)
    # -> logMdot = log[1.932*10^-21] + 3.7logL + logR - 3.1logM
    # https://ui.adsabs.harvard.edu/abs/1995A&A...297..727B, 1995
    print('Bloecker LRM form: logMdot =',np.log10(1.932*10**-21),'+3.7logL +␣
↪logR - 3.1logM')
    Aprint = np.log10(1.932*10**-21) + aRLM
    Bprint = 3.7 + bRLM
    Cprint = -3.1 + cRLM
```

```
    print('Bloecker LM form: logMdot =',Aprint,'+',Bprint,'logL - ',np.
↪abs(Cprint), 'logM')
    fit = linear_fit_1var(logMdotData,Bloecker_log_mdot)
    print('Calc vs Obsv Slope:', fit[0][1])
    print('Calc vs Obsv R^2:',fit[1])
    print('SDev of Residual:',np.nanstd(Bloecker_log_mdot-logMdotData))
    print('Mean distance from 1-to-1 line:',np.nanmean(np.
↪abs(Bloecker_log_mdot-logMdotData)))

    print('_______________')

    Wachter_log_mdot = 9.84423 + 1.54*logLData - 1.95*logMData + 0.959*logPData␣
↪- 6.8*logTData
    # logMdot = -4.52 - 6.81(logT/2600 K) + 1.54log(L/10^4 Lsun) - 1.95log(M/
↪Msun)+0.959log(P/650d)
    # -> logMdot = 9.84423 + 1.54logL -1.95logM + 0.959logP - 6.8logT
    # https://ui.adsabs.harvard.edu/abs/2002A&A...384..452W, 2002
    # from definition of effective temp., logT = 0.940482 + 0.25logL - 0.5logR␣
↪(in solar units and Tsun = 5780K)
    WachterA = -6.39528 + 9.84423 + 0.959*aPML + 2.4*aRLM
    WachterB = -1.7 + 1.54 + 0.959*bPML+2.4*bRLM
    WachterC = -1.95 + 0.959*cPML+2.4*cRLM
    if WachterC <0:
        print('Wachter LRM form: logMdot =',WachterA,' + ',WachterB,'logL -␣
↪',np.abs(WachterC),'logM + logR')
    else:
        print('Wachter LRM form: logMdot =',WachterA,' + ',WachterB,'logL +␣
↪',WachterC,'logM + logR')
    Aprint = WachterA + aRLM
    Bprint = WachterB + bRLM
    Cprint = WachterC + cRLM
    if Cprint < 0:
        print('Wachter LM form: logMdot =',Aprint,' + ',Bprint,'logL - ',np.
↪abs(Cprint),'logM')
    else:
        print('Wachter LM form: logMdot =',Aprint,' + ',Bprint,'logL +␣
↪',Cprint,'logM')
    fit = linear_fit_1var(logMdotData,Wachter_log_mdot)
    print('Calc vs Obsv Slope:', fit[0][1])
    print('Calc vs Obsv R^2:',fit[1])
    print('SDev of Residual:',np.nanstd(Wachter_log_mdot-logMdotData))
    print('Mean distance from 1-to-1 line:',np.nanmean(np.
↪abs(Wachter_log_mdot-logMdotData)))

    print('_______________')
```

```
    van_Loon_log_mdot = 12.4776 + 1.05*logLData - 6.3*logTData
    # logMdot = -5.65 + 1.05 log(L/10000Lsun) - 6.3log(T/3500K)
    # -> logMdot = 12.4776 + 1.05logL - 6.3logT
    vanLoonA = -5.92504 + 12.4776 + 2.15*aRLM
    vanLoonB = -1.575+1.05+2.15*bRLM
    vanLoonC = 2.15*cRLM
    if vanLoonC <0:
        print('van Loon LRM form: logMdot =',vanLoonA,' + ',vanLoonB,'logL␣
↪-',np.abs(vanLoonC),'logM + logR')
    else:
        print('van Loon LRM form: logMdot =',vanLoonA,' + ',vanLoonB,'logL␣
↪+',vanLoonC,'logM + logR')
    Aprint = vanLoonA + aRLM
    Bprint = vanLoonB + bRLM
    Cprint = vanLoonC + cRLM
    if Cprint <0:
        print('van Loon LM form: logMdot =',Aprint,' + ',Bprint,'logL -',np.
↪abs(Cprint),'logM')
    else:
        print('van Loon LM form: logMdot =',Aprint,' + ',Bprint,'logL␣
↪+',Cprint,'logM')
    fit = linear_fit_1var(logMdotData,van_Loon_log_mdot)
    print('Calc vs Obsv Slope:', fit[0][1])
    print('Calc vs Obsv R^2:',fit[1])
    print('SDev of Residual:',np.nanstd(van_Loon_log_mdot-logMdotData))
    print('Mean distance from 1-to-1 line:',np.nanmean(np.
↪abs(van_Loon_log_mdot-logMdotData)))

    print('_____________')

    g = 10**logMData/(10**logRData)**2
    Schroeder_Cuntz_log_mdot = np.log10(0.2*10**Reimers_log_mdot*(10**logTData/
↪4000)**3.5*(1+1/(4300*g)))
    # Mdot = (Mdot_R=(4*10^-13) * eta * LR/M)(T/4000 K)^3.5 (1 + g_sun/
↪4300g_star)
    # http://adsabs.harvard.edu/abs/2005ApJ...630L..73S, 2005
    apprx_fit =␣
↪multilinear_fit_2var([logLData,logMData],Schroeder_Cuntz_log_mdot)
    apprx_fit_coeffs = apprx_fit[0]
    print('Schroeder and Cuntz LRM form: N/A')
    if apprx_fit_coeffs[2]>0:
        print('Schroeder and Cuntz LM form apprx.:
↪',apprx_fit_coeffs[0],'+',apprx_fit_coeffs[1],'logL','+',apprx_fit_coeffs[2],'logM')
    else:
```

```
        print('Schroeder and Cuntz LM form apprx.:
↪',apprx_fit_coeffs[0],'+',apprx_fit_coeffs[1],'logL','-',np.
↪abs(apprx_fit_coeffs[2]),'logM')

    fit = linear_fit_1var(logMdotData,Schroeder_Cuntz_log_mdot)
    print('Calc vs Obsv Slope:', fit[0][1])
    print('Calc vs Obsv R^2:',fit[1])
    print('SDev of Residual:',np.nanstd(Schroeder_Cuntz_log_mdot-logMdotData))
    print('Mean distance from 1-to-1 line:',np.nanmean(np.
↪abs(Schroeder_Cuntz_log_mdot-logMdotData)))
    print('_______________')

    de_Beck_log_mdot = calc_de_beck_mdot(logPData)
    # log Mdot = -7.37 + 3.42*10^-3 * P (P<850)
    #          = -4.46                  (P>850)
    # https://ui.adsabs.harvard.edu/abs/2010A&A...523A..18D, 2010
    apprx_fit = multilinear_fit_2var([logLData,logMData],de_Beck_log_mdot)
    apprx_fit_coeffs = apprx_fit[0]
    print('de Beck LRM form: N/A')
    if apprx_fit_coeffs[2]>0:
        print('de Beck LM form apprx.:
↪',apprx_fit_coeffs[0],'+',apprx_fit_coeffs[1],'logL','+',apprx_fit_coeffs[2],'logM')
    else:
        print('de Beck LM form apprx.:
↪',apprx_fit_coeffs[0],'+',apprx_fit_coeffs[1],'logL','-',np.
↪abs(apprx_fit_coeffs[2]),'logM')

    fit = linear_fit_1var(logMdotData,de_Beck_log_mdot)
    print('Calc vs Obsv Slope:', fit[0][1])
    print('Calc vs Obsv R^2:',fit[1])
    print('SDev of Residual:',np.nanstd(de_Beck_log_mdot-logMdotData))
    print('Mean distance from 1-to-1 line:',np.nanmean(np.
↪abs(de_Beck_log_mdot-logMdotData)))
    print('_______________')

    obsv_data_range = np.asarray([-9.5,-3.5])

    others_lw = 3
    others_alpha = 0.5

    # Type 1: Obsv vs. Calc

    form_vs_fed_fit = linear_fit_1var(our_log_mdot,logMdotData)
    # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
    form_vs_fed_Coeffs = form_vs_fed_fit[0]
    #x_data = obsv_data_range
    #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
```

```
    x_data = obsv_data_range
    y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
    plot.scatter(our_log_mdot,logMdotData,s=marker_size, label='Multilinear␣
↪Fit', color=color_list[0])
    #plot.plot(y_data,x_data, label='Multilinear Fit',linewidth=4)

    dz_log_mdot = dz_values[0] + dz_values[1]*np.log10(star_data[LIndex]) +␣
↪dz_values[2]*np.log10(star_data[MIndex])
    #print(len(dz_log_mdot),len(logMdotData))
    form_vs_fed_fit = linear_fit_1var(dz_log_mdot,logMdotData)
    # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
    form_vs_fed_Coeffs = form_vs_fed_fit[0]
    #x_data = obsv_data_range
    #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
    x_data = obsv_data_range
    y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
    plot.scatter(dz_log_mdot,logMdotData,s=marker_size, label='Death Zone␣
↪Analysis', color=color_list[1])
    #plot.plot(y_data,x_data, label='Death Zone Analysis',linewidth=4)

    form_vs_fed_fit = linear_fit_1var(Reimers_log_mdot,logMdotData)
    # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
    form_vs_fed_Coeffs = form_vs_fed_fit[0]
    #x_data = obsv_data_range
    #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
    x_data = obsv_data_range
    y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
    plot.scatter(Reimers_log_mdot,logMdotData,s=marker_size, label='Reimers␣
↪(1975)', alpha = others_alpha, color=color_list[2])
    #plot.plot(y_data,x_data, label='Reimers (1975)',linewidth=others_lw, alpha␣
↪= others_alpha)

    form_vs_fed_fit = linear_fit_1var(Reimers_log_mdot_2,logMdotData)
    # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
    form_vs_fed_Coeffs = form_vs_fed_fit[0]
    #x_data = obsv_data_range
    #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
    x_data = obsv_data_range
    y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
    plot.scatter(Reimers_log_mdot_2,logMdotData,s=marker_size, label='Reimers␣
↪(1977)', alpha = others_alpha, color=color_list[3])
    #plot.plot(y_data,x_data, label='Reimers (1977)',linewidth=others_lw, alpha␣
↪= others_alpha)

    if spec_mode_label == 'C0' or spec_mode_label == 'C0x' or spec_mode_label␣
↪== 'C0 and C0x':
```

```
        form_vs_fed_fit = linear_fit_1var(VW_log_mdot,logMdotData)
        # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
        form_vs_fed_Coeffs = form_vs_fed_fit[0]
        #x_data = obsv_data_range
        #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
        x_data = obsv_data_range
        y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
        plot.scatter(VW_log_mdot,logMdotData,s=marker_size, label='Vassiliadis␣
↪and Wood', alpha = others_alpha, color=color_list[4])
        #plot.plot(y_data,x_data, label='Vassiliadis and␣
↪Wood',linewidth=others_lw, alpha = others_alpha)

    form_vs_fed_fit = linear_fit_1var(Bloecker_log_mdot,logMdotData)
    # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
    form_vs_fed_Coeffs = form_vs_fed_fit[0]
    #x_data = obsv_data_range
    #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
    x_data = obsv_data_range
    y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
    plot.scatter(Bloecker_log_mdot,logMdotData,s=marker_size, label=u'Blöcker',␣
↪alpha = others_alpha, color=color_list[5])
    #plot.plot(y_data,x_data, label=u'Blöcker',linewidth=others_lw, alpha =␣
↪others_alpha)

    form_vs_fed_fit = linear_fit_1var(Wachter_log_mdot,logMdotData)
    # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
    form_vs_fed_Coeffs = form_vs_fed_fit[0]
    #x_data = obsv_data_range
    #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
    x_data = obsv_data_range
    y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
    plot.scatter(Wachter_log_mdot,logMdotData,s=marker_size, label='Wachter et␣
↪al.', alpha = others_alpha, color=color_list[6])
    #plot.plot(y_data,x_data, label='Wachter et al.',linewidth=others_lw, alpha␣
↪= others_alpha)

    form_vs_fed_fit = linear_fit_1var(van_Loon_log_mdot,logMdotData)
    # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
    form_vs_fed_Coeffs = form_vs_fed_fit[0]
    #x_data = obsv_data_range
    #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
    x_data = obsv_data_range
    y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
    plot.scatter(van_Loon_log_mdot,logMdotData,s=marker_size, label='van Loon',␣
↪alpha = others_alpha, color=color_list[7])
```

```
   #plot.plot(y_data,x_data, label=u'van Loon',linewidth=others_lw, alpha =␣
↪others_alpha)

   form_vs_fed_fit = linear_fit_1var(Schroeder_Cuntz_log_mdot,logMdotData)
   # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
   form_vs_fed_Coeffs = form_vs_fed_fit[0]
   #x_data = obsv_data_range
   #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
   x_data = obsv_data_range
   y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
   plot.scatter(Schroeder_Cuntz_log_mdot,logMdotData,s=marker_size,␣
↪label=u'Schröder and Cuntz', alpha = others_alpha, color=color_list[8])
   #plot.plot(y_data,x_data, label=u'Schröder and Cuntz',linewidth=others_lw,␣
↪alpha = others_alpha)

   form_vs_fed_fit = linear_fit_1var(de_Beck_log_mdot,logMdotData)
   # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
   form_vs_fed_Coeffs = form_vs_fed_fit[0]
   #x_data = obsv_data_range
   #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
   x_data = obsv_data_range
   y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
   plot.scatter(de_Beck_log_mdot,logMdotData,s=marker_size, label='de Beck et␣
↪al.', alpha = others_alpha, color=color_list[9])
   #plot.plot(y_data,x_data, label='de Beck et al.',linewidth=others_lw, alpha␣
↪= others_alpha)

   plot.xlim(-8,-4)
   plot.ylim(-8,-4)

   plot.title(spec_mode_label)
   plot.plot([-12,-2],[-12,-2],color = 'black', alpha =␣
↪1,linestyle='--',linewidth=1.5)
   plot.ylabel(r'$\log \dot{M},\ \mathrm{Calc.}$')
   plot.xlabel(r'$\log \dot{M},\ \mathrm{Obsv.}$')
   plot.tight_layout()
   plot.legend(ncol=2,fontsize='small',loc=2)
   plot.savefig('Final_Fit/Form_Comparison_'+spec_mode_label+'.png')
   plot.savefig('Final_Fit/Form_Comparison_'+spec_mode_label+'.pdf')
   plot.show()
   plot.close()

   print('\n','_____________','\n')

   # Type 2: Residual vs. Obsv
```

```
    obsv_data_range = np.asarray([-9.5,-3.5])

    form_vs_fed_fit = linear_fit_1var(logMdotData,our_log_mdot-logMdotData)
    # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
    form_vs_fed_Coeffs = form_vs_fed_fit[0]
    #x_data = obsv_data_range
    #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
    x_data = obsv_data_range
    y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
    plot.scatter(logMdotData,our_log_mdot-logMdotData, s=marker_size,␣
↪label='Multilinear Fit', color=color_list[0])
    #plot.plot(x_data,y_data, label='Multilinear Fit',linewidth=4)
    print('Dust-Fitting Results:',form_vs_fed_fit)

    dz_log_mdot = dz_values[0] + dz_values[1]*np.log10(star_data[LIndex]) +␣
↪dz_values[2]*np.log10(star_data[MIndex])
    #print(len(dz_log_mdot),len(logMdotData))
    form_vs_fed_fit = linear_fit_1var(logMdotData,dz_log_mdot-logMdotData)
    # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
    form_vs_fed_Coeffs = form_vs_fed_fit[0]
    #x_data = obsv_data_range
    #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
    x_data = obsv_data_range
    y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
    plot.scatter(logMdotData,dz_log_mdot-logMdotData, s=marker_size,␣
↪label='Death Zone Analysis', color=color_list[1])
    #plot.plot(x_data,y_data, label='Death Zone Analysis',linewidth=4)
    print('Death Zone-Fitting Results:',form_vs_fed_fit)

    form_vs_fed_fit = linear_fit_1var(logMdotData,Reimers_log_mdot-logMdotData)
    # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
    form_vs_fed_Coeffs = form_vs_fed_fit[0]
    #x_data = obsv_data_range
    #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
    x_data = obsv_data_range
    y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
    plot.scatter(logMdotData,Reimers_log_mdot-logMdotData, s=marker_size,␣
↪label='Reimers (1975)', color=color_list[2])
    #plot.plot(x_data,y_data, label='Reimers (1975)',linewidth=others_lw, alpha␣
↪= others_alpha)
    print('Reimers 1975 Results:',form_vs_fed_fit)

    form_vs_fed_fit =␣
↪linear_fit_1var(logMdotData,Reimers_log_mdot_2-logMdotData)
    # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
    form_vs_fed_Coeffs = form_vs_fed_fit[0]
```

```
   #x_data = obsv_data_range
   #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
   x_data = obsv_data_range
   y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
   plot.scatter(logMdotData,Reimers_log_mdot_2-logMdotData, s=marker_size,␣
↪label='Reimers (1977)', color=color_list[3])
   #plot.plot(x_data,y_data, label='Reimers (1977)',linewidth=others_lw, alpha␣
↪= others_alpha)
   print('Reimers 1977 Results:',form_vs_fed_fit)

   if spec_mode_label == 'CO' or spec_mode_label == 'COx' or spec_mode_label␣
↪== 'CO and COx':
       form_vs_fed_fit = linear_fit_1var(logMdotData,VW_log_mdot-logMdotData)
       # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
       form_vs_fed_Coeffs = form_vs_fed_fit[0]
       #x_data = obsv_data_range
       #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
       x_data = obsv_data_range
       y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
       plot.scatter(logMdotData,VW_log_mdot-logMdotData,s=marker_size,␣
↪label='Vassiliadis and Wood', color=color_list[4])
       #plot.plot(x_data,y_data, label='Vassiliadis and␣
↪Wood',linewidth=others_lw, alpha = others_alpha)
       print('VW Results:',form_vs_fed_fit)

   form_vs_fed_fit = linear_fit_1var(logMdotData,Bloecker_log_mdot-logMdotData)
   # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
   form_vs_fed_Coeffs = form_vs_fed_fit[0]
   #x_data = obsv_data_range
   #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
   x_data = obsv_data_range
   y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
   plot.scatter(logMdotData,Bloecker_log_mdot-logMdotData, s=marker_size,␣
↪label=u'Blöcker', color=color_list[5])
   #plot.plot(x_data,y_data, label=u'Blöcker',linewidth=others_lw, alpha =␣
↪others_alpha)
   print('Bloecker Results:',form_vs_fed_fit)

   form_vs_fed_fit = linear_fit_1var(logMdotData,Wachter_log_mdot-logMdotData)
   # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
   form_vs_fed_Coeffs = form_vs_fed_fit[0]
   #x_data = obsv_data_range
   #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
   x_data = obsv_data_range
   y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
```

```
   plot.scatter(logMdotData,Wachter_log_mdot-logMdotData, s=marker_size,␣
↪label='Wachter et al.', color=color_list[6])
   #plot.plot(x_data,y_data, label='Wachter et al.',linewidth=others_lw, alpha␣
↪= others_alpha)
   print('Wachter et al. Results:',form_vs_fed_fit)

   form_vs_fed_fit = linear_fit_1var(logMdotData,van_Loon_log_mdot-logMdotData)
   # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
   form_vs_fed_Coeffs = form_vs_fed_fit[0]
   #x_data = obsv_data_range
   #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
   x_data = obsv_data_range
   y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
   plot.scatter(logMdotData,van_Loon_log_mdot-logMdotData, s=marker_size,␣
↪label=u'van Loon', color=color_list[7])
   #plot.plot(x_data,y_data, label=u'van Loon',linewidth=others_lw, alpha =␣
↪others_alpha)
   print('van Loon Results:',form_vs_fed_fit)

   form_vs_fed_fit =␣
↪linear_fit_1var(logMdotData,Schroeder_Cuntz_log_mdot-logMdotData)
   # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
   form_vs_fed_Coeffs = form_vs_fed_fit[0]
   #x_data = obsv_data_range
   #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
   x_data = obsv_data_range
   y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
   plot.scatter(logMdotData,Schroeder_Cuntz_log_mdot-logMdotData,␣
↪s=marker_size, label=u'Schröder and Cuntz', color=color_list[8])
   #plot.plot(x_data,y_data, label=u'Schröder and Cuntz',linewidth=others_lw,␣
↪alpha = others_alpha)
   print('Schroeder and Cuntz Results:',form_vs_fed_fit)

   form_vs_fed_fit = linear_fit_1var(logMdotData,de_Beck_log_mdot-logMdotData)
   # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
   form_vs_fed_Coeffs = form_vs_fed_fit[0]
   #x_data = obsv_data_range
   #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
   x_data = obsv_data_range
   y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
   plot.scatter(logMdotData,de_Beck_log_mdot-logMdotData, s=marker_size,␣
↪label='de Beck et al.', color=color_list[9])
   #plot.plot(x_data,y_data, label='de Beck et al.',linewidth=others_lw, alpha␣
↪= others_alpha)
   print('de Beck et al. Results:',form_vs_fed_fit)
```

```
   plot.xlim(-8,-4)
   plot.ylim(-5,3)

   plot.title(spec_mode_label)
   plot.plot([-12,-2],[0,0],color = 'black', alpha =␣
↪1,linestyle='--',linewidth=1.5)
   plot.ylabel(r'$\log \dot{M},\ \mathrm{Calc.}-\log \dot{M},\ \mathrm{Obsv.
↪}$')
   plot.xlabel(r'$\log \dot{M},\ \mathrm{Obsv.}$')
   plot.tight_layout()
   plot.legend(ncol=2,fontsize='small',loc=3)
   plot.savefig('Final_Fit/Form_Comparison_2_'+spec_mode_label+'.png')
   plot.savefig('Final_Fit/Form_Comparison_2_'+spec_mode_label+'.pdf')
   plot.show()
   plot.close()

   print('\n','_____________','\n')



   if spec_mode_label == 'C0' or spec_mode_label == 'C0x' or spec_mode_label␣
↪== 'C0 and C0x':
       residual_list = [our_log_mdot-logMdotData, dz_log_mdot-logMdotData,␣
↪Reimers_log_mdot-logMdotData,
                        Reimers_log_mdot_2-logMdotData, VW_log_mdot-logMdotData,␣
↪Bloecker_log_mdot-logMdotData,
                        Wachter_log_mdot-logMdotData,␣
↪van_Loon_log_mdot-logMdotData, Schroeder_Cuntz_log_mdot-logMdotData,
                        de_Beck_log_mdot-logMdotData]
       frame = pandas.DataFrame({"Fit to Dust": residual_list[0], "Fit to␣
↪Deathzone": residual_list[1],
                                 "Reimers (1975)": residual_list[2], "Reimers␣
↪(1977)": residual_list[3],
                                "Vassiliadis and Wood": residual_list[4],␣
↪u"Blöcker": residual_list[5],
                                "Wachter et al.": residual_list[6], "van Loon":␣
↪residual_list[7],
                                u"Schröder and Cuntz": residual_list[8], "de Beck␣
↪et al.": residual_list[9]})
       boxplot_labels = ["Fit to Dust", "Fit to Deathzone", "Reimers␣
↪(1975)","Reimers (1977)","Vassiliadis and Wood",
            u"Blöcker", "Wachter et al.", "van Loon", u"Schröder and Cuntz",␣
↪"de Beck et al."]
   else:
```

```
      residual_list = [our_log_mdot-logMdotData, dz_log_mdot-logMdotData,␣
↪Reimers_log_mdot-logMdotData,
                       Reimers_log_mdot_2-logMdotData,␣
↪Bloecker_log_mdot-logMdotData,
                       Wachter_log_mdot-logMdotData,␣
↪van_Loon_log_mdot-logMdotData, Schroeder_Cuntz_log_mdot-logMdotData,
                       de_Beck_log_mdot-logMdotData]
      frame = pandas.DataFrame({"Fit to Dust": residual_list[0], "Fit to␣
↪Deathzone": residual_list[1],
                                "Reimers (1975)": residual_list[2], "Reimers␣
↪(1977)": residual_list[3],
                                u"Blöcker": residual_list[4],
                               "Wachter et al.": residual_list[5], "van Loon":␣
↪residual_list[6],
                               u"Schröder and Cuntz": residual_list[7], "de Beck␣
↪et al.": residual_list[8]})
      boxplot_labels = ["Fit to Dust", "Fit to Deathzone", "Reimers␣
↪(1975)","Reimers (1977)",
           u"Blöcker", "Wachter et al.", "van Loon", u"Schröder and Cuntz",␣
↪"de Beck et al."]
      boxplot_pal = {"Fit to Dust":color_list[0], "Fit to Deathzone":
↪color_list[1], "Reimers (1975)":color_list[2],"Reimers (1977)":color_list[3],
           u"Blöcker":color_list[5], "Wachter et al.":color_list[6], "van␣
↪Loon":color_list[7], u"Schröder and Cuntz":color_list[8], "de Beck et al.":
↪color_list[9]}

   # Type 2 Diagnostic: individual scatter plots
   for i in range(0, len(residual_list)):
      if spec_mode_label == 'C0' or spec_mode_label == 'C0x' or␣
↪spec_mode_label == 'C0 and C0x':
          plot.scatter(logMdotData,residual_list[i], s=marker_size,␣
↪label=boxplot_labels[i],color=color_list[i])
      else:
          if i < 4:
              plot.scatter(logMdotData,residual_list[i], s=2,␣
↪label=boxplot_labels[i],color=color_list[i])
          else:
              plot.scatter(logMdotData,residual_list[i], s=2,␣
↪label=boxplot_labels[i],color=color_list[i+1])
      plot.xlim(-8,-4)
      plot.ylim(-5,3)
      plot.title(spec_mode_label)
      plot.plot([-12,-2],[0,0],color = 'black', alpha =␣
↪1,linestyle='--',linewidth=1.5)
      plot.ylabel(r'$\log \dot{M},\ \mathrm{Calc.}-\log \dot{M},\␣
↪\mathrm{Obsv.}$')
```

```
        plot.xlabel(r'$\log \dot{M},\ \mathrm{Obsv.}$')
        plot.tight_layout()
        plot.legend(ncol=2,fontsize='small',loc=3)
        plot.savefig('Final_Fit/
↪Form_Comparison_2_diagnostic_'+str(i)+spec_mode_label+'.png')
        plot.savefig('Final_Fit/
↪Form_Comparison_2_diagnostic_'+str(i)+spec_mode_label+'.pdf')
        plot.show()
        plot.close()
        print('\n','_____________','\n')


    # Type 3: Box and Whisker Plot
    if spec_mode_label == 'C0' or spec_mode_label == 'C0x' or spec_mode_label␣
↪== 'C0 and C0x':
        sns.boxplot(data=frame,orient='h')
    else:
        sns.boxplot(data=frame,orient='h',palette = boxplot_pal)

    current_limits = plot.gca().get_ylim()
    #breathing_room = np.abs(current_limits[0]-current_limits[1])/10
    plot.vlines(0,current_limits[0]+1.5,current_limits[1]-1.
↪5,colors='black',linestyles='--')
    plot.ylim(current_limits[0]+0.5,current_limits[1]-0.5)
    plot.xlabel(r'$\log \dot{M}_{\mathrm{calc.}} - \log \dot{M}_{\mathrm{obsv.
↪}}$')
    plot.title(spec_mode_label)
    plot.tight_layout()
    plot.savefig('Final_Fit/Form_Comparison_3_'+spec_mode_label+'.png')
    plot.savefig('Final_Fit/Form_Comparison_3_'+spec_mode_label+'.pdf')
    plot.show()
    plot.close()


    alpha_pgram = 0.2
    # Type 4: Boxes around residuals
    for i in range(0, len(residual_list)):
        parallelogram_x, parallelogram_y =␣
↪get_parallelogram_points(logMdotData,residual_list[i])
        pgrm_y1 = parallelogram_y[0]
        pgrm_y2 = parallelogram_y[1]
        plot.fill(parallelogram_x,parallelogram_y,label=boxplot_labels[i])
        if spec_mode_label == 'C0' or spec_mode_label == 'C0x' or␣
↪spec_mode_label == 'C0 and C0x':
            plot.
↪fill_between(parallelogram_x,pgrm_y1,pgrm_y2,color=color_list[i],label=boxplot_labels[i],al
```

```
        else:
            if i < 4:
                plot.
↪fill_between(parallelogram_x,pgrm_y1,pgrm_y2,color=color_list[i],label=boxplot_labels[i],al
            else:
                plot.
↪fill_between(parallelogram_x,pgrm_y1,pgrm_y2,color=color_list[i+1],label=boxplot_labels[i],
    plot.xlim(-8,-4)
    plot.ylim(-5,3)
    plot.title(spec_mode_label)
    plot.plot([-12,-2],[0,0],color = 'black', alpha =␣
↪1,linestyle='--',linewidth=1.5)
    plot.ylabel(r'$\log \dot{M},\ \mathrm{Calc.}-\log \dot{M},\ \mathrm{Obsv.
↪}$')
    plot.xlabel(r'$\log \dot{M},\ \mathrm{Obsv.}$')
    plot.tight_layout()
    #plot.legend(ncol=2,fontsize='small',loc=3)
    plot.savefig('Final_Fit/Form_Comparison_4_'+spec_mode_label+'.png')
    plot.savefig('Final_Fit/Form_Comparison_4_'+spec_mode_label+'.pdf')
    plot.show()
    plot.close()

    print('\n','_____________','\n')
```

[379]:
```
def mdot_formula_comparison_multidata (star_data, spec_mode_label_list =␣
↪[],marker_size=2):
    spec_mode_label = ''
    for i in range(0, len(spec_mode_label_list)):
        if i == 0:
            spec_mode_label = spec_mode_label_list[0]
        elif i == len(spec_mode_label_list)-1:
            if len(spec_mode_label_list) == 2:
                spec_mode_label = spec_mode_label + ' and ' +␣
↪spec_mode_label_list[i]
            else:
                spec_mode_label = spec_mode_label + ', and ' +␣
↪spec_mode_label_list[i]
        else:
            spec_mode_label = spec_mode_label + ', ' + spec_mode_label_list[i]
    print(spec_mode_label)
    # Use artificial star data to create plots demonstrating how our formulas␣
↪differ from eachother and other notable formulas
    logLData = []
    for i in range(0, len(star_data)):
        logLData.append(np.log10(star_data[i][LIndex]))
    stacked_logLData = np.hstack(logLData)
```

```
    logMData = []
    for i in range(0, len(star_data)):
        logMData.append(np.log10(star_data[i][MIndex]))
    stacked_logMData = np.hstack(logMData)

    logPData = []
    for i in range(0, len(star_data)):
        logPData.append(np.log10(star_data[i][PIndex]))
    stacked_logPData = np.hstack(logPData)

    logRData = []
    for i in range(0, len(star_data)):
        logRData.append(np.log10(star_data[i][RIndex]))
    stacked_logRData = np.hstack(logRData)

    logTData = []
    for i in range(0, len(star_data)):
        logTData.append(np.log10(star_data[i][TeffCalcIndex]))
    stacked_logTData = np.hstack(logTData)

    logMdotData = []
    for i in range(0, len(star_data)):
        logMdotData.append(np.log10(star_data[i][MdotIndex]))
    stacked_logMdotData = np.hstack(logMdotData)


    color_list = ['tab:blue','tab:orange','tab:green','tab:red','tab:
↪purple','tab:brown', 'tab:pink', 'tab:gray', 'tab:olive', 'tab:cyan']

    LPmultilinear_fit = []
    LPCoeffs = []
    our_log_mdot = []
    for i in range(0, len(star_data)):
        LPmultilinear_fit =␣
↪multilinear_fit_2var([logLData[i],logPData[i]],logMdotData[i])
        LPCoeffs.append(LPmultilinear_fit[0])
        our_log_mdot.append(LPCoeffs[i][0] + LPCoeffs[i][1]*logLData[i] +␣
↪LPCoeffs[i][2]*logPData[i])
    # log Mdot = A + BlogL + ClogP
    if spec_mode_label[0] == 'M':
        RLM = RLM_M
    if spec_mode_label[0] == 'C':
        RLM = RLM_C
    if spec_mode_label == 'M0':
        PMR = PMR_M0
        dz_values = deathzone_values_M0
```

```
    if spec_mode_label == 'M1':
        PMR = PMR_M1
        dz_values = deathzone_values_M1
    if spec_mode_label == 'C0':
        PMR = PMR_C0
        dz_values = deathzone_values_C0
    if spec_mode_label == 'C0x':
        PMR = PMR_C0
        dz_values = deathzone_values_C0x
    if spec_mode_label == 'C0 and C0x':
        PMR = PMR_C0
        dz_values = deathzone_values_C0_and_C0x
    if spec_mode_label == 'C1':
        PMR = PMR_C1
        dz_values = deathzone_values_C1
    if spec_mode_label == 'C0 and C0x':
        PMR = PMR_C0
        dz_values = deathzone_values_C0_and_C0x

    #artificial_data = make_artificial_set([star_data],0.2,grid_spacing = 75j)
    #artificial_logLData = artificial_data[0]
    #artificial_logMData = artificial_data[1]
    #artificial_logRData = RLM[0] + RLM[1]*artificial_logLData +␣
↪RLM[2]*artificial_logMData
    #artificial_logPData = PMR[0] + PMR[1]*artificial_logMData +␣
↪PMR[2]*artificial_logRData
    #artificial_logTData = np.log10(5778*np.sqrt(10**(artificial_logRData)/np.
↪sqrt(10**artificial_logLData)))

    for i in range(0, len(star_data)):
        a = LPCoeffs[i][0]
        arlm = RLM[0]
        aRLM = arlm
        apmr = PMR[0]
        aPMR = apmr
        c = LPCoeffs[i][2]
        cpmr = PMR[2]
        cPMR = cpmr
        b = LPCoeffs[i][1]
        brlm = RLM[1]
        bRLM = brlm
        bpmr = PMR[1]
        bPMR = bpmr
        crlm = RLM[2]
        cRLM = crlm
        PMLCoeffs = calculate_PML_coeffs(RLM, PMR)
        aPML = PMLCoeffs[0]
```

```
        bPML = PMLCoeffs[1]
        cPML = PMLCoeffs[2]

        Anew = a - arlm + apmr*c + arlm*c*cpmr
        Bnew = b - brlm + brlm*c*cpmr
        Cnew = bpmr*c - crlm + c*cpmr*crlm
        print('Data set', i,':')
        print('log Mdot ~ logL + logP:')
        print('LRM form: logMdot =',Anew,' + ',Bnew,'logL + logR␣
↪-',-Cnew,'logM')
        print('LM form: logMdot =',Anew+arlm,' + ',Bnew+brlm,'logL␣
↪-',-(Cnew+crlm),'logM')

    print('_______________')

    dz_log_mdot = []
    for i in range(0, len(star_data)):
        dz_log_mdot.append( dz_values[0] + dz_values[1]*np.
↪log10(star_data[i][LIndex]) + dz_values[2]*np.log10(star_data[i][MIndex]))

    print('log Mdot ~ logL + logM')
    print('LM form: logMdot =',dz_values[0],' + ',dz_values[1],'logL -',np.
↪abs(dz_values[2]),'logM')

    print('_______________')

    Reimers_log_mdot = []
    for i in range(0, len(star_data)):
        Reimers_log_mdot.append(np.log10(4*10**-13) + logLData[i] + logRData[i]␣
↪- logMData[i])
    # Mdot = (4*10^-13) * eta * LR/M => logMdot = log(4*10^-13) + log(eta) +␣
↪logL + logR - logM
    # 1975
    stacked_Reimers_log_mdot = np.hstack(Reimers_log_mdot)
    print('Reimers 1975 LRM form: logMdot =',np.log10(4*10**-13),'+ logL + logR␣
↪- logM')
    Aprint = np.log10(4*10**-13) + aRLM
    Bprint = 1 + bRLM
    Cprint = -1 + cRLM
    print('Reimers 1975 LM form: logMdot =', Aprint,'+',Bprint,'logL - ',np.
↪abs(Cprint),'logM')
    fit = linear_fit_1var(stacked_logMdotData,stacked_Reimers_log_mdot)
    print('Calc vs Obsv Slope:', fit[0][1])
    print('Calc vs Obsv R^2:',fit[1])
    print('SDev of Residual:',np.
↪nanstd(stacked_Reimers_log_mdot-stacked_logMdotData))
```

```
    print('Mean distance from 1-to-1 line:',np.nanmean(np.
↪abs(stacked_Reimers_log_mdot-stacked_logMdotData)))

    print('_____________')

    Reimers_log_mdot_2 = []
    for i in range(0, len(star_data)):
        Reimers_log_mdot_2.append(Reimers_log_mdot[i] +np.log10(0.35))
    # Mdot = (4*10^-13) * 0.35 * LR/M => logMdot = log(4*10^-13) + log(0.35) +␣
↪logL + logR - logM
    # http://cdsads.u-strasbg.fr/pdf/1977A%26A....61..217R, 1977
    stacked_Reimers_log_mdot_2 = np.hstack(Reimers_log_mdot_2)
    print('Reimers 1977 LRM form: logMdot =',np.log10(4*10**-13)+np.log10(0.
↪35),'+ logL + logR - logM')
    Aprint = np.log10(4*10**-13)+np.log10(0.35) + aRLM
    Bprint = 1 + bRLM
    Cprint = -1 + cRLM
    print('Reimers 1977 LM form: logMdot =', Aprint,'+',Bprint,'logL - ',np.
↪abs(Cprint),'logM')
    fit = linear_fit_1var(stacked_logMdotData,stacked_Reimers_log_mdot_2)
    print('Calc vs Obsv Slope:', fit[0][1])
    print('Calc vs Obsv R^2:',fit[1])
    print('SDev of Residual:',np.
↪nanstd(stacked_Reimers_log_mdot_2-stacked_logMdotData))
    print('Mean distance from 1-to-1 line:',np.nanmean(np.
↪abs(stacked_Reimers_log_mdot_2-stacked_logMdotData)))

    print('_____________')

    VW_log_mdot = []
    for i in range(0, len(star_data)):
        VW_log_mdot.append(calc_vw_mdot(logPData[i], logMData[i]))
    # log Mdot = -11.4 + 0.0125(P-100(M-2.5))
    # http://adsabs.harvard.edu/abs/1993ApJ...413..641V, 1993
    stacked_VW_log_mdot = np.hstack(VW_log_mdot)
    apprx_fit =␣
↪multilinear_fit_2var([stacked_logLData,stacked_logMData],stacked_VW_log_mdot)
    apprx_fit_coeffs = apprx_fit[0]
    print('VW LRM form: N/A')
    if apprx_fit_coeffs[2]>0:
        print('VW LM form apprx.:
↪',apprx_fit_coeffs[0],'+',apprx_fit_coeffs[1],'logL','+',apprx_fit_coeffs[2],'logM')
    else:
        print('VW LM form apprx.:
↪',apprx_fit_coeffs[0],'+',apprx_fit_coeffs[1],'logL','-',np.
↪abs(apprx_fit_coeffs[2]),'logM')
```

```
    fit = linear_fit_1var(stacked_logMdotData,stacked_VW_log_mdot)
    print('Calc vs Obsv Slope:', fit[0][1])
    print('Calc vs Obsv R^2:',fit[1])
    print('SDev of Residual:',np.
↪nanstd(stacked_VW_log_mdot-stacked_logMdotData))
    print('Mean distance from 1-to-1 line:',np.nanmean(np.
↪abs(stacked_VW_log_mdot-stacked_logMdotData)))

    print('_______________')

    Bloecker_log_mdot = []
    for i in range(0, len(star_data)):
        Bloecker_log_mdot.append(np.log10(1.932*10**-21) + 3.7*logLData[i] +␣
↪logRData[i] - 3.1*logMData[i])
    stacked_Bloecker_log_mdot = np.hstack(Bloecker_log_mdot)
    # Mdot = (4.83*10^-9) (M^-2.1)(L^2.7)(Mdot_R=(4*10^-13) * eta * LR/M) = (1.
↪932*10^-21)(L^3.7)(R)(M^-3.1)
    # -> logMdot = log[1.932*10^-21] + 3.7logL + logR - 3.1logM
    # https://ui.adsabs.harvard.edu/abs/1995A&A...297..727B, 1995
    print('Bloecker LRM form: logMdot =',np.log10(1.932*10**-21),'+3.7logL +␣
↪logR - 3.1logM')
    Aprint = np.log10(1.932*10**-21) + aRLM
    Bprint = 3.7 + bRLM
    Cprint = -3.1 + cRLM
    print('Bloecker LM form: logMdot =',Aprint,'+',Bprint,'logL - ',np.
↪abs(Cprint), 'logM')
    fit = linear_fit_1var(stacked_logMdotData,stacked_Bloecker_log_mdot)
    print('Calc vs Obsv Slope:', fit[0][1])
    print('Calc vs Obsv R^2:',fit[1])
    print('SDev of Residual:',np.
↪nanstd(stacked_Bloecker_log_mdot-stacked_logMdotData))
    print('Mean distance from 1-to-1 line:',np.nanmean(np.
↪abs(stacked_Bloecker_log_mdot-stacked_logMdotData)))


    print('_______________')

    Wachter_log_mdot = []
    for i in range(0, len(star_data)):
        Wachter_log_mdot.append(9.84423 + 1.54*logLData[i] - 1.95*logMData[i] +␣
↪0.959*logPData[i] - 6.8*logTData[i])
    # logMdot = -4.52 - 6.81(logT/2600 K) + 1.54log(L/10^4 Lsun) - 1.95log(M/
↪Msun)+0.959log(P/650d)
    # -> logMdot = 9.84423 + 1.54logL -1.95logM + 0.959logP - 6.8logT
    # https://ui.adsabs.harvard.edu/abs/2002A&A...384..452W, 2002
```

```
    # from definition of effective temp., logT = 0.940482 + 0.25logL - 0.5logR␣
↪(in solar units and Tsun = 5780K)
    WachterA = -6.39528 + 9.84423 + 0.959*aPML + 2.4*aRLM
    WachterB = -1.7 + 1.54 + 0.959*bPML+2.4*bRLM
    WachterC = -1.95 + 0.959*cPML+2.4*cRLM
    if WachterC <0:
        print('Wachter LRM form: logMdot =',WachterA,' + ',WachterB,'logL -␣
↪',np.abs(WachterC),'logM + logR')
    else:
        print('Wachter LRM form: logMdot =',WachterA,' + ',WachterB,'logL +␣
↪',WachterC,'logM + logR')
    Aprint = WachterA + aRLM
    Bprint = WachterB + bRLM
    Cprint = WachterC + cRLM
    if Cprint < 0:
        print('Wachter LM form: logMdot =',Aprint,' + ',Bprint,'logL - ',np.
↪abs(Cprint),'logM')
    else:
        print('Wachter LM form: logMdot =',Aprint,' + ',Bprint,'logL +␣
↪',Cprint,'logM')
    stacked_Wachter_log_mdot = np.hstack(Wachter_log_mdot)
    fit = linear_fit_1var(stacked_logMdotData,stacked_Wachter_log_mdot)
    print('Calc vs Obsv Slope:', fit[0][1])
    print('Calc vs Obsv R^2:',fit[1])
    print('SDev of Residual:',np.
↪nanstd(stacked_Wachter_log_mdot-stacked_logMdotData))
    print('Mean distance from 1-to-1 line:',np.nanmean(np.
↪abs(stacked_Wachter_log_mdot-stacked_logMdotData)))

    print('_______________')

    van_Loon_log_mdot = []
    for i in range(0, len(star_data)):
        van_Loon_log_mdot.append(12.4776 + 1.05*logLData[i] - 6.3*logTData[i])
    # logMdot = -5.65 + 1.05 log(L/10000Lsun) - 6.3log(T/3500K)
    # -> logMdot = 12.4776 + 1.05logL - 6.3logT
    vanLoonA = -5.92504 + 12.4776 + 2.15*aRLM
    vanLoonB = -1.575+1.05+2.15*bRLM
    vanLoonC = 2.15*cRLM
    if vanLoonC <0:
        print('van Loon LRM form: logMdot =',vanLoonA,' + ',vanLoonB,'logL␣
↪-',np.abs(vanLoonC),'logM + logR')
    else:
        print('van Loon LRM form: logMdot =',vanLoonA,' + ',vanLoonB,'logL␣
↪+',vanLoonC,'logM + logR')
    Aprint = vanLoonA + aRLM
```

```
    Bprint = vanLoonB + bRLM
    Cprint = vanLoonC + cRLM
    if Cprint <0:
        print('van Loon LM form: logMdot =',Aprint,' + ',Bprint,'logL -',np.
↪abs(Cprint),'logM')
    else:
        print('van Loon LM form: logMdot =',Aprint,' + ',Bprint,'logL␣
↪+',Cprint,'logM')
    stacked_van_Loon_log_mdot = np.hstack(van_Loon_log_mdot)
    fit = linear_fit_1var(stacked_logMdotData,stacked_van_Loon_log_mdot)
    print('Calc vs Obsv Slope:', fit[0][1])
    print('Calc vs Obsv R^2:',fit[1])
    print('SDev of Residual:',np.
↪nanstd(stacked_van_Loon_log_mdot-stacked_logMdotData))
    print('Mean distance from 1-to-1 line:',np.nanmean(np.
↪abs(stacked_van_Loon_log_mdot-stacked_logMdotData)))

    print('_______________')

    Schroeder_Cuntz_log_mdot = []
    for i in range(0, len(star_data)):
        g = 10**logMData[i]/(10**logRData[i])**2
        Schroeder_Cuntz_log_mdot.append(np.log10(0.
↪2*10**Reimers_log_mdot[i]*(10**logTData[i]/4000)**3.5*(1+1/(4300*g[i]))))
    # Mdot = (Mdot_R=(4*10^-13) * eta * LR/M)(T/4000 K)^3.5 (1 + g_sun/
↪4300g_star)
    # http://adsabs.harvard.edu/abs/2005ApJ...630L..73S, 2005
    stacked_Schroeder_Cuntz_log_mdot = np.hstack(Schroeder_Cuntz_log_mdot)
    apprx_fit =␣
↪multilinear_fit_2var([stacked_logLData,stacked_logMData],stacked_Schroeder_Cuntz_log_mdot)
    apprx_fit_coeffs = apprx_fit[0]
    print('Schroeder and Cuntz LRM form: N/A')
    if apprx_fit_coeffs[2]>0:
        print('Schroeder and Cuntz LM form apprx.:
↪',apprx_fit_coeffs[0],'+',apprx_fit_coeffs[1],'logL','+',apprx_fit_coeffs[2],'logM')
    else:
        print('Schroeder and Cuntz LM form apprx.:
↪',apprx_fit_coeffs[0],'+',apprx_fit_coeffs[1],'logL','-',np.
↪abs(apprx_fit_coeffs[2]),'logM')
    fit = linear_fit_1var(stacked_logMdotData,stacked_Schroeder_Cuntz_log_mdot)
    print('Calc vs Obsv Slope:', fit[0][1])
    print('Calc vs Obsv R^2:',fit[1])
    print('SDev of Residual:',np.
↪nanstd(stacked_Schroeder_Cuntz_log_mdot-stacked_logMdotData))
    print('Mean distance from 1-to-1 line:',np.nanmean(np.
↪abs(stacked_Schroeder_Cuntz_log_mdot-stacked_logMdotData)))
```

```
    print('_______________')

    de_Beck_log_mdot = []
    for i in range(0, len(star_data)):
        de_Beck_log_mdot.append(calc_de_beck_mdot(logPData[i]))
    # log Mdot = -7.37 + 3.42*10^-3 * P (P<850)
    #          = -4.46                  (P>850)
    # https://ui.adsabs.harvard.edu/abs/2010A&A...523A..18D, 2010
    stacked_de_Beck_log_mdot = np.hstack(de_Beck_log_mdot)
    apprx_fit =␣
↪multilinear_fit_2var([stacked_logLData,stacked_logMData],stacked_de_Beck_log_mdot)
    apprx_fit_coeffs = apprx_fit[0]
    print('de Beck LRM form: N/A')
    if apprx_fit_coeffs[2]>0:
        print('de Beck LM form apprx.:
↪',apprx_fit_coeffs[0],'+',apprx_fit_coeffs[1],'logL','+',apprx_fit_coeffs[2],'logM')
    else:
        print('de Beck LM form apprx.:
↪',apprx_fit_coeffs[0],'+',apprx_fit_coeffs[1],'logL','-',np.
↪abs(apprx_fit_coeffs[2]),'logM')
    fit = linear_fit_1var(stacked_logMdotData,stacked_de_Beck_log_mdot)
    print('Calc vs Obsv Slope:', fit[0][1])
    print('Calc vs Obsv R^2:',fit[1])
    print('SDev of Residual:',np.
↪nanstd(stacked_de_Beck_log_mdot-stacked_logMdotData))
    print('Mean distance from 1-to-1 line:',np.nanmean(np.
↪abs(stacked_de_Beck_log_mdot-stacked_logMdotData)))

    print('_______________')

    obsv_data_range = np.asarray([-9.5,-3.5])

    others_lw = 3
    others_alpha = 0.5

    shape_list = ['s','v','1','8','h','+','^','D','o','p']

    # Type 1: Obsv vs. Calc
    stacked_our_log_mdot = np.hstack(our_log_mdot)
    form_vs_fed_fit = linear_fit_1var(stacked_our_log_mdot,stacked_logMdotData)
    # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
    form_vs_fed_Coeffs = form_vs_fed_fit[0]
    #x_data = obsv_data_range
    #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
    x_data = obsv_data_range
    y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
```

```
    for i in range(0, len(star_data)):
        plot.scatter(our_log_mdot[i],logMdotData[i],s=2, label='Multilinear␣
↪Fit'+' ('+spec_mode_label_list[i]+')', color=color_list[0],␣
↪marker=shape_list[i])
    #plot.plot(y_data,x_data, label='Multilinear Fit',linewidth=4)

    #print(len(dz_log_mdot),len(logMdotData))
    stacked_dz_log_mdot = np.hstack(dz_log_mdot)
    form_vs_fed_fit = linear_fit_1var(stacked_dz_log_mdot,stacked_logMdotData)
    # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
    form_vs_fed_Coeffs = form_vs_fed_fit[0]
    #x_data = obsv_data_range
    #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
    x_data = obsv_data_range
    y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
    for i in range(0, len(star_data)):
        plot.scatter(dz_log_mdot[i],logMdotData[i],s=2, label='Death Zone␣
↪Analysis'+' ('+spec_mode_label_list[i]+')', color=color_list[1],␣
↪marker=shape_list[i])
    #plot.plot(y_data,x_data, label='Death Zone Analysis',linewidth=4)

    stacked_Reimers_log_mdot = np.hstack(Reimers_log_mdot)
    form_vs_fed_fit =␣
↪linear_fit_1var(stacked_Reimers_log_mdot,stacked_logMdotData)
    # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
    form_vs_fed_Coeffs = form_vs_fed_fit[0]
    #x_data = obsv_data_range
    #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
    x_data = obsv_data_range
    y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
    for i in range(0, len(star_data)):
        plot.scatter(Reimers_log_mdot[i],logMdotData[i],s=2, label='Reimers␣
↪(1975)'+' ('+spec_mode_label_list[i]+')', alpha = others_alpha,␣
↪color=color_list[2], marker=shape_list[i])
        #plot.plot(y_data,x_data, label='Reimers (1975)',linewidth=others_lw,␣
↪alpha = others_alpha)

    stacked_Reimers_log_mdot_2 = np.hstack(Reimers_log_mdot_2)
    form_vs_fed_fit =␣
↪linear_fit_1var(stacked_Reimers_log_mdot_2,stacked_logMdotData)
    # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
    form_vs_fed_Coeffs = form_vs_fed_fit[0]
    #x_data = obsv_data_range
    #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
    x_data = obsv_data_range
    y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
```

```
   for i in range(0, len(star_data)):
       plot.scatter(Reimers_log_mdot_2[i],logMdotData[i],s=2, label='Reimers␣
↪(1977)'+' ('+spec_mode_label_list[i]+')', alpha = others_alpha,␣
↪color=color_list[3], marker=shape_list[i])
   #plot.plot(y_data,x_data, label='Reimers (1977)',linewidth=others_lw, alpha␣
↪= others_alpha)

   if spec_mode_label == 'C0' or spec_mode_label == 'C0x' or spec_mode_label␣
↪== 'C0 and C0x':
       stacked_VW_log_mdot = np.hstack(VW_log_mdot)
       form_vs_fed_fit =␣
↪linear_fit_1var(stacked_VW_log_mdot,stacked_logMdotData)
       # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
       form_vs_fed_Coeffs = form_vs_fed_fit[0]
       #x_data = obsv_data_range
       #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
       x_data = obsv_data_range
       y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
       for i in range(0, len(star_data)):
           plot.scatter(VW_log_mdot[i],logMdotData[i],s=2, label='Vassiliadis␣
↪and Wood'+' ('+spec_mode_label_list[i]+')', alpha = others_alpha,␣
↪color=color_list[4], marker=shape_list[i])
       #plot.plot(y_data,x_data, label='Vassiliadis and␣
↪Wood',linewidth=others_lw, alpha = others_alpha)

   stacked_Bloecker_log_mdot = np.hstack(Bloecker_log_mdot)
   form_vs_fed_fit =␣
↪linear_fit_1var(stacked_Bloecker_log_mdot,stacked_logMdotData)
   # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
   form_vs_fed_Coeffs = form_vs_fed_fit[0]
   #x_data = obsv_data_range
   #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
   x_data = obsv_data_range
   y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
   for i in range(0, len(star_data)):
       plot.scatter(Bloecker_log_mdot[i],logMdotData[i],s=2,␣
↪label=u'Blöcker'+' ('+spec_mode_label_list[i]+')', alpha = others_alpha,␣
↪color=color_list[5], marker=shape_list[i])
   #plot.plot(y_data,x_data, label=u'Blöcker',linewidth=others_lw, alpha =␣
↪others_alpha)

   stacked_Wachter_log_mdot = np.hstack(Wachter_log_mdot)
   form_vs_fed_fit =␣
↪linear_fit_1var(stacked_Wachter_log_mdot,stacked_logMdotData)
   # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
   form_vs_fed_Coeffs = form_vs_fed_fit[0]
```

```
    #x_data = obsv_data_range
    #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
    x_data = obsv_data_range
    y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
    for i in range(0, len(star_data)):
        plot.scatter(Wachter_log_mdot[i],logMdotData[i],s=2, label='Wachter et␣
↪al.'+' ('+spec_mode_label_list[i]+')', alpha = others_alpha,␣
↪color=color_list[6], marker=shape_list[i])
    #plot.plot(y_data,x_data, label='Wachter et al.',linewidth=others_lw, alpha␣
↪= others_alpha)

    stacked_van_Loon_log_mdot = np.hstack(van_Loon_log_mdot)
    form_vs_fed_fit =␣
↪linear_fit_1var(stacked_van_Loon_log_mdot,stacked_logMdotData)
    # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
    form_vs_fed_Coeffs = form_vs_fed_fit[0]
    #x_data = obsv_data_range
    #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
    x_data = obsv_data_range
    y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
    for i in range(0, len(star_data)):
        plot.scatter(van_Loon_log_mdot[i],logMdotData[i],s=2, label='van␣
↪Loon'+' ('+spec_mode_label_list[i]+')', alpha = others_alpha,␣
↪color=color_list[7], marker=shape_list[i])
    #plot.plot(y_data,x_data, label=u'van Loon',linewidth=others_lw, alpha =␣
↪others_alpha)

    stacked_Schroeder_Cuntz_log_mdot = np.hstack(Schroeder_Cuntz_log_mdot)
    form_vs_fed_fit =␣
↪linear_fit_1var(stacked_Schroeder_Cuntz_log_mdot,stacked_logMdotData)
    # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
    form_vs_fed_Coeffs = form_vs_fed_fit[0]
    #x_data = obsv_data_range
    #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
    x_data = obsv_data_range
    y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
    for i in range(0, len(star_data)):
        plot.scatter(Schroeder_Cuntz_log_mdot[i],logMdotData[i],s=2,␣
↪label=u'Schröder and Cuntz'+' ('+spec_mode_label_list[i]+')', alpha =␣
↪others_alpha, color=color_list[8], marker=shape_list[i])
    #plot.plot(y_data,x_data, label=u'Schröder and Cuntz',linewidth=others_lw,␣
↪alpha = others_alpha)

    stacked_de_Beck_log_mdot = np.hstack(de_Beck_log_mdot)
    form_vs_fed_fit =␣
↪linear_fit_1var(stacked_de_Beck_log_mdot,stacked_logMdotData)
```

```
    # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
    form_vs_fed_Coeffs = form_vs_fed_fit[0]
    #x_data = obsv_data_range
    #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
    x_data = obsv_data_range
    y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
    for i in range(0, len(star_data)):
        plot.scatter(de_Beck_log_mdot[i],logMdotData[i],s=2, label='de Beck et␣
↪al.'+' ('+spec_mode_label_list[i]+')', alpha = others_alpha,␣
↪color=color_list[9], marker=shape_list[i])
    #plot.plot(y_data,x_data, label='de Beck et al.',linewidth=others_lw, alpha␣
↪= others_alpha)

    plot.xlim(-8,-4)
    plot.ylim(-8,-4)

    plot.title(spec_mode_label)
    plot.plot([-12,-2],[-12,-2],color = 'black', alpha =␣
↪1,linestyle='--',linewidth=1.5)
    plot.ylabel(r'$\log \dot{M},\ \mathrm{Calc.}$')
    plot.xlabel(r'$\log \dot{M},\ \mathrm{Obsv.}$')
    plot.tight_layout()
    #plot.legend(ncol=2,fontsize='small',loc=2)
    plot.savefig('Final_Fit/Form_Comparison_'+spec_mode_label+'.png')
    plot.savefig('Final_Fit/Form_Comparison_'+spec_mode_label+'.pdf')
    plot.show()
    plot.close()

    print('\n','_____________','\n')

    # Type 2: Residual vs. Obsv

    obsv_data_range = np.asarray([-9.5,-3.5])

    form_vs_fed_fit =␣
↪linear_fit_1var(stacked_logMdotData,stacked_our_log_mdot-stacked_logMdotData)
    # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
    form_vs_fed_Coeffs = form_vs_fed_fit[0]
    #x_data = obsv_data_range
    #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
    x_data = obsv_data_range
    y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
    for i in range(0, len(star_data)):
        plot.scatter(logMdotData[i],our_log_mdot[i]-logMdotData[i],␣
↪s=marker_size, label='Multilinear Fit'+' ('+spec_mode_label_list[i]+')',␣
↪color=color_list[0], marker=shape_list[i])
    print('Dust-Fitting Results:',form_vs_fed_fit)
```

```
    #plot.plot(x_data,y_data, label='Multilinear Fit',linewidth=4)

    #print(len(dz_log_mdot),len(logMdotData))
    form_vs_fed_fit =␣
↪linear_fit_1var(stacked_logMdotData,stacked_dz_log_mdot-stacked_logMdotData)
    # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
    form_vs_fed_Coeffs = form_vs_fed_fit[0]
    #x_data = obsv_data_range
    #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
    x_data = obsv_data_range
    y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
    for i in range(0, len(star_data)):
        plot.scatter(logMdotData[i],dz_log_mdot[i]-logMdotData[i],␣
↪s=marker_size, label='Death Zone Analysis'+' ('+spec_mode_label_list[i]+')',␣
↪color=color_list[1], marker=shape_list[i])
    print('Death Zone-Fitting Results:',form_vs_fed_fit)
    #plot.plot(x_data,y_data, label='Death Zone Analysis',linewidth=4)

    form_vs_fed_fit =␣
↪linear_fit_1var(stacked_logMdotData,stacked_Reimers_log_mdot-stacked_logMdotData)
    # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
    form_vs_fed_Coeffs = form_vs_fed_fit[0]
    #x_data = obsv_data_range
    #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
    x_data = obsv_data_range
    y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
    for i in range(0, len(star_data)):
        plot.scatter(logMdotData[i],Reimers_log_mdot[i]-logMdotData[i],␣
↪s=marker_size, label='Reimers (1975)'+' ('+spec_mode_label_list[i]+')',␣
↪color=color_list[2], marker=shape_list[i])
    print('Reimers 1975 Results:',form_vs_fed_fit)
    plot.plot(x_data,y_data, label='Reimers (1975)',linewidth=others_lw, alpha␣
↪= others_alpha)

    form_vs_fed_fit =␣
↪linear_fit_1var(stacked_logMdotData,stacked_Reimers_log_mdot_2-stacked_logMdotData)
    # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
    form_vs_fed_Coeffs = form_vs_fed_fit[0]
    #x_data = obsv_data_range
    #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
    x_data = obsv_data_range
    y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
    for i in range(0, len(star_data)):
        plot.scatter(logMdotData[i],Reimers_log_mdot_2[i]-logMdotData[i],␣
↪s=marker_size, label='Reimers (1977)'+' ('+spec_mode_label_list[i]+')',␣
↪color=color_list[3], marker=shape_list[i])
```

```
   print('Reimers 1977 Results:',form_vs_fed_fit)
   #plot.plot(x_data,y_data, label='Reimers (1977)',linewidth=others_lw, alpha␣
↪= others_alpha)

   if spec_mode_label == 'CO' or spec_mode_label == 'COx' or spec_mode_label␣
↪== 'CO and COx':
       form_vs_fed_fit =␣
↪linear_fit_1var(stacked_logMdotData,stacked_VW_log_mdot-stacked_logMdotData)
       # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
       form_vs_fed_Coeffs = form_vs_fed_fit[0]
       #x_data = obsv_data_range
       #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
       x_data = obsv_data_range
       y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data

       for i in range(0, len(star_data)):
           plot.
↪scatter(logMdotData[i],VW_log_mdot[i]-logMdotData[i],s=marker_size,␣
↪label='Vassiliadis and Wood'+' ('+spec_mode_label_list[i]+')',␣
↪color=color_list[4], marker=shape_list[i])
       print('VW Results:',form_vs_fed_fit)
       #plot.plot(x_data,y_data, label='Vassiliadis and␣
↪Wood',linewidth=others_lw, alpha = others_alpha)

   form_vs_fed_fit =␣
↪linear_fit_1var(stacked_logMdotData,stacked_Bloecker_log_mdot-stacked_logMdotData)
   # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
   form_vs_fed_Coeffs = form_vs_fed_fit[0]
   #x_data = obsv_data_range
   #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
   x_data = obsv_data_range
   y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
   for i in range(0, len(star_data)):
       plot.scatter(logMdotData[i],Bloecker_log_mdot[i]-logMdotData[i],␣
↪s=marker_size, label=u'Blöcker'+' ('+spec_mode_label_list[i]+')',␣
↪color=color_list[5], marker=shape_list[i])
   print('Bloecker Results:',form_vs_fed_fit)
   #plot.plot(x_data,y_data, label=u'Blöcker',linewidth=others_lw, alpha =␣
↪others_alpha)

   form_vs_fed_fit =␣
↪linear_fit_1var(stacked_logMdotData,stacked_Wachter_log_mdot-stacked_logMdotData)
   # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
   form_vs_fed_Coeffs = form_vs_fed_fit[0]
   #x_data = obsv_data_range
   #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
```

```
    x_data = obsv_data_range
    y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
    for i in range(0, len(star_data)):
        plot.scatter(logMdotData[i],Wachter_log_mdot[i]-logMdotData[i],␣
↪s=marker_size, label='Wachter et al.'+' ('+spec_mode_label_list[i]+')',␣
↪color=color_list[6], marker=shape_list[i])
    print('Wachter et al. Results:',form_vs_fed_fit)
    #plot.plot(x_data,y_data, label='Wachter et al.',linewidth=others_lw, alpha␣
↪= others_alpha)

    form_vs_fed_fit =␣
↪linear_fit_1var(stacked_logMdotData,stacked_van_Loon_log_mdot-stacked_logMdotData)
    # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
    form_vs_fed_Coeffs = form_vs_fed_fit[0]
    #x_data = obsv_data_range
    #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
    x_data = obsv_data_range
    y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
    for i in range(0, len(star_data)):
        plot.scatter(logMdotData[i],van_Loon_log_mdot[i]-logMdotData[i],␣
↪s=marker_size, label=u'van Loon'+' ('+spec_mode_label_list[i]+')',␣
↪color=color_list[7], marker=shape_list[i])
    print('van Loon Results:',form_vs_fed_fit)
    #plot.plot(x_data,y_data, label=u'van Loon',linewidth=others_lw, alpha =␣
↪others_alpha)

    form_vs_fed_fit =␣
↪linear_fit_1var(stacked_logMdotData,stacked_Schroeder_Cuntz_log_mdot-stacked_logMdotData)
    # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
    form_vs_fed_Coeffs = form_vs_fed_fit[0]
    #x_data = obsv_data_range
    #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
    x_data = obsv_data_range
    y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
    for i in range(0, len(star_data)):
        plot.scatter(logMdotData[i],Schroeder_Cuntz_log_mdot[i]-logMdotData[i],␣
↪s=marker_size, label=u'Schröder and Cuntz'+' ('+spec_mode_label_list[i]+')',␣
↪color=color_list[8], marker=shape_list[i])
    print('Schroeder and Cuntz Results:',form_vs_fed_fit)
    #plot.plot(x_data,y_data, label=u'Schröder and Cuntz',linewidth=others_lw,␣
↪alpha = others_alpha)

    form_vs_fed_fit =␣
↪linear_fit_1var(stacked_logMdotData,stacked_de_Beck_log_mdot-stacked_logMdotData)
    # Returns [(b=intercept, m=slope), r^2] for equation x = my + b
    form_vs_fed_Coeffs = form_vs_fed_fit[0]
```

```
   #x_data = obsv_data_range
   #y_data = (x_data - form_vs_fed_Coeffs[0])/form_vs_fed_Coeffs[1]
   x_data = obsv_data_range
   y_data = form_vs_fed_Coeffs[0] + form_vs_fed_Coeffs[1]*x_data
   for i in range(0, len(star_data)):
       plot.scatter(logMdotData[i],de_Beck_log_mdot[i]-logMdotData[i],␣
↪s=marker_size, label='de Beck et al.'+' ('+spec_mode_label_list[i]+')',␣
↪color=color_list[9], marker=shape_list[i])
   print('de Beck et al. Results:',form_vs_fed_fit)
   #plot.plot(x_data,y_data, label='de Beck et al.',linewidth=others_lw, alpha␣
↪= others_alpha)

   plot.xlim(-8,-4)
   plot.ylim(-5,3)

   plot.title(spec_mode_label)
   plot.plot([-12,-2],[0,0],color = 'black', alpha =␣
↪1,linestyle='--',linewidth=1.5)
   plot.ylabel(r'$\log \dot{M},\ \mathrm{Calc.}-\log \dot{M},\ \mathrm{Obsv.
↪}$')
   plot.xlabel(r'$\log \dot{M},\ \mathrm{Obsv.}$')
   plot.tight_layout()
   #plot.legend(ncol=2,fontsize='small',loc=3)
   plot.savefig('Final_Fit/Form_Comparison_2_'+spec_mode_label+'.png')
   plot.savefig('Final_Fit/Form_Comparison_2_'+spec_mode_label+'.pdf')
   plot.show()
   plot.close()

   print('\n','_____________','\n')



   if spec_mode_label == 'C0' or spec_mode_label == 'C0x' or spec_mode_label␣
↪== 'C0 and C0x':
       residual_list = []
       for i in range(0, len(star_data)):
           residual_list.append([our_log_mdot[i]-logMdotData[i],␣
↪dz_log_mdot[i]-logMdotData[i], Reimers_log_mdot[i]-logMdotData[i],
                   Reimers_log_mdot_2[i]-logMdotData[i],␣
↪VW_log_mdot[i]-logMdotData[i], Bloecker_log_mdot[i]-logMdotData[i],
                   Wachter_log_mdot[i]-logMdotData[i],␣
↪van_Loon_log_mdot[i]-logMdotData[i],␣
↪Schroeder_Cuntz_log_mdot[i]-logMdotData[i],
                   de_Beck_log_mdot[i]-logMdotData[i]])
```

```
        stacked_residual_list = [stacked_our_log_mdot-stacked_logMdotData,␣
↪stacked_dz_log_mdot-stacked_logMdotData,␣
↪stacked_Reimers_log_mdot-stacked_logMdotData,
                    stacked_Reimers_log_mdot_2-stacked_logMdotData,␣
↪stacked_VW_log_mdot-stacked_logMdotData,␣
↪stacked_Bloecker_log_mdot-stacked_logMdotData,
                    stacked_Wachter_log_mdot-stacked_logMdotData,␣
↪stacked_van_Loon_log_mdot-stacked_logMdotData,␣
↪stacked_Schroeder_Cuntz_log_mdot-stacked_logMdotData,
                    stacked_de_Beck_log_mdot-stacked_logMdotData]
        frame = pandas.DataFrame({"Fit to Dust": stacked_residual_list[0], "Fit␣
↪to Deathzone": stacked_residual_list[1],
                              "Reimers (1975)": stacked_residual_list[2],␣
↪"Reimers (1977)": stacked_residual_list[3],
                              "Vassiliadis and Wood": stacked_residual_list[4],␣
↪u"Blöcker": stacked_residual_list[5],
                              "Wachter et al.": stacked_residual_list[6], "van␣
↪Loon": stacked_residual_list[7],
                              u"Schröder and Cuntz": stacked_residual_list[8],␣
↪"de Beck et al.": stacked_residual_list[9]})
        boxplot_labels = ["Fit to Dust", "Fit to Deathzone", "Reimers␣
↪(1975)","Reimers (1977)","Vassiliadis and Wood",
             u"Blöcker", "Wachter et al.", "van Loon", u"Schröder and Cuntz",␣
↪"de Beck et al."]
    else:
        residual_list = []
        for i in range(0, len(star_data)):
            residual_list.append([our_log_mdot[i]-logMdotData[i],␣
↪dz_log_mdot[i]-logMdotData[i], Reimers_log_mdot[i]-logMdotData[i],
                    Reimers_log_mdot_2[i]-logMdotData[i],␣
↪Bloecker_log_mdot[i]-logMdotData[i],
                    Wachter_log_mdot[i]-logMdotData[i],␣
↪van_Loon_log_mdot[i]-logMdotData[i],␣
↪Schroeder_Cuntz_log_mdot[i]-logMdotData[i],
                    de_Beck_log_mdot[i]-logMdotData[i]])
        stacked_residual_list = [stacked_our_log_mdot-stacked_logMdotData,␣
↪stacked_dz_log_mdot-stacked_logMdotData,␣
↪stacked_Reimers_log_mdot-stacked_logMdotData,
                    stacked_Reimers_log_mdot_2-stacked_logMdotData,␣
↪stacked_Bloecker_log_mdot-stacked_logMdotData,
                    stacked_Wachter_log_mdot-stacked_logMdotData,␣
↪stacked_van_Loon_log_mdot-stacked_logMdotData,␣
↪stacked_Schroeder_Cuntz_log_mdot-stacked_logMdotData,
                    stacked_de_Beck_log_mdot-stacked_logMdotData]
        frame = pandas.DataFrame({"Fit to Dust": stacked_residual_list[0], "Fit␣
↪to Deathzone": stacked_residual_list[1],
```

```
                              "Reimers (1975)": stacked_residual_list[2],␣
↪"Reimers (1977)": stacked_residual_list[3],
                              u"Blöcker": stacked_residual_list[4],
                             "Wachter et al.": stacked_residual_list[5], "van␣
↪Loon": stacked_residual_list[6],
                            u"Schröder and Cuntz": stacked_residual_list[7],␣
↪"de Beck et al.": stacked_residual_list[8]})
      boxplot_labels = ["Fit to Dust", "Fit to Deathzone", "Reimers␣
↪(1975)","Reimers (1977)",
           u"Blöcker", "Wachter et al.", "van Loon", u"Schröder and Cuntz",␣
↪"de Beck et al."]
      boxplot_pal = {"Fit to Dust":color_list[0], "Fit to Deathzone":
↪color_list[1], "Reimers (1975)":color_list[2],"Reimers (1977)":color_list[3],
           u"Blöcker":color_list[5], "Wachter et al.":color_list[6], "van␣
↪Loon":color_list[7], u"Schröder and Cuntz":color_list[8], "de Beck et al.":
↪color_list[9]}


   # Type 2 Diagnostic: individual scatter plots
   for i in range(0, len(stacked_residual_list)):
       if spec_mode_label == 'C0' or spec_mode_label == 'C0x' or␣
↪spec_mode_label == 'C0 and C0x':
           for k in range(0, len(star_data)):
               print('len(logMdotData[k]):',len(logMdotData[k]))
               print('len(residual_list[k][i]):',len(residual_list[k][i]))
               plot.scatter(logMdotData[k],residual_list[k][i], s=marker_size,␣
↪label=boxplot_labels[i],color=color_list[i], marker=shape_list[k])
      else:
           if i < 4:
               for k in range(0, len(star_data)):
                   plot.scatter(logMdotData[k],residual_list[k][i], s=2,␣
↪label=boxplot_labels[i]+'␣
↪('+spec_mode_label_list[i]+')',color=color_list[i], marker=shape_list[k])
           else:
               for k in range(0, len(star_data)):
                   plot.scatter(logMdotData[k],residual_list[k][i], s=2,␣
↪label=boxplot_labels[i]+'␣
↪('+spec_mode_label_list[i]+')',color=color_list[i+1], marker=shape_list[k])
      plot.xlim(-8,-4)
      plot.ylim(-5,3)
      plot.title(spec_mode_label)
      plot.plot([-12,-2],[0,0],color = 'black', alpha =␣
↪1,linestyle='--',linewidth=1.5)
      plot.ylabel(r'$\log \dot{M},\ \mathrm{Calc.}-\log \dot{M},\␣
↪\mathrm{Obsv.}$')
      plot.xlabel(r'$\log \dot{M},\ \mathrm{Obsv.}$')
```

```
        plot.tight_layout()
        plot.legend(ncol=2,fontsize='small',loc=3)
        plot.savefig('Final_Fit/
↪Form_Comparison_2_diagnostic_'+str(i)+spec_mode_label+'.png')
        plot.savefig('Final_Fit/
↪Form_Comparison_2_diagnostic_'+str(i)+spec_mode_label+'.pdf')
        plot.show()
        plot.close()
        print('\n','_____________','\n')


    # Type 3: Box and Whisker Plot
    if spec_mode_label == 'C0' or spec_mode_label == 'C0x' or spec_mode_label␣
↪== 'C0 and C0x':
        sns.boxplot(data=frame,orient='h')
    else:
        sns.boxplot(data=frame,orient='h',palette = boxplot_pal)

    current_limits = plot.gca().get_ylim()
    #breathing_room = np.abs(current_limits[0]-current_limits[1])/10
    plot.vlines(0,current_limits[0]+1.5,current_limits[1]-1.
↪5,colors='black',linestyles='--')
    plot.ylim(current_limits[0]+0.5,current_limits[1]-0.5)
    plot.xlabel(r'$\log \dot{M}_{\mathrm{calc.}} - \log \dot{M}_{\mathrm{obsv.
↪}}$')
    plot.title(spec_mode_label)
    plot.tight_layout()
    plot.savefig('Final_Fit/Form_Comparison_3_'+spec_mode_label+'.png')
    plot.savefig('Final_Fit/Form_Comparison_3_'+spec_mode_label+'.pdf')
    plot.show()
    plot.close()

    # Type 4: Boxes around residuals

    print('\n','_____________','\n')
```

[380]:
```
mdot_formula_comparison (M0StarData,spec_mode_label='M0')
```

```
log Mdot ~ logL + logP:
LRM form: logMdot = -12.697730017492958  +  1.155822471134391 logL + logR -
2.3769303131009343 logM
LM form: logMdot = -12.938730017492958  +  1.845822471134391 logL -
2.627930313100934 logM

--------------
log Mdot ~ logL + logM
LM form: logMdot = -34.975963392304195  +  8.131127742677283 logL -
13.698081169196328 logM
```

```
--------------
Reimers 1975 LRM form: logMdot = -12.397940008672037 + logL + logR - logM
Reimers 1975 LM form: logMdot = -12.638940008672037 + 1.69 logL -  1.251 logM
Calc vs Obsv Slope: 0.13967368557362794
Calc vs Obsv R^2: 0.09570462768042853
SDev of Residual: 0.37602575504755364
Mean distance from 1-to-1 line: 0.35232982740614965

--------------
Reimers 1977 LRM form: logMdot = -12.853871964321762 + logL + logR - logM
Reimers 1977 LM form: logMdot = -13.094871964321761 + 1.69 logL -  1.251 logM
Calc vs Obsv Slope: 0.13967368557362772
Calc vs Obsv R^2: 0.09570462768042898
SDev of Residual: 0.3760257550475536
Mean distance from 1-to-1 line: 0.704312240980575

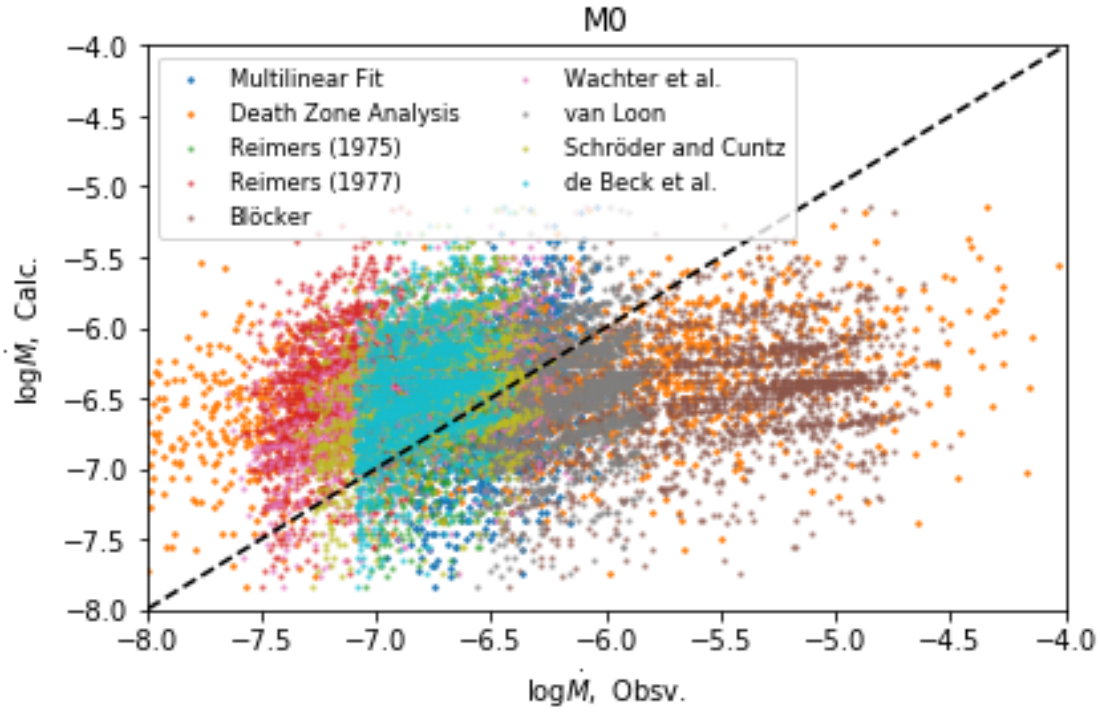
--------------
VW LRM form: N/A
VW LM form apprx.: -31.00304107225585 + 6.1486387143435515 logL -
5.675513835101315 logM
Calc vs Obsv Slope: 0.5223443923410124
Calc vs Obsv R^2: 0.11694853859090815
SDev of Residual: 0.5915955061807282
Mean distance from 1-to-1 line: 2.922400465620822

--------------
Bloecker LRM form: logMdot = -20.713992877920525 +3.7logL + logR - 3.1logM
Bloecker LM form: logMdot = -20.954992877920525 + 4.390000000000001 logL -
3.351 logM
Calc vs Obsv Slope: 0.3671589357655025
Calc vs Obsv R^2: 0.10042022562972341
SDev of Residual: 0.49593109555797443
Mean distance from 1-to-1 line: 0.9569528353109681

--------------
Wachter LRM form: logMdot = 0.25966578699999976  +  0.259871057 logL -
1.21111383 logM + logR
Wachter LM form: logMdot = 0.018665786999999767  +  0.949871057 logL -
1.4621138299999998 logM
Calc vs Obsv Slope: 0.3524874174598196
Calc vs Obsv R^2: 0.18248234803958863
SDev of Residual: 0.38633802455905064
Mean distance from 1-to-1 line: 0.40795719551987925

--------------
van Loon LRM form: logMdot = 6.03441  +  0.9584999999999999 logL - 0.53965 logM
+ logR
van Loon LM form: logMdot = 5.793410000000001  +  1.6484999999999999 logL -
0.79065 logM
Calc vs Obsv Slope: 0.11788282349807896
Calc vs Obsv R^2: 0.052366446508107956
SDev of Residual: 0.39682570644619375
Mean distance from 1-to-1 line: 0.3959191389022616
```

```
--------------
Schroeder and Cuntz LRM form: N/A
Schroeder and Cuntz LM form apprx.: -15.598510193224802 + 2.5093951937734835
logL - 2.0658231374487004 logM
Calc vs Obsv Slope: 0.21545052663429817
Calc vs Obsv R^2: 0.11237823898772759
SDev of Residual: 0.3875751727269456
Mean distance from 1-to-1 line: 0.40257204247056094
--------------
de Beck LRM form: N/A
de Beck LM form apprx.: -12.820601663992997 + 1.709621496183304 logL -
1.5780697004915836 logM
Calc vs Obsv Slope: 0.14523722128506208
Calc vs Obsv R^2: 0.11694853859090804
SDev of Residual: 0.3689217861230519
Mean distance from 1-to-1 line: 0.4209789405265968
--------------
```

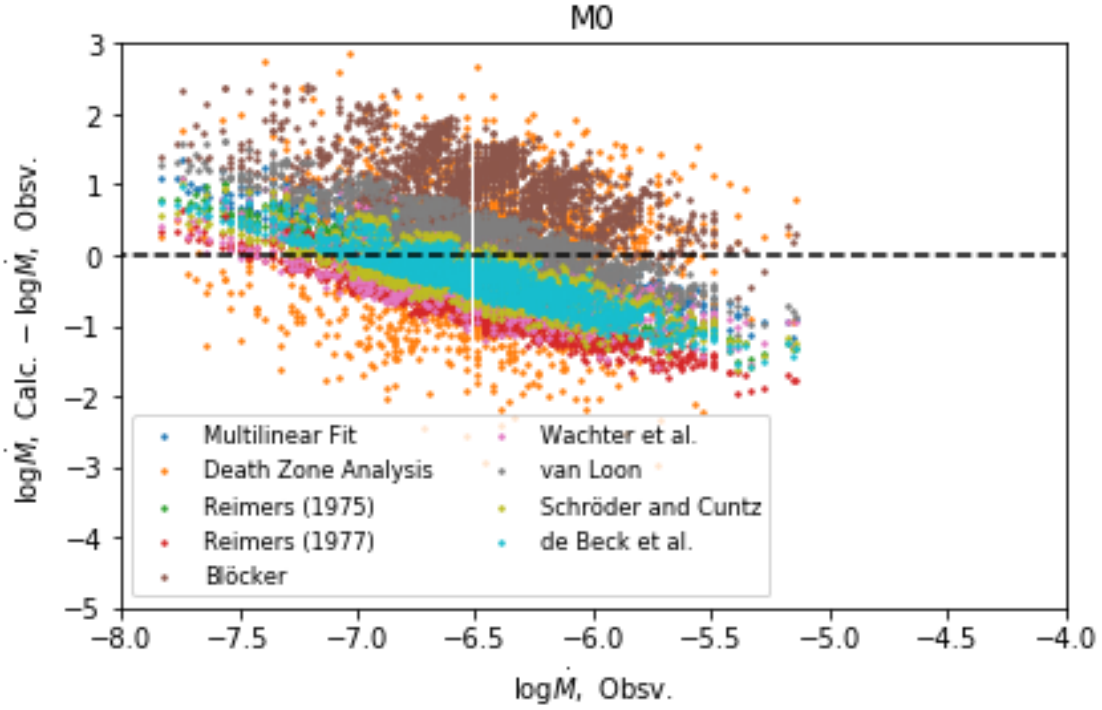


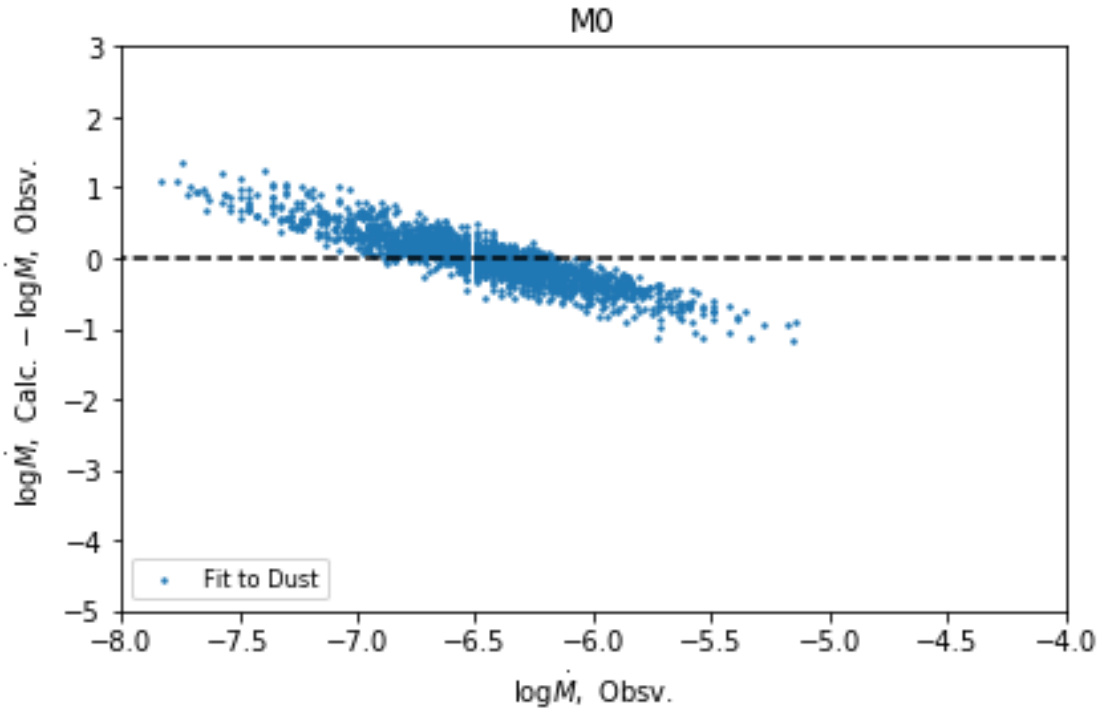

```
 --------------

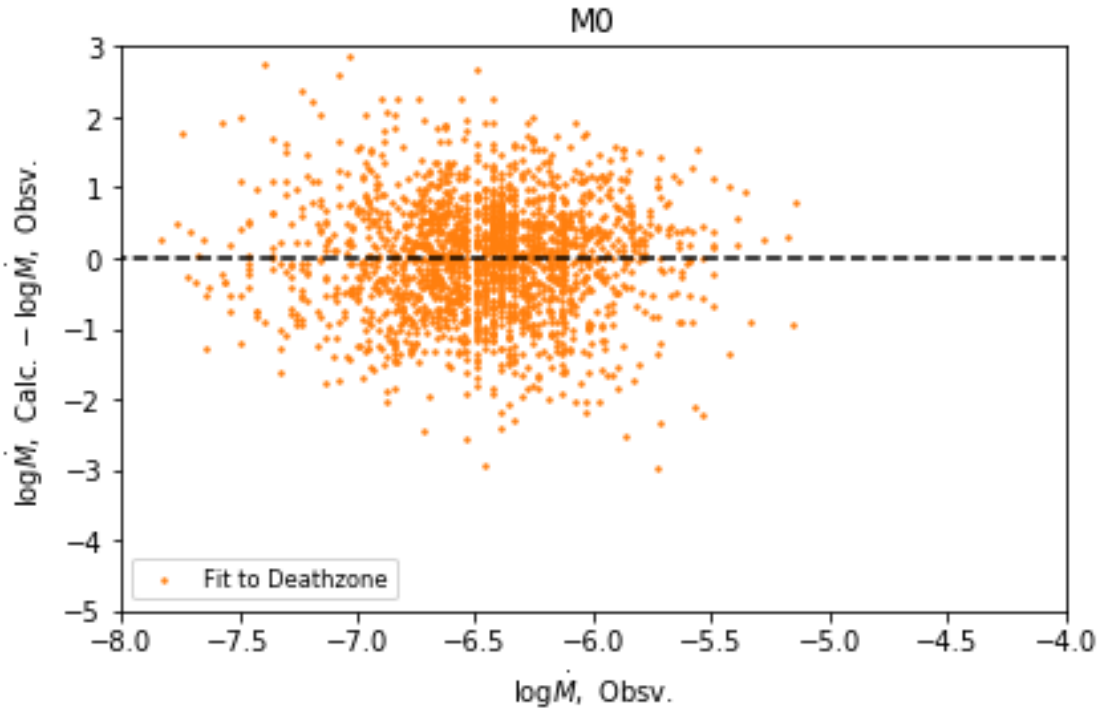
Dust-Fitting Results: (array([-5.12781483, -0.79363455]), 0.7936345478649771,
Intercept    0.059337
x            0.009167
```

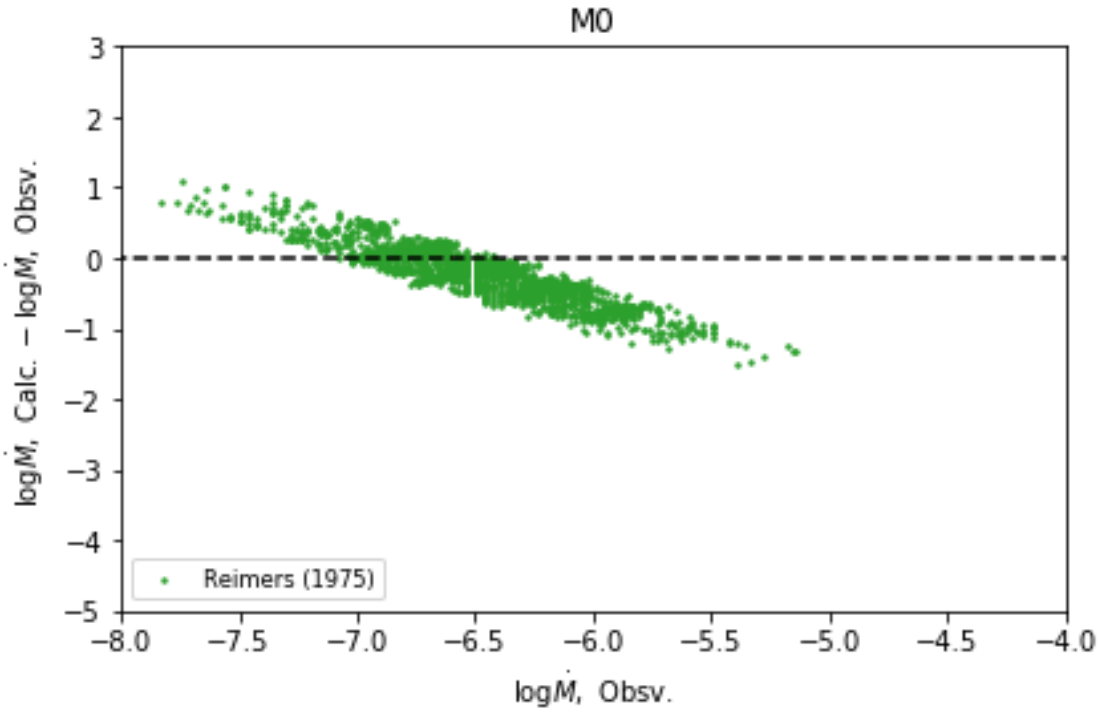
```
dtype: float64,            Intercept         x
Intercept   0.140418  0.021653
x           0.021653  0.003351)
Death Zone-Fitting Results: (array([2.44558732e-05, 3.78504811e-06]),
3.400058012914542e-12, Intercept    0.300979
x            0.046498
dtype: float64,            Intercept         x
Intercept   0.140418  0.021653
x           0.021653  0.003351)
Reimers 1975 Results: (array([-5.79241547, -0.86032631]), 0.8006108160021924,
Intercept    0.062951
x            0.009725
dtype: float64,            Intercept         x
Intercept   0.140418  0.021653
x           0.021653  0.003351)
Reimers 1977 Results: (array([-6.24834743, -0.86032631]), 0.8006108160021923,
Intercept    0.062951
x            0.009725
dtype: float64,            Intercept         x
Intercept   0.140418  0.021653
x           0.021653  0.003351)
Bloecker Results: (array([-3.14029839, -0.63284106]), 0.24904443862191805,
Intercept    0.161125
x            0.024892
dtype: float64,            Intercept         x
Intercept   0.140418  0.021653
x           0.021653  0.003351)
Wachter et al. Results: (array([-4.49983999, -0.64751258]), 0.4296271694125202,
Intercept    0.109391
x            0.016900
dtype: float64,            Intercept         x
Intercept   0.140418  0.021653
x           0.021653  0.003351)
van Loon Results: (array([-5.41700842, -0.88211718]), 0.755758751404028,
Intercept    0.073527
x            0.011359
dtype: float64,            Intercept         x
Intercept   0.140418  0.021653
x           0.021653  0.003351)
Schroeder and Cuntz Results: (array([-5.38001717, -0.78454947]),
0.6266989910640294, Intercept    0.088781
x            0.013716
dtype: float64,            Intercept         x
Intercept   0.140418  0.021653
x           0.021653  0.003351)
de Beck et al. Results: (array([-5.87093304, -0.85476278]), 0.821018272528812,
Intercept    0.058516
x            0.009040
```

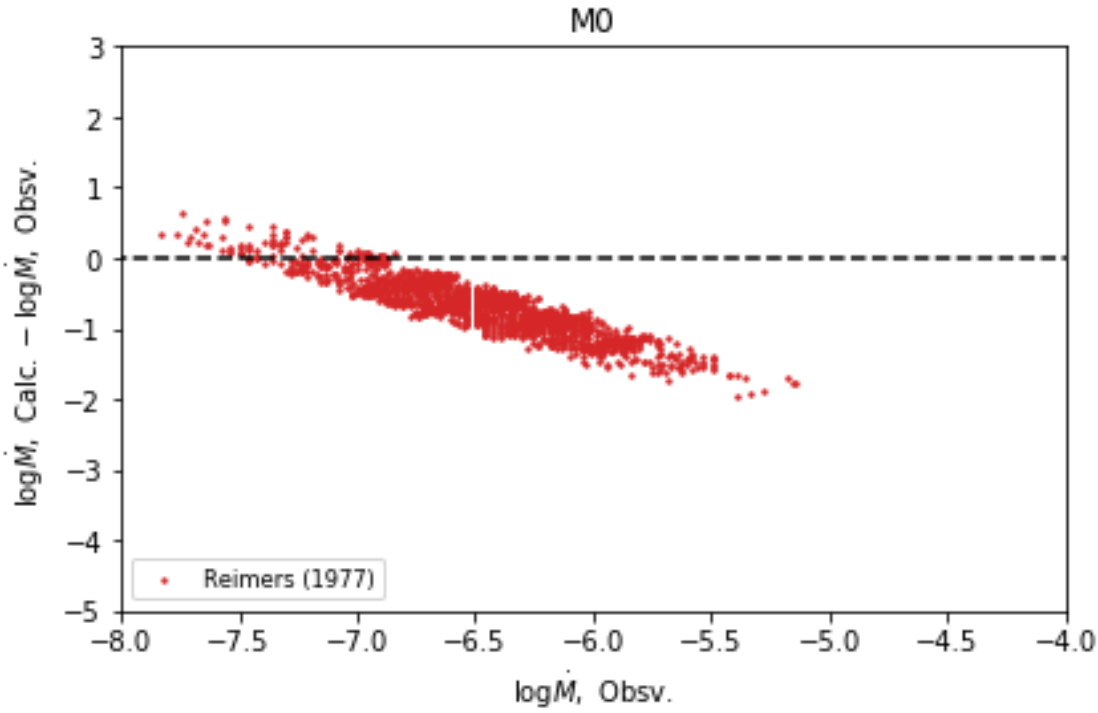
```
dtype: float64,             Intercept         x
Intercept    0.140418   0.021653
x            0.021653   0.003351)
```

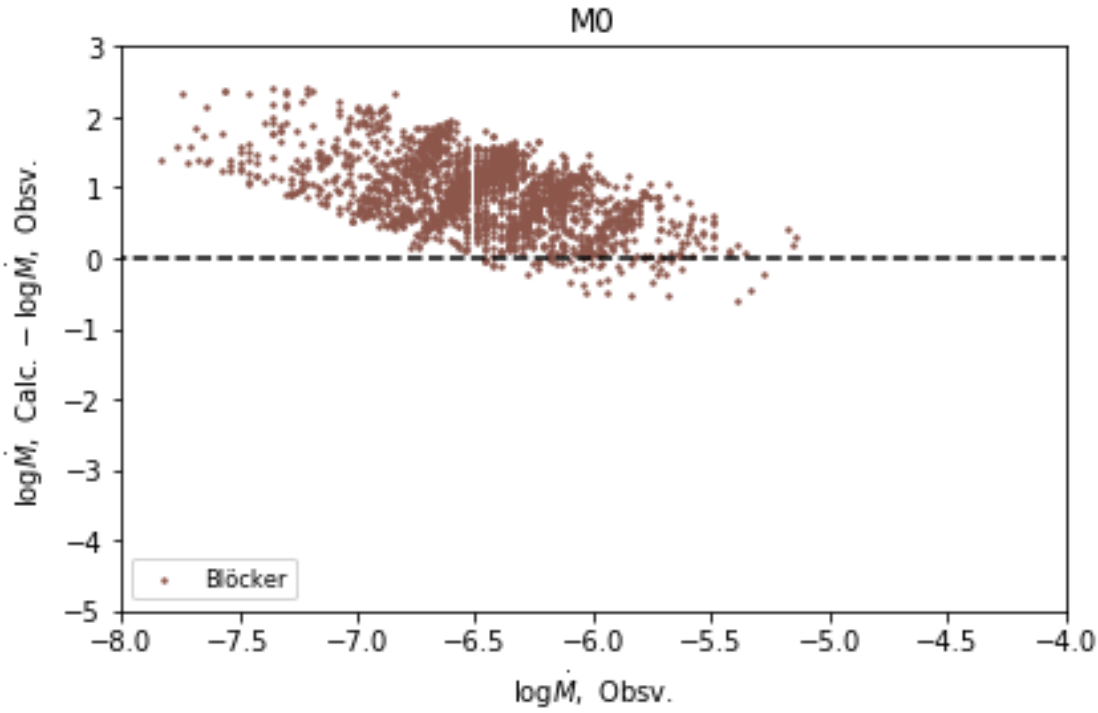


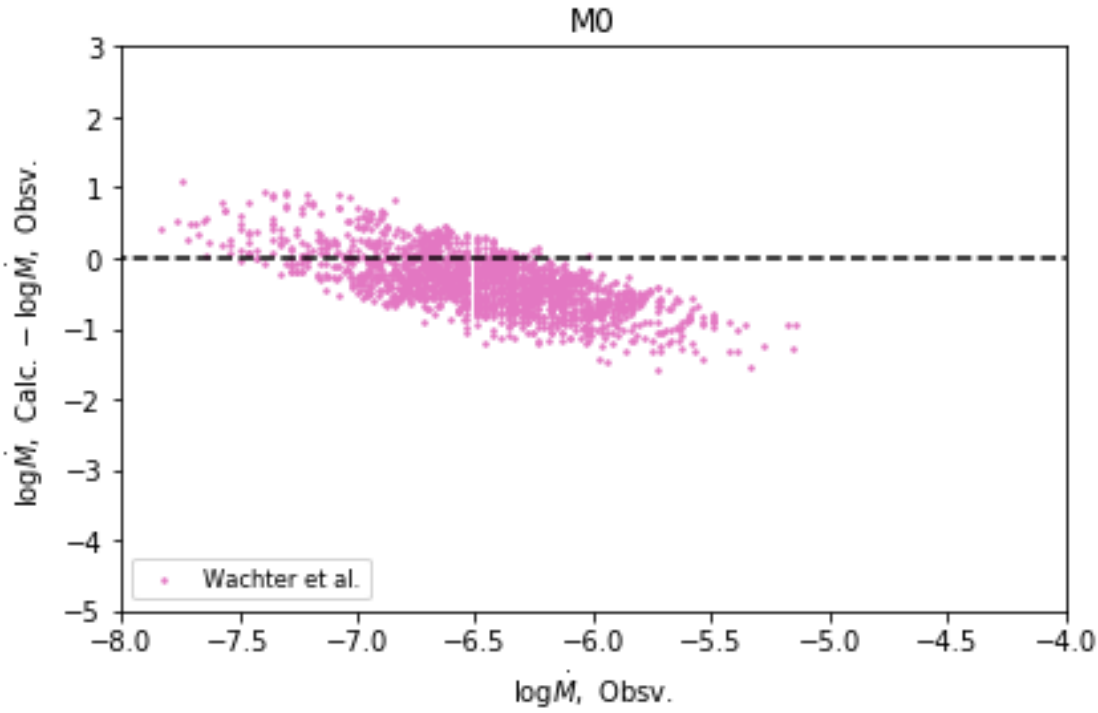

M0
3
2
1
0
−1
−2
−3
−4
−5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
log$\dot{M}$, Calc. − log$\dot{M}$, Obsv.
log$\dot{M}$, Obsv.
Fit to Dust

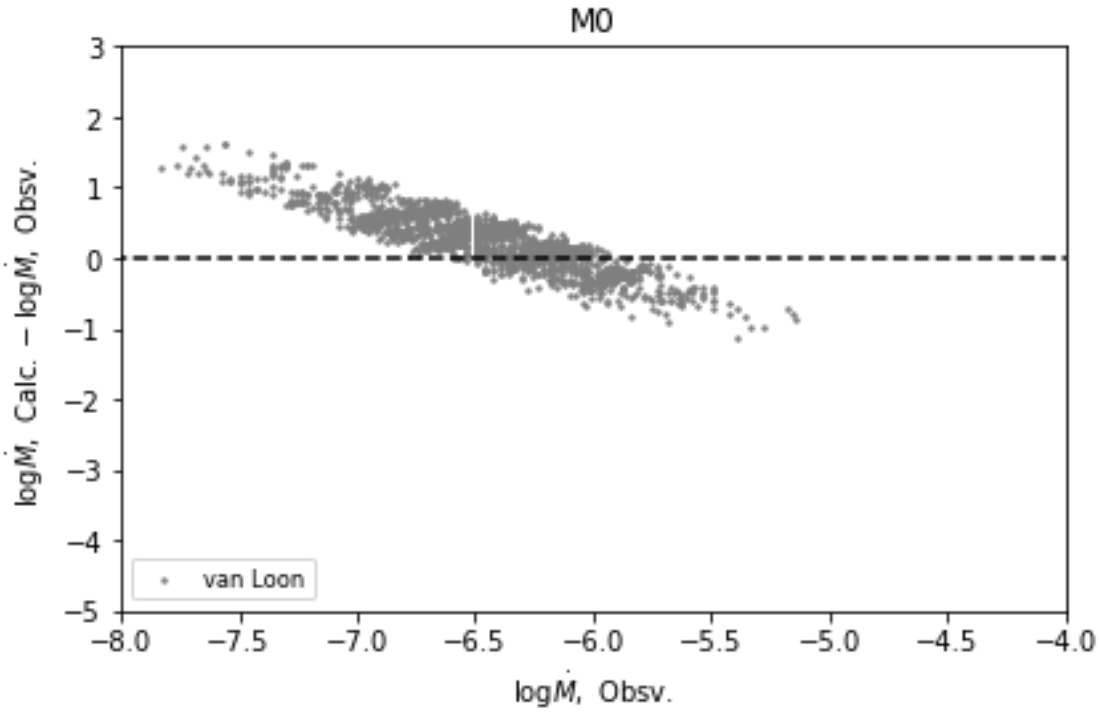

M0
3
2
1
0
−1
−2
−3
−4
−5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
log$\dot{M}$, Calc. − log$\dot{M}$, Obsv.
log$\dot{M}$, Obsv.
Fit to Deathzone

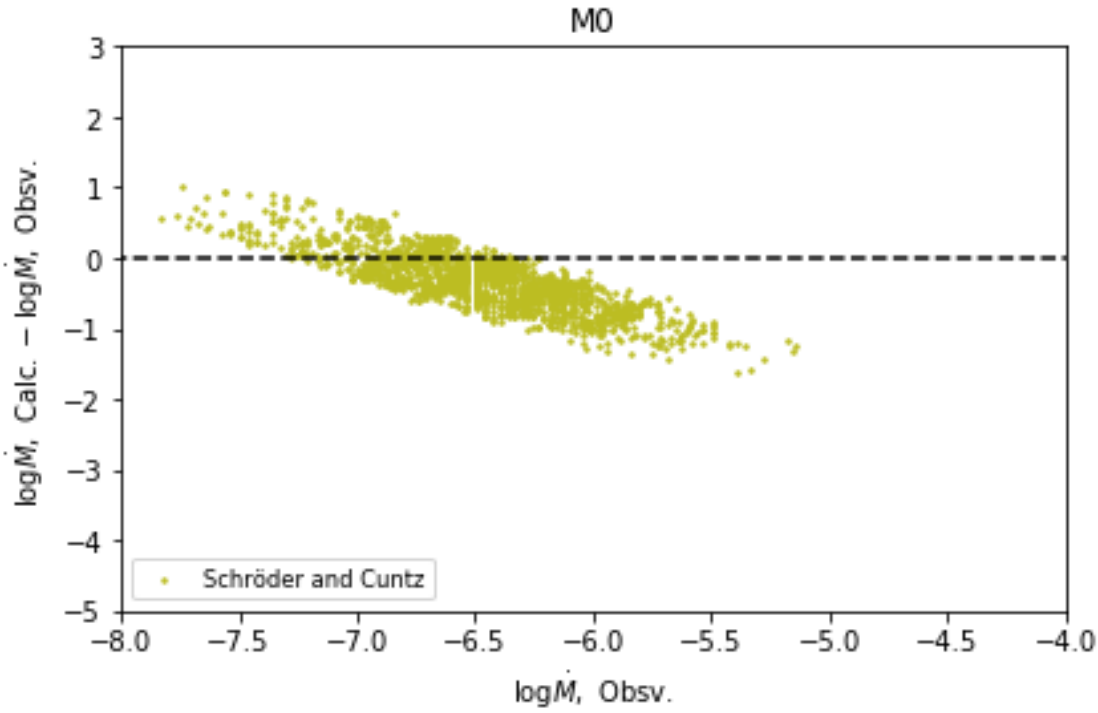

M0
3
2
1
0
−1
−2
−3
−4
−5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
log$\dot{M}$, Calc. − log$\dot{M}$, Obsv.
log$\dot{M}$, Obsv.
Reimers (1975)

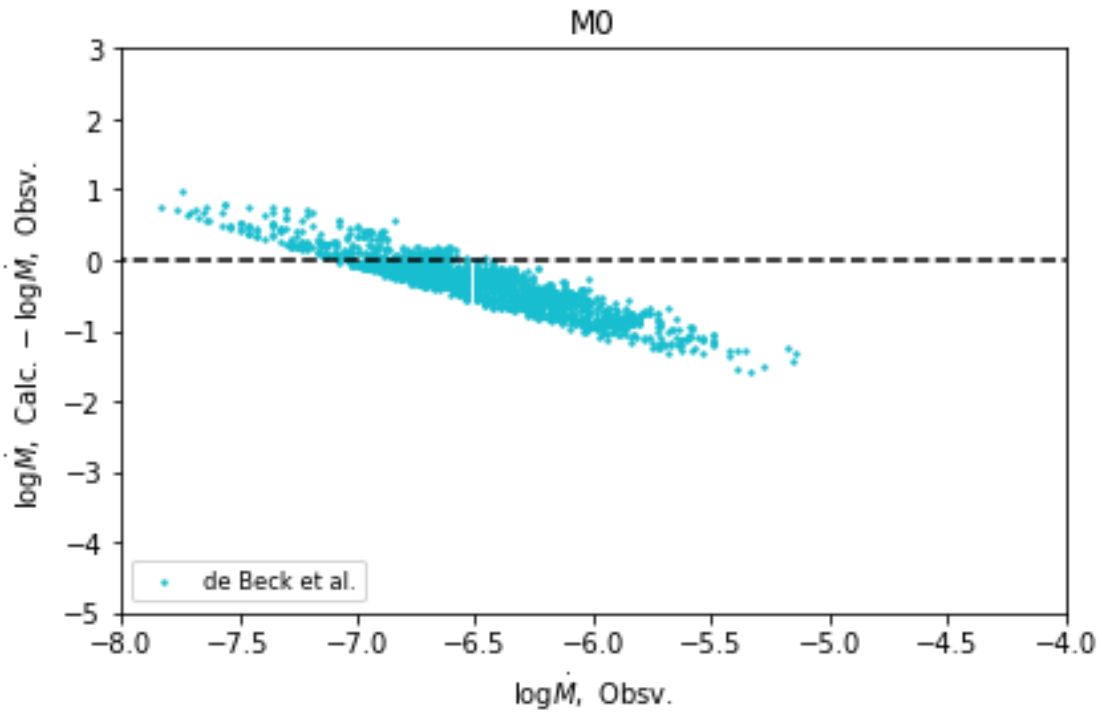

M0
3
2
1
0
−1
−2
−3
−4
−5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
log$\dot{M}$, Calc. − log$\dot{M}$, Obsv.
log$\dot{M}$, Obsv.
Reimers (1977)

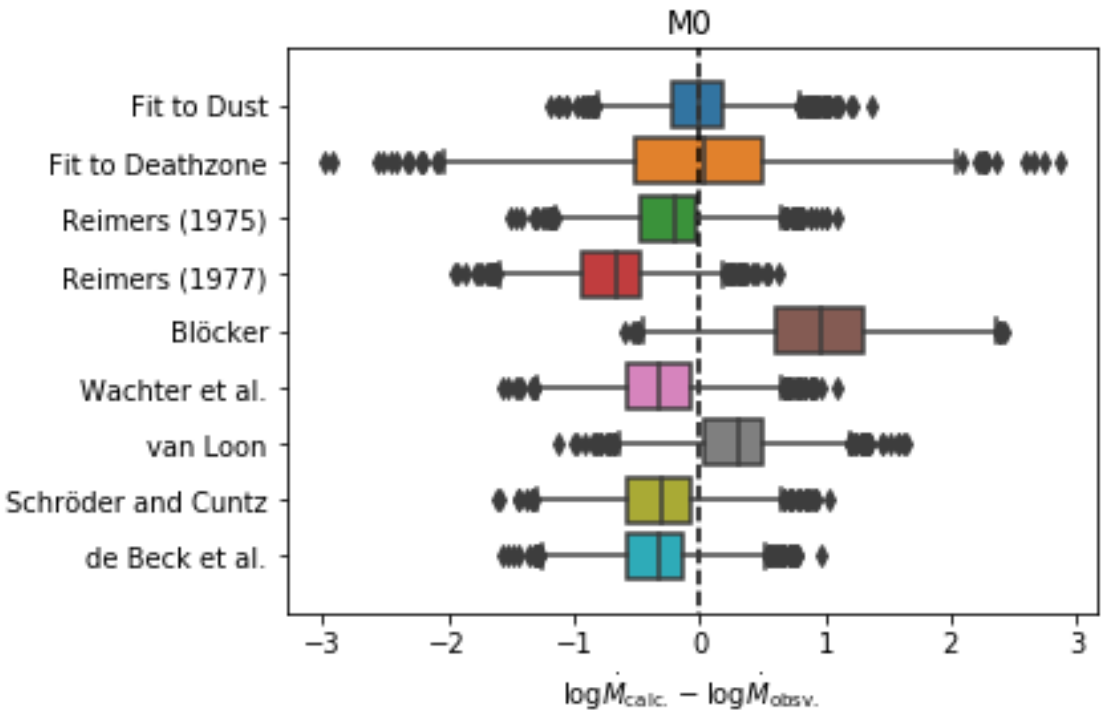

M0
3
2
1
0
−1
−2
−3
−4
−5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
log$\dot{M}$, Calc. − log$\dot{M}$, Obsv.
log$\dot{M}$, Obsv.
Blöcker


M0
3
2
1
0
−1
−2
−3
−4
−5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
log$\dot{M}$, Calc. − log$\dot{M}$, Obsv.
log$\dot{M}$, Obsv.
Wachter et al.


M0
3
2
1
0
−1
−2
−3
−4
−5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
van Loon
log$\dot{M}$, Calc. − log$\dot{M}$, Obsv.
log$\dot{M}$, Obsv.

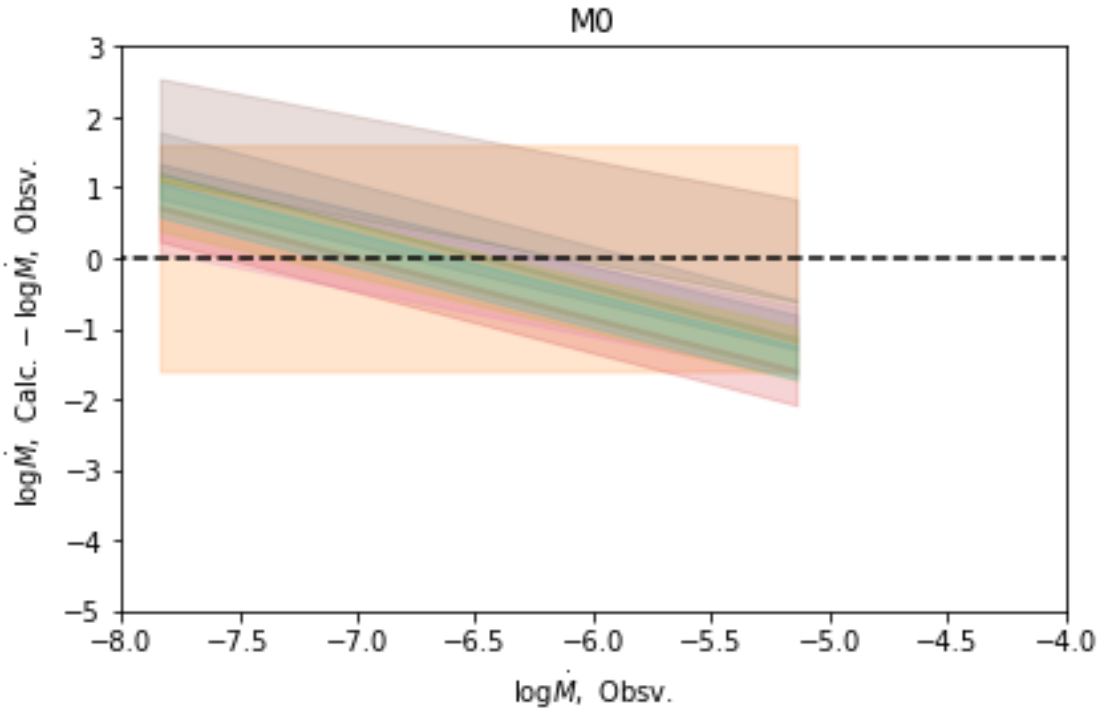

M0
3
2
1
0
−1
−2
−3
−4
−5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
log$\dot{M}$, Calc. − log$\dot{M}$, Obsv.
log$\dot{M}$, Obsv.
Schröder and Cuntz


M0
3
2
1
0
−1
−2
−3
−4
−5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
log$\dot{M}$, Calc. − log$\dot{M}$, Obsv.
log$\dot{M}$, Obsv.
de Beck et al.




```
Point List: [[-7.8366895110363135, 1.3395942444455295], [-7.8366895110363135,
0.8437111625216405], [-5.137719506700295, -1.298284676570706],
[-5.137719506700295, -0.8024015946468168]]
x-values: [-7.83668951 -5.13771951]
 y-values: [[ 1.33959424 -0.80240159]
 [ 0.84371116 -1.29828468]]
Point List: [[-7.8366895110363135, 1.6055739131134825], [-7.8366895110363135,
-1.6055843258607918], [-5.137719506700295, -1.6055741101294754],
[-5.137719506700295, 1.605584128844799]]
x-values: [-7.83668951 -5.13771951]
 y-values: [[ 1.60557391  1.60558413]
 [-1.60558433 -1.60557411]]
Point List: [[-7.8366895110363135, 1.2042626234990834], [-7.8366895110363135,
0.6951268419225336], [-5.137719506700295, -1.626868074655202],
[-5.137719506700295, -1.1177322930786522]]
x-values: [-7.83668951 -5.13771951]
 y-values: [[ 1.20426262 -1.11773229]
 [ 0.69512684 -1.62686807]]
Point List: [[-7.8366895110363135, 0.7483306678493564], [-7.8366895110363135,
0.23919488627280483], [-5.137719506700295, -2.082800030304929],
[-5.137719506700295, -1.5736642487283774]]
x-values: [-7.83668951 -5.13771951]
 y-values: [[ 0.74833067 -1.57366425]
 [ 0.23919489 -2.08280003]]
```

```
Point List: [[-7.8366895110363135, 2.5453851296257577], [-7.8366895110363135,
1.092775953162434], [-5.137719506700295, -0.6152430967185576],
[-5.137719506700295, 0.8373660797447662]]
x-values: [-7.83668951 -5.13771951]
 y-values: [[ 2.54538513  0.83736608]
 [ 1.09277595 -0.6152431 ]]
Point List: [[-7.8366895110363135, 1.0643434762154573], [-7.8366895110363135,
0.08468666943772263], [-5.137719506700295, -1.6629303682683738],
[-5.137719506700295, -0.6832735614906391]]
x-values: [-7.83668951 -5.13771951]
 y-values: [[ 1.06434348 -0.68327356]
 [ 0.08468667 -1.66293037]]
Point List: [[-7.8366895110363135, 1.7900122900055742], [-7.8366895110363135,
1.201727725560458], [-5.137719506700295, -1.1790800741278105],
[-5.137719506700295, -0.5907955096826942]]
x-values: [-7.83668951 -5.13771951]
 y-values: [[ 1.79001229 -0.59079551]
 [ 1.20172773 -1.17908007]]
Point List: [[-7.8366895110363135, 1.1408678659561744], [-7.8366895110363135,
0.3956390518424424], [-5.137719506700295, -1.721836443689205],
[-5.137719506700295, -0.9766076295754731]]
x-values: [-7.83668951 -5.13771951]
 y-values: [[ 1.14086787 -0.97660763]
 [ 0.39563905 -1.72183644]]
Point List: [[-7.8366895110363135, 1.0648611395533516], [-7.8366895110363135,
0.5902937883596584], [-5.137719506700295, -1.7166853122148664],
[-5.137719506700295, -1.2421179610211732]]
x-values: [-7.83668951 -5.13771951]
 y-values: [[ 1.06486114 -1.24211796]
 [ 0.59029379 -1.71668531]]
```



---

[381]:
```
mdot_formula_comparison (M1StarData,spec_mode_label='M1')
```

```
log Mdot ~ logL + logP:
LRM form: logMdot = -14.090671442964444  +  1.4758349824618766 logL + logR -
2.253190194588611 logM
LM form: logMdot = -14.331671442964444  +  2.1658349824618766 logL -
2.504190194588611 logM

--------------
log Mdot ~ logL + logM
LM form: logMdot = -42.82979371999287  +  10.152304120143375 logL -
20.474047210262466 logM

--------------
Reimers 1975 LRM form: logMdot = -12.397940008672037 + logL + logR - logM
Reimers 1975 LM form: logMdot = -12.638940008672037 + 1.69 logL -  1.251 logM
Calc vs Obsv Slope: 0.3162836681148926
Calc vs Obsv R^2: 0.30717890466589926
SDev of Residual: 0.36910257219732795
Mean distance from 1-to-1 line: 0.28127390404930247

--------------
Reimers 1977 LRM form: logMdot = -12.853871964321762 + logL + logR - logM
```

```
Reimers 1977 LM form: logMdot = -13.094871964321761 + 1.69 logL -  1.251 logM
Calc vs Obsv Slope: 0.31628366811489506
Calc vs Obsv R^2: 0.30717890466589914
SDev of Residual: 0.36910257219732795
Mean distance from 1-to-1 line: 0.4979062829965457

--------------
VW LRM form: N/A
VW LM form apprx.: -20.133427587831438 + 2.7597899069390692 logL -
2.398954272614154 logM
Calc vs Obsv Slope: 0.49165250647487246
Calc vs Obsv R^2: 0.2932939287664449
SDev of Residual: 0.40654951048212534
Mean distance from 1-to-1 line: 3.673990107455385

--------------
Bloecker LRM form: logMdot = -20.713992877920525 +3.7logL + logR - 3.1logM
Bloecker LM form: logMdot = -20.954992877920525 + 4.390000000000001 logL -
3.351 logM
Calc vs Obsv Slope: 0.8145493180261095
Calc vs Obsv R^2: 0.310172127723807
SDev of Residual: 0.5448052327121296
Mean distance from 1-to-1 line: 1.3438962630795532

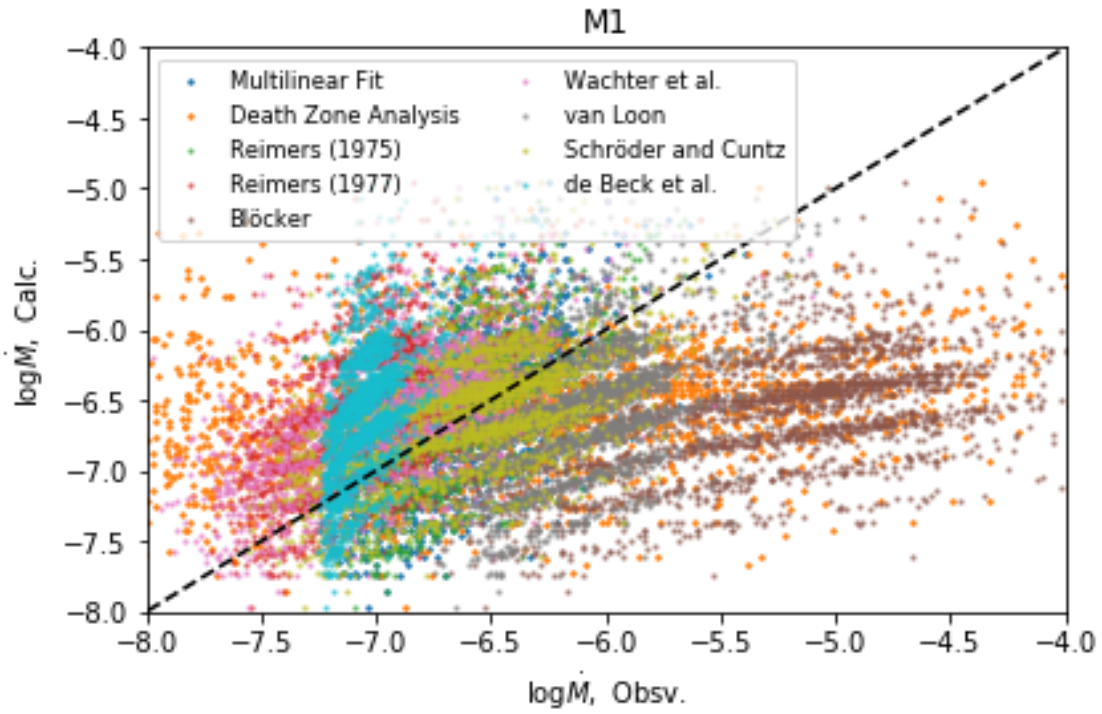
--------------
Wachter LRM form: logMdot = 1.0174071699999998  +  0.6107758700000001 logL -
1.5135153 logM + logR
Wachter LM form: logMdot = 0.7764071699999998  +  1.30077587 logL -
1.7645152999999998 logM
Calc vs Obsv Slope: 0.5132066479639752
Calc vs Obsv R^2: 0.34287497764363695
SDev of Residual: 0.38183720988504644
Mean distance from 1-to-1 line: 0.38314928019711036

--------------
van Loon LRM form: logMdot = 6.03441  +  0.9584999999999999 logL - 0.53965 logM
+ logR
van Loon LM form: logMdot = 5.793410000000001  +  1.6484999999999999 logL -
0.79065 logM
Calc vs Obsv Slope: 0.3383551074446642
Calc vs Obsv R^2: 0.2729844249336426
SDev of Residual: 0.3820760372037357
Mean distance from 1-to-1 line: 0.5525811486254371

--------------
Schroeder and Cuntz LRM form: N/A
Schroeder and Cuntz LM form apprx.: -15.704670632996207 + 2.5405106378568325
logL - 2.0956523635419346 logM
Calc vs Obsv Slope: 0.4602838685423597
Calc vs Obsv R^2: 0.3175278122244699
SDev of Residual: 0.38309938267570026
Mean distance from 1-to-1 line: 0.293324628144288

--------------
```

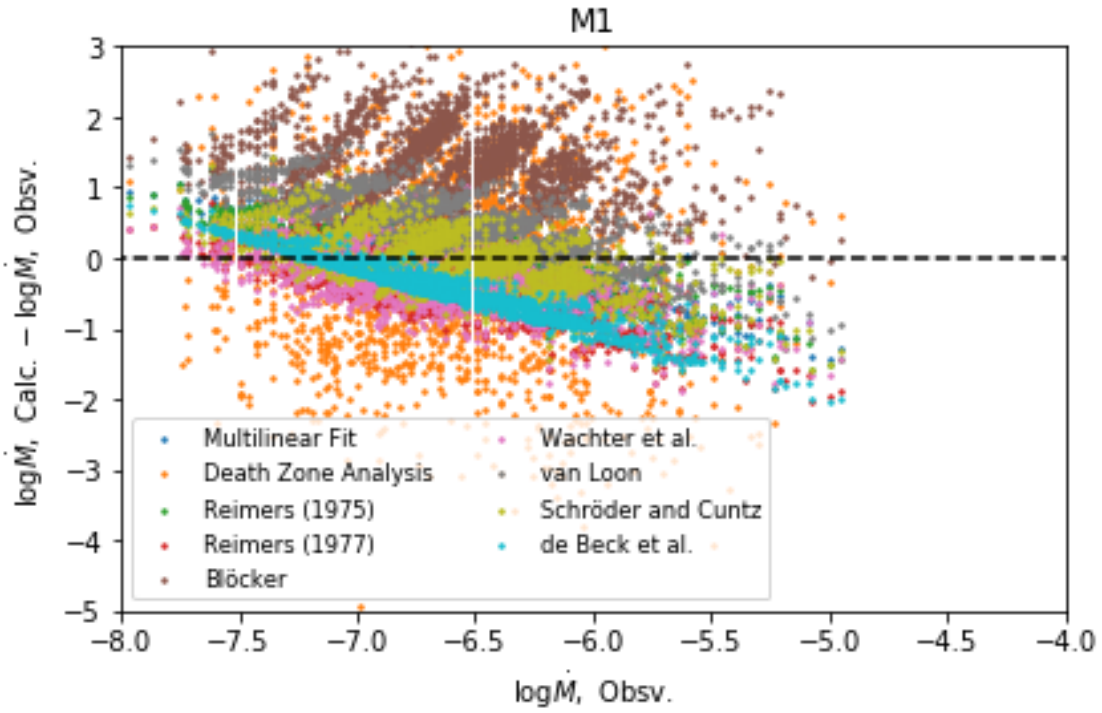
```
de Beck LRM form: N/A
de Beck LM form apprx.: -9.764657758377425 + 0.7584655969671098 logL -
0.580239418357421 logM
Calc vs Obsv Slope: 0.14155327211020763
Calc vs Obsv R^2: 0.29222824370570943
SDev of Residual: 0.3929291292596686
Mean distance from 1-to-1 line: 0.5119672201868225
--------------
```

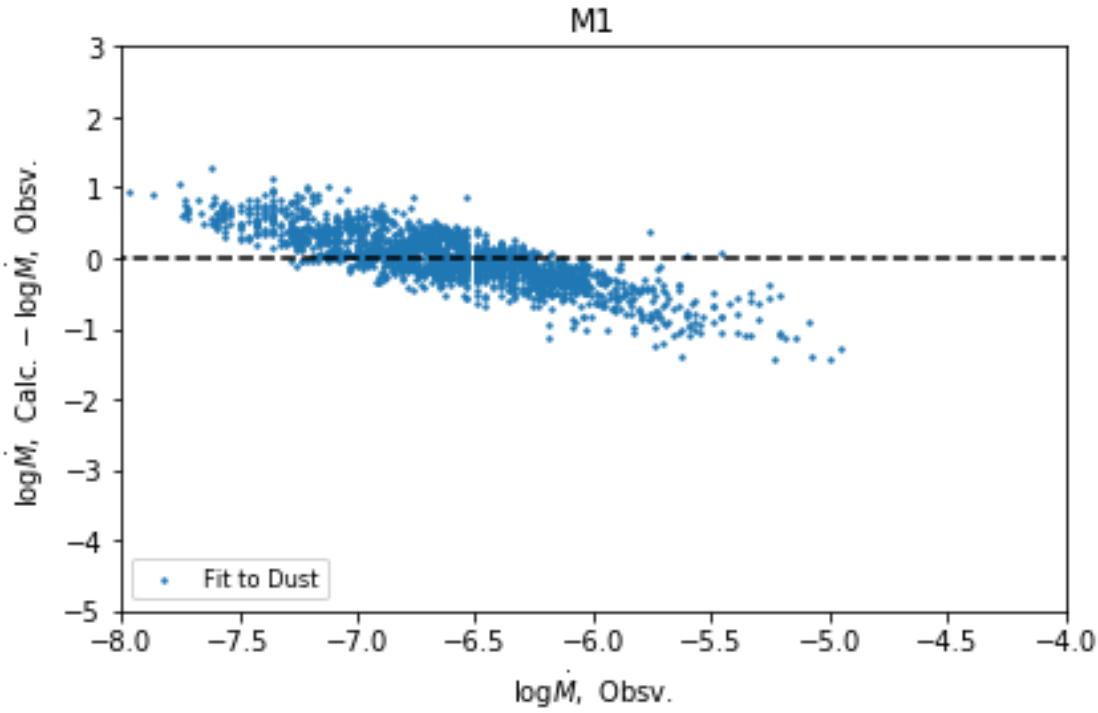


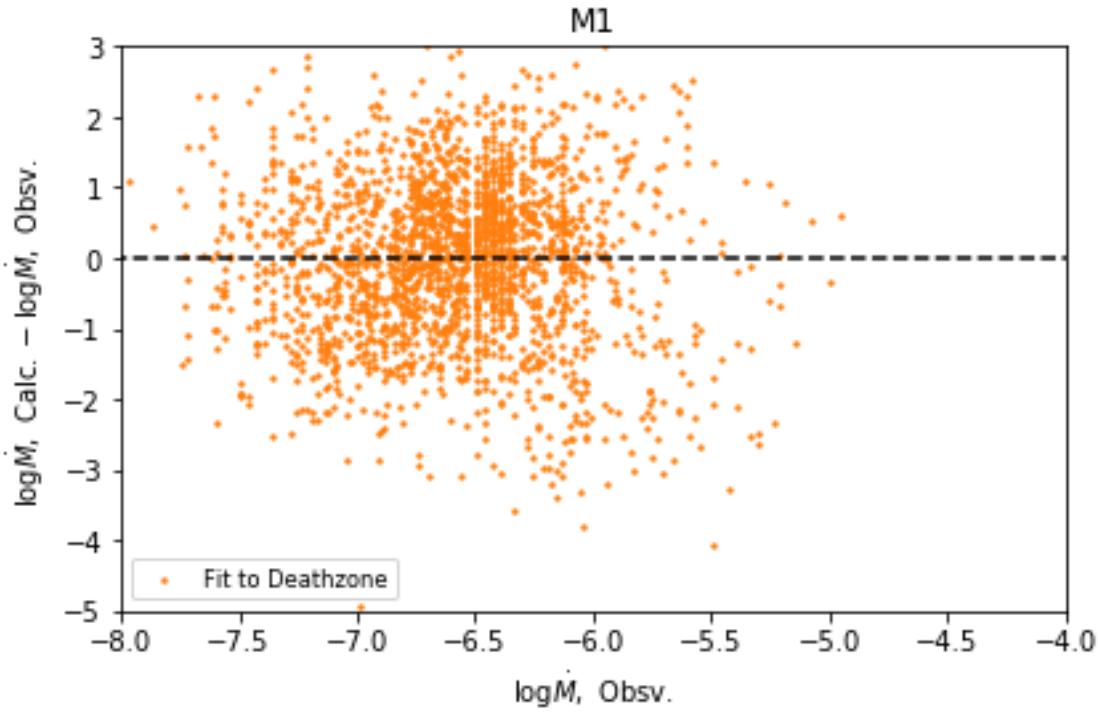

```
 --------------

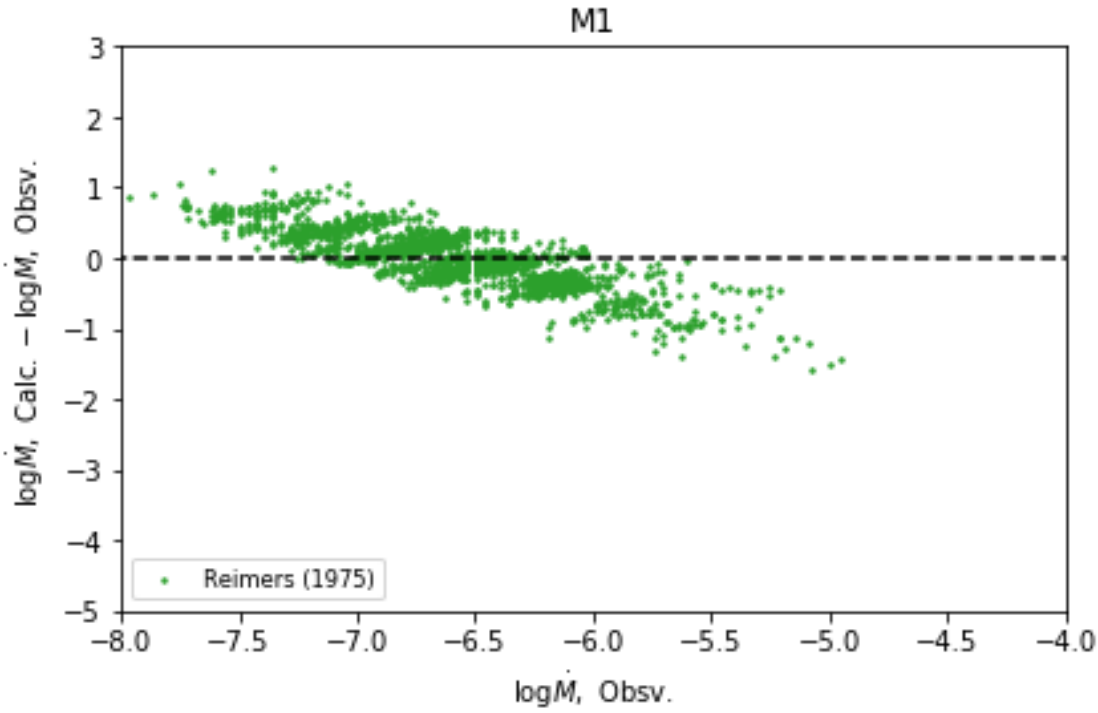
Dust-Fitting Results: (array([-4.32854911, -0.65707902]), 0.6570790219180425,
Intercept    0.067435
x            0.010214
dtype: float64,            Intercept         x
Intercept   0.102578  0.015501
x           0.015501  0.002353)
Death Zone-Fitting Results: (array([-2.15233777e-05, -3.26727493e-06]),
1.4868106745780096e-12, Intercept    0.380615
x            0.057647
dtype: float64,            Intercept         x
Intercept   0.102578  0.015501
x           0.015501  0.002353)
```

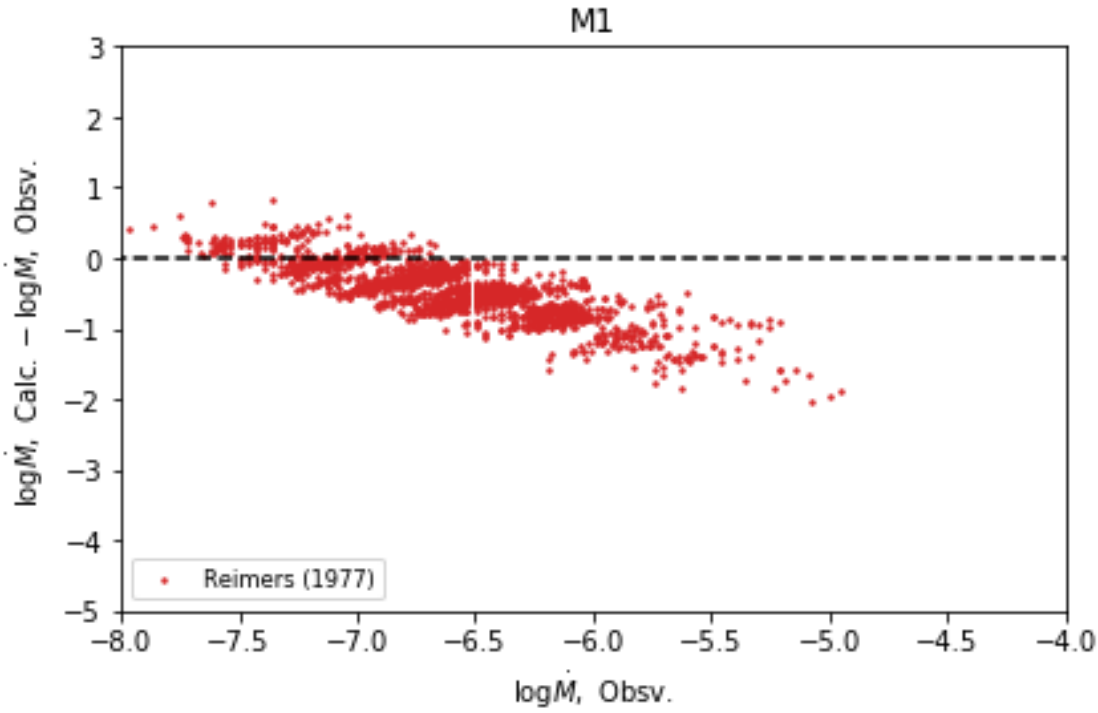
```
Reimers 1975 Results: (array([-4.5088404 , -0.68371633]), 0.6744685034117281,
Intercept    0.067479
x            0.010220
dtype: float64,            Intercept         x
Intercept   0.102578  0.015501
x           0.015501  0.002353)
Reimers 1977 Results: (array([-4.96477236, -0.68371633]), 0.6744685034117281,
Intercept    0.067479
x            0.010220
dtype: float64,            Intercept         x
Intercept   0.102578  0.015501
x           0.015501  0.002353)
Bloecker Results: (array([ 0.11701374, -0.18545068]), 0.022776041115711076,
Intercept    0.172570
x            0.026137
dtype: float64,            Intercept         x
Intercept   0.102578  0.015501
x           0.015501  0.002353)
Wachter et al. Results: (array([-3.49410932, -0.48679335]), 0.31947486138531767,
Intercept    0.100932
x            0.015287
dtype: float64,            Intercept         x
Intercept   0.102578  0.015501
x           0.015501  0.002353)
van Loon Results: (array([-3.84802131, -0.66164489]), 0.5894598596763561,
Intercept    0.078443
x            0.011881
dtype: float64,            Intercept         x
Intercept   0.102578  0.015501
x           0.015501  0.002353)
Schroeder and Cuntz Results: (array([-3.58431156, -0.53971613]),
0.39013220499891954, Intercept    0.095864
x            0.014519
dtype: float64,            Intercept         x
Intercept   0.102578  0.015501
x           0.015501  0.002353)
de Beck et al. Results: (array([-6.11861092, -0.85844673]), 0.9382145344700554,
Intercept    0.031296
x            0.004740
dtype: float64,            Intercept         x
Intercept   0.102578  0.015501
x           0.015501  0.002353)
```

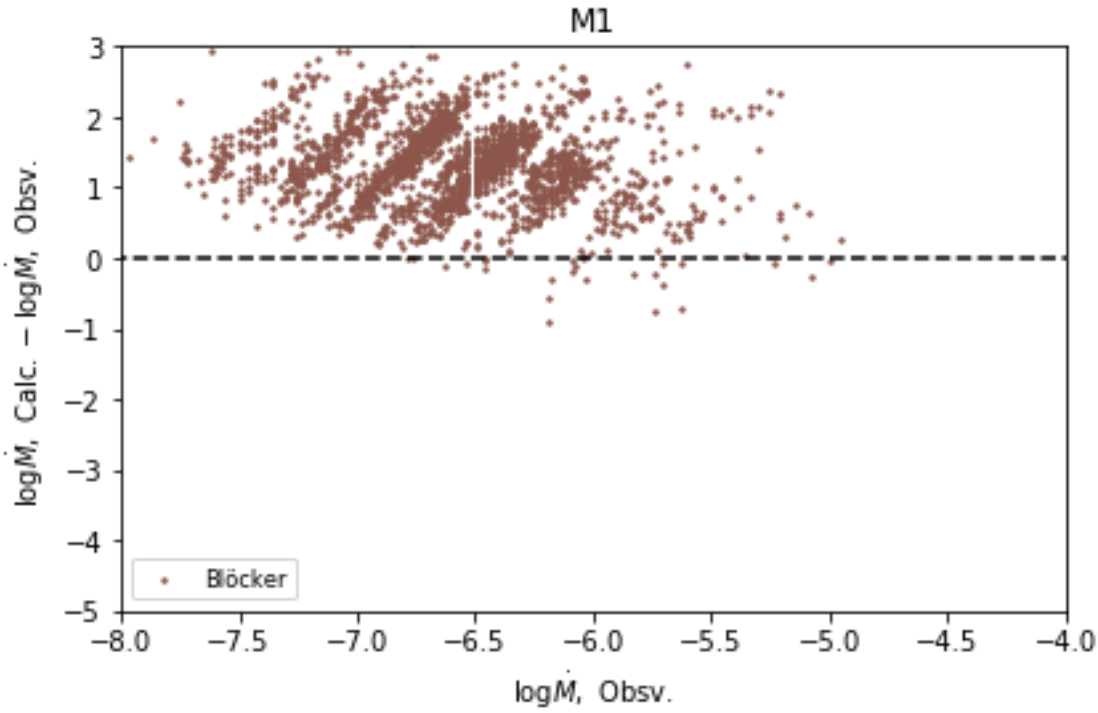

M1
3
2
1
0
−1
−2
−3
−4
−5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
log$\dot{M}$, Calc. − log$\dot{M}$, Obsv.
log$\dot{M}$, Obsv.
Multilinear Fit
Death Zone Analysis
Reimers (1975)
Reimers (1977)
Blöcker
Wachter et al.
van Loon
Schröder and Cuntz
de Beck et al.

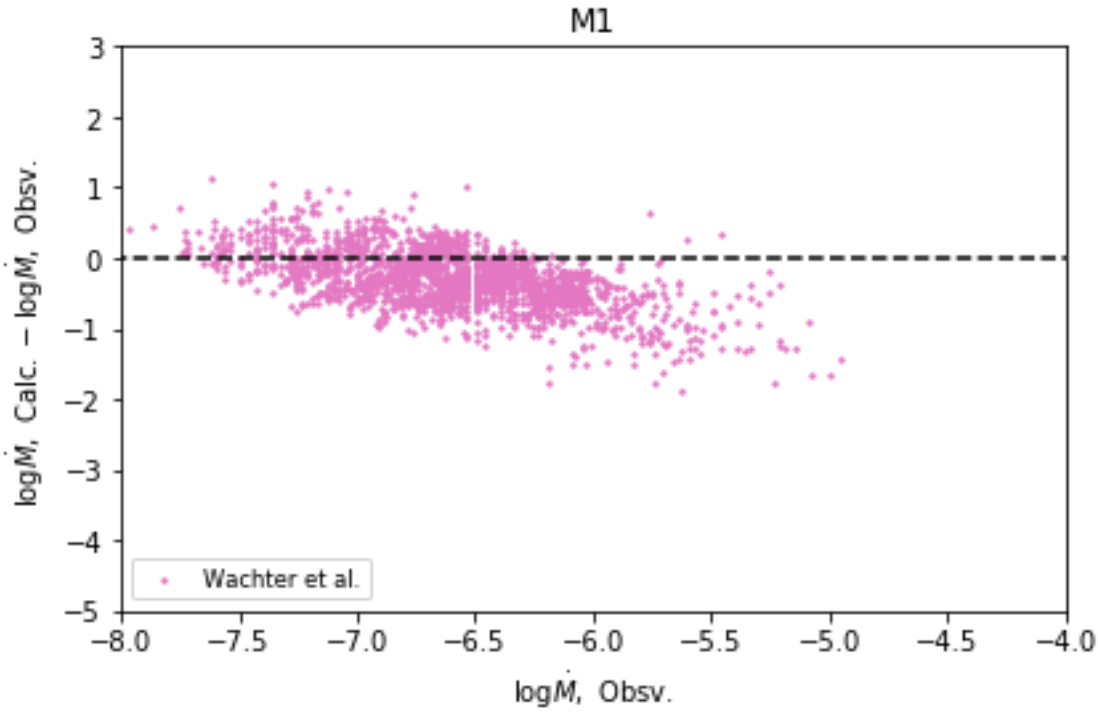

M1
3
2
1
0
−1
−2
−3
−4
−5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
log$\dot{M}$, Calc. − log$\dot{M}$, Obsv.
log$\dot{M}$, Obsv.
Fit to Dust

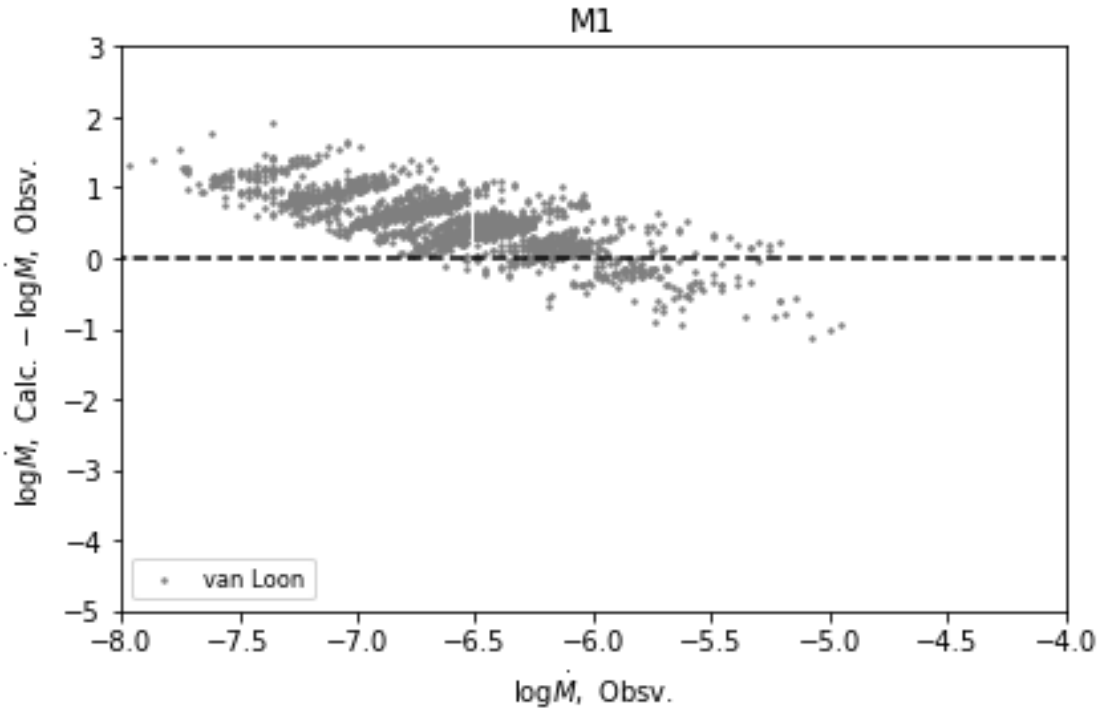

M1
3
2
1
0
−1
−2
−3
−4
−5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
log$\dot{M}$, Calc. − log$\dot{M}$, Obsv.
log$\dot{M}$, Obsv.
Fit to Deathzone

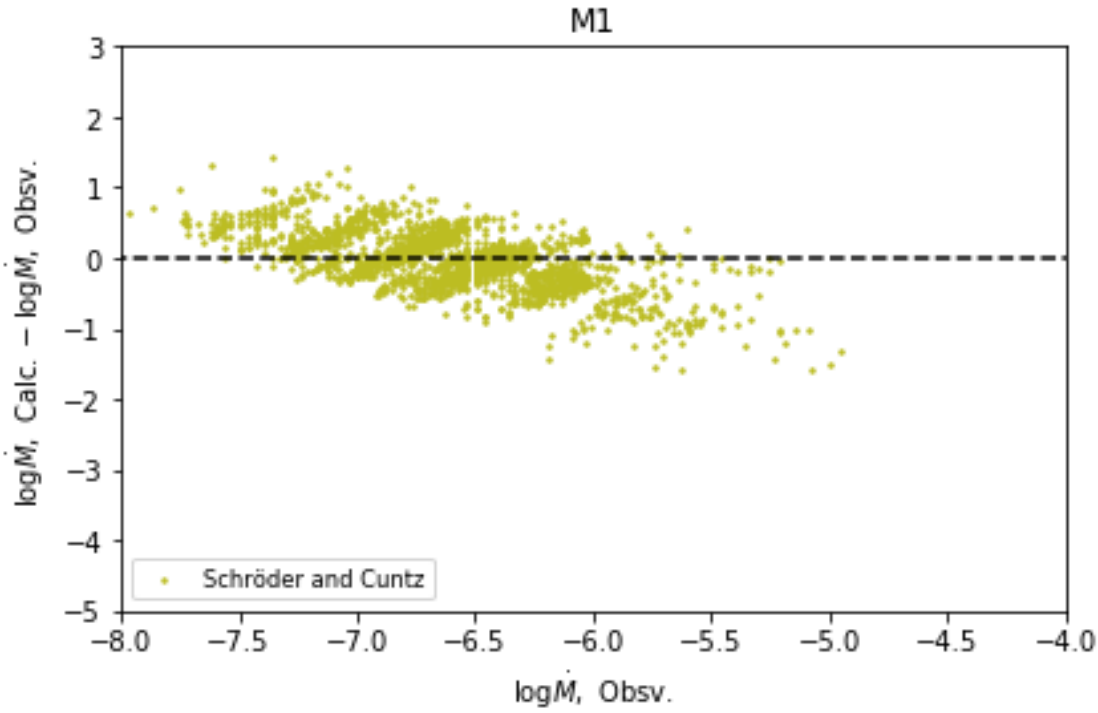

M1
3
2
1
0
−1
−2
−3
−4
−5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
log$\dot{M}$, Calc. − log$\dot{M}$, Obsv.
log$\dot{M}$, Obsv.
Reimers (1975)

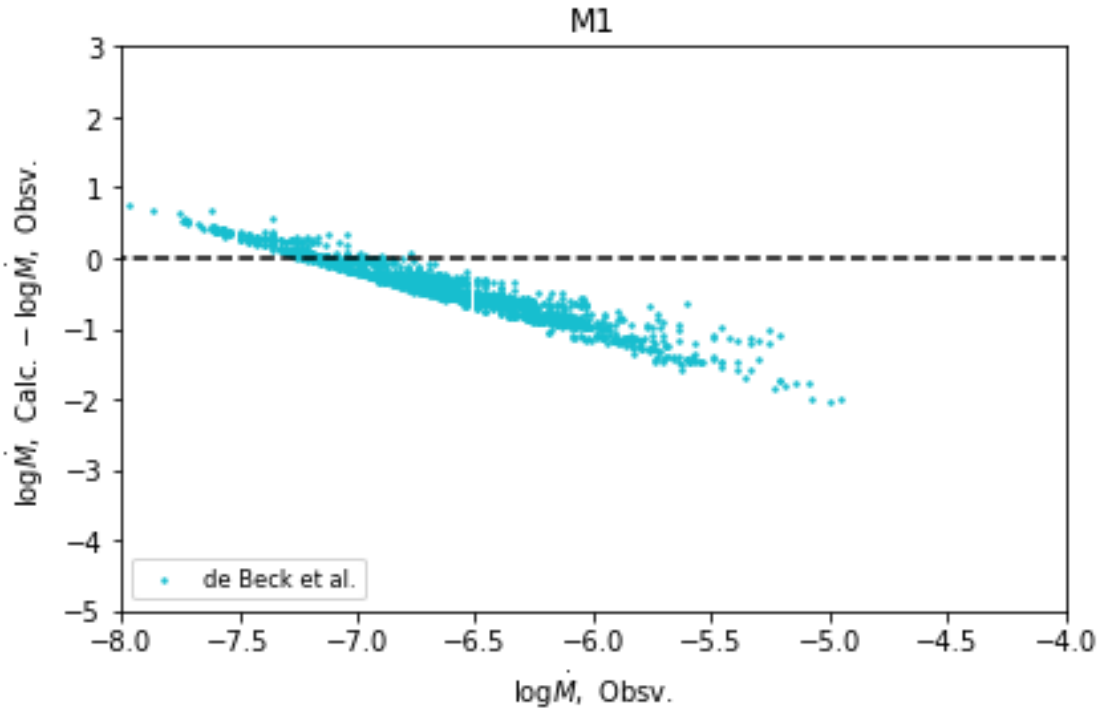

M1
3
2
1
0
−1
−2
−3
−4
−5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
log$\dot{M}$, Calc. − log$\dot{M}$, Obsv.
log$\dot{M}$, Obsv.
Reimers (1977)

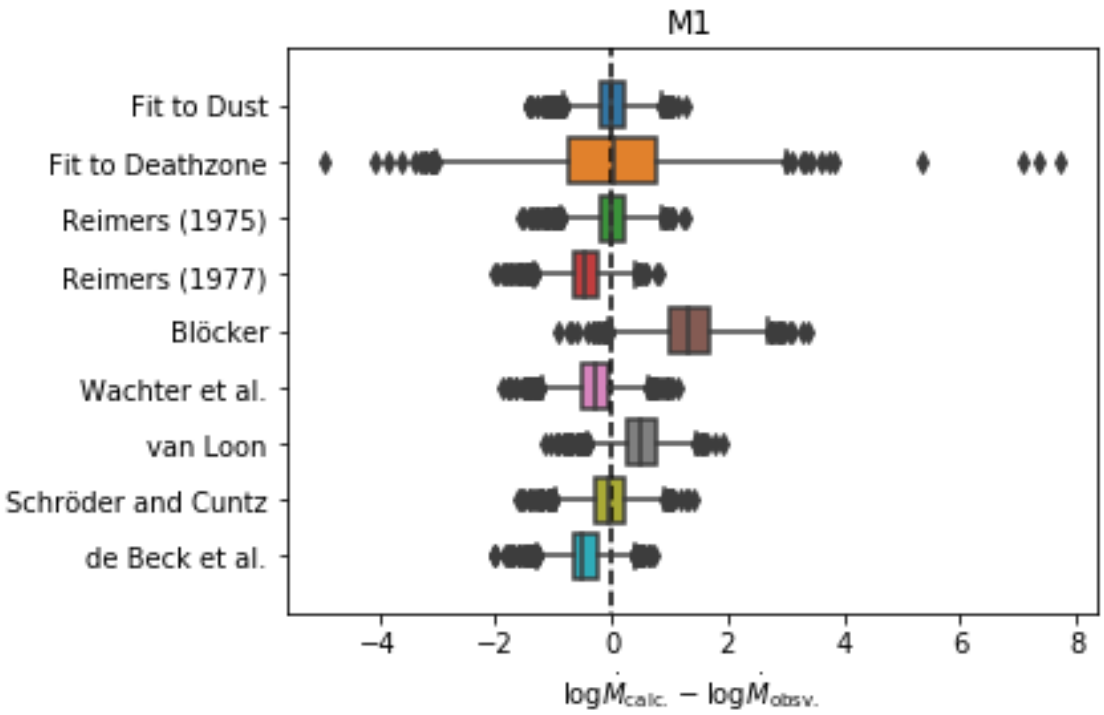

M1
3
2
1
0
−1
−2
−3
−4
−5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
log$\dot{M}$, Calc. − log$\dot{M}$, Obsv.
log$\dot{M}$, Obsv.
Blöcker


M1
3
2
1
0
−1
−2
−3
−4
−5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
log$\dot{M}$, Calc. − log$\dot{M}$, Obsv.
log$\dot{M}$, Obsv.
Wachter et al.


M1
3
2
1
0
−1
−2
−3
−4
−5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
van Loon
log$\dot{M}$, Calc. − log$\dot{M}$, Obsv.
log$\dot{M}$, Obsv.

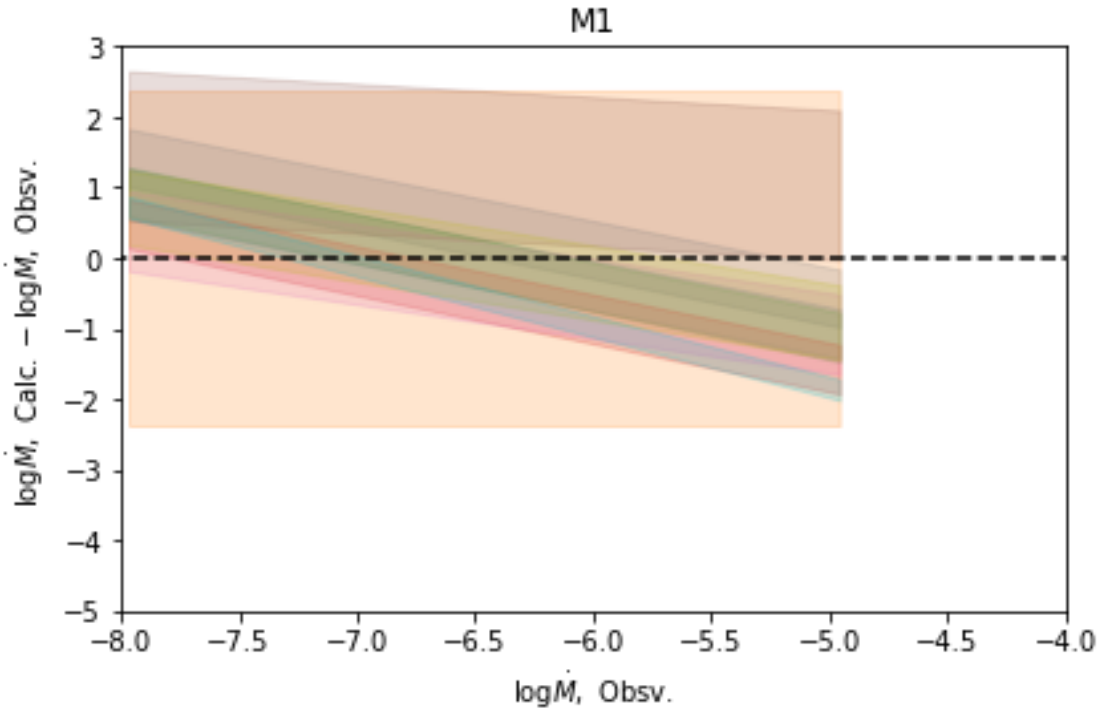

M1
3
2
1
0
−1
−2
−3
−4
−5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
log$\dot{M}$, Calc. − log$\dot{M}$, Obsv.
log$\dot{M}$, Obsv.
Schröder and Cuntz


M1
3
2
1
0
−1
−2
−3
−4
−5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
log$\dot{M}$, Calc. − log$\dot{M}$, Obsv.
log$\dot{M}$, Obsv.
de Beck et al.




```
Point List: [[-7.967457791305338, 1.2584661767135188], [-7.967457791305338,
0.5549343533517312], [-4.955875918755522, -1.4239129178894068],
[-4.955875918755522, -0.7203810945276192]]
x-values: [-7.96745779 -4.95587592]
 y-values: [[ 1.25846618 -0.72038109]
 [ 0.55493435 -1.42391292]]
Point List: [[-7.967457791305338, 2.375685723336336], [-7.967457791305338,
-2.3756767063415736], [-4.955875918755522, -2.375686546007537],
[-4.955875918755522, 2.3756758836703726]]
x-values: [-7.96745779 -4.95587592]
 y-values: [[ 2.37568572  2.37567588]
 [-2.37567671 -2.37568655]]
Point List: [[-7.967457791305338, 1.2863282517966468], [-7.967457791305338,
0.5909529726634215], [-4.955875918755522, -1.4681147384080218],
[-4.955875918755522, -0.7727394592747965]]
x-values: [-7.96745779 -4.95587592]
 y-values: [[ 1.28632825 -0.77273946]
 [ 0.59095297 -1.46811474]]
Point List: [[-7.967457791305338, 0.8303962961469153], [-7.967457791305338,
0.13502101701369007], [-4.955875918755522, -1.9240466940577505],
[-4.955875918755522, -1.2286714149245253]]
x-values: [-7.96745779 -4.95587592]
 y-values: [[ 0.8303963  -1.22867141]
 [ 0.13502102 -1.92404669]]
```

```
Point List: [[-7.967457791305338, 2.653656872130033], [-7.967457791305338,
0.53551157686669], [-4.955875918755522, -0.02298833521788235],
[-4.955875918755522, 2.0951569600454603]]
x-values: [-7.96745779 -4.95587592]
 y-values: [[ 2.65365687  2.09515696]
 [ 0.53551158 -0.02298834]]
Point List: [[-7.967457791305338, 0.950832548290323], [-7.967457791305338,
-0.18204021113792068], [-4.955875918755522, -1.6480582458073743],
[-4.955875918755522, -0.5151854863791305]]
x-values: [-7.96745779 -4.95587592]
 y-values: [[ 0.95083255 -0.51518549]
 [-0.18204021 -1.64805825]]
Point List: [[-7.967457791305338, 1.8319373938104677], [-7.967457791305338,
1.0152754880214783], [-4.955875918755522, -0.9773222764633416],
[-4.955875918755522, -0.16066037067435213]]
x-values: [-7.96745779 -4.95587592]
 y-values: [[ 1.83193739 -0.16066037]
 [ 1.01527549 -0.97732228]]
Point List: [[-7.967457791305338, 1.2424126647570395], [-7.967457791305338,
0.18929521046306874], [-4.955875918755522, -1.4361041073574747],
[-4.955875918755522, -0.382986653063504]]
x-values: [-7.96745779 -4.95587592]
 y-values: [[ 1.24241266 -0.38298665]
 [ 0.18929521 -1.43610411]]
Point List: [[-7.967457791305338, 0.8692435050427108], [-7.967457791305338,
0.5728107962870919], [-4.955875918755522, -2.0124718079755164],
[-4.955875918755522, -1.7160390992198975]]
x-values: [-7.96745779 -4.95587592]
 y-values: [[ 0.86924351 -1.7160391 ]
 [ 0.5728108  -2.01247181]]
```



---

mdot_formula_comparison (C0StarData_n,spec_mode_label='C0') mdot_formula_comparison (C0StarData_x,spec_mode_label='C0x')

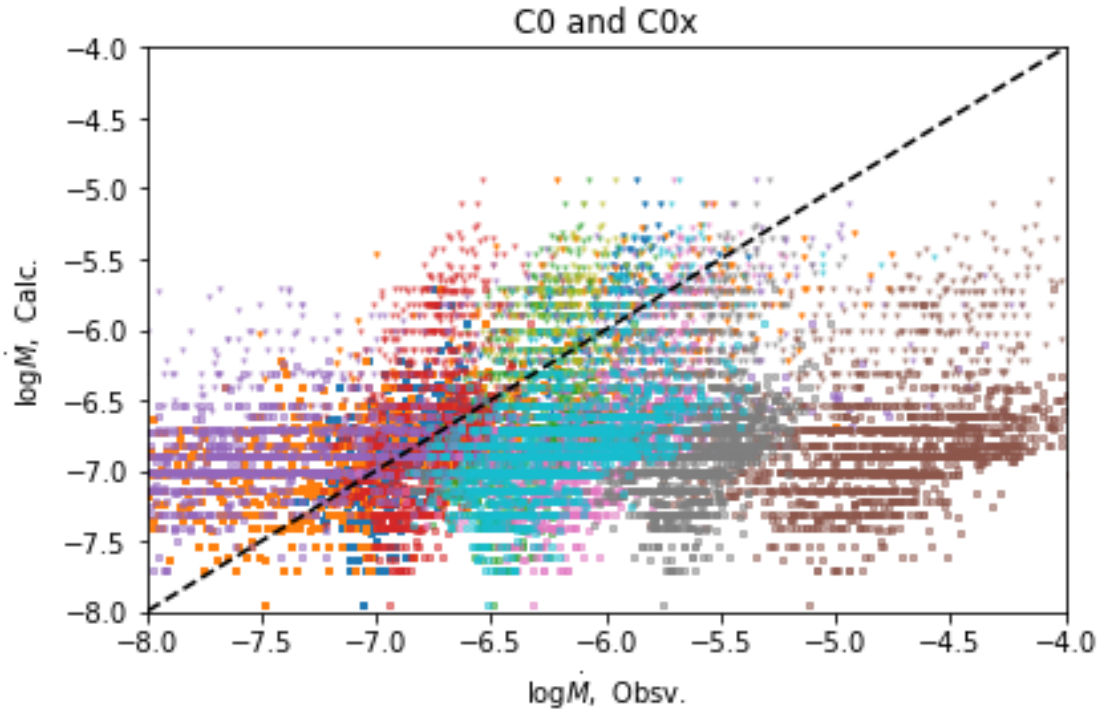
[382]:
```
mdot_formula_comparison_multidata ([C0StarData_n,␣
↪C0StarData_x],spec_mode_label_list=['C0', 'C0x'],marker_size=8)
```

```
C0 and C0x
Data set 0 :
log Mdot ~ logL + logP:
LRM form: logMdot = -14.54811849748131  +  1.4764393908815636 logL + logR -
2.2509424073942057 logM
LM form: logMdot = -14.87111849748131  +  2.2134393908815637 logL -
2.6219424073942057 logM
Data set 1 :
log Mdot ~ logL + logP:
LRM form: logMdot = -18.512591005524992  +  2.7512165735041902 logL + logR -
3.594884381059299 logM
LM form: logMdot = -18.835591005524993  +  3.4882165735041903 logL -
3.965884381059299 logM

--------------
log Mdot ~ logL + logM
```

```
LM form: logMdot = -41.078937026141794  +  9.635192545710597 logL -
13.910423664947327 logM
--------------
Reimers 1975 LRM form: logMdot = -12.397940008672037 + logL + logR - logM
Reimers 1975 LM form: logMdot = -12.720940008672038 + 1.737 logL -  1.371 logM
Calc vs Obsv Slope: 0.13260874175419823
Calc vs Obsv R^2: 0.15765306069333962
SDev of Residual: 0.3947490097064233
Mean distance from 1-to-1 line: 0.5211144588430255
--------------
Reimers 1977 LRM form: logMdot = -12.853871964321762 + logL + logR - logM
Reimers 1977 LM form: logMdot = -13.176871964321762 + 1.737 logL -  1.371 logM
Calc vs Obsv Slope: 0.1326087417541988
Calc vs Obsv R^2: 0.15765306069333918
SDev of Residual: 0.3947490097064233
Mean distance from 1-to-1 line: 0.28913490962848215
--------------
VW LRM form: N/A
VW LM form apprx.: -59.00142916215469 + 14.281670464759578 logL -
15.149773382968814 logM
Calc vs Obsv Slope: 1.3177841201355212
Calc vs Obsv R^2: 0.3396066397867107
SDev of Residual: 0.8002184403531087
Mean distance from 1-to-1 line: 0.7532590058096982
--------------
Bloecker LRM form: logMdot = -20.713992877920525 +3.7logL + logR - 3.1logM
Bloecker LM form: logMdot = -21.036992877920525 + 4.437 logL -  3.471 logM
Calc vs Obsv Slope: 0.33743251471130564
Calc vs Obsv R^2: 0.15441175690012277
SDev of Residual: 0.44230375901812513
Mean distance from 1-to-1 line: 2.019443433684554
--------------
Wachter LRM form: logMdot = -0.15039855100000044  +  0.08978907299999994 logL -
1.3964423309999998 logM + logR
Wachter LM form: logMdot = -0.47339855100000044  +  0.8267890729999999 logL -
1.7674423309999998 logM
Calc vs Obsv Slope: 0.36235660624908494
Calc vs Obsv R^2: 0.4085905377038377
SDev of Residual: 0.33144305532511326
Mean distance from 1-to-1 line: 0.7431883250199941
--------------
van Loon LRM form: logMdot = 5.858110000000001  +  1.05955 logL - 0.79765 logM +
logR
van Loon LM form: logMdot = 5.53511  +  1.7965499999999999 logL - 1.16865 logM
Calc vs Obsv Slope: 0.12669739820382003
Calc vs Obsv R^2: 0.10346074574045483
SDev of Residual: 0.40747163021179383
Mean distance from 1-to-1 line: 1.1649574546997155
```

```
--------------
Schroeder and Cuntz LRM form: N/A
Schroeder and Cuntz LM form apprx.: -11.11753203606868 + 1.322250000000011 logL
- 0.7217499999999999 logM
Calc vs Obsv Slope: 0.19947327861174413
Calc vs Obsv R^2: 0.1952076827417093
SDev of Residual: 0.30654332143420177
Mean distance from 1-to-1 line: 0.2844383042000904
--------------
de Beck LRM form: N/A
de Beck LM form apprx.: -20.662096326053152 + 3.9844729903524785 logL -
4.1631710853753034 logM
Calc vs Obsv Slope: 0.368902935440672
Calc vs Obsv R^2: 0.33830313352962826
SDev of Residual: 0.3497746236516414
Mean distance from 1-to-1 line: 0.5941536278494517
--------------
```

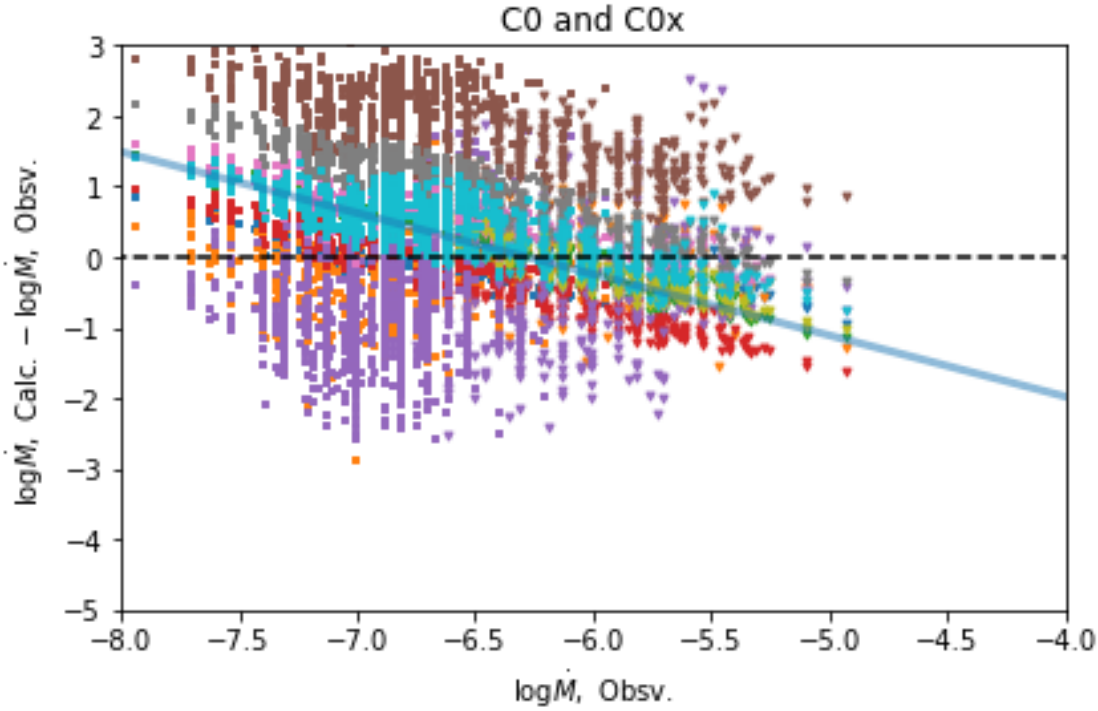



```
 --------------

Dust-Fitting Results: (array([-1.94599718, -0.28786079]), 0.2878607942339241,
Intercept    0.068717
x            0.010144
```

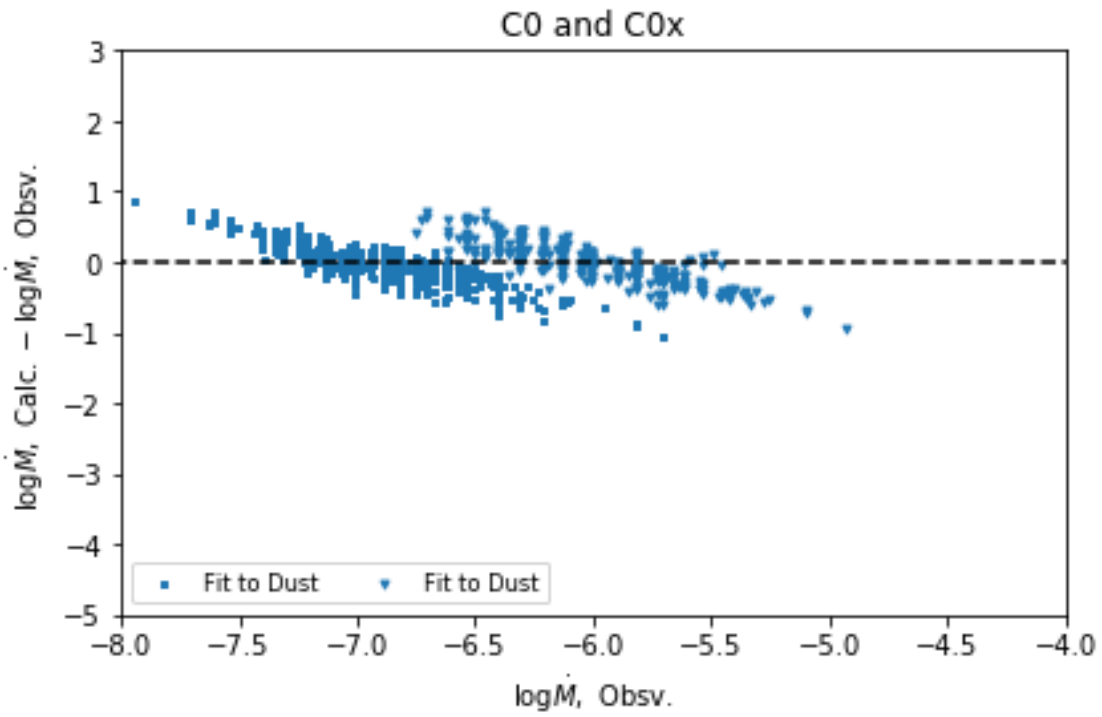
```
dtype: float64,            Intercept         x
Intercept   0.124979  0.018413
x           0.018413  0.002724)
Death Zone-Fitting Results: (array([1.97223984e-04, 2.91742728e-05]),
5.850673279184093e-10, Intercept    0.183057
x            0.027024
dtype: float64,            Intercept         x
Intercept   0.124979  0.018413
x           0.018413  0.002724)
Reimers 1975 Results: (array([-5.44685878, -0.86739126]), 0.8889814567224048,
Intercept    0.046522
x            0.006868
dtype: float64,            Intercept         x
Intercept   0.124979  0.018413
x           0.018413  0.002724)
Reimers 1977 Results: (array([-5.90279074, -0.86739126]), 0.8889814567224048,
Intercept    0.046522
x            0.006868
dtype: float64,            Intercept         x
Intercept   0.124979  0.018413
x           0.018413  0.002724)
VW Results: (array([1.69339067, 0.31778412]), 0.0290370164978786, Intercept
0.278898
x            0.041173
dtype: float64,            Intercept         x
Intercept   0.124979  0.018413
x           0.018413  0.002724)
Bloecker Results: (array([-2.45964674, -0.66256749]), 0.41316516424009897,
Intercept    0.119843
x            0.017692
dtype: float64,            Intercept         x
Intercept   0.124979  0.018413
x           0.018413  0.002724)
Wachter et al. Results: (array([-3.57866846, -0.63764339]), 0.6814635013024971,
Intercept    0.066164
x            0.009768
dtype: float64,            Intercept         x
Intercept   0.124979  0.018413
x           0.018413  0.002724)
van Loon Results: (array([-4.74142028, -0.8733026 ]), 0.8457450664185273,
Intercept    0.056605
x            0.008356
dtype: float64,            Intercept         x
Intercept   0.124979  0.018413
x           0.018413  0.002724)
Schroeder and Cuntz Results: (array([-4.99550178, -0.80052672]),
0.7961915203513223, Intercept    0.132815
x            0.021998
```

```
dtype: float64,            Intercept         x
Intercept   0.915662  0.151415
x           0.151415  0.025119)
de Beck et al. Results: (array([-3.70960584, -0.63109706]), 0.5994051126350111,
Intercept    0.078303
x            0.011560
dtype: float64,            Intercept         x
Intercept   0.124979  0.018413
x           0.018413  0.002724)
```



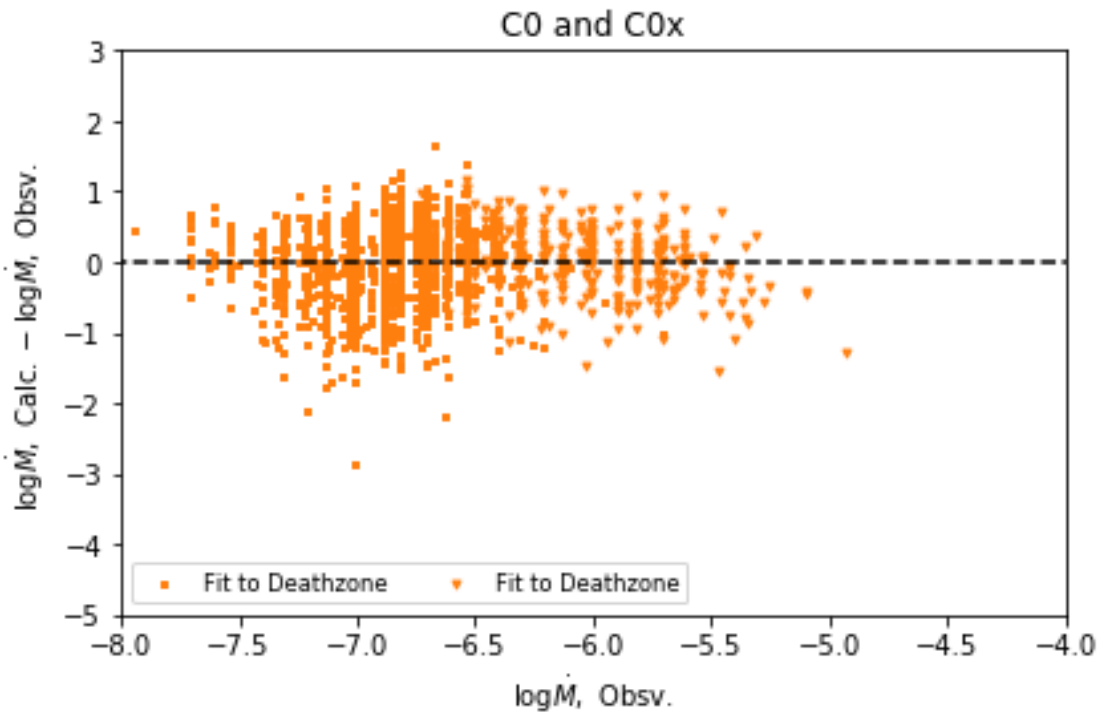

```
 --------------

len(logMdotData[k]): 1654
len(residual_list[k][i]): 1654
len(logMdotData[k]): 341
len(residual_list[k][i]): 341
```



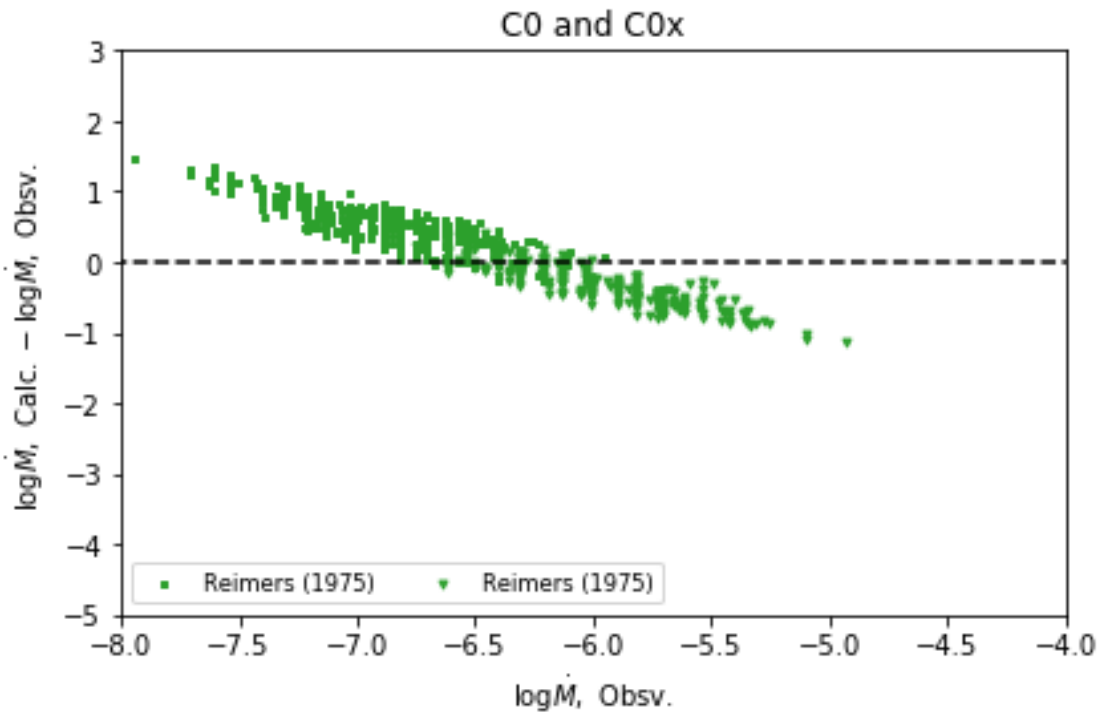

```
 --------------

len(logMdotData[k]): 1654
len(residual_list[k][i]): 1654
len(logMdotData[k]): 341
len(residual_list[k][i]): 341
```



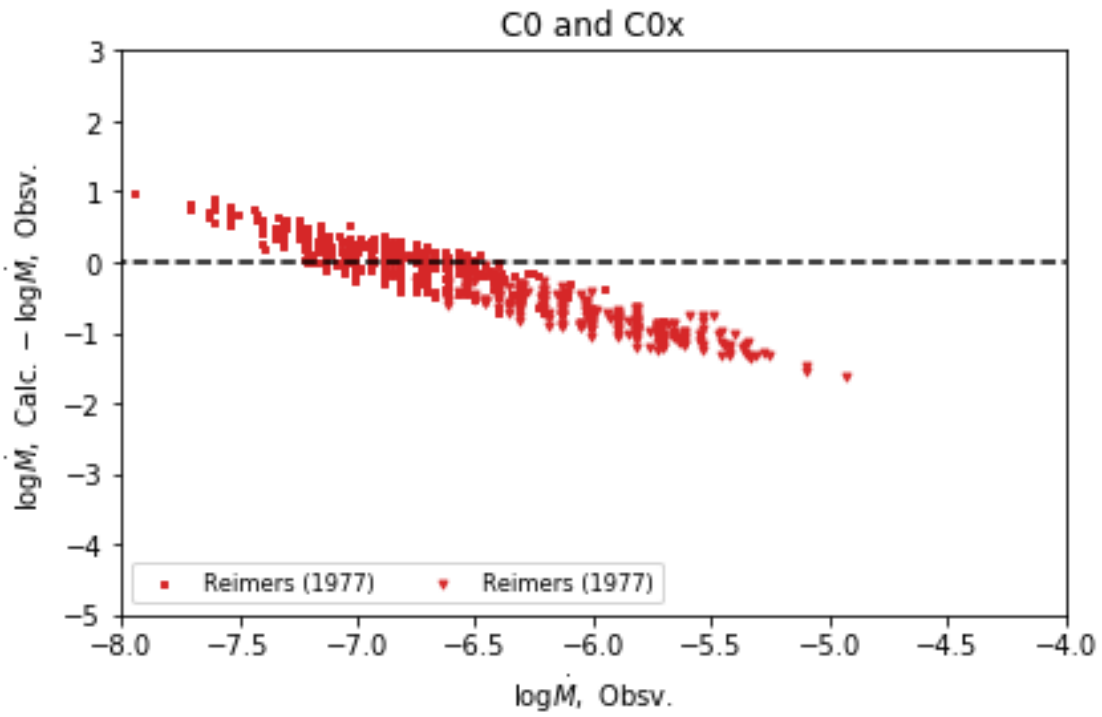

```
 --------------

len(logMdotData[k]): 1654
len(residual_list[k][i]): 1654
len(logMdotData[k]): 341
len(residual_list[k][i]): 341
```

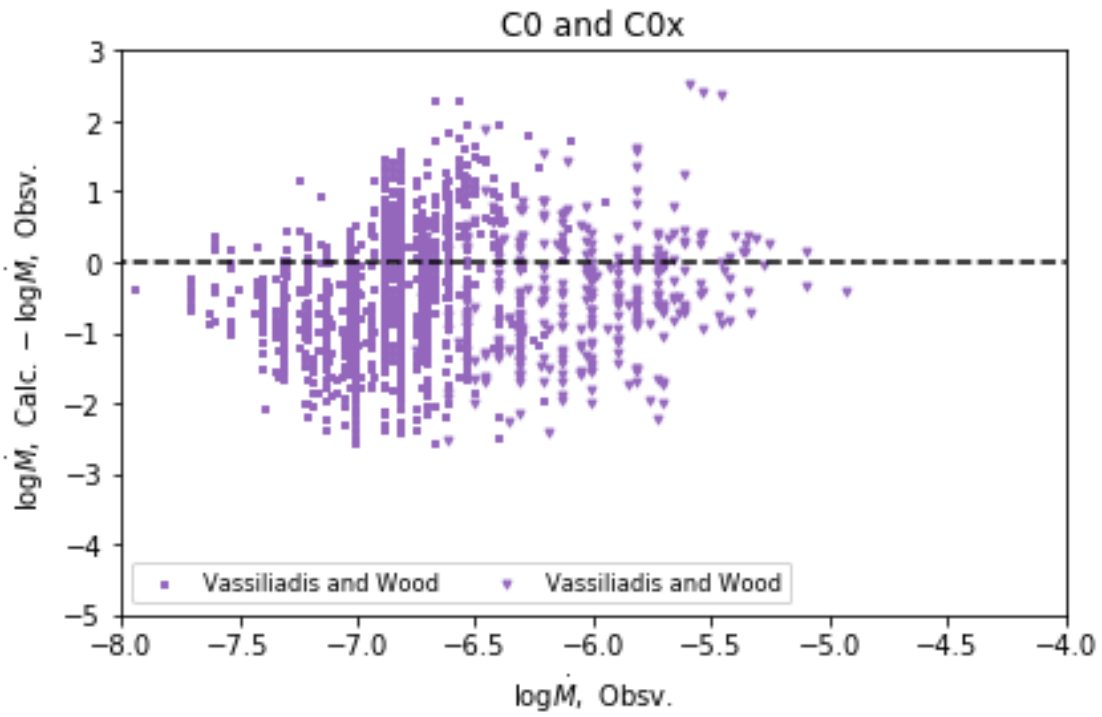



```
 --------------

len(logMdotData[k]): 1654
len(residual_list[k][i]): 1654
len(logMdotData[k]): 341
len(residual_list[k][i]): 341
```

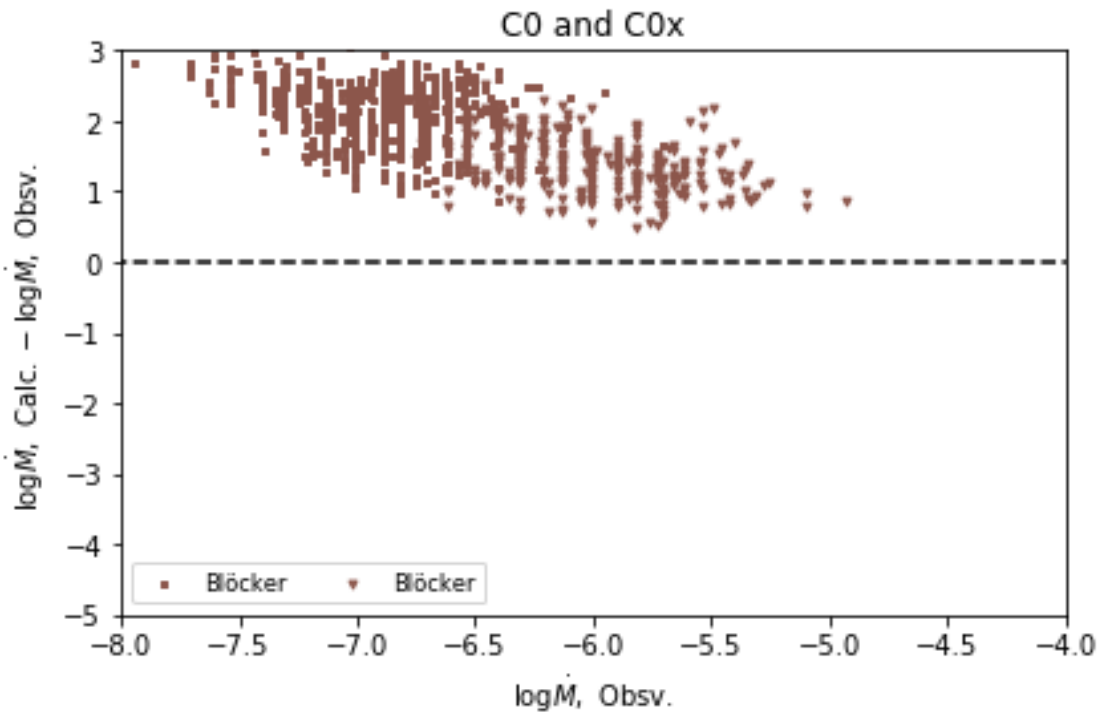


```
len(logMdotData[k]): 1654
len(residual_list[k][i]): 1654
len(logMdotData[k]): 341
len(residual_list[k][i]): 341
```



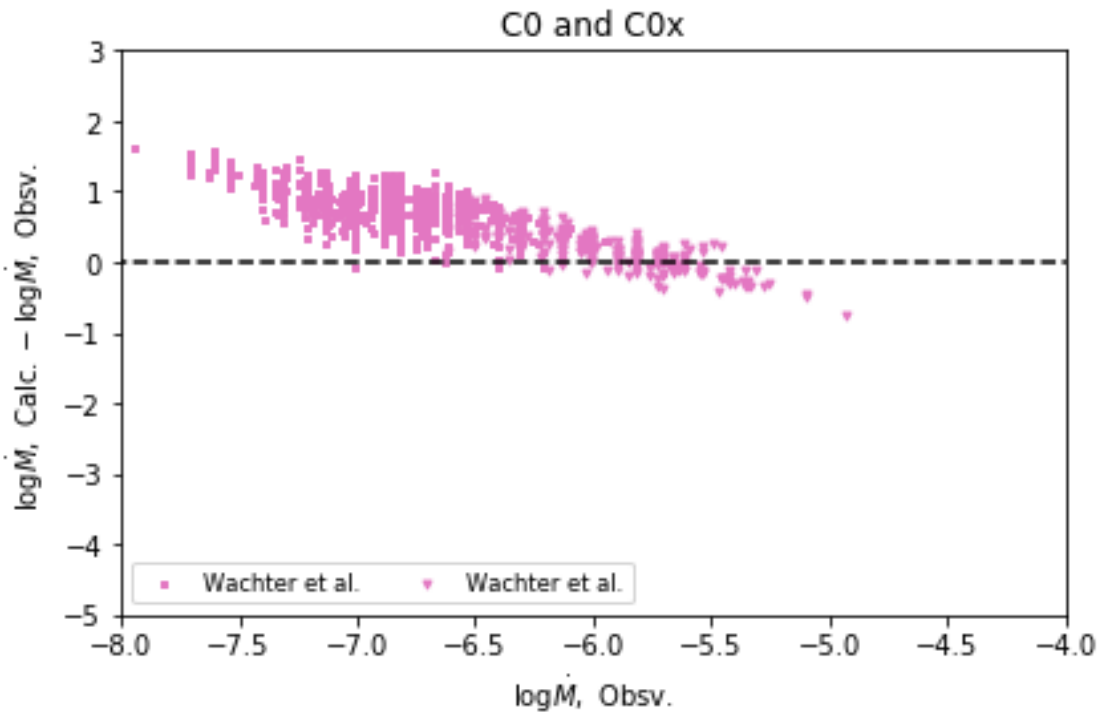

```
 --------------

len(logMdotData[k]): 1654
len(residual_list[k][i]): 1654
len(logMdotData[k]): 341
len(residual_list[k][i]): 341
```



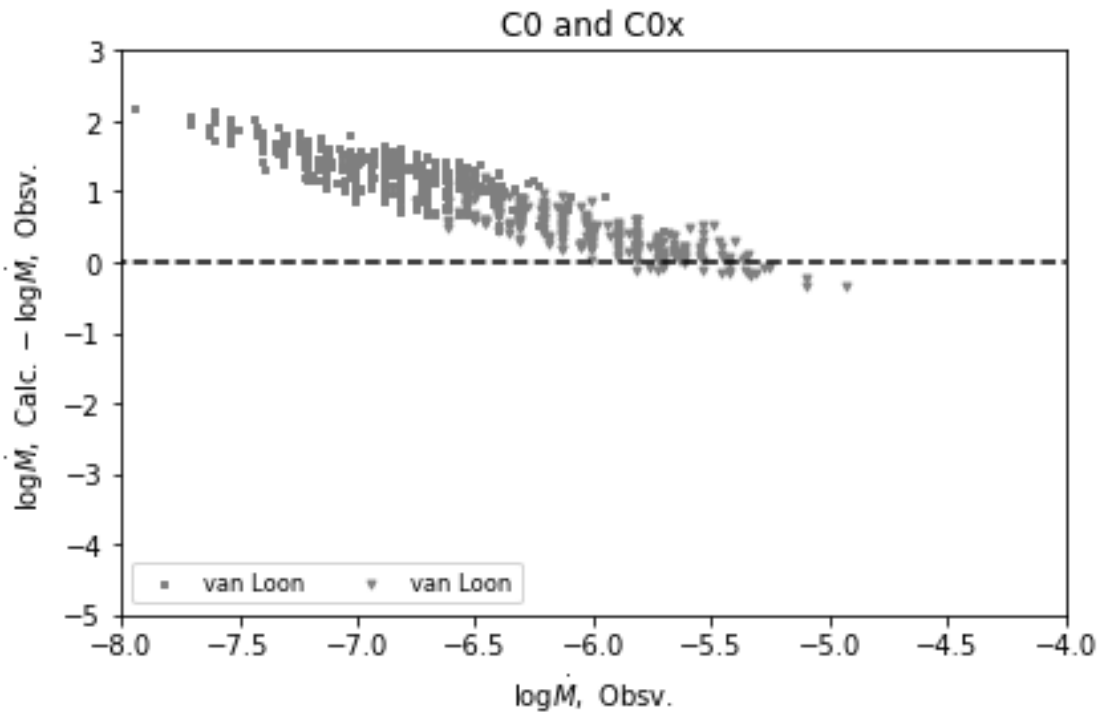

```
 --------------

len(logMdotData[k]): 1654
len(residual_list[k][i]): 1654
len(logMdotData[k]): 341
len(residual_list[k][i]): 341
```



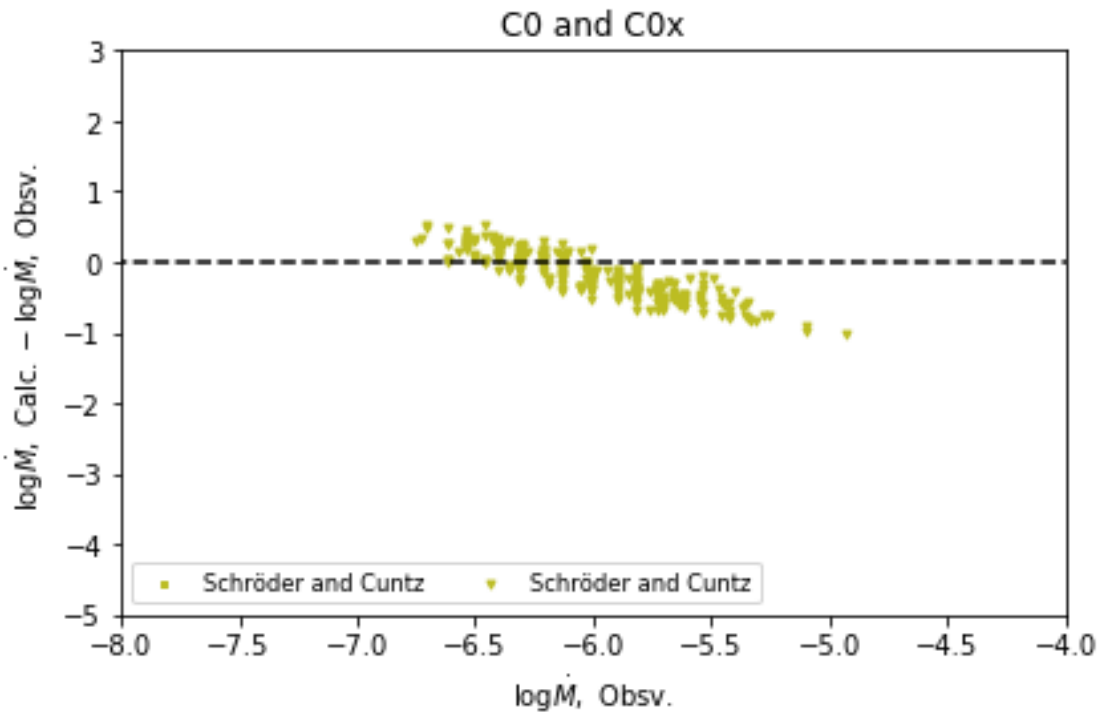

```
 --------------

len(logMdotData[k]): 1654
len(residual_list[k][i]): 1654
len(logMdotData[k]): 341
len(residual_list[k][i]): 341
```



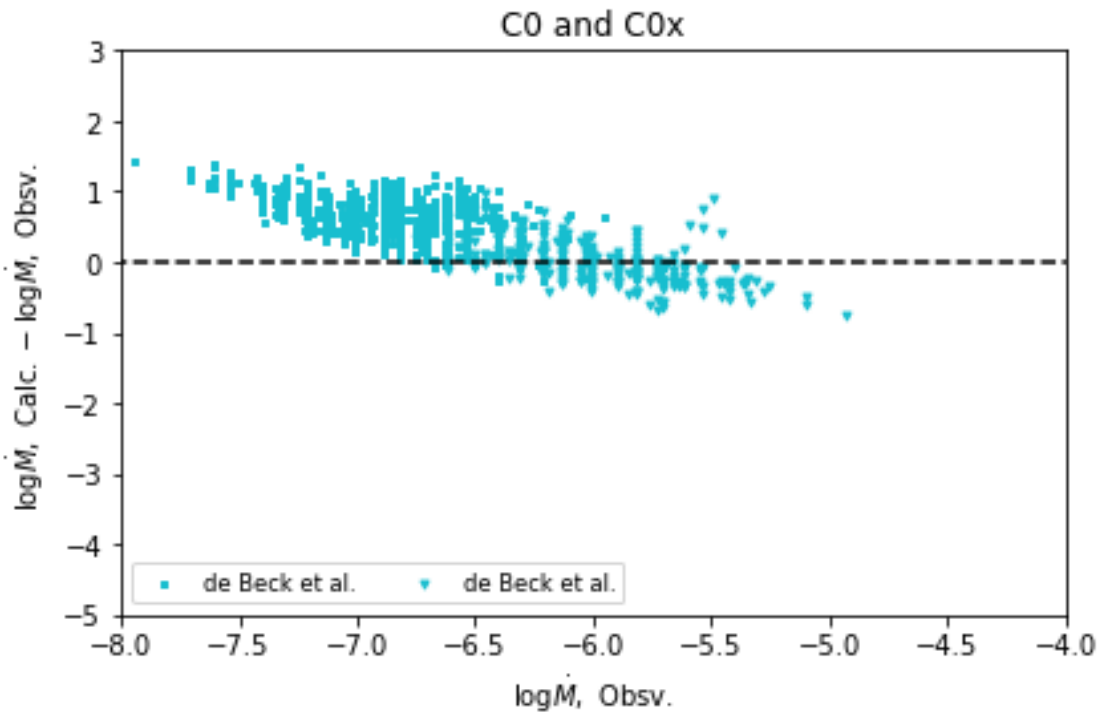

```
 --------------

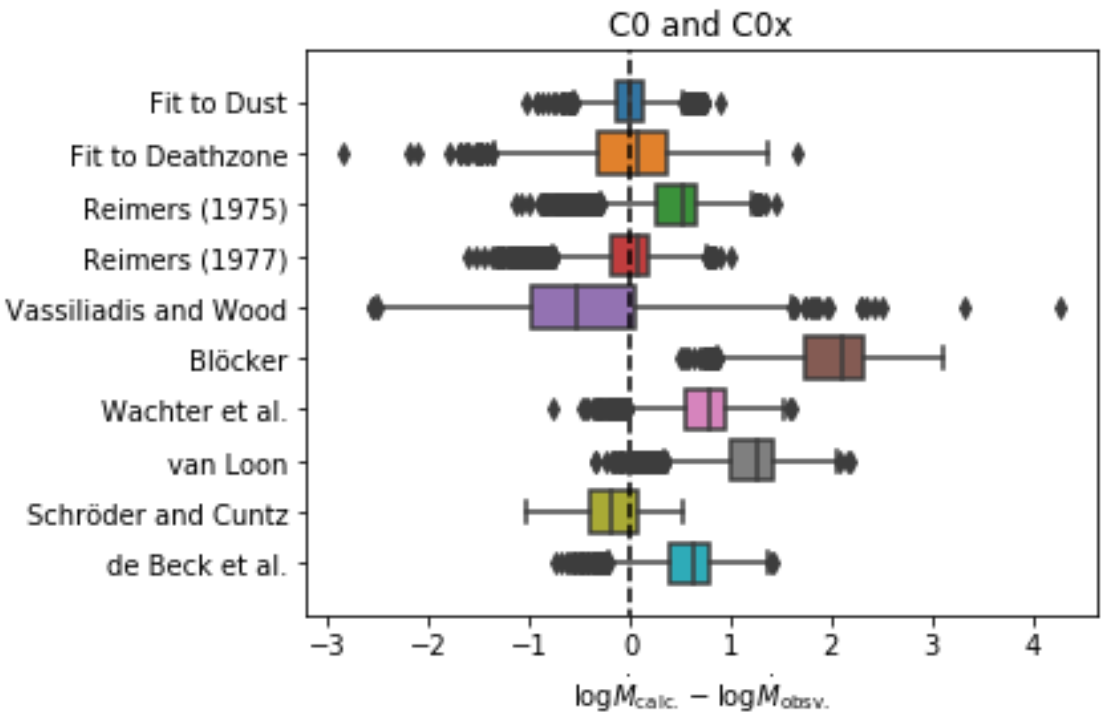
len(logMdotData[k]): 1654
len(residual_list[k][i]): 1654
len(logMdotData[k]): 341
len(residual_list[k][i]): 341
```




```
 --------------

len(logMdotData[k]): 1654
len(residual_list[k][i]): 1654
len(logMdotData[k]): 341
len(residual_list[k][i]): 341
```


C0 and C0x
3
2
1
0
−1
−2
−3
−4
−5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
log$\dot{M}$, Calc. − log$\dot{M}$, Obsv.
log$\dot{M}$, Obsv.
de Beck et al.
de Beck et al.



---

[349]:
```
mdot_formula_comparison (C1StarData,spec_mode_label='C1')
```

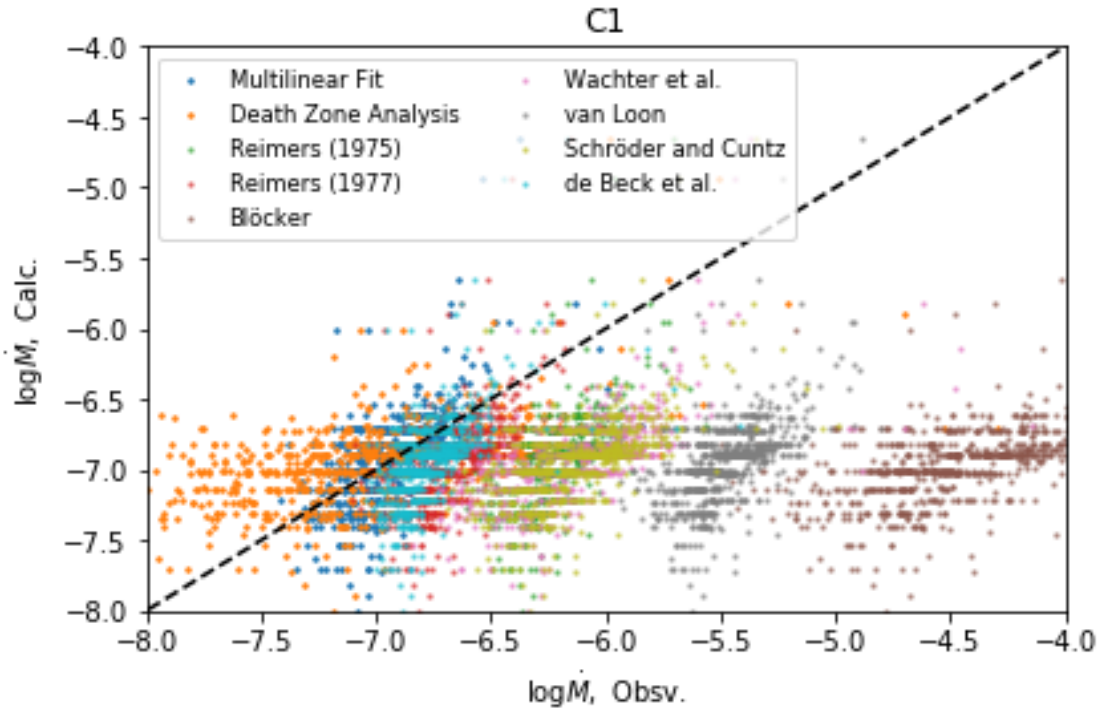
```
log Mdot ~ logL + logP:
LRM form: logMdot = -14.779464244877559  +  1.47292367511773 logL + logR -
1.9670226554375763 logM
LM form: logMdot = -15.10246424487756  +  2.20992367511773 logL -
2.3380226554375763 logM

--------------
log Mdot ~ logL + logM
LM form: logMdot = -28.14202038380528  +  5.88519600584648 logL -
9.437891494192556 logM

--------------
Reimers 1975 LRM form: logMdot = -12.397940008672037 + logL + logR - logM
Reimers 1975 LM form: logMdot = -12.720940008672038 + 1.737 logL -  1.371 logM
Calc vs Obsv Slope: 0.30308775674652416
Calc vs Obsv R^2: 0.33968315689806505
SDev of Residual: 0.23810151793061063

--------------
Reimers 1977 LRM form: logMdot = -12.853871964321762 + logL + logR - logM
Reimers 1977 LM form: logMdot = -13.176871964321762 + 1.737 logL -  1.371 logM
```

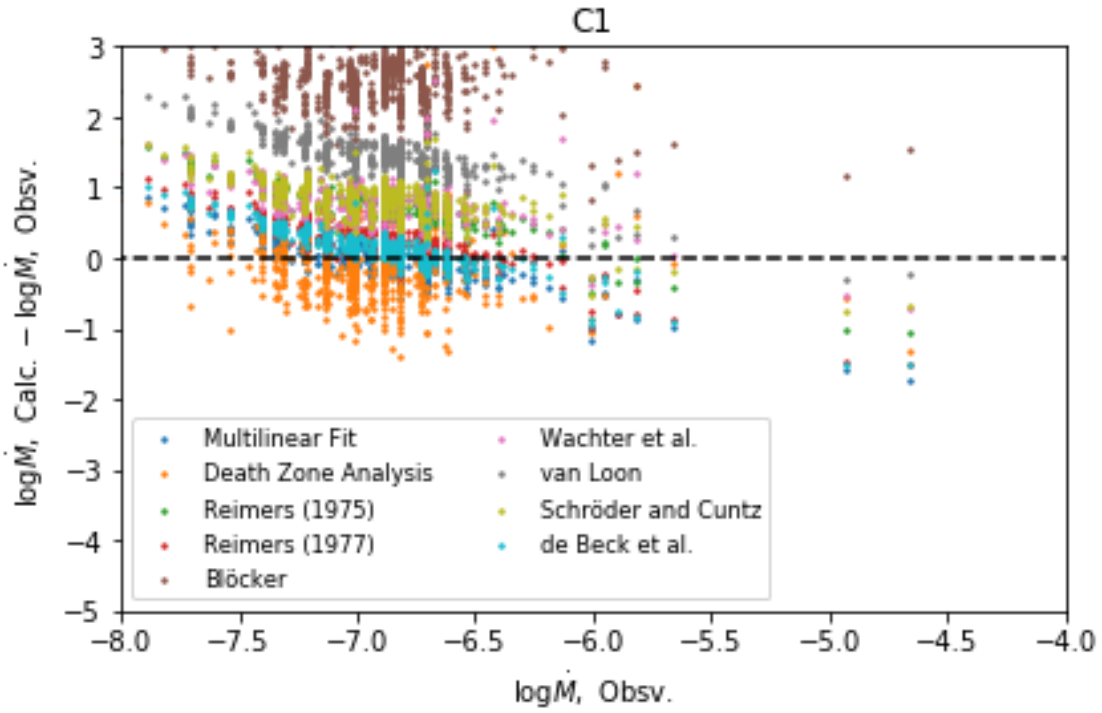
```
Calc vs Obsv Slope: 0.3030877567465255
Calc vs Obsv R^2: 0.33968315689806505
SDev of Residual: 0.23810151793061063
--------------
VW LRM form: N/A
VW LM form apprx.: -32.85116340847547 + 6.383442617052769 logL -
6.650025006906933 logM
Calc vs Obsv Slope: 1.072637507690733
Calc vs Obsv R^2: 0.3368305435617601
SDev of Residual: 0.4402080673857604
--------------
Bloecker LRM form: logMdot = -20.713992877920525 +3.7logL + logR - 3.1logM
Bloecker LM form: logMdot = -21.036992877920525 + 4.437 logL -  3.471 logM
Calc vs Obsv Slope: 0.7743830112649057
Calc vs Obsv R^2: 0.3377197954782637
SDev of Residual: 0.3235887585909702
--------------
Wachter LRM form: logMdot = 0.6971455099999998  +  0.5429002699999999 logL -
1.7307506899999998 logM + logR
Wachter LM form: logMdot = 0.3741455099999998  +  1.2799002699999997 logL -
2.10175069 logM
Calc vs Obsv Slope: 0.5941734899222416
Calc vs Obsv R^2: 0.3613140899439664
SDev of Residual: 0.25945703687585375
--------------
van Loon LRM form: logMdot = 5.858110000000001  +  1.05955 logL - 0.79765 logM +
logR
van Loon LM form: logMdot = 5.53511  +  1.7965499999999999 logL - 1.16865 logM
Calc vs Obsv Slope: 0.314877335147559
Calc vs Obsv R^2: 0.2974283613329236
SDev of Residual: 0.24505003693594068
--------------
Schroeder and Cuntz LRM form: N/A
Schroeder and Cuntz LM form apprx.: -16.20701155177513 + 2.7044206728255027 logL
- 2.3596524801564076 logM
Calc vs Obsv Slope: 0.47031974340936034
Calc vs Obsv R^2: 0.3600375624895372
SDev of Residual: 0.23979511933066028
--------------
de Beck LRM form: N/A
de Beck LM form apprx.: -13.417163844124017 + 1.7959251566881385 logL -
1.831507256260383 logM
Calc vs Obsv Slope: 0.3051114871111316
Calc vs Obsv R^2: 0.34527107509409594
SDev of Residual: 0.23722900274764322
--------------
```

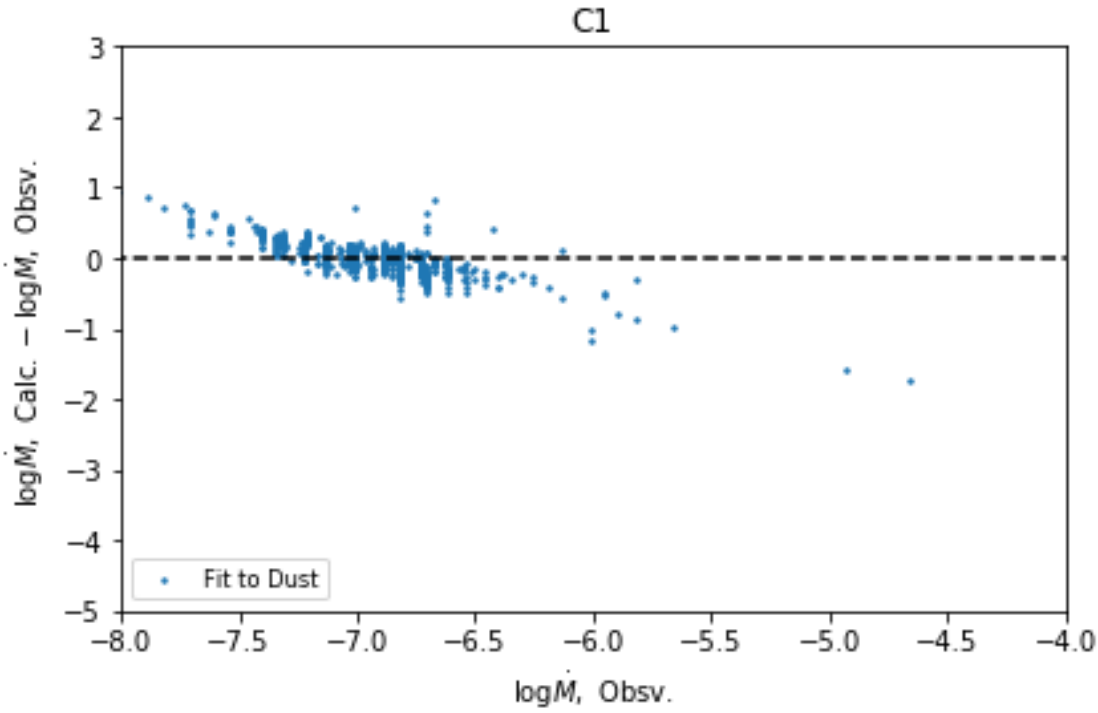


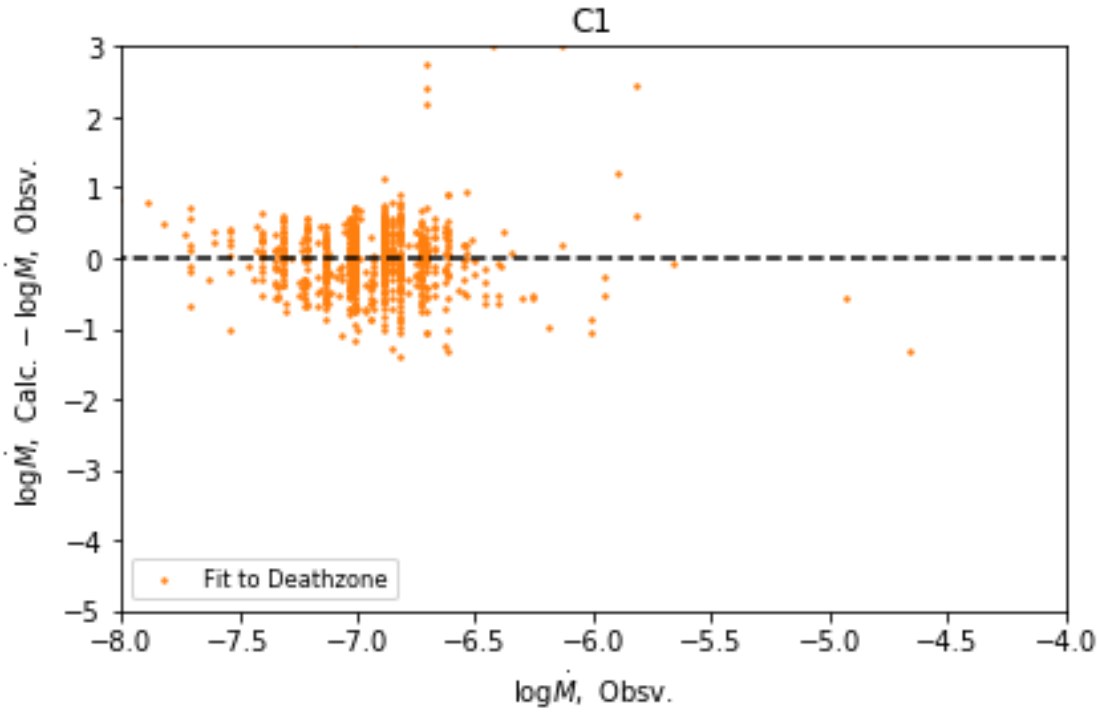

```
 --------------

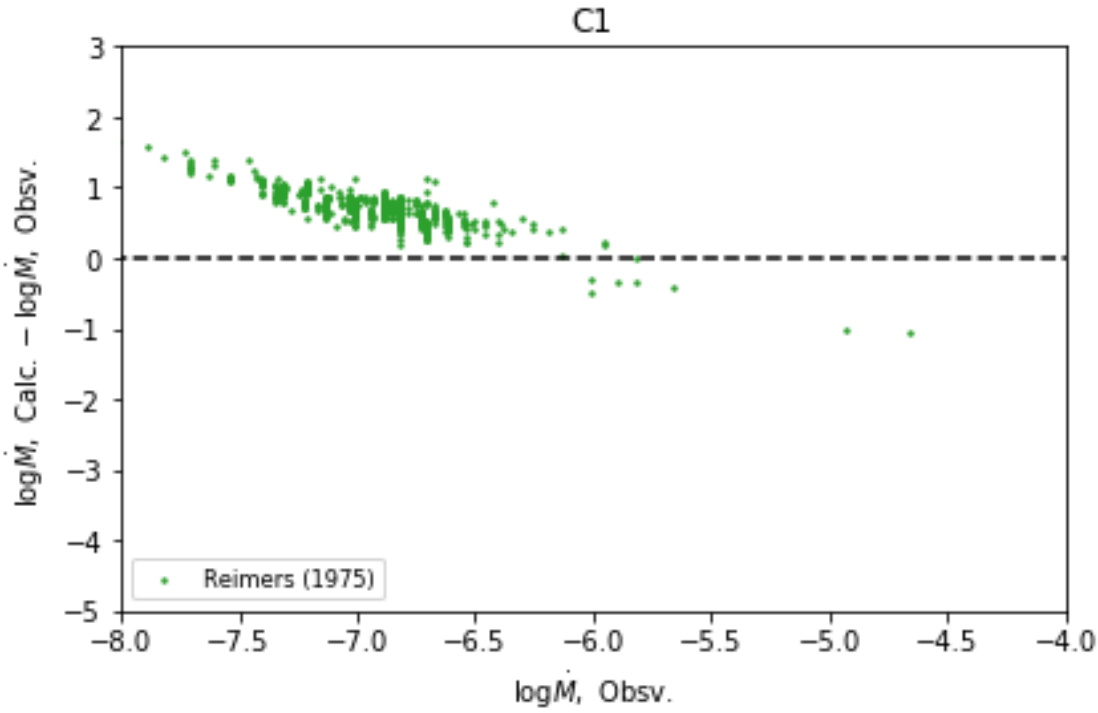
Dust-Fitting Results: (array([-4.29007922, -0.61787789]), 0.6178778936298202,
Intercept    0.120985
x            0.017409
dtype: float64,            Intercept         x
Intercept   0.724529  0.104166
x           0.104166  0.015002)
Death Zone-Fitting Results: (array([1.26131297e-04, 1.81660375e-05]),
1.0862422072932532e-10, Intercept    0.433985
x            0.062449
dtype: float64,            Intercept         x
Intercept   0.724529  0.104166
x           0.104166  0.015002)
Reimers 1975 Results: (array([-4.12523359, -0.69691224]), 0.7311701844793773,
Intercept    0.105217
x            0.015140
dtype: float64,            Intercept         x
Intercept   0.724529  0.104166
x           0.104166  0.015002)
Reimers 1977 Results: (array([-4.58116554, -0.69691224]), 0.7311701844793772,
Intercept    0.105217
x            0.015140
```

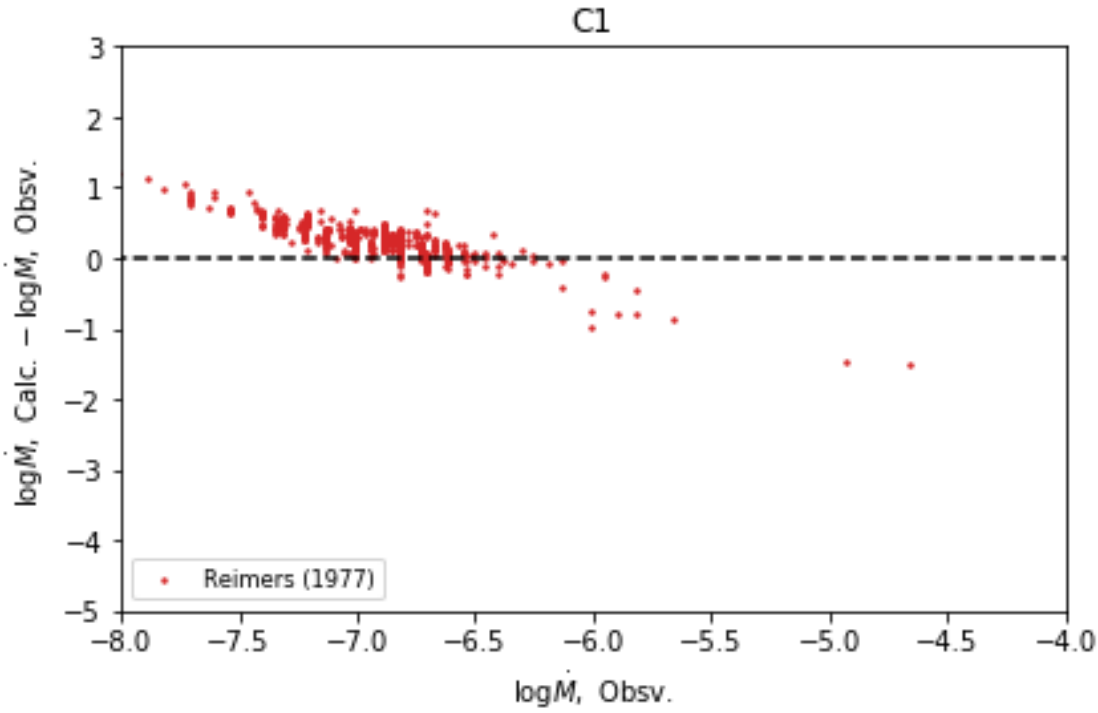
```
dtype: float64,            Intercept         x
Intercept   0.724529  0.104166
x           0.104166  0.015002)
Bloecker Results: (array([ 0.92628038, -0.22561699]), 0.04148999270482123,
Intercept    0.270008
x            0.038853
dtype: float64,            Intercept         x
Intercept   0.724529  0.104166
x           0.104166  0.015002)
Wachter et al. Results: (array([-2.02607265, -0.40582651]), 0.20880290285997705,
Intercept    0.196695
x            0.028304
dtype: float64,            Intercept         x
Intercept   0.724529  0.104166
x           0.104166  0.015002)
van Loon Results: (array([-3.29458002, -0.68512266]), 0.6671350194601728,
Intercept    0.120496
x            0.017339
dtype: float64,            Intercept         x
Intercept   0.724529  0.104166
x           0.104166  0.015002)
Schroeder and Cuntz Results: (array([-2.87216871, -0.52968026]),
0.4164218458727448, Intercept    0.156126
x            0.022466
dtype: float64,            Intercept         x
Intercept   0.724529  0.104166
x           0.104166  0.015002)
de Beck et al. Results: (array([-4.65583836, -0.69488851]), 0.7322869789487301,
Intercept    0.104613
x            0.015054
dtype: float64,            Intercept         x
Intercept   0.724529  0.104166
x           0.104166  0.015002)
```

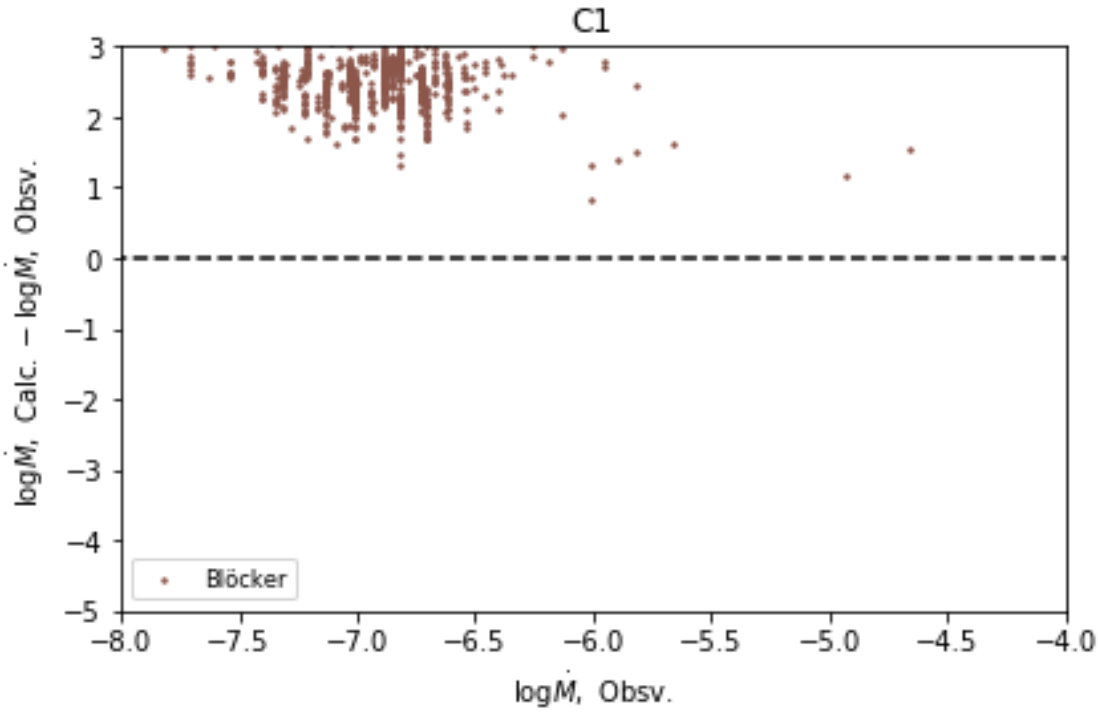

C1
3
2
1
0
−1
−2
−3
−4
−5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
log$\dot{M}$, Calc. − log$\dot{M}$, Obsv.
log$\dot{M}$, Obsv.
Multilinear Fit
Death Zone Analysis
Reimers (1975)
Reimers (1977)
Blöcker
Wachter et al.
van Loon
Schröder and Cuntz
de Beck et al.

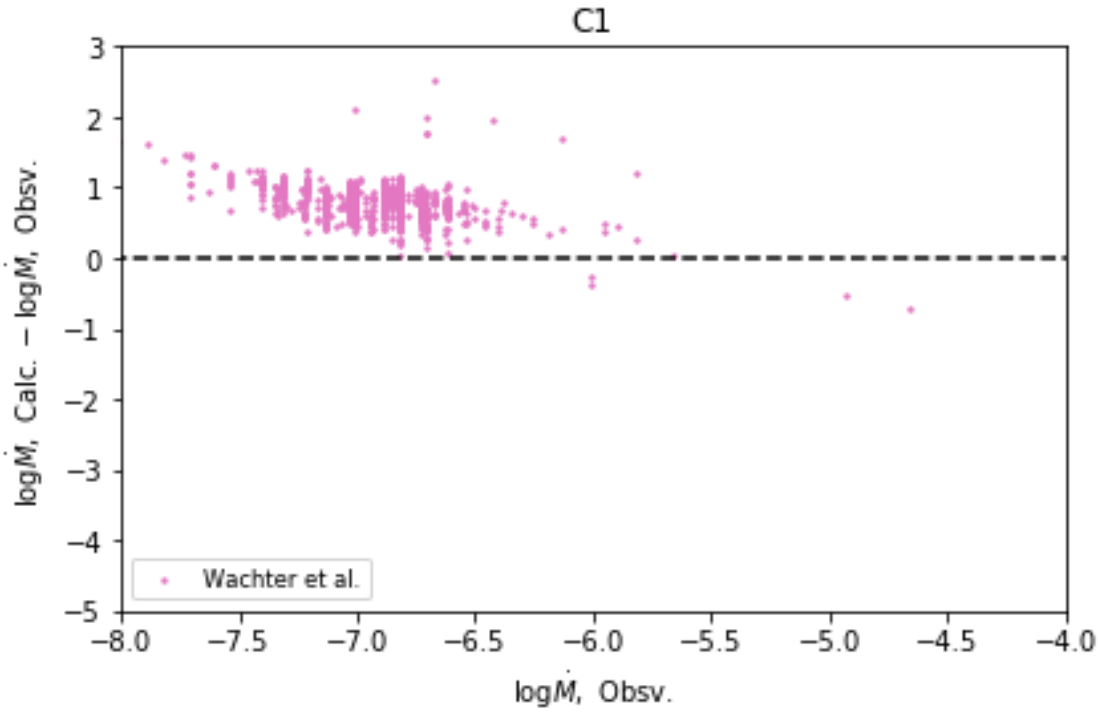

C1
3
2
1
0
−1
−2
−3
−4
−5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
log$\dot{M}$, Calc. − log$\dot{M}$, Obsv.
log$\dot{M}$, Obsv.
Fit to Dust

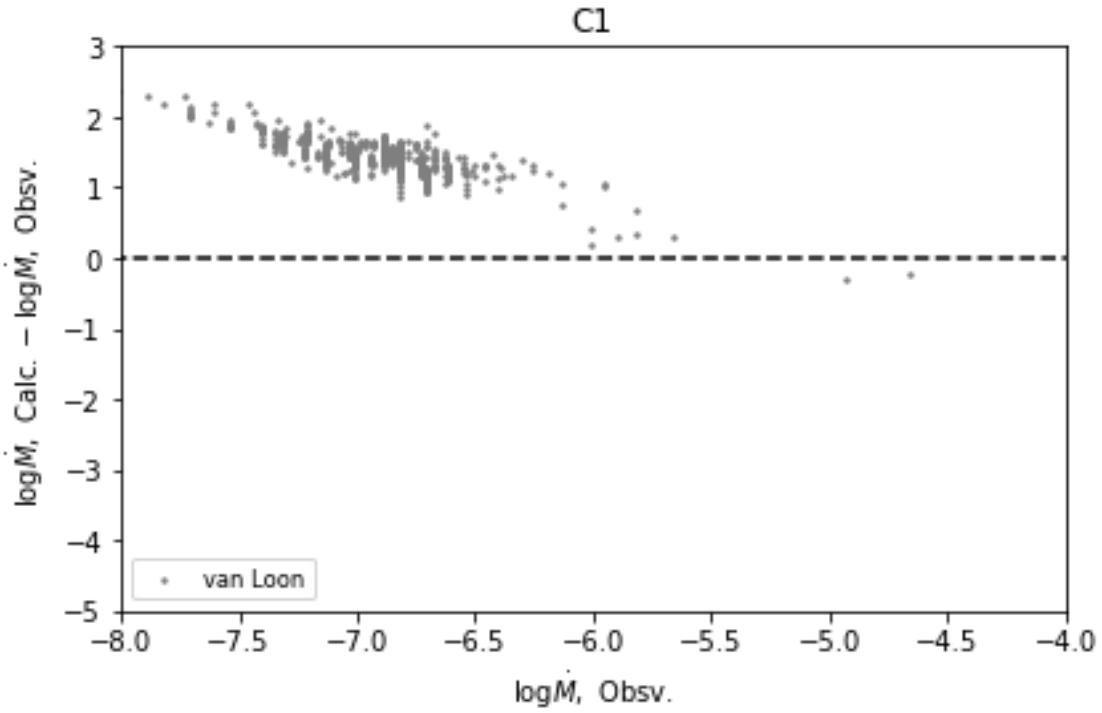

C1
3
2
1
0
−1
−2
−3
−4
−5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
log$\dot{M}$, Calc. − log$\dot{M}$, Obsv.
log$\dot{M}$, Obsv.
Fit to Deathzone

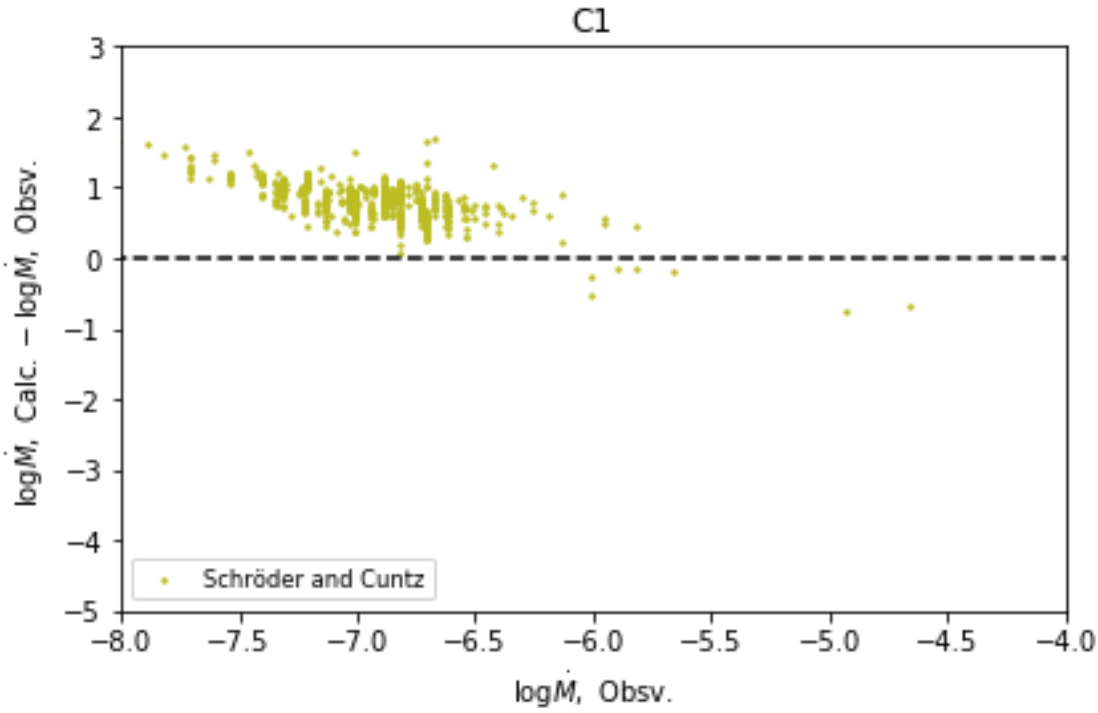

C1
3
2
1
0
−1
−2
−3
−4
−5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
log$\dot{M}$, Calc. − log$\dot{M}$, Obsv.
log$\dot{M}$, Obsv.
Reimers (1975)

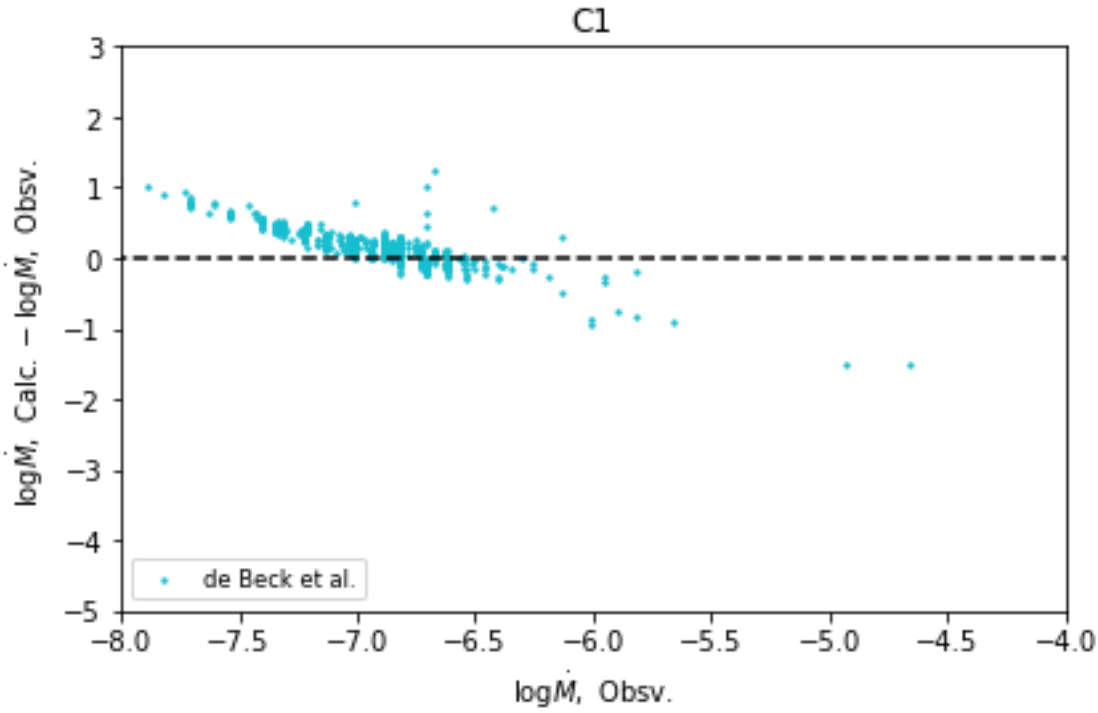

C1
3
2
1
0
−1
−2
−3
−4
−5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
logṀ, Calc. − logṀ, Obsv.
logṀ, Obsv.
Reimers (1977)

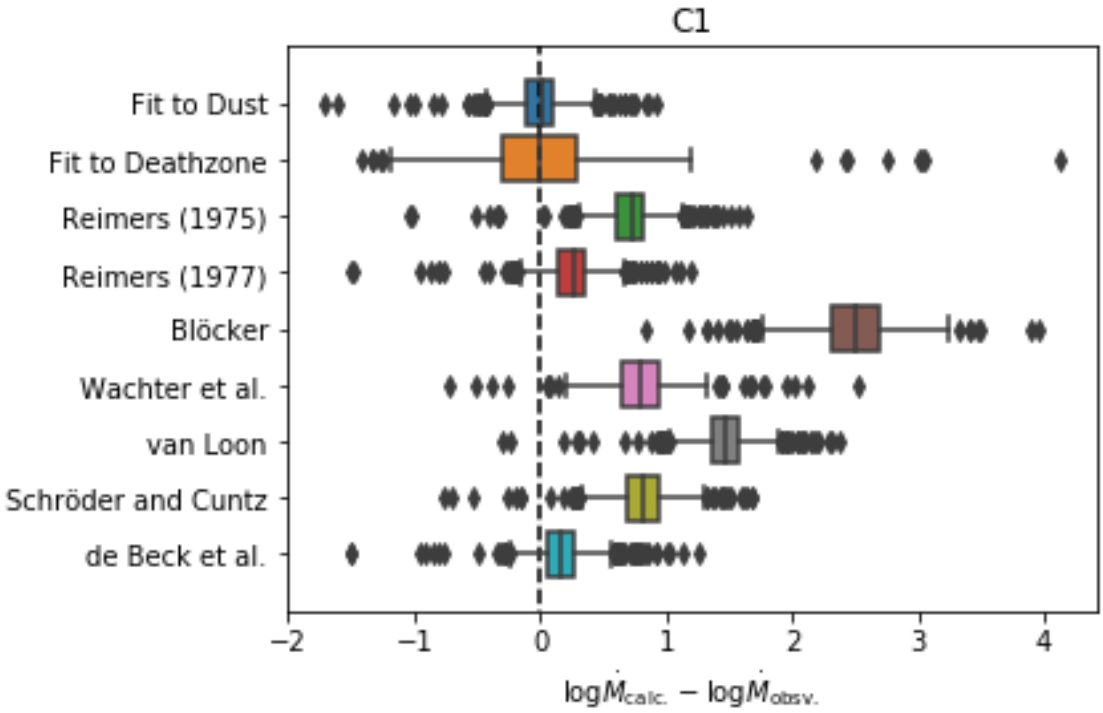

C1
3
2
1
0
−1
−2
−3
−4
−5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
log$\dot{M}$, Calc. − log$\dot{M}$, Obsv.
log$\dot{M}$, Obsv.
Blöcker


C1
3
2
1
0
−1
−2
−3
−4
−5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
log$\dot{M}$, Calc. − log$\dot{M}$, Obsv.
log$\dot{M}$, Obsv.
Wachter et al.


C1
3
2
1
0
−1
−2
−3
−4
−5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
van Loon
log$\dot{M}$, Calc. − log$\dot{M}$, Obsv.
log$\dot{M}$, Obsv.

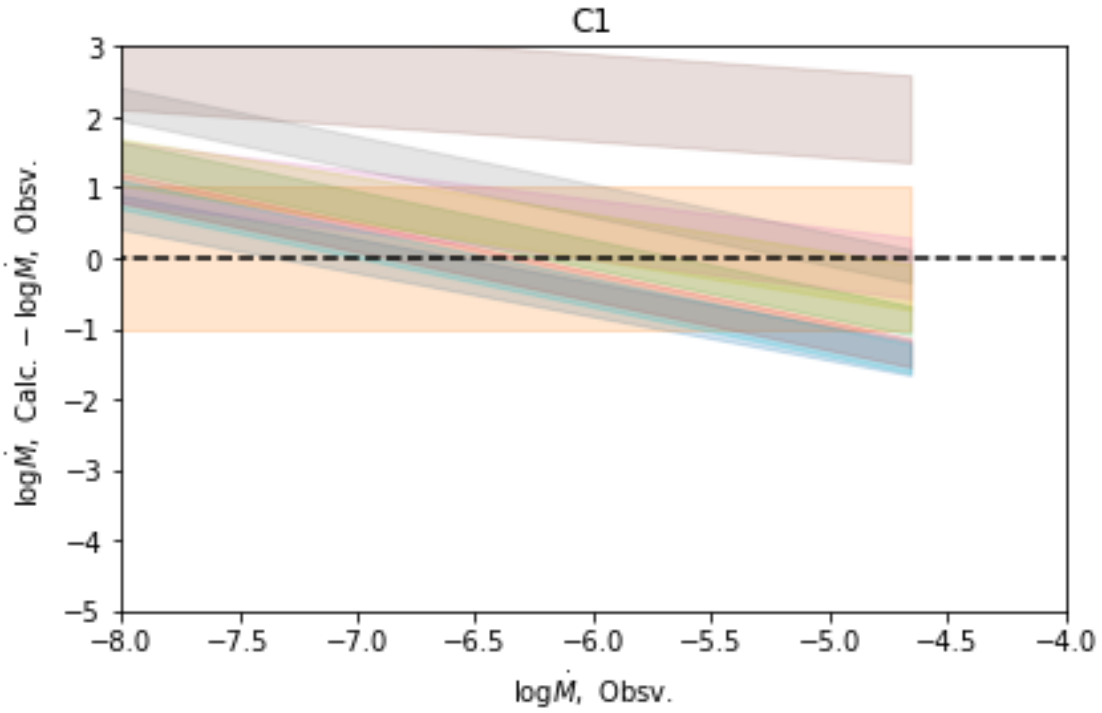

C1
3
2
1
0
−1
−2
−3
−4
−5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
log$\dot{M}$, Calc. − log$\dot{M}$, Obsv.
log$\dot{M}$, Obsv.
Schröder and Cuntz


C1
3
2
1
0
−1
−2
−3
−4
−5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
log$\dot{M}$, Calc. − log$\dot{M}$, Obsv.
log$\dot{M}$, Obsv.
de Beck et al.




```
Point List: [[-8.009394788576081, 0.9002711253057445], [-8.009394788576081,
0.41722640161043323], [-4.6549204736851335, -1.6554291223097195],
[-4.6549204736851335, -1.1723843986144082]]
x-values: [-8.00939479 -4.65492047]
 y-values: [[ 0.90027113 -1.1723844 ]
 [ 0.4172264  -1.65542912]]
Point List: [[-8.009394788576081, 1.0183843341415293], [-8.009394788576081,
-1.0184230694803214], [-4.6549204736851335, -1.0183621319739713],
[-4.6549204736851335, 1.0184452716478793]]
x-values: [-8.00939479 -4.65492047]
 y-values: [[ 1.01838433  1.01844527]
 [-1.01842307 -1.01836213]]
Point List: [[-8.009394788576081, 1.6591780336564068], [-8.009394788576081,
1.2540453709834685], [-4.6549204736851335, -1.0837288487433443],
[-4.6549204736851335, -0.678596186070406]]
x-values: [-8.00939479 -4.65492047]
 y-values: [[ 1.65917803 -0.67859619]
 [ 1.25404537 -1.08372885]]
Point List: [[-8.009394788576081, 1.2032460780066847], [-8.009394788576081,
0.7981134153337459], [-4.6549204736851335, -1.5396608043930673],
[-4.6549204736851335, -1.1345281417201285]]
x-values: [-8.00939479 -4.65492047]
 y-values: [[ 1.20324608 -1.13452814]
 [ 0.79811342 -1.5396608 ]]
```

```
Point List: [[-8.009394788576081, 3.3514098433071426], [-8.009394788576081,
2.115261982129148], [-4.6549204736851335, 1.3584355884142285],
[-4.6549204736851335, 2.5945834495922235]]
x-values: [-8.00939479 -4.65492047]
 y-values: [[3.35140984 2.59458345]
 [2.11526198 1.35843559]]
Point List: [[-8.009394788576081, 1.6520446105742448], [-8.009394788576081,
0.7966595653393251], [-4.6549204736851335, -0.5646750390183495],
[-4.6549204736851335, 0.29071000621657017]]
x-values: [-8.00939479 -4.65492047]
 y-values: [[ 1.65204461  0.29071001]
 [ 0.79665957 -0.56467504]]
Point List: [[-8.009394788576081, 2.4261030176066996], [-8.009394788576081,
1.9595727511369203], [-4.6549204736851335, -0.33865363066023635],
[-4.6549204736851335, 0.12787663580954291]]
x-values: [-8.00939479 -4.65492047]
 y-values: [[ 2.42610302  0.12787664]
 [ 1.95957275 -0.33865363]]
Point List: [[-8.009394788576081, 1.6940069608397637], [-8.009394788576081,
1.0464921916840306], [-4.6549204736851335, -0.7303066241541152],
[-4.6549204736851335, -0.08279185499838215]]
x-values: [-8.00939479 -4.65492047]
 y-values: [[ 1.69400696 -0.08279185]
 [ 1.04649219 -0.73030662]]
Point List: [[-8.009394788576081, 1.1113936421258392], [-8.009394788576081,
0.7082025060454873], [-4.6549204736851335, -1.6227831621529982],
[-4.6549204736851335, -1.2195920260726463]]
x-values: [-8.00939479 -4.65492047]
 y-values: [[ 1.11139364 -1.21959203]
 [ 0.70820251 -1.62278316]]
```



---

1 Method does not work. Saved for archival purposes.

def deathline_analysis_use_const_mdot (star_data,spec_mode_label = ''): # Find deathzone using lines of pseudo-constant Mdot bound_line_1_indices = [] bound_line_2_indices = [] i = 0 while i < len(star_data[MdotIndex]): if np.log10(star_data[MdotIndex][i]) < -7.4: i +=1 elif np.log10(star_data[MdotIndex][i]) > -7.3: i +=1 else: bound_line_1_indices.append(i) i +=1 i = 0 while i < len(star_data[MdotIndex]): if np.log10(star_data[MdotIndex][i]) < -5.15: i +=1 elif np.log10(star_data[MdotIndex][i]) > -5.05: i +=1 else: bound_line_2_indices.append(i) i +=1 print(bound_line_1_indices) print(bound_line_2_indices)

```
plot.scatter(star_data[LIndex], star_data[MIndex],s=2)
plot.scatter(star_data[LIndex][bound_line_1_indices], star_data[MIndex][bound_line_1_indices],s
plot.scatter(star_data[LIndex][bound_line_2_indices], star_data[MIndex][bound_line_2_indices],s
#plot.plot(10**line_x_vals,10**line_y_vals,linestyle='--',color='black')
#plot.plot(10**line_x_vals,10**line_y_vals_up,linestyle='--',color='red')
#plot.plot(10**line_x_vals,10**line_y_vals_down,linestyle='--',color='red')
#plot.errorbar(star_data[LIndex],star_data[MIndex],xerr=star_data[LErrorIndex],fmt='none',marke
plot.xscale('log')
plot.yscale('log')
plot.xlabel(r'$L$')
```

```
plot.ylabel(r'$M$')
plot.title(spec_mode_label)
'''
# Find width of deltaLogL and deltaLogM using fit line, and plot them
logL_left = (np.average(line_y_vals) - deathline_coeffs[0]-std_multiplier*np.nanstd(np.log10(st
logL_right = (np.average(line_y_vals) - deathline_coeffs[0]+std_multiplier*np.nanstd(np.log10(s
delta_logL = logL_right - logL_left
print(' logL =',delta_logL)
plot.plot([10**logL_left,10**logL_right],[10**(np.average(line_y_vals)),10**(np.average(line_y_
delta_logM = line_y_vals_up[1] - line_y_vals_down[1]
print(' logM =',delta_logM)
plot.vlines(10**np.average(line_x_vals),10**(np.average(line_y_vals)-delta_logM/2),10**(np.aver
# Using data, calculate deltaLogMdot
delta_logMdot = np.nanmax(np.log10(star_data[MdotIndex])) - np.nanmin(np.log10(star_data[MdotIn
print(' logMdot =',delta_logMdot)
# Calculate B = deltaLogMdot/deltaLogL and C = deltaLogMdot/deltaLogM
B = delta_logMdot/delta_logL
C = delta_logMdot/delta_logM
print('For set '+spec_mode_label+', B=',B, ', C=',C )
'''
plot.show()
plot.close()
```

[350]:
```
def sort_data (stellar_data,sorting_index,num_bins=2):
    a = stellar_data[sorting_index]
    ind=np.argsort(a)
    sorted_data=np.asarray(np.transpose(combined_data_set)[ind])

    return np.array_split(sorted_data,num_bins)
```

def deathline_analysis_use_bins (star_data,sorting_index,num_bins=2,spec_mode_label = ''): # Find deathzone using lines through bins sorted_data = sort_data(star_data, sorting_index, num_bins = num_bins) for i in range (0, len(sorted_data)): bin_label = str(np.log10(np.nanmin(np.transpose(sorted_data[i])[MdotIndex])))[0:7] +'<Mdot<' + str(np.log10(np.nanmax(np.transpose(sorted_data[i])[MdotIndex])))[0:7] plot.scatter(np.transpose(sorted_data[i])[LIndex], np.transpose(sorted_data[i])[MIndex],s=2,label=bin_label)

```
plot.xscale('log')
plot.yscale('log')
plot.xlabel(r'$L$')
plot.ylabel(r'$M$')
plot.title(spec_mode_label)
plot.legend()
'''
# Find width of deltaLogL and deltaLogM using fit line, and plot them
logL_left = (np.average(line_y_vals) - deathline_coeffs[0]-std_multiplier*np.nanstd(np.log10(st
logL_right = (np.average(line_y_vals) - deathline_coeffs[0]+std_multiplier*np.nanstd(np.log10(s
delta_logL = logL_right - logL_left
```

```
print(' logL =',delta_logL)
plot.plot([10**logL_left,10**logL_right],[10**(np.average(line_y_vals)),10**(np.average(line_y
delta_logM = line_y_vals_up[1] - line_y_vals_down[1]
print(' logM =',delta_logM)
plot.vlines(10**np.average(line_x_vals),10**(np.average(line_y_vals)-delta_logM/2),10**(np.aver
# Using data, calculate deltaLogMdot
delta_logMdot = np.nanmax(np.log10(star_data[MdotIndex])) - np.nanmin(np.log10(star_data[MdotIn
print(' logMdot =',delta_logMdot)
# Calculate B = deltaLogMdot/deltaLogL and C = deltaLogMdot/deltaLogM
B = delta_logMdot/delta_logL
C = delta_logMdot/delta_logM
print('For set '+spec_mode_label+', B=',B, ', C=',C )
'''
plot.show()
plot.close()
```

deathline_analysis_use_bins (combined_data_set, MdotIndex, num_bins = 3, spec_mode_label = 'All Stars')

c0_true_mdots = deathzone_values_C0[0] + deathzone_values_C0[1]*np.log10(C0StarData_n[LIndex])* - *deathzone_values_C1[2]*np.log10(C0StarData_n[MIndex])

[351]:
```
def deathzone_mlr_check (star_data, deathzone_ABC,spec_mode_label):

    print(spec_mode_label+':')
    logLData = np.log10(star_data[LIndex])
    logMData = np.log10(star_data[MIndex])
    logPData = np.log10(star_data[PIndex])
    logRData = np.log10(star_data[RIndex])
    logTData = np.log10(star_data[TeffCalcIndex])
    logMdotData = np.log10(star_data[MdotIndex])

    color_list = ['tab:blue','tab:orange','tab:green','tab:red','tab:
↪purple','tab:brown', 'tab:pink', 'tab:gray', 'tab:olive', 'tab:cyan']

    # log Mdot = A + BlogL + ClogP
    if spec_mode_label[0] == 'M':
        RLM = RLM_M
    if spec_mode_label[0] == 'C':
        RLM = RLM_C
    if spec_mode_label == 'M0':
        PMR = PMR_M0
    if spec_mode_label == 'M1':
        PMR = PMR_M1
    if spec_mode_label == 'C0':
        PMR = PMR_C0
    if spec_mode_label == 'C0x':
        PMR = PMR_C0
```

```
    if spec_mode_label == 'C0 and C0x':
        PMR = PMR_C0
    if spec_mode_label == 'C1':
        PMR = PMR_C1
    # Take stellar data and associated death zone analysis results to see if we␣
↪get MLR fit results similar to fitting to
    # dust mass loss rate directly, or if we recover our deathzone formulas
    calculated_mdot = deathzone_ABC[0] + deathzone_ABC[1]*logLData +␣
↪deathzone_ABC[2]*logMData


    print('Death Zone Values:',deathzone_ABC)

    artificial_fit = multilinear_fit_2var([logLData,logPData],calculated_mdot)
    LPCoeffs = artificial_fit[0]
    a = LPCoeffs[0]
    arlm = RLM[0]
    aRLM = arlm
    apmr = PMR[0]
    aPMR = apmr
    c = LPCoeffs[2]
    cpmr = PMR[2]
    cPMR = cpmr
    b = LPCoeffs[1]
    brlm = RLM[1]
    bRLM = brlm
    bpmr = PMR[1]
    bPMR = bpmr
    crlm = RLM[2]
    cRLM = crlm
    PMLCoeffs = calculate_PML_coeffs(RLM, PMR)
    aPML = PMLCoeffs[0]
    bPML = PMLCoeffs[1]
    cPML = PMLCoeffs[2]

    Anew = a - arlm + apmr*c + arlm*c*cpmr
    Bnew = b - brlm + brlm*c*cpmr
    Cnew = bpmr*c - crlm + c*cpmr*crlm
    print('Death Zone Mdot Fit:',artificial_fit[0])
    print('     LM form: logMdot =',Anew+arlm,' + ',Bnew+brlm,'logL␣
↪-',-(Cnew+crlm),'logM')


    data_fit = multilinear_fit_2var([logLData,logPData],logMdotData)
    LPCoeffs = data_fit[0]
    a = LPCoeffs[0]
    arlm = RLM[0]
```

```
    aRLM = arlm
    apmr = PMR[0]
    aPMR = apmr
    c = LPCoeffs[2]
    cpmr = PMR[2]
    cPMR = cpmr
    b = LPCoeffs[1]
    brlm = RLM[1]
    bRLM = brlm
    bpmr = PMR[1]
    bPMR = bpmr
    crlm = RLM[2]
    cRLM = crlm
    PMLCoeffs = calculate_PML_coeffs(RLM, PMR)
    aPML = PMLCoeffs[0]
    bPML = PMLCoeffs[1]
    cPML = PMLCoeffs[2]

    Anew = a - arlm + apmr*c + arlm*c*cpmr
    Bnew = b - brlm + brlm*c*cpmr
    Cnew = bpmr*c - crlm + c*cpmr*crlm
    print('Observed Mdot Fit:',data_fit[0])
    print('     LM form: logMdot =',Anew+arlm,' + ',Bnew+brlm,'logL␣
↪-',-(Cnew+crlm),'logM')
```

[352]:
```
deathzone_mlr_check(M0StarData, deathzone_values_M0,'M0')
print('\n')


deathzone_mlr_check(M1StarData, deathzone_values_M1,'M1')
print('\n')


deathzone_mlr_check(C0StarData, deathzone_values_C0_and_C0x,'C0 and C0x')
print('\n')


deathzone_mlr_check(C1StarData, deathzone_values_C1, 'C1')
print('\n')
```

```
M0:
Death Zone Values: (-34.975963392304195, 8.131127742677283, -13.698081169196328)
Death Zone Mdot Fit: [-6.04561596 -6.73129033 10.62971099]
     LM form: logMdot = -34.985078533852665  +  8.135742352261536 logL -
13.70145297927229 logM
Observed Mdot Fit: [-7.38815877 -1.00566532  2.03877208]
     LM form: logMdot = -12.938730017492958  +  1.845822471134391 logL -
```

```
2.627930313100934 logM

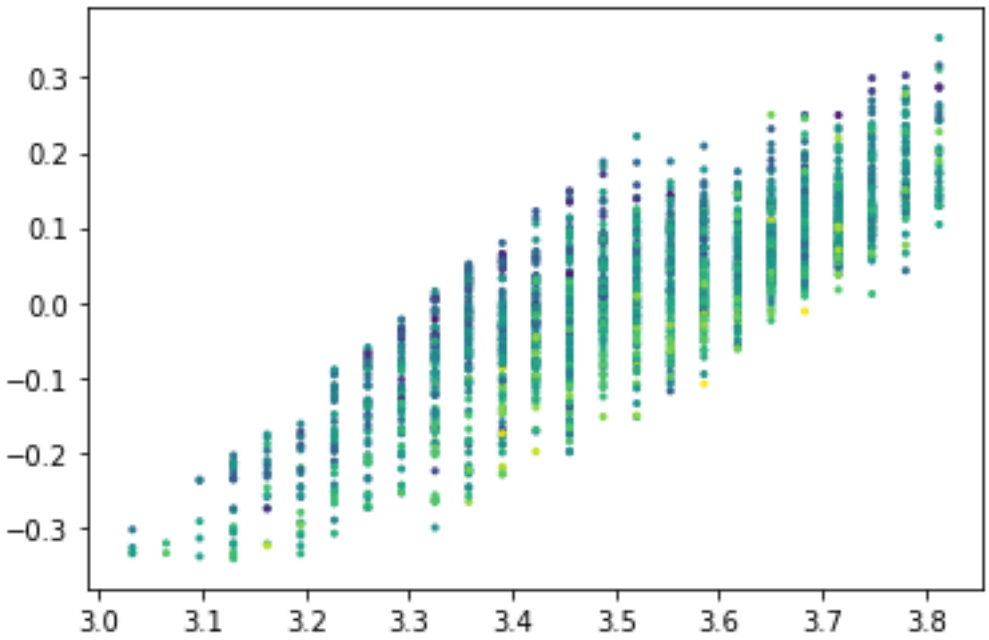

M1:
Death Zone Values: (-42.82979371999287, 10.152304120143375, -20.474047210262466)
Death Zone Mdot Fit: [  0.0502318  -13.88606997  22.18649652]
     LM form: logMdot = -42.82228848299002  +  10.148561710001893 logL -
20.479689341727624 logM
Observed Mdot Fit: [-9.08935829 -0.77304207  2.71289306]
     LM form: logMdot = -14.331671442964444  +  2.1658349824618766 logL -
2.504190194588611 logM


C0 and C0x:
Death Zone Values: (-41.078937026141794, 9.635192545710597, -13.910423664947327)
Death Zone Mdot Fit: [-15.21945943  -3.59362036   8.77747733]
     LM form: logMdot = -41.068155877474055  +  9.622548262718038 logL -
13.903111553374876 logM
Observed Mdot Fit: [-11.36909618  -1.15826195   3.5757003 ]
     LM form: logMdot = -21.899136660341252  +  4.225637806117329 logL -
5.663741216898915 logM


C1:
Death Zone Values: (-28.14202038380528, 5.88519600584648, -9.437891494192556)
Death Zone Mdot Fit: [-10.64416955  -3.93964904   8.49410234]
     LM form: logMdot = -28.151448837407983  +  5.888791842505767 logL -
9.440939933145183 logM
Observed Mdot Fit: [-10.76683497  -0.22406252   2.10354095]
     LM form: logMdot = -15.10246424487756  +  2.20992367511773 logL -
2.3380226554375763 logM
```

[353]:
```
plot.scatter(np.log10(M0StarData[LIndex]),np.log10(M0StarData[MIndex]),c=np.
↪log10(M0StarData[MdotIndex]),s=4)
```

[353]:
```
<matplotlib.collections.PathCollection at 0x223a872ed48>
```



[354]:
```
print(np.nanmin(M0StarData[MdotIndex]))
print(np.nanmin(M1StarData[MdotIndex]))
print(np.nanmin(C0StarData[MdotIndex]))
print(np.nanmin(C1StarData[MdotIndex]))
print(np.nanmax(M0StarData[MdotIndex]))
print(np.nanmax(M1StarData[MdotIndex]))
print(np.nanmax(C0StarData[MdotIndex]))
print(np.nanmax(C1StarData[MdotIndex]))
```

```
1.4564999999999998e-08
1.07781e-08
1.1416999999999999e-08
9.785999999999999e-09
7.2825e-06
1.1069400000000001e-05
1.165e-05
2.2135e-05
```

[355]:
```
np.histogram(np.log10(M0StarData[MdotIndex]),bins=8,range=(-9,-5))
```

[355]:
```
(array([  0,   0,  18, 146, 636, 948, 187,  16], dtype=int64),
 array([-9. , -8.5, -8. , -7.5, -7. , -6.5, -6. , -5.5, -5. ]))
```

[356]:
```
def bin_locations (star_mdot_data):
    # Digitize will return which bin a value falls into
```

```
    digitized_list = np.digitize(np.
↪log10(star_mdot_data),[-10,-9,-8,-7,-6,-5,-4])
    bin_0, bin_1, bin_2, bin_3, bin_4, bin_5, bin_6 = [],[],[],[],[],[],[]
    for i in range(0, len(star_mdot_data)):
        if digitized_list[i] == 0:
            bin_0.append(i)
        elif digitized_list[i] == 1:
            bin_1.append(i)
        elif digitized_list[i] == 2:
            bin_2.append(i)
        elif digitized_list[i] == 3:
            bin_3.append(i)
        elif digitized_list[i] == 4:
            bin_4.append(i)
        elif digitized_list[i] == 5:
            bin_5.append(i)
        elif digitized_list[i] == 6:
            bin_6.append(i)
        else:
            print('Sorting Failed for index',i)
    return bin_0, bin_1, bin_2, bin_3, bin_4, bin_5, bin_6
```

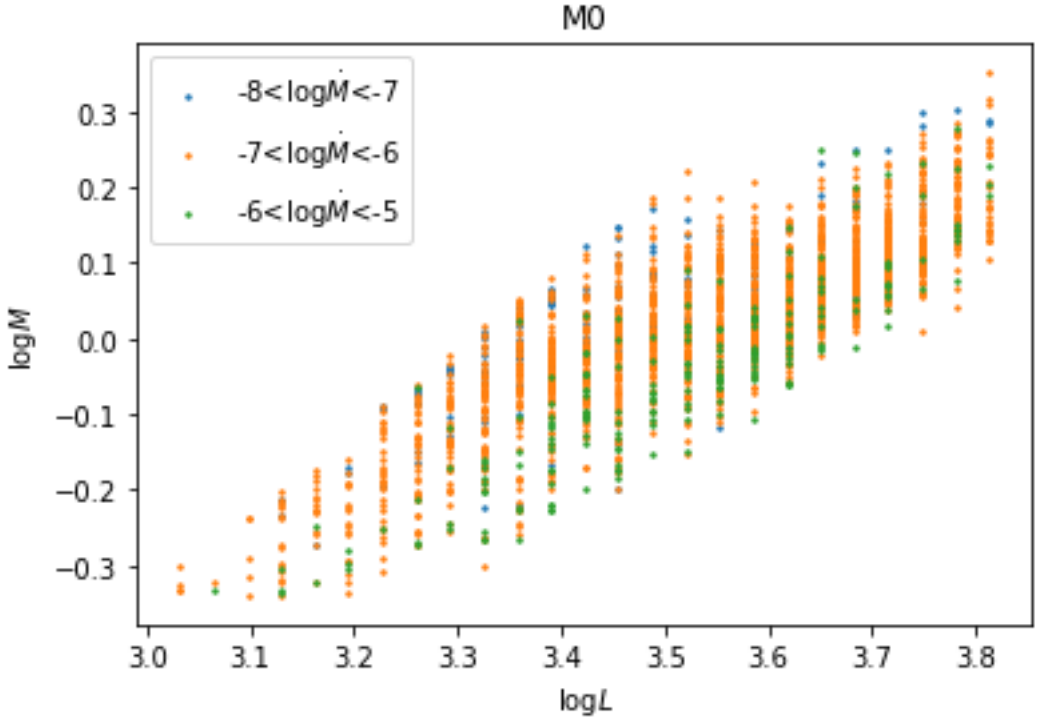
[357]:
```
def plot_binned_stellar_data (star_data, bin_list,spec_mode_label=''):
    for i in range(0, len(bin_list)):
        logLData = np.log10(star_data[LIndex][bin_list[i]])
        logMData = np.log10(star_data[MIndex][bin_list[i]])
        logMdotData = np.log10(star_data[MdotIndex][bin_list[i]])
        if len(logLData)>0:
            plot.scatter(logLData,logMData,label=str(-11+i)+'<'+r'$\log␣
↪\dot{M}$'+'<'+str(-10+i),s=2)
            print('Min observed value in bin:',np.nanmin(logMdotData))
            print('Max observed value in bin:',np.nanmax(logMdotData))
        else:
            print('Bin list ',i,' is empty. Continuing to next bin.')
    plot.legend()
    plot.title(spec_mode_label)
    plot.xlabel(r'$\log L$')
    plot.ylabel(r'$\log M$')
    plot.show()
    plot.close()
```

[358]:
```
M0_bin_ids = bin_locations (M0StarData[MdotIndex])
M1_bin_ids = bin_locations (M1StarData[MdotIndex])
C0_bin_ids = bin_locations (C0StarData[MdotIndex])
C1_bin_ids = bin_locations (C1StarData[MdotIndex])
```

```
Sorting Failed for index 0
```

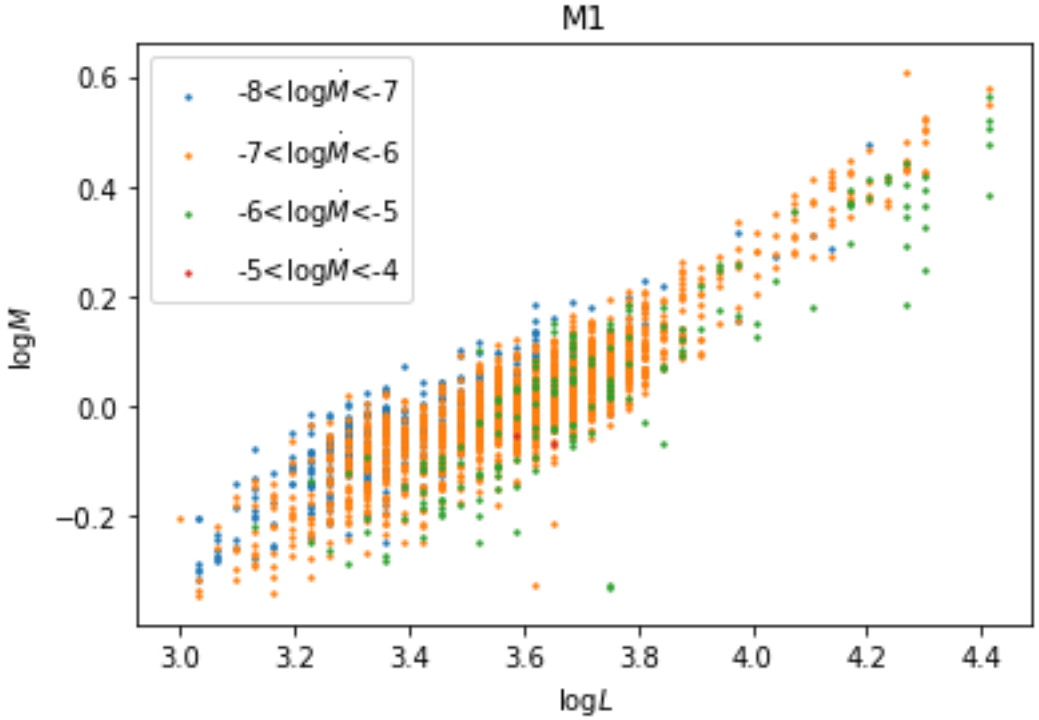
[359]:
```
plot_binned_stellar_data(M0StarData,M0_bin_ids,'M0')
plot_binned_stellar_data(M1StarData,M1_bin_ids,'M1')
plot_binned_stellar_data(C0StarData,C0_bin_ids,'C0')
plot_binned_stellar_data(C1StarData,C1_bin_ids,'C1')
```

```
Bin list  0  is empty. Continuing to next bin.
Bin list  1  is empty. Continuing to next bin.
Bin list  2  is empty. Continuing to next bin.
Min observed value in bin: -7.8366895110363135
Max observed value in bin: -7.004180598330078
Min observed value in bin: -6.9915914710220575
Max observed value in bin: -6.004180598330078
Min observed value in bin: -5.9915914710220575
Max observed value in bin: -5.137719506700295
Bin list  6  is empty. Continuing to next bin.
```

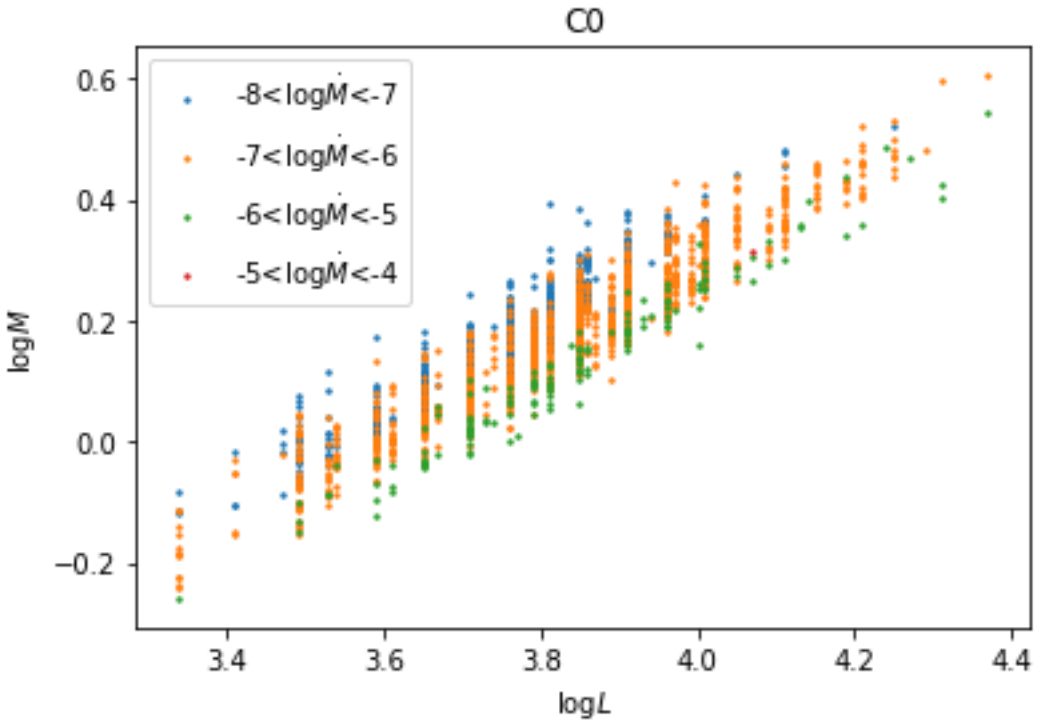



```
Bin list  0  is empty. Continuing to next bin.
Bin list  1  is empty. Continuing to next bin.
Bin list  2  is empty. Continuing to next bin.
Min observed value in bin: -7.967457791305338
Max observed value in bin: -7.004180598330078
Min observed value in bin: -6.9915914710220575
Max observed value in bin: -6.004180598330078
Min observed value in bin: -5.9915914710220575
```

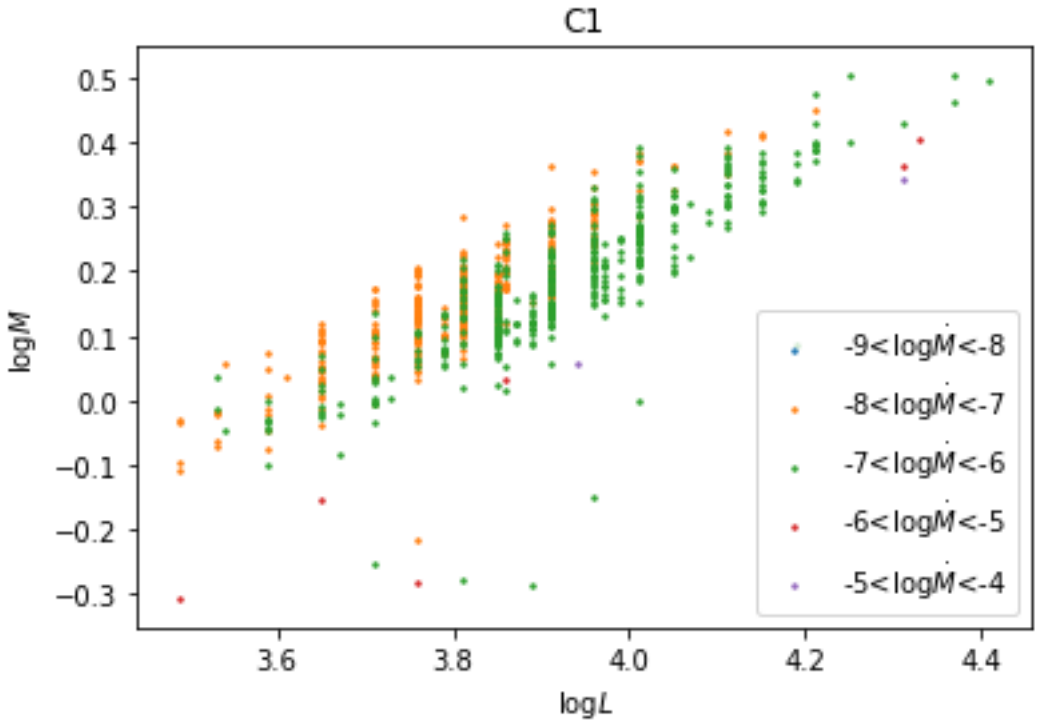
```
Max observed value in bin: -5.073261517473377
Min observed value in bin: -4.9915914710220575
Max observed value in bin: -4.955875918755522
```




```
Bin list  0  is empty. Continuing to next bin.
Bin list  1  is empty. Continuing to next bin.
Bin list  2  is empty. Continuing to next bin.
Min observed value in bin: -7.942447998945467
Max observed value in bin: -7.0042551489236695
Min observed value in bin: -6.989191402487793
Max observed value in bin: -6.00939478857608
Min observed value in bin: -5.984284067993049
Max observed value in bin: -5.101165161931726
Min observed value in bin: -4.933674074637962
Max observed value in bin: -4.933674074637962
```




```
Bin list  0  is empty. Continuing to next bin.
Bin list  1  is empty. Continuing to next bin.
Min observed value in bin: -8.009394788576081
Max observed value in bin: -8.009394788576081
Min observed value in bin: -7.892281389479737
Max observed value in bin: -7.0042551489236695
Min observed value in bin: -6.984284067993049
Max observed value in bin: -6.0042551489236695
Min observed value in bin: -5.955950469349115
Max observed value in bin: -5.6549204736851335
Min observed value in bin: -4.933674074637962
Max observed value in bin: -4.6549204736851335
```



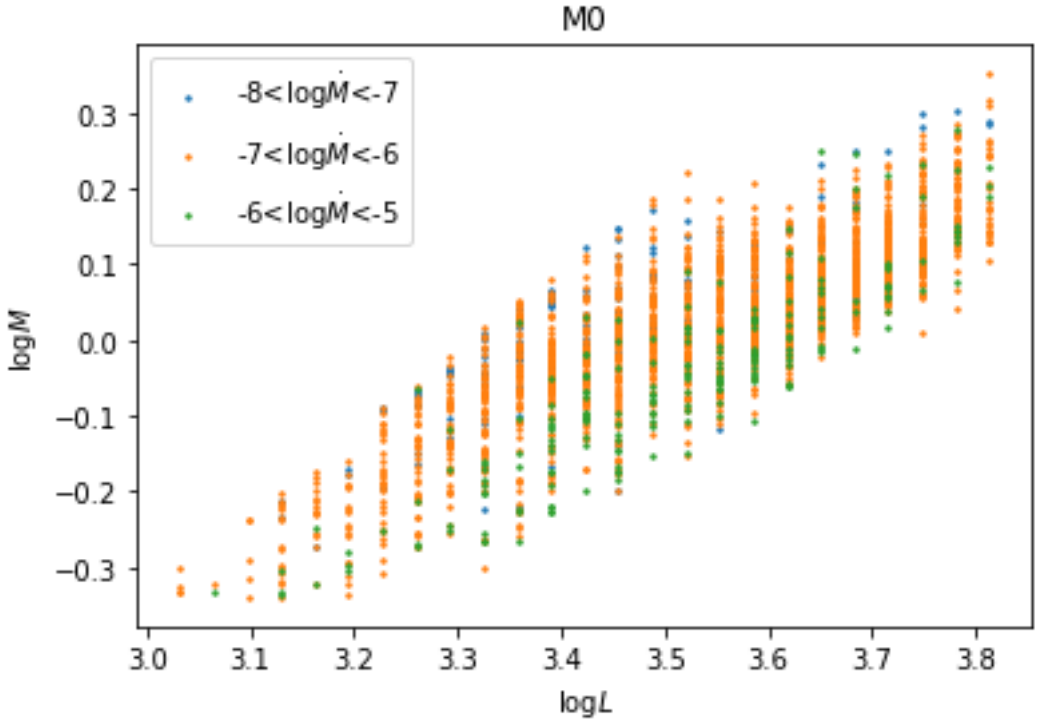
deathzone_M0_mdot = 10**(deathzone_values_M0[0] + deathzone_values_M0[1]*np.log10(M0StarData[LIndex])+ deathzone_values_M0[2]*np.log10(M0StarData[MIndex])) deathzone_M1_mdot = 10**(deathzone_values_M1[0] + deathzone_values_M1[1]*np.log10(M1StarData[LIndex])+ deathzone_values_M1[2]*np.log10(M1StarData[MIndex])) deathzone_C0_mdot = 10**(deathzone_values_C0[0] + deathzone_values_C0[1]*np.log10(C0StarData[LIndex])+ deathzone_values_C0[2]*np.log10(C0StarData[MIndex])) deathzone_C1_mdot = 10**(deathzone_values_C1[0] + deathzone_values_C1[1]*np.log10(C1StarData[LIndex])+ deathzone_values_C1[2]*np.log10(C1StarData[MIndex]))

M0_bin_ids = bin_locations (deathzone_M0_mdot) M1_bin_ids = bin_locations (deathzone_M1_mdot) C0_bin_ids = bin_locations (deathzone_C0_mdot) C1_bin_ids = bin_locations (deathzone_C1_mdot)

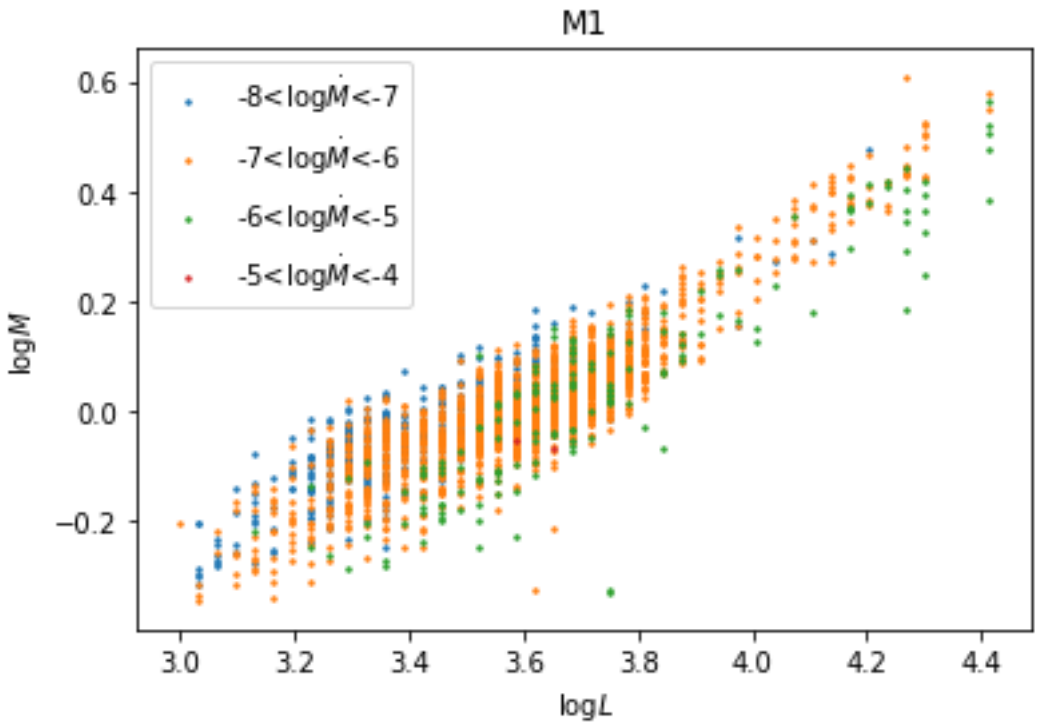
[360]:
```
plot_binned_stellar_data(M0StarData,M0_bin_ids,'M0')
plot_binned_stellar_data(M1StarData,M1_bin_ids,'M1')
plot_binned_stellar_data(C0StarData,C0_bin_ids,'C0')
plot_binned_stellar_data(C1StarData,C1_bin_ids,'C1')
```

```
Bin list  0  is empty. Continuing to next bin.
Bin list  1  is empty. Continuing to next bin.
Bin list  2  is empty. Continuing to next bin.
Min observed value in bin: -7.8366895110363135
Max observed value in bin: -7.004180598330078
```

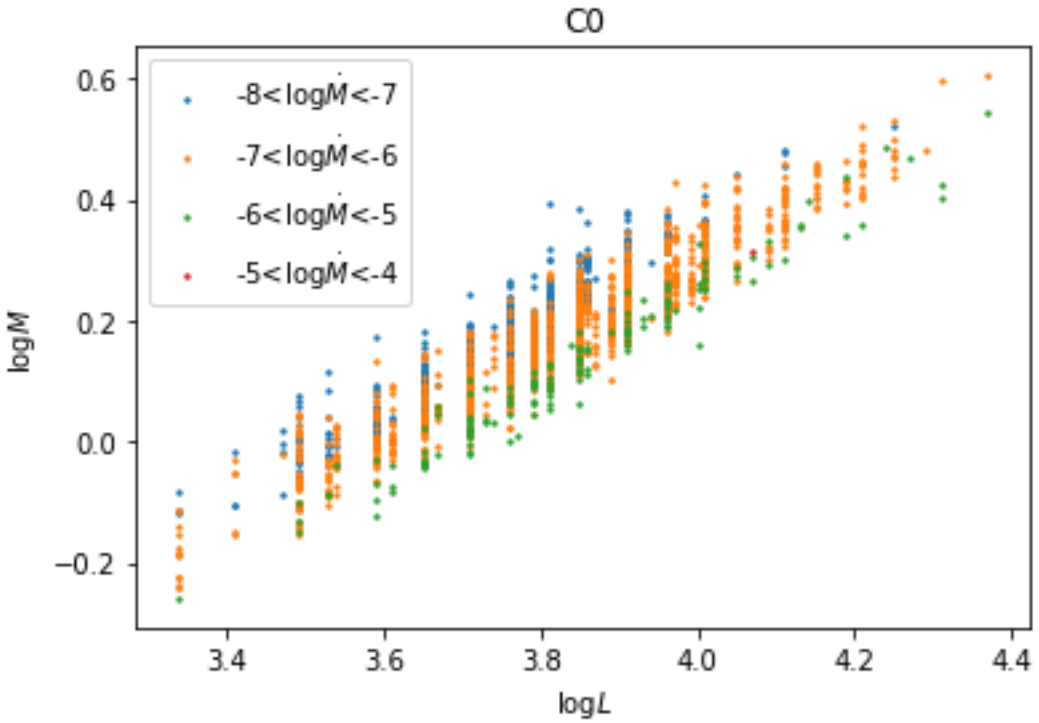
```
Min observed value in bin: -6.9915914710220575
Max observed value in bin: -6.004180598330078
Min observed value in bin: -5.9915914710220575
Max observed value in bin: -5.137719506700295
Bin list  6  is empty. Continuing to next bin.
```



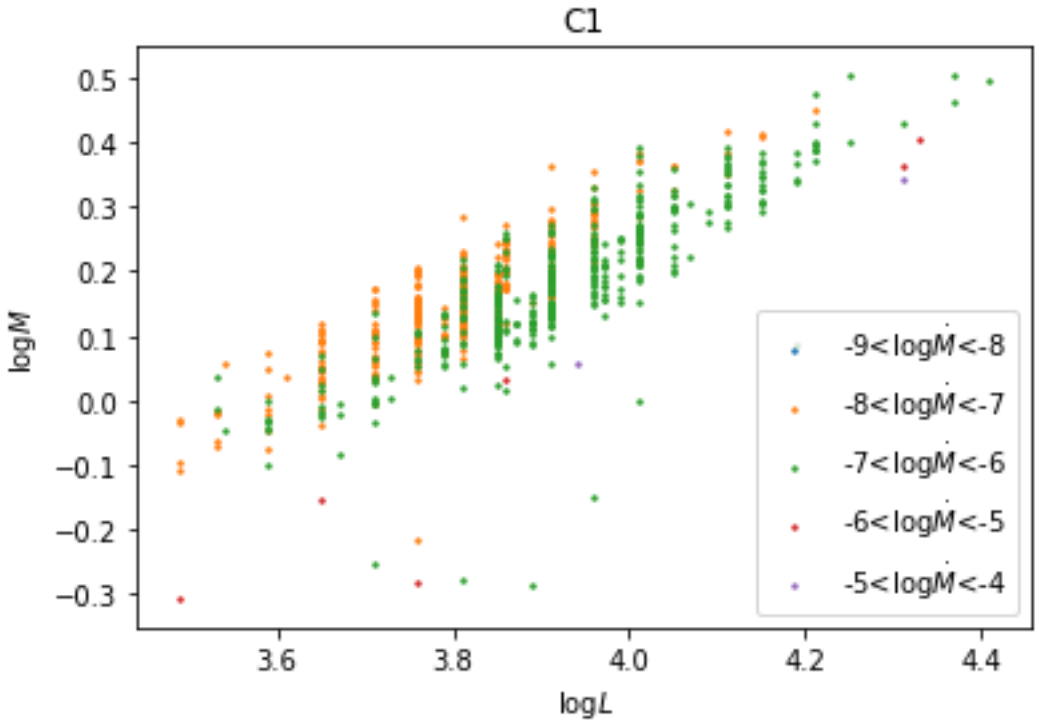

```
Bin list  0  is empty. Continuing to next bin.
Bin list  1  is empty. Continuing to next bin.
Bin list  2  is empty. Continuing to next bin.
Min observed value in bin: -7.967457791305338
Max observed value in bin: -7.004180598330078
Min observed value in bin: -6.9915914710220575
Max observed value in bin: -6.004180598330078
Min observed value in bin: -5.9915914710220575
Max observed value in bin: -5.073261517473377
Min observed value in bin: -4.9915914710220575
Max observed value in bin: -4.955875918755522
```




```
Bin list  0  is empty. Continuing to next bin.
Bin list  1  is empty. Continuing to next bin.
Bin list  2  is empty. Continuing to next bin.
Min observed value in bin: -7.942447998945467
Max observed value in bin: -7.0042551489236695
Min observed value in bin: -6.989191402487793
Max observed value in bin: -6.00939478857608
Min observed value in bin: -5.984284067993049
Max observed value in bin: -5.101165161931726
Min observed value in bin: -4.933674074637962
Max observed value in bin: -4.933674074637962
```




```
Bin list  0  is empty. Continuing to next bin.
Bin list  1  is empty. Continuing to next bin.
Min observed value in bin: -8.009394788576081
Max observed value in bin: -8.009394788576081
Min observed value in bin: -7.892281389479737
Max observed value in bin: -7.0042551489236695
Min observed value in bin: -6.984284067993049
Max observed value in bin: -6.0042551489236695
Min observed value in bin: -5.955950469349115
Max observed value in bin: -5.6549204736851335
Min observed value in bin: -4.933674074637962
Max observed value in bin: -4.6549204736851335
```



[361]:
```
def quick_mlr_fit(star_data):
    logLData = np.log10(star_data[LIndex])
    logPData = np.log10(star_data[PIndex])
    logMdotData = np.log10(star_data[MdotIndex])
    fit = multilinear_fit_2var([logLData,logPData],logMdotData)
    return fit
```

[362]:
```
quick_mlr_fit(M0StarData)
```

[362]:
```
(array([-7.38815877, -1.00566532,  2.03877208]),
 0.20636545213502622,
 Intercept    0.168589
 x0           0.076281
 x1           0.093964
 dtype: float64,
             Intercept        x0        x1
 Intercept    0.233797 -0.067902  0.002839
 x0          -0.067902  0.047865 -0.045992
 x1           0.002839 -0.045992  0.072627)
```

[363]:
```
coeffs_M0 = quick_mlr_fit(M0StarData)[0]
coeffs_M1 = quick_mlr_fit(M1StarData)[0]
coeffs_C0 = quick_mlr_fit(C0StarData)[0]
```

```
coeffs_C1 = quick_mlr_fit(C1StarData)[0]
```

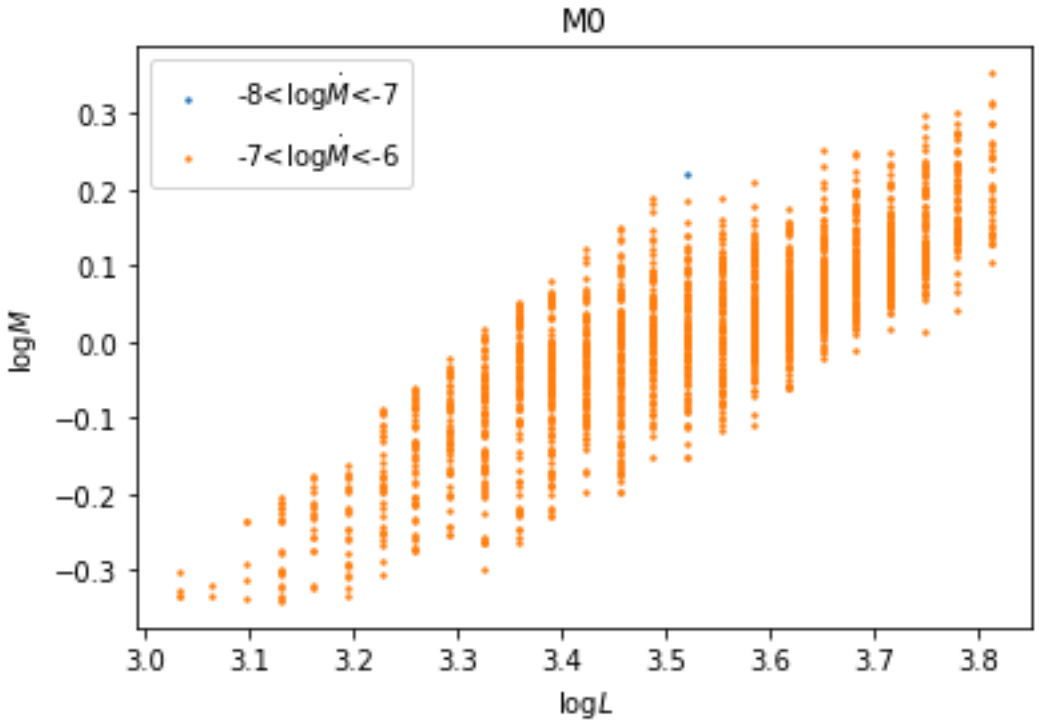
[364]:
```
dustfit_M0_mdot = 10**(coeffs_M0[0] + coeffs_M0[1]*np.
 ↪log10(M0StarData[LIndex])+ coeffs_M0[2]*np.log10(M0StarData[PIndex]))
dustfit_M1_mdot = 10**(coeffs_M1[0] + coeffs_M1[1]*np.
 ↪log10(M1StarData[LIndex])+ coeffs_M1[2]*np.log10(M1StarData[PIndex]))
dustfit_C0_mdot = 10**(coeffs_C0[0] + coeffs_C0[1]*np.
 ↪log10(C0StarData[LIndex])+ coeffs_C0[2]*np.log10(C0StarData[PIndex]))
dustfit_C1_mdot = 10**(coeffs_C1[0] + coeffs_C1[1]*np.
 ↪log10(C1StarData[LIndex])+ coeffs_C1[2]*np.log10(C1StarData[PIndex]))

M0_bin_ids = bin_locations (dustfit_M0_mdot)
M1_bin_ids = bin_locations (dustfit_M1_mdot)
C0_bin_ids = bin_locations (dustfit_C0_mdot)
C1_bin_ids = bin_locations (dustfit_C1_mdot)
```

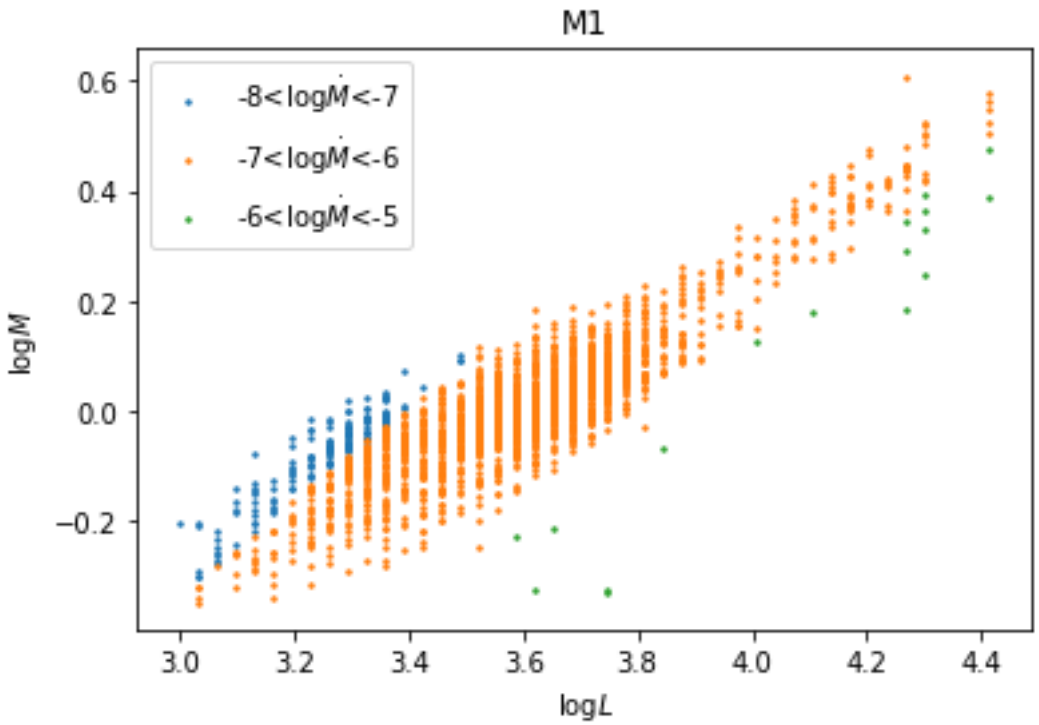
```
Sorting Failed for index 0
```

[365]:
```
plot_binned_stellar_data(M0StarData,M0_bin_ids,'M0')
plot_binned_stellar_data(M1StarData,M1_bin_ids,'M1')
plot_binned_stellar_data(C0StarData,C0_bin_ids,'C0')
plot_binned_stellar_data(C1StarData,C1_bin_ids,'C1')
```

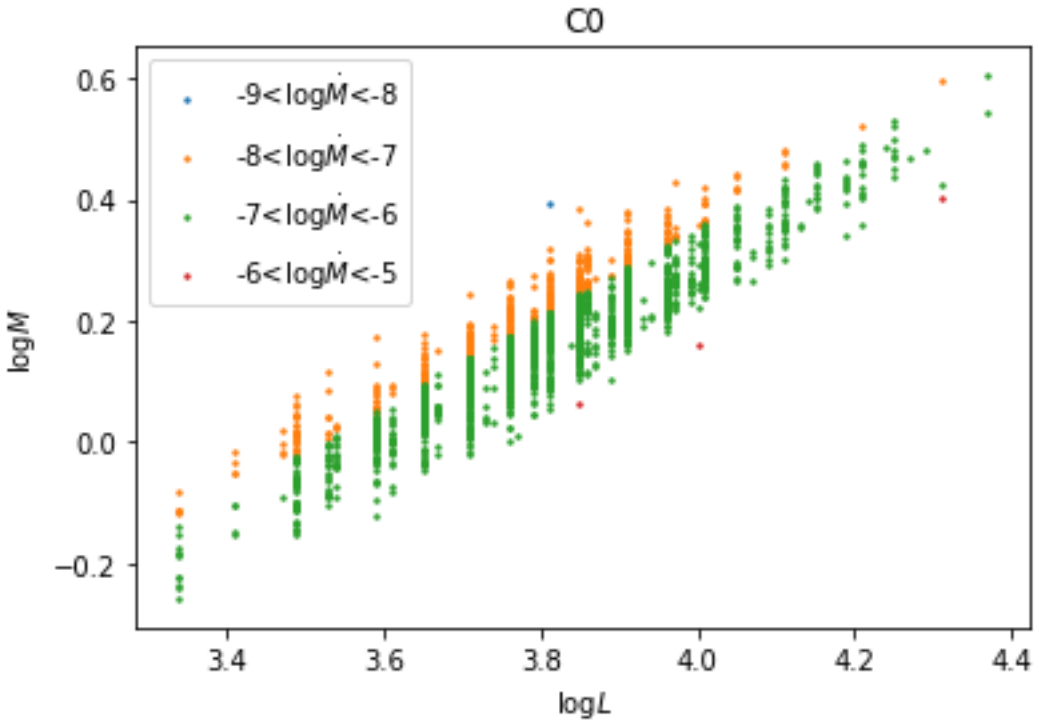
```
Bin list  0  is empty. Continuing to next bin.
Bin list  1  is empty. Continuing to next bin.
Bin list  2  is empty. Continuing to next bin.
Min observed value in bin: -6.456478269324708
Max observed value in bin: -6.456478269324708
Min observed value in bin: -7.8366895110363135
Max observed value in bin: -5.137719506700295
Bin list  5  is empty. Continuing to next bin.
Bin list  6  is empty. Continuing to next bin.
```



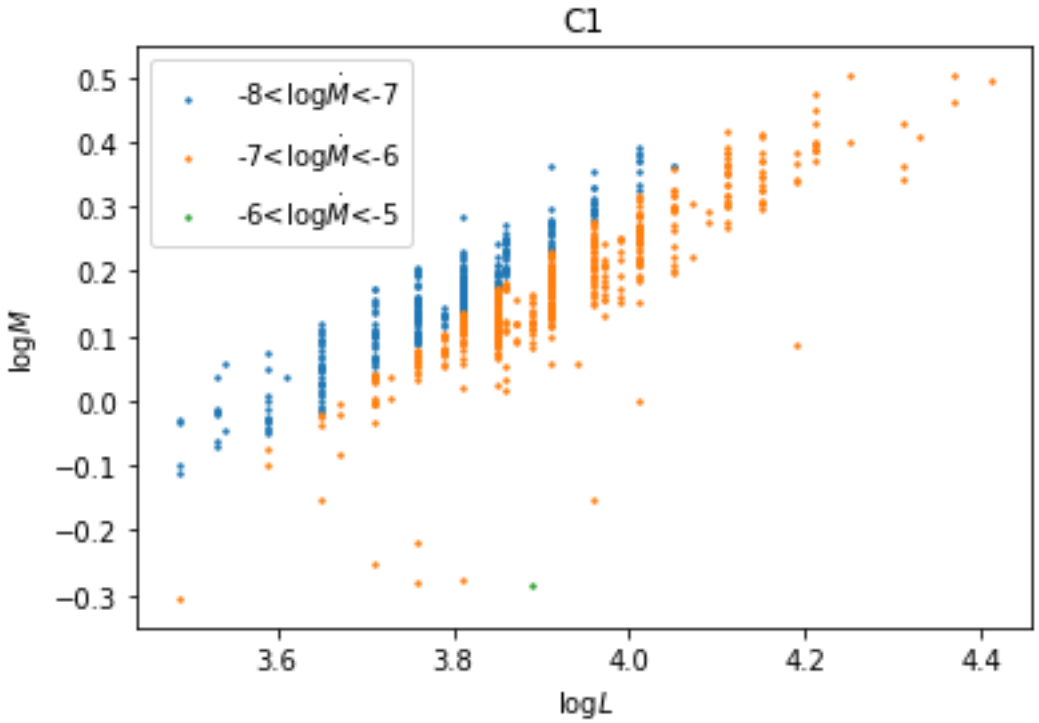

```
Bin list  0  is empty. Continuing to next bin.
Bin list  1  is empty. Continuing to next bin.
Bin list  2  is empty. Continuing to next bin.
Min observed value in bin: -7.967457791305338
Max observed value in bin: -5.743267825874079
Min observed value in bin: -7.863561657436615
Max observed value in bin: -4.955875918755522
Min observed value in bin: -6.7648075037301885
Max observed value in bin: -5.088501484030114
Bin list  6  is empty. Continuing to next bin.
```




```
Bin list  0  is empty. Continuing to next bin.
Bin list  1  is empty. Continuing to next bin.
Min observed value in bin: -7.00939478857608
Max observed value in bin: -7.00939478857608
Min observed value in bin: -7.942447998945467
Max observed value in bin: -6.0360469833475205
Min observed value in bin: -7.703225153259688
Max observed value in bin: -4.933674074637962
Min observed value in bin: -5.611454779904043
Max observed value in bin: -5.4565528199183
Bin list  6  is empty. Continuing to next bin.
```




```
Bin list  0  is empty. Continuing to next bin.
Bin list  1  is empty. Continuing to next bin.
Bin list  2  is empty. Continuing to next bin.
Min observed value in bin: -8.009394788576081
Max observed value in bin: -6.0042551489236695
Min observed value in bin: -7.729554091982037
Max observed value in bin: -4.6549204736851335
Min observed value in bin: -6.678401569534656
Max observed value in bin: -6.678401569534656
Bin list  6  is empty. Continuing to next bin.
```



[366]:
```
M0BCResid = np.asarray([[1.8458,-2.6279,-0.7936],[1,-1,-0.8603],[4.3900,-3.
↪351,0.6328],[0.9499,-1.4621,-0.6475],[1.6485,-0.7907,-0.8821],[2.5094,-2.
↪066,-0.7845],[1.7096,-1.5781,-0.8548]])
M1BCResid = np.asarray([[2.1658, -2.5042, -0.6571],[1, -1, -0.6837],[4.3900, -3.
↪351, -0.1855],[1.3008, -1.7645, -0.4868],[1.6485, -0.7907, -0.6616],[2.5405,␣
↪-2.0957, -0.5397],[0.7585, -0.5802, -0.8584]])
C0BCResid = np.asarray([[2.2134, -2.6219, -0.7292],[1, -1, -0.7599],[4.4370, -3.
↪4710, -0.3854],[0.8268, -1.7674, -0.5466],[1.7965, -1.1687, -0.7415],[2.
↪6618, -2.3039, -0.6396],[3.8115, -3.9779, -0.4846],[13.6825, -14.5687, 0.
↪8139]])
C1BCResid = np.asarray([[2.2099, -2.3380, -0.6179],[1, -1, -0.6969],[4.437, -3.
↪471, -0.2256],[1.2799, 2.1018, -0.4058],[1.7965, -1.1687, -0.6851],[2.7044,␣
↪-2.3597, -0.5297],[1.7959, -1.8315, -0.6949]])
```

def randrange(n, vmin, vmax): """ Helper function to make an array of random numbers having shape (n,) with each number distributed Uniform(vmin, vmax). """ return (vmax - vmin)*np.random.rand(n) + vmin for az_angle in range(0,360): fig = plt.figure() ax = fig.add_subplot(projection='3d') to_plot_list = [M0BCResid,M1BCResid,C0BCResid,C1BCResid] marker_list = ['o', 'v', '^', 's'] label_list = ['M0','M1','C0','C1'] for i in range (0,4): xs = np.transpose(to_plot_list[i])[0] ys = np.transpose(to_plot_list[i])[1] zs = np.transpose(to_plot_list[i])[2] ax.scatter(xs, ys, zs, marker=marker_list[i]) plot.tight_layout() ax.view_init(10, az_angle)

```
ax.set_xlabel('B')
```

```
ax.set_ylabel('C')
ax.set_zlabel('Residual Slope')

#plt.show()
plt.savefig('BCResid_gif_elem/'+'BCResid_gif_elem_'+str(az_angle)+'.jpg')
plt.close()
```

for i in range (0,4): xs = np.transpose(to_plot_list[i])[0] ys = np.transpose(to_plot_list[i])[1] zs = np.transpose(to_plot_list[i])[2] plt.scatter(xs, ys, c=zs, marker=marker_list[i],label=label_list[i],cmap='gray') plt.legend() plt.xlabel('B') plt.ylabel('C') plt.show() plt.close()

for i in range (0,4): xs = np.transpose(to_plot_list[i])[0] ys = np.transpose(to_plot_list[i])[1] xs = xs+ys zs = np.transpose(to_plot_list[i])[2] plt.scatter(xs, zs, marker=marker_list[i],label=label_list[i],cmap='gray') plt.legend() plt.xlabel('B+C') plt.ylabel('Residual Slope') plt.show() plt.close()

for i in range (0,4): xs = np.transpose(to_plot_list[i])[0] ys = np.transpose(to_plot_list[i])[1] xs = xs-ys zs = np.transpose(to_plot_list[i])[2] plt.scatter(xs, zs, marker=marker_list[i],label=label_list[i],cmap='gray') plt.legend() plt.xlabel('B-C') plt.ylabel('Residual Slope') plt.show() plt.close()

for i in range (0,4): xs = np.transpose(to_plot_list[i])[0] ys = np.transpose(to_plot_list[i])[1] xs = xs*ys zs = np.transpose(to_plot_list[i])[2] plt.scatter(xs, zs, marker=marker_list[i],label=label_list[i],cmap='gray') plt.legend() plt.xlabel('BC') plt.ylabel('Residual Slope') plt.show() plt.close()

for i in range (0,4): xs = np.transpose(to_plot_list[i])[0] ys = np.transpose(to_plot_list[i])[1] xs = xs/ys zs = np.transpose(to_plot_list[i])[2] plt.scatter(xs, zs, marker=marker_list[i],label=label_list[i],cmap='gray') plt.legend() plt.xlabel('B/C') plt.ylabel('Residual Slope') plt.show() plt.close()

[387]:
```
# Attenuation Test - What do we get when we fit log L(log Mdot, log P) as␣
↪opposed to log Mdot(log L, log P)?
def attenuation_test (star_data,spec_mode_label='',text_x=0.03):
    color_list = ['tab:blue','tab:orange','tab:green','tab:red','tab:
↪purple','tab:brown', 'tab:pink', 'tab:gray', 'tab:olive', 'tab:cyan']
    logMData = np.log10(star_data[MIndex])
    logLData = np.log10(star_data[LIndex])
    logPData = np.log10(star_data[PIndex])
    logMdotData = np.log10(star_data[MdotIndex])
    logMdotperMData = np.log10(star_data[MdotIndex]/star_data[MIndex])
    fit_primary = multilinear_fit_2var([logLData,logPData],logMdotData)
    # Returns logMdot = A + BlogL + ClogP
    fit_secondary = multilinear_fit_2var([logMdotData,logPData],logLData)
    # Returns logL = A + BlogMdot + ClogP

    LPCoeffs = fit_primary[0]
    MdotPCoeffs = fit_secondary[0]
    our_log_mdot = LPCoeffs[0] + LPCoeffs[1]*logLData + LPCoeffs[2]*logPData
```

```
    attenuation_log_mdot = (logLData - MdotPCoeffs[0] -␣
↪MdotPCoeffs[2]*logPData)/MdotPCoeffs[1]
    # log Mdot = A + BlogL + ClogP
    if spec_mode_label[0] == 'M':
        RLM = RLM_M
    if spec_mode_label[0] == 'C':
        RLM = RLM_C
    if spec_mode_label == 'M0':
        PMR = PMR_M0
        dz_values = deathzone_values_M0
    if spec_mode_label == 'M1':
        PMR = PMR_M1
        dz_values = deathzone_values_M1
    if spec_mode_label == 'C0':
        PMR = PMR_C0
        dz_values = deathzone_values_C0
    if spec_mode_label == 'C0x':
        PMR = PMR_C0
        dz_values = deathzone_values_C0x
    if spec_mode_label == 'C0 and C0x':
        PMR = PMR_C0
        dz_values = deathzone_values_C0_and_C0x
    if spec_mode_label == 'C1':
        PMR = PMR_C1
        dz_values = deathzone_values_C1
    if spec_mode_label == 'C0 and C0x':
        PMR = PMR_C0
        dz_values = deathzone_values_C0_and_C0x

    #artificial_data = make_artificial_set([star_data],0.2,grid_spacing = 75j)
    #artificial_logLData = artificial_data[0]
    #artificial_logMData = artificial_data[1]
    #artificial_logRData = RLM[0] + RLM[1]*artificial_logLData +␣
↪RLM[2]*artificial_logMData
    #artificial_logPData = PMR[0] + PMR[1]*artificial_logMData +␣
↪PMR[2]*artificial_logRData
    #artificial_logTData = np.log10(5778*np.sqrt(10**(artificial_logRData)/np.
↪sqrt(10**artificial_logLData)))

    a = LPCoeffs[0]
    arlm = RLM[0]
    aRLM = arlm
    apmr = PMR[0]
    aPMR = apmr
    c = LPCoeffs[2]
    cpmr = PMR[2]
    cPMR = cpmr
```

```
    b = LPCoeffs[1]
    brlm = RLM[1]
    bRLM = brlm
    bpmr = PMR[1]
    bPMR = bpmr
    crlm = RLM[2]
    cRLM = crlm
    PMLCoeffs = calculate_PML_coeffs(RLM, PMR)
    aPML = PMLCoeffs[0]
    bPML = PMLCoeffs[1]
    cPML = PMLCoeffs[2]


    # Coefficients for the Mdot(L,P) fit as a function Mdot(L, P)
    aMdotLP = a
    bMdotLP   = b
    cMdotLP  = c

    Anew = a - arlm + apmr*c + arlm*c*cpmr
    Bnew = b - brlm + brlm*c*cpmr
    Cnew = bpmr*c - crlm + c*cpmr*crlm
    print('log Mdot(logL, logP):')
    #print('LRM form: logMdot =',Anew,' + ',Bnew,'logL + logR -',-Cnew,'logM')
    print('LM form: logMdot =',Anew+arlm,' + ',Bnew+brlm,'logL␣
↪-',-(Cnew+crlm),'logM')
    #####

    # Type 1: Calc Mdot vs Obsv Mdot
    fig, ax = plt.subplots()
    plot.scatter(logMdotData,our_log_mdot,s=2,color=color_list[0])
    title=spec_mode_label+r' ($\log \dot{M}(\log L, \log P)$)'
    ax.text(text_x, .
↪97,title,horizontalalignment='center',verticalalignment='center',transform =␣
↪ax.transAxes,fontsize=10)
    plot.xlabel(r'$\log \dot{M}_{obsv.}$')
    plot.ylabel(r'$\log \dot{M}_{calc.}$')
    plot.plot([-9,-3],[-9,-3],color = 'black', alpha =␣
↪1,linestyle='--',linewidth=1.5)
    plot.xlim([-8,-4])
    plot.ylim([-8,-4])
    plot.tight_layout()
    plot.savefig('Final_Fit/Attenuation_Demonstration/
↪'+spec_mode_label+'_logMdot(logL,logP)_calc_vs_obsv'+'.pdf',format='pdf')
    plot.savefig('Final_Fit/Attenuation_Demonstration/
↪'+spec_mode_label+'_logMdot(logL,logP)_calc_vs_obsv'+'.png',format='png')
    plot.show()
    plot.close()
```

```
    # Type 2: Residual vs Obsv Mdot
    fig, ax = plt.subplots()
    plot.scatter(logMdotData,our_log_mdot-logMdotData,s=2,color=color_list[0])
    title=spec_mode_label+r' ($\log \dot{M}(\log L, \log P)$)'
    ax.text(text_x, .
↪97,title,horizontalalignment='center',verticalalignment='center',transform =␣
↪ax.transAxes,fontsize=10)
    plot.xlabel(r'$\log \dot{M}_{obsv.}$')
    plot.ylabel(r'$\log \dot{M}_{calc.}-\log \dot{M}_{obsv.}$')
    plot.plot([-8,-4],[0,0],color = 'black', alpha =␣
↪1,linestyle='--',linewidth=1.5)
    plot.xlim([-8,-4])
    plot.tight_layout()
    plot.savefig('Final_Fit/Attenuation_Demonstration/
↪'+spec_mode_label+'_logL(logMdot,logP)_resid_vs_obsv'+'.pdf',format='pdf')
    plot.savefig('Final_Fit/Attenuation_Demonstration/
↪'+spec_mode_label+'_logL(logMdot,logP)_resid_vs_obsv'+'.png',format='png')
    plot.show()
    plot.close()


    # Returns logL = A + BlogMdot + ClogP
    a_LMdotP = MdotPCoeffs[0]
    b_LMdotP = MdotPCoeffs[1]
    c_LMdotP = MdotPCoeffs[2]
    a = -a_LMdotP/b_LMdotP
    b = 1/b_LMdotP
    c = -c_LMdotP/b_LMdotP
    arlm = RLM[0]
    aRLM = arlm
    apmr = PMR[0]
    aPMR = apmr
    cpmr = PMR[2]
    cPMR = cpmr
    brlm = RLM[1]
    bRLM = brlm
    bpmr = PMR[1]
    bPMR = bpmr
    crlm = RLM[2]
    cRLM = crlm
    PMLCoeffs = calculate_PML_coeffs(RLM, PMR)
    aPML = PMLCoeffs[0]
    bPML = PMLCoeffs[1]
    cPML = PMLCoeffs[2]

    # Coefficients for the L(Mdot,P) fit as a function Mdot(L, P)
    aLMdotP = a
```

```
    bLMdotP  = b
    cLMdotP  = c


    Anew = a - arlm + apmr*c + arlm*c*cpmr
    Bnew = b - brlm + brlm*c*cpmr
    Cnew = bpmr*c - crlm + c*cpmr*crlm
    print('log L(logMdot, logP):')
    print('LM form: logMdot =',Anew+arlm,' + ',Bnew+brlm,'logL␣
↪-',-(Cnew+crlm),'logM' )

    # Type 1: Calc Mdot vs Obsv Mdot
    fig, ax = plt.subplots()
    plot.scatter(logMdotData,attenuation_log_mdot,s=2,color=color_list[1])
    title=spec_mode_label+r' ($\log L(\log \dot{M}, \log P)$)'
    ax.text(text_x, .
↪97,title,horizontalalignment='center',verticalalignment='center',transform =␣
↪ax.transAxes,fontsize=10)
    plot.xlabel(r'$\log \dot{M}_{obsv.}$')
    plot.ylabel(r'$\log \dot{M}_{calc.}$')
    plot.plot([-9,-3],[-9,-3],color = 'black', alpha =␣
↪1,linestyle='--',linewidth=1.5)
    plot.xlim([-8,-4])
    plot.ylim([-8,-4])
    plot.tight_layout()
    plot.savefig('Final_Fit/Attenuation_Demonstration/
↪'+spec_mode_label+'_logL(logMdot,logP)_calc_vs_obsv'+'.pdf',format='pdf')
    plot.savefig('Final_Fit/Attenuation_Demonstration/
↪'+spec_mode_label+'_logL(logMdot,logP)_calc_vs_obsv'+'.png',format='png')
    plot.show()
    plot.close()

    # Type 2: Residual vs Obsv Mdot
    plot.
↪scatter(logMdotData,attenuation_log_mdot-logMdotData,s=2,color=color_list[1])
    title=spec_mode_label+r' ($\log L(\log \dot{M}, \log P)$)'
    ax.text(text_x, .
↪97,title,horizontalalignment='center',verticalalignment='center',transform =␣
↪ax.transAxes,fontsize=10)
    plot.xlabel(r'$\log \dot{M}_{obsv.}$')
    plot.ylabel(r'$\log \dot{M}_{calc.}-\log \dot{M}_{obsv.}$')
    plot.plot([-8,-4],[0,0],color = 'black', alpha =␣
↪1,linestyle='--',linewidth=1.5)
    plot.xlim([-8,-4])
    plot.tight_layout()
```

```
   plot.savefig('Final_Fit/Attenuation_Demonstration/
↪'+spec_mode_label+'_logL(logMdot,logP)_resid_vs_obsv'+'.pdf',format='pdf')
   plot.savefig('Final_Fit/Attenuation_Demonstration/
↪'+spec_mode_label+'_logL(logMdot,logP)_resid_vs_obsv'+'.png',format='png')
   plot.show()
   plot.close()


   dz_log_mdot = dz_values[0] + dz_values[1]*np.log10(star_data[LIndex]) +␣
↪dz_values[2]*np.log10(star_data[MIndex])

   print('log Mdot(logL, logM):')
   print('LM form: logMdot =',dz_values[0],' + ',dz_values[1],'logL␣
↪-',-(dz_values[2]),'logM' )
   # Type 1: Calc Mdot vs Obsv Mdot
   plot.scatter(logMdotData,dz_log_mdot,s=2,color=color_list[2])
   title=spec_mode_label+r' ($\log \dot{M}(\log L, \log M)$)'
   ax.text(text_x, .
↪97,title,horizontalalignment='center',verticalalignment='center',transform =␣
↪ax.transAxes,fontsize=10)
   plot.xlabel(r'$\log \dot{M}_{obsv.}$')
   plot.ylabel(r'$\log \dot{M}_{calc.}$')
   plot.plot([-9,-3],[-9,-3],color = 'black', alpha =␣
↪1,linestyle='--',linewidth=1.5)
   plot.xlim([-8,-4])
   plot.ylim([-8,-4])
   plot.savefig('Final_Fit/Attenuation_Demonstration/
↪'+spec_mode_label+'_logMdot(logL,logM)_calc_vs_obsv'+'.pdf',format='pdf')
   plot.savefig('Final_Fit/Attenuation_Demonstration/
↪'+spec_mode_label+'_logMdot(logL,logM)_calc_vs_obsv'+'.png',format='png')
   plot.tight_layout()
   plot.show()
   plot.close()

   # Type 2: Residual vs Obsv Mdot
   plot.scatter(logMdotData,dz_log_mdot-logMdotData,s=2,color=color_list[2])
   title=spec_mode_label+r' ($\log \dot{M}(\log L, \log M)$)'
   ax.text(text_x, .
↪97,title,horizontalalignment='center',verticalalignment='center',transform =␣
↪ax.transAxes,fontsize=10)
   plot.xlabel(r'$\log \dot{M}_{obsv.}$')
   plot.ylabel(r'$\log \dot{M}_{calc.}-\log \dot{M}_{obsv.}$')
   plot.plot([-8,-4],[0,0],color = 'black', alpha =␣
↪1,linestyle='--',linewidth=1.5)
   plot.xlim([-8,-4])
   plot.tight_layout()
```

```
   plot.savefig('Final_Fit/Attenuation_Demonstration/
↪'+spec_mode_label+'_logMdot(logL,logM)_resid_vs_obsv'+'.pdf',format='pdf')
   plot.savefig('Final_Fit/Attenuation_Demonstration/
↪'+spec_mode_label+'_logMdot(logL,logM)_resid_vs_obsv'+'.png',format='png')
   plot.show()
   plot.close()



   # Now an overlaid version
   # Type 1: Calc Mdot vs Obsv Mdot
   fig, ax = plt.subplots()
   alpha_set=0.33
   plot.scatter(logMdotData,our_log_mdot,s=2,label=r'$\log \dot{M}(\log L,␣
↪\log P)$',alpha=alpha_set,color=color_list[0])
   fit = linear_fit_1var(logMdotData,our_log_mdot)
   fit_params = fit[0]
   fit_x = np.asarray([-9,-3])
   fit_y = fit_params[0]+fit_params[1]*fit_x
   plot.plot(fit_x,fit_y,color = color_list[0])
   print('Mdot(L,P) Calc. vs. Obsv. Slope=',fit_params[1])
   print('Mdot(L,P) Calc. vs. Obsv. R^2=',fit[1])
   print('Mdot(L,P) Std. Dev. of Residual: sigma=',np.
↪nanstd(our_log_mdot-logMdotData))
   print('Mdot(L,P) Mean Distance from 1-to-1 Line:',np.nanmean(np.
↪abs(our_log_mdot-logMdotData)))

   plot.scatter(logMdotData,attenuation_log_mdot,s=2,label=r'$\log L(\log␣
↪\dot{M}, \log P)$',alpha=alpha_set,color=color_list[1])
   fit = linear_fit_1var(logMdotData,attenuation_log_mdot)
   fit_params = fit[0]
   fit_x = np.asarray([-9,-3])
   fit_y = fit_params[0]+fit_params[1]*fit_x
   plot.plot(fit_x,fit_y,color = color_list[1])
   print('L(Mdot,P) Calc. vs. Obsv. Slope=',fit_params[1])
   print('L(Mdot,P) Calc. vs. Obsv. R^2=',fit[1])
   print('L(Mdot,P) Std. Dev. of Residual: sigma=',np.
↪nanstd(attenuation_log_mdot-logMdotData))
   print('L(Mdot,P) Mean Distance from 1-to-1 Line:',np.nanmean(np.
↪abs(attenuation_log_mdot-logMdotData)))

   plot.scatter(logMdotData,dz_log_mdot,s=2,label='LM Strip␣
↪Method',alpha=alpha_set,color=color_list[2])
   fit = linear_fit_1var(logMdotData,dz_log_mdot)
   fit_params = fit[0]
   fit_x = np.asarray([-9,-3])
```

```
    fit_y = fit_params[0]+fit_params[1]*fit_x
    plot.plot(fit_x,fit_y,color = color_list[2])
    print('LM Strip Calc. vs. Obsv. Slope=',fit_params[1])
    print('LM Strip Calc. vs. Obsv. R^2=',fit[1])
    print('LM Strip Std. Dev. of Residual: sigma=',np.
↪nanstd(dz_log_mdot-logMdotData))
    print('LM Strip Mean Distance from 1-to-1 Line:',np.nanmean(np.
↪abs(dz_log_mdot-logMdotData)))

    #plot.title(spec_mode_label)
    ax.text(text_x, .
↪97,spec_mode_label,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
    plot.xlabel(r'$\log \dot{M}_{obsv.}$')
    plot.ylabel(r'$\log \dot{M}_{calc.}$')
    plot.plot([-9,-3],[-9,-3],color = 'black', alpha =␣
↪1,linestyle='--',linewidth=1.5)
    plot.xlim([-8,-4])
    plot.ylim([-8,-4])
    plot.tight_layout()
    plot.legend(loc=4)
    plot.savefig('Final_Fit/Attenuation_Demonstration/
↪'+spec_mode_label+'_all_calc_vs_obsv'+'.pdf',format='pdf')
    plot.savefig('Final_Fit/Attenuation_Demonstration/
↪'+spec_mode_label+'_all_calc_vs_obsv'+'.png',format='png')
    plot.show()
    plot.close()

    # Type 2: Residual vs Obsv Mdot
    fig, ax = plt.subplots()
    plot.scatter(logMdotData,our_log_mdot-logMdotData,s=2,label=r'$\log␣
↪\dot{M}(\log L, \log P)$',alpha=alpha_set,color=color_list[0])
    fit = linear_fit_1var(logMdotData,our_log_mdot-logMdotData)
    fit_params = fit[0]
    fit_x = np.asarray([-9,-3])
    fit_y = fit_params[0]+fit_params[1]*fit_x
    plot.plot(fit_x,fit_y,color = color_list[0])
    print('Mdot(L,P) Slope of Residual=',fit_params[1])
    print('Mdot(L,P) R^2 of Residual=',fit[1])


    plot.scatter(logMdotData,attenuation_log_mdot-logMdotData,s=2,␣
↪label=r'$\log L(\log \dot{M}, \log P)$',alpha=alpha_set,color=color_list[1])
    fit = linear_fit_1var(logMdotData,attenuation_log_mdot-logMdotData)
    fit_params = fit[0]
    fit_x = np.asarray([-9,-3])
```

```
    fit_y = fit_params[0]+fit_params[1]*fit_x
    plot.plot(fit_x,fit_y,color = color_list[1])
    print('L(Mdot,P) Slope of Residual=',fit_params[1])
    print('L(Mdot,P) R^2 of Residual=',fit[1])

    plot.scatter(logMdotData,dz_log_mdot-logMdotData,s=2,label=r'$\log␣
↪\dot{M}(\log L, \log M)$',alpha=alpha_set,color=color_list[2])
    fit = linear_fit_1var(logMdotData,dz_log_mdot-logMdotData)
    fit_params = fit[0]
    fit_x = np.asarray([-9,-3])
    fit_y = fit_params[0]+fit_params[1]*fit_x
    plot.plot(fit_x,fit_y,color = color_list[2])
    print('Mdot(L,M) Slope of Residual=',fit_params[1])
    print('Mdot(L,M) R^2 of Residual=',fit[1])

    #plot.title(spec_mode_label)
    ax.text(text_x, .
↪97,spec_mode_label,horizontalalignment='center',verticalalignment='center',transform␣
↪= ax.transAxes,fontsize=10)
    plot.xlabel(r'$\log \dot{M}_{obsv.}$')
    plot.ylabel(r'$\log \dot{M}_{calc.}-\log \dot{M}_{obsv.}$')
    plot.plot([-8,-4],[0,0],color = 'black', alpha =␣
↪1,linestyle='--',linewidth=1.5)
    plot.xlim([-8,-4])
    plot.tight_layout
    plot.legend(loc=4)
    plot.savefig('Final_Fit/Attenuation_Demonstration/
↪'+spec_mode_label+'_all_resid_vs_obsv'+'.pdf',format='pdf')
    plot.savefig('Final_Fit/Attenuation_Demonstration/
↪'+spec_mode_label+'_all_resid_vs_obsv'+'.png',format='png')
    plot.show()
    plot.close()

    # Test plots: Residual vs log M and vs log P

    resid_MdotLP = our_log_mdot-logMdotData
    resid_LMdotP = attenuation_log_mdot-logMdotData
    resid_MdotLM = dz_log_mdot-logMdotData

    plot.scatter(logPData,resid_MdotLP,s=2)
    plot.show()
    plot.close()
    plot.scatter(logPData,resid_LMdotP,s=2)
    plot.show()
    plot.close()
    plot.scatter(logMData,resid_MdotLM,s=2)
    plot.show()
```

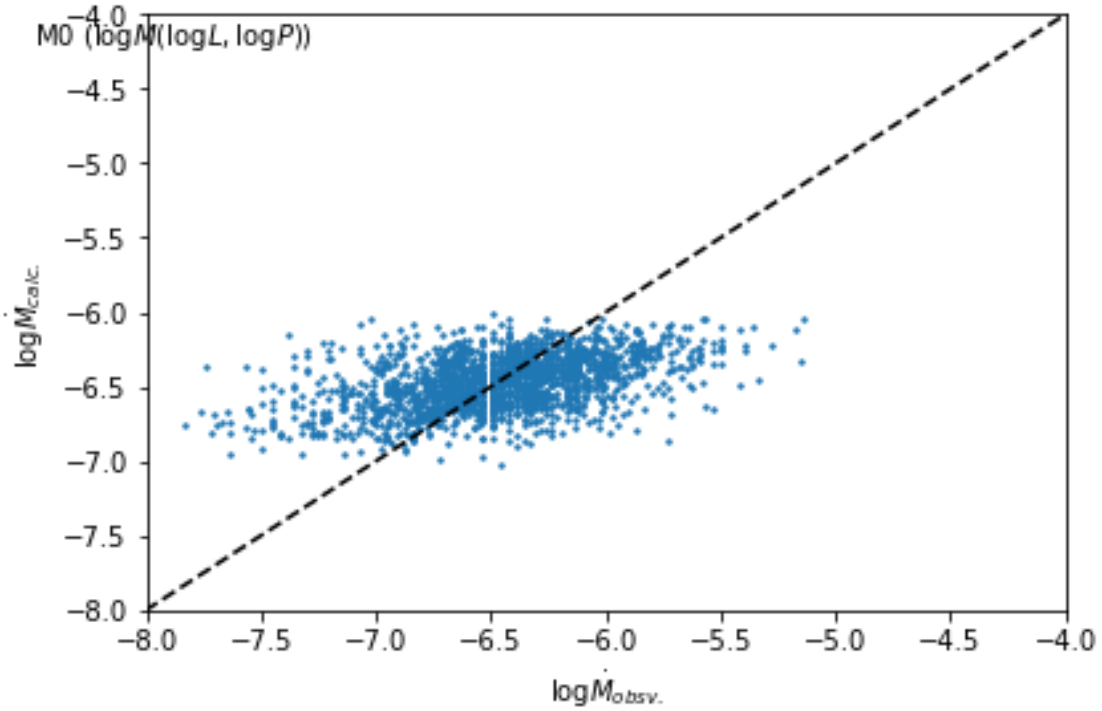
```
    plot.close()

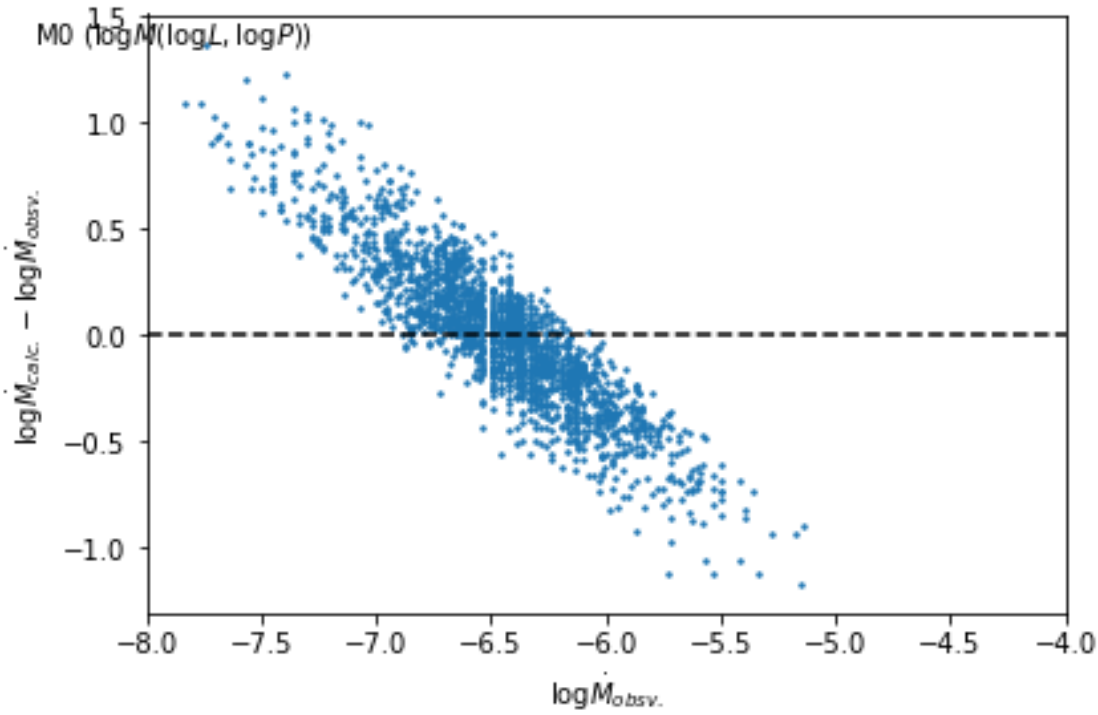
    plot.scatter(logLData,resid_MdotLP,s=2)
    plot.show()
    plot.close()
    plot.scatter(logLData,resid_LMdotP,s=2)
    plot.show()
    plot.close()
    plot.scatter(logLData,resid_MdotLM,s=2)
    plot.show()
    plot.close()

    return [aMdotLP, bMdotLP, cMdotLP], [aLMdotP, bLMdotP, cLMdotP]
```

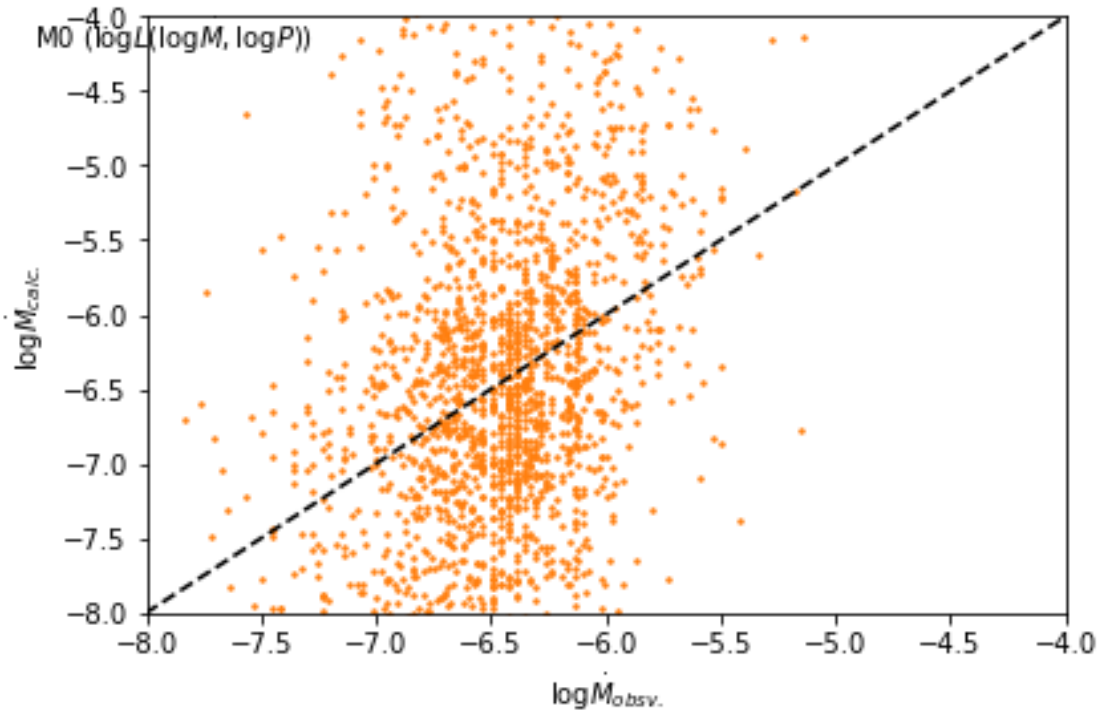
[389]:
```
print('M0:')
MdotLP_Coeffs_M0, LMdotP_Coeffs_M0 = attenuation_test␣
↪(M0StarData,spec_mode_label='M0')
```

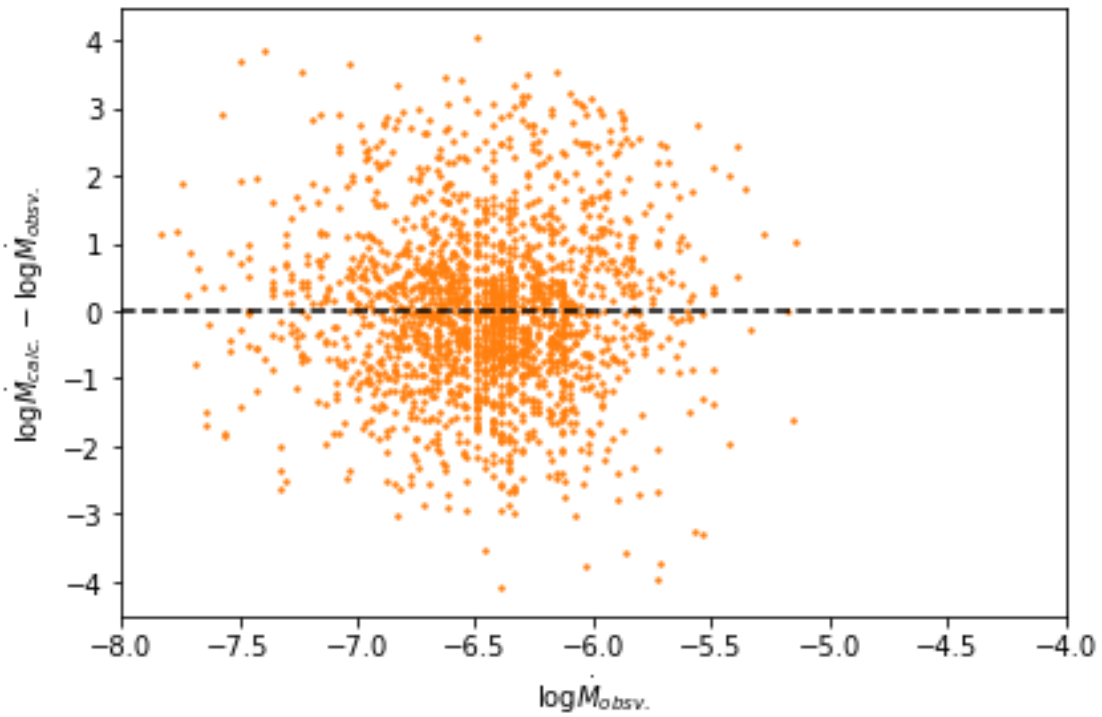
```
M0:
log Mdot(logL, logP):
LM form: logMdot = -12.938730017492958  +  1.845822471134391 logL -
2.627930313100934 logM
```



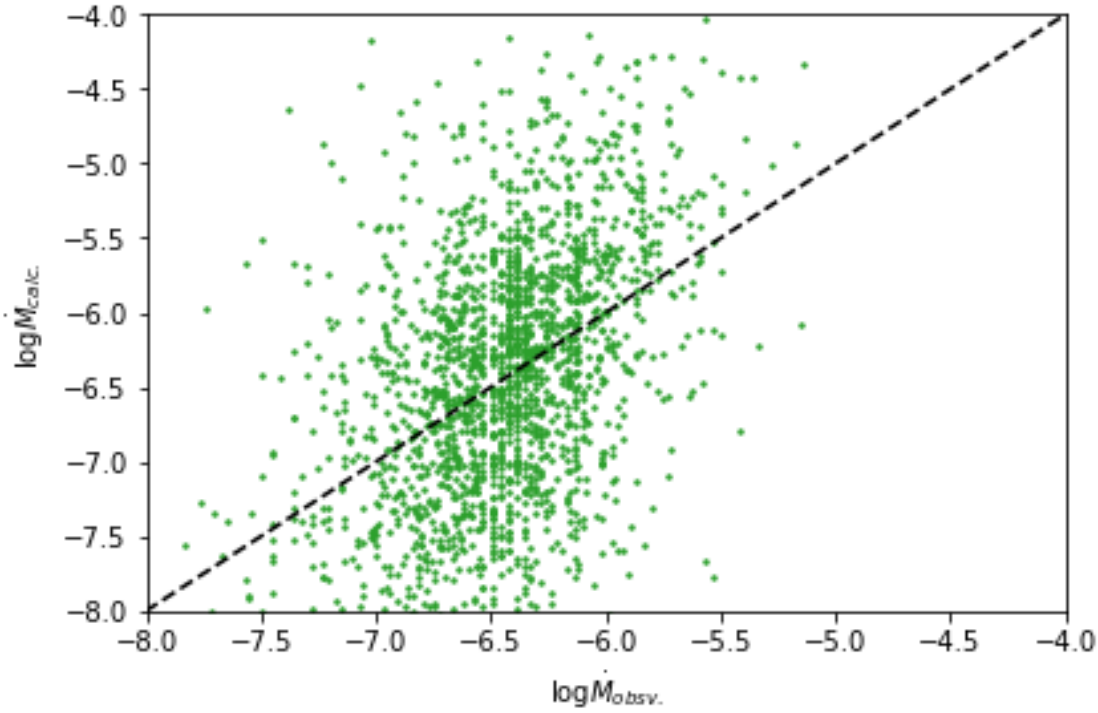


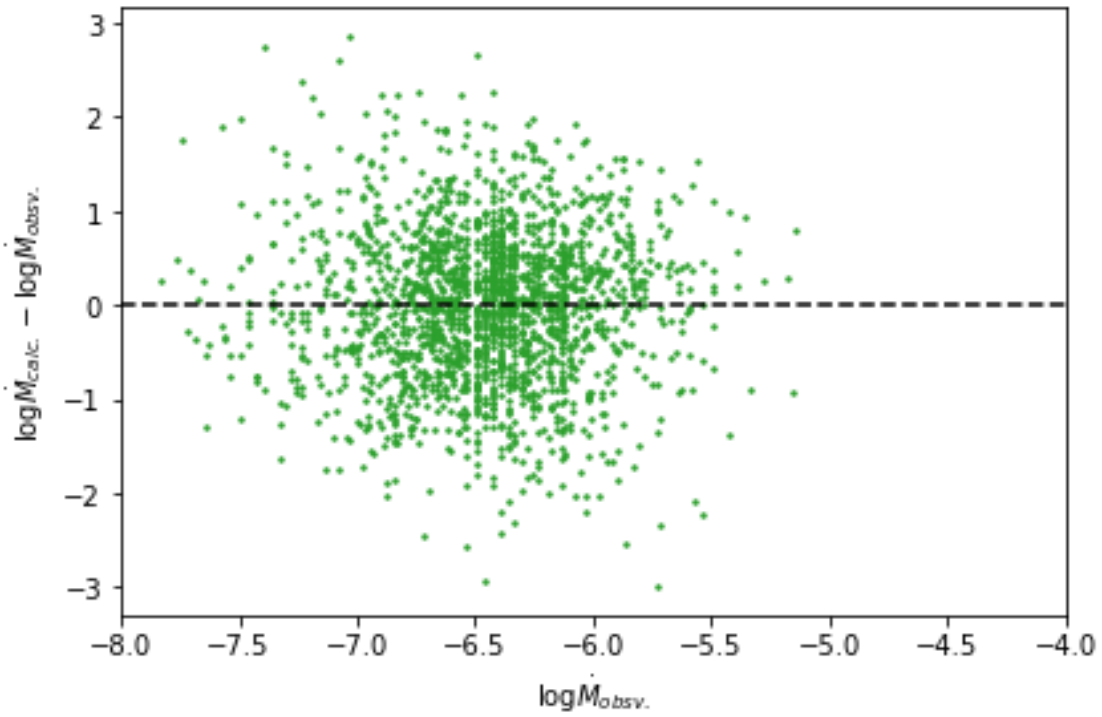

```
log L(logMdot, logP):
LM form: logMdot = -26.434182089563695  +  5.721974364433084 logL -
16.587679509001223 logM
```


M0 ($\log L(\log M, \log P)$)
$\log \dot{M}_{calc.}$
$\log \dot{M}_{obsv.}$
−4.0
−4.5
−5.0
−5.5
−6.0
−6.5
−7.0
−7.5
−8.0

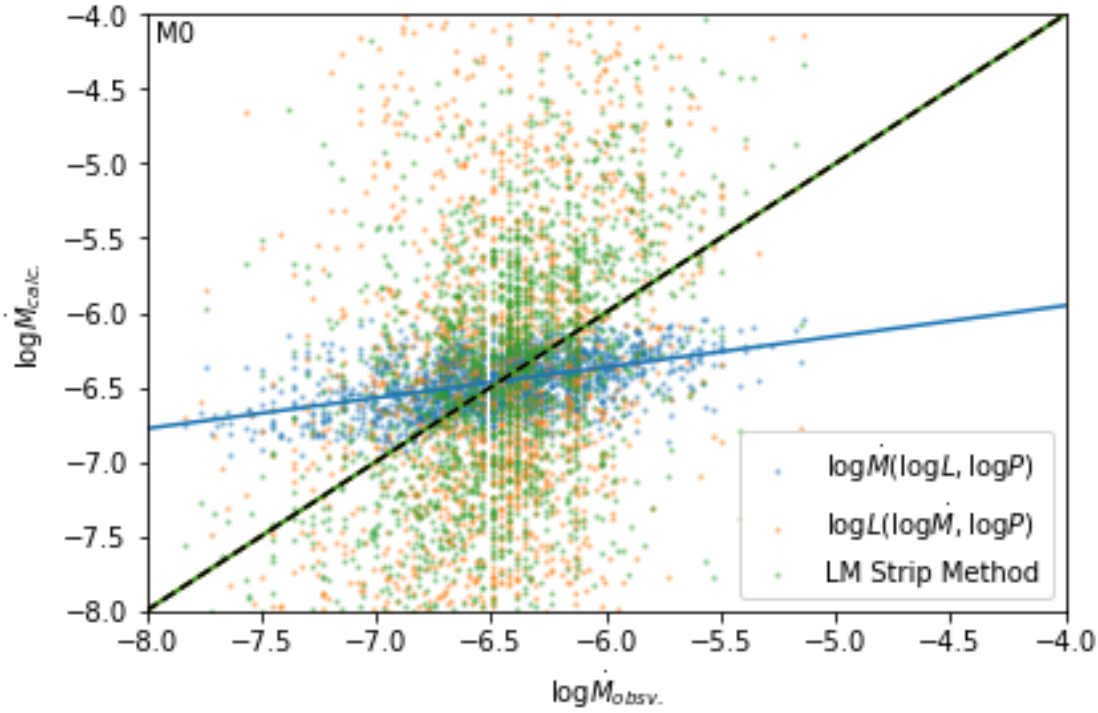

$\log \dot{M}_{calc.} - \log \dot{M}_{obsv.}$
$\log \dot{M}_{obsv.}$
4
3
2
1
0
−1
−2
−3
−4
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0

```
log Mdot(logL, logM):
LM form: logMdot = -34.975963392304195  +  8.131127742677283 logL -
13.698081169196328 logM
```

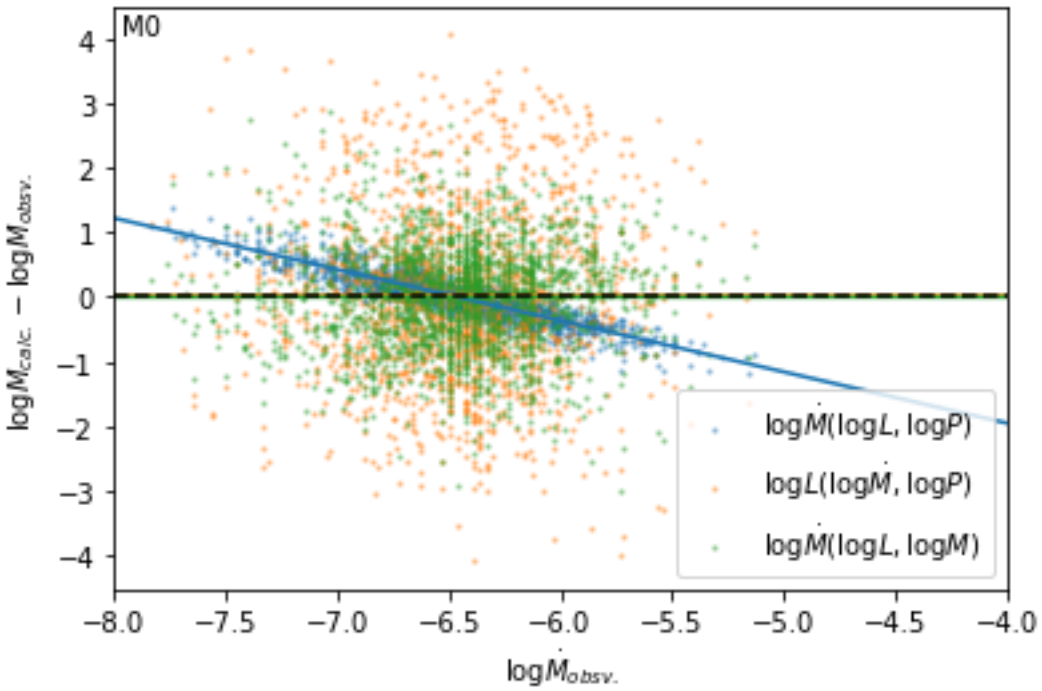


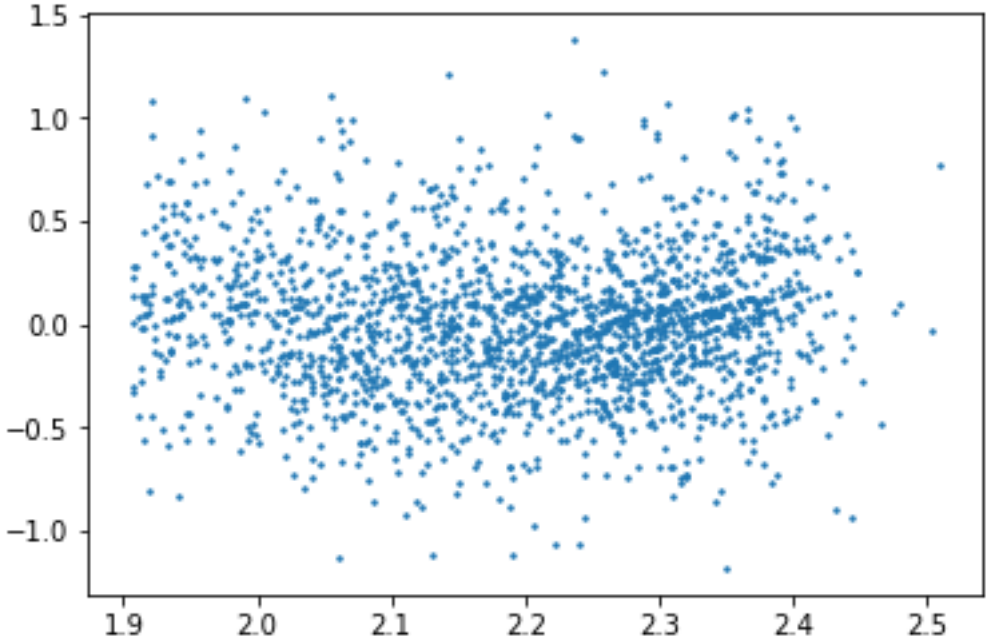


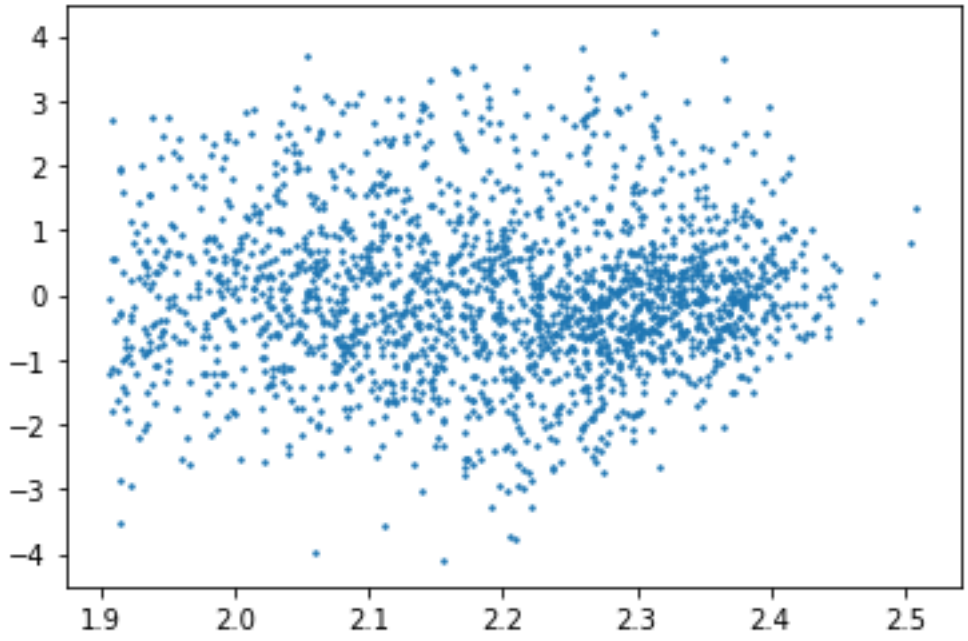

```
Mdot(L,P) Calc. vs. Obsv. Slope= 0.20636545213502516
Mdot(L,P) Calc. vs. Obsv. R^2= 0.20636545213502588
Mdot(L,P) Std. Dev. of Residual: sigma= 0.34839779223147993
Mdot(L,P) Mean Distance from 1-to-1 Line: 0.2671331082033727
L(Mdot,P) Calc. vs. Obsv. Slope= 1.0000000000000078
L(Mdot,P) Calc. vs. Obsv. R^2= 0.09355922406515038
L(Mdot,P) Std. Dev. of Residual: sigma= 1.217282905178631
L(Mdot,P) Mean Distance from 1-to-1 Line: 0.9302424274456704
LM Strip Calc. vs. Obsv. Slope= 1.0000037850481107
LM Strip Calc. vs. Obsv. R^2= 0.19180011520815876
LM Strip Std. Dev. of Residual: sigma= 0.802789559750684
LM Strip Mean Distance from 1-to-1 Line: 0.6296524292131949
```

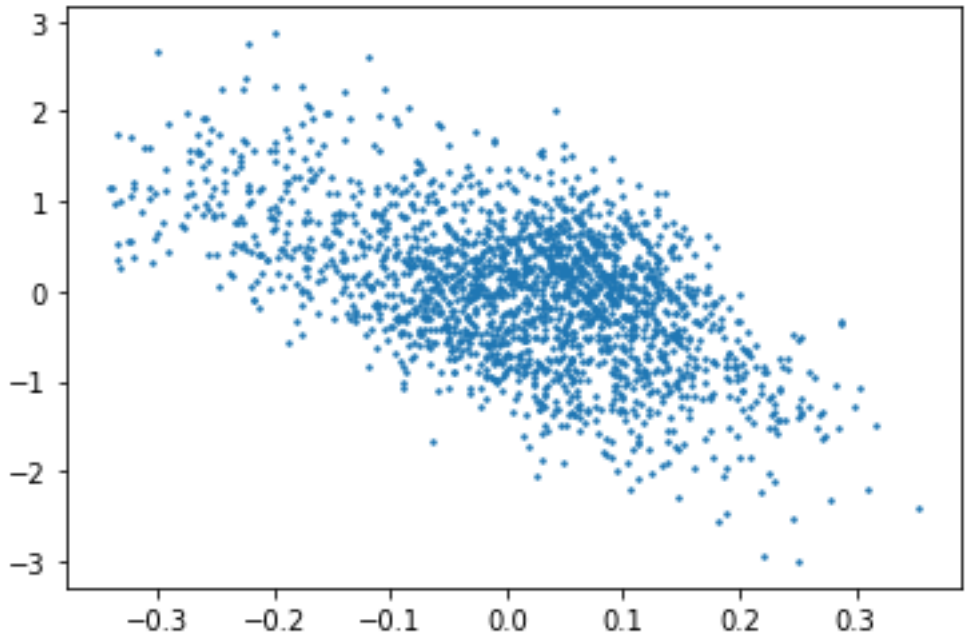


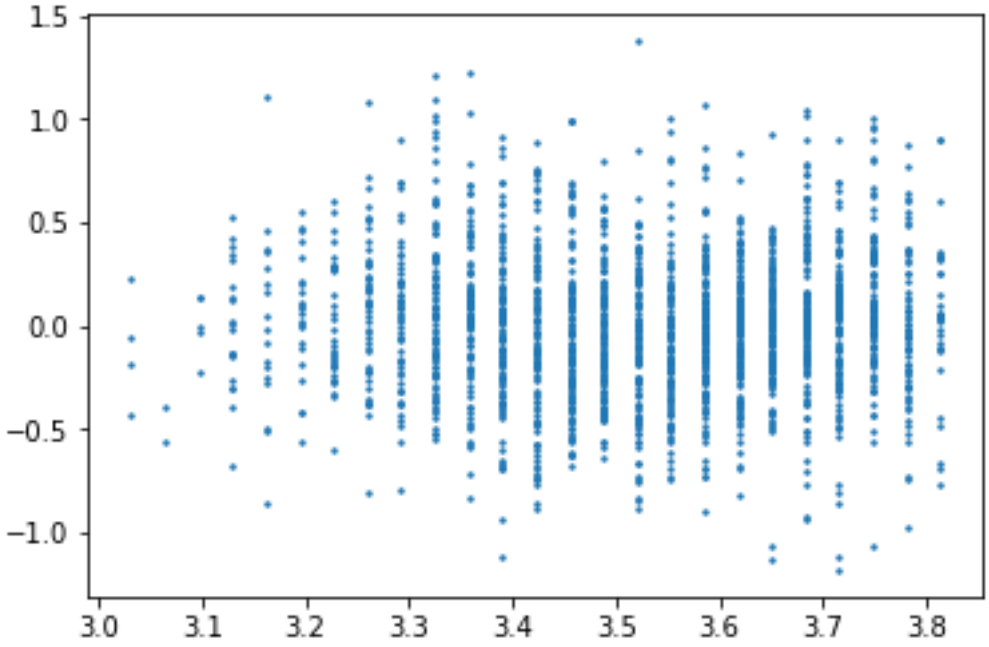

```
Mdot(L,P) Slope of Residual= -0.7936345478649754
Mdot(L,P) R^2 of Residual= 0.7936345478649771
L(Mdot,P) Slope of Residual= 7.02476271596808e-15
L(Mdot,P) R^2 of Residual= -2.220446049250313e-16
Mdot(L,M) Slope of Residual= 3.7850481109201603e-06
Mdot(L,M) R^2 of Residual= 3.400058012914542e-12
```

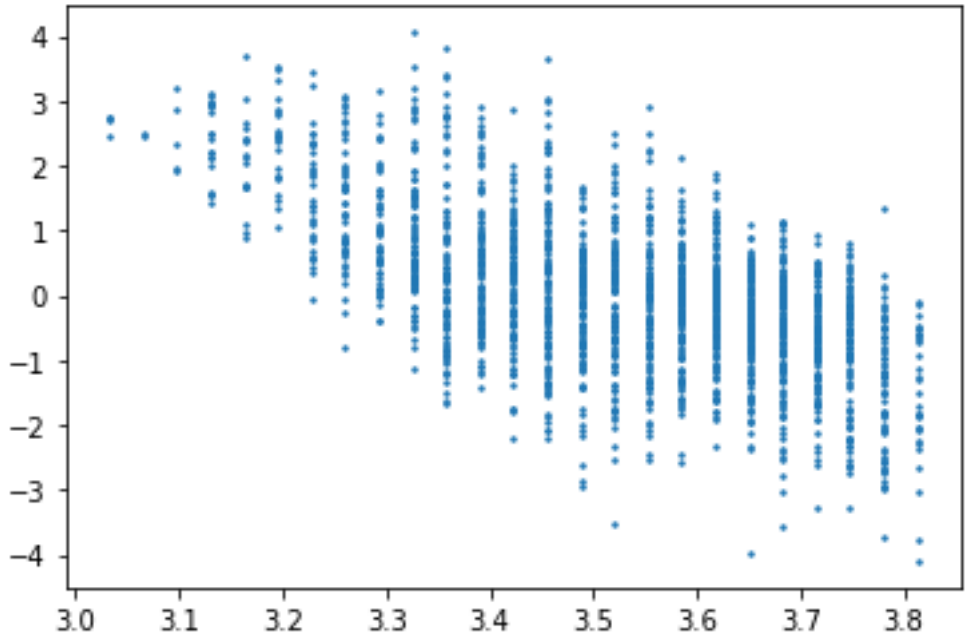

M0
$\log\dot{M}_{calc.} - \log\dot{M}_{obsv.}$
$\log\dot{M}_{obsv.}$
$\log\dot{M}(\log L, \log P)$
$\log L(\log\dot{M}, \log P)$
$\log\dot{M}(\log L, \log M)$

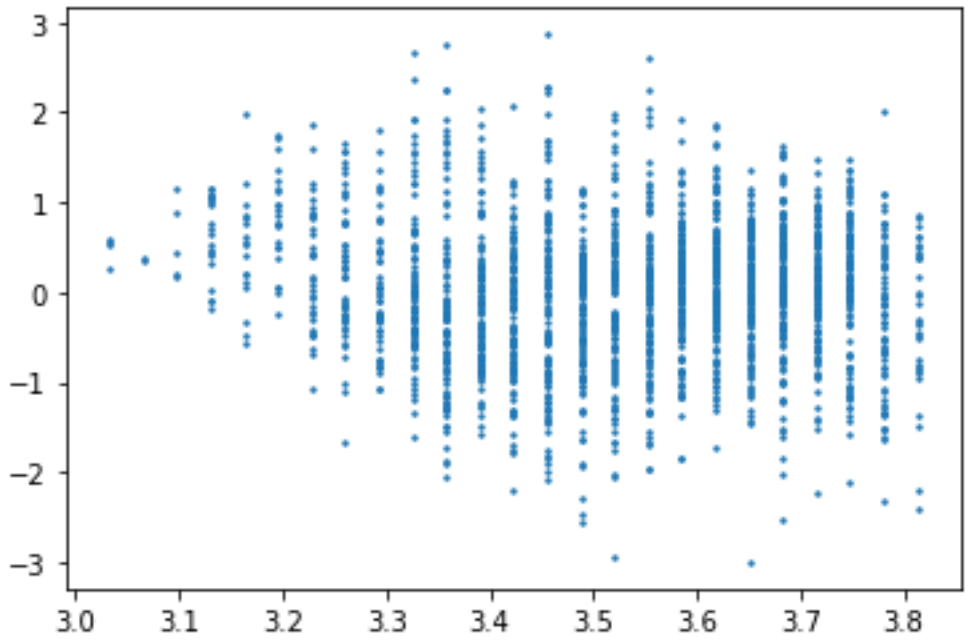



4
3
2
1
0
−1
−2
−3
−4
1.9
2.0
2.1
2.2
2.3
2.4
2.5


3
2
1
0
−1
−2
−3
−0.3
−0.2
−0.1
0.0
0.1
0.2
0.3

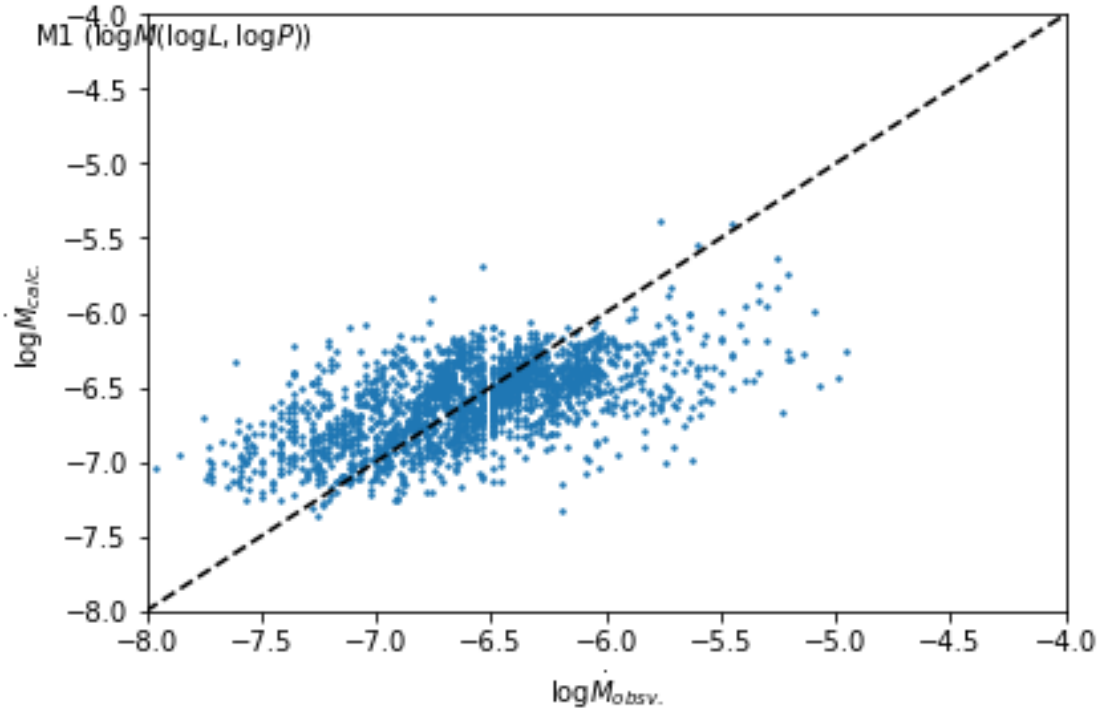

1.5
1.0
0.5
0.0
−0.5
−1.0
3.0
3.1
3.2
3.3
3.4
3.5
3.6
3.7
3.8

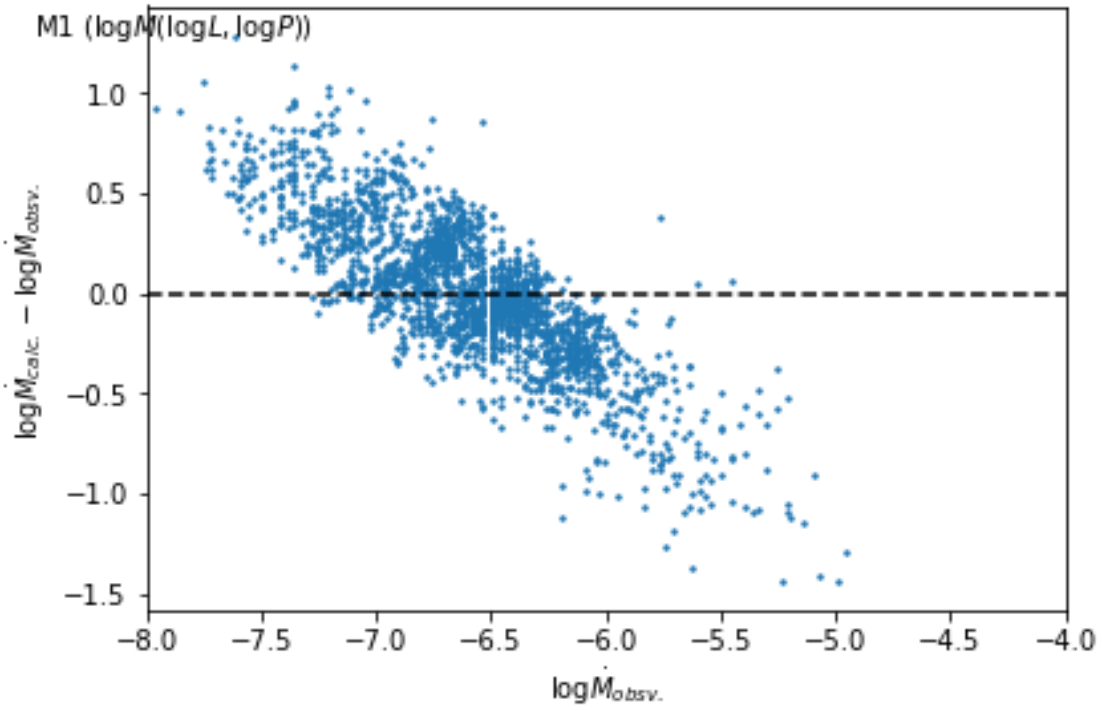

4
3
2
1
0
−1
−2
−3
−4
3.0
3.1
3.2
3.3
3.4
3.5
3.6
3.7
3.8



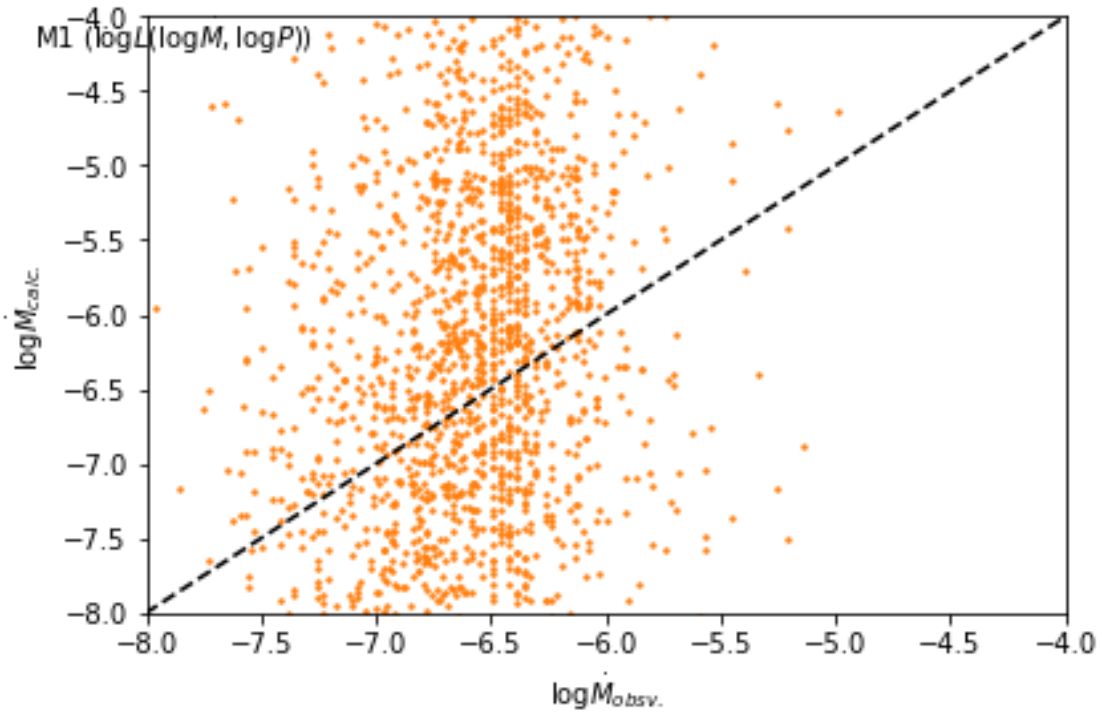
[390]:
```
print('M1:')
MdotLP_Coeffs_M1, LMdotP_Coeffs_M1 = attenuation_test␣
 ↪(M1StarData,spec_mode_label='M1')
```

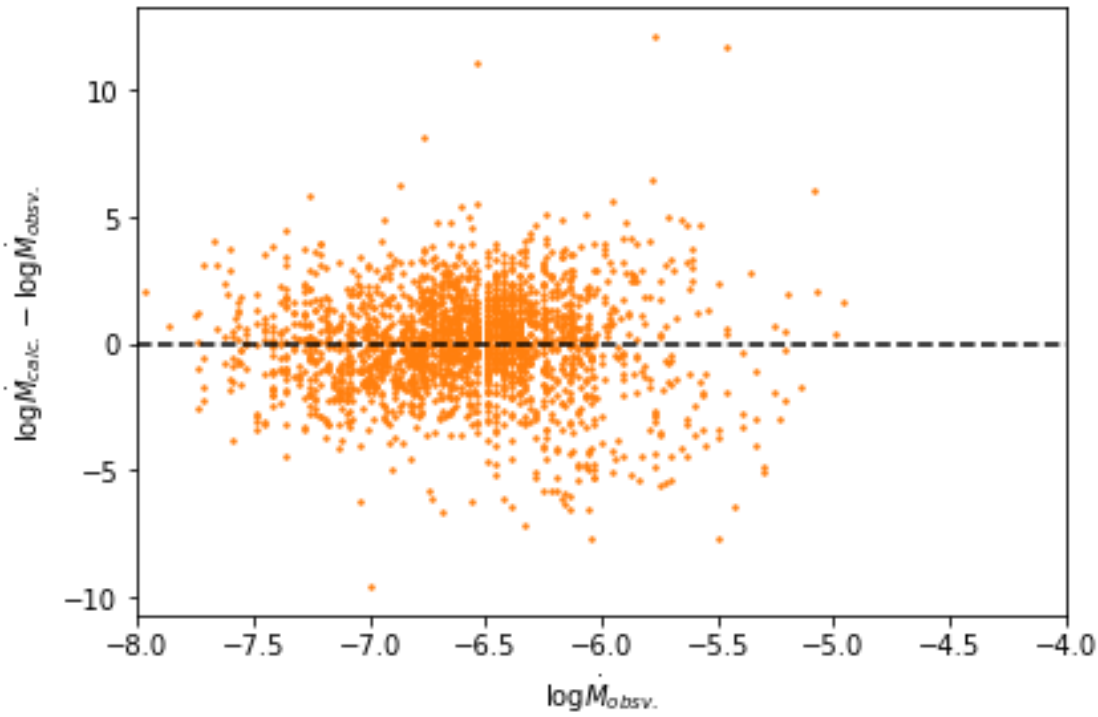
```
M1:
log Mdot(logL, logP):
LM form: logMdot = -14.331671442964444  +  2.1658349824618766 logL -
2.504190194588611 logM
```


M1 ($\log\dot{M}(\log L, \log P)$)
$\log\dot{M}_{calc.}$
$\log\dot{M}_{obsv.}$
−4.0
−4.5
−5.0
−5.5
−6.0
−6.5
−7.0
−7.5
−8.0

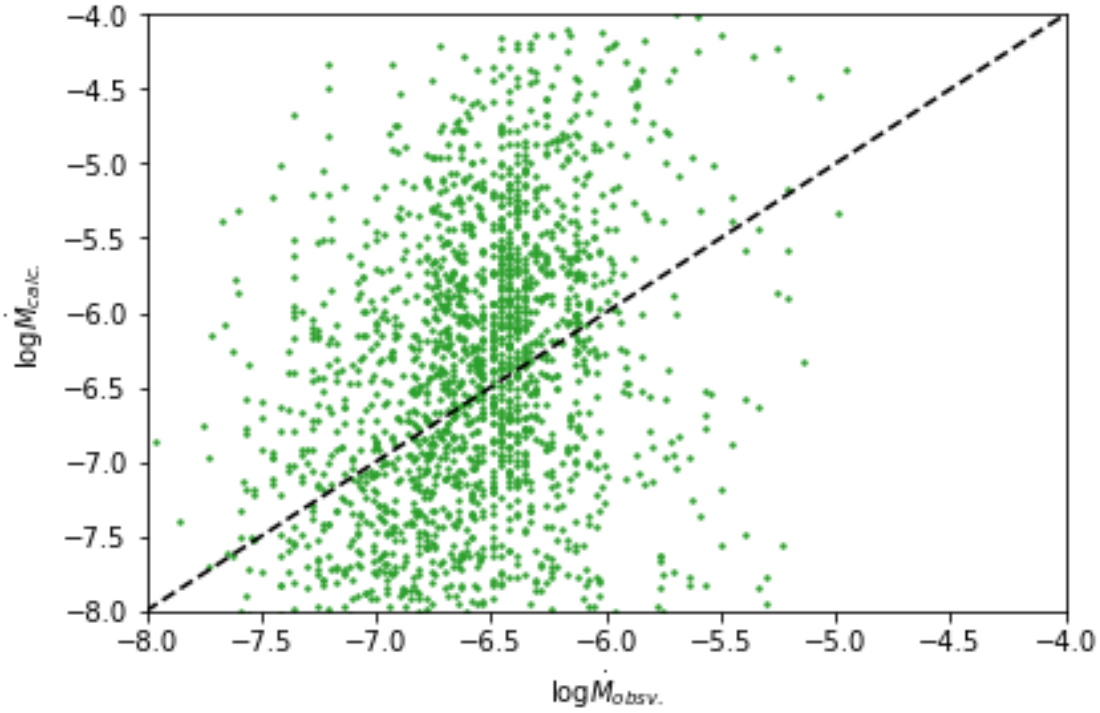

M1 ($\log\dot{M}(\log L, \log P)$)
$\log\dot{M}_{calc.} - \log\dot{M}_{obsv.}$
$\log\dot{M}_{obsv.}$
1.0
0.5
0.0
−0.5
−1.0
−1.5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0

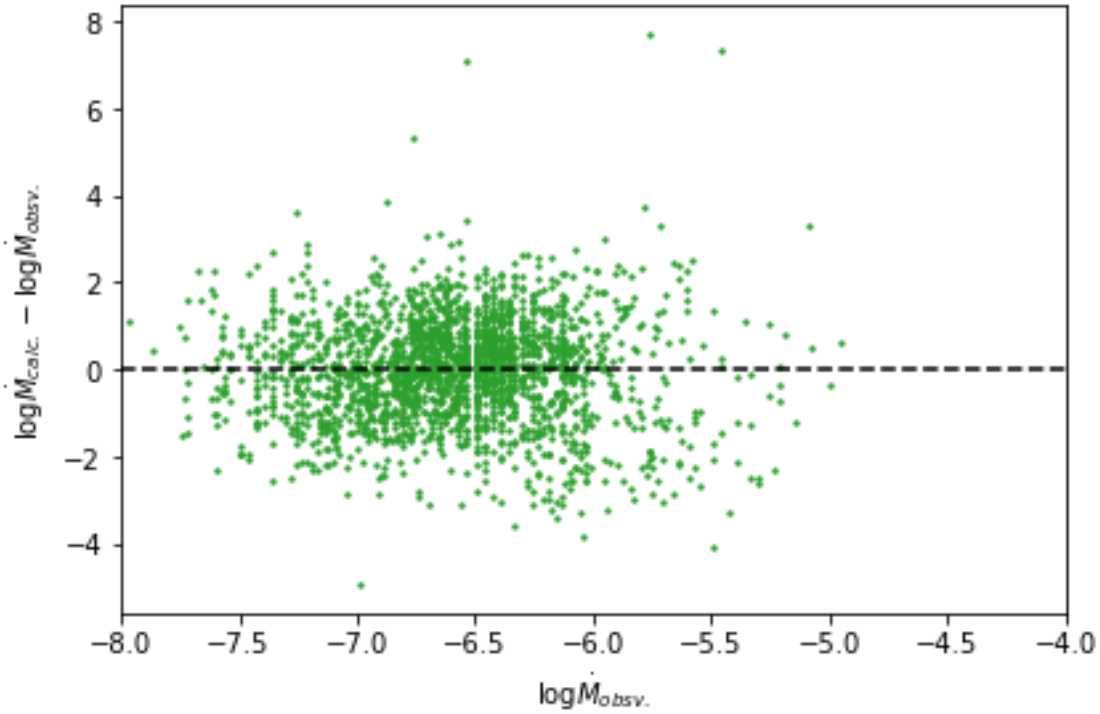
log L(logMdot, logP):
LM form: logMdot = -54.82132888171585 + 13.517235076993826 logL -
32.06134003667905 logM



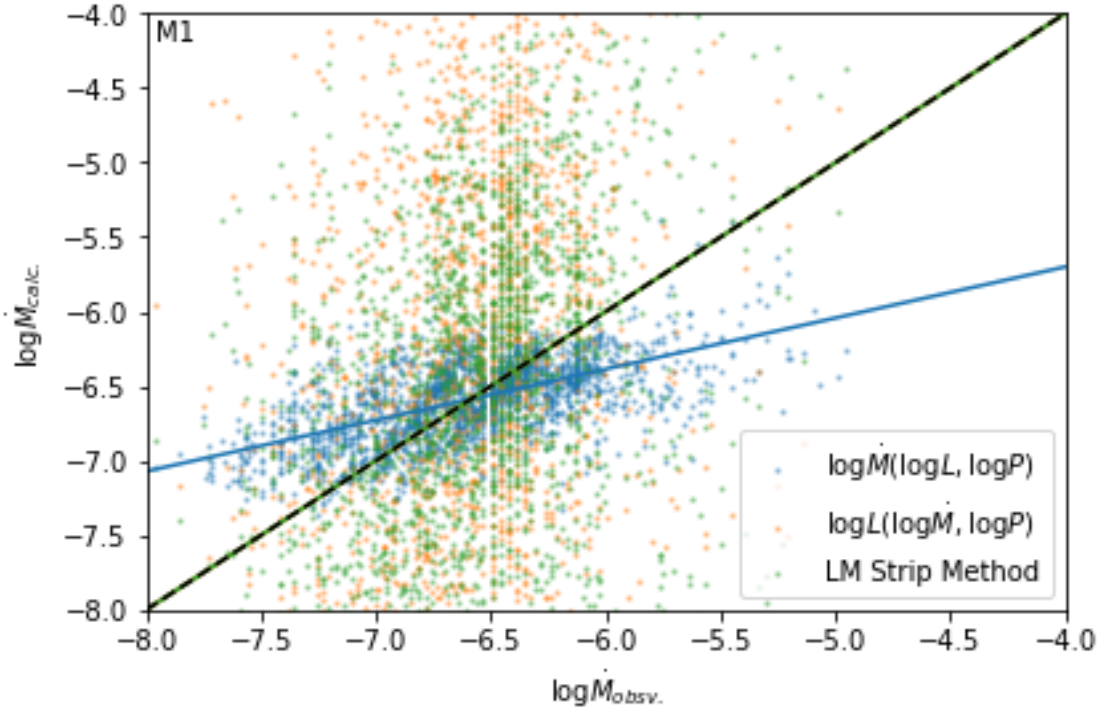



```
log Mdot(logL, logM):
LM form: logMdot = -42.82979371999287  +  10.152304120143375 logL -
20.474047210262466 logM
```

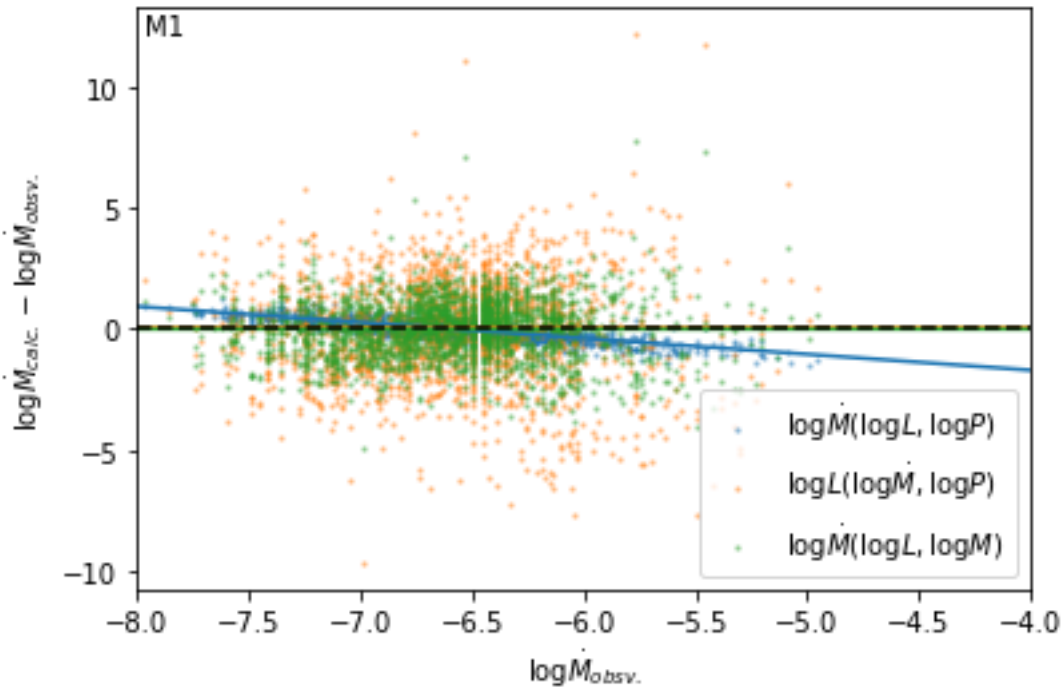

−4.0
−4.5
−5.0
−5.5
−6.0
−6.5
−7.0
−7.5
−8.0
$\log \dot{M}_{calc.}$
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
$\log \dot{M}_{obsv.}$

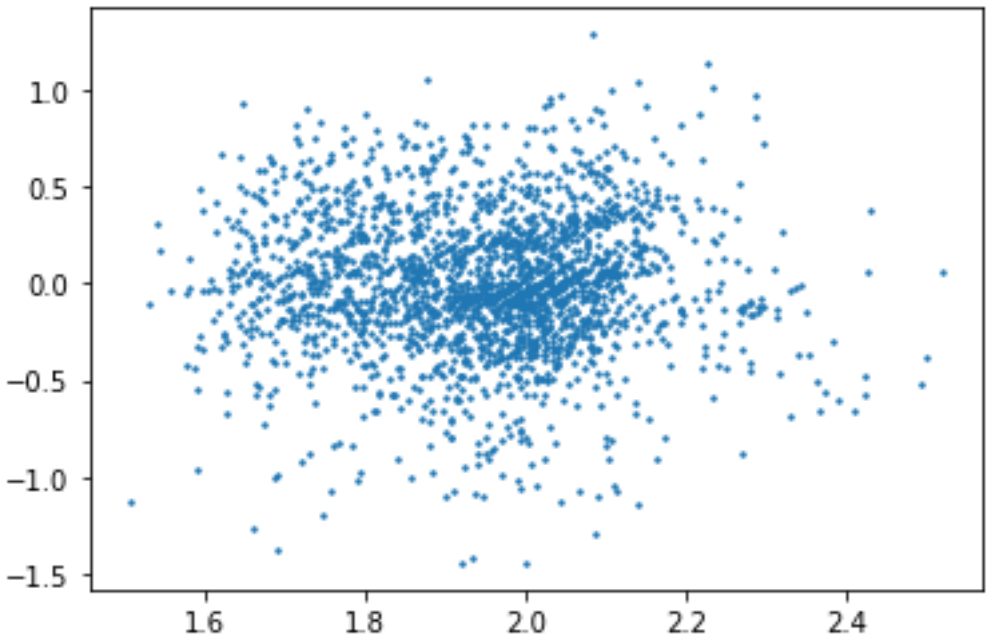

8
6
4
2
0
−2
−4
$\log \dot{M}_{calc.} - \log \dot{M}_{obsv.}$
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
$\log \dot{M}_{obsv.}$

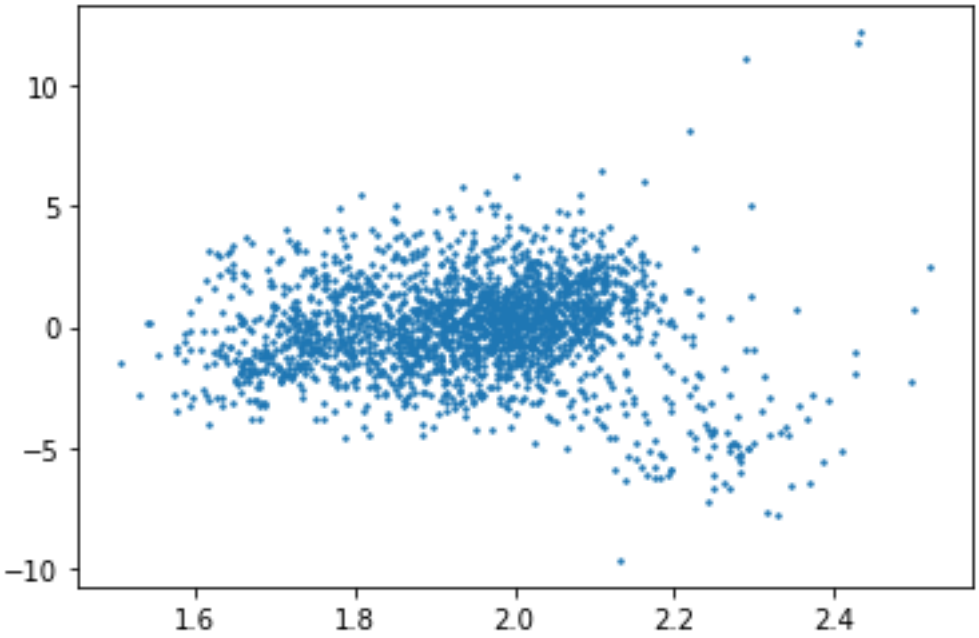
```
Mdot(L,P) Calc. vs. Obsv. Slope= 0.3429209780819571
Mdot(L,P) Calc. vs. Obsv. R^2= 0.34292097808196
Mdot(L,P) Std. Dev. of Residual: sigma= 0.35938566943043987
Mdot(L,P) Mean Distance from 1-to-1 Line: 0.271620712736137
L(Mdot,P) Calc. vs. Obsv. Slope= 0.999999999999977
L(Mdot,P) Calc. vs. Obsv. R^2= 0.04652718946690659
L(Mdot,P) Std. Dev. of Residual: sigma= 2.0070246269810417
L(Mdot,P) Mean Distance from 1-to-1 Line: 1.5085531841835311
LM Strip Calc. vs. Obsv. Slope= 0.9999967327250667
LM Strip Calc. vs. Obsv. R^2= 0.12227636713260515
LM Strip Std. Dev. of Residual: sigma= 1.1878406074267007
LM Strip Mean Distance from 1-to-1 Line: 0.9194940439999479
```

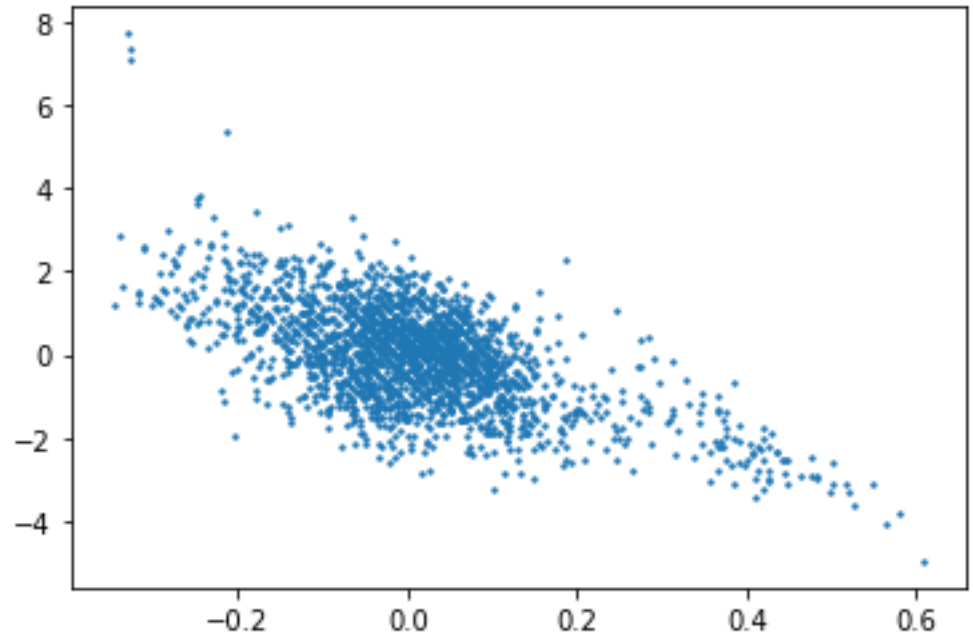


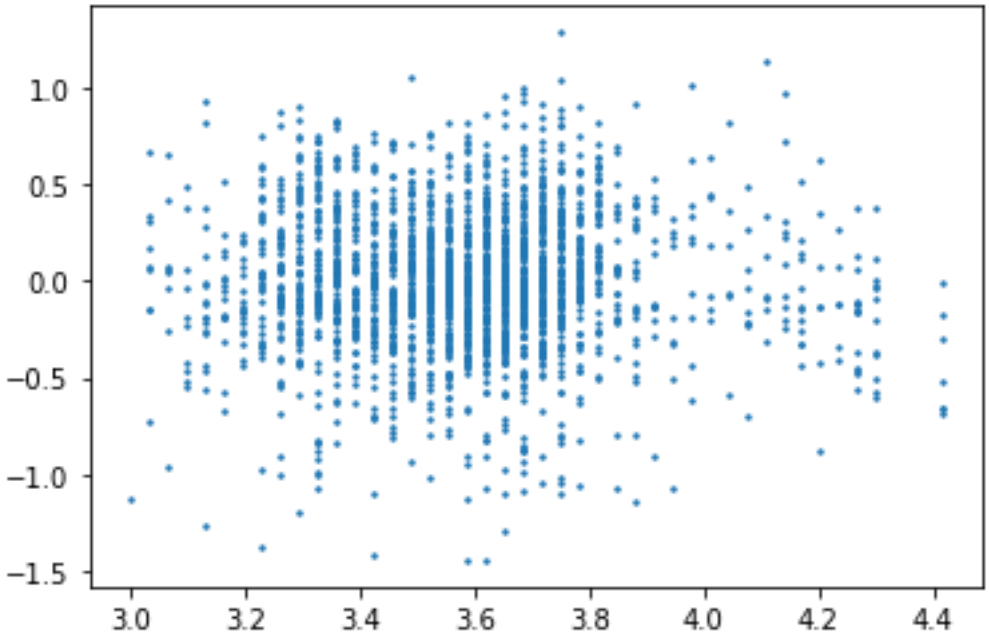

```
Mdot(L,P) Slope of Residual= -0.6570790219180418
Mdot(L,P) R^2 of Residual= 0.6570790219180425
L(Mdot,P) Slope of Residual= -2.079239558305801e-14
L(Mdot,P) R^2 of Residual= -2.220446049250313e-16
Mdot(L,M) Slope of Residual= -3.2672749339223567e-06
Mdot(L,M) R^2 of Residual= 1.4868106745780096e-12
```

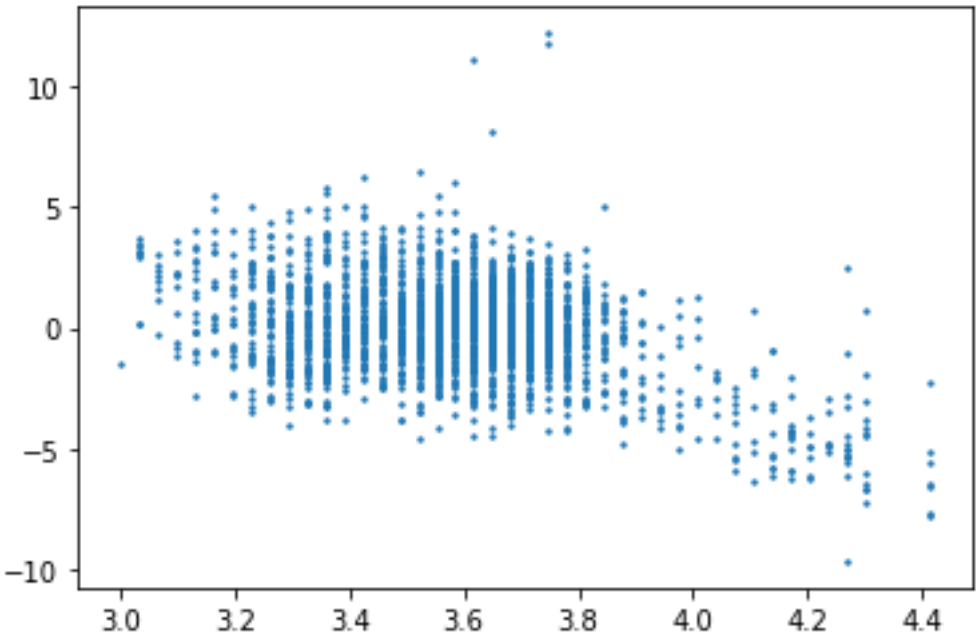

M1
10
5
0
−5
−10
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
$\log\dot{M}_{calc.} - \log\dot{M}_{obsv.}$
$\log\dot{M}_{obsv.}$
$\log\dot{M}(\log L, \log P)$
$\log L(\log\dot{M}, \log P)$
$\log\dot{M}(\log L, \log M)$

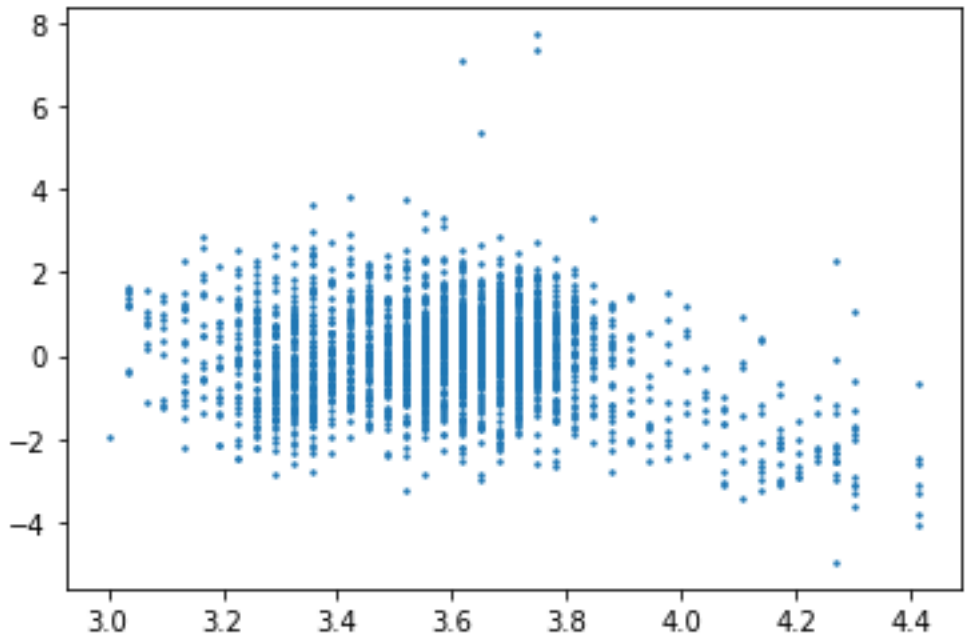

1.0
0.5
0.0
−0.5
−1.0
−1.5
1.6
1.8
2.0
2.2
2.4


10
5
0
−5
−10
1.6
1.8
2.0
2.2
2.4


8
6
4
2
0
−2
−4
−0.2
0.0
0.2
0.4
0.6

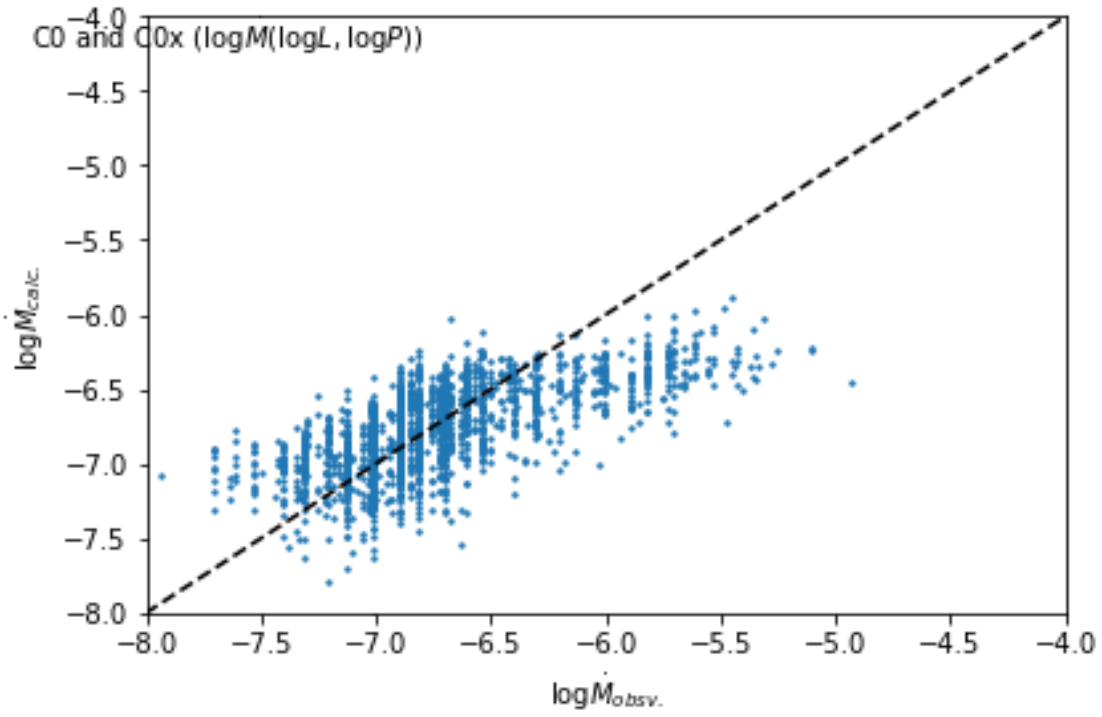

1.0
0.5
0.0
−0.5
−1.0
−1.5
3.0
3.2
3.4
3.6
3.8
4.0
4.2
4.4

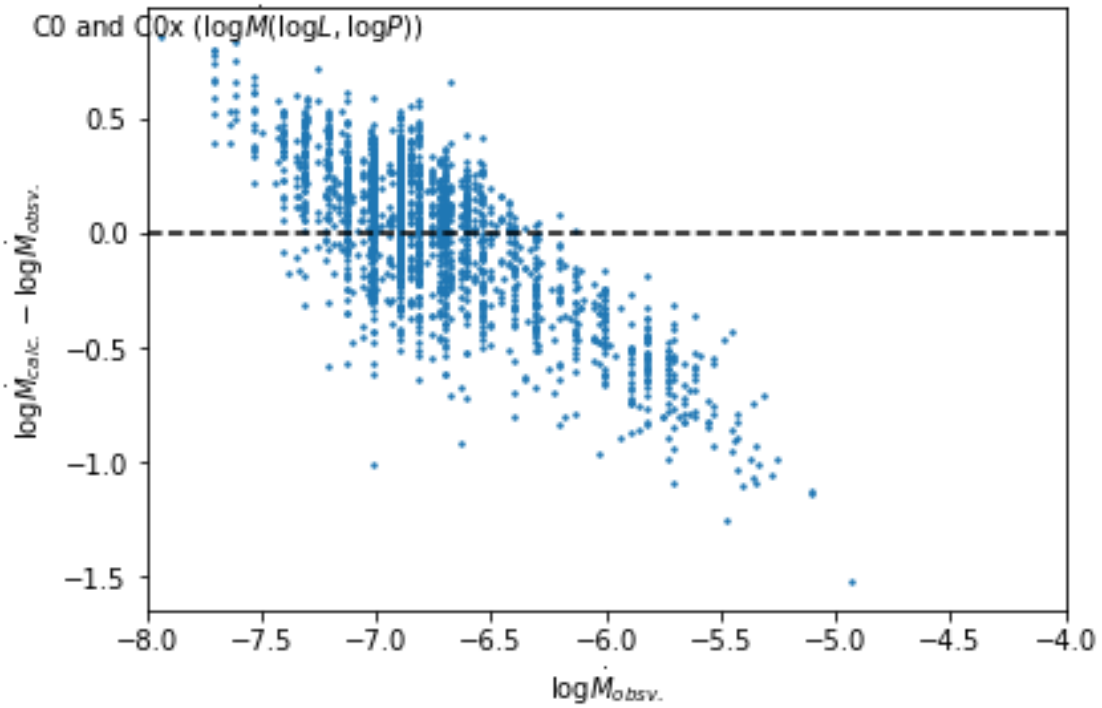

10
5
0
−5
−10
3.0
3.2
3.4
3.6
3.8
4.0
4.2
4.4



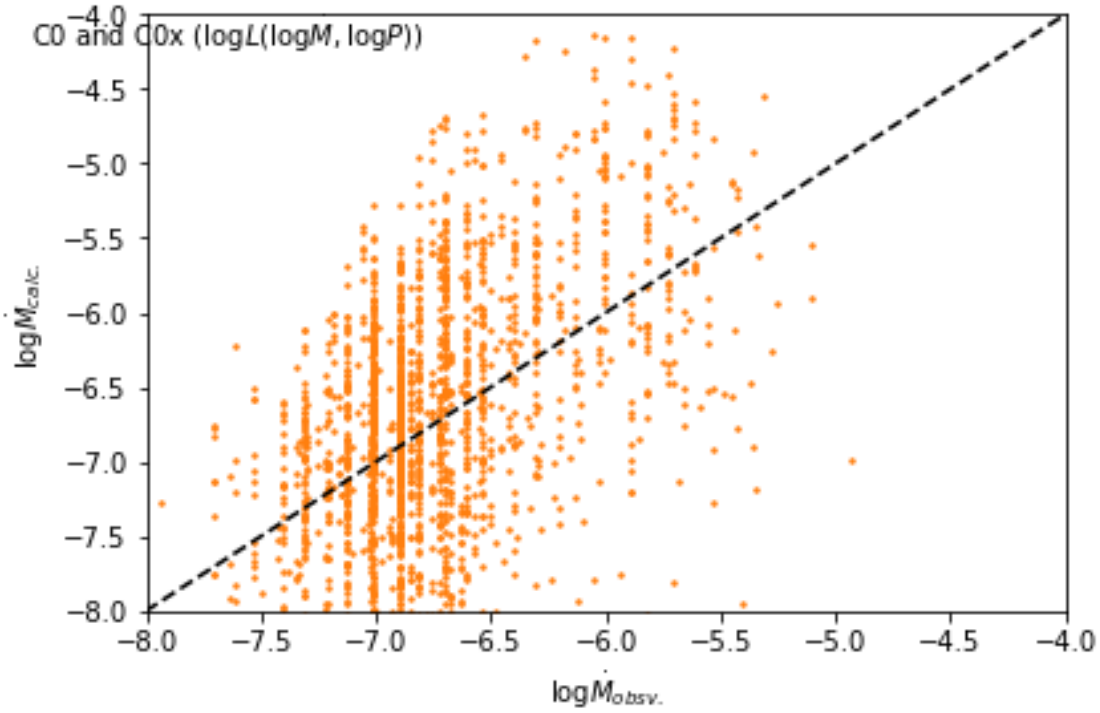
[391]:
```
print('C0 & C0x:')
MdotLP_Coeffs_C0andC0x, LMdotP_Coeffs_C0andC0x = attenuation_test␣
↪(C0StarData,spec_mode_label='C0 and C0x',text_x=0.09)
```

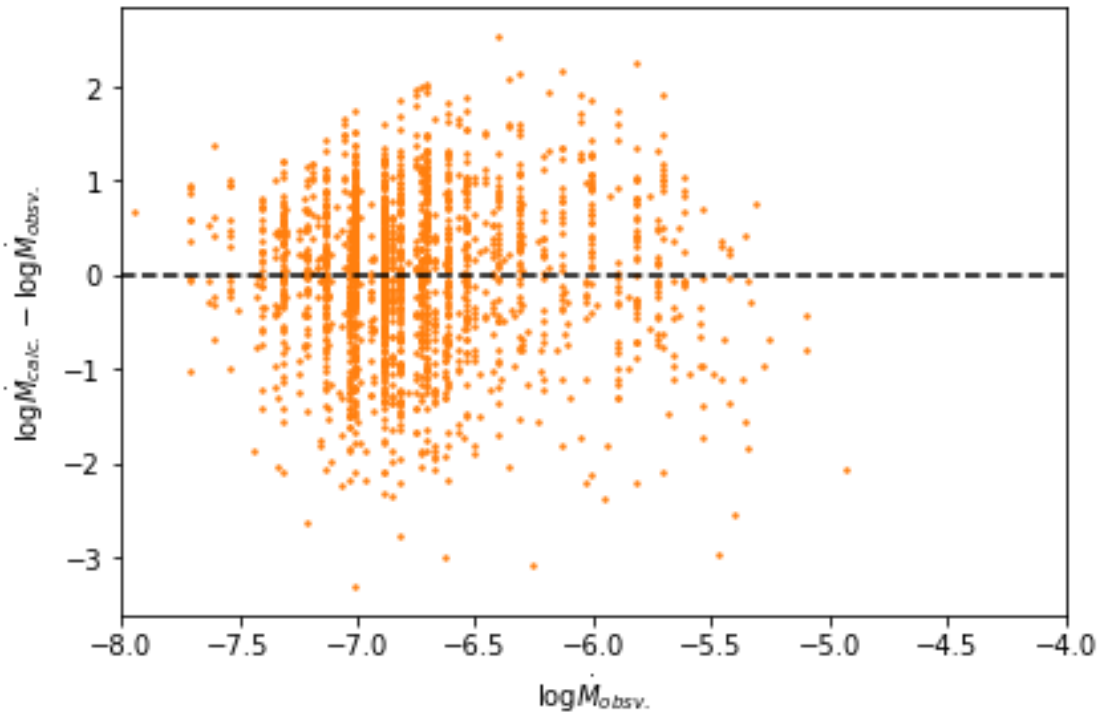
```
C0 & C0x:
log Mdot(logL, logP):
LM form: logMdot = -21.899136660341252  +  4.225637806117329 logL -
5.663741216898915 logM
```


C0 and C0x (logM(logL, logP))
−4.0
−4.5
−5.0
−5.5
−6.0
−6.5
−7.0
−7.5
−8.0
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
$\log \dot{M}_{calc.}$
$\log \dot{M}_{obsv.}$

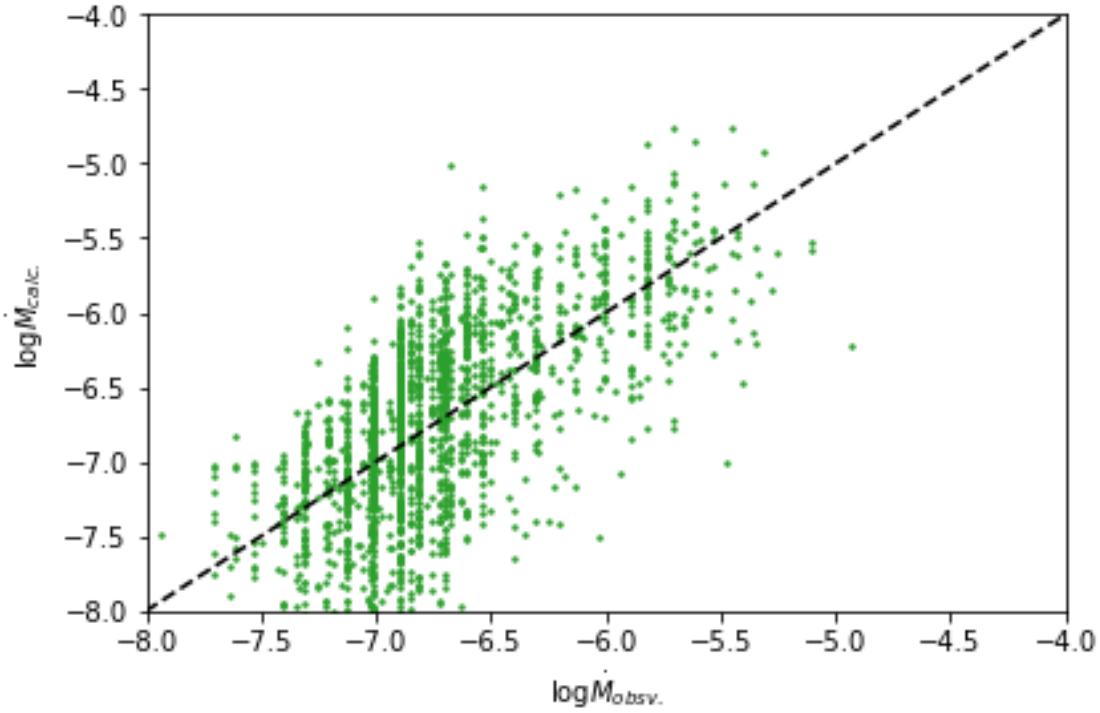

C0 and C0x (logM(logL, logP))
0.5
0.0
−0.5
−1.0
−1.5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
$\log \dot{M}_{calc.} - \log \dot{M}_{obsv.}$
$\log \dot{M}_{obsv.}$

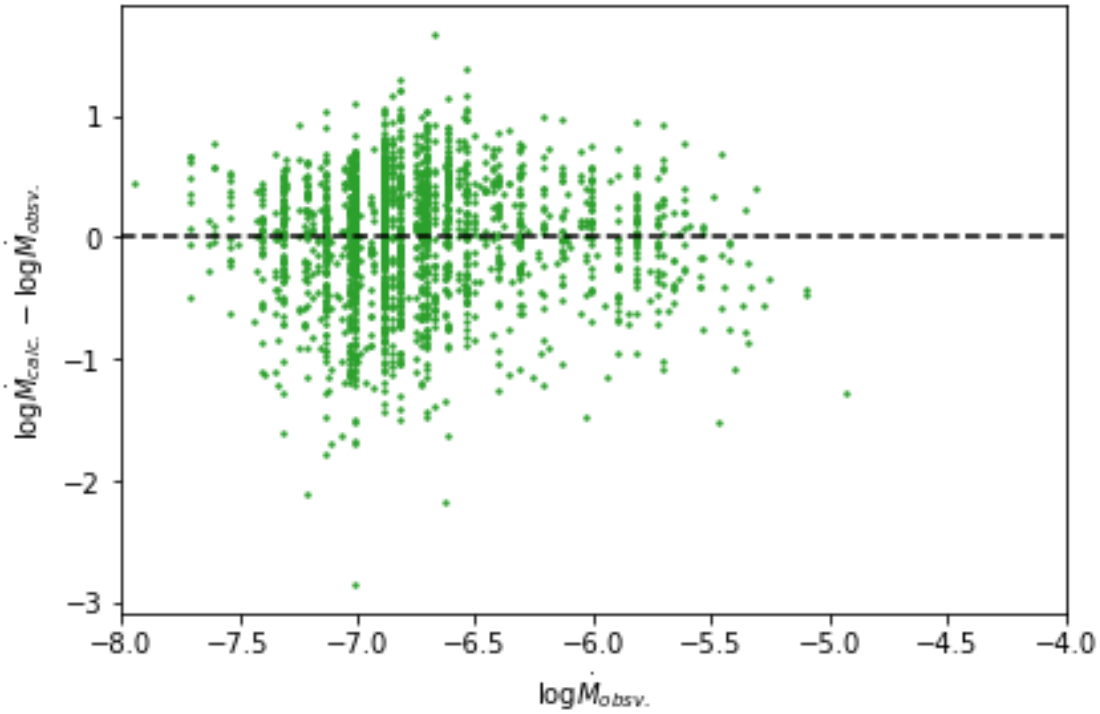
log L(logMdot, logP):
LM form: logMdot = -33.57042057795684 + 7.7359052335317 logL - 15.844802884488407 logM



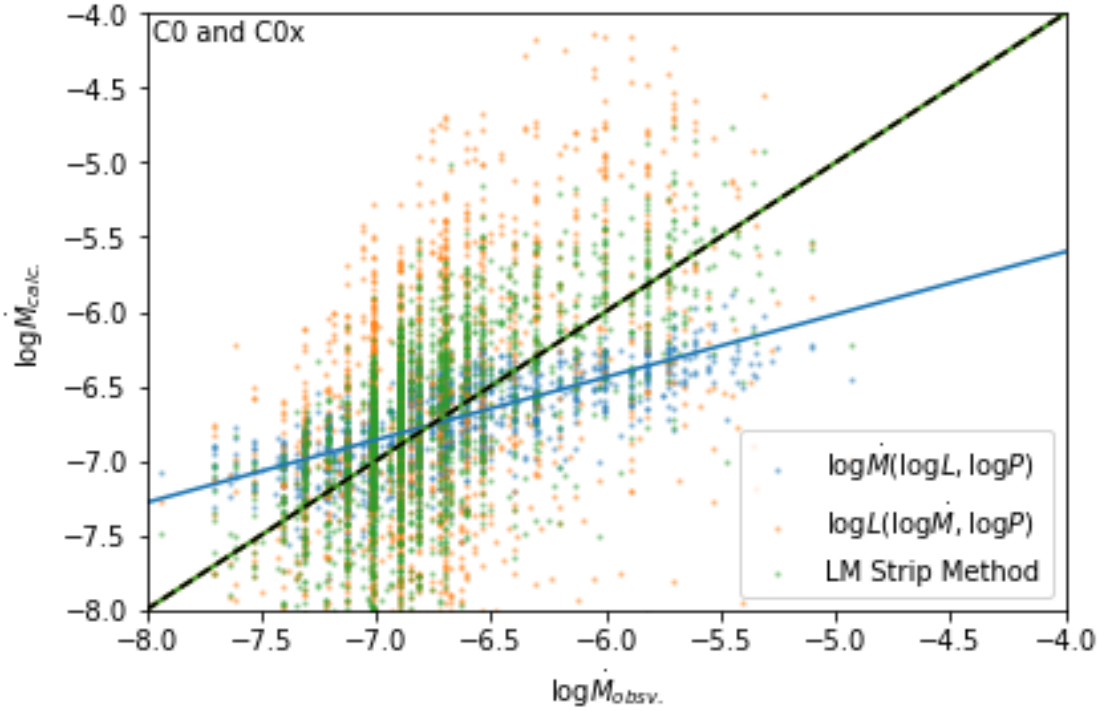



```
log Mdot(logL, logM):
LM form: logMdot = -41.078937026141794  +  9.635192545710597 logL -
13.910423664947327 logM
```

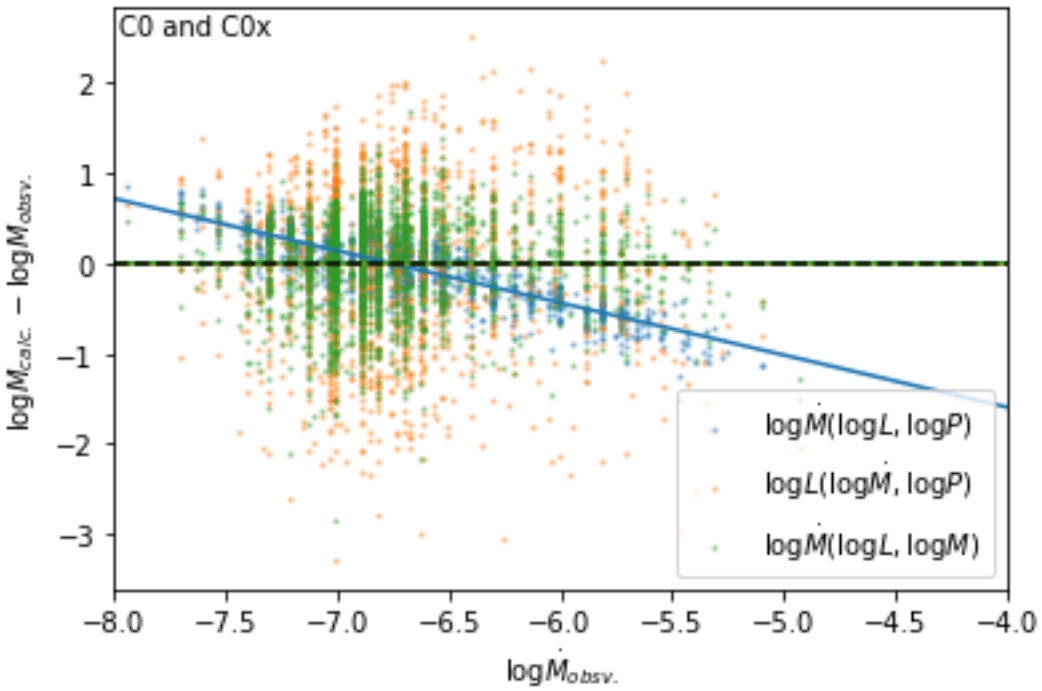

−4.0
−4.5
−5.0
−5.5
−6.0
−6.5
−7.0
−7.5
−8.0
$\log \dot{M}_{calc.}$
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
$\log \dot{M}_{obsv.}$

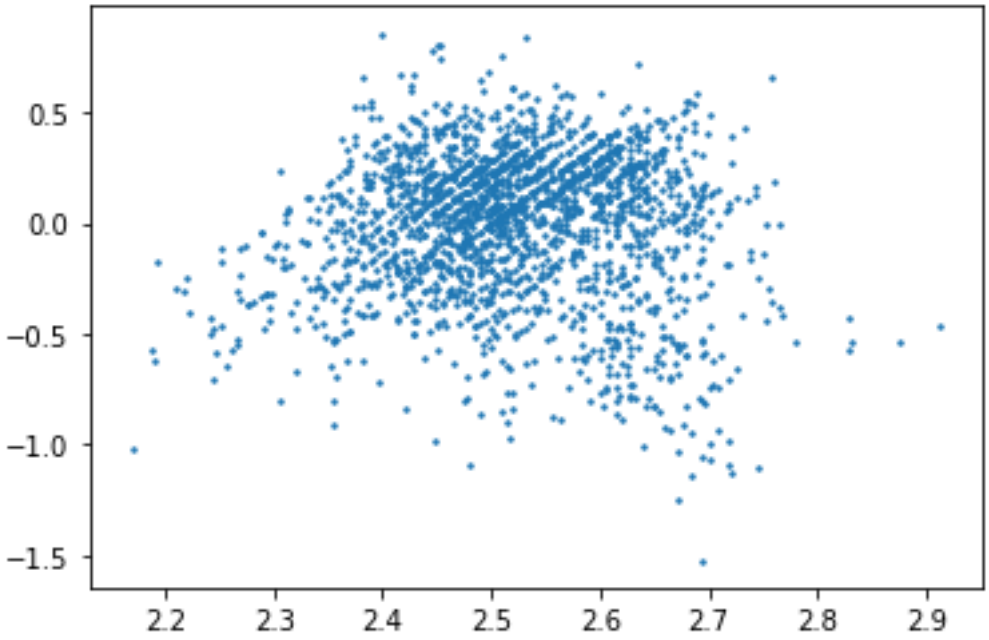

1
0
−1
−2
−3
$\log \dot{M}_{calc.} - \log \dot{M}_{obsv.}$
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
$\log \dot{M}_{obsv.}$

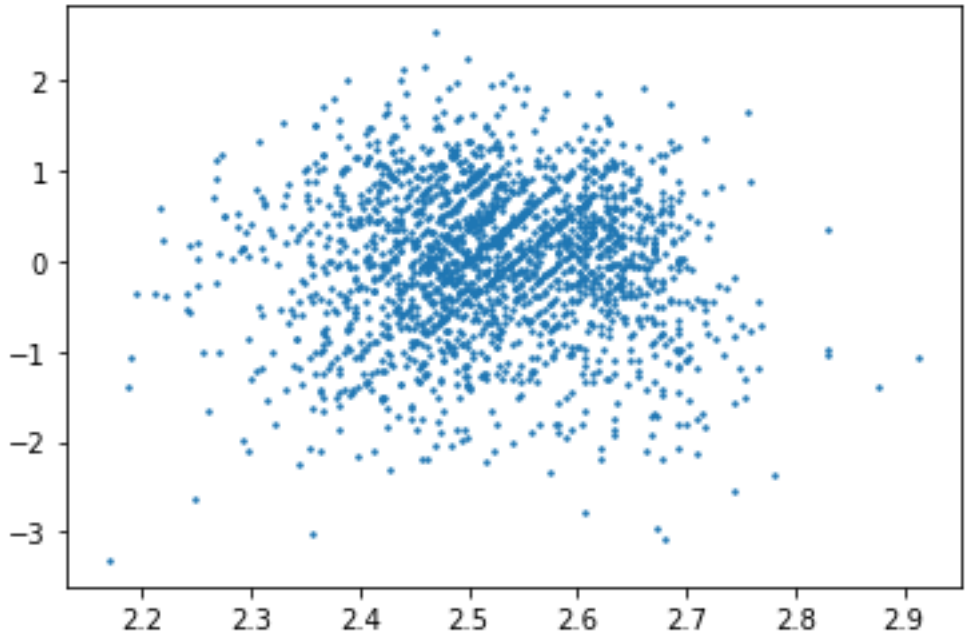
```
Mdot(L,P) Calc. vs. Obsv. Slope= 0.4206020792634797
Mdot(L,P) Calc. vs. Obsv. R^2= 0.42060207926348403
Mdot(L,P) Std. Dev. of Residual: sigma= 0.3266185342252046
Mdot(L,P) Mean Distance from 1-to-1 Line: 0.25853283112369024
L(Mdot,P) Calc. vs. Obsv. Slope= 1.000000000000007
L(Mdot,P) Calc. vs. Obsv. R^2= 0.2143761971586724
L(Mdot,P) Std. Dev. of Residual: sigma= 0.8214315786386992
L(Mdot,P) Mean Distance from 1-to-1 Line: 0.6481866165320419
LM Strip Calc. vs. Obsv. Slope= 1.0000291742728293
LM Strip Calc. vs. Obsv. R^2= 0.40738409029419864
LM Strip Std. Dev. of Residual: sigma= 0.517546827333577
LM Strip Mean Distance from 1-to-1 Line: 0.41014364984003904
```

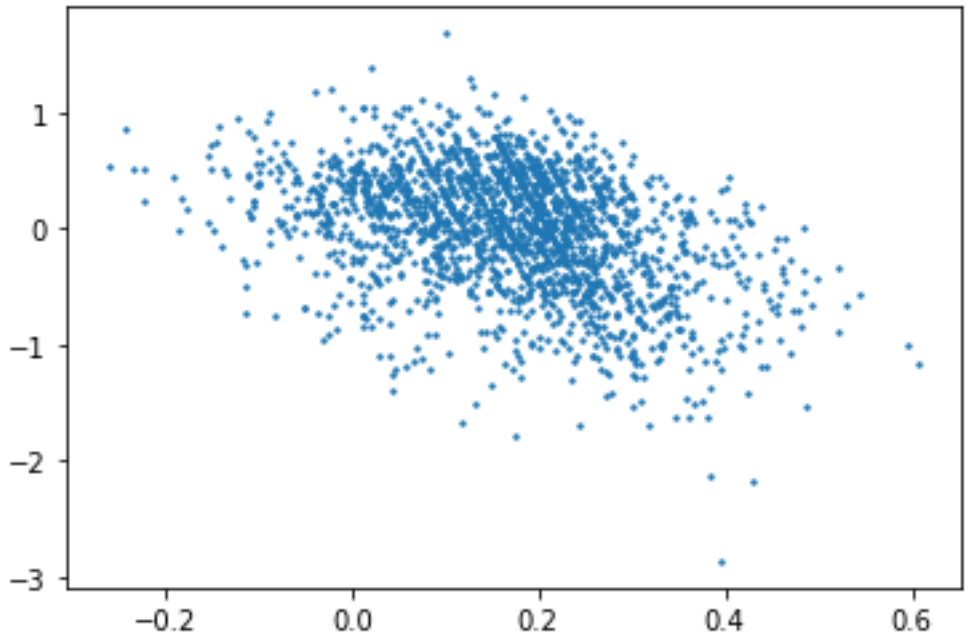


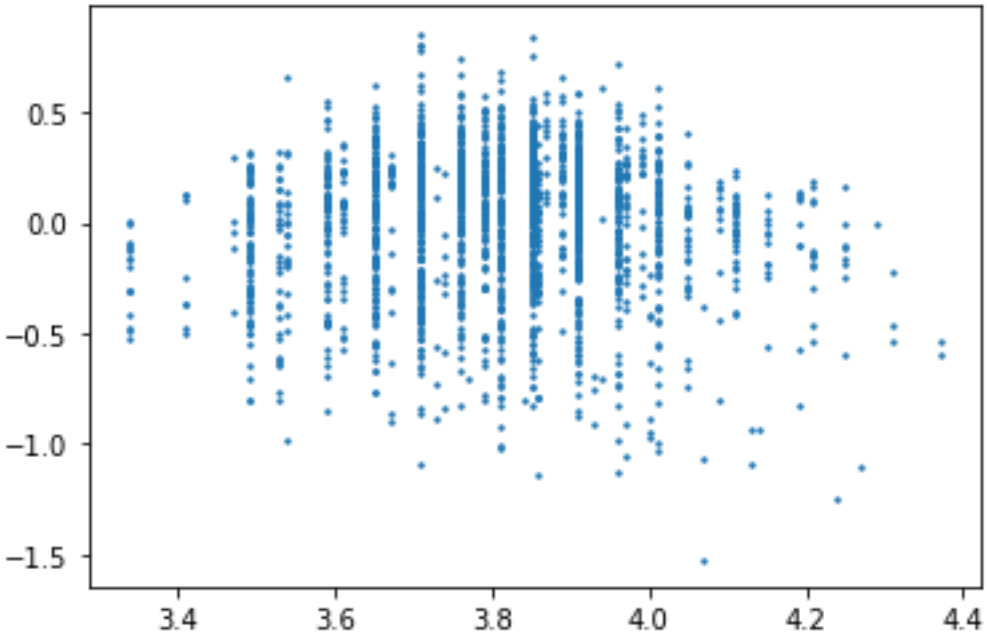

```
Mdot(L,P) Slope of Residual= -0.5793979207365173
Mdot(L,P) R^2 of Residual= 0.5793979207365181
L(Mdot,P) Slope of Residual= 6.612765890423589e-15
L(Mdot,P) R^2 of Residual= 2.220446049250313e-16
Mdot(L,M) Slope of Residual= 2.9174272822807615e-05
Mdot(L,M) R^2 of Residual= 5.850673279184093e-10
```

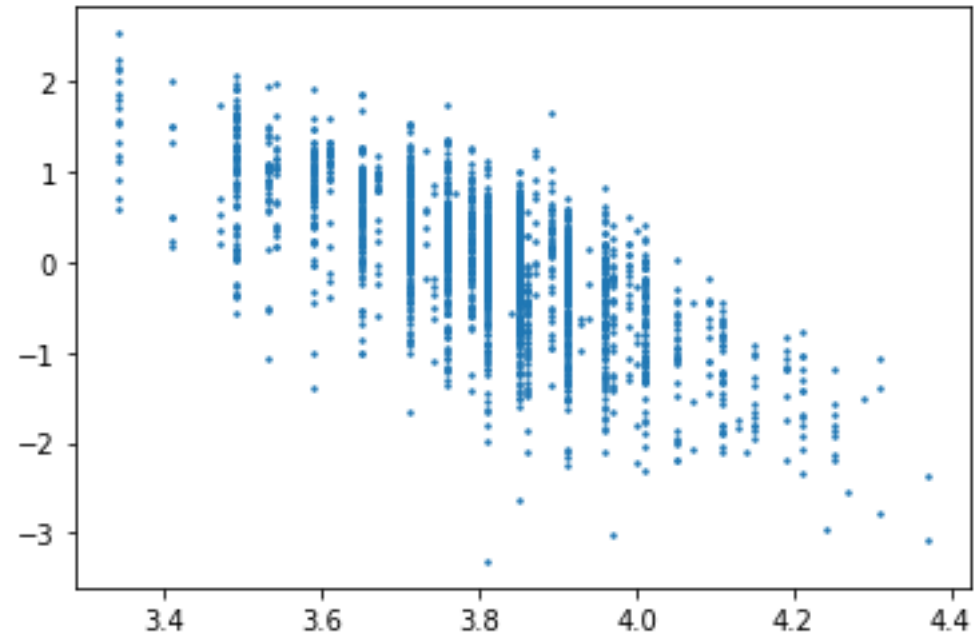

C0 and C0x
2
1
0
−1
−2
−3
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
$\log\dot{M}_{calc.} - \log\dot{M}_{obsv.}$
$\log\dot{M}_{obsv.}$
$\log\dot{M}(\log L, \log P)$
$\log L(\log\dot{M}, \log P)$
$\log\dot{M}(\log L, \log M)$

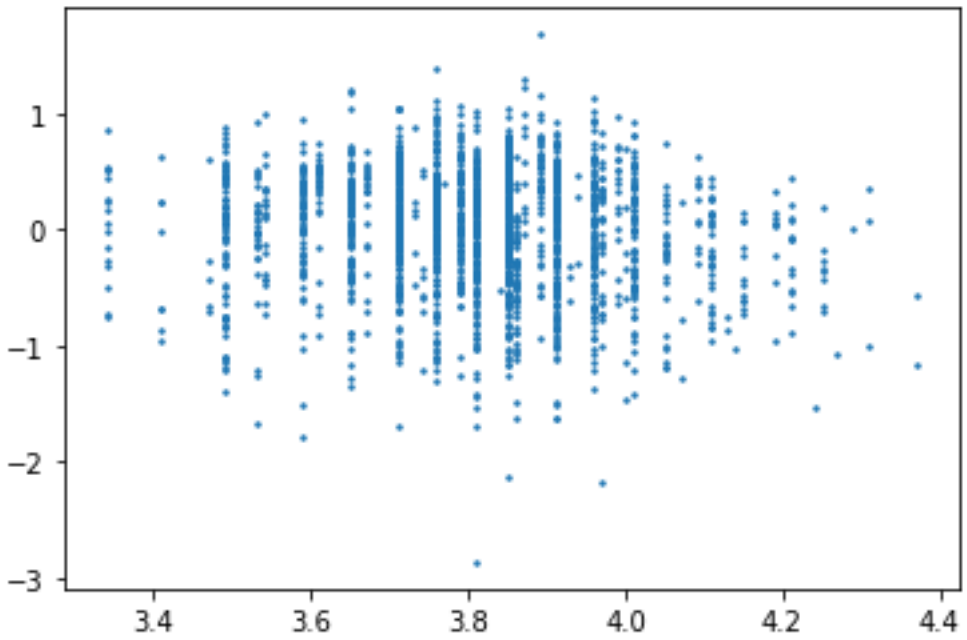

0.5
0.0
−0.5
−1.0
−1.5
2.2
2.3
2.4
2.5
2.6
2.7
2.8
2.9


2
1
0
−1
−2
−3
2.2
2.3
2.4
2.5
2.6
2.7
2.8
2.9


1
0
−1
−2
−3
−0.2
0.0
0.2
0.4
0.6

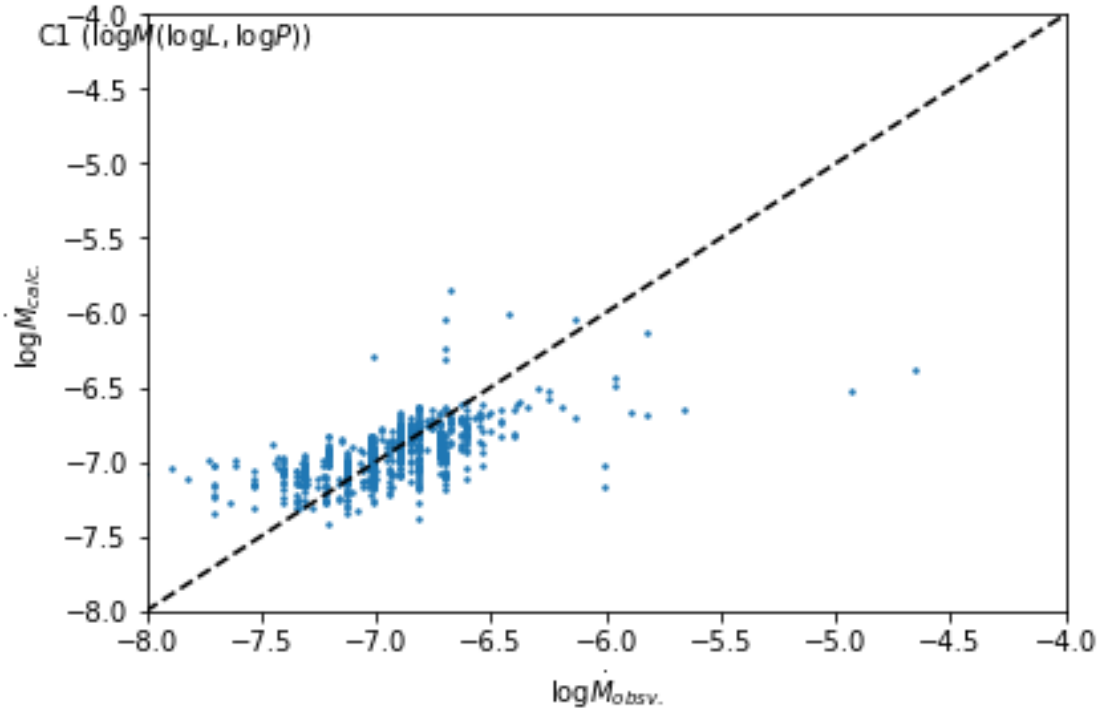

0.5
0.0
−0.5
−1.0
−1.5
3.4
3.6
3.8
4.0
4.2
4.4

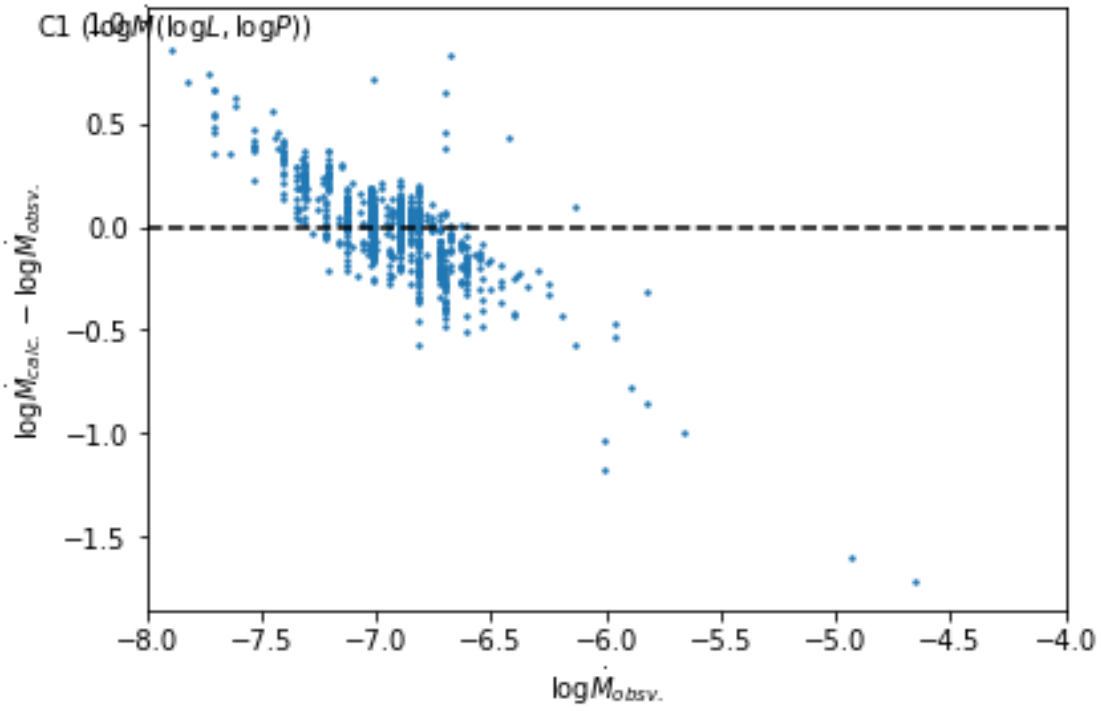

2
1
0
−1
−2
−3
3.4
3.6
3.8
4.0
4.2
4.4



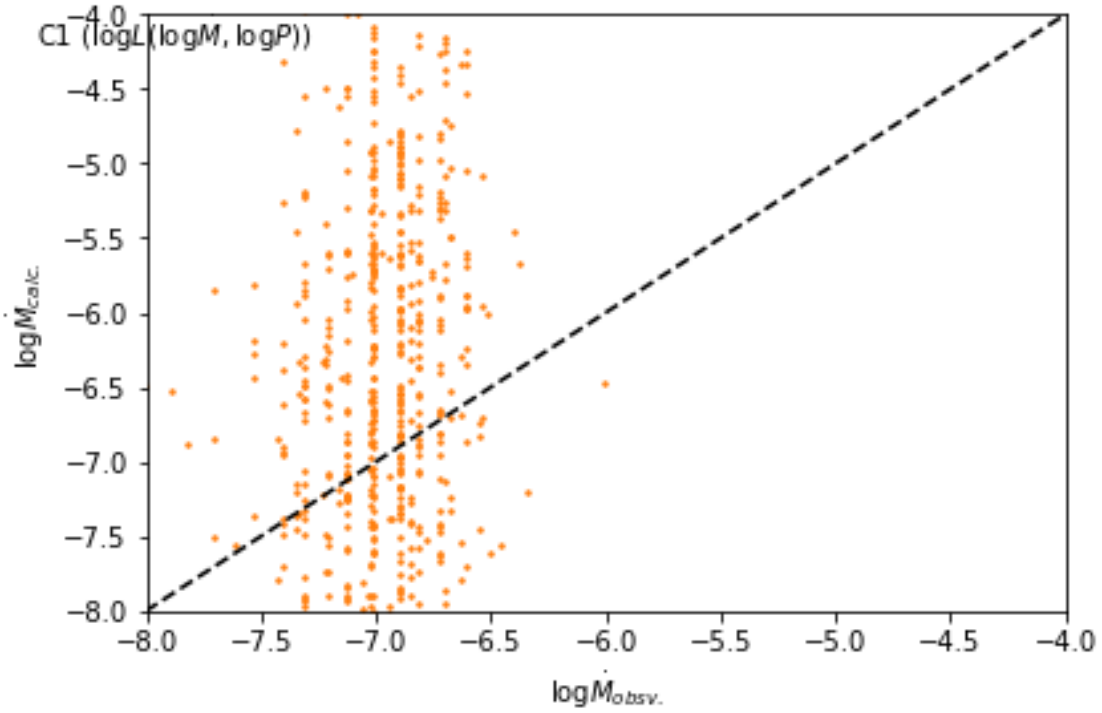
[392]:
```
print('C1:')
MdotLP_Coeffs_C1, LMdotP_Coeffs_C1 = attenuation_test␣
 ↪(C1StarData,spec_mode_label='C1')
```

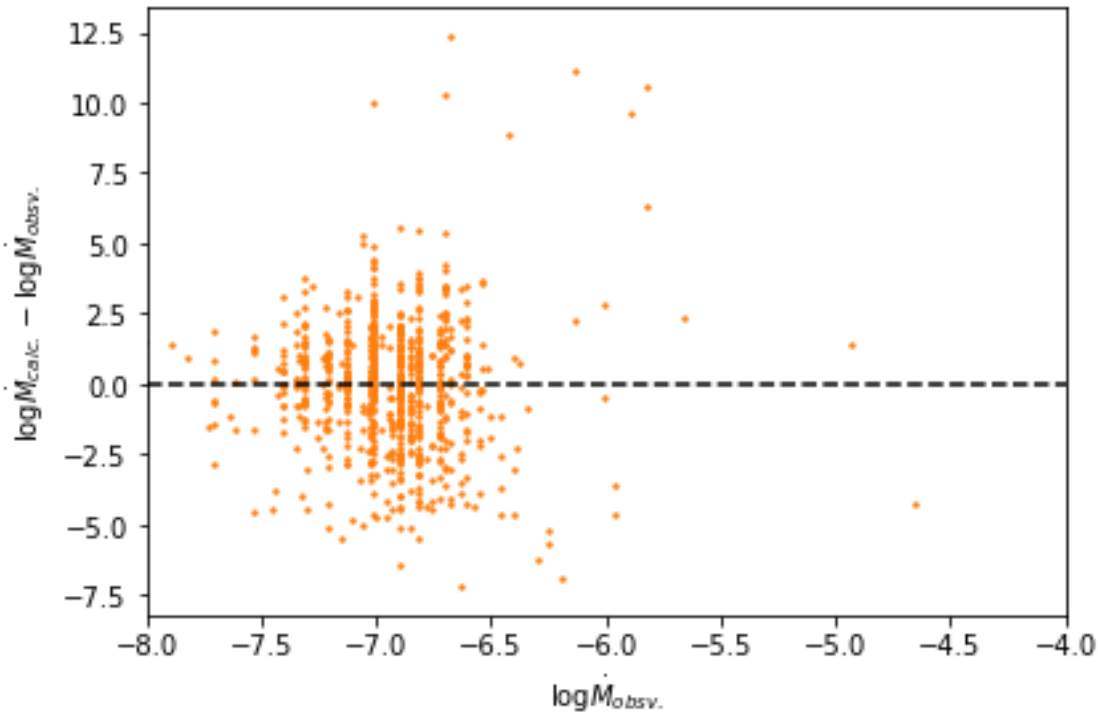
```
C1:
log Mdot(logL, logP):
LM form: logMdot = -15.10246424487756  +  2.20992367511773 logL -
2.3380226554375763 logM
```


C1 ($\log\dot{M}(\log L, \log P)$)
$\log\dot{M}_{calc.}$
$\log\dot{M}_{obsv.}$
−4.0
−4.5
−5.0
−5.5
−6.0
−6.5
−7.0
−7.5
−8.0

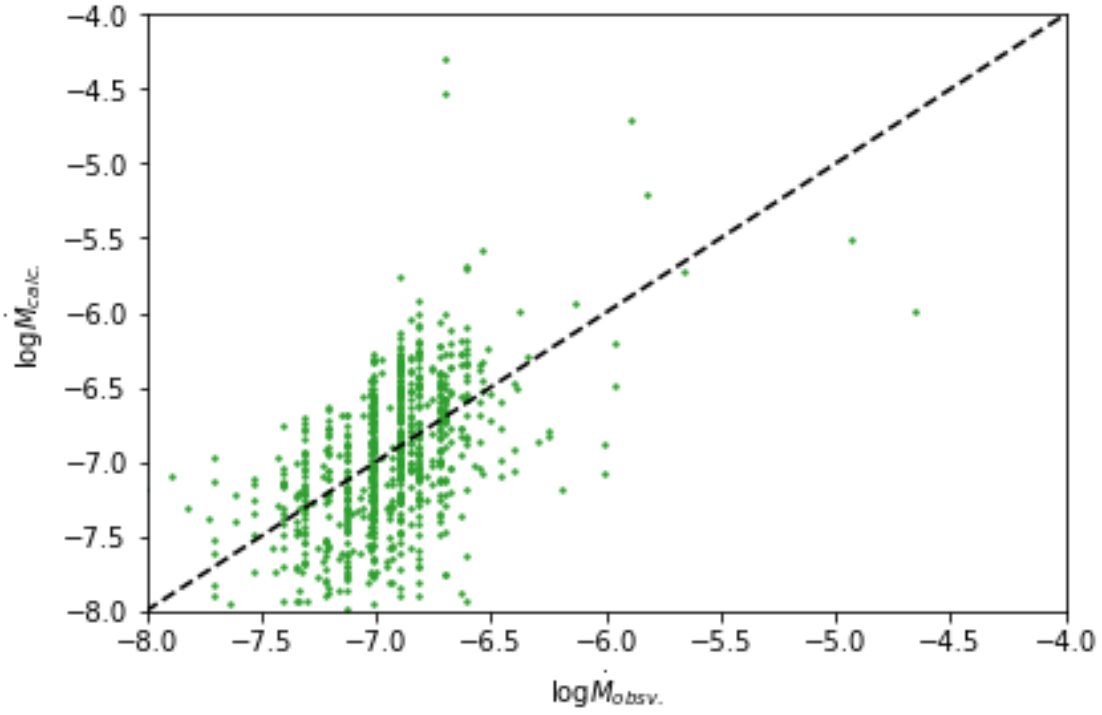

C1 ($\log\dot{M}(\log L, \log P)$)
$\log\dot{M}_{calc.} - \log\dot{M}_{obsv.}$
$\log\dot{M}_{obsv.}$
1.0
0.5
0.0
−0.5
−1.0
−1.5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0

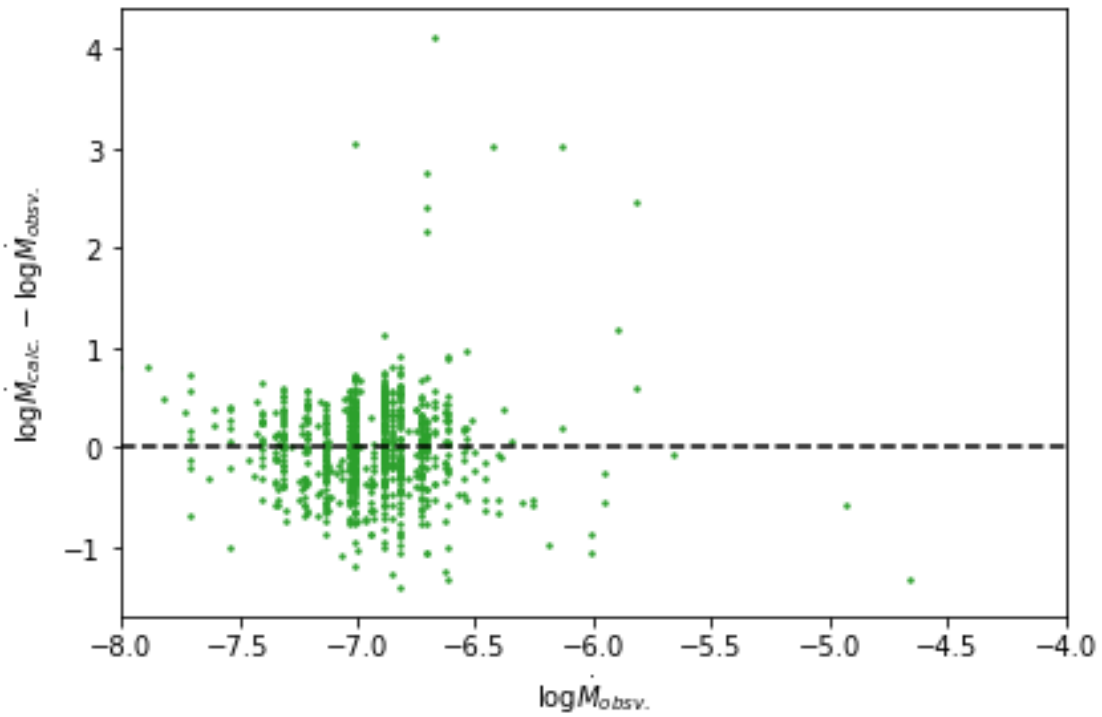
log L(logMdot, logP):
LM form: logMdot = -27.354879778189876 + 6.4691326966646425 logL -
27.737910863060566 logM



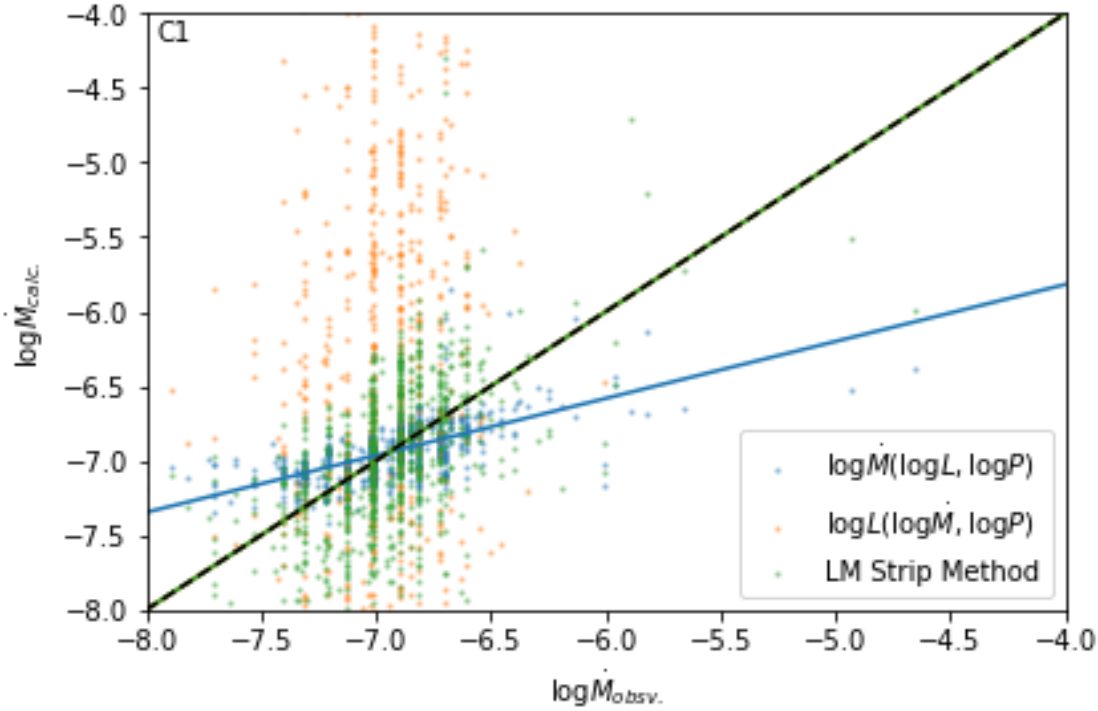



```
log Mdot(logL, logM):
LM form: logMdot = -28.14202038380528  +  5.88519600584648 logL -
9.437891494192556 logM
```

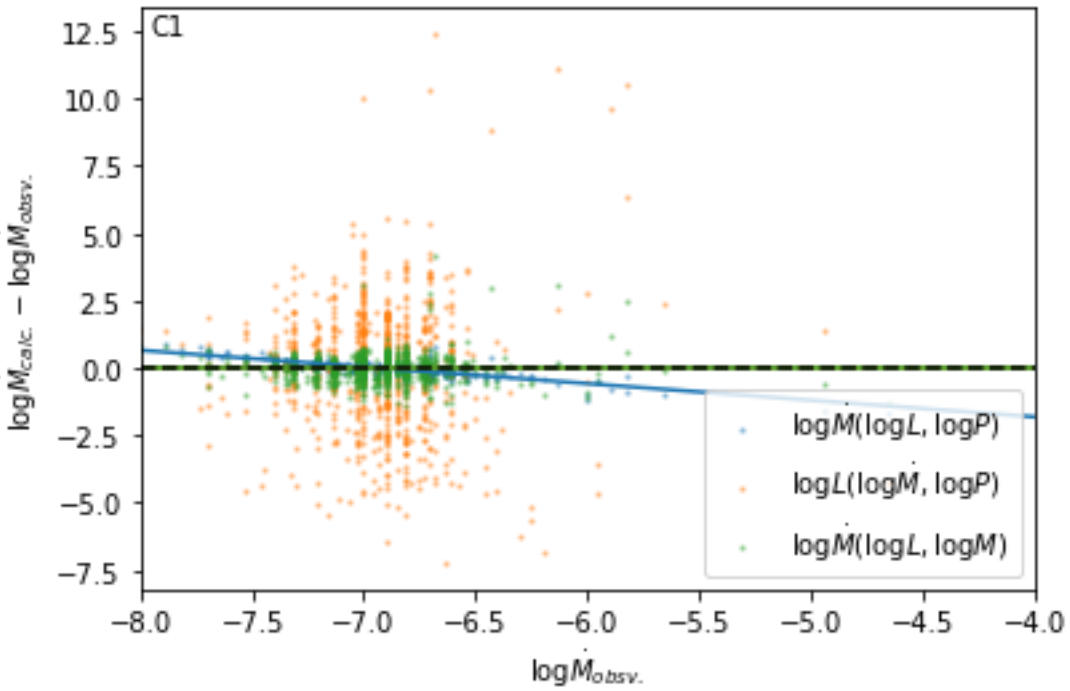

−4.0
−4.5
−5.0
−5.5
−6.0
−6.5
−7.0
−7.5
−8.0
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
$\log \dot{M}_{calc.}$
$\log \dot{M}_{obsv.}$

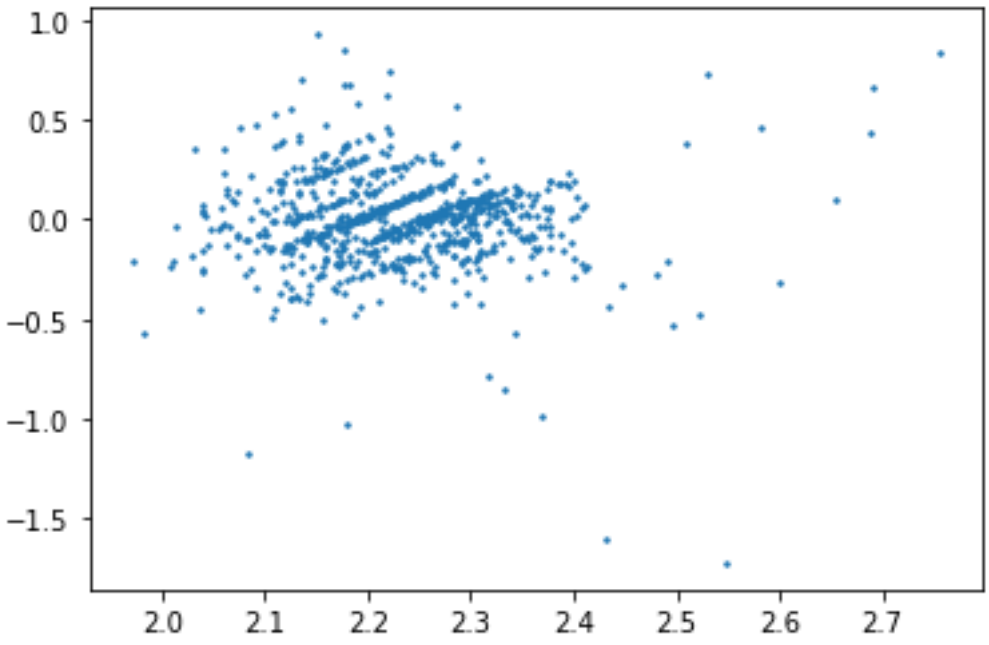

4
3
2
1
0
−1
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
$\log \dot{M}_{calc.} - \log \dot{M}_{obsv.}$
$\log \dot{M}_{obsv.}$

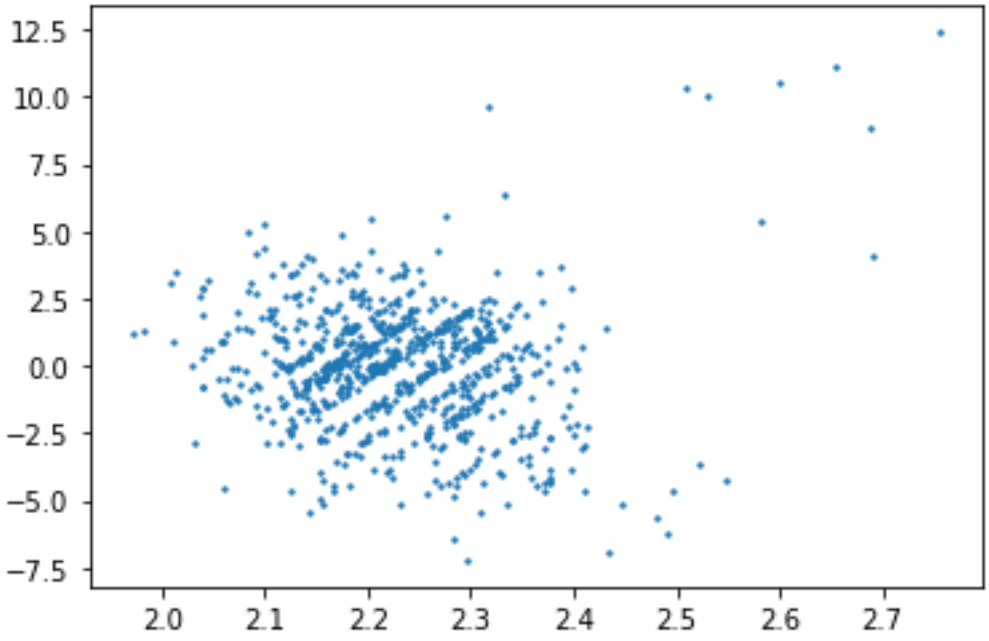
```
Mdot(L,P) Calc. vs. Obsv. Slope= 0.38212210637018057
Mdot(L,P) Calc. vs. Obsv. R^2= 0.3821221063701793
Mdot(L,P) Std. Dev. of Residual: sigma= 0.22963846506509997
Mdot(L,P) Mean Distance from 1-to-1 Line: 0.15594692855182238
L(Mdot,P) Calc. vs. Obsv. Slope= 1.000000000000022
L(Mdot,P) Calc. vs. Obsv. R^2= 0.015925948936884482
L(Mdot,P) Std. Dev. of Residual: sigma= 2.296437008808248
L(Mdot,P) Mean Distance from 1-to-1 Line: 1.6930783986375417
LM Strip Calc. vs. Obsv. Slope= 1.0000181660375453
LM Strip Calc. vs. Obsv. R^2= 0.24765173220056858
LM Strip Std. Dev. of Residual: sigma= 0.509201851017138
LM Strip Mean Distance from 1-to-1 Line: 0.3596709179565797
```

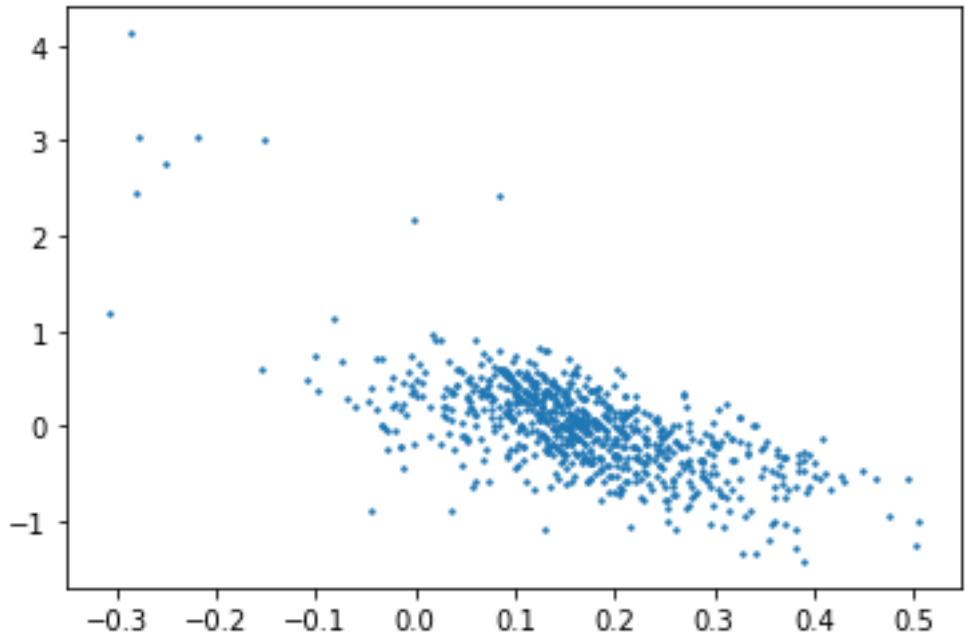


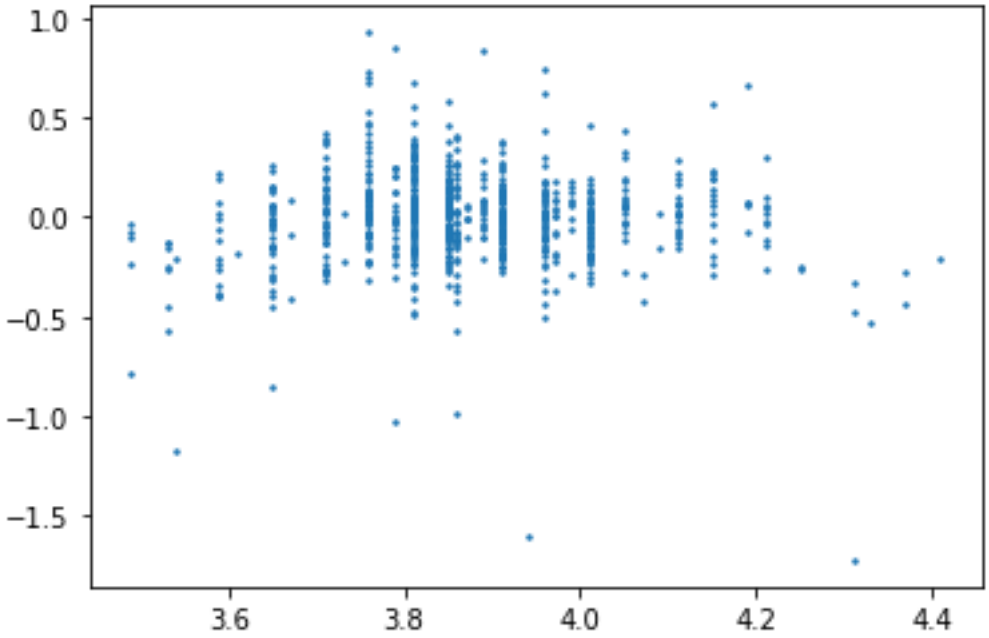

```
Mdot(L,P) Slope of Residual= -0.6178778936298202
Mdot(L,P) R^2 of Residual= 0.6178778936298202
L(Mdot,P) Slope of Residual= 2.2923503373295517e-14
L(Mdot,P) R^2 of Residual= -4.440892098500626e-16
Mdot(L,M) Slope of Residual= 1.81660375456129e-05
Mdot(L,M) R^2 of Residual= 1.0862422072932532e-10
```

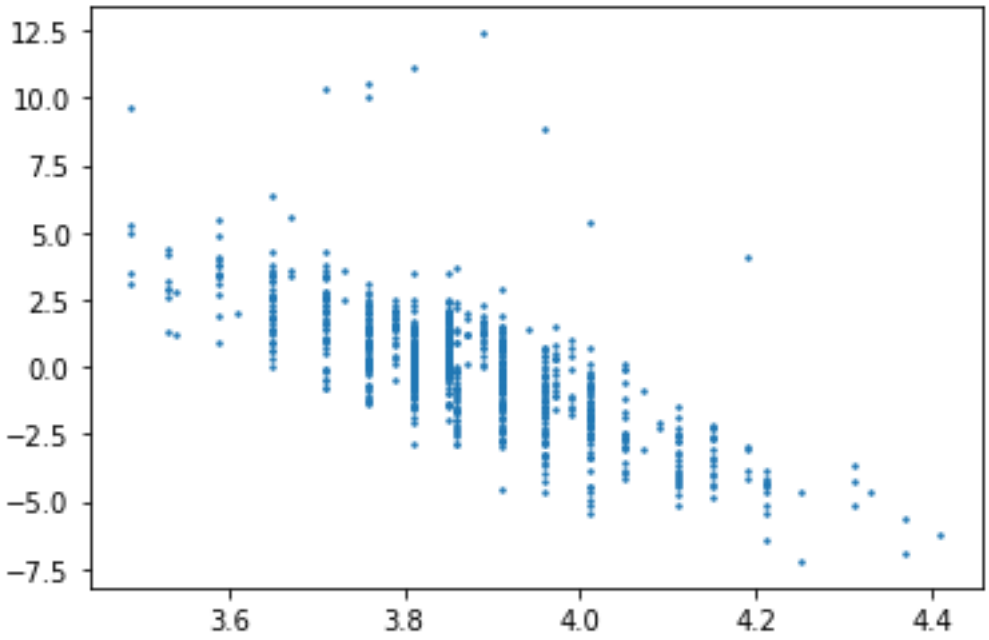

C1
12.5
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−8.0
−7.5
−7.0
−6.5
−6.0
−5.5
−5.0
−4.5
−4.0
$\log\dot{M}_{calc.} - \log\dot{M}_{obsv.}$
$\log\dot{M}_{obsv.}$
$\log\dot{M}(\log L, \log P)$
$\log L(\log\dot{M}, \log P)$
$\log\dot{M}(\log L, \log M)$

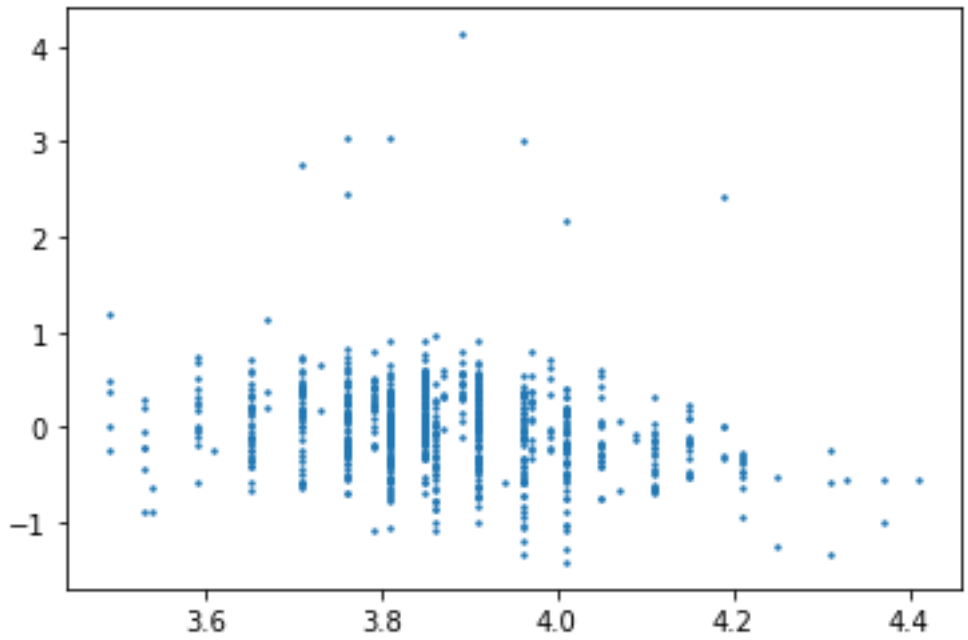

1.0
0.5
0.0
−0.5
−1.0
−1.5
2.0
2.1
2.2
2.3
2.4
2.5
2.6
2.7


12.5
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
2.0
2.1
2.2
2.3
2.4
2.5
2.6
2.7


4
3
2
1
0
−1
−0.3
−0.2
−0.1
0.0
0.1
0.2
0.3
0.4
0.5


1.0
0.5
0.0
−0.5
−1.0
−1.5
3.6
3.8
4.0
4.2
4.4


12.5
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
3.6
3.8
4.0
4.2
4.4



[369]:
```
np.shape(M0StarData)[1]+np.shape(M1StarData)[1]+np.shape(C0StarData)[1]+np.
↪shape(C1StarData)[1]
```

[369]: 6889

[370]:
```
def calculate_formula_correction (coeffs_1, coeffs_2, data_set,coeffs_labels =␣
↪['',''],grid_spacing=50,Mdotlimits=[-8,␣
↪-4],Mdotdustlimits=[-12,-8],Llimits=[2.8, 4.6],Plimits=[1.2, 3.
↪4],deltaMdotlimits=[-1, 1.75]):
    coeffs_1_LPMdot = coeffs_1[0]
    coeffs_1_PML = coeffs_1[1]
    coeffs_2_LPMdot = coeffs_2[0]
    coeffs_2_PML = coeffs_2[1]


    # create data sets of constant mass
    checked_log_m_array = []
    working_log_m = 0
    proceed = False
    constant_mass_lists = []
    i = 0
    while i < len(data_set[1]):
        indices_of_current_mass = []
        if i == 0:
            checked_log_m_array.append(data_set[1][0])
            working_log_m = data_set[1][0]
```

```
            for k in range(0, len(data_set[1])):
                if working_log_m == data_set[1][k]:
                    indices_of_current_mass.append(k)
            constant_mass_lists.append(indices_of_current_mass)
        else:
            working_log_m = data_set[1][i]
            for k in range(0, len(checked_log_m_array)):
                if working_log_m == checked_log_m_array[k]:
                    proceed = False
                    break
                else:
                    proceed = True
            if proceed == True:
                for l in range(0, len(data_set[1])):
                    checked_log_m_array.append(data_set[1][i])
                    if working_log_m == data_set[1][l]:
                        indices_of_current_mass.append(l)
                constant_mass_lists.append(indices_of_current_mass)
        #print('i:',i)
        i += 1
    #print('constant_mass_lists:',constant_mass_lists)
    i = 0
    constant_mass_arrays = []
    for i in range(0, len(constant_mass_lists)):
        constant_mass_arrays.append(np.transpose(np.
↪transpose(data_set)[constant_mass_lists[i]]))
    #print(constant_mass_arrays)


    # Calculate pulsation periods
    #print(coeffs_1_PML)
    logP_1 = coeffs_1_PML[0] + coeffs_1_PML[1]*data_set[1] +␣
↪coeffs_1_PML[2]*data_set[0]
    #print(logP_1)
    #print(coeffs_2_PML)
    logP_2 = coeffs_2_PML[0] + coeffs_2_PML[1]*data_set[1] +␣
↪coeffs_2_PML[2]*data_set[0]
    #print(logP_2)

    # some testing plots
    plot.scatter(data_set[0],logP_1,s=2,label=coeffs_labels[0])
    plot.scatter(data_set[0],logP_2,s=2,label=coeffs_labels[1])
    plot.legend()
    plot.xlabel(r'$\log L$')
    plot.xlim(Llimits)
    plot.ylabel(r'$\log P$')
    plot.ylim(Plimits)
```

```
    plot.show()
    plot.close()

    # Now, figure out the difference between the mass-loss rates
    logMdot_1 = coeffs_1_LPMdot[0] + coeffs_1_LPMdot[1]*data_set[0] +␣
↪coeffs_1_LPMdot[2]*logP_1
    logMdot_2 = coeffs_2_LPMdot[0] + coeffs_2_LPMdot[1]*data_set[0] +␣
↪coeffs_2_LPMdot[2]*logP_2

    #print(logMdot_1)
    #print(logMdot_2)

    plot.scatter(data_set[0],logMdot_1,s=2,label=coeffs_labels[0])
    plot.scatter(data_set[0],logMdot_2,s=2,label=coeffs_labels[1])
    plot.legend()
    plot.xlabel(r'$\log L$')
    plot.xlim(Llimits)
    plot.ylabel(r'$log \dot{M}_{\mathrm{tot.}}$'+ ' (' + coeffs_labels[0] + '),␣
↪(' + coeffs_labels[1] + ')' )
    plot.ylim(Mdotlimits)
    plot.show()
    plot.close()

    plot.scatter(logP_1,logMdot_1,s=2,label=coeffs_labels[0])
    plot.scatter(logP_2,logMdot_2,s=2,label=coeffs_labels[1])
    plot.legend()
    plot.xlabel(r'$\log P$')
    plot.xlim(Plimits)
    plot.ylabel(r'$\log \dot{M}_{\mathrm{tot.}}$'+ ' (' + coeffs_labels[0] +␣
↪'), (' + coeffs_labels[1] + ')' )
    plot.ylim(Mdotlimits)
    plot.show()
    plot.close()

    differences = logMdot_1 - logMdot_2

    diff_vs_logL_fit = linear_fit_1var(data_set[0],differences)
    diff_vs_logL_Coeffs = diff_vs_logL_fit[0]
    print('Difference vs logL fit:
↪','logMdot_1-logMdot_2=',diff_vs_logL_Coeffs[0],'+',diff_vs_logL_Coeffs[1],'logL')
    diff_vs_logL_err = diff_vs_logL_fit[2]
    print('Difference vs logL fit errors:',diff_vs_logL_err[0],',␣
↪',diff_vs_logL_err[1])

    diff_vs_logL_yval = diff_vs_logL_Coeffs[0] + diff_vs_logL_Coeffs[1]*np.
↪asarray(Llimits)
```

```
    ####
    i = 0
    while i < len(constant_mass_arrays):
        subset = constant_mass_arrays[i]
        #print(coeffs_1_PML)
        logP_subset_1 = coeffs_1_PML[0] + coeffs_1_PML[1]*subset[1] +␣
↪coeffs_1_PML[2]*subset[0]
        #print(logP_1)
        #print(coeffs_2_PML)
        logP_subset_2 = coeffs_2_PML[0] + coeffs_2_PML[1]*subset[1] +␣
↪coeffs_2_PML[2]*subset[0]

        logMdot_subset_1 = coeffs_1_LPMdot[0] + coeffs_1_LPMdot[1]*subset[0] +␣
↪coeffs_1_LPMdot[2]*logP_subset_1
        logMdot_subset_2 = coeffs_2_LPMdot[0] + coeffs_2_LPMdot[1]*subset[0] +␣
↪coeffs_2_LPMdot[2]*logP_subset_2

        differences_subset = logMdot_subset_1 - logMdot_subset_2
        if subset[1][0] < 0:
            label_mass = str(10**(subset[1][0]))[0:6] # Take the current mass␣
↪and make string
        else:
            label_mass = str(10**(subset[1][0]))[0:5] # Take the current mass␣
↪and make string
        plot.plot(subset[0],differences_subset,linewidth=1,linestyle =␣
↪'--',label=label_mass)
        #if coeffs_labels[1] == 'C1':
        #    i = i + 3
        #else:
        #    i = i + 5
        i = i + 5
    ####
    #plot.scatter(data_set[0],differences,s=2)
    #plot.plot(np.asarray(Llimits),diff_vs_logL_yval,color='orange')
    plot.xlabel('log L')
    plot.xlim(Llimits)
    #plot.ylabel(r'$\Delta \log \dot{M}_{\mathrm{tot.}}$' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
    plot.ylabel(r'$\log \dot{M}_{\mathrm{tot.}}$'+' (' + coeffs_labels[0] + ')␣
↪' + r'$-\log \dot{M}_{\mathrm{tot.}}$' + ' ('+ coeffs_labels[1] + ')')
    #plot.ylim(deltaMdotlimits)
    plot.legend(title=r'$M$',ncol=2)
    plot.tight_layout(pad=1.2)
    plot.savefig('Final_Fit/
↪'+'deltalogMdot_vs_L_'+coeffs_labels[0]+'_'+coeffs_labels[1]+'.png')
```

```
   plot.savefig('Final_Fit/
↪'+'deltalogMdot_vs_L_'+coeffs_labels[0]+'_'+coeffs_labels[1]+'.pdf')
   plot.show()
   plot.close()

   plot.scatter(logP_1,differences,s=2)
   plot.xlabel('log P'+ ' (' + coeffs_labels[0] + ')')
   plot.xlim(Plimits)
   plot.ylabel(r'$\Delta \log \dot{M}_{\mathrm{tot.}}$' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
   plot.ylim(deltaMdotlimits)
   plot.show()
   plot.close()

   plot.scatter(logP_2,differences,s=2)
   plot.xlabel('log P'+ ' (' + coeffs_labels[1] + ')')
   plot.xlim(Plimits)
   plot.ylabel(r'$\Delta \log \dot{M}_{\mathrm{tot.}}$' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
   plot.ylim(deltaMdotlimits)
   plot.show()
   plot.close()

   plot.scatter(logMdot_1,differences,s=2)
   plot.xlabel(r'$log Mdot_{\mathrm{tot.}}$' + ' (' + coeffs_labels[0] + ')' )
   plot.xlim(Mdotlimits)
   plot.ylabel(r'$\Delta \log \dot{M}_{\mathrm{tot.}}$' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
   plot.ylim(deltaMdotlimits)
   plot.show()
   plot.close()

   plot.scatter(logMdot_1,differences,s=2)
   plot.xlabel(r'$log Mdot_{\mathrm{tot.}}$' + ' (' + coeffs_labels[1] + ')' )
   plot.xlim(Mdotlimits)
   plot.ylabel(r'$\Delta \log \dot{M}_{\mathrm{tot.}}$' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
   plot.ylim(deltaMdotlimits)
   plot.show()
   plot.close()

   # And again without the dust scaling

   # Now, figure out the difference between the mass-loss rates
   logMdot_1 = coeffs_1_LPMdot[0] + coeffs_1_LPMdot[1]*data_set[0] +␣
↪coeffs_1_LPMdot[2]*logP_1
```

```
    logMdot_2 = coeffs_2_LPMdot[0] + coeffs_2_LPMdot[1]*data_set[0] +␣
↪coeffs_2_LPMdot[2]*logP_2
    if coeffs_labels[0][0] == 'M':
        logMdot_1 = logMdot_1-np.log10(ratios_M[1])
    elif coeffs_labels[0][0] == 'C':
        logMdot_1 = logMdot_1-np.log10(ratios_C[1])
    else:
        print('Error. Invalid spectral type encountered.')
    if coeffs_labels[1][0] == 'M':
        logMdot_2 = logMdot_2-np.log10(ratios_M[1])
    elif coeffs_labels[1][0] == 'C':
        logMdot_2 = logMdot_2-np.log10(ratios_C[1])
    else:
        print('Error. Invalid spectral type encountered.')


    plot.scatter(data_set[0],logMdot_1,s=2,label=coeffs_labels[0])
    plot.scatter(data_set[0],logMdot_2,s=2,label=coeffs_labels[1])
    plot.legend()
    plot.xlabel(r'$\log L$')
    plot.xlim(Llimits)
    plot.ylabel(r'$log \dot{M}_{\mathrm{dust}}$'+ ' (' + coeffs_labels[0] + '),␣
↪(' + coeffs_labels[1] + ')' )
    plot.ylim(Mdotdustlimits)
    plot.tight_layout()
    plot.show()
    plot.close()

    plot.scatter(logP_1,logMdot_1,s=2,label=coeffs_labels[0])
    plot.scatter(logP_2,logMdot_2,s=2,label=coeffs_labels[1])
    plot.legend()
    plot.xlabel(r'$\log P$')
    plot.xlim(Plimits)
    plot.ylabel(r'$\log \dot{M}_{\mathrm{dust}}$'+ ' (' + coeffs_labels[0] +␣
↪'), (' + coeffs_labels[1] + ')' )
    plot.ylim(Mdotdustlimits)
    plot.tight_layout()
    plot.show()
    plot.close()

    differences = logMdot_1 - logMdot_2

    diff_vs_logL_fit = linear_fit_1var(data_set[0],differences)
    diff_vs_logL_Coeffs = diff_vs_logL_fit[0]
    print('Difference vs logL fit:
↪','logMdot_1-logMdot_2=',diff_vs_logL_Coeffs[0],'+',diff_vs_logL_Coeffs[1],'logL')
    diff_vs_logL_err = diff_vs_logL_fit[2]
```

```
   print('Difference vs logL fit errors:',diff_vs_logL_err[0],',␣
↪',diff_vs_logL_err[1])

   diff_vs_logL_yval = diff_vs_logL_Coeffs[0] + diff_vs_logL_Coeffs[1]*np.
↪asarray(Llimits)

   ####
   i = 0
   while i < len(constant_mass_arrays):
       subset = constant_mass_arrays[i]
       #print(coeffs_1_PML)
       logP_subset_1 = coeffs_1_PML[0] + coeffs_1_PML[1]*subset[1] +␣
↪coeffs_1_PML[2]*subset[0]
       #print(logP_1)
       #print(coeffs_2_PML)
       logP_subset_2 = coeffs_2_PML[0] + coeffs_2_PML[1]*subset[1] +␣
↪coeffs_2_PML[2]*subset[0]

       logMdot_subset_1 = coeffs_1_LPMdot[0] + coeffs_1_LPMdot[1]*subset[0] +␣
↪coeffs_1_LPMdot[2]*logP_subset_1
       logMdot_subset_2 = coeffs_2_LPMdot[0] + coeffs_2_LPMdot[1]*subset[0] +␣
↪coeffs_2_LPMdot[2]*logP_subset_2
       if coeffs_labels[0][0] == 'M':
           logMdot_subset_1 = logMdot_subset_1-np.log10(ratios_M[1])
       elif coeffs_labels[0][0] == 'C':
           logMdot_subset_1 = logMdot_subset_1-np.log10(ratios_C[1])
       else:
           print('Error. Invalid spectral type encountered.')
       if coeffs_labels[1][0] == 'M':
           logMdot_subset_2 = logMdot_subset_2-np.log10(ratios_M[1])
       elif coeffs_labels[1][0] == 'C':
           logMdot_subset_2 = logMdot_subset_2-np.log10(ratios_C[1])
       else:
           print('Error. Invalid spectral type encountered.')


       differences_subset = logMdot_subset_1 - logMdot_subset_2
       if subset[1][0] < 0:
           label_mass = str(10**(subset[1][0]))[0:6] # Take the current mass␣
↪and make string
       else:
           label_mass = str(10**(subset[1][0]))[0:5] # Take the current mass␣
↪and make string
       plot.plot(subset[0],differences_subset,linewidth=1,linestyle =␣
↪'--',label=label_mass)
       #if coeffs_labels[1] == 'C1':
```

```
        #    i = i + 3
        #else:
        #    i = i + 5
        i = i + 5
    ####
    #plot.scatter(data_set[0],differences,s=2)
    #plot.plot(np.asarray(Llimits),diff_vs_logL_yval,color='orange')
    plot.xlabel('log L')
    plot.xlim(Llimits)
    #plot.ylabel(r'$\Delta \log \dot{M}_{\mathrm{dust}}$' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
    plot.ylabel(r'$\log \dot{M}_{\mathrm{dust}}$'+' (' + coeffs_labels[0] + ')␣
↪' + r'$-\log \dot{M}_{\mathrm{dust}}$' + ' ('+ coeffs_labels[1] + ')')
    #plot.ylim(deltaMdotlimits)
    plot.legend(title=r'$M$',ncol=2)
    plot.tight_layout(pad=1.2)
    plot.savefig('Final_Fit/
↪'+'deltalogMdotdust_vs_L_'+coeffs_labels[0]+'_'+coeffs_labels[1]+'.png')
    plot.savefig('Final_Fit/
↪'+'deltalogMdotdust_vs_L_'+coeffs_labels[0]+'_'+coeffs_labels[1]+'.pdf')
    plot.show()
    plot.close()

    plot.scatter(logP_1,differences,s=2)
    plot.xlabel('log P'+ ' (' + coeffs_labels[0] + ')')
    plot.xlim(Plimits)
    plot.ylabel(r'$\Delta \log \dot{M}_{\mathrm{dust}}$' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
    plot.ylim(deltaMdotlimits)
    plot.tight_layout()
    plot.show()
    plot.close()

    plot.scatter(logP_2,differences,s=2)
    plot.xlabel('log P'+ ' (' + coeffs_labels[1] + ')')
    plot.xlim(Plimits)
    plot.ylabel(r'$\Delta \log \dot{M}_{\mathrm{tot.}}$' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
    plot.ylim(deltaMdotlimits)
    plot.tight_layout()
    plot.show()
    plot.close()

    plot.scatter(logMdot_1,differences,s=2)
    plot.xlabel(r'$log Mdot_{\mathrm{tot.}}$' + ' (' + coeffs_labels[0] + ')' )
    plot.xlim(Mdotlimits)
```

```
    plot.ylabel(r'$\Delta \log \dot{M}_{\mathrm{dust}}$' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
    plot.ylim(deltaMdotlimits)
    plot.tight_layout()
    plot.show()
    plot.close()

    plot.scatter(logMdot_1,differences,s=2)
    plot.xlabel(r'$log Mdot_{\mathrm{tot.}}$' + ' (' + coeffs_labels[1] + ')' )
    plot.xlim(Mdotlimits)
    plot.ylabel(r'$\Delta \log \dot{M}_{\mathrm{dust}}$' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
    plot.ylim(deltaMdotlimits)
    plot.tight_layout()
    plot.show()
    plot.close()



    '''
    # Figure out average difference at a given luminosity
    working_array = np.transpose(np.vstack((data_set[0],differences)))
    i = 0
    luminosity_pos_array = []
    luminosity_pos_array_all = []
    # Loop finds all of the indices where luminosities match, and saves them in␣
↪arrays
    while i < len(working_array):
        current_L = working_array[i][0]
        if i == len(working_array)-1:
            next_L = 0 # reset
        else:
            next_L = working_array[i+1][0]
        if i == len(working_array):
            # Handles the final value
            luminosity_pos_array.append(i)
            luminosity_pos_array_all.append(luminosity_pos_array)
            luminosity_pos_array = [] # reset after saving
        elif current_L == next_L:
            luminosity_pos_array.append(i)
        else:
            luminosity_pos_array.append(i) # Last position of current␣
↪luminosity needs to be appended
            luminosity_pos_array_all.append(luminosity_pos_array)
            luminosity_pos_array = [] # reset after saving
        i += 1
    #print(luminosity_pos_array_all)
```

```
    # Now, we need to average the Mdots at each luminosity
    i = 0
    average_differences = []
    for i in range(0,len(luminosity_pos_array_all)):
        differences_sum = 0
        for k in range(0,len(luminosity_pos_array_all[i])):
            differences_sum = differences_sum +␣
↪differences[luminosity_pos_array_all[i][k]]
        differences_mean = differences_sum/len(luminosity_pos_array_all[i])
        average_differences.append(differences_mean)
    unique_luminosities = []
    for i in range(0, len(luminosity_pos_array_all)):
        unique_luminosities.append(data_set[0][luminosity_pos_array_all[i][0]])
    #print(len(average_differences))
    #print(len(unique_luminosities))
    # And now plot
    fit_line = chi_sq_fit_2d(unique_luminosities,average_differences)[0]
    left_point = np.nanmin(unique_luminosities)
    right_point = np.nanmax(unique_luminosities)
    fit_x = np.asarray([left_point,right_point])
    fit_y = fit_line[0]+fit_line[1]*fit_x
    plot.scatter(unique_luminosities, average_differences,s=2)
    if fit_line[1]<0:
        fit_label = str(fit_line[0]) + str(fit_line[1])+'logL'
    else:
        fit_label = str(fit_line[0]) + '+' + str(fit_line[1])+'logL'
    plot.plot(fit_x, fit_y,color = 'black',label=fit_label)
    plot.xlabel('log L')
    plot.ylabel('<log Mdot - log Mdot>' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
    plot.legend()
    plot.show()
    plot.close()
    '''

    '''# Figure out average difference at a given period
    working_array = np.transpose(np.vstack((data_set[1],differences)))
    i = 0
    print(data_set[1])
    i = 0
    average_periods = []
    for i in range(0,len(luminosity_pos_array_all)):
        sum_P = 0
        for k in range(0,len(luminosity_pos_array_all[i])):
            sum_P = sum_P + data_set[1][luminosity_pos_array_all[i][k]]
        average_P = sum_P/len(luminosity_pos_array_all[i])
        average_periods.append(average_P)
```

```
    #print(len(average_differences))
    #print(len(average_periods))
    # Now plot
    fit_line = chi_sq_fit_2d(average_periods,average_differences)[0]
    left_point = np.nanmin(average_periods)
    right_point = np.nanmax(average_periods)
    fit_x = np.asarray([left_point,right_point])
    fit_y = fit_line[0]+fit_line[1]*fit_x
    plot.scatter(average_periods, average_differences,s=2)
    fit_label = str(fit_line[0]) + str(fit_line[1])+'<logP>'
    plot.plot(fit_x, fit_y,color = 'black',label=fit_label)
    plot.xlabel('log P')
    plot.ylabel('<log Mdot - log Mdot>' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
    plot.legend()
    plot.show()
    plot.close()
     '''
```

[371]:
```
def calculate_formula_correction_MdotP (coeffs_1, coeffs_2,␣
↪data_set,coeffs_labels = ['',''],grid_spacing=50,Mdotlimits=[-8,␣
↪-4],Mdotdustlimits=[-12,-8],Llimits=[2.8, 4.6],Plimits=[1.2, 3.
↪4],deltaMdotlimits=[-7, 7]):
    coeffs_1_LPMdot = coeffs_1[0]
    coeffs_1_PML = coeffs_1[1]
    coeffs_2_LPMdot = coeffs_2[0]
    coeffs_2_PML = coeffs_2[1]


    # create data sets of constant mass
    checked_log_m_array = []
    working_log_m = 0
    proceed = False
    constant_mass_lists = []
    i = 0
    while i < len(data_set[1]):
        indices_of_current_mass = []
        if i == 0:
            checked_log_m_array.append(data_set[1][0])
            working_log_m = data_set[1][0]
            for k in range(0, len(data_set[1])):
                if working_log_m == data_set[1][k]:
                    indices_of_current_mass.append(k)
            constant_mass_lists.append(indices_of_current_mass)
        else:
            working_log_m = data_set[1][i]
            for k in range(0, len(checked_log_m_array)):
```

```
                if working_log_m == checked_log_m_array[k]:
                    proceed = False
                    break
                else:
                    proceed = True
            if proceed == True:
                for l in range(0, len(data_set[1])):
                    checked_log_m_array.append(data_set[1][i])
                    if working_log_m == data_set[1][l]:
                        indices_of_current_mass.append(l)
                constant_mass_lists.append(indices_of_current_mass)
        #print('i:',i)
        i += 1
    #print('constant_mass_lists:',constant_mass_lists)
    i = 0
    constant_mass_arrays = []
    for i in range(0, len(constant_mass_lists)):
        constant_mass_arrays.append(np.transpose(np.
↪transpose(data_set)[constant_mass_lists[i]]))
    #print(constant_mass_arrays)


    # Calculate pulsation periods
    #print(coeffs_1_PML)
    logP_1 = coeffs_1_PML[0] + coeffs_1_PML[1]*data_set[1] +␣
↪coeffs_1_PML[2]*data_set[0]
    #print(logP_1)
    #print(coeffs_2_PML)
    logP_2 = coeffs_2_PML[0] + coeffs_2_PML[1]*data_set[1] +␣
↪coeffs_2_PML[2]*data_set[0]
    #print(logP_2)

    # some testing plots
    plot.scatter(data_set[0],logP_1,s=2,label=coeffs_labels[0])
    plot.scatter(data_set[0],logP_2,s=2,label=coeffs_labels[1])
    plot.legend()
    plot.xlabel(r'$\log L$')
    plot.xlim(Llimits)
    plot.ylabel(r'$\log P$')
    plot.ylim(Plimits)
    plot.show()
    plot.close()

    # Now, figure out the difference between the mass-loss rates
    logMdot_1 = coeffs_1_LPMdot[0] + coeffs_1_LPMdot[1]*data_set[0] +␣
↪coeffs_1_LPMdot[2]*logP_1
```

```
   logMdot_2 = coeffs_2_LPMdot[0] + coeffs_2_LPMdot[1]*data_set[0] +␣
↪coeffs_2_LPMdot[2]*logP_2

   #print(logMdot_1)
   #print(logMdot_2)

   plot.scatter(data_set[0],logMdot_1,s=2,label=coeffs_labels[0])
   plot.scatter(data_set[0],logMdot_2,s=2,label=coeffs_labels[1])
   plot.legend()
   plot.xlabel(r'$\log L$')
   plot.xlim(Llimits)
   plot.ylabel(r'$log \dot{M}_{\mathrm{tot.}}$'+ ' (' + coeffs_labels[0] + '),␣
↪(' + coeffs_labels[1] + ')' )
   plot.ylim(Mdotlimits)
   plot.show()
   plot.close()

   plot.scatter(logP_1,logMdot_1,s=2,label=coeffs_labels[0])
   plot.scatter(logP_2,logMdot_2,s=2,label=coeffs_labels[1])
   plot.legend()
   plot.xlabel(r'$\log P$')
   plot.xlim(Plimits)
   plot.ylabel(r'$\log \dot{M}_{\mathrm{tot.}}$'+ ' (' + coeffs_labels[0] +␣
↪'), (' + coeffs_labels[1] + ')' )
   plot.ylim(Mdotlimits)
   plot.show()
   plot.close()

   differences = logMdot_1 - logMdot_2

   diff_vs_logL_fit = linear_fit_1var(data_set[0],differences)
   diff_vs_logL_Coeffs = diff_vs_logL_fit[0]
   print('Difference vs logL fit:
↪','logMdot_1-logMdot_2=',diff_vs_logL_Coeffs[0],'+',diff_vs_logL_Coeffs[1],'logL')
   diff_vs_logL_err = diff_vs_logL_fit[2]
   print('Difference vs logL fit errors:',diff_vs_logL_err[0],',␣
↪',diff_vs_logL_err[1])

   diff_vs_logL_yval = diff_vs_logL_Coeffs[0] + diff_vs_logL_Coeffs[1]*np.
↪asarray(Llimits)

   ####
   i = 0
   while i < len(constant_mass_arrays):
       subset = constant_mass_arrays[i]
       #print(coeffs_1_PML)
```

```
        logP_subset_1 = coeffs_1_PML[0] + coeffs_1_PML[1]*subset[1] +␣
↪coeffs_1_PML[2]*subset[0]
        #print(logP_1)
        #print(coeffs_2_PML)
        logP_subset_2 = coeffs_2_PML[0] + coeffs_2_PML[1]*subset[1] +␣
↪coeffs_2_PML[2]*subset[0]

        logMdot_subset_1 = coeffs_1_LPMdot[0] + coeffs_1_LPMdot[1]*subset[0] +␣
↪coeffs_1_LPMdot[2]*logP_subset_1
        logMdot_subset_2 = coeffs_2_LPMdot[0] + coeffs_2_LPMdot[1]*subset[0] +␣
↪coeffs_2_LPMdot[2]*logP_subset_2

        differences_subset = logMdot_subset_1 - logMdot_subset_2
        if subset[1][0] < 0:
            label_mass = str(10**(subset[1][0]))[0:6] # Take the current mass␣
↪and make string
        else:
            label_mass = str(10**(subset[1][0]))[0:5] # Take the current mass␣
↪and make string
        plot.plot(subset[0],differences_subset,linewidth=1,linestyle =␣
↪'--',label=label_mass)
        #if coeffs_labels[1] == 'C1':
        #    i = i + 3
        #else:
        #    i = i + 5
        i = i + 5
    ####
    #plot.scatter(data_set[0],differences,s=2)
    #plot.plot(np.asarray(Llimits),diff_vs_logL_yval,color='orange')
    plot.xlabel(r'$\log L$')
    plot.xlim(Llimits)
    #plot.ylabel(r'$\Delta \log \dot{M}_{\mathrm{tot.}}$' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
    plot.ylabel(r'$\log \dot{M}_{\mathrm{tot.}}$'+ ' (' + coeffs_labels[0] + ')␣
↪' + r'$-\log \dot{M}_{\mathrm{tot.}}$' + ' ('+ coeffs_labels[1] + ')')
    #plot.ylim(deltaMdotlimits)
    plot.legend(title=r'$M$',ncol=2)
    plot.tight_layout(pad=1.2)
    plot.savefig('Final_Fit/
↪'+'LMdotP_'+'deltalogMdot_vs_L_'+coeffs_labels[0]+'_'+coeffs_labels[1]+'.
↪png')
    plot.savefig('Final_Fit/
↪'+'LMdotP_'+'deltalogMdot_vs_L_'+coeffs_labels[0]+'_'+coeffs_labels[1]+'.
↪pdf')
    plot.show()
    plot.close()
```

```
   plot.scatter(logP_1,differences,s=2)
   plot.xlabel('log P'+ ' (' + coeffs_labels[0] + ')')
   plot.xlim(Plimits)
   plot.ylabel(r'$\Delta \log \dot{M}_{\mathrm{tot.}}$' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
   plot.ylim(deltaMdotlimits)
   plot.show()
   plot.close()

   plot.scatter(logP_2,differences,s=2)
   plot.xlabel('log P'+ ' (' + coeffs_labels[1] + ')')
   plot.xlim(Plimits)
   plot.ylabel(r'$\Delta \log \dot{M}_{\mathrm{tot.}}$' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
   plot.ylim(deltaMdotlimits)
   plot.show()
   plot.close()

   plot.scatter(logMdot_1,differences,s=2)
   plot.xlabel(r'$log Mdot_{\mathrm{tot.}}$' + ' (' + coeffs_labels[0] + ')' )
   plot.xlim(Mdotlimits)
   plot.ylabel(r'$\Delta \log \dot{M}_{\mathrm{tot.}}$' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
   plot.ylim(deltaMdotlimits)
   plot.show()
   plot.close()

   plot.scatter(logMdot_1,differences,s=2)
   plot.xlabel(r'$log Mdot_{\mathrm{tot.}}$' + ' (' + coeffs_labels[1] + ')' )
   plot.xlim(Mdotlimits)
   plot.ylabel(r'$\Delta \log \dot{M}_{\mathrm{tot.}}$' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
   plot.ylim(deltaMdotlimits)
   plot.show()
   plot.close()

   # And again without the dust scaling

   # Now, figure out the difference between the mass-loss rates
   logMdot_1 = coeffs_1_LPMdot[0] + coeffs_1_LPMdot[1]*data_set[0] +␣
↪coeffs_1_LPMdot[2]*logP_1
   logMdot_2 = coeffs_2_LPMdot[0] + coeffs_2_LPMdot[1]*data_set[0] +␣
↪coeffs_2_LPMdot[2]*logP_2
   if coeffs_labels[0][0] == 'M':
       logMdot_1 = logMdot_1-np.log10(ratios_M[1])
```

```
    elif coeffs_labels[0][0] == 'C':
        logMdot_1 = logMdot_1-np.log10(ratios_C[1])
    else:
        print('Error. Invalid spectral type encountered.')
    if coeffs_labels[1][0] == 'M':
        logMdot_2 = logMdot_2-np.log10(ratios_M[1])
    elif coeffs_labels[1][0] == 'C':
        logMdot_2 = logMdot_2-np.log10(ratios_C[1])
    else:
        print('Error. Invalid spectral type encountered.')


    plot.scatter(data_set[0],logMdot_1,s=2,label=coeffs_labels[0])
    plot.scatter(data_set[0],logMdot_2,s=2,label=coeffs_labels[1])
    plot.legend()
    plot.xlabel(r'$\log L$')
    plot.xlim(Llimits)
    plot.ylabel(r'$log \dot{M}_{\mathrm{dust}}$'+ ' (' + coeffs_labels[0] + '),␣
↪(' + coeffs_labels[1] + ')' )
    plot.ylim(Mdotdustlimits)
    plot.tight_layout()
    plot.show()
    plot.close()

    plot.scatter(logP_1,logMdot_1,s=2,label=coeffs_labels[0])
    plot.scatter(logP_2,logMdot_2,s=2,label=coeffs_labels[1])
    plot.legend()
    plot.xlabel(r'$\log P$')
    plot.xlim(Plimits)
    plot.ylabel(r'$\log \dot{M}_{\mathrm{dust}}$'+ ' (' + coeffs_labels[0] +␣
↪'), (' + coeffs_labels[1] + ')' )
    plot.ylim(Mdotdustlimits)
    plot.tight_layout()
    plot.show()
    plot.close()

    differences = logMdot_1 - logMdot_2

    diff_vs_logL_fit = linear_fit_1var(data_set[0],differences)
    diff_vs_logL_Coeffs = diff_vs_logL_fit[0]
    print('Difference vs logL fit:
↪','logMdot_1-logMdot_2=',diff_vs_logL_Coeffs[0],'+',diff_vs_logL_Coeffs[1],'logL')
    diff_vs_logL_err = diff_vs_logL_fit[2]
    print('Difference vs logL fit errors:',diff_vs_logL_err[0],',␣
↪',diff_vs_logL_err[1])
```

```
   diff_vs_logL_yval = diff_vs_logL_Coeffs[0] + diff_vs_logL_Coeffs[1]*np.
↪asarray(Llimits)

   ####
   i = 0
   while i < len(constant_mass_arrays):
       subset = constant_mass_arrays[i]
       #print(coeffs_1_PML)
       logP_subset_1 = coeffs_1_PML[0] + coeffs_1_PML[1]*subset[1] +␣
↪coeffs_1_PML[2]*subset[0]
       #print(logP_1)
       #print(coeffs_2_PML)
       logP_subset_2 = coeffs_2_PML[0] + coeffs_2_PML[1]*subset[1] +␣
↪coeffs_2_PML[2]*subset[0]

       logMdot_subset_1 = coeffs_1_LPMdot[0] + coeffs_1_LPMdot[1]*subset[0] +␣
↪coeffs_1_LPMdot[2]*logP_subset_1
       logMdot_subset_2 = coeffs_2_LPMdot[0] + coeffs_2_LPMdot[1]*subset[0] +␣
↪coeffs_2_LPMdot[2]*logP_subset_2
       if coeffs_labels[0][0] == 'M':
           logMdot_subset_1 = logMdot_subset_1-np.log10(ratios_M[1])
       elif coeffs_labels[0][0] == 'C':
           logMdot_subset_1 = logMdot_subset_1-np.log10(ratios_C[1])
       else:
           print('Error. Invalid spectral type encountered.')
       if coeffs_labels[1][0] == 'M':
           logMdot_subset_2 = logMdot_subset_2-np.log10(ratios_M[1])
       elif coeffs_labels[1][0] == 'C':
           logMdot_subset_2 = logMdot_subset_2-np.log10(ratios_C[1])
       else:
           print('Error. Invalid spectral type encountered.')


       differences_subset = logMdot_subset_1 - logMdot_subset_2
       if subset[1][0] < 0:
           label_mass = str(10**(subset[1][0]))[0:6] # Take the current mass␣
↪and make string
       else:
           label_mass = str(10**(subset[1][0]))[0:5] # Take the current mass␣
↪and make string
       plot.plot(subset[0],differences_subset,linewidth=1,linestyle =␣
↪'--',label=label_mass)
       #if coeffs_labels[1] == 'C1':
       #    i = i + 3
       #else:
       #    i = i + 5
```

```
        i = i + 5
    ####
    #plot.scatter(data_set[0],differences,s=2)
    #plot.plot(np.asarray(Llimits),diff_vs_logL_yval,color='orange')
    plot.xlabel(r'$\log L$')
    plot.xlim(Llimits)
    #plot.ylabel(r'$\Delta \log \dot{M}_{\mathrm{dust}}$' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
    plot.ylabel(r'$\log \dot{M}_{\mathrm{dust}}$'+' (' + coeffs_labels[0] + ')␣
↪' + r'$-\log \dot{M}_{\mathrm{dust}}$' + ' ('+ coeffs_labels[1] + ')')
    #plot.ylim(deltaMdotlimits)
    plot.legend(title=r'$M$',ncol=2)
    plot.tight_layout(pad=1.2)
    plot.savefig('Final_Fit/
↪'+'LMdotP_'+'deltalogMdotdust_vs_L_'+coeffs_labels[0]+'_'+coeffs_labels[1]+'.
↪png')
    plot.savefig('Final_Fit/
↪'+'LMdotP_'+'deltalogMdotdust_vs_L_'+coeffs_labels[0]+'_'+coeffs_labels[1]+'.
↪pdf')
    plot.show()
    plot.close()

    plot.scatter(logP_1,differences,s=2)
    plot.xlabel('log P'+ ' (' + coeffs_labels[0] + ')')
    plot.xlim(Plimits)
    plot.ylabel(r'$\Delta \log \dot{M}_{\mathrm{dust}}$' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
    plot.ylim(deltaMdotlimits)
    plot.tight_layout()
    plot.show()
    plot.close()

    plot.scatter(logP_2,differences,s=2)
    plot.xlabel('log P'+ ' (' + coeffs_labels[1] + ')')
    plot.xlim(Plimits)
    plot.ylabel(r'$\Delta \log \dot{M}_{\mathrm{tot.}}$' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
    plot.ylim(deltaMdotlimits)
    plot.tight_layout()
    plot.show()
    plot.close()

    plot.scatter(logMdot_1,differences,s=2)
    plot.xlabel(r'$log Mdot_{\mathrm{tot.}}$' + ' (' + coeffs_labels[0] + ')' )
    plot.xlim(Mdotlimits)
```

```
    plot.ylabel(r'$\Delta \log \dot{M}_{\mathrm{dust}}$' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
    plot.ylim(deltaMdotlimits)
    plot.tight_layout()
    plot.show()
    plot.close()

    plot.scatter(logMdot_1,differences,s=2)
    plot.xlabel(r'$log Mdot_{\mathrm{tot.}}$' + ' (' + coeffs_labels[1] + ')' )
    plot.xlim(Mdotlimits)
    plot.ylabel(r'$\Delta \log \dot{M}_{\mathrm{dust}}$' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
    plot.ylim(deltaMdotlimits)
    plot.tight_layout()
    plot.show()
    plot.close()



    '''
    # Figure out average difference at a given luminosity
    working_array = np.transpose(np.vstack((data_set[0],differences)))
    i = 0
    luminosity_pos_array = []
    luminosity_pos_array_all = []
    # Loop finds all of the indices where luminosities match, and saves them in␣
↪arrays
    while i < len(working_array):
        current_L = working_array[i][0]
        if i == len(working_array)-1:
            next_L = 0 # reset
        else:
            next_L = working_array[i+1][0]
        if i == len(working_array):
            # Handles the final value
            luminosity_pos_array.append(i)
            luminosity_pos_array_all.append(luminosity_pos_array)
            luminosity_pos_array = [] # reset after saving
        elif current_L == next_L:
            luminosity_pos_array.append(i)
        else:
            luminosity_pos_array.append(i) # Last position of current␣
↪luminosity needs to be appended
            luminosity_pos_array_all.append(luminosity_pos_array)
            luminosity_pos_array = [] # reset after saving
        i += 1
    #print(luminosity_pos_array_all)
```

```
    # Now, we need to average the Mdots at each luminosity
    i = 0
    average_differences = []
    for i in range(0,len(luminosity_pos_array_all)):
        differences_sum = 0
        for k in range(0,len(luminosity_pos_array_all[i])):
            differences_sum = differences_sum +␣
↪differences[luminosity_pos_array_all[i][k]]
        differences_mean = differences_sum/len(luminosity_pos_array_all[i])
        average_differences.append(differences_mean)
    unique_luminosities = []
    for i in range(0, len(luminosity_pos_array_all)):
        unique_luminosities.append(data_set[0][luminosity_pos_array_all[i][0]])
    #print(len(average_differences))
    #print(len(unique_luminosities))
    # And now plot
    fit_line = chi_sq_fit_2d(unique_luminosities,average_differences)[0]
    left_point = np.nanmin(unique_luminosities)
    right_point = np.nanmax(unique_luminosities)
    fit_x = np.asarray([left_point,right_point])
    fit_y = fit_line[0]+fit_line[1]*fit_x
    plot.scatter(unique_luminosities, average_differences,s=2)
    if fit_line[1]<0:
        fit_label = str(fit_line[0]) + str(fit_line[1])+'logL'
    else:
        fit_label = str(fit_line[0]) + '+' + str(fit_line[1])+'logL'
    plot.plot(fit_x, fit_y,color = 'black',label=fit_label)
    plot.xlabel('log L')
    plot.ylabel('<log Mdot - log Mdot>' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
    plot.legend()
    plot.show()
    plot.close()
    '''

    '''# Figure out average difference at a given period
    working_array = np.transpose(np.vstack((data_set[1],differences)))
    i = 0
    print(data_set[1])
    i = 0
    average_periods = []
    for i in range(0,len(luminosity_pos_array_all)):
        sum_P = 0
        for k in range(0,len(luminosity_pos_array_all[i])):
            sum_P = sum_P + data_set[1][luminosity_pos_array_all[i][k]]
        average_P = sum_P/len(luminosity_pos_array_all[i])
        average_periods.append(average_P)
```

```
    #print(len(average_differences))
    #print(len(average_periods))
    # Now plot
    fit_line = chi_sq_fit_2d(average_periods,average_differences)[0]
    left_point = np.nanmin(average_periods)
    right_point = np.nanmax(average_periods)
    fit_x = np.asarray([left_point,right_point])
    fit_y = fit_line[0]+fit_line[1]*fit_x
    plot.scatter(average_periods, average_differences,s=2)
    fit_label = str(fit_line[0]) + str(fit_line[1])+'<logP>'
    plot.plot(fit_x, fit_y,color = 'black',label=fit_label)
    plot.xlabel('log P')
    plot.ylabel('<log Mdot - log Mdot>' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
    plot.legend()
    plot.show()
    plot.close()
     '''
```

[372]:
```
def calculate_formula_correction_dz (coeffs_1, coeffs_2, data_set,coeffs_labels␣
↪= ['',''],grid_spacing=50,Mdotlimits=[-8, -4],
                                     Mdotdustlimits=[-12,-8],Llimits=[2.8, 4.
↪6],Plimits=[1.2, 3.4],deltaMdotlimits=[-2.5, 3.5]):
    coeffs_1_LMMdot = coeffs_1[0]
    coeffs_1_PML = coeffs_1[1]
    coeffs_2_LMMdot = coeffs_2[0]
    coeffs_2_PML = coeffs_2[1]


    # create data sets of constant mass
    checked_log_m_array = []
    working_log_m = 0
    proceed = False
    constant_mass_lists = []
    i = 0
    while i < len(data_set[1]):
        indices_of_current_mass = []
        if i == 0:
            checked_log_m_array.append(data_set[1][0])
            working_log_m = data_set[1][0]
            for k in range(0, len(data_set[1])):
                if working_log_m == data_set[1][k]:
                    indices_of_current_mass.append(k)
            constant_mass_lists.append(indices_of_current_mass)
        else:
            working_log_m = data_set[1][i]
            for k in range(0, len(checked_log_m_array)):
```

```
                if working_log_m == checked_log_m_array[k]:
                    proceed = False
                    break
                else:
                    proceed = True
            if proceed == True:
                for l in range(0, len(data_set[1])):
                    checked_log_m_array.append(data_set[1][i])
                    if working_log_m == data_set[1][l]:
                        indices_of_current_mass.append(l)
                constant_mass_lists.append(indices_of_current_mass)
        #print('i:',i)
        i += 1
    #print('constant_mass_lists:',constant_mass_lists)
    i = 0
    constant_mass_arrays = []
    for i in range(0, len(constant_mass_lists)):
        constant_mass_arrays.append(np.transpose(np.
↪transpose(data_set)[constant_mass_lists[i]]))
    #print(constant_mass_arrays)


    # Calculate pulsation periods
    #print('coeffs_1_PML',coeffs_1_PML)
    logP_1 = coeffs_1_PML[0] + coeffs_1_PML[1]*data_set[1] +␣
↪coeffs_1_PML[2]*data_set[0]
    #print('logP_1',logP_1)
    #print('coeffs_2_PML',coeffs_2_PML)
    logP_2 = coeffs_2_PML[0] + coeffs_2_PML[1]*data_set[1] +␣
↪coeffs_2_PML[2]*data_set[0]
    #print('logP_2',logP_2)

    # some testing plots
    plot.scatter(data_set[0],logP_1,s=2,label=coeffs_labels[0])
    plot.scatter(data_set[0],logP_2,s=2,label=coeffs_labels[1])
    plot.legend()
    plot.xlabel('log L')
    plot.xlim(Llimits)
    plot.ylabel('log P')
    plot.ylim(Plimits)
    plot.show()
    plot.close()

    # Now, figure out the difference between the mass-loss rates
    logMdot_1 = coeffs_1_LMMdot[0] + coeffs_1_LMMdot[1]*data_set[0] -␣
↪coeffs_1_LMMdot[2]*data_set[1]
```

```
   logMdot_2 = coeffs_2_LMMdot[0] + coeffs_2_LMMdot[1]*data_set[0] -␣
↪coeffs_2_LMMdot[2]*data_set[1]

   differences = logMdot_1 - logMdot_2


   #print(logMdot_1)
   #print(logMdot_2)

   plot.scatter(data_set[0],logMdot_1,s=2,label=coeffs_labels[0])
   plot.scatter(data_set[0],logMdot_2,s=2,label=coeffs_labels[1])
   plot.legend()
   plot.xlabel(r'$\log L$')
   plot.xlim(Llimits)
   plot.ylabel(r'$log \dot{M}_{\mathrm{tot.}}$'+ ' (' + coeffs_labels[0] + '),␣
↪(' + coeffs_labels[1] + ')' )
   plot.ylim(Mdotlimits)
   plot.show()
   plot.close()

   plot.scatter(logP_1,logMdot_1,s=2,label=coeffs_labels[0])
   plot.scatter(logP_2,logMdot_2,s=2,label=coeffs_labels[1])
   plot.legend()
   plot.xlabel(r'$\log P$')
   plot.xlim(Plimits)
   plot.ylabel(r'$\log \dot{M}_{\mathrm{tot.}}$'+ ' (' + coeffs_labels[0] +␣
↪'), (' + coeffs_labels[1] + ')' )
   plot.ylim(Mdotlimits)
   plot.show()
   plot.close()

   differences = logMdot_1 - logMdot_2

   diff_vs_logL_fit = linear_fit_1var(data_set[0],differences)
   diff_vs_logL_Coeffs = diff_vs_logL_fit[0]
   print('Difference vs logL fit:
↪','logMdot_1-logMdot_2=',diff_vs_logL_Coeffs[0],'+',diff_vs_logL_Coeffs[1],'logL')
   diff_vs_logL_err = diff_vs_logL_fit[2]
   print('Difference vs logL fit errors:',diff_vs_logL_err[0],',␣
↪',diff_vs_logL_err[1])

   diff_vs_logL_yval = diff_vs_logL_Coeffs[0] + diff_vs_logL_Coeffs[1]*np.
↪asarray(Llimits)

   ####
   i = 0
```

```
    while i < len(constant_mass_arrays):
        subset = constant_mass_arrays[i]

        logMdot_subset_1 = coeffs_1_LMMdot[0] + coeffs_1_LMMdot[1]*subset[0] +␣
↪coeffs_1_LMMdot[2]*subset[1]
        logMdot_subset_2 = coeffs_2_LMMdot[0] + coeffs_2_LMMdot[1]*subset[0] +␣
↪coeffs_2_LMMdot[2]*subset[1]

        differences_subset = logMdot_subset_1 - logMdot_subset_2
        if subset[1][0] < 0:
            label_mass = str(10**(subset[1][0]))[0:6] # Take the current mass␣
↪and make string
        else:
            label_mass = str(10**(subset[1][0]))[0:5] # Take the current mass␣
↪and make string
        plot.plot(subset[0],differences_subset,linewidth=1,linestyle =␣
↪'--',label=label_mass)
        #if coeffs_labels[1] == 'C1':
        #    i = i + 3
        #else:
        #    i = i + 5
        i = i + 5
    ####
    #plot.scatter(data_set[0],differences,s=2)
    #plot.plot(np.asarray(Llimits),diff_vs_logL_yval,color='orange')
    plot.xlabel('log L')
    plot.xlim(Llimits)
    #plot.ylabel(r'$\Delta \log \dot{M}_{\mathrm{tot.}}$' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
    plot.ylabel(r'$\log \dot{M}_{\mathrm{tot.}}$'+' (' + coeffs_labels[0] + ')␣
↪' + r'$-\log \dot{M}_{\mathrm{tot.}}$' + ' ('+ coeffs_labels[1] + ')')

    #plot.ylim(deltaMdotlimits)
    plot.legend(title=r'$M$',ncol=2)
    plot.tight_layout(pad=1.2)
    plot.savefig('Final_Fit/
↪'+'dz_'+'deltalogMdot_vs_L_'+coeffs_labels[0]+'_'+coeffs_labels[1]+'.png')
    plot.savefig('Final_Fit/
↪'+'dz_'+'deltalogMdot_vs_L_'+coeffs_labels[0]+'_'+coeffs_labels[1]+'.pdf')
    plot.show()
    plot.close()

    plot.scatter(logP_1,differences,s=2)
    plot.xlabel('log P'+ ' (' + coeffs_labels[0] + ')')
    plot.xlim(Plimits)
```

```
    plot.ylabel(r'$\Delta \log \dot{M}_{\mathrm{tot.}}$' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
    plot.ylim(deltaMdotlimits)
    plot.show()
    plot.close()

    plot.scatter(logP_2,differences,s=2)
    plot.xlabel('log P'+ ' (' + coeffs_labels[1] + ')')
    plot.xlim(Plimits)
    plot.ylabel(r'$\Delta \log \dot{M}_{\mathrm{tot.}}$' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
    plot.ylim(deltaMdotlimits)
    plot.show()
    plot.close()

    plot.scatter(logMdot_1,differences,s=2)
    plot.xlabel(r'$log Mdot_{\mathrm{tot.}}$' + ' (' + coeffs_labels[0] + ')' )
    plot.xlim(Mdotlimits)
    plot.ylabel(r'$\Delta \log \dot{M}_{\mathrm{tot.}}$' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
    plot.ylim(deltaMdotlimits)
    plot.show()
    plot.close()

    plot.scatter(logMdot_1,differences,s=2)
    plot.xlabel(r'$log Mdot_{\mathrm{tot.}}$' + ' (' + coeffs_labels[1] + ')' )
    plot.xlim(Mdotlimits)
    plot.ylabel(r'$\Delta \log \dot{M}_{\mathrm{tot.}}$' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
    plot.ylim(deltaMdotlimits)
    plot.show()
    plot.close()

    # And again without the dust scaling


    logMdot_1 = coeffs_1_LMMdot[0] + coeffs_1_LMMdot[1]*data_set[0] -␣
↪coeffs_1_LMMdot[2]*data_set[1]
    logMdot_2 = coeffs_2_LMMdot[0] + coeffs_2_LMMdot[1]*data_set[0] -␣
↪coeffs_2_LMMdot[2]*data_set[1]
    if coeffs_labels[0][0] == 'M':
        logMdot_1 = logMdot_1-np.log10(ratios_M[1])
    elif coeffs_labels[0][0] == 'C':
        logMdot_1 = logMdot_1-np.log10(ratios_C[1])
    else:
        print('Error. Invalid spectral type encountered.')
```

```
    if coeffs_labels[1][0] == 'M':
        logMdot_2 = logMdot_2-np.log10(ratios_M[1])
    elif coeffs_labels[1][0] == 'C':
        logMdot_2 = logMdot_2-np.log10(ratios_C[1])
    else:
        print('Error. Invalid spectral type encountered.')

    plot.scatter(data_set[0],logMdot_1,s=2,label=coeffs_labels[0])
    plot.scatter(data_set[0],logMdot_2,s=2,label=coeffs_labels[1])
    plot.legend()
    plot.xlabel(r'$\log L$')
    plot.xlim(Llimits)
    plot.ylabel(r'$log \dot{M}_{\mathrm{dust}}$'+ ' (' + coeffs_labels[0] + '),␣
↪(' + coeffs_labels[1] + ')' )
    plot.ylim(Mdotdustlimits)
    plot.tight_layout()
    plot.show()
    plot.close()

    plot.scatter(logP_1,logMdot_1,s=2,label=coeffs_labels[0])
    plot.scatter(logP_2,logMdot_2,s=2,label=coeffs_labels[1])
    plot.legend()
    plot.xlabel(r'$\log P$')
    plot.xlim(Plimits)
    plot.ylabel(r'$\log \dot{M}_{\mathrm{dust}}$'+ ' (' + coeffs_labels[0] +␣
↪'), (' + coeffs_labels[1] + ')' )
    plot.ylim(Mdotdustlimits)
    plot.tight_layout()
    plot.show()
    plot.close()

    differences = logMdot_1 - logMdot_2

    diff_vs_logL_fit = linear_fit_1var(data_set[0],differences)
    diff_vs_logL_Coeffs = diff_vs_logL_fit[0]
    print('Difference vs logL fit:
↪','logMdot_1-logMdot_2=',diff_vs_logL_Coeffs[0],'+',diff_vs_logL_Coeffs[1],'logL')
    diff_vs_logL_err = diff_vs_logL_fit[2]
    print('Difference vs logL fit errors:',diff_vs_logL_err[0],',␣
↪',diff_vs_logL_err[1])

    diff_vs_logL_yval = diff_vs_logL_Coeffs[0] + diff_vs_logL_Coeffs[1]*np.
↪asarray(Llimits)

    ####
    i = 0
    while i < len(constant_mass_arrays):
```

```
        subset = constant_mass_arrays[i]

        logMdot_subset_1 = coeffs_1_LMMdot[0] + coeffs_1_LMMdot[1]*subset[0] -␣
↪coeffs_1_LMMdot[2]*subset[1]
        logMdot_subset_2 = coeffs_2_LMMdot[0] + coeffs_2_LMMdot[1]*subset[0] -␣
↪coeffs_2_LMMdot[2]*subset[1]
        if coeffs_labels[0][0] == 'M':
            logMdot_subset_1 = logMdot_subset_1-np.log10(ratios_M[1])
        elif coeffs_labels[0][0] == 'C':
            logMdot_subset_1 = logMdot_subset_1-np.log10(ratios_C[1])
        else:
            print('Error. Invalid spectral type encountered.')
        if coeffs_labels[1][0] == 'M':
            logMdot_subset_2 = logMdot_subset_2-np.log10(ratios_M[1])
        elif coeffs_labels[1][0] == 'C':
            logMdot_subset_2 = logMdot_subset_2-np.log10(ratios_C[1])
        else:
            print('Error. Invalid spectral type encountered.')

        differences_subset = logMdot_subset_1 - logMdot_subset_2
        if subset[1][0] < 0:
            label_mass = str(10**(subset[1][0]))[0:6] # Take the current mass␣
↪and make string
        else:
            label_mass = str(10**(subset[1][0]))[0:5] # Take the current mass␣
↪and make string
        plot.plot(subset[0],differences_subset,linewidth=1,linestyle =␣
↪'--',label=label_mass)
        #if coeffs_labels[1] == 'C1':
        #    i = i + 3
        #else:
        #    i = i + 5
        i = i + 5
    ####
    #plot.scatter(data_set[0],differences,s=2)
    #plot.plot(np.asarray(Llimits),diff_vs_logL_yval,color='orange')
    plot.xlabel('log L')
    plot.xlim(Llimits)
    #plot.ylabel(r'$\Delta \log \dot{M}_{\mathrm{dust}}$' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
    plot.ylabel(r'$\log \dot{M}_{\mathrm{dust}}$'+' (' + coeffs_labels[0] + ')␣
↪' + r'$-\log \dot{M}_{\mathrm{dust}}$' + ' ('+ coeffs_labels[1] + ')')
    #plot.ylim(deltaMdotlimits)
    plot.legend(title=r'$M$',ncol=2)
    plot.tight_layout(pad=1.2)
```

```
   plot.savefig('Final_Fit/
↪'+'dz_'+'deltalogMdotdust_vs_L_'+coeffs_labels[0]+'_'+coeffs_labels[1]+'.
↪png')
   plot.savefig('Final_Fit/
↪'+'dz_'+'deltalogMdotdust_vs_L_'+coeffs_labels[0]+'_'+coeffs_labels[1]+'.
↪pdf')
   plot.show()
   plot.close()

   plot.scatter(logP_1,differences,s=2)
   plot.xlabel('log P'+ ' (' + coeffs_labels[0] + ')')
   plot.xlim(Plimits)
   plot.ylabel(r'$\Delta \log \dot{M}_{\mathrm{dust}}$' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
   plot.ylim(deltaMdotlimits)
   plot.tight_layout()
   plot.show()
   plot.close()

   plot.scatter(logP_2,differences,s=2)
   plot.xlabel('log P'+ ' (' + coeffs_labels[1] + ')')
   plot.xlim(Plimits)
   plot.ylabel(r'$\Delta \log \dot{M}_{\mathrm{tot.}}$' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
   plot.ylim(deltaMdotlimits)
   plot.tight_layout()
   plot.show()
   plot.close()

   plot.scatter(logMdot_1,differences,s=2)
   plot.xlabel(r'$log Mdot_{\mathrm{tot.}}$' + ' (' + coeffs_labels[0] + ')' )
   plot.xlim(Mdotlimits)
   plot.ylabel(r'$\Delta \log \dot{M}_{\mathrm{dust}}$' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
   plot.ylim(deltaMdotlimits)
   plot.tight_layout()
   plot.show()
   plot.close()

   plot.scatter(logMdot_1,differences,s=2)
   plot.xlabel(r'$log Mdot_{\mathrm{tot.}}$' + ' (' + coeffs_labels[1] + ')' )
   plot.xlim(Mdotlimits)
   plot.ylabel(r'$\Delta \log \dot{M}_{\mathrm{dust}}$' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
   plot.ylim(deltaMdotlimits)
   plot.tight_layout()
```

```
    plot.show()
    plot.close()


    print('_____________________________________________________________________')



    '''
    # Figure out average difference at a given luminosity
    working_array = np.transpose(np.vstack((data_set[0],differences)))
    i = 0
    luminosity_pos_array = []
    luminosity_pos_array_all = []
    # Loop finds all of the indices where luminosities match, and saves them in␣
↪arrays
    while i < len(working_array):
        current_L = working_array[i][0]
        if i == len(working_array)-1:
            next_L = 0 # reset
        else:
            next_L = working_array[i+1][0]
        if i == len(working_array):
            # Handles the final value
            luminosity_pos_array.append(i)
            luminosity_pos_array_all.append(luminosity_pos_array)
            luminosity_pos_array = [] # reset after saving
        elif current_L == next_L:
            luminosity_pos_array.append(i)
        else:
            luminosity_pos_array.append(i) # Last position of current␣
↪luminosity needs to be appended
            luminosity_pos_array_all.append(luminosity_pos_array)
            luminosity_pos_array = [] # reset after saving
        i += 1
    #print(luminosity_pos_array_all)
    # Now, we need to average the Mdots at each luminosity
    i = 0
    average_differences = []
    for i in range(0,len(luminosity_pos_array_all)):
        differences_sum = 0
        for k in range(0,len(luminosity_pos_array_all[i])):
            differences_sum = differences_sum +␣
↪differences[luminosity_pos_array_all[i][k]]
        differences_mean = differences_sum/len(luminosity_pos_array_all[i])
        average_differences.append(differences_mean)
    unique_luminosities = []
```

```
    for i in range(0, len(luminosity_pos_array_all)):
        unique_luminosities.append(data_set[0][luminosity_pos_array_all[i][0]])
    #print(len(average_differences))
    #print(len(unique_luminosities))
    # And now plot
    fit_line = chi_sq_fit_2d(unique_luminosities,average_differences)[0]
    left_point = np.nanmin(unique_luminosities)
    right_point = np.nanmax(unique_luminosities)
    fit_x = np.asarray([left_point,right_point])
    fit_y = fit_line[0]+fit_line[1]*fit_x
    plot.scatter(unique_luminosities, average_differences,s=2)
    if fit_line[1]<0:
        fit_label = str(fit_line[0]) + str(fit_line[1])+'logL'
    else:
        fit_label = str(fit_line[0]) + '+' + str(fit_line[1])+'logL'
    plot.plot(fit_x, fit_y,color = 'black',label=fit_label)
    plot.xlabel('log L')
    plot.ylabel('<log Mdot - log Mdot>' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
    plot.legend()
    plot.show()
    plot.close()
    '''

    '''# Figure out average difference at a given period
    working_array = np.transpose(np.vstack((data_set[1],differences)))
    i = 0
    print(data_set[1])
    i = 0
    average_periods = []
    for i in range(0,len(luminosity_pos_array_all)):
        sum_P = 0
        for k in range(0,len(luminosity_pos_array_all[i])):
            sum_P = sum_P + data_set[1][luminosity_pos_array_all[i][k]]
        average_P = sum_P/len(luminosity_pos_array_all[i])
        average_periods.append(average_P)
    #print(len(average_differences))
    #print(len(average_periods))
    # Now plot
    fit_line = chi_sq_fit_2d(average_periods,average_differences)[0]
    left_point = np.nanmin(average_periods)
    right_point = np.nanmax(average_periods)
    fit_x = np.asarray([left_point,right_point])
    fit_y = fit_line[0]+fit_line[1]*fit_x
    plot.scatter(average_periods, average_differences,s=2)
    fit_label = str(fit_line[0]) + str(fit_line[1])+'<logP>'
    plot.plot(fit_x, fit_y,color = 'black',label=fit_label)
```

```
    plot.xlabel('log P')
    plot.ylabel('<log Mdot - log Mdot>' + '␣
↪('+coeffs_labels[0]+'->'+coeffs_labels[1]+')')
    plot.legend()
    plot.show()
    plot.close()
     '''
```

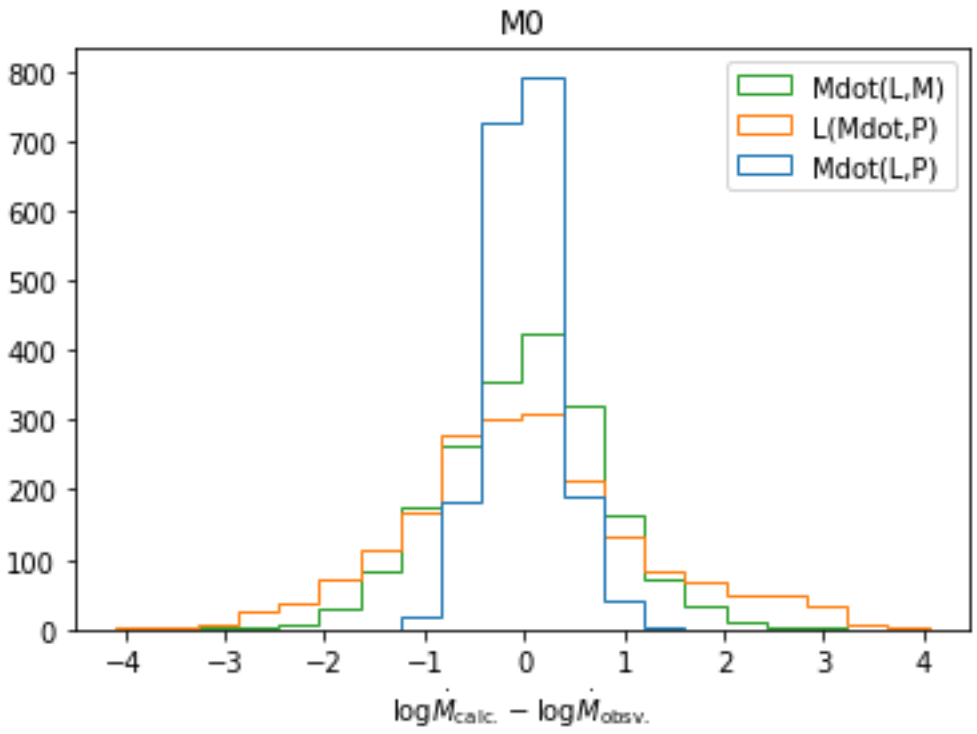
[373]:
```
def residual_histograms (star_data,MdotLP_coeffs, LMdotP_coeffs,␣
↪MdotLM_coeffs,spec_mode_label=''):
    # Ex: residual_histograms (M0StarData, MdotLP_Coeffs_M0 ,LMdotP_Coeffs_M0,␣
↪deathzone_values_M0)
    logPData = np.log10(star_data[PIndex])
    logLData = np.log10(star_data[LIndex])
    logMData = np.log10(star_data[MIndex])
    logMdotData = np.log10(star_data[MdotIndex])

    logMdot_MdotLP =␣
↪MdotLP_coeffs[0]+MdotLP_coeffs[1]*logLData+MdotLP_coeffs[2]*logPData
    logMdot_LMdotP =␣
↪LMdotP_coeffs[0]+LMdotP_coeffs[1]*logLData+LMdotP_coeffs[2]*logPData
    logMdot_MdotLM =␣
↪MdotLM_coeffs[0]+MdotLM_coeffs[1]*logLData+MdotLM_coeffs[2]*logMData

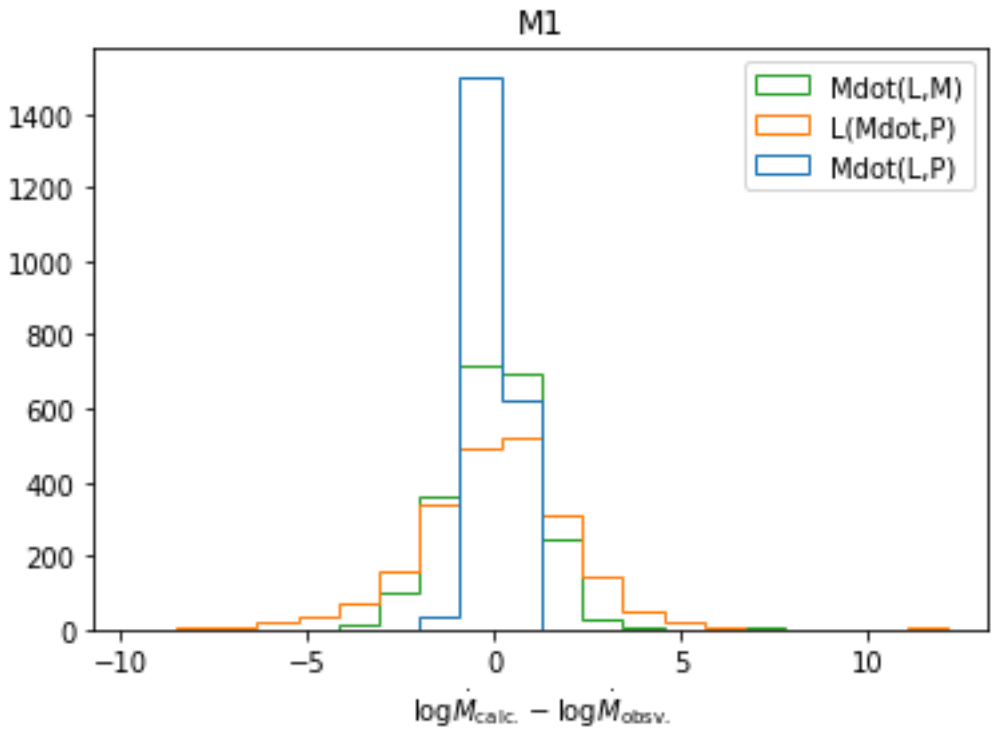
    residual_MdotLP = logMdot_MdotLP - logMdotData
    residual_LMdotP = logMdot_LMdotP - logMdotData
    residual_MdotLM = logMdot_MdotLM - logMdotData

    plot.hist([residual_MdotLP, residual_LMdotP,␣
↪residual_MdotLM],bins=20,histtype='step',label=['Mdot(L,P)','L(Mdot,P)','Mdot(L,M)'])
    plot.legend()
    plot.xlabel(r'$\log \dot{M}_{\mathrm{calc.}}-\log \dot{M}_{\mathrm{obsv.
↪}}$')
    plot.title(spec_mode_label)
```

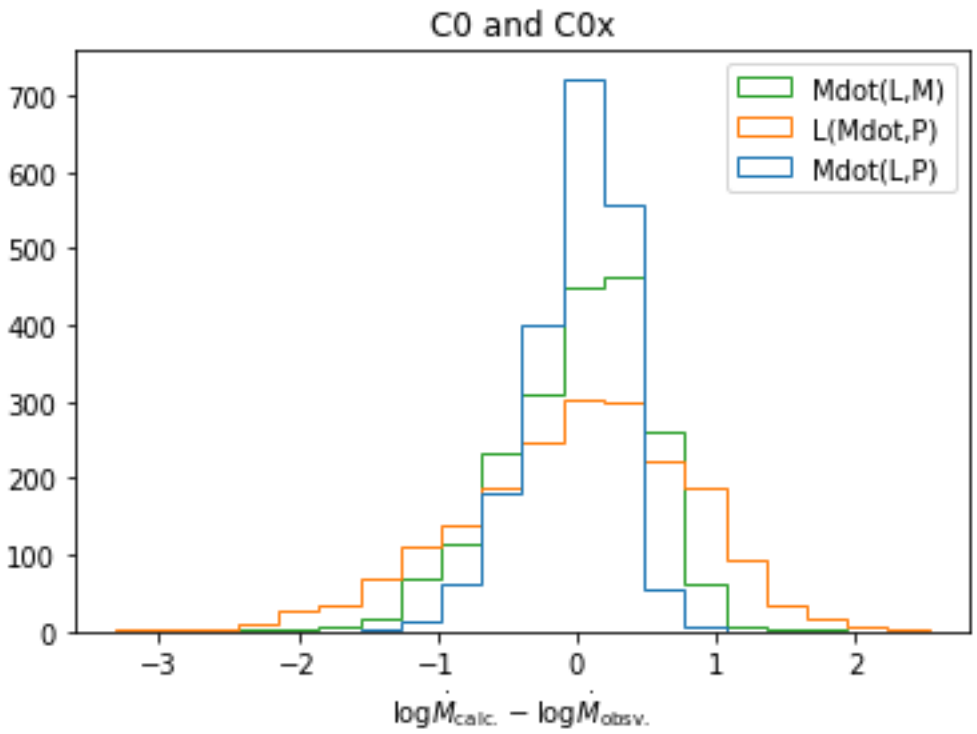
[374]:
```
residual_histograms (M0StarData, MdotLP_Coeffs_M0 ,LMdotP_Coeffs_M0,␣
↪deathzone_values_M0,spec_mode_label='M0')
```



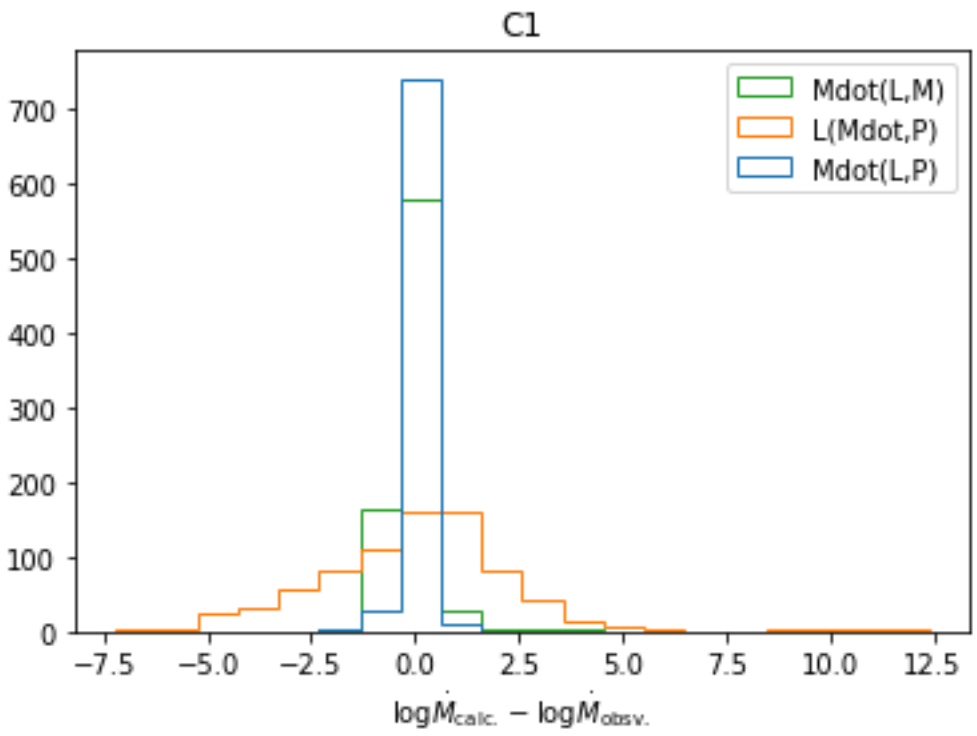
[375]:
```
residual_histograms (M1StarData, MdotLP_Coeffs_M1 ,LMdotP_Coeffs_M1,␣
↪deathzone_values_M1,spec_mode_label='M1')
```



[376]:
```
residual_histograms (C0StarData, MdotLP_Coeffs_C0andC0x␣
↪,LMdotP_Coeffs_C0andC0x, deathzone_values_C0_and_C0x,spec_mode_label='C0 and␣
↪C0x')
```



[377]:
```
residual_histograms (C1StarData, MdotLP_Coeffs_C1 ,LMdotP_Coeffs_C1,␣
↪deathzone_values_C1,spec_mode_label='C1')
```


C1
Mdot(L,M)
L(Mdot,P)
Mdot(L,P)
700
600
500
400
300
200
100
0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0
12.5
$\log \dot{M}_{\mathrm{calc.}} - \log \dot{M}_{\mathrm{obsv.}}$

[ ]:

[ ]: